



Secrets of the JavaScript Ninja

SEGUNDA EDIÇÃO

John Resig
Bear Bibeault
Josip Maras



TRIPULAÇÃO

Folha de dicas ES6

Literals de modelo embute expressões em strings: ``$ { ninja} ``.

Variáveis com escopo de bloco:

- Use as novas `let` palavra-chave para criar variáveis de nível de bloco: `let ninja = "Yoshi"`.
- Use as novas `const` palavra-chave para criar variáveis constantes de nível de bloco cujo valor não pode ser reatribuído a um valor completamente novo: `const ninja = "Yoshi"`.

Parâmetros de função:

- **Parâmetros de descanso** crie uma matriz de argumentos que não foram correspondidos aos parâmetros:

```
função multiMax (primeiro, ... restante) {/"..."/}multiMax(2, 3, 4, 5); // primeiro: 2;  
restantes: [3, 4, 5]
```

- **Parâmetros padrão** especifique os valores de parâmetro padrão que são usados se nenhum valor for fornecido durante a invocação:

```
função do (ninja, action = "skulk") { retornar ninja + "" + ação;} do ("Fuma"); // "Fuma skulk"
```

Operadores de spread expanda uma expressão onde vários itens são necessários: `[... itens, 3, 4, 5]`.

Funções de seta vamos criar funções com menos confusão sintática. Eles não têm seus próprios esta parâmetro. Em vez disso, eles o herdam do contexto em que foram criados:

```
valores const = [0, 3, 2, 5, 7, 4, 8, 1];  
valores.sort ((v1, v2) => v1 - v2); / * OR * / valores.sort ((v1, v2) => {return v1 - v2;}); value.forEach (value => console.log (value));
```

Geradores gerar sequências de valores por solicitação. Uma vez que um valor é gerado, o gerador suspende sua execução sem bloqueio. Usar produzem para gerar valores:

```
função * IdGenerator () {  
  deixe id = 0;  
  enquanto (verdadeiro) { rendimento ++ eu ia; }  
}
```

Promessas são marcadores de posição para o resultado de um cálculo. Uma promessa é uma garantia de que eventualmente saberemos o resultado de alguns cálculos. A promessa pode ser bem-sucedida ou falhar e, depois de cumprida, não haverá mais mudanças:

- Crie uma nova promessa com nova promessa `((resolver, rejeitar) => {})` ;.
- Ligar para resolver função para resolver explicitamente uma promessa. Ligar para rejeitar função para rejeitar explicitamente uma promessa. Uma promessa é rejeitada implicitamente se ocorrer um erro.
- O objeto de promessa tem um `então` método que retorna uma promessa e recebe dois retornos de chamada, um retorno de sucesso e um retorno de falha:

```
myPromise.then (val => console.log ("Sucesso"), err => console.log ("Erro"));
```

- Corrente em um pegar método para detectar falhas de promessa: `myPromise.catch (e => alert (e))` ;.

(continua na contracapa)

Elogios pela Primeira Edição

Finalmente, de um mestre, um livro que oferece o que um aspirante a desenvolvedor de JavaScript precisa para aprender a arte de criar um JavaScript eficaz para vários navegadores.

- Glenn Stokol, desenvolvedor sênior de currículo principal,
Oracle Corporation

Consistente com o lema da jQuery, "Escreva menos, faça mais".

- André Roberge, Université Saint-Anne

Técnicas interessantes e originais.

- Scott Sauyet, Four Winds Software

Leia este livro, e você não vai mais inserir cegamente um trecho de código e se maravilhar com a forma como ele funciona - você entenderá "por que" funciona.

- Joe Litton, desenvolvedor de software colaborativo, JoeLitton.net

Ajudará você a elevar seu JavaScript ao reino dos mestres.

- Christopher Haupt, greenstack.com

As coisas que os ninjas precisam saber.

- Chad Davis, autor de *Struts 2 em ação*

Leitura obrigatória para qualquer JavaScript Master.

- John J. Ryan III, Princigration LLC

Este livro é obrigatório para qualquer programador JS sério. Seu conhecimento do idioma se expandirá dramaticamente.

- S., leitor da Amazon

Segredos do JavaScript Ninja,
Segunda edição

JOHN RESIG
BEAR BIBLEAULT
e JOSIP MARAS



MANNING
Shelter Island

Para informações online e pedidos deste e de outros livros da Manning, visite www.manning.com. O editor oferece descontos neste livro quando solicitado em quantidade. Para mais informações por favor entre em contato

Departamento de Vendas Especiais
Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964 Email:
orders@manning.com

© 2016 por Manning Publications Co. Todos os direitos reservados.

Nenhuma parte desta publicação pode ser reproduzida, armazenada em um sistema de recuperação ou transmitida, em qualquer forma ou por meio eletrônico, mecânico, fotocópia ou outro, sem a permissão prévia por escrito do editor.

Muitas das designações usadas por fabricantes e vendedores para distinguir seus produtos são reivindicadas como marcas registradas. Onde essas designações aparecem no livro, e a Manning Publications estava ciente de uma reivindicação de marca registrada, as designações foram impressas em maiúsculas iniciais ou todas em maiúsculas.

☺ Reconhecendo a importância de preservar o que foi escrito, é política de Manning fazer com que os livros que publicamos sejam impressos em papel sem ácido, e fazemos o possível para esse fim. Reconhecendo também nossa responsabilidade de conservar os recursos de nosso planeta, os livros de Manning são impressos em papel que é pelo menos 15% reciclado e processado sem o uso de cloro elementar.

 Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964

Editor de desenvolvimento:	Dan Maharry
Editor de desenvolvimento técnico:	Gregor Zurowski
Editor de resenhas:	Ozren Harlovic
Editor de projeto:	Tiffany Taylor
Editor de cópia:	Sharon Wilkey
Revisor:	Alyson Brenner
Revisores técnicos:	Mathias Bynens, Jon Borgman
Tipógrafo:	Gordan Salinovic
Designer de capa:	Marija Tudor

ISBN 9781617292859

Impresso nos Estados Unidos da América

1 2 3 4 5 6 7 8 9 10 - EBM - 21 20 19 18 17 16

conteúdo

introdução do autor xi
agradecimentos xiii
sobre este livro xv
sobre a ilustração da capa xxi

P ARTE 1 W ARMANDO 1

1 *JavaScript está em toda parte 3*

1.1 Compreendendo a linguagem JavaScript 4

Como o JavaScript evoluirá? 6. Transpilers nos dão acesso ao JavaScript de amanhã, hoje 6

1.2 Compreendendo o navegador 7

1.3 Usando as melhores práticas atuais 8

Depurando 9. Teste 9. Análise de desempenho 10

1.4 Aumentando a capacidade de transferência 10

1.5 Resumo 11

2 *Construindo a página no tempo de execução 13*

2.1 A visão geral do ciclo de vida 14

2.2 A fase de construção da página 17

Analisando o HTML e construindo o DOM 18 . Executando o código JavaScript 20

2.3 Tratamento de eventos 23

Visão geral do tratamento de eventos 24 . Registrando manipuladores de eventos 25 Gerenciando eventos 27

2.4 Resumo 29

2.5 Exercícios 29

PARTE 2 U FUNÇÕES DE ENTENDIMENTO 31

3

Funções de primeira classe para o novato: definições e argumentos 33

3.1 O que há com a diferença funcional? 34

Funciona como objetos de primeira classe 35 . Funções de retorno de chamada 36

3.2 Diversão com funções como objetos 40

Armazenando funções 40 . Funções de auto-memorização 42

3.3 Definindo funções 44

Declarações de funções e expressões de funções 45 . Funções de seta 50

3.4 Argumentos e parâmetros de função 52

Parâmetros de descanso 54 . Parâmetros padrão 55

3.5 Resumo 58

3.6 Exercícios 59

4

Funções para o jornalista: função de compreensão

4.1 Usando parâmetros de função implícita 62

O parâmetro de argumentos 62 . O parâmetro this: introduzindo o contexto da função 67

4.2 Chamando funções 67

Invocação como uma função 68 . Invocação como método 69 Invocação como construtor 72 . Chamada com os métodos apply e call 77

4.3 Resolvendo o problema de contextos de função 83

Usando funções de seta para contornar contextos de função 83 Usando o método bind 86

4.4 Resumo 88

4.5 Exercícios 88

5

Funções para o mestre: fechamentos e escopos 91

5.1 Compreendendo os fechamentos 92

5.2 Colocando fechamentos para funcionar 95

Imitando variáveis privadas 95 . Usando fechamentos com retornos de chamada 96

5.3 Acompanhamento da execução do código com contextos de execução 99

5.4 Acompanhar identificadores com ambientes lexicais 103

Aninhamento de código 103 . Aninhamento de código e ambientes lexicais 104

5.5 Noções básicas sobre tipos de variáveis JavaScript 106

Mutabilidade variável 107 . Palavras-chave de definição de variável e ambientes lexicais 109 . Registrando identificadores em ambientes lexicais 113

5.6 Explorando como funcionam os fechamentos 117

Revisitando; imitando variáveis privadas com fechamentos 117 . Advertência de variáveis privadas 121 . Revisitando o exemplo de fechamentos e retornos de chamada 122

5,7 Resumo 124

5,8 Exercícios 124

6

Funções para o futuro: geradores e promessas 126

6.1 Tornando nosso código assíncrono elegante com geradores e promessas 127

6.2 Trabalhando com funções do gerador 129

Controlando o gerador através do objeto iterador 130 . Usando geradores 133 . Comunicando-se com um gerador 136 Explorando geradores sob o capô 139

6.3 Trabalho com promessas 146

Compreendendo os problemas com retornos de chamada simples 147 . Mergulhando nas promessas 149 . Rejeitando promessas 152 . Criando nossa primeira promessa do mundo real 154 . Encadeando promessas 155 Esperando por uma série de promessas 156

6.4 Combinando geradores e promessas 158

Olhando para o futuro - a função assíncrona 161

6,5 Resumo 162

6.6 Exercícios 163

PARTE 3 DIGGING EM OBJETOS E FORTIFICANDO

SEU CÓDIGO 165

7

Orientação do objeto com protótipos 167

7.1 Compreendendo os protótipos 168

7.2 Construção de objetos e protótipos 171

Propriedades da instância 173 . Efeitos colaterais da natureza dinâmica do JavaScript 176 . Digitação de objetos por meio de construtores 179

7.3 Atingindo herança 181

O problema de substituir a propriedade do construtor 184 . A instância do operador 187

7.4 Usando “classes” JavaScript no ES6 190

Usando a palavra-chave class 190 . Implementando herança 193

7.5 Resumo 195

7.6 Exercícios 196

8

Controlando o acesso a objetos 199

8.1 Controle de acesso às propriedades com getters e setters 200

Definindo getters e setters 202 . Usando getters e setters para validar valores de propriedade 207 . Usando getters e setters para definir propriedades computadas 208

8.2 Usando proxies para controlar o acesso 210

Usando proxies para registro 214 . Usando proxies para medir o desempenho 215 . Usando proxies para preencher automaticamente propriedades 217 Usando proxies para implementar índices de array negativos 218 Custos de desempenho de proxies 220

8.3 Resumo 221

8.4 Exercícios 222

9

Lidando com coleções 224

9.1 Arrays 225

Criação de matrizes 225 . Adicionando e removendo itens em qualquer extremidade de uma matriz 227 . Adicionar e remover itens em qualquer local da matriz 230 . Operações comuns em matrizes 232 . Reutilizando funções de array integradas 242

9.2 Mapas 244

Não use objetos como mapas 245 . Criando nosso primeiro mapa 247 Iterando sobre mapas 250

9.3 Conjuntos 251

Criando nosso primeiro conjunto 252 . União de conjuntos 253 . Intersecção de conjuntos 255 . Diferença de conjuntos 255

9.4 Resumo 256

9.5 Exercícios 256

10

Wrangling expressões regulares 259

10.1 Por que as expressões regulares são 260

10.2 Um atualizador de expressão regular 261

Expressões regulares explicadas 261 . Termos e operadores 263

10.3 Compilando expressões regulares 267

10.4 Capturando segmentos correspondentes 269

Executando capturas simples 269 . Correspondência usando expressões globais 271 . Referenciando capturas 272 . Grupos de não captura 273

10.5 Substituindo usando funções 274

10.6 Resolvendo problemas comuns com expressões regulares 276

Novas linhas correspondentes 277 . Unicode 277 correspondente . Caracteres de escape correspondentes 278

10.7 Resumo 279

10.8 Exercícios 280

11

Técnicas de modularização de código 282

11.1 Modularizando código em JavaScript pré-ES6 283

Usando objetos, encerramentos e funções imediatas para especificar módulos 284
Modularizando aplicativos JavaScript com AMD e CommonJS 290

11.2 Módulos ES6 294

Funcionalidade de exportação e importação 294

11.3 Resumo 300

11.4 Exercícios 301

P ARTE 4 B RECONHECIMENTO DE ROWSER 303

12

Trabalhando no DOM 305

12.1 Injetando HTML no DOM 306

Convertendo HTML em DOM 307 . Inserindo elementos no documento 311

12.2 Usando atributos e propriedades DOM 313

12.3 Dores de cabeça de atributo de estilo 315

Onde estão meus estilos? 315. Nomenclatura de propriedades de estilo 318 Buscando estilos computados 319. Convertendo valores de pixel 322 Medindo alturas e larguras 323

12.4 Minimizando layout thrashing 327

12.5 Resumo 330

12.6 Exercícios 330

13

Sobrevivendo eventos 332

13.1 Mergulho no loop de eventos 333

Um exemplo com apenas macrotarefas 336. Um exemplo com macro e microtarefas 339

13.2 Temporizadores de domesticação: tempos limite e intervalos 344

Execução do cronômetro dentro do loop de evento 345. Lidando com processamento computacionalmente caro 350

13.3 Trabalhando com eventos 353

Propagando eventos através da DOM 354. Eventos personalizados 360

13.4 Resumo 364

13.5 Exercícios 364

14

Desenvolvimento de estratégias para vários navegadores 367

14.1 Considerações entre navegadores 368

14.2 As cinco principais questões de desenvolvimento 370

Bugs e diferenças do navegador 371. Correções de bug do navegador 371 Código externo e marcação 373. Regressões 376

14.3 Estratégias de implementação 378

Correções seguras para navegadores diferentes 378. Detecção de recursos e polyfills 379 Problemas não testáveis do navegador 381

14.4 Reduzindo suposições 383

14.5 Resumo 384

14.6 Exercícios 385

apêndice A Recursos ES6 adicionais 387

Apêndice B Armando com teste e depuração 392 Respostas de

apêndice C exercícios 411

índice 433

introdução do autor

É incrível pensar no quanto o mundo do JavaScript mudou desde que comecei a escrever *Segredos do JavaScript Ninja* em 2008. O mundo em que escrevemos JavaScript agora, embora ainda esteja amplamente centrado no navegador, é quase irreconhecível.

A popularidade do JavaScript como uma linguagem de plataforma cruzada cheia de recursos explodiu. Node.js é uma plataforma formidável contra a qual inúmeros aplicativos de produção são desenvolvidos. Os desenvolvedores estão, na verdade, escrevendo aplicativos em uma linguagem - JavaScript - que podem ser executados em um navegador, em um servidor e até mesmo em um aplicativo nativo em um dispositivo móvel.

É mais importante agora, do que nunca, que o conhecimento de um desenvolvedor da linguagem JavaScript esteja no auge. Ter uma compreensão fundamental da linguagem e das maneiras pelas quais ela pode ser melhor escrita permitirá que você crie aplicativos que podem funcionar em praticamente qualquer plataforma, o que é uma afirmação de que poucas outras linguagens podem se gabar legitimamente.

Ao contrário das eras anteriores no crescimento do JavaScript, não houve um crescimento igual nas incompatibilidades de plataforma. Antes, você ficava salivando só de pensar em usar os novos recursos mais básicos do navegador e, ainda assim, ficava bloqueado por navegadores desatualizados que tinham participação de mercado excessiva. Entramos em uma época harmoniosa em que a maioria dos usuários está em navegadores atualizados rapidamente que competem para ser a plataforma mais compatível com os padrões. Os fornecedores de navegadores até se esforçam para desenvolver recursos voltados especificamente para desenvolvedores, na esperança de tornar suas vidas mais fáceis.

As ferramentas que temos agora, fornecidas pelos navegadores e pela comunidade de código aberto, estão anos-luz à frente das práticas antigas. Temos uma infinidade de estruturas de teste para escolher, a capacidade de fazer testes de integração contínua, gerar relatórios de cobertura de código, teste de desempenho em dispositivos móveis reais em todo o mundo e até mesmo carregar navegadores virtuais automaticamente em qualquer plataforma para testar.

A primeira edição do livro se beneficiou tremendamente da visão de desenvolvimento fornecida por Bear Bibeault. Esta edição recebeu ajuda substancial de Josip Maras para explorar os conceitos por trás do ECMAScript 6 e 7, mergulhar nas melhores práticas de teste e entender as técnicas empregadas por estruturas JavaScript populares.

Tudo isso é uma longa maneira de dizer: a maneira como escrevemos JavaScript mudou substancialmente. Felizmente, este livro está aqui para ajudá-lo a se manter atualizado sobre as práticas recomendadas atuais. Não apenas isso, mas ajudará a melhorar a forma como você pensa sobre suas práticas de desenvolvimento como um todo para garantir que você estará pronto para escrever JavaScript no futuro.

J OHN R ESIG

agradecimentos

O número de pessoas envolvidas na escrita de um livro surpreenderia a maioria das pessoas. Foi necessário um esforço colaborativo por parte de muitos colaboradores com uma variedade de talentos para trazer o volume que você está segurando (ou e-book que está lendo na tela) em prática.

A equipe da Manning trabalhou incansavelmente conosco para garantir que este livro atingisse o nível de qualidade que esperávamos e agradecemos a eles por seus esforços. Sem eles, este livro não teria sido possível. Os “créditos finais” deste livro incluem não apenas nosso editor, Marjan Bace, e o editor Dan Maharry, mas também os seguintes colaboradores: Ozren Harlovic, Gregor Zurowski, Kevin Sullivan, Janet Vail, Tiffany Taylor, Sharon Wilkey, Alyson Brener e Gordan Salinovic.

Não se pode dizer o suficiente para agradecer aos nossos revisores que ajudaram a moldar a forma final do livro, desde a captura de erros de digitação simples até a correção de erros de terminologia e código, e ajuda na organização dos capítulos do livro. Cada passagem por um ciclo de revisão acabou melhorando muito o produto final. Por reservar um tempo para revisar o livro, gostaríamos de agradecer a Jacob Andresen, Tidjani Belmansour, Francesco Bianchi, Matthew Halverson, Becky Huett, Daniel Lamb, Michael Lund, Kariem Ali Elkoush, Elyse Kolker Gordon, Christopher Haupt, Mike Hatfield, Gerd Klevesaat, Alex Lucas, Arun Noronha, Adam Scheller, David Starkey e Gregor Zurowski.

Agradecimentos especiais a Mathias Bynens e Jon Borgman, que atuaram como revisores técnicos do livro. Além de verificar cada amostra de código de exemplo em vários ambientes, eles também ofereceram contribuições inestimáveis para a precisão técnica do texto, localizaram informações que estavam faltando originalmente e se mantiveram a par das rápidas mudanças no suporte a JavaScript e HTML5 nos navegadores .

John Resig

Gostaria de agradecer a meus pais por seu apoio e incentivo constantes ao longo dos anos. Eles me forneceram os recursos e ferramentas de que eu precisava para despertar meu interesse inicial em programação - e eles têm me incentivado desde então.

Bear Bibeault

O elenco de personagens que eu gostaria de agradecer por esta sétima tentativa tem uma longa lista de "suspeitos do costume", incluindo, mais uma vez, os membros e a equipe do coderanch.com (antigo JavaRanch). Sem meu envolvimento no CodeRanch, eu nunca teria tido a oportunidade de começar a escrever em primeiro lugar, então agradeço sinceramente a Paul Wheaton e Kathy Sierra por começarem tudo, bem como colegas de equipe que me encorajaram e apoiaram, incluindo (mas certamente não limitado a) Eric Pascarello, Ernest Friedman-Hill, Andrew Monkhouse, Jeanne Boyarsky, Bert Bates e Max Habibi.

Meu marido, Jay, e meus cachorros, Ursinho e Cozmo, recebem os agradecimentos calorosos de sempre por tolerar a presença sombria que compartilhava sua casa e raramente erguia os olhos do teclado, exceto para xingar Word, os navegadores, minha falta de dedos gordos de habilidades de digitação, ou qualquer outra coisa que atraiu minha ira enquanto eu estava trabalhando neste projeto.

Por fim, gostaria de agradecer aos meus co-autores, John Resig e Josip Maras, sem os quais este projeto não existiria.

Josip Maras

Os maiores agradecimentos vão para minha esposa, Josipa, por suportar todas as horas gastas para escrever este livro.

Também gostaria de agradecer a Maja Stula, Darko Stipanicev, Ivica Crnkovic, Jan Carlson e Bert Bates: todos eles pela orientação e conselhos úteis, e alguns deles por serem tolerantes em minhas atribuições de "trabalho diário" conforme os prazos dos livros se aproximavam .

Por fim, gostaria de agradecer ao resto da minha família - Jere, dois Marijas, Vitomir e Zdenka - por sempre estarem disponíveis para mim.

sobre este livro

JavaScript é importante. Nem sempre foi assim, mas agora é verdade. JavaScript se tornou uma das linguagens de programação mais importantes e mais amplamente usadas hoje.

Espera-se que os aplicativos da Web ofereçam aos usuários uma experiência de interface de usuário rica e, sem JavaScript, você pode muito bem mostrar fotos de gatinhos. Mais do que nunca, os desenvolvedores da web precisam ter uma compreensão sólida da linguagem que dá vida aos aplicativos da web.

E como suco de laranja e café da manhã, JavaScript não é mais apenas para navegadores. A linguagem há muito derrubou as paredes do navegador e está sendo usada no servidor graças ao Node.js, em dispositivos desktop e móveis por meio de plataformas como Apache Cordova, e até mesmo em dispositivos embarcados com Espruino e Tessel.

Embora este livro se concentre principalmente em JavaScript executado no navegador, os fundamentos da linguagem apresentada neste livro são aplicáveis em todo o quadro. Compreender verdadeiramente os conceitos e aprender várias dicas e truques farão de você um desenvolvedor JavaScript completo.

Com cada vez mais desenvolvedores usando JavaScript em um mundo cada vez mais JavaScript, é mais importante do que nunca entender seus fundamentos para que você possa se tornar um ninja especialista na linguagem.

Público

Se você não está familiarizado com JavaScript, este provavelmente não deve ser seu primeiro livro. Mesmo se for, não se preocupe muito; tentamos apresentar os conceitos fundamentais do JavaScript de uma forma que seja compreensível mesmo para iniciantes. Mas para ser honesto,

este livro provavelmente será mais adequado para pessoas que já conhecem um pouco de JavaScript e desejam aprofundar sua compreensão de JavaScript como uma linguagem, bem como do navegador como o ambiente no qual o código JavaScript é executado.

Roteiro

Este livro é organizado para levá-lo de um aprendiz a um ninja em quatro partes.

A Parte 1 apresenta o tópico e prepara o cenário para que você possa progredir facilmente pelo resto do livro:

- O Capítulo 1 apresenta a linguagem JavaScript e seus recursos mais importantes, enquanto sugere as melhores práticas atuais que devemos seguir ao desenvolver aplicativos, incluindo testes e análise de desempenho.
- Como nossa exploração de JavaScript é feita no contexto de navegadores, no capítulo 2 definiremos o cenário, introduzindo o ciclo de vida dos aplicativos da Web do lado do cliente. Isso nos ajudará a entender a função do JavaScript no processo de desenvolvimento de aplicativos da web.

A Parte 2 concentra-se em um dos pilares do JavaScript: funções. Estudaremos por que as funções são tão importantes em JavaScript, os diferentes tipos de funções, bem como os detalhes essenciais de definir e invocar funções. Colocaremos um foco especial em um novo tipo de função - funções geradoras - que são especialmente úteis ao lidar com código assíncrono:

- O Capítulo 3 inicia nossa incursão nos fundamentos da linguagem, começando, talvez para sua surpresa, com um exame completo do *função* conforme definido por JavaScript. Embora você possa ter esperado o *objeto* para ser o alvo de nosso primeiro foco, uma compreensão sólida da função e do JavaScript como uma linguagem funcional, começa nossa transformação de codificadores comuns de JavaScript para ninjas de JavaScript!
- Continuamos esse encadeamento funcional no capítulo 4, explorando o mecanismo exato de invocar funções, bem como os meandros dos parâmetros de função implícitos.
- Ainda não concluindo as funções, no capítulo 5 levamos nossa discussão para o próximo nível, estudando dois conceitos intimamente relacionados: *escopos* e *fechamentos*. Um conceito-chave na programação funcional, os encerramentos nos permitem exercer um controle refinado sobre o escopo dos objetos que declaramos e criamos em nossos programas. O controle desses escopos é o fator chave para escrever um código digno de um ninja. Mesmo que você pare de ler depois deste capítulo (mas esperamos que não pare), você será um desenvolvedor de JavaScript muito melhor do que quando começou.
- Concluimos nossa exploração de funções no capítulo 6, dando uma olhada em um tipo completamente novo de função (*funções do gerador*) e um novo tipo de objeto (*promessas*) que nos ajudam a lidar com valores assíncronos. Também mostraremos como combinar geradores e promessas para obter elegância ao lidar com código assíncrono.

A Parte 3 trata do segundo pilar do JavaScript: objetos. Exploraremos exaustivamente a orientação a objetos em JavaScript e estudaremos como proteger o acesso a objetos e como lidar com coleções e expressões regulares:

- Os objetos são finalmente tratados no capítulo 7, onde aprendemos exatamente como funciona o sabor ligeiramente estranho do JavaScript de orientação a objetos. Também apresentaremos uma nova adição ao JavaScript: classes, que, bem no fundo, podem não ser exatamente o que você espera.
- Continuaremos nossa exploração de objetos no capítulo 8, onde estudaremos diferentes técnicas para proteger o acesso a nossos objetos.
- No capítulo 9, colocaremos um foco especial em diferentes tipos de coleções que existem em JavaScript; em arrays, que fazem parte do JavaScript desde seu início; e em mapas e conjuntos, que são uma adição recente ao JavaScript.
- O Capítulo 10 enfoca as expressões regulares, um recurso frequentemente esquecido da linguagem que pode fazer o trabalho de dezenas de linhas de código quando usado corretamente. Aprenderemos como construir e usar expressões regulares e como resolver alguns problemas recorrentes de maneira elegante, usando expressões regulares e os métodos que funcionam com elas.
- No capítulo 11, aprenderemos diferentes técnicas para organizar nosso código em módulos: segmentos menores e relativamente fracamente acoplados que melhoram a estrutura e a organização de nosso código.

Finalmente, a parte 4 encerra o livro estudando como o JavaScript interage com nossas páginas da web e como os eventos são processados pelo navegador. Terminaremos o livro examinando um tópico importante, o desenvolvimento entre navegadores:

- O Capítulo 12 explora como podemos modificar dinamicamente nossas páginas por meio de APIs de manipulação de DOM e como podemos lidar com atributos, propriedades e estilos de elementos, bem como algumas considerações importantes de desempenho.
- O Capítulo 13 discute a importância do modelo de execução de thread único do JavaScript e as consequências que esse modelo tem no loop de eventos. Também aprenderemos como temporizadores e intervalos funcionam e como podemos usá-los para melhorar o desempenho percebido de nossos aplicativos da web.
- O Capítulo 14 conclui o livro examinando as cinco principais preocupações de desenvolvimento com relação a esses problemas entre navegadores: diferenças de navegador, bugs e correções de bugs, código externo e marcação, recursos ausentes e regressões. Estratégias como simulação de recursos e detecção de objetos são discutidas detalhadamente para nos ajudar a lidar com esses desafios entre navegadores.

Convenções de código

Todo o código-fonte nas listagens ou no texto está em um fonte de largura fixa como esta para separá-lo do texto comum. Nomes de métodos e funções, propriedades, elementos XML e atributos no texto também são apresentados nesta mesma fonte.

Em alguns casos, o código-fonte original foi reformatado para caber nas páginas. Em geral, o código original foi escrito com as limitações de largura da página em mente, mas às vezes você pode encontrar uma pequena diferença de formatação entre o código do livro e o fornecido no download do código-fonte. Em alguns casos raros, onde longas linhas não puderam ser reformatadas sem alterar seu significado, as listas de livros contêm marcadores de continuação de linha.

As anotações de código acompanham muitas das listagens; estes destacam conceitos importantes.

Downloads de código

O código-fonte de todos os exemplos de trabalho neste livro (junto com alguns extras que nunca foram incluídos no texto) está disponível para download na página do livro em <https://manning.com/books/secrets-of-the-javascript-ninja-segunda-edição>.

Os exemplos de código para este livro são organizados por capítulo, com pastas separadas para cada capítulo. O layout está pronto para ser servido por um servidor web local, como o servidor Apache HTTP. Descompacte o código baixado em uma pasta de sua escolha e torne essa pasta a raiz do documento do aplicativo.

Com algumas exceções, a maioria dos exemplos não requer a presença de um servidor web e pode ser carregada diretamente em um navegador para execução, se desejar.

Autor Online

Os autores e a Manning Publications convidam você para o fórum do livro, administrado pela Manning Publications, onde você pode fazer comentários sobre o livro, fazer perguntas técnicas e receber ajuda dos autores e de outros usuários. Para acessar e assinar o fórum, aponte seu navegador para <https://manning.com/books/secrets-of-the-javascript-ninja-second-edition> e clique no link Autor Online. Esta página fornece informações sobre como entrar no fórum depois de registrado, que tipo de ajuda está disponível e as regras de conduta do fórum.

O compromisso de Manning com nossos leitores é fornecer um espaço onde um diálogo significativo entre leitores individuais e entre leitores e autores possa ocorrer. Não é um compromisso com qualquer quantidade específica de participação por parte dos autores, cuja contribuição para o fórum do livro permanece voluntária (e gratuita). Sugerimos que você tente fazer algumas perguntas desafiadoras aos autores, para que o interesse deles não se desvie!

O fórum do autor online e os arquivos das discussões anteriores estarão acessíveis no site da editora, desde que o livro esteja sendo impresso.

Sobre os autores



John Resig é engenheiro da Khan Academy e criador da biblioteca jQuery JavaScript.

Além da primeira edição do

Segredos do JavaScript Ninja, ele também é o autor do livro *Técnicas de JavaScript Pro*.

John desenvolveu um abrangente banco de dados de impressão de blocos de madeira japonês e um mecanismo de busca de imagens: Ukiyo-e.org. Ele é membro do conselho da Sociedade de Arte Japonesa da América e é um visitante

Pesquisador da Ritsumeikan University trabalhando no estudo de Ukiyo-e.

John está localizado em Brooklyn, NY.



Bear Bibeault vem escrevendo software há mais de três décadas, começando com um programa Tic-Tac-Toe escrito em um supercomputador Control Data Cyber por meio de um teletipo de 100 baud. Tendo dois diplomas em engenharia elétrica, Bear deveria estar projetando antenas ou algo parecido, mas desde seu primeiro emprego na Digital Equipment Corporation, ele sempre foi muito mais fascinado por programação.

Bear também atuou em empresas como Dragon Systems, Works.com, Spreadfast, Logitech, Caringo e mais de um punhado de outras. Bear até serviu nas forças armadas dos Estados Unidos, liderando e treinando um pelotão de soldados de infantaria antitanques - habilidades que são úteis durante as reuniões do scrum. "Isso é *Sargento* Tenha paciência com você, estagiário! "

Bear é atualmente um desenvolvedor front-end sênior para um provedor líder de software de armazenamento de objetos que fornece escalabilidade de armazenamento massivo e proteção de conteúdo.

Além da primeira edição deste livro, Bear também é autor de vários outros livros de Manning, incluindo *jQuery em ação* (primeira, segunda e terceira edições), *Ajax na prática*, e *Protótipo e Scriptaculous em ação*; e ele tem sido um revisor técnico de muitos dos livros focados na web "Use a Cabeça" da O'Reilly Publishing, como *Head First Ajax*, *Head Rush Ajax*, e *Use primeiro Servlets e JSP*.

Além de seu trabalho diário, Bear também escreve livros (duh!), Dirige uma pequena empresa que cria aplicativos da web e oferece outros serviços de mídia (mas não videografia de casamento - nunca, jamais, videografia de casamento) e ajuda em CodeRanch.com como um "marechal" (moderador uber).

Quando não está plantado na frente de um computador, Bear gosta de cozinhar *grande* comida (o que explica o tamanho de seu jeans), mergulhe em fotografia e vídeo, ande em sua Yamaha V-Star e use camisetas com estampas tropicais.

Ele trabalha e mora em Austin, Texas, uma cidade que ele ama, exceto pelo trânsito e os motoristas completamente insanos.



Josip Maras é pesquisador de pós-doutorado na faculdade de engenharia elétrica, engenharia mecânica e arquitetura naval da Universidade de Split, Croácia. Ele tem um PhD em engenharia de software, com a tese "Automatizando o Reuso no Desenvolvimento de Aplicativos Web", que entre outras coisas incluiu a implementação de um interpretador JavaScript em JavaScript. Durante sua pesquisa, ele publicou mais de uma dúzia de artigos em jornais e conferências científicas, a maioria lidando com análise de programas de aplicativos da Web do lado do cliente.

Quando não está fazendo pesquisa, Josip passa seu tempo ensinando desenvolvimento para web, análise e design de sistemas e desenvolvimento para Windows (algumas centenas de alunos nos últimos seis anos). Ele também possui uma pequena empresa de desenvolvimento de software.

Em seu tempo livre, Josip gosta de ler, correr longas e, se o tempo permitir, nadar no Adriático.

sobre a ilustração da capa

A figura na capa do *Segredos do JavaScript Ninja*, segunda edição tem como legenda "Ator Noh, Samurai" de uma impressão em xilogravura de um artista japonês desconhecido de meados do século XIX. Derivado da palavra japonesa para *talento* ou *habilidade*, Noh é uma forma de drama musical clássico japonês que é apresentada desde o século XIV. Muitos personagens são mascarados, com homens desempenhando papéis masculinos e femininos. O samurai, uma figura heróica no Japão por centenas de anos, era freqüentemente apresentado nas apresentações, e nesta gravura o artista representa com grande habilidade a beleza do traje e a ferocidade do samurai.

Samurais e ninjas eram guerreiros que se destacavam na arte da guerra japonesa, conhecidos por sua bravura e astúcia. Samurais eram soldados de elite, homens bem-educados que sabiam ler e escrever, além de lutar, e estavam vinculados a um rígido código de honra chamado Bushido (O Caminho do Guerreiro), que era transmitido oralmente de geração em geração, começando no século 10. Recrutados na aristocracia e nas classes altas, análogos aos cavaleiros europeus, os samurais iam para a batalha em grandes formações, usando armaduras elaboradas e vestidos coloridos destinados a impressionar e intimidar. Os ninjas foram escolhidos por suas habilidades nas artes marciais, e não por sua posição social ou educação. Vestidos de preto e com o rosto coberto, eles foram enviados em missões sozinhos ou em pequenos grupos para atacar o inimigo com subterfúgio e furtividade, usar qualquer tática para garantir o sucesso; seu único código era o sigilo.

A ilustração da capa é de um conjunto de três gravuras japonesas de propriedade por muitos anos de um editor da Manning, e quando estávamos procurando por um ninja para a capa deste livro, o

Impressão impressionante de samurai chamou nossa atenção e foi selecionada por seus detalhes intrincados, cores vibrantes e representação vívida de um guerreiro feroz pronto para atacar - e vencer.

Em uma época em que é difícil distinguir um livro de computador do outro, Manning celebra a inventividade e a iniciativa do setor de informática com capas de livros baseadas em ilustrações de duzentos anos que retratam a rica diversidade de trajes tradicionais de todo o mundo , trazido de volta à vida por estampas como esta.

Aquecendo

Tua parte do livro definirá o cenário para seu treinamento ninja de JavaScript. No capítulo 1, veremos o estado atual do JavaScript e exploraremos alguns dos ambientes nos quais o código JavaScript pode ser executado. Colocaremos um foco especial no ambiente onde tudo começou - *o navegador*—E discutiremos algumas das melhores práticas ao desenvolver aplicativos JavaScript.

Como nossa exploração de JavaScript será feita no contexto de navegadores, no capítulo 2 ensinaremos a você o ciclo de vida dos aplicativos da Web do lado do cliente e como a execução do código JavaScript se encaixa nesse ciclo de vida.

Quando terminar esta parte do livro, você estará pronto para embarcar em seu treinamento como um ninja JavaScript!

JavaScript está em todo lugar



Este capítulo cobre

- Os principais recursos da linguagem JavaScript
- Os principais itens de um motor JavaScript
- Três melhores práticas no desenvolvimento de JavaScript

Vamos falar sobre Bob. Depois de passar alguns anos aprendendo como criar aplicativos de desktop em C ++, ele se formou como desenvolvedor de software no início dos anos 2000 e então saiu para o mundo inteiro. Nesse ponto, a web tinha acabado de começar e todos queriam ser a próxima Amazon. Então, a primeira coisa que ele fez foi aprender desenvolvimento web.

Ele aprendeu um pouco de PHP para que pudesse gerar páginas da web dinamicamente, que normalmente ele borrifava com JavaScript para obter funcionalidades complexas, como validação de formulário e até mesmo relógios dinâmicos na página! Alguns anos depois, os smartphones se tornaram uma coisa, então, antecipando a abertura de um grande mercado, Bob foi em frente e aprendeu Objective-C e Java para desenvolver aplicativos móveis que rodam em iOS e Android.

Ao longo dos anos, Bob criou muitos aplicativos de sucesso que precisam ser mantidos e ampliados. Infelizmente, saltar diariamente entre todas essas diferentes linguagens de programação e estruturas de aplicativos realmente começou a desgastar o pobre Bob.

Agora vamos falar sobre Ann. Dois anos atrás, Ann se formou em desenvolvimento de software, com especialização em aplicativos baseados na web e na nuvem. Ela criou alguns aplicativos da web de médio porte com base em estruturas modernas de model-view-controller (MVC), junto com os aplicativos móveis que os acompanham que rodam em iOS e Android. Ela criou um aplicativo de desktop que roda em Linux, Windows e SO

X, e até começou a construir uma versão sem servidor desse aplicativo totalmente baseada na nuvem. E *tudo o que ela fez foi escrito em JavaScript*.

Isso é extraordinário! O que Bob levou 10 anos e 5 idiomas para fazer, Ann conseguiu em 2 anos e em *apenas um idioma*. Ao longo da história da computação, é raro que um determinado conjunto de conhecimentos seja facilmente transferível e útil em tantos domínios diferentes.

O que começou como um humilde projeto de 10 dias em 1995 é agora uma das linguagens de programação mais amplamente usadas no mundo. JavaScript é literalmente *em toda parte*, graças a mecanismos JavaScript mais poderosos e à introdução de estruturas como Node, Apache Cordova, Ionic e Electron, que levaram a linguagem além da humilde página da web. E, como o HTML, a própria linguagem agora está recebendo atualizações há muito atrasadas com o objetivo de tornar o JavaScript ainda mais adequado para o desenvolvimento de aplicativos modernos.

Neste livro, vamos garantir que você saiba tudo o que precisa saber sobre JavaScript para que, seja como Ann ou Bob, você possa desenvolver todos os tipos de aplicativos em um campo verde ou marrom.

O que são Babel e Traceur, e por que eles são importantes

para os desenvolvedores de JavaScript de hoje?

Você sabe?

Quais são as partes principais do JavaScript de qualquer navegador da web

API usada por aplicativos da web?

1,1 *Compreender a linguagem JavaScript*

À medida que avançam em suas carreiras, muitos programadores de JavaScript como Bob e Ann chegam ao ponto em que estão usando ativamente o vasto número de elementos que formam a linguagem. Em muitos casos, entretanto, essas habilidades não podem ser levadas além dos níveis fundamentais. Nossa suposição é que isso geralmente ocorre porque o JavaScript, usando uma sintaxe semelhante a C, tem uma semelhança superficial com outras linguagens semelhantes a C (como C # e Java) e, portanto, deixa a impressão de familiaridade.

As pessoas geralmente acham que, se conhecerem C # ou Java, já terão um conhecimento bastante sólido de como o JavaScript funciona. Mas é uma armadilha! Quando comparado a outras linguagens convencionais, JavaScript é muito mais *funcionalmente* orientado. Alguns conceitos de JavaScript diferem fundamentalmente daqueles da maioria das outras linguagens.

Essas diferenças incluem o seguinte:

- *Funções são objetos de primeira classe*— Em JavaScript, as funções coexistem e podem ser tratadas como qualquer outro objeto JavaScript. Eles podem ser criados por meio de literais, referenciados por variáveis, passados como argumentos de função e até mesmo retornados como valores de retorno de função. Dedicamos grande parte do capítulo 3 para explorar alguns dos benefícios maravilhosos que funções como objetos de primeira classe trazem para nosso código JavaScript.
- *Fechamentos de funções*— O conceito de encerramentos de função é geralmente mal compreendido, mas ao mesmo tempo exemplifica fundamental e irrevogavelmente a importância das funções para o JavaScript. Por enquanto, basta saber que uma função é *um fechamento quando ele mantém ativamente (“fecha”) as variáveis externas usadas em seu corpo*. Não se preocupe por enquanto se você não vê os muitos benefícios dos fechamentos; vamos ter certeza de que tudo está claro como cristal no capítulo 5. Além dos encerramentos, exploraremos completamente os muitos aspectos das próprias funções nos capítulos 3 e 4, bem como os escopos de identificador no capítulo 5.
- *Âmbitos* - Até recentemente, o JavaScript não tinha variáveis de nível de bloco (como em outras linguagens semelhantes a C); em vez disso, tivemos que confiar apenas em variáveis globais e variáveis de nível de função.
- *Orientação a objetos baseada em protótipo* - Ao contrário de outras linguagens de programação convencionais (como C #, Java e Ruby), que usam orientação a objetos baseada em classe, JavaScript usa protótipos. Frequentemente, quando os desenvolvedores vêm para o JavaScript de linguagens baseadas em classe (como Java), eles tentam usar JavaScript como se fosse Java, essencialmente escrevendo o código baseado em classe do Java usando a sintaxe do JavaScript. Então, por algum motivo, eles ficam surpresos quando os resultados diferem do que eles esperam. É por isso que nos aprofundaremos em protótipos, como funciona a objetororientação baseada em protótipos e como é implementada em JavaScript.

JavaScript consiste em um relacionamento próximo entre objetos e protótipos, funções e encerramentos. Compreender as fortes relações entre esses conceitos pode melhorar muito sua capacidade de programação JavaScript, dando a você uma base sólida para qualquer tipo de desenvolvimento de aplicativo, independentemente de seu código JavaScript ser executado em uma página da web, em um aplicativo de desktop ou em um aplicativo móvel , ou no servidor.

Além desses conceitos fundamentais, outros recursos de JavaScript podem ajudá-lo a escrever um código mais elegante e mais eficiente. Alguns desses são recursos que desenvolvedores experientes como Bob reconhecerão de outras linguagens, como Java e C ++. Em particular, nos concentramos no seguinte:

- *Geradores*, que são funções que podem gerar vários valores por solicitação e podem suspender sua execução entre solicitações
- *Promessas*, que nos dá melhor controle sobre o código assíncrono
- *Proxies*, que nos permitem controlar o acesso a certos objetos
- *Métodos avançados de array*, que tornam o código de manipulação de array muito mais elegante

- *Mapas*, que podemos usar para criar coleções de dicionário; e *conjuntos*, que nos permitem lidar com coleções de itens exclusivos
- *Expressões regulares*, que nos permite simplificar o que de outra forma seriam partes complicadas de código
- *Módulos*, que podemos usar para quebrar o código em partes menores e relativamente autocontidas que tornam os projetos mais gerenciáveis

Ter um profundo conhecimento dos fundamentos e aprender como usar os recursos avançados de linguagem da melhor maneira possível pode elevar seu código a níveis mais altos. Aperfeiçoar suas habilidades para unir esses conceitos e recursos lhe dará um nível de compreensão que coloca a criação de qualquer tipo de aplicativo JavaScript ao seu alcance.

1.1.1 *Como o JavaScript evoluirá?*

O comitê ECMAScript, responsável pela padronização da linguagem, acaba de finalizar a versão ES7 / ES2016 do JavaScript. A versão ES7 é uma atualização relativamente pequena para o JavaScript (pelo menos, quando comparada ao ES6), porque a meta do comitê daqui para frente é se concentrar em mudanças incrementais menores e anuais no idioma.

Neste livro, exploramos completamente o ES6, mas também nos concentramos nos recursos do ES7, como a nova função assíncrona, que o ajudará a lidar com o código assíncrono (discutido no capítulo 6).



NOTA Quando cobrimos os recursos de JavaScript definidos no ES6 / ES2015 ou ES7 / ES2016, você verá esses ícones ao lado de um link para informações sobre se eles são suportados pelo seu navegador.

Atualizações incrementais anuais da especificação do idioma são excelentes notícias, mas isso não significa que os desenvolvedores da web terão acesso instantâneo aos novos recursos depois que a especificação for lançada. O código JavaScript deve ser executado por um mecanismo JavaScript, portanto, muitas vezes ficamos esperando impacientemente por atualizações em nossos mecanismos JavaScript favoritos que incorporam esses novos e interessantes recursos.

Infelizmente, embora os desenvolvedores do mecanismo de JavaScript estejam tentando se manter atualizados e melhorando o tempo todo, sempre há uma chance de você encontrar recursos que está morrendo de vontade de usar, mas que ainda não têm suporte.

Felizmente, você pode acompanhar o estado do suporte a recursos nos vários navegadores por meio das listas em <https://kangax.github.io/compat-table/es6/> , <http://kangax.github.io/mesa/compativel/es2016plus/> , e <https://kangax.github.io/compat-table/esnext/> .

1.1.2 *Transpilers nos dão acesso ao JavaScript de amanhã hoje*

Devido aos rápidos ciclos de lançamento dos navegadores, geralmente não temos que esperar muito para que um recurso JavaScript seja compatível. Mas o que acontece se quisermos aproveitar

todos os benefícios dos mais novos recursos de JavaScript, mas são reféns de uma dura realidade: os usuários de nossos aplicativos da web ainda podem estar usando navegadores mais antigos?

Uma resposta para este problema é usar *transpiladores* ("Transformação + compilação"), ferramentas que pegam código JavaScript de ponta e o transformam em código equivalente (ou, se isso não for possível, semelhante) que funciona corretamente na maioria dos navegadores atuais.

Os transpiladores mais populares de hoje são Traceur (<https://github.com/google/traceur-compiler>) e Babel (<https://babeljs.io/>) Configurá-los é fácil; basta seguir um dos tutoriais, como <https://github.com/google/traceur-compiler/wiki/Getting-Started> ou <http://babeljs.io/docs/setup/> .

Neste livro, colocamos um foco especial na execução de código JavaScript no navegador. Para usar a plataforma do navegador de forma eficaz, você precisa sujar as mãos e estudar o funcionamento interno dos navegadores. Vamos começar!

1,2 Compreender o navegador

Atualmente, os aplicativos JavaScript podem ser executados em muitos ambientes. Mas o ambiente a partir do qual tudo começou, o ambiente a partir do qual todos os outros ambientes tiveram ideias e o ambiente no qual nos concentraremos, é o *navegador*. O navegador fornece vários conceitos e APIs para explorar completamente; veja a figura 1.1.

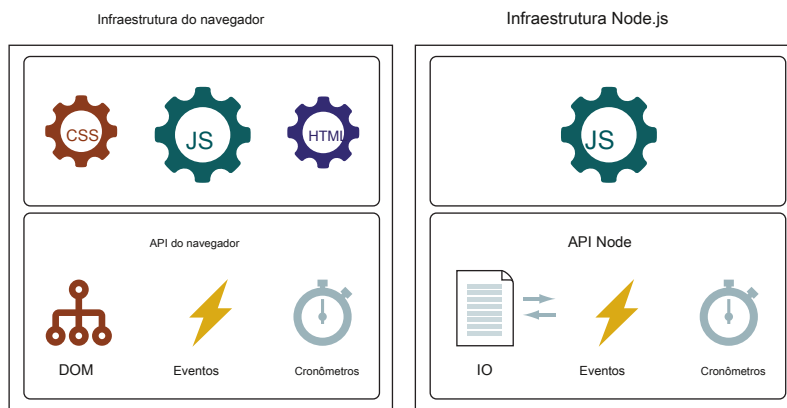


Figura 1.1 Os aplicativos da web do lado do cliente dependem da infraestrutura fornecida pelo navegador. Vamos nos concentrar particularmente no DOM, eventos, temporizadores e APIs do navegador.

Vamos nos concentrar no seguinte:

- *O Document Object Model (DOM)* - O DOM é uma representação estruturada da IU de um aplicativo da web do lado do cliente que é, pelo menos inicialmente, construído a partir do código HTML de um aplicativo da web. Para desenvolver ótimos aplicativos, você precisa não apenas ter um conhecimento profundo da mecânica básica do JavaScript, mas também estudar como o DOM é construído (capítulo 2) e como escrever um código eficaz que manipule o DOM (capítulo 12). Isso colocará a criação de interfaces de usuário avançadas e altamente dinâmicas ao seu alcance.

- *Eventos*—A grande maioria dos aplicativos JavaScript são *orientado por eventos* aplicativos, o que significa que a maior parte do código é executada no contexto de uma resposta a um determinado evento. Os exemplos de eventos incluem eventos de rede, cronômetros e eventos gerados pelo usuário, como cliques, movimentos do mouse, pressionamentos de teclado e assim por diante. Por esse motivo, exploraremos completamente os mecanismos por trás dos eventos no capítulo 13. Prestaremos atenção especial a *temporizadores*, que são frequentemente um mistério, mas vamos lidar com tarefas complexas de codificação, como cálculos de longa duração e animações suaves.
- *API do navegador*—Para nos ajudar a interagir com o mundo, o navegador oferece uma API que nos permite acessar informações sobre dispositivos, armazenar dados localmente ou nos comunicar com servidores remotos. Exploraremos algumas dessas APIs ao longo do livro.

Aperfeiçoar suas habilidades de programação JavaScript e alcançar um conhecimento profundo das APIs oferecidas pelo navegador o levará longe. Mas mais cedo, ao invés de mais tarde, você vai se deparar com *os navegadores* e seus vários problemas e inconsistências. Em um mundo perfeito, todos os navegadores estariam livres de bugs e ofereceriam suporte aos padrões da web de maneira consistente; infelizmente, não vivemos nesse mundo.

A qualidade dos navegadores melhorou muito ultimamente, mas todos eles ainda têm alguns bugs, APIs ausentes e peculiaridades específicas do navegador com as quais precisamos lidar. O desenvolvimento de uma estratégia abrangente para lidar com esses problemas do navegador e se familiarizar intimamente com suas diferenças e peculiaridades pode ser quase tão importante quanto a proficiência no próprio JavaScript.

Quando estamos escrevendo aplicativos de navegador ou bibliotecas JavaScript para serem usados neles, escolher quais navegadores dar suporte é uma consideração importante. Gostaríamos de oferecer suporte a todos eles, mas as limitações nos recursos de desenvolvimento e teste ditam o contrário. Por esse motivo, exploraremos exaustivamente as estratégias de desenvolvimento entre navegadores no capítulo 14.

O desenvolvimento de código eficaz para vários navegadores pode depender significativamente da habilidade e experiência dos desenvolvedores. Este livro tem como objetivo aumentar esse nível de habilidade, então vamos começar examinando as melhores práticas atuais.

1,3

Usando as melhores práticas atuais

O domínio da linguagem JavaScript e uma compreensão dos problemas de codificação entre navegadores são partes importantes para se tornar um desenvolvedor especialista de aplicativos da Web, mas não são o quadro completo. Para entrar nas grandes ligas, você também precisa exibir as características que dezenas de desenvolvedores anteriores comprovaram serem benéficas para o desenvolvimento de código de qualidade. Essas características são conhecidas como *Melhores Práticas*, e, além do domínio da linguagem, incluem elementos como

- Habilidades de depuração
- Testando
- Análise de desempenho

É de vital importância aderir a essas práticas ao codificar e as usaremos ao longo do livro. Vamos examinar alguns deles a seguir.

1.3.1 *Depurando*

Depurar JavaScript costumava significar usar alerta para verificar o valor das variáveis. Felizmente, a capacidade de depurar código JavaScript melhorou drasticamente, em grande parte por causa da popularidade da extensão do desenvolvedor Firebug para Firefox. Ferramentas semelhantes foram desenvolvidas para todos os principais navegadores:

- *Firebug*—A popular extensão de desenvolvedor para Firefox que fez a bola rolar (<http://getfirebug.com/>)
- *Chrome DevTools*—Desenvolvido pela equipe do Chrome e usado no Chrome e no Opera
- *Ferramentas de desenvolvedor do Firefox*—Uma ferramenta incluída no Firefox, desenvolvida pela equipe do Firefox
- *Ferramentas de desenvolvedor F12*—Incluído no Internet Explorer e Microsoft Edge
- *Inspetor WebKit*—Usado pelo Safari

Como você pode ver, todos os principais navegadores oferecem ferramentas de desenvolvedor que podemos usar para depurar nossos aplicativos da web. Os dias de usar alertas JavaScript para depuração já se foram!

Todas essas ferramentas são baseadas em ideias semelhantes, que foram introduzidas principalmente pelo Firebug, de modo que oferecem funcionalidades semelhantes: explorar o DOM, depurar JavaScript, editar estilos CSS, rastrear eventos de rede e assim por diante. Qualquer um deles fará um bom trabalho; use aquele oferecido pelo navegador de sua escolha ou no navegador no qual você está investigando bugs.

Além disso, você pode usar alguns deles, como o Chrome Dev Tools, para depurar outros tipos de aplicativos, como aplicativos Node.js. (Apresentaremos algumas técnicas de depuração no Apêndice B.)

1.3.2 *Testando*

Ao longo deste livro, aplicaremos técnicas de teste para garantir que o código de exemplo opere conforme pretendido e para servir como exemplos de como testar o código em geral. A principal ferramenta que usaremos para teste é um afirmar função, cujo propósito é afirmar que uma premissa é verdadeira ou falsa. Ao especificar asserções, podemos verificar se o código está se comportando conforme o esperado.

A forma geral desta função é a seguinte:

afirmar (condição, mensagem);

O primeiro parâmetro é uma condição que deveria ser verdadeira, e o segundo é uma mensagem que será exibida se não for.

Considere isto, por exemplo:

```
assert (a === 1, "Desastre! a não é 1!");
```

Se o valor da variável `uma` não é igual a 1, a afirmação falha e a mensagem um tanto excessivamente dramática é exibida.

NOTA O afirmar função não é um recurso padrão da linguagem, então vamos implementá-la nós mesmos no apêndice B.

1.3.3 *Análise de desempenho*

Outra prática importante é a análise de desempenho. Os motores JavaScript deram passos surpreendentes no desempenho do JavaScript, mas isso não é desculpa para escrever código desleixado e ineficiente.

Usaremos um código como o seguinte posteriormente neste livro para coletar informações de desempenho:

```
console.time("Minha operação");
```

← Inicia o cronômetro

```
para (var n = 0; n < maxCount; n++) {
  /* realizar a operação a ser medida */
}
```

| Executa
a operação
várias vezes

```
console.timeEnd("Minha operação");
```

← Para o cronômetro

Aqui, colocamos entre parênteses a execução do código a ser medido com duas chamadas para o `Time` e `timeEnd` métodos do embutido `console` objeto.

Antes que a operação comece a ser executada, a chamada para `console.time` inicia um cronômetro com um nome (neste caso, `Minha operação`). Em seguida, executamos o código no `for` loop um certo número de vezes (neste caso, `maxCount` vezes). Como uma única operação do código acontece muito rapidamente para ser medida de maneira confiável, precisamos executar o código muitas vezes para obter um valor mensurável. Frequentemente, essa contagem pode chegar a dezenas de milhares, ou mesmo milhões, dependendo da natureza do código que está sendo medido. Uma pequena tentativa e erro nos permite escolher um valor razoável.

Quando a operação termina, chamamos o `console.timeEnd` método com o mesmo nome. Isso faz com que o navegador exiba o tempo decorrido desde que o cronômetro foi iniciado.

Essas técnicas de prática recomendada, junto com outras que você aprenderá ao longo do caminho, irão aprimorar muito o seu desenvolvimento de JavaScript. O desenvolvimento de aplicativos com os recursos restritos que um navegador oferece, juntamente com o mundo cada vez mais complexo da capacidade e compatibilidade do navegador, requer um conjunto robusto e completo de habilidades.

1,4 *Aumentando a capacidade de transferência de habilidades*

Quando Bob estava aprendendo desenvolvimento web pela primeira vez, cada navegador tinha sua própria maneira de interpretar estilos de script e IU, pregando que seu jeito era o melhor e fazendo todo desenvolvedor ranger de frustração. Felizmente, as guerras dos navegadores terminaram com a padronização de HTML, CSS, a API DOM e o JavaScript e o foco do desenvolvedor voltando-se para aplicativos JavaScript entre navegadores eficazes. Na verdade, este foco em

tratar sites como aplicativos levou a muitas ideias, ferramentas e técnicas que passaram de aplicativos de desktop para aplicativos da web. E agora, essa transferência de conhecimento e ferramentas aconteceu novamente, à medida que ideias, ferramentas e técnicas originadas no desenvolvimento da Web no cliente também permearam outros domínios de aplicação.

Alcançar um entendimento profundo dos princípios fundamentais do JavaScript com o conhecimento das principais APIs pode, portanto, torná-lo um desenvolvedor mais versátil. Usando os navegadores e Node.js (um ambiente derivado do navegador), você pode desenvolver quase qualquer tipo de aplicativo imaginável:

- *Aplicativos de desktop*, usando, por exemplo, NW.js (<http://nwjs.io/>) ou elêtron (<http://electron.atom.io/>)
Essas tecnologias geralmente envolvem o navegador para que possamos construir UIs de desktop com HTML, CSS e JavaScript padrão (dessa forma, podemos confiar em nosso JavaScript básico e conhecimento do navegador), com suporte adicional que torna possível interagir com o sistema de arquivos. Podemos construir aplicativos de desktop verdadeiramente independentes de plataforma que tenham a mesma aparência no Windows, Mac e Linux.
- *Aplicativos móveis com estruturas*, como Apache Cordova (<https://cordova.apache.org/>) Semelhante aos aplicativos de desktop desenvolvidos com tecnologias da web, as estruturas para aplicativos móveis usam um navegador empacotado, mas com APIs adicionais específicas da plataforma que nos permitem interagir com a plataforma móvel.
- *Aplicativos do lado do servidor e aplicativos para dispositivos incorporados com Node.js*, um ambiente derivado do navegador que usa muitos dos mesmos princípios básicos do navegador. Por exemplo, Node.js executa código JavaScript e depende de eventos.

Ann não sabe o quão sortuda ela é (embora Bob tenha uma boa ideia). Não importa se ela precisa construir um aplicativo de desktop padrão, um aplicativo móvel, um aplicativo do lado do servidor ou mesmo um aplicativo incorporado - todos esses tipos de aplicativos compartilham alguns dos mesmos princípios básicos dos aplicativos da web do lado do cliente padrão. Ao entender como a mecânica principal do JavaScript funciona e compreender as APIs principais fornecidas pelos navegadores (como eventos, que também têm muito em comum com os mecanismos fornecidos pelo Node.js), ela pode impulsionar suas habilidades de desenvolvimento em todo o quadro. Como você pode. No processo, você se tornará um desenvolvedor mais versátil e ganhará o conhecimento e a compreensão para lidar com uma ampla variedade de problemas.

1,5 Resumo

- Os aplicativos da Web do lado do cliente estão entre os mais populares hoje, e os conceitos, ferramentas e técnicas antes usados apenas para seu desenvolvimento permearam outros domínios de aplicativo. Compreender os fundamentos dos aplicativos da web do lado do cliente o ajudará a desenvolver aplicativos para uma ampla variedade de domínios.

- Para melhorar suas habilidades de desenvolvimento, você precisa obter um conhecimento profundo da mecânica central do JavaScript, bem como da infraestrutura fornecida pelos navegadores.
- Este livro enfoca os principais mecanismos do JavaScript, como funções, encerramentos de função e protótipos, bem como novos recursos do JavaScript, como geradores, promessas, proxies, mapas, conjuntos e módulos.
- JavaScript pode ser executado em um grande número de ambientes, mas o ambiente onde tudo começou, e o ambiente no qual nos concentraremos, é o navegador.
- Além do JavaScript, exploraremos os aspectos internos do navegador, como o DOM (uma representação estruturada da IU da página da web) e eventos, porque os aplicativos da web do lado do cliente são aplicativos orientados a eventos.
- Faremos essa exploração com as melhores práticas em mente: depuração, teste e análise de desempenho.

Prédio

a página em tempo de execução

Este capítulo cobre

- Etapas no ciclo de vida de um aplicativo da web
- Processando código HTML para produzir uma página da web
- Ordem de execução do código JavaScript
- Alcançando interatividade com eventos
- O loop de eventos

Nossa exploração de JavaScript é realizada no contexto de aplicativos da Web do lado do cliente e do navegador como o mecanismo que executa o código JavaScript. Para ter uma base sólida a partir da qual continuar explorando JavaScript como linguagem e o navegador como plataforma, primeiro temos que entender o ciclo de vida completo do aplicativo da Web e, especialmente, como nosso código JavaScript se encaixa nesse ciclo de vida.

Neste capítulo, exploraremos completamente o ciclo de vida dos aplicativos da Web do lado do cliente, desde o momento em que a página é solicitada, por meio de várias interações realizadas pelo usuário, até o fechamento da página. Primeiro, vamos explorar como a página é construída processando o código HTML. Em seguida, vamos nos concentrar na execução do código JavaScript, que adiciona a dinâmica muito necessária às nossas páginas. E finalmente

veremos como os eventos são tratados para desenvolver aplicativos interativos que respondem às ações dos usuários.

Durante esse processo, exploraremos alguns conceitos fundamentais de aplicativos da web, como o DOM (uma representação estruturada de uma página da web) e o loop de eventos (determina como os eventos são tratados pelos aplicativos). Vamos mergulhar!

O navegador sempre constrói a página exatamente de acordo
para o HTML fornecido?

Você sabe?

Quantos eventos um aplicativo da web pode manipular de uma vez? Por que os navegadores
devem usar uma fila de eventos para processar eventos?

2,1

A visão geral do ciclo de vida

O ciclo de vida de um aplicativo da Web típico do lado do cliente começa com o usuário digitando uma URL na barra de endereços do navegador ou clicando em um link. Digamos que desejamos pesquisar um termo e acessar a página inicial do Google. Nós digitamos o URL www.google.com, conforme mostrado no canto superior esquerdo na figura 2.1.

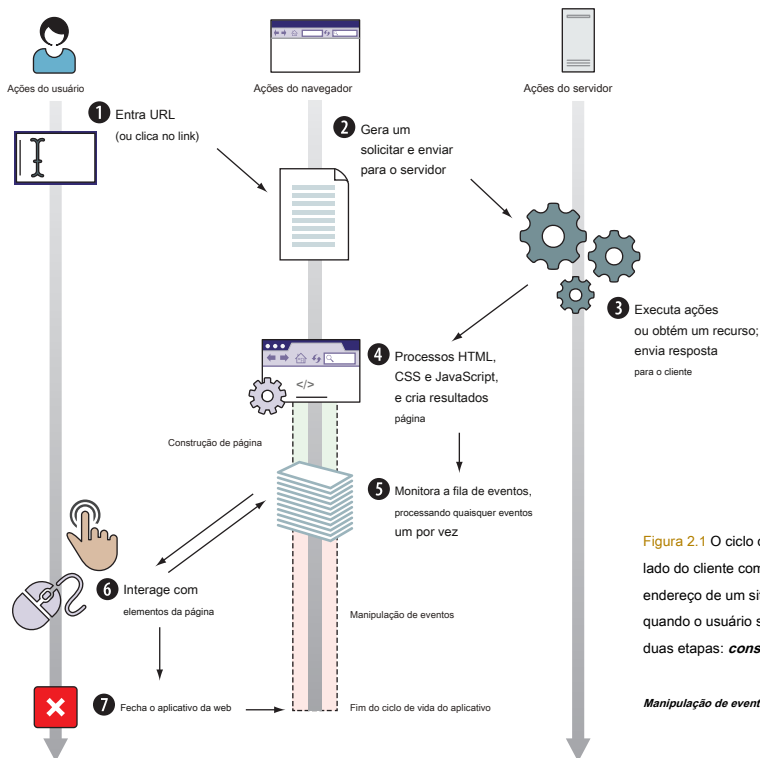


Figura 2.1 O ciclo de vida de um aplicativo da web do lado do cliente começa com o usuário especificando o endereço de um site (ou clicando em um link) e termina quando o usuário sai da página da web. É composto de duas etapas: **construção de página** e

Manipulação de eventos.

Em nome do usuário, o navegador formula uma solicitação que é enviada a um servidor **C**, que processa o pedido **D** e formula uma resposta que geralmente é composta de Código HTML, CSS e JavaScript. No momento em que o navegador recebe esta resposta **E** é quando nosso aplicativo da web do lado do cliente realmente começa a ganhar vida.

Como os aplicativos da web do lado do cliente são aplicativos de interface gráfica do usuário (GUI), seu ciclo de vida segue fases semelhantes a outros aplicativos de GUI (pense em aplicativos de desktop padrão ou aplicativos móveis) e é realizado nas duas etapas a seguir:

- 1 *Construção de página*—Configure a interface do usuário.
- 2 *Manipulação de eventos*—Digite um loop **F** esperando que os eventos ocorram **G**, e iniciar invok-manipuladores de eventos.

O ciclo de vida do aplicativo termina quando o usuário fecha ou sai da página da web **H**.

Agora, vamos ver um exemplo de aplicativo da web com uma interface de usuário simples que reage ao usuário ações: sempre que um usuário move o mouse ou clica na página, uma mensagem é exibida. Usaremos esse aplicativo ao longo do capítulo.

Listagem 2.1 Pequeno aplicativo da web com uma GUI reagindo a eventos

```
<!DOCTYPE html>
<html>
  <head>
    <title> Ciclo de vida do aplicativo da web </title>
    <style>
      # primeira {cor: verde;}
      # segundo {cor: vermelho;}
    </style>
  </head>
  <body>
    <ul id = "first"> </ul>

    <script>
      function addMessage (elemento, mensagem) {
        var messageElement = document.createElement ("li"); messageElement.textContent =
          mensagem;
        element.appendChild (messageElement);
      }

      var primeiro = document.getElementById ("primeiro"); addMessage (primeiro,
        "Carregamento de página");
    </script>

    <ul id = "second"> </ul>

    <script>
      document.body.addEventListener ("mousemove", function () {
        var segundo = document.getElementById ("segundo"); addMessage (segundo, "Evento:
          mousemove");
      });

      document.body.addEventListener ("click", function () {
```

Define uma função
que adiciona uma
mensagem a um elemento

Anexa mousemove
manipulador de eventos para o corpo

Anexa evento de clique
manipulador para o corpo

```

        var segundo = document.getElementById ("segundo"); addMessage (segundo, "Evento:
        clique");
    });
</script>
</body>
</html>

```

A Listagem 2.1 primeiro define duas regras CSS, # primeiro e # segundo, que especificam a cor do texto para os elementos com os IDs primeiro e segundo (para que possamos distinguir facilmente entre eles). Continuamos definindo um elemento de lista com o id primeiro:

```
<ul id = "first"> </ul>
```

Então, definimos um adicionar mensagem função que, quando chamada, cria um novo elemento de item de lista, define seu conteúdo de texto e o anexa a um elemento existente:

```

function addMessage (elemento, mensagem) {
    var messageElement = document.createElement ("li"); messageElement.textContent =
    mensagem;
    elemento.appendChild (messageElement);
}

```

Seguimos isso usando o integrado getElementById método para buscar um elemento com o ID primeiro do documento e adicionando uma mensagem a ele que nos notifica que a página está carregando:

```

var primeiro = document.getElementById ("primeiro"); addMessage (primeiro,
"Carregamento de página");

```

Em seguida, definimos outro elemento de lista, agora com o atributo ID segundo:

```
<ul id = "second"> </ul>
```

Finalmente, anexamos dois manipuladores de eventos ao corpo da página da web. Começamos com o mousemove manipulador de eventos, que é executado toda vez que o usuário move o mouse e adiciona uma mensagem " Evento: mousemove " ao segundo elemento de lista chamando o adicionar mensagem função:

```

document.body.addEventListener ("mousemove", function () {
    var segundo = document.getElementById ("segundo"); addMessage (segundo, "Evento:
    mousemove");
});

```

Também registramos um clique manipulador de eventos, que, sempre que o usuário clica na página, registra uma mensagem " Evento: clique em ", também para o segundo elemento da lista:

```

document.body.addEventListener ("click", function () {
    var segundo = document.getElementById ("segundo"); addMessage (segundo, "Evento:
    clique");
});

```

O resultado da execução e interação com este aplicativo é mostrado na figura 2.2.

Usaremos este aplicativo de exemplo para explorar e ilustrar as diferenças entre as diferentes fases do ciclo de vida do aplicativo da web. Vamos começar com a fase de criação de página.

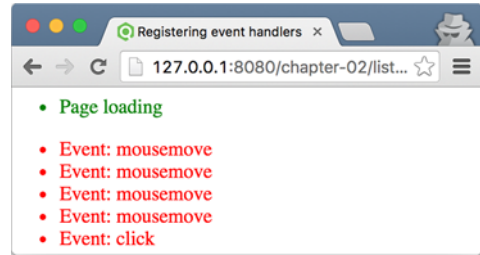


Figura 2.2 Quando o código da listagem 2.1 é executado, as mensagens são registradas dependendo das ações do usuário.

2,2

A fase de construção da página

Antes que um aplicativo da web possa ser interagido ou até mesmo exibido, a página deve ser

construído a partir das informações em sua resposta recebida do servidor (geralmente código HTML, CSS e JavaScript). O objetivo desta fase de construção de página é configurar a UI de um aplicativo e isso é feito em duas etapas distintas:

- 1 Analisando o HTML e construindo o Document Object Model (DOM)
- 2 Executando código JavaScript

A etapa 1 é realizada quando o navegador está processando nós HTML, e a etapa 2 é realizada sempre que um tipo especial de elemento HTML - o roteiro elemento (que contém ou se refere ao código JavaScript) - é encontrado. Durante a fase de construção da página, o navegador pode alternar entre essas duas etapas quantas vezes forem necessárias, conforme mostrado na figura 2.3.

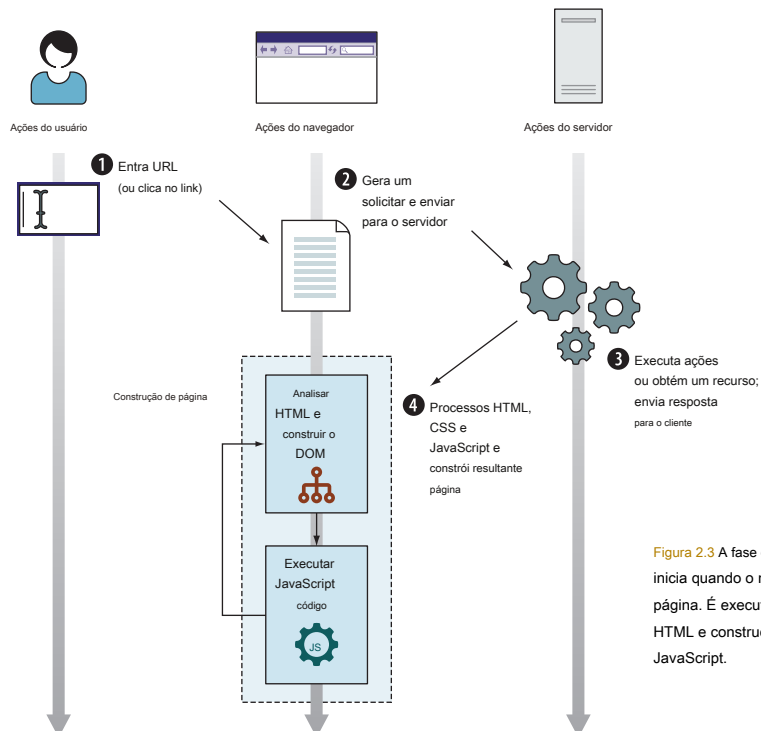


Figura 2.3 A fase de construção da página inicia quando o navegador recebe o código da página. É executado em duas etapas: análise do HTML e construção do DOM e execução do código JavaScript.

2.2.1 Analisando o HTML e construindo o DOM

A fase de construção da página começa com o navegador recebendo o código HTML, que é usado como base sobre a qual o navegador constrói a IU da página. O navegador faz isso analisando o código HTML, um elemento HTML por vez, e construindo um DOM, uma representação estruturada da página HTML na qual cada elemento HTML é representado como um nó. Por exemplo, a figura 2.4 mostra o DOM da página de exemplo que foi construída até o primeiro roteiro elemento é alcançado.

Observe como os nós na figura 2.4 são organizados de tal forma que cada nó, exceto o primeiro (a raiz `html` nó **B**) tem exatamente um dos pais. Por exemplo, o cabeça nó **C** tem o `html` nó **B** como seu pai. Ao mesmo tempo, um nó pode ter qualquer número de filhos. Por exemplo, o `html` nó **B** tem dois filhos: o cabeça nó **C** e a corpo nó **H**. Filhos do mesmo elemento são chamados *irmãos*. (O cabeça nó **C** e a corpo nó **H** são irmãos.)

É importante enfatizar que, embora o HTML e o DOM sejam intimamente vinculados, com o DOM sendo construído a partir de HTML, eles não são o mesmo. Você deve pensar no código HTML como um *planta* o navegador segue ao construir o DOM inicial - a IU - da página. O navegador pode até corrigir problemas que

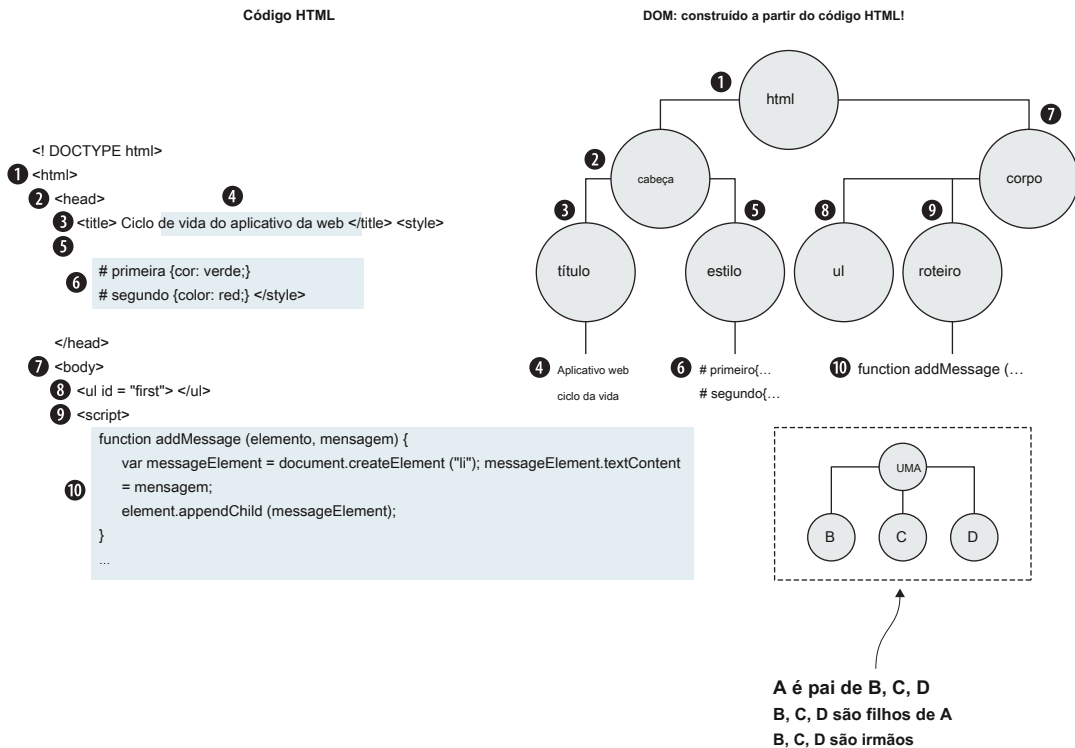


Figura 2.4 No momento em que o navegador encontra o primeiro **roteiro** elemento, ele já criou um DOM com vários elementos HTML (os nós à direita).

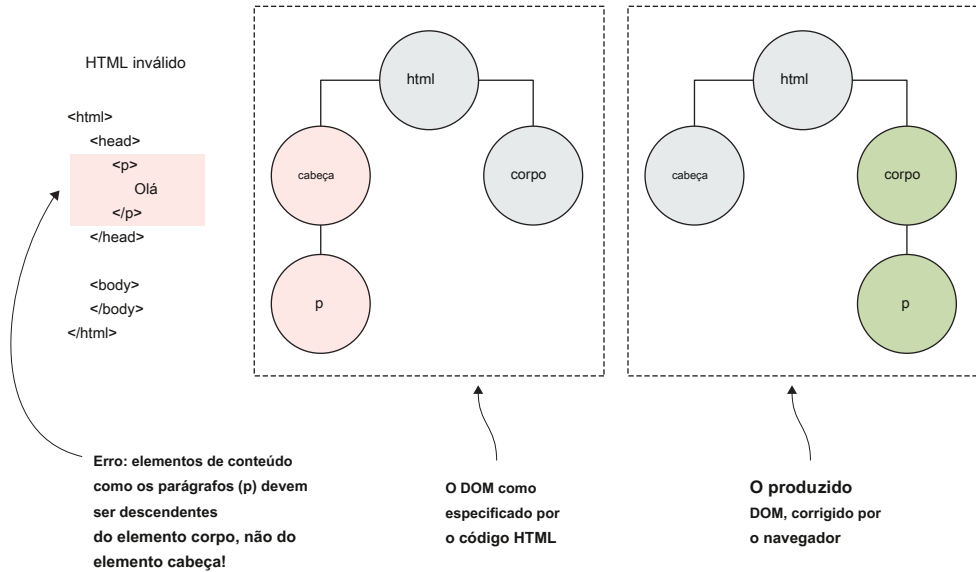


Figura 2.5 Um exemplo de HTML inválido corrigido pelo navegador

encontra com este blueprint para criar um DOM válido. Vamos considerar o exemplo mostrado na figura 2.5.

A Figura 2.5 dá um exemplo simples de código HTML incorreto no qual um elemento de parágrafo é colocado no cabeçalho elemento. A intenção do cabeçalho elemento é usado para fornecer informações gerais da página: por exemplo, o título da página, codificações de caracteres e estilos e scripts externos. Não se destina a definir o conteúdo da página, como neste exemplo. Por ser um erro, o navegador o corrige silenciosamente, construindo o DOM correto (à direita na figura 2.5), no qual o elemento de parágrafo é colocado no corpo elemento, onde o conteúdo da página deveria estar.

Especificação HTML e especificação DOM

A versão atual do HTML é HTML5, cuja especificação está disponível em <https://html.spec.whatwg.org/>. Se você precisar de algo mais legível, recomendamos o guia HTML5 da Mozilla, disponível em <https://developer.mozilla.org/en-US/docs/Guide/HTML/HTML5>.

O DOM, por outro lado, está evoluindo um pouco mais devagar. A versão atual é DOM3, cuja especificação está disponível em <https://dom.spec.whatwg.org/>. Novamente, a Mozilla preparou um relatório que pode ser encontrado em https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model.

Durante a construção da página, o navegador pode encontrar um tipo especial de elemento HTML, o roteiro elemento, que é usado para incluir o código JavaScript. Quando isso acontece, o navegador pausa a construção do DOM a partir do código HTML e começa a executar o código JavaScript.

2.2.2 Executando código JavaScript

Todo o código JavaScript contido no roteiro elemento é executado pelo mecanismo JavaScript do navegador; por exemplo, Spidermonkey do Firefox, Chrome e Opera's V8, ou Edge's (IE's) Chakra. Como o objetivo principal do código JavaScript é fornecer dinamismo à página, o navegador fornece uma API por meio de um objeto global que pode ser usado pelo mecanismo JavaScript para interagir e modificar a página.

OBJETOS GLOBAIS EM JAVASCRIPT

O principal objeto global que o navegador expõe ao mecanismo JavaScript é o `global` objeto, que representa a janela na qual uma página está contida. O `global` objeto é o objeto global por meio do qual todos os outros objetos globais, variáveis globais (mesmo aquelas definidas pelo usuário) e APIs do navegador são acessíveis. Uma das propriedades mais importantes do `global` objeto é o `document`, que representa o DOM da página atual. Ao usar este objeto, o código JavaScript pode alterar o DOM da página em qualquer grau, modificando ou removendo elementos existentes e até mesmo criando e inserindo novos.

Vejamos um snippet de código da listagem 2.1:

```
var primeiro = document.getElementById ("primeiro");
```

Este exemplo usa o `global document` objeto para selecionar um elemento com o ID `primeiro` do DOM e atribui-lo a uma variável `primeiro`. Podemos então usar o código JavaScript para fazer todos os tipos de modificações nesse elemento, como alterar seu conteúdo textual, modificar seus atributos, criar e adicionar dinamicamente novos filhos a ele e até mesmo remover o elemento do DOM.

APIs de navegador

Ao longo do livro, usamos uma série de objetos e funções embutidos no navegador (por exemplo, `window` e `document`). Infelizmente, cobrir tudo que é suportado pelo navegador está além do escopo de um livro sobre JavaScript. Felizmente, a Mozilla novamente nos apoia com <https://developer.mozilla.org/en-US/docs/Web/API>, onde você pode encontrar o status atual das interfaces de API da web.

Com esse entendimento básico dos objetos globais fornecidos pelo navegador, vamos examinar dois tipos diferentes de código JavaScript que definem exatamente quando esse código é executado.

DIFERENTES TIPOS DE CÓDIGO JAVASCRIPT

Diferenciamos amplamente dois tipos diferentes de código JavaScript: *código global* e *código de função*. A listagem a seguir o ajudará a entender as diferenças entre esses dois tipos de código.

Listagem 2.2 Código JavaScript global e de função

```
<script>
  function addMessage (elemento, mensagem) {
    var messageElement = document.createElement ("li"); messageElement.textContent =
    mensagem;
    element.appendChild (messageElement);
  }

  var primeiro = document.getElementById ("primeiro"); addMessage (primeiro,
  "Carregamento de página");
</script>
```

Código de função é o código contido em uma função.

Código global é o código fora das funções.

A principal diferença entre esses dois tipos de código é o seu posicionamento: o código contido em uma função é chamado *código de função*, enquanto o código colocado fora de todas as funções é chamado *código global*.

Esses dois tipos de código também diferem em sua execução (você verá algumas diferenças adicionais posteriormente, especialmente no capítulo 5). O código global é executado automaticamente pelo mecanismo JavaScript (mais sobre isso em breve) de maneira direta, linha por linha, conforme é encontrado. Por exemplo, na listagem 2.2, as partes do código global que definem o adicionar mensagem função usar o embutido getElementById método para buscar o elemento com ID primeiro e ligue para o adicionar mensagem função; eles são executados um após o outro conforme são encontrados, conforme mostrado na figura 2.6.

Por outro lado, o código da função, para ser executado, deve ser chamado por outra coisa: ou pelo código global (por exemplo, o adicionar mensagem chamada de função no código global causa a execução do adicionar mensagem código de função), por alguma outra função ou pelo navegador (mais sobre isso em breve).

EXECUÇÃO DO CÓDIGO JAVASCRIPT NA FASE DE CONSTRUÇÃO DE PÁGINA

Quando o navegador atinge o roteiro na fase de construção da página, ele pausa a construção do DOM com base no código HTML e começa a executar o código JavaScript.

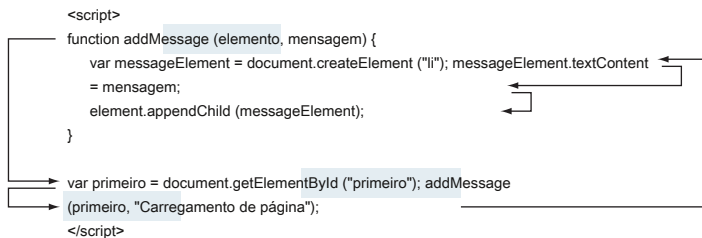


Figura 2.6 Fluxo de execução do programa ao executar o código JavaScript

Isso significa executar o código JavaScript global contido no roteiro elemento (e funções chamadas pelo código global também são executadas). Vamos voltar ao exemplo da listagem 2.1.

A Figura 2.7 mostra o estado do DOM depois que o código JavaScript global foi executado. Vamos caminhar lentamente por sua execução. Primeiro uma função adicionar mensagem é definido:

```
function addMessage (elemento, mensagem) {
    var messageElement = document.createElement ("li"); messageElement.textContent =
    mensagem;
    element.appendChild (messageElement);
}
```

Em seguida, um elemento existente é buscado no DOM usando o global documento objeto e seu getElementById método:

```
var primeiro = document.getElementById ("primeiro");
```

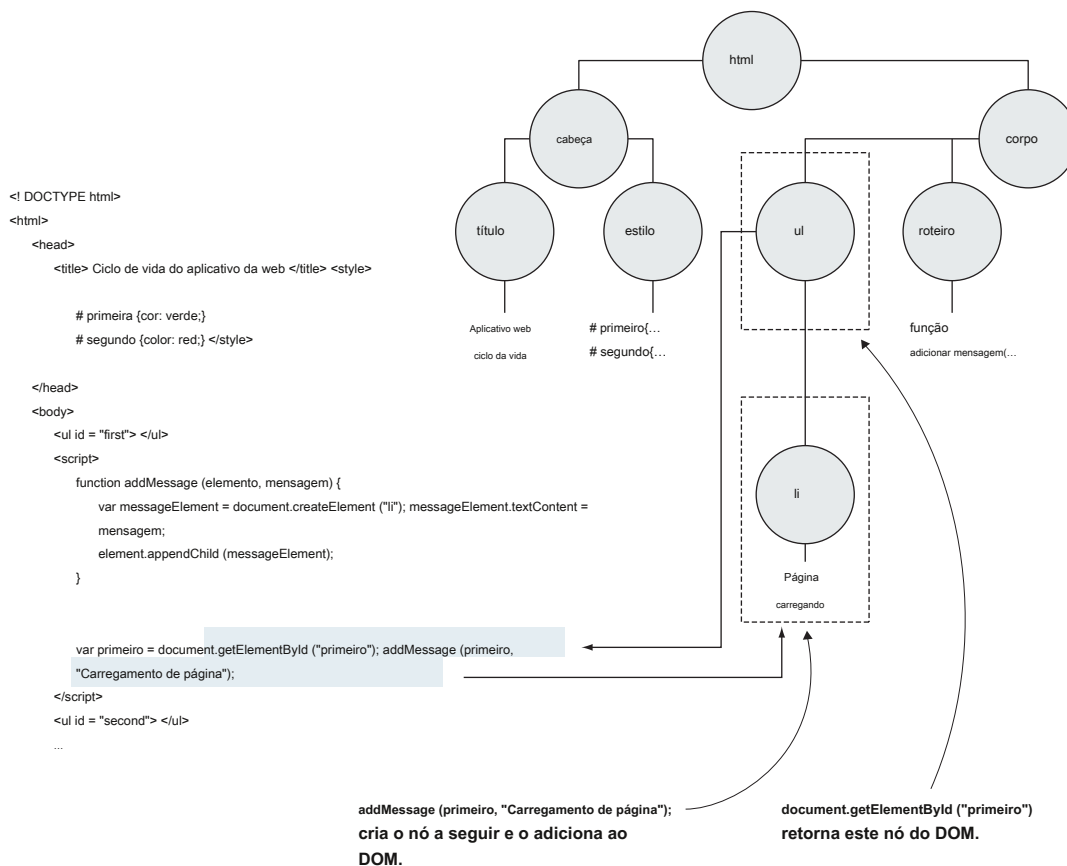


Figura 2.7 O DOM da página após a execução do código JavaScript contido no **roteiro** elemento

Isso é seguido por uma chamada para o adicionar mensagem função

```
addMessage (primeiro, "Carregamento de página");
```

que causa a criação de um novo li elemento, a modificação de seu conteúdo de texto e, finalmente, sua inserção no DOM.

Neste exemplo, o código JavaScript modifica o DOM atual criando um novo elemento e inserindo-o no DOM. Mas, em geral, o código JavaScript pode modificar o DOM em qualquer grau: ele pode criar novos nós e modificar ou remover nós DOM existentes. Mas também há algumas coisas que ele não pode fazer, como selecionar e modificar elementos que ainda não foram criados. Por exemplo, não podemos selecionar e modificar o ul elemento com o ID segundo, porque esse elemento é encontrado após o atual roteiro nó e ainda não foi alcançado e criado. Essa é uma das razões pelas quais as pessoas tendem a colocar seus roteiro elementos na parte inferior da página. Dessa forma, não precisamos nos preocupar se um determinado elemento HTML foi atingido.

Depois que o mecanismo de JavaScript executa a última linha do código JavaScript no roteiro elemento (na figura 2.5, isso significa retornar do adicionar mensagem), o navegador sai do modo de execução JavaScript e continua construindo nós DOM processando o código HTML restante. Se, durante esse processamento, o navegador encontrar novamente um roteiro , a criação do DOM a partir do código HTML é novamente pausada e o tempo de execução do JavaScript começa a executar o código JavaScript contido. É importante observar que o estado global do aplicativo JavaScript persiste enquanto isso. Todas as variáveis globais definidas pelo usuário criadas durante a execução do código JavaScript em um roteiro elemento são normalmente acessíveis ao código JavaScript em outro roteiro elementos Isso acontece porque o global janela O objeto, que armazena todas as variáveis JavaScript globais, está ativo e acessível durante todo o ciclo de vida da página.

Essas duas etapas

- 1 Construindo o DOM a partir de HTML
- 2 Executando código JavaScript

são repetidos enquanto houver HTML elementos para processar e código JavaScript a ser executado.

Finalmente, quando o navegador fica sem elementos HTML para processar, a fase de construção da página está concluída. O navegador então passa para a segunda parte do ciclo de vida do aplicativo da web: *Manipulação de eventos*.

2,3

Manipulação de eventos

Os aplicativos da Web do lado do cliente são aplicativos GUI, o que significa que eles reagem a diferentes tipos de eventos: movimentos do mouse, cliques, pressionamentos do teclado e assim por diante. Por esse motivo, o código JavaScript executado durante a fase de construção da página, além de influenciar o estado global do aplicativo e modificar o DOM, também pode registrar ouvintes (ou manipuladores) de eventos: funções que são executadas pelo navegador quando ocorre um evento. Com esses manipuladores de eventos, nós fornecemos interatividade para nossos aplicativos. Mas antes de examinar mais de perto o registro de manipuladores de eventos, vamos examinar as idéias gerais por trás do tratamento de eventos.

2.3.1 Visão geral do tratamento de eventos

O ambiente de execução do navegador é, em sua essência, baseado na ideia de que apenas uma única parte do código pode ser executada de uma vez: o chamado *single-threaded* modelo de execução. Pense em uma fila no banco. Todos ficam em uma única fila e têm que esperar sua vez de serem “processados” pelos caixas. Mas com JavaScript, apenas 1 a janela do caixa está aberta! Os clientes (eventos) são processados apenas um de cada vez, quando chega a sua vez. Basta uma pessoa que ache apropriado fazer seu planejamento financeiro para todo o ano fiscal enquanto está na janela do caixa (todos nós encontramos com ela!) Para obstruir o trabalho.

Sempre que ocorre um evento, o navegador deve executar a função de manipulador de eventos associada. Mas não há garantia de que temos usuários extremamente pacientes que sempre esperarão um período de tempo apropriado antes de acionar outro evento. Por esse motivo, o navegador precisa de uma maneira de rastrear os eventos que ocorreram, mas ainda não foram processados. Para fazer isso, o navegador usa um *fila de eventos*, conforme mostrado na figura 2.8.

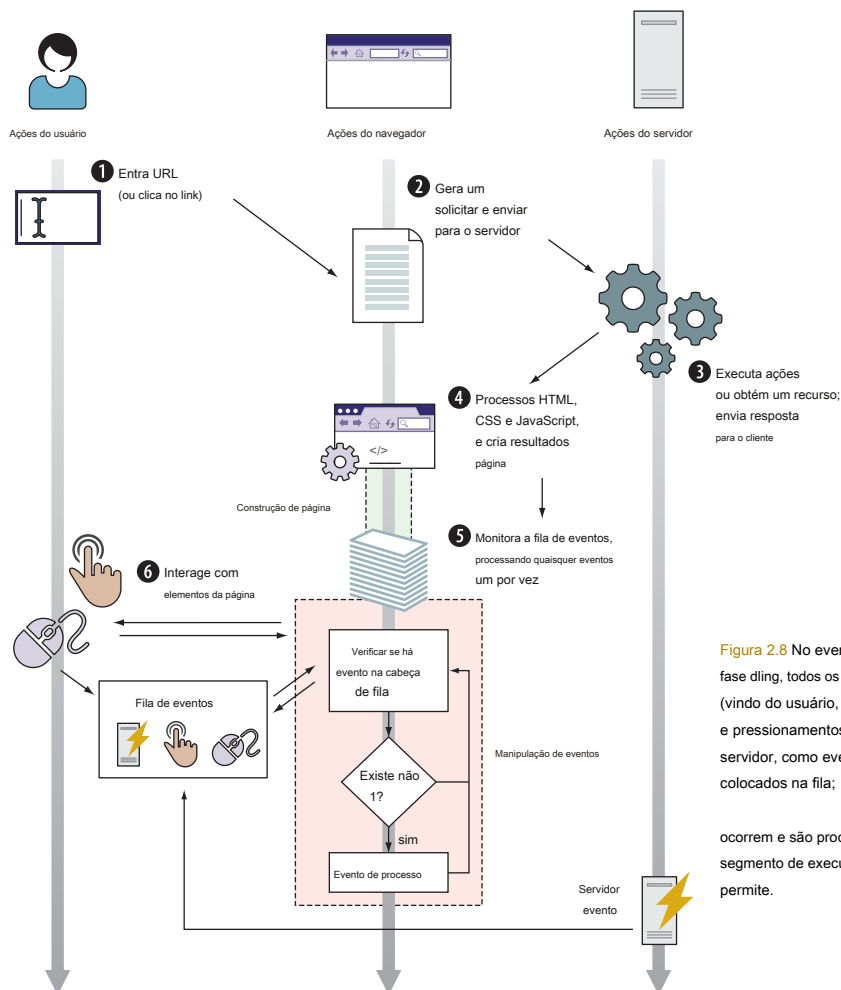


Figura 2.8 No evento-handling, todos os eventos (vindo do usuário, como cliques do mouse e pressionamentos de tecla, ou vindo do servidor, como eventos Ajax) são colocados na fila;

ocorrem e são processados como o único segmento de execução permite.

Todos os eventos gerados (não importa se são gerados pelo usuário, como movimentos do mouse ou pressionamentos de tecla, ou gerados pelo servidor, como eventos Ajax) são colocados na mesma fila de eventos, na ordem em que são detectado pelo navegador. Conforme mostrado no meio da figura 2.8, o processo de manipulação de eventos pode então ser descrito com um fluxograma simples:

- O navegador verifica o início da fila de eventos. Se não houver
- eventos, o navegador continua verificando.
- Se houver um evento no topo da fila de eventos, o navegador o pega e executa o manipulador associado (se houver). Durante essa execução, o restante dos eventos aguardam pacientemente na fila de eventos pelo seu turno de processamento.

Como apenas um evento é tratado por vez, temos que ser extremamente cuidadosos sobre a quantidade de tempo necessária para tratar os eventos; escrever manipuladores de eventos que levam muito tempo para executar leva a aplicativos da web que não respondem! (Não se preocupe se isso soar um pouco vago; voltaremos ao loop de eventos no capítulo 13 e veremos exatamente como isso afeta o desempenho percebido dos aplicativos da web).

É importante observar que o mecanismo do navegador que coloca eventos *para* a fila é externa às fases de criação de página e tratamento de eventos. O processamento necessário para determinar quando os eventos ocorreram e que os empurra para a fila de eventos não participa do thread que está *tratando* os eventos.

EVENTOS SÃO ASSÍNCRONOS

Os eventos, quando acontecem, podem ocorrer em momentos imprevisíveis e em uma ordem imprevisível (é difícil forçar os usuários a pressionar teclas ou clicar em alguma ordem específica). Dizemos que o tratamento de eventos e, portanto, a invocação de suas funções de tratamento, é *assíncrono*.

Os seguintes tipos de eventos podem ocorrer, entre outros:

- Eventos do navegador, como quando uma página termina de carregar ou quando deve ser descarregada. Eventos de rede, como respostas vindas do servidor (eventos Ajax, eventos do lado do servidor)
- Eventos do usuário, como cliques do mouse, movimentos do mouse e pressionamentos de tecla. Eventos de cronômetro, como quando um tempo limite expira ou um intervalo é disparado

A grande maioria do código é executada como resultado de tais eventos!

O conceito de manipulação de eventos é central para os aplicativos da Web e é algo que você verá repetidas vezes ao longo dos exemplos neste livro: O código é configurado com antecedência para ser executado posteriormente. Exceto pelo código global, a grande maioria do código que colocamos em uma página será executada como resultado de algum evento.

Antes que os eventos possam ser manipulados, nosso código deve notificar o navegador de que estamos interessados em manipular eventos específicos. Vejamos como registrar manipuladores de eventos.

2.3.2 Registrando manipuladores de eventos

Como já mencionamos, os manipuladores de eventos são funções que desejamos executar sempre que ocorrer um evento específico. Para que isso aconteça, temos que notificar o navegador

que estamos interessados em um evento. Isso é chamado *registro de manipulador de eventos*. Em aplicativos da web do lado do cliente, existem duas maneiras de registrar eventos:

- Atribuindo funções a propriedades especiais
- Usando o integrado `addEventListener` método

Por exemplo, escrever o código a seguir atribui uma função ao especial carregando propriedade do janela objeto:

```
window.onload = function () {};
```

Um manipulador de eventos para o carregar evento (quando o DOM está pronto e totalmente construído) é registrado. (Não se preocupe se a notação no lado direito do operador de atribuição parecer um pouco estranha; falaremos muito sobre as funções nos capítulos posteriores.) Da mesma forma, se quisermos registrar um manipulador para o clique evento no corpo do documento, podemos escrever algo neste sentido:

```
document.body.onclick = function () {};
```

Atribuir funções a propriedades especiais é uma maneira fácil e direta de registrar manipuladores de eventos, e provavelmente você já se deparou com isso. Mas não recomendamos que você registre manipuladores de eventos dessa maneira, porque fazer isso tem uma desvantagem: só é possível registrar um manipulador de função para um determinado evento. Isso significa que é fácil sobrescrever funções de manipulador de eventos anteriores, o que pode ser um pouco frustrante. Felizmente, há uma alternativa: o `addEventListener` método nos permite registrar quantas funções de manipulador de eventos forem necessárias. Para mostrar um exemplo, a listagem a seguir retorna a um trecho do exemplo da listagem 2.1.

Listagem 2.3 Registrando manipuladores de eventos

```
<script>
  document.body.addEventListener("mousemove", function () {
    var segundo = document.getElementById("segundo"); addMessage(segundo, "Evento:
    mousemove");
  });

  document.body.addEventListener("click", function () {
    var segundo = document.getElementById("segundo"); addmessage(segundo, "Evento:
    clique");
  });
</script>
```

← Registra um manipulador para a mousemove evento

← Registra um manipulador para o evento de clique

Este exemplo usa o integrado `addEventListener` método em um elemento HTML para especificar o tipo de evento (`mousemove` ou clique) e a função de manipulador de eventos. Isso significa que sempre que o mouse é movido sobre a página, o navegador chama uma função que adiciona uma mensagem, "Evento: mousemove ", para o elemento da lista com o ID segundo (uma mensagem semelhante, "Evento: clique em ", é adicionado ao mesmo elemento sempre que o corpo é clicado).

Agora que você sabe como configurar manipuladores de eventos, vamos relembrar o fluxograma simples que você viu antes e dar uma olhada mais de perto em como os eventos são tratados.

2.3.3 Tratamento de eventos

A ideia principal por trás do tratamento de eventos é que, quando um evento ocorre, o navegador chama o tratador de eventos associado. Como já mencionamos, devido ao modelo de execução de thread único, apenas um único manipulador de eventos pode ser executado por vez. Todos os eventos a seguir são processados somente após a execução do manipulador de eventos atual estar totalmente concluída!

Vamos voltar ao aplicativo da listagem 2.1. A Figura 2.9 mostra um exemplo de execução em que um usuário rápido moveu e clicou com o mouse.

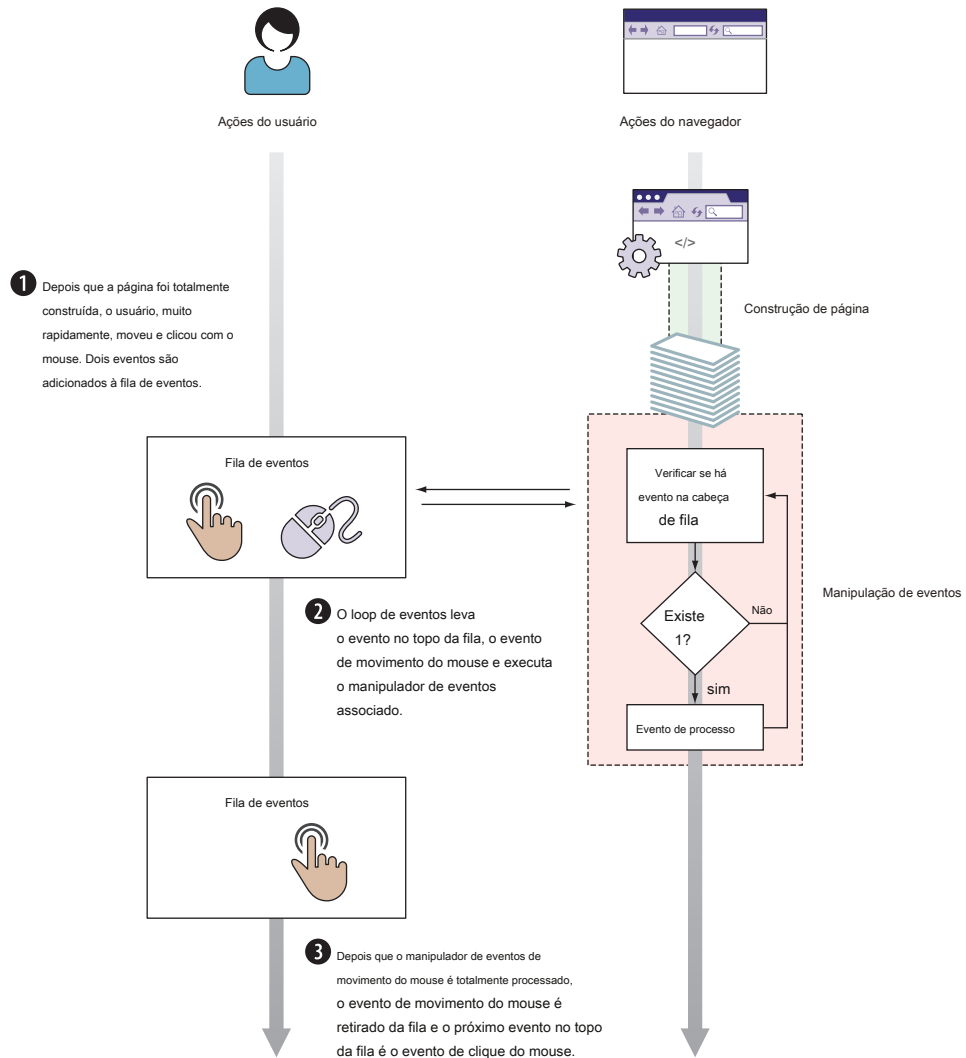


Figura 2.9 Exemplo de uma fase de tratamento de eventos em que dois eventos— **mousemove** e **click** — são manuseados

Vamos examinar o que está acontecendo aqui. Como resposta a essas ações do usuário, o navegador coloca o mousemove e clique eventos na fila de eventos na ordem em que eles ocorreram: primeiro o mousemove evento e então o clique evento **B**.

Na fase de tratamento de eventos, o loop de eventos verifica a fila, vê que há um mousemove evento na frente da fila e executa o evento associado manipulador **C**. Enquanto o mousemove manipulador está sendo processado, o clique evento espera na fila por sua vez. Quando a última linha do mousemove função de manipulador tem fim-executado e o mecanismo JavaScript sai da função de manipulador, o mousemove evento está totalmente processado **D**, e o loop de eventos verifica novamente a fila. Desta vez, na frente da fila, o loop de eventos encontra o clique evento e processa-o. Uma vez a clique handler terminou a execução, não há novos eventos na fila e o loop de eventos continua em loop, esperando que novos eventos sejam tratados. Este loop continuará em execução até que o usuário feche o aplicativo da web.

Agora que temos uma noção das etapas gerais que acontecem na fase de tratamento de eventos, vamos ver como essa execução influencia o DOM (figura 2.10). A execução de

```
<ul id = "first"> </ul>
```

```
<script>
function addMessage (elemento, mensagem) {
  var messageElement = document.createElement ("li"); messageElement.textContent =
  mensagem;
  element.appendChild (messageElement);
}
</script>
```

```
<ul id = "second"> </ul>
```

```
<script>
document.body.addEventListener ('mousemove', function () {
  var segundo = document.getElementById ('segundo'); addMessage (segundo, 'Evento:
  mousemove');
});

document.body.addEventListener ('click', function () {
  var segundo = document.getElementById ('segundo'); addMessage (segundo, 'Evento:
  clique');
});
</script>
```

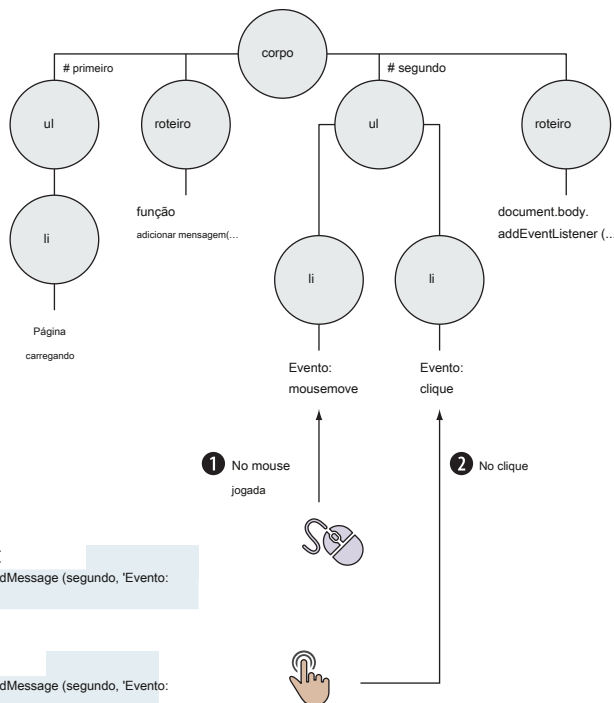


Figura 2.10 O DOM do aplicativo de exemplo após lidar com o **mousemove** e **click** eventos

a mousemove handler seleciona o segundo elemento da lista com ID segundo e, usando a adicionar mensagem função, adiciona um novo elemento de item de lista B com o texto " Evento: mousemove ". Uma vez que a execução do mousemove o manipulador está concluído, o loop de eventos executa o clique manipulador, o que leva à criação de outro elemento de item de lista

C , que também é anexado ao segundo elemento da lista com o ID segundo.

Armado com esta compreensão sólida do ciclo de vida dos aplicativos da web do lado do cliente, na próxima parte do livro, começaremos a nos concentrar na linguagem JavaScript, aprendendo os meandros das funções.

2,4 Resumo

- O código HTML recebido pelo navegador é usado como um blueprint para criar o DOM, uma representação interna da estrutura de um aplicativo da web do lado do cliente. Usamos o código JavaScript para modificar dinamicamente o DOM para trazer comportamento dinâmico para aplicativos da web.
- A execução de aplicativos da web do lado do cliente é realizada em duas fases:
 - *Construção de página* - o código HTML é processado para criar o DOM e o código JavaScript global é executado quando os nós de script são encontrados. Durante essa execução, o código JavaScript pode modificar o DOM atual em qualquer grau e pode até registrar manipuladores de eventos - funções que são executadas quando um evento específico ocorre (por exemplo, um clique do mouse ou um pressionamento de teclado). Registrar manipuladores de eventos é fácil: use o addEventListener método.
 - *Manipulação de eventos* —Vários eventos são processados um de cada vez, na ordem em que foram gerados. A fase de tratamento de eventos depende muito da fila de eventos, na qual todos os eventos são armazenados na ordem em que ocorreram. O loop de eventos sempre verifica se há eventos no topo da fila e, se um evento for encontrado, a função manipuladora de eventos correspondente é chamada.

2,5 Exercícios

- 1 Quais são as duas fases do ciclo de vida de um aplicativo da web do lado do cliente? Qual é a principal vantagem de usar o addEventListener método para registrar um manipulador de eventos versus atribuir um manipulador a uma propriedade de elemento específico? Quantos eventos podem ser processados de uma vez?
- 2
- 3
- 4 Em que ordem os eventos da fila de eventos são processados?

Funções de compreensão

Nomo que você está mentalmente preparado e entende o ambiente no qual o código JavaScript é executado, você está pronto para aprender os fundamentos dos recursos JavaScript disponíveis para você.

No capítulo 3, você aprenderá tudo sobre o conceito básico mais importante de JavaScript: não, não o objeto, mas a função. Este capítulo ensinará por que entender as funções do JavaScript é a chave para desvendar os segredos da linguagem.

O Capítulo 4 continua nossa exploração detalhada das funções, estudando como as funções são chamadas, juntamente com todos os detalhes dos parâmetros implícitos que são acessíveis durante a execução do código da função.

O Capítulo 5 leva as funções para o próximo nível com fechamentos - provavelmente um dos aspectos mais mal compreendidos (e até mesmo desconhecidos) da linguagem JavaScript. Como você verá em breve, os fechamentos estão intimamente ligados aos escopos. Neste capítulo, além dos encerramentos, colocamos um foco especial nos mecanismos de escopo em JavaScript.

Nossa exploração das funções será concluída no capítulo 6, onde discutimos um tipo de função completamente novo - a função geradora - que possui algumas propriedades especiais e é especialmente útil ao lidar com código assíncrono.



Primeira classe

funções para o novato: definições e argumentos

Este capítulo cobre

- Por que entender as funções é tão crucial
- Como as funções são objetos de primeira classe
- As formas de definir uma função
- Os segredos de como os parâmetros são atribuídos

Você pode se surpreender, ao consultar esta parte do livro dedicada aos fundamentos do JavaScript, para ver que o primeiro tópico de discussão é *funções* ao invés de *objetos*. Certamente estaremos prestando muita atenção aos objetos na parte 3 do livro, mas quando se trata de detalhes, a principal diferença entre escrever código JavaScript como a Jill (ou Joe) comum e escrevê-lo como um ninja JavaScript é entender JavaScript como um *linguagem funcional*. O nível de sofisticação de todo o código que você escreverá em JavaScript depende dessa compreensão.

Se você está lendo este livro, você não é um iniciante. Estamos assumindo que você conhece fundamentos de objetos suficientes para sobreviver (e vamos dar uma olhada em

conceitos de objetos mais avançados no capítulo 7), mas *mesmo* compreender as funções em JavaScript é a arma mais importante que você pode usar. Tão importante, na verdade, que este e os três capítulos seguintes são dedicados a uma compreensão completa das funções em JavaScript.

Mais importante, em JavaScript, as funções são *objetos de primeira classe*, ou *cidadãos de primeira classe* como são frequentemente chamados. Eles coexistem e podem ser tratados como qualquer outro objeto JavaScript. Assim como os tipos de dados JavaScript mais comuns, eles podem ser referenciados por variáveis, declarados com literais e até mesmo passados como parâmetros de função. Neste capítulo, vamos primeiro dar uma olhada na diferença que esta orientação para funções traz, e você verá como isso pode nos ajudar a escrever um código mais compacto e fácil de entender, permitindo-nos definir funções exatamente onde Nós precisamos deles. Também exploraremos como tirar proveito das funções como objetos de primeira classe para escrever funções de melhor desempenho. Você verá várias maneiras de definir funções, incluindo alguns novos tipos, como *seta* funções, que o ajudarão a escrever um código mais elegante. Finalmente, veremos a diferença entre parâmetros de função e argumentos de função, com um foco especial nas adições ES6, como o resto e os parâmetros padrão.

Vamos começar examinando alguns dos benefícios da programação funcional.

.....

	Em quais situações as funções de retorno de chamada podem ser usadas sincronicamente? De forma assíncrona?
Você sabe?	Qual é a diferença entre uma função de seta e um expressão de função?
	Por que você pode precisar usar valores de parâmetro padrão em um função?

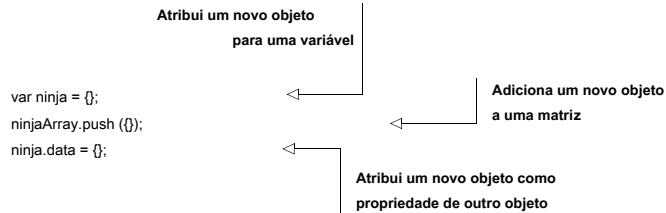
.....

3.1 Qual é a diferença funcional?

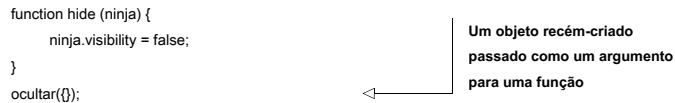
Uma das razões pelas quais funções e conceitos funcionais são tão importantes em JavaScript é que as funções são unidades modulares primárias de execução. Exceto pelo código JavaScript global executado na fase de construção de página, todo o código de script que escreveremos para nossas páginas estará dentro de uma função.

Como a maior parte do nosso código será executado como resultado de uma invocação de função, você verá que ter funções que são construções versáteis e poderosas nos dá uma grande flexibilidade e controle ao escrever código. Partes significativas deste livro explicam como a natureza das funções como objetos de primeira classe pode ser explorada para nosso grande benefício. Mas primeiro, vamos dar uma olhada em algumas das ações que podemos realizar com objetos. Em JavaScript, os objetos desfrutam de certos recursos:

- Eles podem ser criados por meio de literais: {}
- Eles podem ser atribuídos a variáveis, entradas de array e propriedades de outros objetos:



- Eles podem ser passados como argumentos para funções:



- Eles podem ser retornados como valores de funções:



- Eles podem possuir propriedades que podem ser criadas e atribuídas dinamicamente:



Acontece que, ao contrário de muitas outras linguagens de programação, em JavaScript podemos fazer quase exatamente as mesmas coisas com funções também.

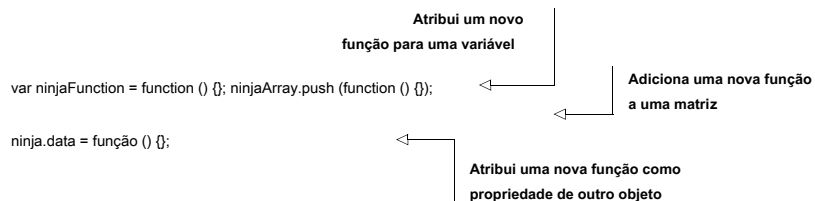
3.1.1 Funciona como objetos de primeira classe

As funções em JavaScript possuem todos os recursos de objetos e, portanto, são tratadas como qualquer outro objeto na linguagem. Dizemos que as funções são *primeira classe* objetos, que também podem ser

- Criado por meio de literais

```
function ninjaFunction () {}
```

- Atribuído a variáveis, entradas de array e propriedades de outros objetos



- Passado como argumentos para outras funções

```
chamada de função (ninjaFunction) {
  ninjaFunction ();
}
chamar (função () {});
```

← Uma função recém-criada
passada como um argumento
para uma função

- Retornado como valores de funções

```
function returnNewNinjaFunction () {
  função de retorno () {};
}
```

← Retorna um novo
função

- Eles podem possuir propriedades que podem ser criadas e atribuídas dinamicamente:

```
var ninjaFunction = function () {}; ninjaFunction.name = "Hanzo";
```

← Adiciona uma nova
propriedade a uma função

O que quer que possamos fazer com objetos, também podemos fazer com funções. Funções *estamos* objetos, apenas com uma capacidade especial adicional de ser *invokable*: As funções podem ser chamadas ou invocadas para executar uma ação.

Programação funcional em JavaScript

Ter funções como objetos de primeira classe é o primeiro passo para *programação funcional*, um estilo de programação que se concentra na solução de problemas por meio da composição de funções (em vez de especificar sequências de etapas, como na programação mais convencional e imperativa). A programação funcional pode nos ajudar a escrever códigos mais fáceis de testar, estender e modularizar. Mas é um grande tópico, e neste livro nós apenas concordamos com ele (por exemplo, no capítulo 9). Se você estiver interessado em aprender como tirar proveito dos conceitos de programação funcional e aplicá-los aos seus programas JavaScript, recomendamos *Programação Funcional em JavaScript* por Luis Atencio (Manning, 2016), disponível em www.manning.com/books/functional-programming-in-javascript

Uma das características dos objetos de primeira classe é que eles podem ser passados para funções como argumentos. No caso de funções, isso significa que passamos uma função como um argumento para outra função que pode, em um ponto posterior da execução do aplicativo, chamar a função passada. Este é um exemplo de um conceito mais geral conhecido como um *função de retorno de chamada*. Vamos explorar este importante conceito.

3.1.2 Funções de retorno de chamada

Sempre que configuramos uma função para ser chamada posteriormente, seja pelo navegador na fase de manipulação de eventos ou por outro código, estamos configurando um *ligar de volta*. O termo deriva do fato de que estamos estabelecendo uma função que outro código irá "chamar de volta" posteriormente em um ponto apropriado de execução.

Callbacks são uma parte essencial do uso eficaz do JavaScript, e estamos dispostos a apostar que você já os usa muito em seu código - seja executando código com um clique de botão, recebendo dados de um servidor ou animando partes de sua IU.

Nesta seção, vamos ver como usar callbacks para manipular eventos ou para classificar coleções facilmente - exemplos típicos do mundo real de como callbacks são usados. Mas é um pouco complexo, então antes de mergulhar, vamos despir o conceito de callback completamente e examiná-lo em sua forma mais simples. Começaremos com um exemplo esclarecedor de uma função completamente inútil que aceita uma referência a outra função como parâmetro e chama essa função como retorno de chamada:

```
function useless (ninjaCallback) {  
    return ninjaCallback ();  
}
```

Por mais inútil que seja, essa função demonstra a capacidade de passar uma função como um argumento para outra função e, posteriormente, invocar essa função por meio do parâmetro passado.

Podemos testar essa função inútil com o código da listagem a seguir.

Listagem 3.1 Um exemplo simples de retorno de chamada

```
var text = "Domo arigato!";  
relatório ("Antes de definir funções");
```

```
function useless (ninjaCallback) {  
    relatório ("Em função inútil");  
    return ninjaCallback ();  
}
```

Define uma função que recebe uma
função de retorno de chamada e a
invoca imediatamente

```
function getText () {  
    report ("Na função getText");  
    texto de retorno;  
}
```

Define um simples
função que retorna
uma variável global

```
relatório ("Antes de fazer todas as ligações");
```

```
assert (useless (getText) === text,  
        "A função inútil funcional" + Texto);
```

Chama nossa função inútil com a função
getText como um retorno de chamada

```
relatório ("Após as ligações terem sido feitas");
```

Nesta lista, usamos um personalizado relatório função para produzir várias mensagens conforme nosso código está sendo executado, para que possamos acompanhar a execução de nosso programa. Nós também usamos o

afirmar função de teste que mencionamos no capítulo 1. O afirmar A função geralmente leva dois argumentos. O primeiro argumento é uma expressão cuja premissa é afirmada. Neste caso, queremos estabelecer se o resultado de invocar nosso sem utilidade função com o argumento getText retorna um valor que é igual ao valor da variável

texto (inútil (getText) === texto). Se o primeiro argumento for avaliado como verdade, a afirmação passa; caso contrário, é considerado um fracasso. O segundo argumento é a mensagem associada, que geralmente é registrada com um indicador de aprovação / reprovação apropriado. (Apêndice C

discute os testes em geral, bem como nossa pequena implementação do afirmar e relatório funções).

Quando executamos esse código, chegamos ao resultado mostrado na figura 3.1. Como você pode ver, chamando o `sem utilidade` funcionar com o nosso `getText` a função de retorno de chamada como um argumento retorna o valor esperado.

Também podemos dar uma olhada em como exatamente este exemplo simples de callback é executado. Como mostra a figura 3.2, passamos no `getText` função para o `sem utilidade` funcionar como um argumento. Isso significa que dentro do corpo do `sem utilidade` função, o `getText` função pode ser referenciada através do `ligar de volta`

parâmetro. Então, fazendo o `ligar de volta()` chamada, nós causamos a execução do `getText` função; a `getText` função, que passamos como um argumento, é chamada de volta pelo `sem utilidade` função.

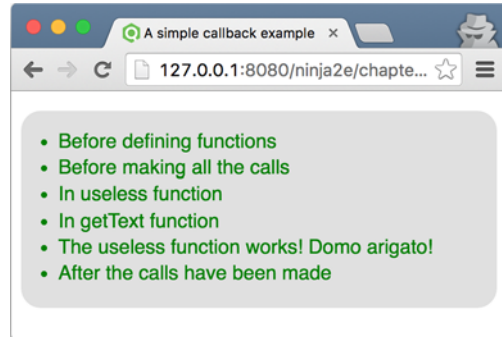


Figura 3.1 O resultado da execução do código da listagem 3.1

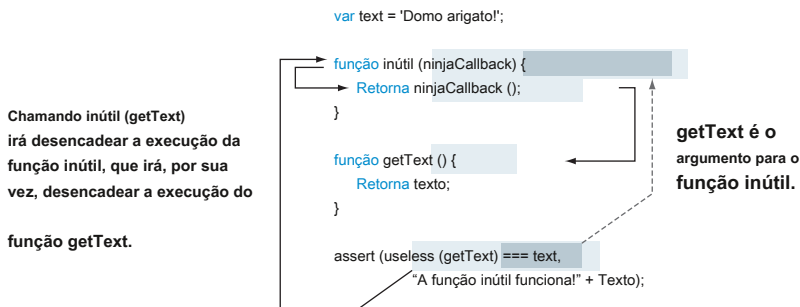


Figura 3.2 O fluxo de execução ao fazer o `inútil (getText)` chamar. O `sem utilidade` função é chamada com `getText` como um argumento. No corpo do `sem utilidade` função é uma chamada para a função passada, que neste caso dispara a execução do `getText` função (nós "chamamos de volta" para o `getText` função).

Isso é fácil, porque a natureza funcional do JavaScript nos permite lidar com funções como objetos de primeira classe. Podemos até dar um passo adiante, reescrevendo nosso código da seguinte maneira:

```
var text = 'Domo arigato!';

function useless (ninjaCallback) {
  return ninjaCallback ();
}

afirmar (inútil ( função () {texto de retorno;} ) === texto,
  "A função inútil funciona!" + Texto);
```

Define um retorno de chamada
funcionar diretamente
como um argumento

Um dos recursos mais importantes do JavaScript é a capacidade de criar funções no código em qualquer lugar em que uma expressão possa aparecer. Além de tornar o código mais compacto e fácil de entender (colocando as definições de função perto de onde são usadas), esse recurso também pode eliminar a necessidade de poluir o namespace global com nomes desnecessários quando uma função não for referenciada de vários lugares dentro do código.

No exemplo anterior de um retorno de chamada, chamamos nosso próprio retorno de chamada. Mas os retornos de chamada também podem ser chamados pelo navegador. Pense no capítulo 2, que tem um exemplo com o seguinte snippet:

```
document.body.addEventListener("mousemove", function () {  
    var segundo = document.getElementById("segundo"); addMessage(segundo, "Evento:  
    mousemove");  
});
```

Essa também é uma função de retorno de chamada, definida como um manipulador de eventos para o mousemove evento, e que será chamado pelo navegador quando esse evento ocorrer.

NOTA Esta seção apresenta retornos de chamada como funções que outro código “chamará de volta” posteriormente em um ponto apropriado de execução. Você viu um exemplo em que nosso próprio código chama imediatamente o retorno de chamada fornecido (o sem utilidade exemplo de função), bem como um exemplo em que o navegador faz a chamada (o mousemove exemplo) sempre que um determinado evento acontece. É importante observar que, ao contrário de nós, algumas pessoas acreditam que um retorno de chamada deve ser chamado de forma assíncrona e, portanto, o primeiro exemplo não é realmente um retorno de chamada. Mencionamos isso apenas no caso de você tropeçar em alguma discussão acalorada.

Agora vamos considerar o uso de callbacks que simplificarão muito a forma como classificamos as coleções.

CLASSIFICANDO COM UM COMPARADOR

Quase assim que nós *ter* uma coleção de dados, provavelmente precisaremos classificá-la. Digamos que temos uma matriz de números em ordem aleatória: 0, 3, 2, 5, 7, 4, 8, 1. Essa ordem pode ser adequada, mas é provável que, mais cedo ou mais tarde, queiramos para reorganizá-lo.

Normalmente, implementar algoritmos de classificação não é a mais trivial das tarefas de programação; temos que selecionar o melhor algoritmo para o trabalho em questão, implementá-lo, adaptá-lo às nossas necessidades atuais (de modo que os itens sejam classificados em uma ordem específica) e ter cuidado para não introduzir bugs. Destas tarefas, a única que é específica do aplicativo é a ordem de classificação. Felizmente, todas as matrizes de JavaScript têm acesso ao organizar método que exige que definamos apenas um algoritmo de comparação que diga ao algoritmo de classificação como os valores devem ser ordenados.

É aqui que entram os callbacks! Em vez de deixar o algoritmo de classificação decidir quais valores vão antes de outros valores, *Nós vamos* fornecer uma função que executa a comparação. Forneceremos ao algoritmo de classificação acesso a essa função como um retorno de chamada, e o algoritmo chamará o retorno de chamada sempre que precisar fazer uma comparação. O retorno de chamada deve retornar um número positivo se a ordem dos valores passados for

invertido, um número negativo se não, e zero se os valores forem iguais; subtrair os valores comparados produz o valor de retorno desejado para classificar a matriz:

```
valores var = [0, 3, 2, 5, 7, 4, 8, 1];
```

```
values.sort ( função (valor1, valor2) {  
    retorno valor1 - valor2;  
});
```

Não há necessidade de pensar sobre os detalhes de baixo nível de um algoritmo de classificação (ou mesmo que algoritmo de classificação escolher). Fornecemos um retorno de chamada que o mecanismo JavaScript chamará sempre que precisar comparar dois itens.

O *funcional* abordagem nos permite criar uma função como uma entidade autônoma, assim como podemos qualquer outro tipo de objeto, e passá-la como um argumento para um método, assim como qualquer outro tipo de objeto, que pode aceitá-la como um parâmetro, assim como qualquer outro tipo de objeto. É esse status de primeira classe entrando em jogo.

3,2 *Diversão com funções como objetos*

Nesta seção, examinaremos maneiras de explorar as semelhanças que as funções compartilham com outros tipos de objetos. Um recurso que pode ser surpreendente é que nada nos impede de anexar propriedades às funções:

```
var ninja = {};  
ninja.name = "hitsuke";
```

└ Cria um objeto e atribui uma nova
propriedade a ele

```
var wieldSword = function () {}; wieldSword.swordType =  
"katana";
```

└ Cria uma função e atribui uma nova
propriedade a ela

Vejamos algumas das coisas mais interessantes que podem ser feitas com esse recurso:

- *Armazenamento de funções em uma coleção* nos permite gerenciar facilmente funções relacionadas - por exemplo, retornos de chamada que precisam ser invocados quando algo de interesse ocorre.
- *Memoização* permite que a função lembre-se de valores calculados anteriormente, melhorando assim o desempenho de invocações subsequentes.

Vamos começar.

3.2.1 *Armazenando funções*

Em certos casos (por exemplo, quando precisamos gerenciar coleções de callbacks que devem ser invocadas quando um certo evento ocorre), queremos armazenar coleções de funções únicas. Ao adicionar funções a tal coleção, um desafio que podemos enfrentar é determinar quais funções são novas para a coleção e devem ser adicionadas e quais já são residentes e não devem ser adicionadas. Em geral, ao gerenciar coleções de retorno de chamada, não queremos duplicatas, porque um único evento resultaria em várias chamadas para o mesmo retorno de chamada.

Uma técnica óbvia, mas ingênua, é armazenar todas as funções em um array e percorrer o array, verificando se há funções duplicadas. Infelizmente, o desempenho é ruim e, como ninja, queremos fazer as coisas funcionarem *Nós vamos*, não apenas trabalhar. Podemos usar propriedades de função para conseguir isso com um nível apropriado de sofisticação, conforme mostrado na próxima listagem.

Listagem 3.2 Armazenando uma coleção de funções exclusivas

```

var store = {
  nextId: 1,
  cache: {},
  add: function (fn) {
    if (!fn.id) {
      fn.id = this.nextId++;
      this.cache [fn.id] = fn;
      return true;
    }
  }
};

função ninja () {}
assert (store.add (ninja),
        "A função foi adicionada com segurança."); assert (!
store.add (ninja),
        "Mas foi adicionado apenas uma vez.");

```

Mantém o controle do próximo ID disponível a ser atribuído

Cria um objeto para servir como um cache no qual iremos armazenar funções

Adiciona funções a o cache, mas apenas se eles forem únicos

Testa que tudo funciona como planejado

Nesta listagem, criamos um objeto atribuído à variável `armazenar`, no qual armazenaremos um conjunto único de funções. Este objeto possui duas propriedades de dados: uma que armazena um próximo disponível eu ia valor, e dentro do qual iremos esconderijo as funções armazenadas. As funções são adicionadas a este cache por meio do `adicionar()` método:

```

add: function (fn) {
  if (!fn.id) {
    fn.id = this.nextId++;
    this.cache [fn.id] = fn;
    return true;
  }
  ...

```

Dentro de `adicionar`, primeiro verificamos se a função já foi adicionada à coleção, procurando a existência do eu ia propriedade. Se a função atual tiver um eu ia propriedade, assumimos que a função já foi processada e a ignoramos. Caso contrário, atribuímos um eu ia propriedade para a função (incrementando o `nextId` propriedade ao longo do caminho) e adicione a função como uma propriedade do cache, usando o eu ia valor como o nome da propriedade. Em seguida, devolvemos o valor verdade, para que possamos saber quando a função foi adicionada após uma chamada para `adicionar()`.

Executar a página no navegador mostra que quando nossos testes tentam adicionar o `ninja()` função duas vezes, a função é adicionada apenas uma vez, conforme mostrado na figura 3.3. O Capítulo 9 mostra uma técnica ainda melhor para trabalhar com coleções de itens exclusivos que utilizam conjuntos, um novo tipo de objeto disponível no ES6.

Outro truque útil para tirar de nossas mangas ao usar as propriedades da função é dar a uma função a capacidade de se modificar. Esta técnica pode ser usada para lembrar valores calculados anteriormente, economizando tempo durante cálculos futuros.

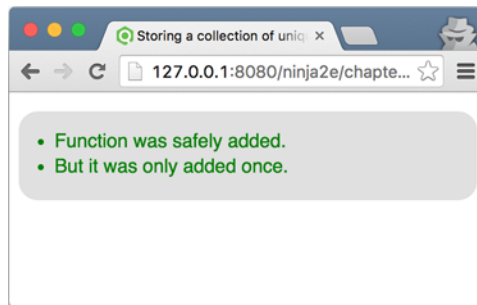


Figura 3.3 Ao adicionar uma propriedade a uma função, podemos controlá-la. Dessa forma, podemos ter certeza de que nossa função foi adicionada apenas uma vez.

3.2.2 *Funções self-memoizing*

Como observado anteriormente, *memoização* (não, isso não é um erro de digitação) é o processo de construção de uma função que é capaz de lembrar seus valores calculados anteriormente. Resumindo, sempre que uma função calcula seu resultado, armazenamos esse resultado junto com os argumentos da função. Dessa forma, quando outra chamada ocorre para o mesmo conjunto de argumentos, podemos retornar o resultado armazenado anteriormente, em vez de calculá-lo novamente. Isso pode aumentar significativamente o desempenho, evitando cálculos complexos desnecessários que já foram executados. A memorização é particularmente útil ao realizar cálculos para animações, pesquisar dados que não mudam com tanta frequência ou qualquer matemática demorada.

Como exemplo, vamos examinar um algoritmo simplista (e certamente não particularmente eficiente) para calcular números primos. Embora este seja um exemplo simples de um cálculo complexo, essa técnica é facilmente aplicável a outros cálculos caros (como derivar o hash MD5 para uma string) que são muito complexos para apresentar aqui.

Do lado de fora, a função parece ser como qualquer função normal, mas vamos construir secretamente em um cache de resposta no qual a função salvará as respostas para os cálculos que realiza. Examine o código a seguir.

Listagem 3.3 Memorizando valores calculados anteriormente

```
function isPrime (value) {
  if (!isPrime.answers) {
    isPrime.answers = {};
  }

  if (isPrime.answers [value] !== undefined) {
    return isPrime.answers [value];
  }

  var prime = valor! == 1; // 1 não é primo
```

Cria o cache

Verifica os valores em cache

```

para (var i = 2; i < valor; i++) {
  if (valor % i === 0) {
    prime = false;
    intervalo;
  }
}
return isPrime.answers [value] = prime;
}

```

← Armazena o valor calculado

```

assert (isPrime (5), "5 é primo!");
assert (isPrime.answers [5], "A resposta foi armazenada em cache!");

```

Testa que isso
tudo funciona

Dentro do `isPrime` função, começamos verificando se o respostas propriedade que usaremos como cache foi criada e, se não, nós a criamos:

```

if (! isPrime.answers) {
  isPrime.answers = {};
}

```

A criação desse objeto inicialmente vazio ocorrerá apenas na primeira chamada para a função; depois disso, o cache existirá.

Em seguida, verificamos se o resultado do valor passado já foi armazenado em cache no respostas:

```

if (isPrime.answers [value] !== undefined) {
  return isPrime.answers [value];
}

```

Dentro desse cache, vamos armazenar a resposta computada (verdade ou falso) usando o argumento valor como a chave de propriedade. Se encontrarmos uma resposta em cache, a devolvemos.

Se nenhum valor em cache for encontrado, prosseguimos e realizamos os cálculos necessários para determinar se o valor é melhor (que pode ser uma operação cara para valores maiores) e armazenar o resultado no cache à medida que o retornamos:

```

return isPrime.answers [value] = prime;

```

Nosso cache é uma propriedade da própria função, portanto, é mantido ativo enquanto a própria função estiver ativa.

Por fim, testamos se a memoização está funcionando!

```

assert (isPrime (5), "5 é primo!");
assert (isPrime.answers [5], "A resposta foi armazenada em cache!");

```

Essa abordagem tem duas vantagens principais:

- O usuário final desfruta de benefícios de desempenho para chamadas de função que solicitam um valor calculado anteriormente.
- Isso acontece perfeitamente e nos bastidores; nem o usuário final nem o autor da página precisam realizar quaisquer solicitações especiais ou fazer qualquer inicialização extra para fazer tudo funcionar.

Mas nem tudo são rosas e violinos; suas desvantagens podem precisar ser pesadas contra suas vantagens:

- Qualquer tipo de cache certamente sacrificará a memória em favor do desempenho. Os puristas podem considerar que o armazenamento em cache é uma preocupação que não deve ser misturada com a lógica de negócios; uma função ou método deve fazer uma coisa e fazê-lo bem. Mas não se preocupe; no capítulo 8, você verá como lidar com essa reclamação.
- É difícil testar a carga ou medir o desempenho de um algoritmo como este, porque nossos resultados dependem das entradas anteriores para a função.

Agora que você viu alguns dos casos de uso prático de funções de primeira classe, vamos explorar as várias maneiras de definir funções.

3.3 Definindo funções

As funções JavaScript são geralmente definidas usando um *literal de função* que cria um valor de função da mesma maneira que, por exemplo, um literal numérico cria um valor numérico. Lembre-se de que, como objetos de primeira classe, as funções são valores que podem ser usados na linguagem como outros valores, como strings e números. E quer você perceba ou não, você tem feito isso o tempo todo.

JavaScript fornece algumas maneiras de definir funções, que podem ser divididas em quatro grupos:

- *Declarações de função e expressões de função* —As duas formas mais comuns, embora sutilmente diferentes, de definir funções. Muitas vezes, as pessoas nem mesmo os consideram separados, mas como você verá, estar ciente de suas diferenças pode nos ajudar a entender quando nossas funções estão disponíveis para invocação:

```
function myFun () {return 1;}
```

- *Funções de seta* (frequentemente chamado *funções lambda*) —Uma adição recente do ES6 ao padrão JavaScript que nos permite definir funções com muito menos confusão sintática. Eles até resolvem um problema comum com funções de retorno de chamada, mas mais sobre isso mais tarde:

```
myArg => myArg * 2
```

- *Construtores de função* —Uma maneira não tão usada de definir funções que nos permite construir dinamicamente uma nova função a partir de uma string que também pode ser gerada dinamicamente. Este exemplo cria dinamicamente uma função com dois parâmetros, *a* e *b*, que retorna a soma desses dois parâmetros:

```
nova função ('a', 'b', 'retornar a + b')
```

- *Funções geradoras* —Esta adição ES6 ao JavaScript nos permite criar funções que, ao contrário das funções normais, podem ser encerradas e reinseridas posteriormente na execução do aplicativo, enquanto mantemos os valores de suas variáveis entre essas reentradas. Podemos definir versões geradoras de *declarações de função*, *expressões de função*, e *construtores de função*:

```
função * myGen () {rendimento 1; }
```

É importante que você entenda essas diferenças, porque a maneira como uma função é definida influencia significativamente quando a função está disponível para ser chamada e como ela se comporta, bem como em qual objeto a função pode ser chamada.

Neste capítulo, exploraremos as declarações de funções, expressões de funções e funções de seta. Você aprenderá sua sintaxe e como funcionam, e voltaremos a ela várias vezes ao longo do livro para explorar suas especificidades. As funções do gerador, por outro lado, são bastante peculiares e são significativamente diferentes das funções padrão. Iremos revisá-los em detalhes no capítulo 6.

Isso nos deixa com construtores de função, um recurso JavaScript que ignoraremos completamente. Embora tenha alguns aplicativos interessantes, especialmente ao criar e avaliar código dinamicamente, nós o consideramos um recurso fundamental da linguagem JavaScript. Se você quiser saber mais sobre construtores de função, visite <http://mng.bz/ZN8e>.

Vamos começar com as maneiras mais simples e tradicionais de definir funções: *declarações de função* e *expressões de função*.

3.3.1 Declarações de funções e expressões de funções

As duas maneiras mais comuns de definir funções em JavaScript são usando declarações e expressões de função. Essas duas técnicas são tão semelhantes que muitas vezes nem fazemos distinção entre elas, mas, como você verá nos próximos capítulos, existem diferenças sutis.

DECLARAÇÕES DE FUNÇÃO

A maneira mais básica de definir uma função em JavaScript é usando declarações de função (consulte a figura 3.4). Como você pode ver, cada declaração de função começa com um obrigatório `função` palavra-chave, seguida por um nome de função obrigatório e uma lista de nomes de parâmetros separados por vírgulas opcionais entre parênteses obrigatórios. O corpo da função, que é uma lista potencialmente vazia de instruções, deve ser colocado entre uma chave de abertura e uma chave de fechamento. Além deste formulário, que cada declaração de função deve satisfazer, há mais uma condição: Uma declaração de função deve ser colocada por conta própria, como uma instrução JavaScript separada (mas pode ser

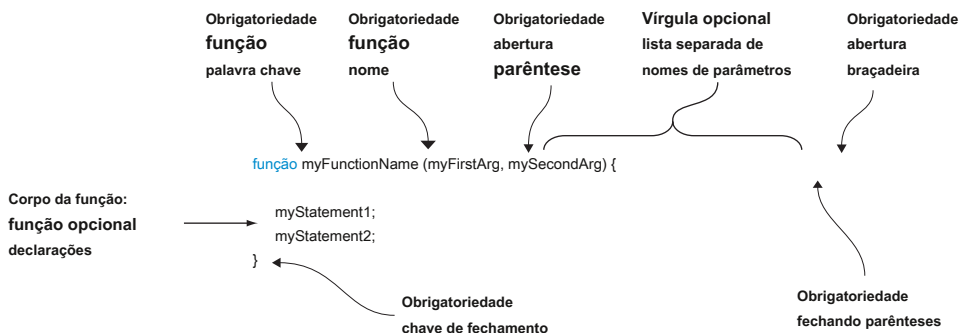


Figura 3.4 A declaração da função é independente, como um bloco separado de código JavaScript! (Pode estar contido em outras funções.)

contido em outra função ou bloco de código; você verá exatamente o que queremos dizer com isso na próxima seção).

Alguns exemplos de declaração de função são mostrados na lista a seguir.

Listagem 3.4 Exemplos de declarações de funções

<pre>function samurai () { retornar "samurai aqui"; } function ninja () { function hiddenNinja () { retornar "ninja aqui"; } return hiddenNinja (); }</pre>	Define a função samurai no código global Define a função ninja no código global Define a função hiddenNinja dentro a função ninja
--	--

Se você olhar mais de perto, verá algo com o qual talvez não esteja acostumado, se não teve muita experiência com linguagens funcionais: uma função definida dentro de outra função!

```
function ninja () {
    function hiddenNinja () {
        retornar "ninja aqui";
    }
    return hiddenNinja ();
}
```

Em JavaScript, isso é perfeitamente normal, e o usamos aqui para enfatizar novamente a importância das funções em JavaScript.

NOTA Ter funções contidas em outras funções pode levantar algumas questões complicadas em relação ao escopo e à resolução do identificador, mas guarde-as por enquanto, porque revisaremos esse caso em detalhes no capítulo 5.

EXPRESSÕES DE FUNÇÃO

Como já mencionamos várias vezes, as funções em JavaScript são objetos de primeira classe, o que, entre outras coisas, significa que podem ser criados por meio de literais, atribuídos a variáveis e propriedades e usados como argumentos e valores de retorno de e para outras funções. Como as funções são construções fundamentais, o JavaScript nos permite tratá-las como quaisquer outras expressões. Então, assim como podemos usar literais de número, por exemplo

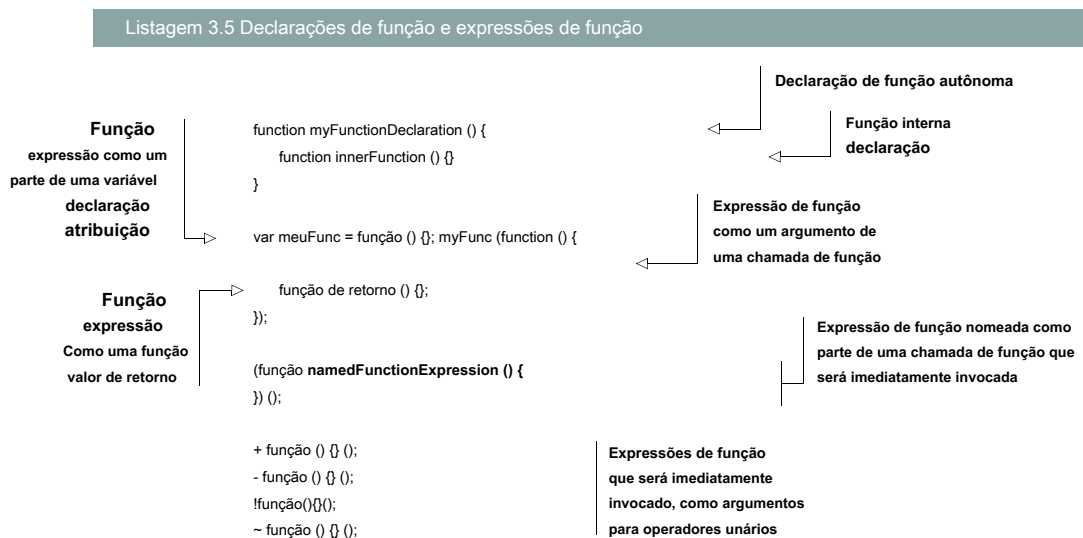
```
var a = 3;
minhaFunção (4);
```

assim também podemos usar literais de função, nos mesmos locais

```
var a = função () {}; minhaFunção (função () {});
```

Essas funções que sempre fazem parte de outra instrução (por exemplo, como o lado direito de uma expressão de atribuição ou como um argumento para outra função) são chamadas *expressões de função*. As expressões de função são ótimas porque nos permitem definir funções exatamente onde precisamos, no processo tornando nosso código mais fácil de entender.

A lista a seguir mostra as diferenças entre as declarações e expressões de função.



Este código de exemplo começa com uma declaração de função padrão que contém outra declaração de função interna:

```
function myFunctionDeclaration () {  
    function innerFunction () {}  
}
```

Aqui você pode ver como as declarações de função são instruções separadas do código JavaScript, mas podem estar contidas no corpo de outras funções.

Em contraste, há expressões de função, que sempre fazem parte de outra instrução. Eles são colocados no nível da expressão, como o lado direito de uma declaração de variável (ou uma atribuição):

```
var meuFunc = função () {};
```

Ou como um argumento para outra chamada de função ou como um valor de retorno de função:

```
myFunc (function () {  
    função de retorno () {};  
});
```

Além da posição no código onde são colocados, há mais uma diferença entre as declarações de função e expressões de função: Para declarações de função, o nome da função é *obrigatoriedade*, enquanto para expressões de função é completamente *opcional*.

As declarações de funções devem ter um nome definido porque são independentes. Como um dos requisitos básicos para uma função é que ela deve ser invocável, devemos ter uma maneira de referenciá-la, e a única maneira de fazer isso é por meio de seu nome.

As expressões de função, por outro lado, são partes de outras expressões JavaScript, portanto, temos maneiras alternativas de invocá-las. Por exemplo, se uma expressão de função for atribuída a uma variável, podemos usar essa variável para invocar a função:

```
var doNothing = function () {}; fazer nada();
```

Ou, se for um argumento para outra função, podemos invocá-lo dentro dessa função por meio do nome do parâmetro correspondente:

```
function doSomething (action) {  
    ação();  
}
```

FUNÇÕES IMEDIATAS

As expressões de função podem até mesmo ser colocadas em posições onde parecem um pouco estranhas no início, como em um local onde normalmente esperaríamos um identificador de função. Vamos parar e examinar mais de perto essa construção (veja a figura 3.5).

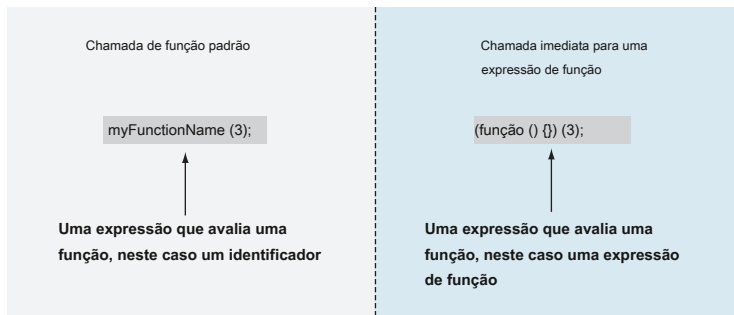


Figura 3.5 Uma comparação de uma chamada de função padrão e uma chamada imediata para uma expressão de função

Quando queremos fazer uma chamada de função, usamos uma expressão que avalia uma função, seguida por um par de parênteses de chamada de função, que pode conter argumentos. Na chamada de função mais básica, colocamos um identificador que avalia uma função, como no lado esquerdo da figura 3.5. Mas a expressão à esquerda do parêntese de chamada não precisa ser um identificador simples; pode ser *nenhum* expressão que avalia uma função. Por exemplo, uma maneira simples de especificar uma expressão avaliada como uma função é usar uma expressão de função. Portanto, no lado direito da figura 3.5, primeiro criamos uma função e, em seguida, invocamos imediatamente essa função recém-criada. A propósito, isso é chamado de *expressão de função imediatamente invocada* (IIFE), ou *função imediata* para resumir, e é um conceito importante no desenvolvimento de JavaScript porque nos permite imitar módulos em JavaScript. Vamos nos concentrar nesta aplicação de IIFEs no capítulo 11.

Parênteses em torno de expressões de função

Mais uma coisa pode estar incomodando você sobre a maneira como chamamos imediatamente nossa expressão de função: os parênteses ao redor da própria expressão de função. Por que ainda precisamos disso? O motivo é puramente sintático. O analisador JavaScript deve ser capaz de diferenciar facilmente entre declarações de função e expressões de função. Se deixarmos os parênteses em torno da expressão da função e colocarmos nossa chamada imediata como uma instrução separada função () {} (3), o analisador JavaScript começará a processá-lo e concluirá, porque é uma instrução separada que começa com a palavra-chave função, que está lidando com uma declaração de função. Como cada declaração de função deve ter um nome (e aqui não especificamos um), um erro será gerado. Para evitar isso, colocamos a expressão da função entre parênteses, sinalizando ao analisador JavaScript que está lidando com uma expressão, não com uma instrução.

Há também uma forma alternativa, ainda mais simples (embora, estranhamente, usada com menos frequência) de atingir o mesmo objetivo: (função () {} (3)). Envolvendo a definição de função imediata e chamada entre parênteses, você também pode notificar o analisador JavaScript que está lidando com uma expressão.

As últimas quatro expressões na listagem 3.5 são variações do mesmo tema de expressões de função imediatamente invocadas, frequentemente encontradas em várias bibliotecas JavaScript:

```
+ função () {} ();  
- função () {} ();  
!função(){}();  
~ função () {} ();
```

Desta vez, em vez de usar parênteses em torno das expressões de função para diferenciá-las das declarações de função, podemos usar operadores unários: +, -, ! E ~. Fazemos isso para sinalizar ao mecanismo JavaScript que está lidando com expressões e não com instruções. Observe como os resultados da aplicação desses operadores unários não são armazenados em lugar nenhum; de uma perspectiva computacional, eles realmente não importam; apenas as chamadas para nossos IIFEs importam.

Agora que estudamos os prós e contras das duas maneiras mais básicas de definir funções em JavaScript (declarações de função e expressões de função), vamos explorar uma nova adição ao padrão JavaScript: *funções de seta*.

3.3.2 Funções de seta



NOTA As funções de seta são uma adição ES6 ao padrão JavaScript (para compatibilidade de navegador, consulte <http://mng.bz/8bnH>)

Porque em nosso JavaScript usamos *muito* de funções, faz sentido adicionar algum açúcar sintático que nos permite criar funções de uma forma mais curta e sucinta, tornando assim nossas vidas como desenvolvedores mais agradáveis.

De várias maneiras, as funções de seta são uma simplificação das expressões de função. Vamos revisar nosso exemplo de classificação da primeira seção deste capítulo:

```
valores var = [0, 3, 2, 5, 7, 4, 8, 1]; valores.sort (função (valor1, valor2) {  
  
    retorno valor1 - valor2; });
```

Este exemplo usa uma expressão de função de retorno de chamada enviada ao método de classificação do objeto de matriz; esse retorno de chamada será invocado pelo mecanismo JavaScript para classificar os valores da matriz em ordem decrescente.

Agora vamos ver como fazer exatamente a mesma coisa com as funções de seta:

```
valores var = [0, 3, 2, 5, 7, 4, 8, 1]; values.sort (( valor1, valor2) => valor1 - valor2);
```

Veja como isso é muito mais sucinto?

Não há desordem causada pelo função palavra-chave, as chaves ou o Retorna demonstração. De uma maneira muito mais simples do que uma expressão de função pode, a função de seta afirma: aqui está uma função que recebe dois argumentos e retorna sua diferença. Observe a introdução de um novo operador, `=>`, o assim chamado *flecha gorda* operador (um sinal de igual imediatamente seguido por um sinal de maior que), que está no centro da definição de uma função de seta.

Agora vamos desconstruir a sintaxe de uma função de seta, começando da maneira mais simples possível:

```
param => expressão
```

Esta função de seta recebe um parâmetro e retorna o valor de uma expressão. Podemos usar essa sintaxe conforme mostrado na lista a seguir.

Listagem 3.6 Comparando uma função de seta e uma expressão de função

```
var greet = name => "Saudações" + nome;
assert (greet ("Oishi") === "Greetings Oishi", "Oishi é devidamente saudado");
```

Define uma função de seta

```
var anotherGreet = function (name) {
    retornar "Saudações" + nome;
};
assert (anotherGreet ("Oishi") === "Saudações Oishi",
    "Novamente, Oishi é devidamente saudado");
```

Define uma função expressão

Pare um pouco para apreciar como as funções de seta tornam o código mais sucinto, sem sacrificar a clareza. Essa é a versão mais simples da sintaxe da função seta, mas, em geral, a função seta pode ser definida de duas maneiras, conforme mostrado na figura 3.6.

Como você pode ver, a definição da função de seta começa com uma lista opcional separada por vírgulas de nomes de parâmetros. Se não houver parâmetros, ou mais de um parâmetro, esta lista deve ser colocada entre parênteses. Mas se tivermos apenas um único parâmetro, os parênteses são opcionais. Essa lista de parâmetros é seguida por um operador de seta grande obrigatório, que nos informa e ao mecanismo JavaScript que estamos lidando com uma função de seta.

Depois do operador fat-arrow, temos duas opções. Se for uma função simples, colocamos uma expressão lá (uma operação matemática, outra chamada de função, qualquer coisa), e o resultado da chamada de função será o valor dessa expressão. Por exemplo, nosso primeiro exemplo de função de seta tem a seguinte função de seta:

```
var greet = name => "Saudações" + nome;
```

Parênteses obrigatórios para 0 ou mais de 1 parâmetro; opcional para 1 parâmetro.

Obrigatoriedade flecha gorda operador (igual a e maior que).

Se o corpo da função de seta for uma expressão, o valor de retorno da função será o valor dessa expressão.

(param1, param2) => expressão

```
{
  myStatement1;
  myStatement2;
}
```

Separado por vírgula opcional lista de nomes de parâmetros.

Se o corpo da função de seta for um bloco de código, o retorno o valor é como seria em uma função padrão (indefinido se não houver instrução de retorno e o valor da expressão de retorno se houver).

Figura 3.6 A sintaxe de uma função de seta

O valor de retorno da função é uma concatenação da string " Saudações " com o valor do nome parâmetro.

Em outros casos, quando nossas funções de seta não são tão simples e exigem mais código, podemos incluir um bloco de código após o operador de seta. Por exemplo:

```
var greet = name => {
  var helloString = 'Saudações'; return helloString + name;
};
```

Nesse caso, o valor de retorno da função de seta se comporta como em uma função padrão. Se não houver instrução de retorno, o resultado da invocação da função será Indefinido, e se houver, o resultado será o valor da expressão de retorno.

Iremos revisitar as funções de seta várias vezes ao longo deste livro. Entre outras coisas, apresentaremos recursos adicionais de funções de seta que nos ajudarão a evitar erros sutis que podem ocorrer com funções mais padrão.

As funções de seta, como todas as outras funções, podem receber argumentos para usá-los para realizar sua tarefa. Vamos ver o que acontece com os valores que passamos para uma função.

3.4 Argumentos e parâmetros de função

Ao discutir funções, costumamos usar os termos *argumento* e *parâmetro* quase indistintamente, como se fossem mais ou menos a mesma coisa. Mas agora, vamos ser mais formais:

- Uma *parâmetro* é uma variável que listamos como parte da definição de uma função.
- A *argumento* é um valor que passamos para a função quando a chamamos.

A Figura 3.7 ilustra a diferença.

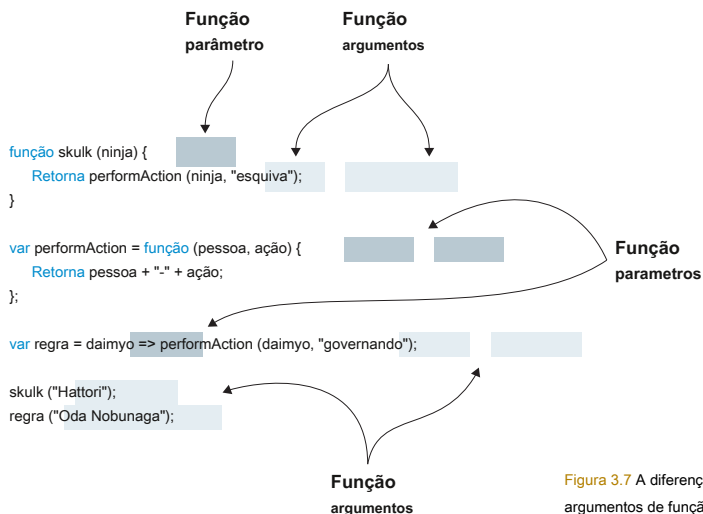


Figura 3.7 A diferença entre parâmetros de função e argumentos de função

Como você pode ver, um parâmetro de função é especificado com a definição da função, e todos os tipos de funções podem ter parâmetros:

- Declarações de funções (o ninja parâmetro para o esconder-se função) Expressões de função (o pessoa e ação parâmetros para o executar a ação função)
- Funções de seta (o daimyo parâmetro)

Os argumentos, por outro lado, estão vinculados à invocação da função; são valores passados para uma função no momento de sua invocação:

- A corda Hattori é passado como um argumento para o esconder-se função.
- A corda Oda Nobunaga é passado como um argumento para o regra função.
- O parâmetro ninja do esconder-se função é passada como um argumento para o executar a ação função.

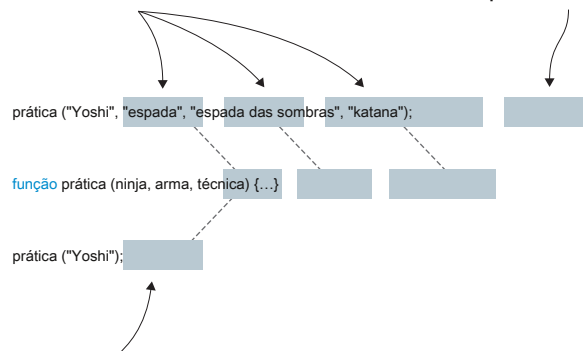
Quando uma lista de argumentos é fornecida como parte de uma chamada de função, esses argumentos são atribuídos aos parâmetros na definição da função na ordem especificada. O primeiro argumento é atribuído ao primeiro parâmetro, o segundo argumento ao segundo parâmetro e assim por diante.

Se tivermos um número diferente de argumentos do que parâmetros, nenhum erro será gerado. JavaScript está perfeitamente bem com essa situação e lida com ela da seguinte maneira. Se forem fornecidos mais argumentos do que parâmetros, os argumentos "em excesso" não serão atribuídos aos nomes dos parâmetros. Por exemplo, veja a figura 3.8.

A Figura 3.8 mostra que se chamássemos o prática funcionar com prática ("Yoshi", "espada", "espada sombra", "katana"), os argumentos Yoshi, espada, e espada sombra seria atribuído aos parâmetros ninja, arma, e técnica,

O argumento "Yoshi" é atribuído ao parâmetro ninja. O argumento "espada" é atribuído ao parâmetro arma. O argumento "espada da sombra" é atribuído ao parâmetro técnica.

Excesso de argumentos não são atribuídos a parâmetros.



O argumento "Yoshi" é atribuído ao parâmetro ninja. undefined é atribuído ao parâmetro arma. undefined é atribuído ao parâmetro técnica.

Figura 3.8 Os argumentos são atribuídos aos parâmetros da função na ordem especificada. Excesso de argumentos não são atribuídos a nenhum parâmetro.

respectivamente. O argumento katana é um argumento em excesso e não seria atribuído a nenhum parâmetro. No próximo capítulo, você verá que embora alguns argumentos não sejam atribuídos a nomes de parâmetros, ainda temos uma maneira de acessá-los.

Por outro lado, se tivermos mais parâmetros do que argumentos, os parâmetros que não têm argumento correspondente são definidos como Indefinido. Por exemplo, se fizéssemos a chamada prática ("Yoshi"), o parâmetro ninja seria atribuído o valor Yoshi, enquanto os parâmetros arma e técnica seria definido para Indefinido.

Lidar com argumentos e parâmetros de função é tão antigo quanto o próprio JavaScript, mas agora vamos explorar dois novos recursos de JavaScript oferecidos pelo ES6: resto e parâmetros padrão.

3.4.1 Parâmetros de descanso



NOTA Os parâmetros de descanso são adicionados pelo padrão ES6 (para compatibilidade do navegador, consulte <http://mng.bz/3go1>)

Para nosso próximo exemplo, vamos construir uma função que multiplica o primeiro argumento pelo maior dos argumentos restantes. Isso provavelmente não é algo particularmente aplicável em nossos aplicativos, mas é um exemplo de ainda mais técnicas para lidar com argumentos dentro de uma função.

Isso pode parecer bastante simples: pegaremos o primeiro argumento e o multiplicaremos pelo maior dos valores de argumento restantes. Nas versões antigas do JavaScript, isso exigiria algumas soluções alternativas (que veremos no próximo capítulo). Felizmente, no ES6, não precisamos passar por nenhum obstáculo. Podemos usar *parâmetros de descanso*, conforme mostrado na lista a seguir.

Listagem 3.7 Usando parâmetros de descanso

```
function multiMax (primeiro, ... restantesNúmeros) {
  var classificado = restantesNúmeros.sort (função (a, b) {
    retorno b - a;
  });
  retorna primeiro * classificado [0];
}
afirmar (multiMax (3, 1, 2, 3) == 9,
  "3 * 3 = 9 (Primeiro argumento, pelo maior.);");
```

← Parâmetros de descanso são prefixado com ...

← Classifique o restante números, decrescentes.

← A função é chamada como qualquer outra função.

Ao prefixar o último argumento nomeado de uma função com reticências (...), nós o transformamos em uma matriz chamada *os demais parâmetros*, que contém os argumentos passados restantes.

```
function multiMax (primeiro, ... restantesNúmeros) {
  ...
}
```

Por exemplo, neste caso, o multiMax função é chamada com quatro argumentos: multiMax (3, 1, 2, 3). No corpo do multiMax função, o valor do primeiro argumento, 3, é atribuído ao primeiro multiMax parâmetro de função, primeiro. Como o segundo parâmetro da função é o parâmetro restante, todos os argumentos restantes (1, 2, 3) são colocados em uma nova matriz: restantesNúmeros. Em seguida, obtemos o maior número classificando a matriz em ordem decrescente (observe como é simples mudar a ordem de classificação) e escolhendo o maior número, que está no primeiro lugar de nossa matriz classificada. (Essa está longe de ser a maneira mais eficiente de determinar o maior número, mas por que não aproveitar as habilidades que adquirimos no início do capítulo?)

NOTA Apenas o último parâmetro de função pode ser um parâmetro de descanso. Tentar colocar reticências na frente de qualquer parâmetro que não seja o último nos trará apenas sor-linha, na forma de SyntaxError: parâmetro após o parâmetro rest.

Na próxima seção, continuaremos adicionando ao nosso cinto de ferramentas JavaScript com funcionalidade ES6 adicional: parâmetros padrão.

3.4.2 Parâmetros padrão



NOTA Os parâmetros padrão são adicionados pelo padrão ES6 (para compatibilidade do navegador, consulte <http://mng.bz/wl8w>)

Muitos componentes de IU da web (especialmente plug-ins jQuery) podem ser configurados. Por exemplo, se estivermos desenvolvendo um componente de controle deslizante, podemos dar aos nossos usuários a opção de especificar um intervalo de tempo após o qual um item é substituído por outro, bem como uma animação que será usada conforme a mudança ocorre. Ao mesmo tempo, talvez alguns usuários não se importem e fiquem felizes em usar quaisquer configurações que oferecemos. Os parâmetros padrão são ideais para tais situações!

Nosso pequeno exemplo com as configurações do componente deslizante é apenas um caso específico de uma situação em que *quase* todas as chamadas de função usam o mesmo valor para um parâmetro específico (observe a ênfase em *quase*). Considere um caso mais simples em que a maioria de nossos ninjas está acostumada a se esgueirar, mas não Yagyu, que se preocupa apenas com uma simples espreita:

```
function performAction (ninja, ação) {
  retornar ninja + "" + ação;
}
performAction ("Fuma", "esquiva");
performAction ("Yoshi", "esquiva");
performAction ("Hattori", "esquiva");
performAction ("Yagyu", "sorrateiro");
```

Não parece tedioso ter que repetir sempre o mesmo argumento, esquivando-se, simplesmente porque Yagyu é obstinado e se recusa a agir como um ninja adequado?

Em outras linguagens de programação, esse problema é mais frequentemente resolvido com sobrecarga de função (especificando funções adicionais com o mesmo nome, mas um conjunto diferente de parâmetros). Infelizmente, o JavaScript não oferece suporte à sobrecarga de funções, portanto, quando confrontados com essa situação no passado, os desenvolvedores frequentemente recorriam a algo como a listagem a seguir.

Listagem 3.8 Lidando com parâmetros padrão antes de ES6

```
function performAction (ninja, ação) {
  action = typeof action === "undefined"? "esquiva": ação; retornar ninja + "" + ação;
}

assert (performAction ("Fuma") === "Fuma se escondendo",
  "O valor padrão é usado para Fuma");

assert (performAction ("Yoshi") === "Yoshi se escondendo",
  "O valor padrão é usado para Yoshi");

assert (performAction ("Hattori") === "Hattori se escondendo",
  "O valor padrão é usado para Hattori");

assert (performAction ("Yagyu", "sneaking") === "Yagyu sneaking",
  "Yagyu pode fazer o que quiser, até esgueirar-se!");
```

Se o parâmetro de ação for indefinido, usamos um valor padrão, skulking, e se estiver definido, mantemos o valor passado.

Não passamos em um segundo argumento, o valor da ação parâmetro; depois de executar a primeira função, a instrução do corpo assumirá como padrão skulking.

Passe uma string como o valor do parâmetro de ação; esse valor será usado em todo o corpo da função.

Aqui nós definimos um executar a ação função, que verifica se o valor do ação parâmetro é Indefinido (usando o tipo de operador), e se for, a função define o valor do ação variável para escondida. Se o ação parâmetro é enviado através de uma chamada de função (não é Indefinido), nós mantemos o valor.

NOTA O tipo de operador retorna uma string que indica o tipo do operando. Se o operando não estiver definido (por exemplo, se não tivermos fornecido um argumento correspondente para um parâmetro de função), o valor de retorno é a string Indefinido.

Este é um padrão que ocorre comumente e é tedioso de escrever, então o padrão ES6 adicionou suporte para *parâmetros padrão*, conforme mostrado na lista a seguir.

Listagem 3.9 Lidando com parâmetros padrão no ES6

```
function performAction (ninja, action = "skulking") {  
  retornar ninja + "" + ação;  
}
```

```
assert (performAction ("Fuma") === "Fuma se escondendo",  
       "O valor padrão é usado para Fuma");
```

```
assert (performAction ("Yoshi") === "Yoshi se escondendo",  
       "O valor padrão é usado para Yoshi");
```

```
assert (performAction ("Hattori") === "Hattori se escondendo",  
       "O valor padrão é usado para Hattori");
```

```
assert (performAction ("Yagyu", "sneaking") === "Yagyu sneaking",  
       "Yagyu pode fazer o que quiser, até esgueirar-se!");
```

← No ES6, é possível atribuir um valor a um parâmetro de função.

Se o valor não for passado, o valor padrão é usado.

O valor passado é usado.

Aqui você pode ver a sintaxe dos parâmetros de função padrão em JavaScript. Para criar um parâmetro padrão, atribuímos um valor a um parâmetro de função:

```
função performAction (ninja, action = "skulking") {  
  retornar ninja + "" + ação;  
}
```

Então, quando fazemos uma chamada de função e o valor do argumento correspondente é deixado de fora, como acontece com Fuma, Yoshi, e Hattori, o valor padrão (neste caso, esquivando), é usado:

```
assert (performAction ("Fuma") === "Fuma se escondendo",  
       "O valor padrão é usado para Fuma");
```

```
assert (performAction ("Yoshi") === "Yoshi se escondendo",  
       "O valor padrão é usado para Yoshi");
```

```
assert (performAction ("Hattori") === "Hattori se escondendo",  
       "O valor padrão é usado para Hattori");
```

Se, por outro lado, especificarmos o valor, o valor padrão será substituído:

```
assert (performAction ("Yagyu", "sneaking") === "Yagyu sneaking",  
       "Yagyu pode fazer o que quiser, até esgueirar-se!");
```

Podemos atribuir qualquer valor aos parâmetros padrão: valores simples e primitivos, como números ou strings, mas também tipos complexos, como objetos, matrizes e até funções. Os valores são avaliados em cada chamada de função, da esquerda para a direita e, ao atribuir valores a parâmetros padrão posteriores, podemos fazer referência a parâmetros anteriores, como na listagem a seguir.

Listagem 3.10 Referenciando parâmetros padrão anteriores

```

função performAction (ninja, ação = "esquiva",
                    mensagem = ninja + "" + ação) {
    mensagem de retorno;
}

assert (performAction (" Yoshi ") === " Yoshi está se escondendo ", " Yoshi está se escondendo ");

```

Podemos colocar expressões arbitrárias como padrão valores de parâmetro, em o processo até referenciando anterior parâmetros de função.

Embora o JavaScript permita que você faça algo assim, recomendamos cautela. Em nossa opinião, isso não melhora a legibilidade do código e deve ser evitado, sempre que possível. Mas o uso moderado de parâmetros padrão - como meio de evitar valores nulos ou como sinalizadores relativamente simples que configuram os comportamentos de nossas funções - pode levar a um código muito mais simples e elegante.

3,5 *Resumo*

- Escrever código sofisticado depende do aprendizado de JavaScript como uma linguagem funcional.
- Funções são objetos de primeira classe tratados como quaisquer outros objetos em JavaScript. Semelhante a qualquer outro tipo de objeto, eles podem ser
 - Criado por meio de literais
 - Atribuído a variáveis ou propriedades
 - Passado como parâmetros
 - Retornado como resultados de função
 - Propriedades e métodos atribuídos
- Funções de retorno de chamada são funções que outro código irá "chamar de volta" posteriormente e são frequentemente usadas, especialmente com tratamento de eventos.
- Podemos tirar vantagem do fato de que as funções podem ter propriedades e que essas propriedades podem ser usadas para armazenar qualquer informação; por exemplo
 - Podemos armazenar funções em propriedades de função para referência posterior e invocação.
 - Podemos usar as propriedades da função para criar um cache (memoização), evitando cálculos desnecessários.
- Existem diferentes tipos de funções: declarações de função, expressões de função, funções de seta e geradores de função.
- Declarações e expressões de funções são os dois tipos mais comuns de funções. As declarações de função devem ter um nome e devem ser colocadas como instruções separadas em nosso código. As expressões de função não precisam ser nomeadas, mas precisam ser parte de outra instrução de código.
- As funções de seta são uma nova adição ao JavaScript, permitindo-nos definir funções de uma forma muito mais sucinta do que com funções padrão.
- Um parâmetro é uma variável que listamos como parte da definição de uma função, enquanto um argumento é um valor que passamos para a função quando a invocamos.

- A lista de parâmetros de uma função e sua lista de argumentos podem ter comprimentos diferentes:
 - Parâmetros não atribuídos avaliados como `Indefinido`.
 - Argumentos extras não estão vinculados a nomes de parâmetros.
- Parâmetros de descanso e parâmetros padrão são novas adições ao JavaScript:
 - Os parâmetros Rest nos permitem fazer referência aos argumentos restantes que não têm nomes de parâmetros correspondentes.
 - Os parâmetros padrão nos permitem especificar os valores dos parâmetros padrão que serão usados se nenhum valor for fornecido durante a chamada da função.

3,6 Exercícios

- 1 No trecho de código a seguir, quais funções são funções de retorno de chamada?

```
numbers.sort (função sortAsc ( a, b) {
    retornar a - b;
});

função ninja () {}
ninja ();

var myButton = document.getElementById ("myButton"); myButton.addEventListener ("click", função handleClick
() {
    alerta ("clicado");
});
```

- 2 No snippet a seguir, categorize as funções de acordo com seu tipo (função declaração, expressão de função ou função de seta).

```
numbers.sort (função sortAsc ( a, b) {
    retornar a - b;
});

numbers.sort (( a, b) => b - a);

( função(){}());

function outer () {
    função interna () {}
    retorno interno;
}

( função(){}());

()=> " Yoshi " ();
```

- 3 Depois de executar o seguinte trecho de código, quais são os valores das variáveis `samurai` e `ninja`?

```
var samurai = (() => "Tomoe") (); var ninja = (() => {"Yoshi"})
();
```


- 4 Dentro do corpo do teste função, quais são os valores dos parâmetros a, b, e c para as duas chamadas de função?

```
teste de função (a, b, ... c) { / * a, b, c * / }
```

```
teste (1, 2, 3, 4, 5); teste();
```

- 5 Depois de executar o seguinte snippet de código, quais são os valores do mensagem1 e mensagem2 variáveis?

```
function getNinjaWieldingWeapon (ninja, weapon = "katana") {  
    retornar ninja + " " + katana;  
}
```

```
var message1 = getNinjaWieldingWeapon ("Yoshi");  
var message2 = getNinjaWieldingWeapon ("Yoshi", "wakizashi");
```

4

Funções para o jornaleiro: *função de compreensão* *invocação*

Este capítulo cobre

- Dois parâmetros de função implícitos: argumentos e este
- Maneiras de invocar funções
- Lidando com problemas de contextos funcionais

No capítulo anterior, você viu que JavaScript é uma linguagem de programação com características orientadas funcionalmente significativas. Exploramos as diferenças entre argumentos de chamada de função e parâmetros de função e como os valores são transferidos de argumentos de chamada para parâmetros de função.

Este capítulo continua em uma linha semelhante, discutindo primeiro algo que escondemos de você no capítulo anterior: os parâmetros de função implícitos `esta` e `argumentos`. Eles são silenciosamente passados para as funções e podem ser acessados como qualquer outro parâmetro de função explicitamente nomeado dentro do corpo da função.

O `esta` parâmetro representa o contexto da função, o objeto no qual nossa função é chamada, enquanto o `argumentos` parâmetro representa todos os argumentos que são transmitidos por meio de uma chamada de função. Ambos os parâmetros são vitais no código JavaScript. O `esta` parâmetro é um dos ingredientes fundamentais do JavaScript orientado a objetos, e o `argumentos` parâmetro nos permite ser criativos com os argumentos que são aceitos por nossas funções. Por esse motivo, exploraremos algumas das armadilhas comuns relacionadas a esses argumentos implícitos.

Em seguida, continuaremos explorando maneiras de invocar funções em JavaScript. A maneira como invocamos uma função tem uma grande influência em como os parâmetros implícitos da função são determinados.

Finalmente, concluiremos o capítulo aprendendo sobre pegadinhas comuns relacionadas ao contexto da função, o `esta` parâmetro. Sem mais delongas, vamos começar a explorar!

.....

Você sabe? Porque é o `esta` parâmetro conhecido como a função *contexto*?

Qual é a diferença entre uma função e um método? O que aconteceria se um construtor funcionasse explicitamente retornou um objeto?

.....

4,1 *Usando parâmetros de função implícitos*

No capítulo anterior, exploramos as diferenças entre funções *parametros* (variáveis listadas como parte de uma definição de função) e função *argumentos* (valores passados para a função quando a invocamos). Mas não mencionamos que, além dos parâmetros que declaramos explicitamente na definição da função, as invocações da função geralmente recebem dois parâmetros implícitos: `argumentos` e `esta`.

De *implícito*, queremos dizer que esses parâmetros não são listados explicitamente na assinatura da função, mas são passados silenciosamente para a função e acessíveis dentro da função. Eles podem ser referenciados dentro da função como qualquer outro parâmetro explicitamente nomeado. Vamos dar uma olhada em cada um desses parâmetros implícitos.

4.1.1 *O parâmetro de argumentos*

O `argumentos` parâmetro é uma coleção de todos os argumentos passados para uma função. É útil porque nos permite acessar todos os argumentos da função, independentemente de o parâmetro correspondente estar definido explicitamente. Isso nos permite implementar a sobrecarga de função, um recurso que o JavaScript não suporta nativamente, e funções variáveis que aceitam um número variável de argumentos. Para ser honesto, com resto parâmetros, introduzidos no capítulo anterior, a necessidade do `argumentos` parâmetro foi bastante reduzido. Ainda assim, é importante entender como o `argumentos` parâmetro funciona, porque você está fadado a topá-lo ao lidar com código legado.

O argumento `objeto` tem uma propriedade chamada `comprimento` que indica o número exato de argumentos. Os valores de argumentos individuais podem ser obtidos usando a notação de indexação de matriz; por exemplo, `argumentos[2]` buscaria o terceiro parâmetro. Dê uma olhada na lista a seguir.

Listagem 4.1 Usando o argumento `parâmetro`

função qualquer (a, b, c) {	← Declara uma função com três parâmetros: a, b e c
<pre> assert (a === 1, 'O valor de a é 1'); assert (b === 2, 'O valor de b é 2'); assert (c === 3, 'O valor de c é 3'); </pre>	Testa os valores corretos
<pre> assert (arguments.length === 5, 'Passamos 5 parâmetros'); </pre>	Ao todo, a função recebe cinco argumentos.
<pre> assert (argumentos [0] === a, 'O primeiro argumento é atribuído a a'); assert (argumentos [1] === b, 'O segundo argumento é atribuído a b'); assert (argumentos [2] === c, 'O terceiro argumento é atribuído a c'); </pre>	Verifica se os três primeiros argumentos correspondem os parâmetros da função
<pre> assert (argumentos [3] === 4, 'Podemos acessar o quarto argumento'); assert (argumentos [4] === 5, 'Podemos acessar o quinto argumento'); } </pre>	Verifica se os argumentos em excesso podem ser acessados por meio do parâmetro <code>argumentos</code>
qualquer que seja (1,2,3,4,5);	← Chama uma função com cinco argumentos

Aqui temos um `qualquer que seja` função que é chamada com cinco argumentos, seja o que for (1,2,3,4,5), embora tenha apenas três parâmetros declarados, a, b, c:

```

função qualquer (a, b, c) {
  ...
}

```

Podemos acessar os três primeiros argumentos por meio de seus respectivos parâmetros de função, a, b, e c:

```

assert (a === 1, 'O valor de a é 1'); assert (b === 2, 'O valor de b é 2');
assert (c === 3, 'O valor de c é 3');

```

Também podemos verificar quantos argumentos no total foram passados para a função usando `argumentos.length` propriedade.

O argumento `parâmetro` também pode ser usado para acessar cada argumento individual por meio de notação de matriz. É importante observar que isso também inclui os argumentos em excesso que não estão associados a nenhum parâmetro de função:

```
assert (argumentos [0] === a, 'O primeiro argumento é atribuído a'); assert (argumentos [1] === b, 'O segundo argumento é atribuído a b'); assert (argumentos [2] === c, 'O terceiro argumento é atribuído a c'); afirmar( argumentos [3] === 4, ' Podemos acessar o quarto argumento '); afirmar( argumentos [4] === 5, ' Podemos acessar o quinto argumento ');
```

Ao longo desta seção, fazemos o possível para evitar chamar o argumentos parâmetro um *variedade*. Você pode se enganar pensando que é um array; afinal, tem um comprimento parâmetro e suas entradas podem ser obtidos usando notação de matriz. Mas é *não* um array JavaScript, e se você tentar usar métodos de array em argumentos (por exemplo, o organizar método usado no capítulo anterior), você não encontrará nada além de tristeza e decepção. Apenas pense em argumentos como um *como matriz* construir e exibir contenção em seu uso.

Como já mencionamos, o ponto principal do argumentos objeto é permitir que acessemos todos os argumentos que foram passados para a função, independentemente de um argumento específico estar associado a um parâmetro de função. Vamos ver como fazer isso implementando uma função que pode calcular a soma de um número arbitrário de argumentos.

Listagem 4.2 Usando o argumentos objeto para executar operações em todos os argumentos de função

```
function sum () {
  var sum = 0;
  para (var i = 0; i < argumentos.length; i++) {
    soma + = argumentos [i];
  }
  soma de retorno;
}
```

← Uma função sem parâmetros definidos explicitamente

Itera por meio de todos os argumentos passados e acessa itens individuais por meio de notação de índice

```
assert (sum (1, 2) === 3, "Podemos somar dois números"); assert (sum (1, 2, 3) === 6, "Podemos somar três números"); assert (sum (1, 2, 3, 4) === 10, "Podemos adicionar quatro números");
```

Chama a função com qualquer número de argumentos

Aqui, primeiro definimos um soma função que não lista explicitamente nenhum parâmetro. Independentemente disso, ainda podemos acessar todos os argumentos da função por meio do argumentos objeto. Nós iteramos todos os argumentos e calculamos sua soma.

Agora vem a recompensa. Podemos chamar a função com qualquer número de argumentos, então testamos alguns casos para ver se tudo funciona. Este é o verdadeiro poder do argumentos objeto. Ele nos permite escrever funções mais versáteis e flexíveis que podem lidar facilmente com diferentes situações.

NOTA Mencionamos anteriormente que, em muitos casos, podemos usar o resto parâmetro em vez do argumentos parâmetro. O resto parâmetro é um array real, o que significa que podemos usar todos os nossos métodos de array favoritos nele. Isso lhe dá uma certa vantagem sobre o argumentos objeto. Como exercício, reescreva a lista 4.2 para usar o resto parâmetro em vez do argumentos parâmetro.

Agora que entendemos como o argumentos objeto funciona, vamos explorar algumas de suas pegadinhas.

ARGUMENTOS OBJETOS COMO ALIAS PARA PARÂMETROS DE FUNÇÃO

O argumentos parâmetro tem uma característica curiosa: ele cria um apelido para parâmetros de função. Se definirmos um novo valor para, por exemplo, argumentos [0], o valor do primeiro parâmetro também será alterado. Dê uma olhada na lista a seguir.

Listagem 4.3 O argumentos parâmetros de função de aliases de objeto

<pre>função infiltrada (pessoa) { afirmar (pessoa === 'jardineiro', 'A pessoa é jardineiro'); assert (argumentos [0] === 'jardineiro', 'O primeiro argumento é um jardineiro'); argumentos [0] = 'ninja'; afirmar (pessoa === 'ninja', 'A pessoa é um ninja agora'); assert (argumentos [0] === 'ninja', 'O primeiro argumento é um ninja'); pessoa = 'jardineiro'; afirmar (pessoa === 'jardineiro', 'A pessoa é jardineiro mais uma vez'); assert (argumentos [0] === 'jardineiro', 'O primeiro argumento é um jardineiro novamente'); }</pre>	O parâmetro pessoa tem o valor "jardineiro" enviado como primeiro argumento.
	Mudando os argumentos objeto também mudará o parâmetro correspondente.
<pre>infiltrar ("jardineiro");</pre>	O alias funciona em ambos os sentidos.

Você pode ver como o argumentos object é um alias para os parâmetros da função. Nós definimos uma função, infiltrar, que leva um único parâmetro, pessoa, e nós o invocamos com o argumento jardineiro. Podemos acessar o valor jardineiro através do parâmetro de função pessoa e através do argumentos objeto:

```
afirme (pessoa === 'jardineiro', 'A pessoa é um jardineiro'); assert (argumentos [0] === 'jardineiro', 'O primeiro argumento é um jardineiro');
```

Porque o argumentos objeto é um alias para os parâmetros da função, se mudarmos o argumentos objeto, a mudança também se reflete no parâmetro de função correspondente:

```
argumentos [0] = 'ninja';  
  
afirme (pessoa === 'ninja', 'A pessoa é um ninja agora'); assert (argumentos [0] === 'ninja', 'O primeiro argumento é um ninja');
```

O mesmo se aplica à outra direção. Se mudarmos um parâmetro, a mudança pode ser observada tanto no parâmetro quanto no argumentos objeto:

```

pessoa = 'jardineiro';

afirmar (pessoa === 'jardineiro',
  'A pessoa é jardineiro mais uma vez'); assert (argumentos [0] ===
'jardineiro',
  'O primeiro argumento é um jardineiro novamente');

```

EVITANDO ALIASES

O conceito de aliasing de parâmetros de função por meio do argumentos objeto pode ser confuso, portanto, o JavaScript fornece uma maneira de recusar usando *modo estrito*.

Modo estrito

O modo estrito é uma adição ES5 ao JavaScript que altera o comportamento dos mecanismos JavaScript para que os erros sejam lançados em vez de detectados silenciosamente. O comportamento de alguns recursos de linguagem é alterado, e alguns recursos de linguagem insegura são completamente banidos (mais sobre isso mais tarde). Uma das coisas que o modo estrito muda é que ele desativa argumentos aliasing.

Como sempre, vamos dar uma olhada em um exemplo simples.

Listagem 4.4 Usando o modo estrito para evitar argumentos aliasing

```

"use estrito";
                                  <----- Ativa o modo estrito

função infiltrada (pessoa) {
  afirmar (pessoa === 'jardineiro',
    'A pessoa é jardineiro'); assert (argumentos [0] === 'jardineiro',
    'O primeiro argumento é um jardineiro');

  argumentos [0] = 'ninja';
                                  <----- Muda o primeiro argumento

  assert (argumentos [0] === 'ninja',
    'O primeiro argumento agora é um ninja');

  afirmar (pessoa === 'jardineiro',
    'A pessoa ainda é um jardineiro');
}

infiltrar ("jardineiro");

```

O argumento da pessoa e o primeiro argumento começam com o mesmo valor.

O primeiro argumento Mudou.

O valor do parâmetro pessoa não mudou.

Aqui começamos colocando a string simples use estrito como a primeira linha de código. Isso informa ao mecanismo JavaScript que desejamos executar o código a seguir no modo estrito. Neste exemplo, o modo estrito altera a semântica do nosso programa de forma que o pessoa parâmetro e o primeiro argumento comece com o mesmo valor:

```
afirme (pessoa === 'jardineiro', 'A pessoa é um jardineiro'); assert (argumentos [0] === 'jardineiro', 'O primeiro argumento é um jardineiro');
```

Mas, ao contrário do modo não estrito, desta vez em torno do argumentos objeto não alias aos parâmetros. Se mudarmos o valor do primeiro argumento, argumentos [0] = 'ninja', o primeiro argumento é alterado, mas o pessoa parâmetro não é:

```
assert (argumentos [0] === 'ninja', 'O primeiro argumento agora é um ninja'); assert (pessoa === 'jardineiro', 'A pessoa ainda é um jardineiro');
```

Vamos revisitar o argumentos objeto posteriormente neste livro, mas por enquanto, vamos nos concentrar em outro parâmetro implícito: esta, o que é de certa forma ainda mais interessante.

4.1.2 O parâmetro this: introduzindo o contexto da função

Quando uma função é chamada, além dos parâmetros que representam os argumentos explícitos fornecidos na chamada da função, um parâmetro implícito denominado esta é passado para a função. O esta parâmetro, um ingrediente vital em JavaScript orientado a objetos, refere-se a um objeto que está associado à invocação da função. Por este motivo, é frequentemente denominado de *contexto da função*.

O contexto da função é uma noção que aqueles que vêm de linguagens orientadas a objetos, como Java, podem pensar que entendem. Em tais linguagens, esta geralmente aponta para uma instância da classe dentro da qual o método é definido.

Mas cuidado! Como veremos em breve, em JavaScript, invocar uma função como um *método* é apenas uma maneira pela qual uma função pode ser chamada. E ao que parece, o que esta parâmetro aponta para não (como em Java ou C #) definido apenas por como e onde a função é definida; também pode ser fortemente influenciado por como a função é *invocada*. Porque compreender a natureza exata do esta parâmetro é um dos pilares mais importantes do JavaScript orientado a objetos, estamos prestes a examinar várias maneiras de invocar funções. Você verá que uma das principais diferenças entre eles é como o valor de esta está determinado. E então daremos uma olhada longa e detalhada nos contextos de função novamente nos próximos capítulos, então não se preocupe se as coisas não se encaixarem imediatamente.

Agora vamos ver, em grande detalhe, como as funções podem ser chamadas.

4,2 Invocando funções

Todos nós chamamos funções JavaScript, mas você já parou para se perguntar o que realmente acontece quando uma função é chamada? Acontece que a maneira como uma função é chamada tem um grande impacto em como o código dentro dela opera, principalmente em como o esta parâmetro, o contexto da função, é estabelecido. Essa diferença é muito mais importante do que pode parecer à primeira vista. Vamos examiná-lo nesta seção e explorá-lo no restante deste livro para ajudar a elevar nosso código ao nível ninja.

Podemos invocar uma função de quatro maneiras, cada uma com suas próprias nuances:

- *Como uma função* - skulk (), em que a função é chamada de maneira direta
- *Como método* - ninja.skulk (), que liga a invocação a um objeto, permitindo a programação orientada a objetos

- *Como construtor* - novo Ninja (), em que um novo objeto é trazido à existência
- *Através da função Aplique ou chamar métodos* - skulk.call (ninja) ou skulk.apply (ninja)

Aqui estão alguns exemplos:

```
função skulk (nome) {}
```

```
função Ninja (nome) {}
```

```
skulk ('Hattori');
```

```
(função (quem) {retornar quem;}) ('Hattori');
```

Chamado como uma função

```
var ninja = {
```

```
  skulk: function () {}
```

```
};
```

```
ninja.skulk ('Hattori');
```

← Invocado como um método de ninja

```
ninja = novo Ninja ('Hattori');
```

← Chamado como um construtor

```
skulk.call (ninja, 'Hattori');
```

← Chamado por meio do método call

```
skulk.apply (ninja, ['Hattori']);
```

← Chamado por meio do método apply

Para todos, exceto o chamar e Aplique abordagens, o operador de invocação de função é um conjunto de parênteses após qualquer expressão avaliada para uma referência de função.

Vamos começar nossa exploração com a forma mais simples, invocando funções como funções.

4.2.1 *Invocação como uma função*

Invocação como função? Bem, é claro que as funções são chamadas como *funções*. Que tolice pensar de outra forma. Mas, na realidade, dizemos que uma função é invocada "como uma função" para distingui-la dos outros mecanismos de invocação: métodos, construtores e

aplicar / ligar. Se uma função não for chamada como um método, como um construtor ou via Aplique ou chamar, é invocado como uma função.

Esse tipo de chamada ocorre quando uma função é chamada usando o operador () e a expressão à qual o operador () é aplicado não faz referência à função como uma propriedade de um objeto. (Nesse caso, teríamos uma invocação de método, mas discutiremos isso a seguir.) Aqui estão alguns exemplos simples:

```
função ninja () {};
```

```
ninja ();
```

Declaração de função

invocado como uma função

Imediatamente invocado
expressão de função,
invocado como uma função

```
var samurai = função () {}; samurai();
```

```
(função(){})(())
```

Expressão de função

invocado como uma função

Quando chamado desta maneira, o contexto da função (o valor do esta palavra-chave) pode ser duas coisas: No modo não estrito, será o contexto global (o janela objeto), enquanto no modo estrito, será Indefinido.

A lista a seguir ilustra a diferença de comportamento entre os modos estrito e não estrito.

Listagem 4.5 Invocação como uma função

<pre>function ninja () { devolva isso; }</pre>	Uma função em modo não estrito
<pre>function samurai () { "use estrito"; devolva isso; }</pre>	Uma função em modo estrito
<pre>janela assert (ninja () ===, "Em uma função ninja 'não estrita'," + "o contexto é o objeto de janela global");</pre>	Como esperado, uma função não estrita tem janela como contexto de função.
<pre>assert (samurai () === indefinido, "Em uma função de samurai 'estrita'," + "o contexto é indefinido");</pre>	A função estrita, por outro lado, possui um contexto indefinido.

NOTA Como você pode ver, o modo estrito é, na maioria dos casos, muito mais simples do que o modo não estrito. Por exemplo, quando a listagem 4.5 invoca uma função como uma função (em oposição a um método), ela não especificou um objeto no qual a função deve ser invocada. Então, em nossa opinião, faz mais sentido que o esta palavra-chave deve ser definida para Indefinido (como no modo estrito), em oposição ao global janela objeto (como no modo não estrito). Em geral, o modo estrito corrige muitas dessas pequenas esquisitices do JavaScript. (Lembra-se dos argumentos com alias do início do capítulo?)

Você provavelmente já escreveu um código como este muitas vezes sem se preocupar muito. Agora vamos subir um degrau, vendo como as funções são chamadas como *métodos*.

4.2.2 Invocação como método

Quando uma função é atribuída a uma propriedade de um objeto e a invocação ocorre referenciando a função usando essa propriedade, então a função é invocada como um *método* desse objeto. Aqui está um exemplo:

```
var ninja = {};
ninja.skulk = função () {};
ninja.skulk ();
```

OK; E daí? A função é chamada de *método* neste caso, mas o que torna isso interessante ou útil? Bem, se você vem de um plano de fundo orientado a objetos, você se lembrará de que o objeto ao qual um método pertence está disponível dentro do corpo do método como esta. A mesma coisa acontece aqui. Quando invocamos uma função como um *método* de um objeto, esse objeto se torna o contexto da função e está disponível dentro

a função através do este parâmetro. Este é um dos principais meios pelos quais o JavaScript permite que o código orientado a objetos seja escrito. (Construtores são outra, e vamos chegar a eles em breve.)

Vamos considerar alguns códigos de teste na próxima listagem para ilustrar as diferenças e semelhanças entre a chamada como uma função e a chamada como um método.

Listagem 4.6 As diferenças entre invocações de função e método

getMyThis obtém um referência ao whatsMyContext função.	<pre>function whatsMyContext () { devolva isso; }</pre>	Retorna o contexto da função que nos permitirá examinar o contexto de fora
Um objeto ninja1 é criado com um getMyThis propriedade que faz referência a whatsMyContext função.	<pre>assert (whatsMyContext () === window, "Chamada de função na janela"); var getMyThis = whatsMyContext; assert (getMyThis () === window, "Outra chamada de função na janela"); var ninja1 = { getMyThis: whatsMyContext }; assert (ninja1.getMyThis () === ninja1, "Trabalhando com o 1º ninja");</pre>	Chamar como uma função define o contexto para o objeto de janela. Invoca a função usando a variável getMyThis. Até embora usemos uma variável, a função ainda é chamada como uma função e o contexto da função é o objeto da janela.
Outro objeto, ninja2, também tem uma propriedade getMyThis referenciando whatsMyContext.	<pre>var ninja2 = { getMyThis: whatsMyContext }; assert (ninja2.getMyThis () === ninja2, "Trabalhando com o 2º ninja");</pre>	Invocar as funções por meio de getMyThis o chama como um método de ninja1. O contexto da função agora é ninja1. Isso é orientação a objetos! Invocar a função como um método de ninja2 mostra que o contexto da função agora é ninja2.

Este teste configura uma função chamada whatsMyContext que usaremos no restante da lista. A única coisa que essa função faz é retornar seu contexto de função para que possamos ver, de fora da função, qual é o contexto da função para a chamada. (Caso contrário, teríamos dificuldade em saber.)

```
function whatsMyContext () {
    devolva isso;
}
```

Quando chamamos a função diretamente pelo nome, este é um caso de invocar a função como uma função, então esperamos que o contexto da função seja o contexto global (janela), porque estamos no modo não estrito. Afirmamos que é assim:

```
assert (whatsMyContext () === window, ...)
```

Em seguida, criamos uma referência para a função `whatsMyContext` em uma variável chamada `getMyThis`: `var getMyThis = whatsMyContext`. Isso não cria uma segunda instância da função; apenas cria uma referência para a mesma função (você sabe, objeto de primeira classe e tudo).

Quando invocamos a função por meio da variável - algo que podemos fazer porque o operador de invocação de função pode ser aplicado a qualquer expressão avaliada como uma função - estamos novamente invocando a função como uma função. Como tal, esperamos novamente que o contexto da função seja janela, e isso é:

```
assert (getMyThis () === window,
        "Outra chamada de função na janela");
```

Agora, ficamos um pouco mais complicados e definimos um objeto em variável `ninja1` com uma propriedade chamada `getMyThis` que recebe uma referência ao `whatsMyContext` função. Ao fazer isso, dizemos que criamos um *método* nomeado `getMyThis` no objeto. Nós não dizemos isso `whatsMyContext` tem *tornar-se* um método de `ninja1`; não tem. Você já viu isso `whatsMyContext` é sua própria função independente que pode ser chamada de várias maneiras:

```
var ninja1 = {
  getMyThis: whatsMyContext
};
```

De acordo com o que afirmamos anteriormente, quando invocamos a função por meio de uma referência de método, esperamos que o contexto da função seja o objeto do método (neste caso, `ninja1`) e afirmamos tanto:

```
assert (ninja1.getMyThis () === ninja1,
        "Trabalhando com o 1º ninja");
```

NOTA Invocar funções como métodos é crucial para escrever JavaScript de uma maneira orientada a objetos. Isso permite que você use esta dentro de qualquer método para fazer referência ao objeto "proprietário" do método - um conceito fundamental na programação orientada a objetos.

Para esclarecer esse ponto, continuamos testando criando outro objeto, `ninja2`, também com uma propriedade chamada `getMyThis` que faz referência a `whatsMyContext` função. Ao invocar esta função por meio do `ninja2` objeto, afirmamos corretamente que seu contexto de função é `ninja2`:

```
var ninja2 = {
  getMyThis: whatsMyContext
};

assert (ninja2.getMyThis () === ninja2,
        "Trabalhando com o 2º ninja");
```

Mesmo que a *mesma* função- `whatsMyContext` —É usado em todo o exemplo, o contexto da função retornado por esta muda dependendo de como `whatsMyContext` é invocado. Por exemplo, a mesma função exata é compartilhada por ambos `ninja1` e `ninja2`,

no entanto, quando é executada, a função tem acesso e pode realizar operações no objeto por meio do qual o método foi invocado. Não precisamos criar cópias separadas de uma função para realizar exatamente o mesmo processamento em objetos diferentes. Este é um princípio da programação orientada a objetos.

Embora seja um recurso poderoso, a maneira como é usado neste exemplo tem limitações. Acima de tudo, quando criamos os dois objetos ninja, podemos compartilhar a mesma função a ser usada como método em cada um, mas temos que usar um pouco de código repetido para configurar os objetos separados e seus métodos.

Mas não há nada para se desesperar - o JavaScript fornece mecanismos para tornar a criação de objetos a partir de um único padrão muito mais fácil do que neste exemplo. Exploraremos esses recursos em profundidade no capítulo 7. Mas, por agora, vamos considerar uma parte desse mecanismo que se relaciona às invocações de função: o *construtor*.

4.2.3 *Invocação como um construtor*

Não há nada de especial em uma função que será usada como construtor. *Funções de construtor* são declarados como quaisquer outras funções, e podemos usar facilmente declarações e expressões de função para construir novos objetos. A única exceção é a função de seta, que, como você verá mais adiante neste capítulo, funciona de maneira um pouco diferente. Mas, em qualquer caso, a principal diferença está em como a função é chamada.

Para invocar a função como um construtor, precedemos a invocação da função com a palavra-chave `new`. Por exemplo, lembre-se do `whatsMyContext` função da seção anterior:

```
função whatsMyContext () {retornar isto; }
```

Se quisermos invocar o `whatsMyContext` funcionar como um construtor, escrevemos isto:

```
new whatsMyContext ();
```

Mas mesmo que possamos invocar `whatsMyContext` como um construtor, essa função não é um construtor particularmente útil. Vamos descobrir o porquê discutindo o que torna os construtores especiais.

NOTA Lembre-se do capítulo 3, quando discutimos maneiras de definir funções? Entre as declarações de função, expressões de função, funções de seta e funções de gerador, também mencionamos *construtores de função*, que nos permitem construir novas funções a partir de strings. Por exemplo: nova função ('a', 'b', 'retornar a + b') cria uma nova função com dois parâmetros,

uma e b, que retorna sua soma. Tenha cuidado para não confundir estes *construtores de função* com *funções de construtor*! A diferença é sutil, mas significativa. Um construtor de função nos permite criar funções a partir de strings criadas dinamicamente. Por outro lado, *funções construtoras*, o tópico desta seção, são funções que usamos para criar e inicializar instâncias de objeto.

AS SUPERPOTÊNCIAS DOS CONSTRUTORES

Invocar uma função como um construtor é um recurso poderoso do JavaScript que exploraremos na listagem a seguir.

Listagem 4.7

Usando um construtor para configurar objetos comuns

Cria dois objetos invocando o construtor com novo. O novo objetos criados são referenciados por ninja1 e ninja2.

```
function Ninja () {
  this.skulk = function () {
    devolva isso;
  };
}

var ninja1 = novo Ninja (); var ninja2 = novo
Ninja ();

assert (ninja1.skulk () === ninja1,
  "O primeiro ninja está se escondendo"); afirmar
(ninja2.skulk () === ninja2,
  "O segundo ninja está se escondendo");
```

Um construtor que cria uma propriedade skulk em qualquer objeto que seja o contexto da função. O método mais uma vez retorna o contexto da função para que possamos testá-lo externamente.

Testa o método skulk dos objetos construídos.
Cada um deve retornar seu próprio objeto construído.

Neste exemplo, criamos uma função chamada Ninja que usaremos para construir, bem, ninjas. Quando invocado com a palavra-chave novo, uma instância de objeto vazio é criada e passada para a função como seu contexto de função, o esta parâmetro. O construtor cria uma propriedade chamada esconder-se neste objeto, ao qual é atribuída uma função, tornando essa função um método do objeto recém-criado.

Em geral, quando um construtor é invocado, algumas ações especiais ocorrem, conforme mostrado na figura 4.1.

Chamando uma função com a palavra-chave novo aciona as seguintes etapas:

- 1 Um novo objeto vazio é criado.
- 2 Este objeto é passado para o construtor como o esta parâmetro e, portanto, torna-se o contexto da função do construtor.
- 3 O objeto recém-construído é retornado como o novo valor do operador (com uma exceção que veremos em breve).

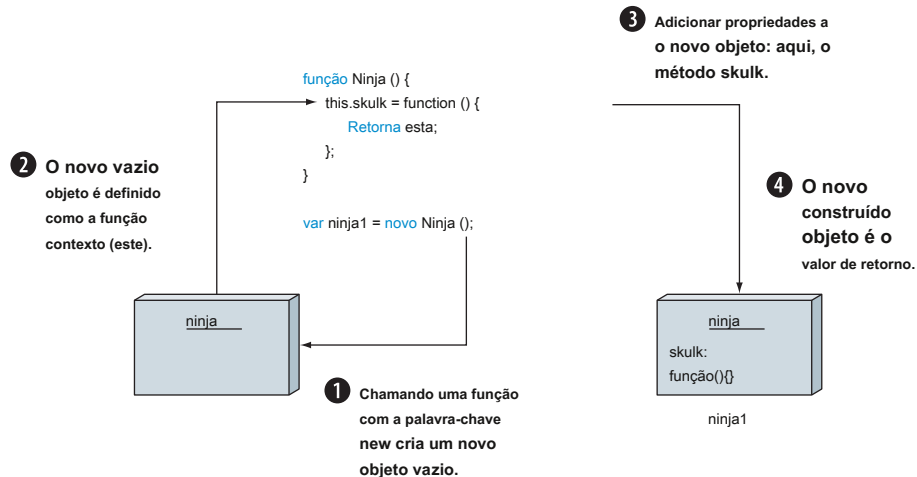


Figura 4.1 Ao chamar uma função com uma palavra-chave **novo**, um novo objeto vazio é criado e definido como o contexto da função do construtor, o **esta** parâmetro.

Os dois últimos pontos tocam no porquê `whatsMyContext` no novo `whatsMyContext ()` faz para um péssimo construtor. O objetivo de um construtor é fazer com que um novo objeto seja criado, configurá-lo e retorná-lo como o valor do construtor. Qualquer coisa que interfira com essa intenção não é apropriada para construtores.

Vamos considerar um construtor mais apropriado, `Ninja`, que configura ninjas furtivos, conforme mostrado na listagem 4.7:

```
function Ninja () {
  this.skulk = function () {
    devolva isso;
  };
}
```

O esconder-se método executa a mesma operação que `whatsMyContext` nas seções anteriores, retornando o contexto da função para que possamos testá-la externamente.

Com o construtor definido, criamos dois novos `Ninja` objetos invocando o construtor duas vezes. Observe que os valores retornados das invocações são armazenados em variáveis que se tornam referências ao recém-criado `Ninja` s:

```
var ninja1 = novo Ninja (); var ninja2 = novo
Ninja ();
```

Em seguida, executamos os testes que garantem que cada invocação do método opere no objeto esperado:

```
assert (ninja1.skulk () === ninja1,
  "O primeiro ninja está se escondendo"); afirmar
(ninja2.skulk () === ninja2,
  "O segundo ninja está se escondendo");
```

É isso! Agora você sabe como criar e inicializar novos objetos com funções construtoras. Chamar uma função com a palavra-chave `novo` retorna o objeto recém-criado. Mas vamos verificar se isso é sempre exatamente verdade.

VALORES DE RETORNO DO CONSTRUTOR

Mencionamos anteriormente que os construtores se destinam a inicializar objetos recém-criados e que o objeto recém-construído é retornado como resultado de uma invocação do construtor (por meio do `novo` operador). Mas o que acontece quando o construtor retorna um valor próprio? Vamos explorar essa situação na lista a seguir.

Listagem 4.8 Construtores retornando valores primitivos

```
function Ninja () {
  this.skulk = function () {
    return true;
  };
  return 1;
}
```

Define uma função construtora chamada `Ninja`

O construtor retorna um valor primitivo específico, o número 1.

```

    }
    afirmar (Ninja () === 1,
        "Valor de retorno honrado quando não chamado como construtor");
    var ninja = novo Ninja ();
    assert (typeof ninja === "objeto",
        "Objeto retornado quando chamado como construtor"); assert (typeof ninja.skulk ===
        "função",
        "objeto ninja tem um método skulk");

```

A função é chamado como um função e seu valor de retorno é 1, conforme esperado.

A função é chamada como um construtor por meio do novo operador.

Os testes verificam se o valor de retorno de 1 é ignorado e se um novo valor inicializado objeto foi voltou de novo.

Se executarmos esta lista, veremos que está tudo bem e bem. O fato de que isso Ninja função retorna um número simples 1 não tem influência significativa sobre como o código se comporta. Se chamarmos o Ninja função como uma função, ele retorna 1 (tal como esperávamos); e se o chamarmos de construtor, com a palavra-chave novo, uma nova ninja objeto é construído e retornado. Até agora tudo bem.

Mas agora vamos tentar algo diferente, uma função construtora que retorna outro objeto, conforme mostrado na lista a seguir.

Listagem 4.9 Construtores que retornam explicitamente os valores do objeto

```

var puppet = {
    regras: falso
};

function Imperador () {
    this.rules = true;
    boneco de retorno;
}

imperador var = novo imperador ();

afirmar (imperador === fantoche,
    "O imperador é apenas um fantoche!"); assert (emperador.rules ===
false,
    "O fantoche não sabe governar!");

```

Cria o nosso objeto com um propriedade conhecida

Retorna aquele objeto apesar de inicializar o objeto passado como este

Invoca a função como um construtor

Os testes mostram que o objeto retornado pelo construtor é atribuído à variável imperador (e não o objeto criado pela nova expressão).

Esta lista tem uma abordagem ligeiramente diferente. Começamos criando um fantoche objeto com a propriedade as regras definido como falso:

```

var puppet = {
    regras: falso
};

```

Então, definimos um Imperador função que adiciona um as regras propriedade para o objeto recém-construído e a define como verdade. Além disso, o Imperador função tem uma peculiaridade; retorna o global fantoche objeto:


```
function Emperor () {
  this.rules = true;
  boneco de retorno;
}
```

Mais tarde, chamamos o Imperador funcionar como um construtor, com a palavra-chave novo:

```
imperador var = novo imperador ();
```

Com isso, criamos uma situação ambígua: obtemos um objeto passado para o construtor como o contexto da função em esta, que inicializamos, mas depois retornamos um completamente diferente fantoche objeto. Qual objeto reinará supremo?

Vamos testar:

```
afirmam (imperador === fantoche, "O imperador é apenas um fantoche!"); assert (emperor.rules === false,
  "O fantoche não sabe governar!");
```

Acontece que nossos testes indicam que o fantoche objeto é retornado como o valor da invocação do construtor, e que a inicialização que realizamos no contexto da função no construtor foi em vão. O fantoche foi exposto!

Agora que passamos por alguns testes, vamos resumir nossas descobertas:

- Se o construtor retorna um objeto, esse objeto é retornado como o valor de todo novo expressão, e o objeto recém-construído passado como esta para o construtor é descartado.
- Se, no entanto, um não-objeto for retornado do construtor, o valor retornado será ignorado e o objeto recém-criado será retornado.

Por causa dessas peculiaridades, as funções destinadas ao uso como construtores são geralmente codificadas de maneira diferente de outras funções. Vamos explorar isso com mais detalhes.

CONSIDERAÇÕES DE CODIFICAÇÃO PARA CONSTRUTORES

A intenção dos construtores é inicializar o novo objeto que será criado pela invocação da função para as condições iniciais. E embora tais funções *posso* ser chamados como funções “normais”, ou mesmo atribuídos a propriedades de objeto para serem chamados como métodos, eles geralmente não são úteis como tal. Por exemplo

```
function Ninja () {
  this.skulk = function () {
    devolva isso;
  };
}
var qualquer = Ninja ();
```

Podemos ligar Ninja como uma função simples, mas o esconder-se propriedade seria criada em janela no modo não estrito - não é uma operação particularmente útil. As coisas ficam ainda mais complicadas no modo estrito, como esta seria indefinido e nosso aplicativo JavaScript travaria. Mas isso é uma coisa boa; se cometermos esse erro no modo não estrito, pode

escapar de nossa atenção (a menos que tenhamos bons testes), mas não há como errar no modo estrito. Este é um bom exemplo de por que o modo estrito foi introduzido.

Como os construtores são geralmente codificados e usados de maneira diferente de outras funções, e não são tão úteis a menos que sejam chamados como construtores, uma convenção de nomenclatura surgiu para distinguir construtores de funções e métodos comuns. Se você tem prestado atenção, talvez já tenha percebido.

Funções e métodos são geralmente nomeados começando com um verbo que descreve o que eles fazem (esquivar, rastejar, esgueirar-se, fazer algo maravilhoso, e assim por diante) e comece com uma letra minúscula. Os construtores, por outro lado, geralmente são nomeados como um substantivo que descreve o objeto que está sendo construído e começam com um caractere maiúsculo:

Ninja, Samurai, Imperador, Ronin, e assim por diante.

É fácil ver como um construtor torna mais elegante criar vários objetos que estejam em conformidade com o mesmo padrão sem ter que repetir o mesmo código indefinidamente. O código comum é escrito apenas uma vez, como o corpo do construtor. No capítulo 7, você verá mais sobre como usar construtores e sobre os outros mecanismos orientados a objetos que o JavaScript fornece para tornar ainda mais fácil configurar padrões de objetos.

Mas não terminamos com as invocações de função ainda. Há ainda outra maneira de o JavaScript nos permitir invocar funções que fornecem um grande controle sobre os detalhes da invocação.

4.2.4 *Invocação com os métodos apply e call*

Até agora, você viu que uma das principais diferenças entre os tipos de invocação de função é o objeto que termina como o contexto de função referenciado pelo implícito esta parâmetro que é passado para a função em execução. Para métodos, é o objeto proprietário do método; para funções de nível superior, é janela ou Indefinido (dependendo do rigor atual); para construtores, é uma instância de objeto recém-criada.

Mas e se quisermos tornar o contexto da função o que quisermos? E se quisermos defini-lo explicitamente? E se ... bem, por que queremos fazer uma coisa dessas?

Para ter uma ideia de por que nos importamos com essa capacidade, veremos um exemplo prático que ilustra um bug surpreendentemente comum relacionado ao tratamento de eventos. Por enquanto, considere que, quando um manipulador de eventos é chamado, o contexto da função é definido para o objeto ao qual o evento foi vinculado. (Não se preocupe se isso parecer vago; você aprenderá sobre o tratamento de eventos em detalhes no capítulo 13.) Dê uma olhada na lista a seguir.

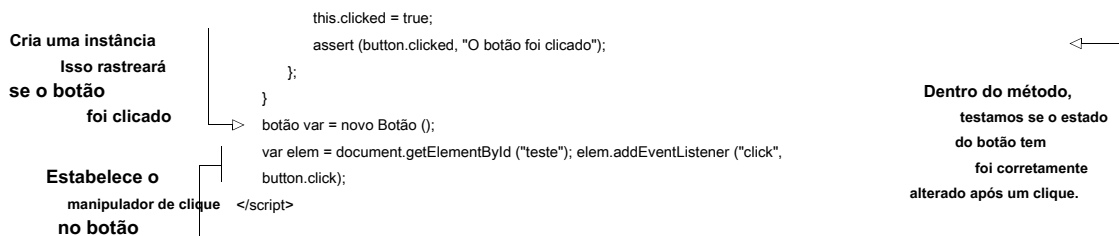
Listagem 4.10 Vinculando um contexto específico a uma função

Uma função construtora que cria objetos que retêm o estado em relação ao nosso botão. Com ele, rastreamos se o botão foi clicado.

```
<button id = "test"> Clique aqui! </button>
<script>
  function Button () {
    this.clicked = false;
    this.click = function () {
```

Um elemento de botão ao qual atribuiremos um manipulador de eventos

Declara o método que usaremos como manipulador de cliques. Por ser um método do objeto, usamos isso dentro da função para obter um referência ao objeto.



Neste exemplo, temos um botão, `< button id = "test"> Clique aqui! </button>`, e nós deseja saber se alguma vez foi clicado. Para reter essas informações de estado, usamos uma função construtora para criar um objeto de apoio denominado botão, no qual armazenaremos o estado clicado:

```

function Button () {
  this.clicked = false;
  this.click = function () {
    this.clicked = true;
    assert (button.clicked, "O botão foi clicado");
  };
}
botão var = novo Botão ();

```

Nesse objeto, também definimos um clique método que servirá como um manipulador de eventos que dispara quando o botão é clicado. O método define a propriedade clicada como verdade e então testa se o estado foi devidamente registrado no objeto de apoio (usamos intencionalmente o botão identificador em vez de esta palavra-chave - afinal, eles devem se referir à mesma coisa, ou não?). Finalmente, estabelecemos o `button.click`

método como um manipulador de cliques para o botão:

```

var elem = document.getElementById ("teste"); elem.addEventListener ("click",
button.click);

```

Quando carregamos o exemplo em um navegador e clicamos no botão, vemos pela exibição da figura 4.2 que algo está errado; o texto marcado indica que o teste falhou. O código na lista

4.10 falha porque o contexto do clique função *não é* referindo-se ao botão objeto como pretendíamos.

Relembrando as lições do início do capítulo, se tivéssemos chamado a função via `button.click ()`, o contexto *seria* foi o botão, porque a função seria invocada como um método no

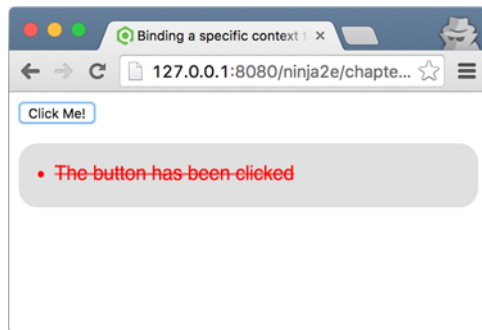


Figura 4.2 Por que nosso teste falhou? Para onde foi a mudança de estado? Normalmente, o contexto do retorno de chamada do evento é o objeto que gera o evento (neste caso, o botão HTML, e não o objeto do botão).

botão objeto. Mas, neste exemplo, o sistema de manipulação de eventos do navegador define o contexto da invocação como o elemento de destino do evento, o que faz com que o contexto seja < botão > elemento, não o botão objeto. Então, nós definimos nosso clique estado no objeto errado!

Este é um problema surpreendentemente comum e, posteriormente neste capítulo, você verá técnicas para evitá-lo completamente. Por enquanto, vamos explorar como lidar com isso examinando como definir explicitamente o contexto da função usando o Aplique e chamar métodos.

USANDO OS MÉTODOS DE APLICAÇÃO E CHAMADA

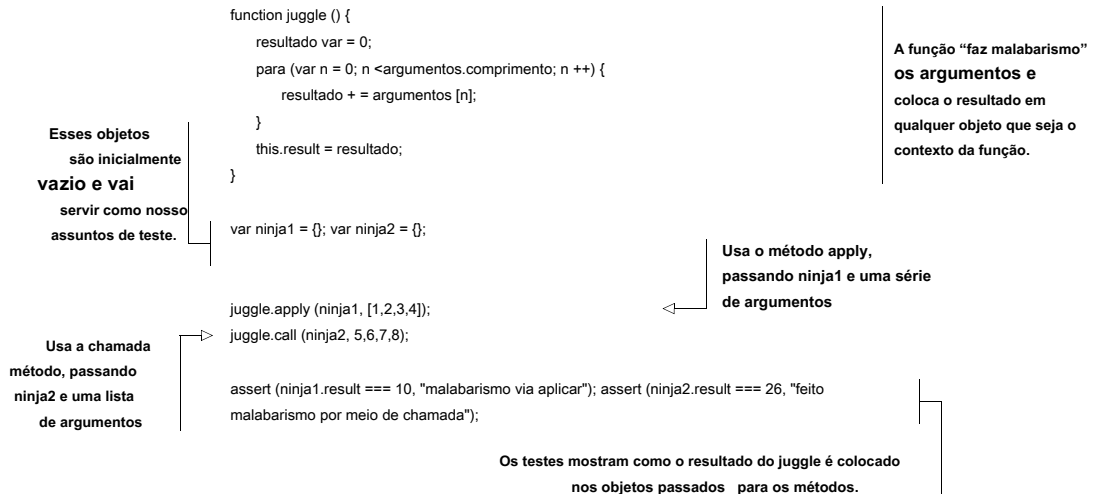
JavaScript fornece um meio para invocarmos uma função e especificarmos explicitamente qualquer objeto que desejamos como o contexto da função. Fazemos isso com o uso de um dos dois métodos existentes para cada função: Aplique e chamar.

Sim, dissemos métodos de funções. Como objetos de primeira classe (criados, aliás, pelo built-in Função construtor), as funções podem ter propriedades como qualquer outro tipo de objeto, incluindo métodos.

Para invocar uma função usando seu Aplique método, passamos dois parâmetros para Aplique: o objeto a ser usado como o contexto da função e uma matriz de valores a ser usada como os argumentos de invocação. O chamar método é usado de maneira semelhante, exceto que os argumentos são passados diretamente na lista de argumentos em vez de como uma matriz.

A lista a seguir mostra os dois métodos em ação.

Listagem 4.11 Usando o Aplique e chamar métodos para fornecer o contexto da função



Neste exemplo, configuramos uma função chamada malabarismo, em que definimos malabarismo como somar todos os argumentos e armazená-los como uma propriedade chamada resultado no contexto da função (referenciado pelo esta palavra-chave). Essa pode ser uma definição um tanto esfarrapada de malabarismo, mas *vontade* nos permitem determinar se os argumentos foram passados para a função corretamente e qual objeto acabou como o contexto da função.

Em seguida, configuramos dois objetos, `ninja1` e `ninja2`, que usaremos como contextos de função, passando o primeiro para o Aplique método, junto com um *variedade* de argumentos, e passando o segundo para o chamar método, junto com um *Lista* de outros argumentos:

```
juggle.apply(ninja1, [1,2,3,4]);
juggle.call(ninja2, 5,6,7,8);
```

Observe que a única diferença entre Aplique e chamar é como os argumentos são fornecidos. No caso de Aplique, usamos uma matriz de argumentos e, no caso de chamar, nós os listamos como argumentos de chamada, após o contexto da função. Veja a figura 4.3.

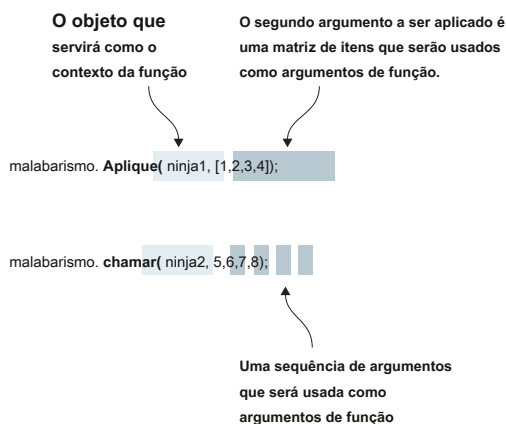


Figura 4.3 Como primeiro argumento, tanto o

chamar e **Aplique** métodos pegam o objeto que será usado como o contexto da função. A diferença está nos seguintes argumentos.

Aplique leva apenas um argumento adicional, uma matriz de valores de argumento; **chamar** recebe qualquer número de argumentos, que serão usados como argumentos de função.

Depois de fornecer os contextos e argumentos de nossa função, continuamos testando! Primeiro, verificamos se `ninja1`, que foi chamado via Aplique, recebeu um resultado propriedade que é o resultado da adição de todos os valores de argumento (1, 2, 3, 4) na matriz passada. Então fazemos o mesmo para `ninja2`, que foi chamado via chamar, onde verificamos o resultado em busca de argumentos 5, 6, 7, e 8:

```
assert(ninja1.result === 10, "malabarismo via aplicar"); assert(ninja2.result === 26, "feito
malabarismo por meio de chamada");
```

A Figura 4.4 fornece uma visão mais detalhada do que está acontecendo na Listagem 4.11.

Esses dois métodos, `chamar` e `Aplique`, pode ser útil sempre que for conveniente usurpar o que normalmente seria o contexto da função com um objeto de nossa própria escolha - algo que pode ser particularmente útil ao invocar funções de retorno de chamada.

FORÇANDO O CONTEXTO DE FUNÇÃO EM CALLBACKS

Vamos considerar um exemplo concreto de forçar o contexto da função a ser um objeto de nossa própria escolha. Usaremos uma função simples para realizar uma operação em cada entrada de um array.

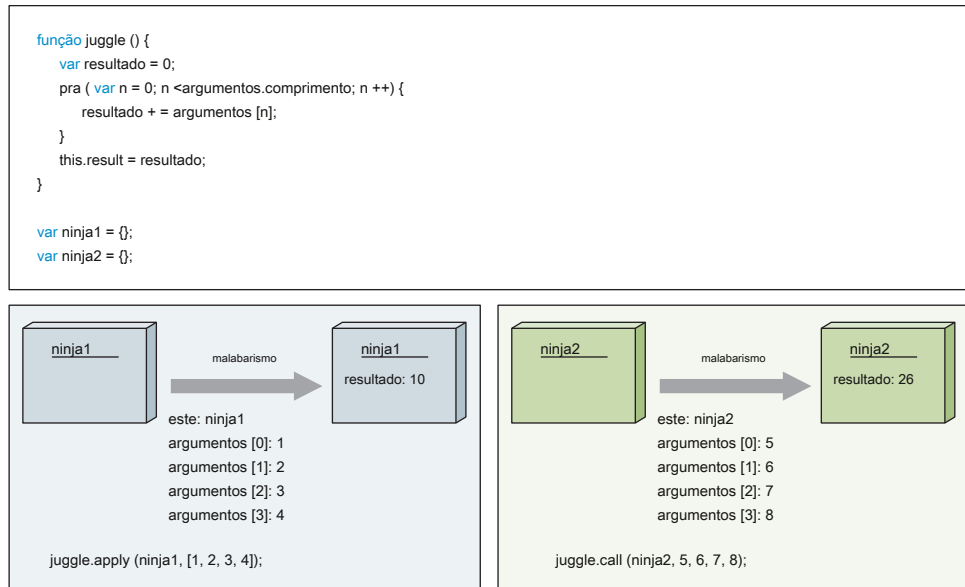


Figura 4.4 Configurando manualmente um contexto de função usando **chamar** e **Aplique** da listagem 4.11 resulta nessas combinações de contextos de função (o **esta** parâmetro) e **argumentos**.

Na programação imperativa, é comum passar o array para um método e usar um pra loop para iterar sobre cada entrada, realizando a operação em cada entrada:

```

função (coleção) {
  para (var n = 0; n < comprimento da coleção; n++) {
    /* fazer algo para a coleção [n] */
  }
}

```

Em contraste, a abordagem funcional é criar uma função que opera em um único elemento e passa cada entrada para essa função:

```

função (item) {
  /* fazer algo para o item */
}

```

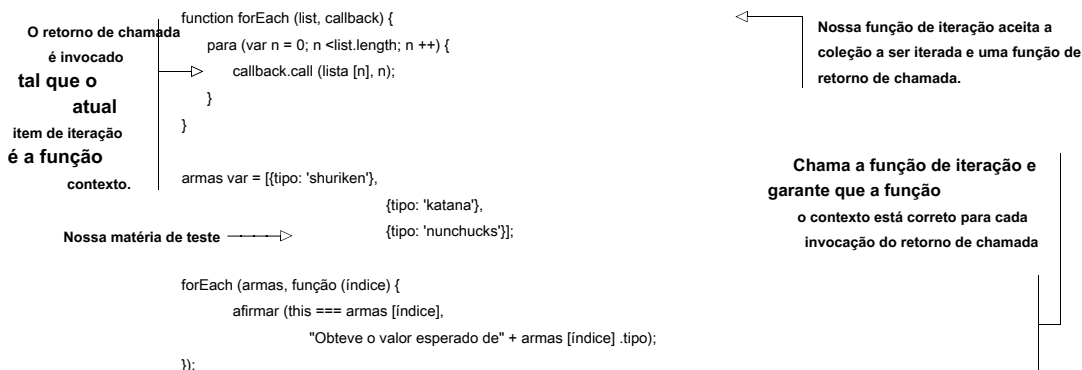
A diferença está em pensar em um nível em que as funções são os principais blocos de construção do programa. Você pode pensar que é discutível e que tudo o que você está fazendo é mover o pra loop out um nível, mas não terminamos de massagear este exemplo ainda.

Para facilitar um estilo mais funcional, todos os objetos de matriz têm acesso a um para cada função que invoca um retorno de chamada em cada elemento de uma matriz. Muitas vezes, é mais sucinto, e este estilo é preferido ao tradicional pra declaração por aqueles familiarizados com a programação funcional. Seus benefícios organizacionais se tornarão ainda mais evidentes (*fosse*, reutilização de código, *fosse*) depois de cobrir os fechamentos no capítulo 5. Tal iteração

função *poderia* passe o elemento atual para o retorno de chamada como um parâmetro, mas a maioria torna o elemento atual o contexto da função do retorno de chamada.

Embora todos os motores JavaScript modernos agora suportem um *para cada* em matrizes, construiremos nossa própria versão (simplificada) de tal função na próxima listagem.

Listagem 4.12 Construindo um *para cada* função para demonstrar a configuração de um contexto de função



A função de iteração exibe uma assinatura simples que espera que a matriz de objetos seja iterada como o primeiro argumento e uma função de retorno de chamada como o segundo. A função itera sobre as entradas da matriz, invocando a função de retorno de chamada para cada entrada:

```
function forEach (list, callback) {
  para (var n = 0; n <list.length; n++) {
    callback.call (lista [n], n);
  }
}
```

Nós usamos o chamar método da função de retorno de chamada, passando a entrada de iteração atual como o primeiro parâmetro e o índice de loop como o segundo. Esta *deve* faz com que a entrada atual se torne o contexto da função e o índice seja passado como o único parâmetro para o retorno de chamada.

Agora, para testar isso! Configuramos um simples armas variedade. Então chamamos o *para cada* função, passando a matriz de teste e um retorno de chamada dentro do qual testamos se a entrada esperada é definida como o contexto da função para cada invocação do retorno de chamada:

```
forEach (armas, função (indice) {
  afirmar (this === armas [indice],
           "Obteve o valor esperado de" + armas [indice] .tipo);
});
```

A Figura 4.5 mostra que nossa função funciona esplendidamente.

Em uma implementação pronta para produção de tal função, haveria muito mais trabalho a fazer. Por exemplo, e se o primeiro argumento não for um array? E se o segundo

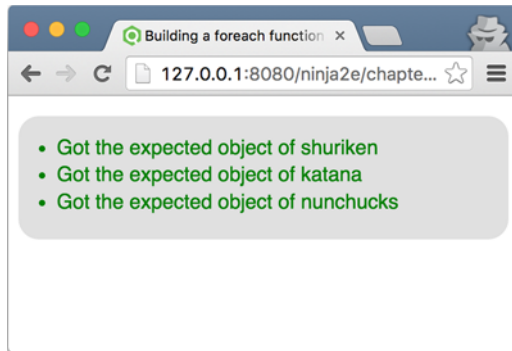


Figura 4.5 Os resultados do teste mostram que temos a capacidade de tornar qualquer objeto que desejamos no contexto de função de uma chamada de retorno.

argumento não é uma função? Como você permitiria que o autor da página encerrasse o loop a qualquer momento? Como exercício, você pode aumentar a função para lidar com essas situações. Outro exercício que você pode fazer é aprimorar a função para que o autor da página possa passar um número arbitrário de argumentos para o retorno de chamada, além do índice de iteração.

Dado que Aplique e chamar fazer praticamente a mesma coisa, aqui está algo que você pode estar se perguntando neste momento: Como decidimos qual usar? A resposta de alto nível é a mesma para muitas dessas perguntas: Usamos aquela que melhora a clareza do código. Uma resposta mais prática é usar aquele que melhor corresponda aos argumentos de que dispomos. Se tivermos um monte de valores não relacionados em variáveis ou especificados como literais, chamar nos permite listá-los diretamente em sua lista de argumentos. Mas se já temos os valores do argumento em uma matriz, ou se é conveniente coletá-los como tal, Aplique poderia ser a melhor escolha.

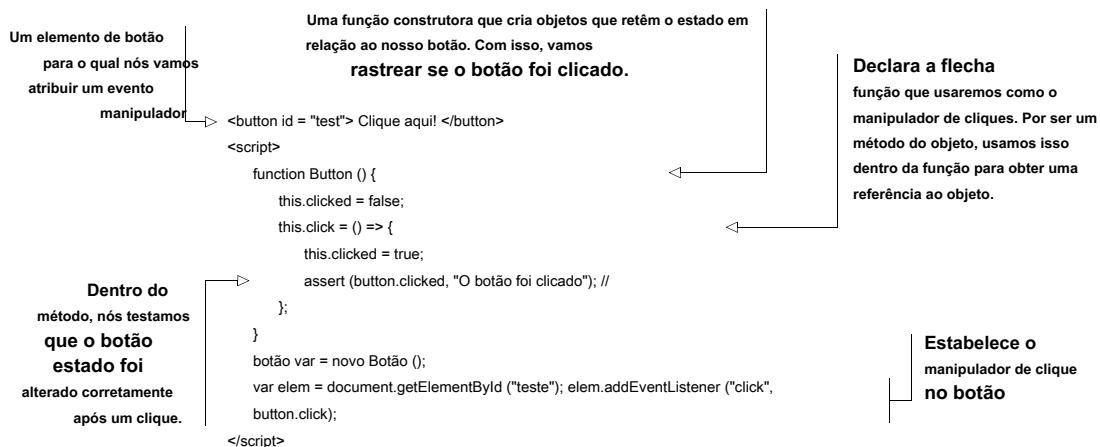
4,3 *Corrigindo o problema de contextos de função*

Na seção anterior, você viu alguns dos problemas que podem ocorrer ao lidar com o contexto de função em JavaScript. Em funções de retorno de chamada (como manipuladores de eventos), o contexto da função pode não ser exatamente o que esperamos, mas podemos usar o chamar e Aplique métodos para contorná-lo. Nesta seção, você verá duas outras opções: funções de seta e o ligar método, que pode, em certos casos, obter o mesmo efeito, mas de uma forma muito mais elegante.

4.3.1 *Usando funções de seta para contornar contextos de função*

Além de nos permitir criar funções de uma maneira mais elegante do que as declarações de função padrão e expressões de função, as funções de seta introduzidas no capítulo anterior têm um recurso que as torna particularmente boas como funções de retorno de chamada: as funções de seta não têm as suas próprias esta valor. Em vez disso, eles se lembram do valor do esta parâmetro no momento da sua definição. Vamos revisitar nosso problema com retornos de chamada de clique de botão na lista a seguir.

Listagem 4.13 Usando funções de seta para contornar contextos de função de retorno de chamada



A única mudança, quando comparada à listagem 4.10, é que a listagem 4.13 usa uma função de seta:

```

this.click = () => {
  this.clicked = true;
  assert (button.clicked, "O botão foi clicado");
};

```

Agora, se executarmos o código, obteremos a saída mostrada na figura 4.6.

Como você pode ver, está tudo bem agora. O objeto de botão rastreia o clicado Estado. O que aconteceu é que nosso manipulador de cliques foi criado dentro do Botão construtor como uma função de seta:

```

function Button () {
  this.clicked = false;
  this.click = () => {
    this.clicked = true;
    assert (button.clicked, "O botão foi clicado");
  };
}

```

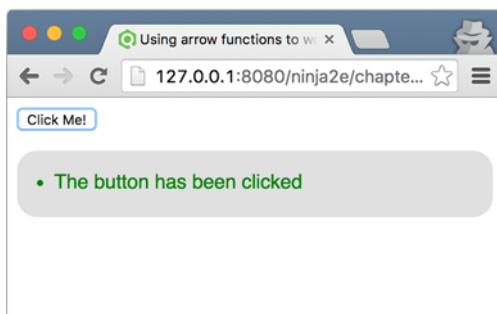


Figura 4.6 As funções de seta não têm contexto próprio. Em vez disso, o contexto é herdado da função na qual foram definidos. O

esta parâmetro em nosso retorno de chamada de função de seta se refere ao objeto de botão.

Como já mencionamos, as funções de seta não têm seus próprios implícitos este parâmetro quando os chamamos; em vez disso, eles se lembram do valor do este parâmetro no momento em que foram criados. No nosso caso, o clique função de seta foi criada dentro de uma função de construtor, onde o este parâmetro é o objeto recém-construído, então sempre que nós (ou o navegador) chamamos o clique função, o valor do este parâmetro sempre será vinculado ao objeto de botão recém-construído.

CAVEAT: FUNÇÕES DE SETA E LITERAIS DE OBJETOS

Porque o valor do este parâmetro é escolhido no momento em que a função de seta é criada, alguns comportamentos aparentemente estranhos podem resultar. Vamos voltar ao nosso exemplo de manipulador de clique de botão.

Digamos que chegamos à conclusão de que não precisamos de uma função construtora, porque temos apenas um botão. Nós o substituímos por um literal de objeto simples, da seguinte maneira.

Listagem 4.14 Funções de seta e literais de objeto

O botão objeto é definido como um objeto literal.

Teste se o botão foi clicado.

clicado é armazenado na janela.

```

<button id = "test"> Clique aqui! </button>
<script>
  afirmar (esta janela ==, "esta janela =="); botão var = {

    clicado: falso,
    clique: () => {
      this.clicado = true;
      assert (button.clicado, "O botão foi clicado"); assert (esta janela ==, "Na função de seta esta janela =="); assert
        (window.clicado, "clicked is stored in window");
    }
  }

  var elem = document.getElementById ("teste"); elem.addEventListener ("click",
    button.click);
</script>

```

O valor deste parâmetro é global code é o objeto global da janela.

Nossa função de seta é uma propriedade de um objeto literal.

O valor de isso em nosso função de seta é o global objeto de janela.

Se executarmos a listagem 4.14, ficaremos novamente desapontados, porque o botão objeto mais uma vez não conseguiu rastrear o clicado Estado. Veja a figura 4.7.

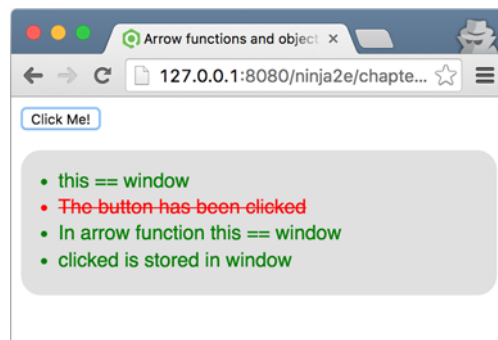


Figura 4.7 Se uma função de seta for definida em um literal de objeto definido no código global, o valor do **esta** parâmetro associado à função de seta é o global **janela** objeto.

Felizmente, espalhamos algumas afirmações em nosso código que ajudarão. Por exemplo, colocamos o seguinte diretamente no código global, a fim de verificar o valor do esta parâmetro:

```
afirmar (esta janela ==, "esta janela ==");
```

Como a afirmação é aprovada, podemos ter certeza de que no código global esta refere-se ao global janela objeto.

Seguimos especificando que o botão literal de objeto tem um clique propriedade da função de seta:

```
botão var = {
  clicado: falso,
  clique: () => {
    this.clicado = true;
    assert (button.clicado, "O botão foi clicado"); assert (esta janela ==, "Na função de seta esta janela =="); assert
    (window.clicado, "Clicked is stored in window");
  }
};
```

Agora, vamos revisitar novamente nossa pequena regra: *As funções de seta captam o valor do esta parâmetro no momento da sua criação*. Porque o clique a função de seta é criada como um valor de propriedade em um literal de objeto, e o literal de objeto é criado em código global, o esta valor da função de seta será o esta valor do código global. E, como vimos desde a primeira afirmação colocada em nosso código global

```
afirmar (esta janela ==, "esta janela ==");
```

o valor do esta parâmetro no código global é o global janela objeto. Portanto, nosso clicado propriedade será definida no global janela objeto, e não em nosso botão objeto. Só para ter certeza, no final, verificamos se o janela objeto foi atribuído a um clicado propriedade:

```
assert (window.clicado, "Clicked is stored in window");
```

Como você pode ver, não se lembrar de todas as consequências das funções das setas pode levar a alguns erros sutis, então tome cuidado!

Agora que exploramos como as funções de seta podem ser usadas para contornar o problema de contextos de função, vamos continuar com outro método para corrigir o mesmo problema.

4.3.2 Usando o método bind

Neste capítulo, você já conheceu dois métodos aos quais toda função tem acesso, chamar e Aplique, e você viu como usá-los para maior controle sobre o contexto e os argumentos de nossas invocações de função.

Além desses métodos, cada função tem acesso ao ligar método que, em suma, cria uma nova função. Esta função tem o mesmo corpo, mas seu contexto é *sempre*

ligado a um certo objeto, *independentemente* da maneira como o invocamos.

Vamos revisitar nosso pequeno problema com manipuladores de clique de botão uma última vez.

Listagem 4.15 Vinculando um contexto específico a um manipulador de eventos

```
<button id = "test"> Clique aqui! </button>
<script>
    botão var = {
        clicado: falso,
        click: function () {
            this.clicado = true;
            assert (button.clicado, "O botão foi clicado");
        }
    };
    var elem = document.getElementById ("teste"); elem.addEventListener ("click", botão.clique.bind (botão));

    var boundFunction = button.click.bind (button); assert (boundFunction! = button.click,
        "Chamar bind cria uma função completamente nova");
</script>
```

Usa a função de ligação para
criar uma nova função
vinculado ao objeto de botão

O molho secreto adicionado aqui é o `ligar()` método:

```
elem.addEventListener ("click", botão.clique.bind (botão));
```

O `ligar` método está disponível para todas as funções e é projetado para *Criar* e devolver um *novo* função que está ligada ao objeto passado (neste caso, o botão objeto). O valor do esta parâmetro é sempre definido para esse objeto, independentemente da forma como a função vinculada foi chamada. Além disso, a função associada se comporta como a função de origem, pois possui o mesmo código em seu corpo.

Sempre que o botão é clicado, essa função associada será invocada com o botão objeto como seu contexto, porque usamos isso botão objeto como um argumento para `ligar`.

Observe que chamar o `ligar` método não modifica a função original. Ele cria uma função completamente nova, um fato afirmado no final do exemplo:

```
var boundFunction = button.click.bind (button); assert (boundFunction! = button.click,
    "Chamar bind cria uma função completamente nova");
```

Com isso, encerraremos nossa exploração do contexto da função. Descanse por enquanto, porque no próximo capítulo, lidaremos com um dos conceitos mais importantes em JavaScript: fechamentos.

4,4 *Resumo*

- Ao invocar uma função, além dos parâmetros explicitamente declarados na definição da função, as invocações da função são passadas em dois parâmetros implícitos: argumentos e esta:
 - O argumento parâmetro é uma coleção de argumentos transmitidos à função. Tem um comprimento propriedade que indica quantos argumentos foram passados e nos permite acessar os valores dos argumentos que não possuem parâmetros correspondentes. No modo não estrito, o argumento objeto alias os parâmetros da função (alterar o argumento altera o valor do parâmetro e vice-versa). Isso pode ser evitado usando o modo estrito. O esta parâmetro representa o contexto da função, um objeto ao qual a invocação da função está associada. Quando esta é determinado pode depender de como uma função é definida, bem como de como ela é chamada.
- Uma função pode ser chamada de quatro maneiras: Como uma
 - função: `skulk ()`
 - Como método: `ninja.skulk ()`
 - Como construtor: `novo Ninja ()`
 - Via seu Aplique e chamar métodos: `skulk.call (ninja)` ou `skulk.apply (ninja)`
- A forma como uma função é chamada influencia o valor do esta parâmetro:
 - Se uma função for invocada como uma função, o valor do esta parâmetro geralmente é o global janela objeto em modo não estrito, e Indefinido no modo estrito. Se uma função é chamada como um método, o valor do esta parâmetro geralmente é o objeto no qual a função foi chamada.
 -
 - Se uma função for invocada como um construtor, o valor do esta parâmetro é o objeto recém-construído.
 - Se uma função é chamada por meio de chamar e Aplique, o valor do esta parâmetro é o primeiro argumento fornecido para chamar e Aplique.
- As funções de seta não têm seu próprio valor de esta parâmetro. Em vez disso, eles o pegam no momento de sua criação.
- Use o `ligar método`, disponível para todas as funções, para criar uma nova função que está sempre ligada ao argumento do `ligar método`. Em todos os outros aspectos, a função vinculada se comporta como a função original.

4,5 *Exercícios*

- 1 A função a seguir calcula a soma dos argumentos transmitidos usando argumentos objeto:

```
function sum () {
  var sum = 0;
  para (var i = 0; i < argument.length; i++) {
    soma += argumentos [i];
  }
}
```

```
soma de retorno;
}
```

```
assert (sum (1, 2, 3) === 6, 'A soma dos três primeiros números é 6'); assert (sum (1, 2, 3, 4) === 10, 'A soma dos quatro primeiros números é 10');
```

Usando os parâmetros restantes introduzidos no capítulo anterior, reescreva o soma função para que não use o argumentos objeto.

- 2 Depois de executar o código a seguir, quais são os valores das variáveis ninja e samurai?

```
function getSamurai (samurai) {
  "use estrito"

  argumentos [0] = "Ishida";

  retorno samurai;
}

function getNinja (ninja) {
  argumentos [0] = "Fuma";
  retornar ninja;
}

var samurai = getSamurai ("Toyotomi"); var ninja = getNinja ("Yoshi");
```

- 3 Ao executar o código a seguir, qual das afirmações será aprovada?

```
function whoAml1 () {
  "use estrito";
  devolva isso;
}

function whoAml2 () {
  devolva isso;
}

assert (whoAml1 () === window, "Window?"); assert (whoAml2 () ===
window, "Window?");
```

- 4 Ao executar o código a seguir, qual das afirmações será aprovada?

```
var ninja1 = {
  whoAml: function () {
    devolva isso;
  }
};

var ninja2 = {
  whoAml: ninja1.whoAml
};
```

```

var identificar = ninja2.whoAml;

assert (ninja1.whoAml () === ninja1, "ninja1?"); assert (ninja2.whoAml () === ninja1, "ninja1
novamente?");

assert (identifique () === ninja1, "ninja1 novamente?");

assert (ninja1.whoAml.call (ninja2) === ninja2, "ninja2 aqui?");

```

5 Ao executar o código a seguir, qual das afirmações será aprovada?

```

function Ninja () {
  this.whoAml = () => isso;
}

var ninja1 = novo Ninja (); var ninja2 = {

  whoAml: ninja1.whoAml
};

assert (ninja1.whoAml () === ninja1, "ninja1 aqui?"); assert (ninja2.whoAml () === ninja2, "ninja2
aqui?");

```

6 Qual das afirmações a seguir será aprovada?

```

function Ninja () {
  this.whoAml = function () {
    devolva isso;
  }.bind (este);
}

var ninja1 = novo Ninja (); var ninja2 = {

  whoAml: ninja1.whoAml
};

assert (ninja1.whoAml () === ninja1, "ninja1 aqui?"); assert (ninja2.whoAml () === ninja2, "ninja2
aqui?");

```

5

Funções para o mestre: fechamentos e escopos

Este capítulo cobre

- Usando fechamentos para simplificar o desenvolvimento
- Acompanhar a execução de programas JavaScript com contextos de execução
- Rastreamento de escopos variáveis com ambientes lexicais
- Compreendendo os tipos de variáveis Explorando como funcionam os fechamentos

Intimamente vinculado às funções que aprendemos nos capítulos anteriores, os encerramentos são um recurso definidor do JavaScript. Embora muitos desenvolvedores de JavaScript possam escrever código sem compreender os benefícios dos encerramentos, seu uso pode não apenas nos ajudar a reduzir a quantidade e a complexidade do código necessária para adicionar recursos avançados, mas também nos permitir fazer coisas que de outra forma não seriam possíveis, ou seria muito complexo para ser viável. Por exemplo, quaisquer tarefas envolvendo retornos de chamada, como manipulação de eventos ou animações, seriam significativamente mais complexas sem encerramentos. Outros, como fornecer suporte para variáveis de objeto privado, seriam totalmente

impossível. A paisagem da linguagem e a maneira como escrevemos nosso código é para sempre moldada pela inclusão de fechamentos.

Tradicionalmente, os fechamentos têm sido uma característica das linguagens de programação puramente funcionais. Vê-los passar para o desenvolvimento dominante é encorajador. É comum encontrar fechamentos que permeiam as bibliotecas JavaScript, junto com outras bases de código avançadas, devido à sua capacidade de simplificar drasticamente operações complexas.

Os fechamentos são um efeito colateral de como os escopos funcionam em JavaScript. Por esse motivo, exploraremos as regras de escopo do JavaScript, com um foco especial nas adições recentes. Isso ajudará você a entender como os fechamentos funcionam nos bastidores. Vamos começar!

.....

Quantos escopos diferentes pode uma variável ou método tem, e quais são eles?

Você sabe? Como os identificadores e seus valores são rastreados?

O que é uma variável mutável e como você define uma em JavaScript?

.....

5.1 *Entendendo fechamentos*

UMA *fecho* permite que uma função acesse e manipule variáveis externas a essa função. Os fechamentos permitem que uma função acesse todas as variáveis, bem como outras funções, que estão no escopo quando a própria função é definida.

NOTA Você provavelmente está familiarizado com o conceito de escopos, mas apenas no caso, um *escopo* refere-se à visibilidade de identificadores em certas partes de um programa. Um escopo é uma parte do programa em que um certo nome está vinculado a uma determinada variável.

Isso pode parecer intuitivo até que você se lembre de que uma função declarada pode ser chamada a qualquer momento posterior, mesmo *depois de* o escopo em que foi declarado desapareceu. Este conceito é provavelmente melhor explicado por meio de código. Mas antes de entrarmos em exemplos concretos que irão ajudá-lo a desenvolver animações mais elegantes em código ou para definir propriedades de objetos privados, vamos começar pequeno, com a seguinte listagem.

Listagem 5.1 Um fechamento simples

```
var outerValue = "ninja"; function outerFunction
() {
    afirmar( outerValue === " ninja ", " posso ver o ninja. ");
}
outerFunction ();
```

Define um valor em escopo global

Declara uma função em escopo global

Executa a função

Neste exemplo de código, declaramos uma variável `outerValue` e uma função `outerFunction` no mesmo escopo - neste caso, o escopo global. Depois, ligamos `outerFunction`.

Como você pode ver na figura 5.1, a função é capaz de “ver” e acessar o `outerValue` variável. Provavelmente, você escreveu um código como este centenas de vezes sem perceber que estava criando um encerramento!

Não estou impressionado? Acho que isso não é surpreendente. Porque ambos `outerValue` e `outerFunction` são declarados em escopo global, esse escopo (que é um encerramento) nunca vai embora (enquanto nosso aplicativo estiver em execução). Não é surpreendente que a função possa acessar a variável, porque ela ainda está no escopo e é viável.

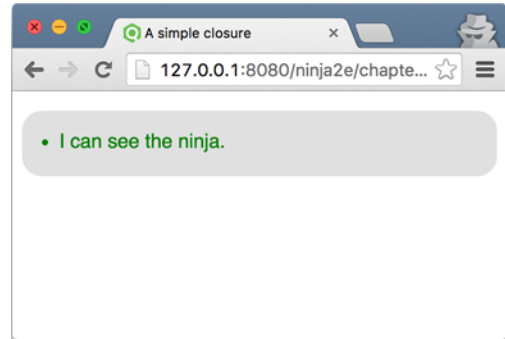
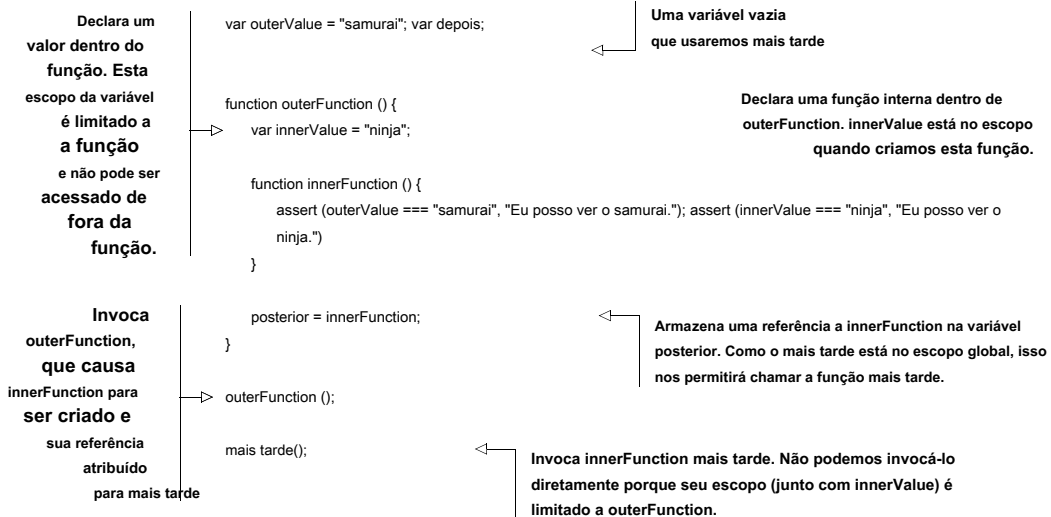


Figura 5.1 Nossa função encontrou o ninja, que estava escondido à vista de todos.

Mesmo que o fechamento exista, seus benefícios ainda não são claros. Vamos incrementar na próxima lista.

Listagem 5.2 Outro exemplo de fechamento



Vamos analisar demais o código em `innerFunction` e ver se podemos prever o que pode acontecer:

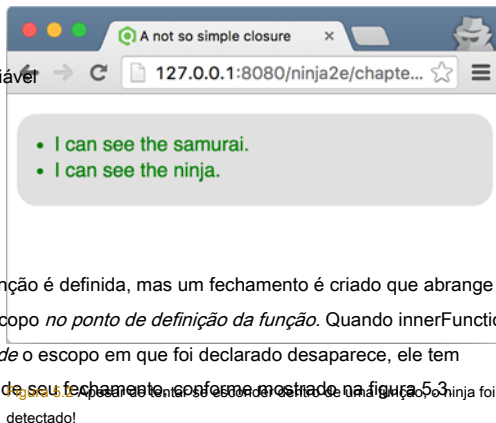
- O primeiro afirmar está certo de passar; `outerValue` está no âmbito global e é visível para tudo. Mas e quanto ao segundo afirmar?
- Estamos executando `innerFunction` *depois de* `outerFunction` foi executado através do truque de copiar uma referência à função para a variável global `mais tarde`.

- Quando `innerFunction` executa, o escopo dentro da função externa há muito se foi e não é visível no ponto em que estamos invocando a função por meio mais tarde.
- Portanto, poderíamos muito bem esperar afirmar falhar, como `innerValue` com certeza será `Indefinido`. Direita?

Mas quando executamos o teste, vemos a tela da figura 5.2.

Como pode ser? Que magia permite o `innerValue` variável ainda estar “viva” quando executamos a função interna, muito depois de o escopo em que foi criada ter desaparecido? A resposta é encerramentos.

Quando declaramos `innerFunction` dentro da função externa, não apenas a declaração da função é definida, mas um fechamento é criado que abrange a definição da função, bem como todas as variáveis no escopo *no ponto de definição da função*. Quando `innerFunction` eventualmente executa, mesmo se for executado *depois de* o escopo em que foi declarado desaparece, ele tem acesso ao escopo original em que foi declarado por meio de seu fechamento, conforme mostrado na figura 5.3.



Isso é o que significa encerramentos. Eles criam uma “bolha de segurança” da função e das variáveis em escopo no ponto da definição da função, de modo que a função tenha tudo o que precisa para ser executada. Esta bolha, contendo a função e suas variáveis, permanece enquanto a função permanece.

Embora toda essa estrutura não seja prontamente visível (não há nenhum objeto de “fechamento” contendo todas as informações que você pode inspecionar), armazenar e referenciar informações dessa forma tem um custo direto. É importante lembrar que cada função que acessa informações por meio de um fecho tem uma “bola e uma corrente” anexada a ela, carregando essas informações. Portanto, embora os fechamentos sejam incrivelmente úteis, eles não são isentos de sobrecarga. Todas essas informações precisam ser mantidas na memória até que seja

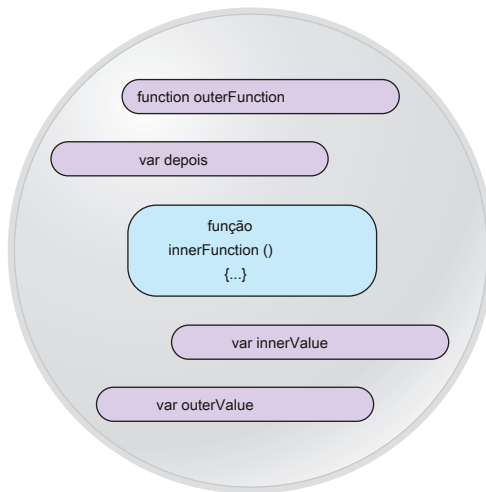


Figura 5.3 Como uma bolha protetora, o fechamento para `innerFunction` mantém as variáveis no escopo da função vivas enquanto a função existir.

absolutamente claro para o mecanismo JavaScript que não é mais necessário (e é seguro coletar lixo) ou até que a página seja descarregada.

Não se preocupe; isso não é tudo o que temos a dizer sobre como funcionam os fechamentos. Mas antes de explorar os mecanismos que permitem fechamentos, vamos dar uma olhada em seus usos práticos.

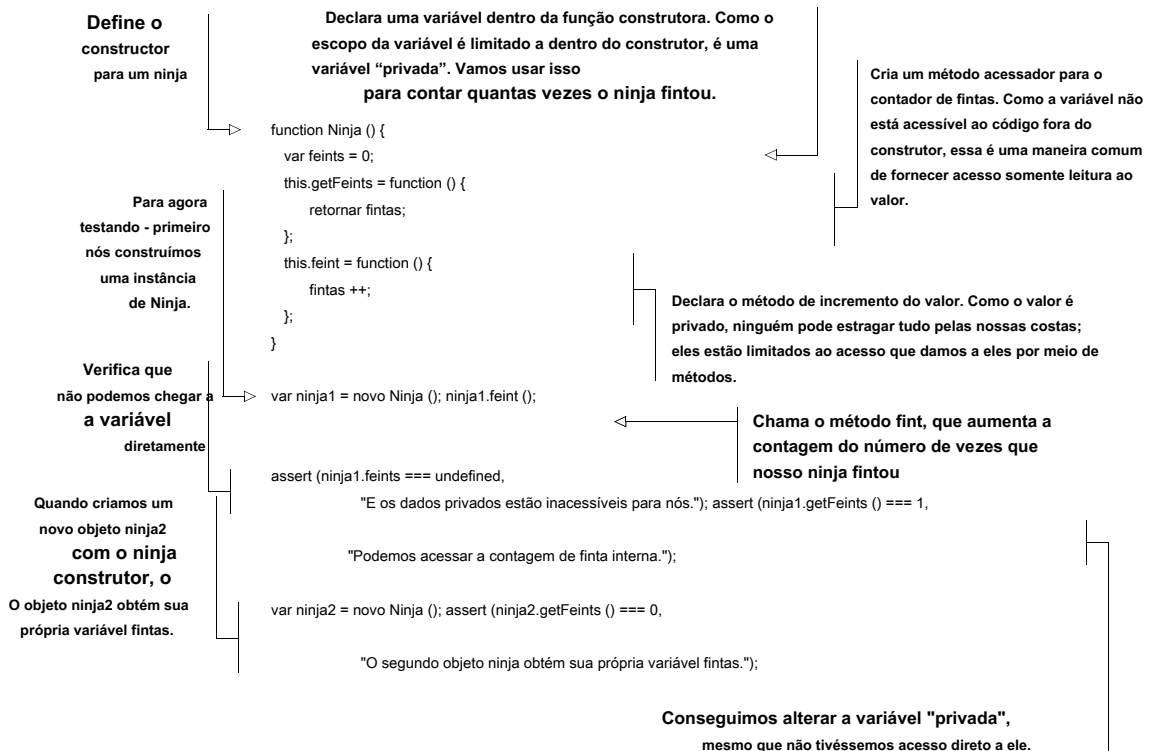
5,2 Colocando fechamentos para funcionar

Agora que temos um entendimento de alto nível sobre fechamentos, vamos ver como colocá-los para funcionar em nossos aplicativos JavaScript. Por enquanto, vamos nos concentrar em seus aspectos práticos e benefícios. Posteriormente neste capítulo, revisitaremos os mesmos exemplos para ver exatamente o que está acontecendo nos bastidores.

5.2.1 Imitando variáveis privadas

Muitas linguagens de programação usam *variáveis privadas*—Propriedades de um objeto que estão ocultas de terceiros. Este é um recurso útil, porque não queremos sobrecarregar os usuários de nossos objetos com detalhes de implementação desnecessários ao acessar esses objetos de outras partes do código. Infelizmente, JavaScript não tem suporte nativo para variáveis privadas. Mas, usando um encerramento, podemos obter uma aproximação aceitável, conforme demonstrado pelo código a seguir.

Listagem 5.3 Usando fechamentos para aproximar variáveis privadas



Aqui criamos uma função, *Ninja*, para servir como um construtor. Introduzimos o uso de uma função como construtor no capítulo 3 (e daremos uma olhada em profundidade no capítulo 7). Por enquanto, lembre-se de que ao usar o novo em uma função, uma nova instância de objeto é criada e a função é chamada com esse novo objeto como seu contexto, para servir como um construtor para esse objeto. Então esta dentro da função se refere a um objeto recém-instanciado.

Dentro do construtor, definimos uma variável para manter o estado, *fintas*. As regras de escopo do JavaScript para esta variável limitam sua acessibilidade a *dentro de* o construtor. Para dar acesso ao valor da variável do código que está fora do escopo, definimos um *acessor* método: *getFeints*, que pode ser usado para ler a variável privada. (Os métodos de acesso são frequentemente chamados *getters*.)

```
function Ninja () {  
  var feints = 0;  
  this.getFeints = function () {  
    retornar fintas;  
  };  
  this.feint = function () {  
    fintas ++;  
  };  
}
```

Um método de implementação, *finta*, é então criado para nos dar controle sobre o valor da variável. Em um aplicativo do mundo real, isso pode ser um método de negócios, mas neste exemplo, ele apenas incrementa o valor de *fintas*.

Depois que o construtor tiver cumprido sua função, podemos chamar o *finta* método no recém-criado *ninja1* objeto:

```
var ninja1 = novo Ninja ();  
ninja1.feint ();
```

Nossos testes mostram que podemos usar o método acessador para obter o valor da variável privada, mas não podemos acessá-lo diretamente. Isso nos impede de fazer alterações não controladas no valor da variável, como se fosse uma verdadeira variável privada. Essa situação é ilustrada na figura 5.4.

O uso de encerramentos permite que o estado do *ninja* seja mantido dentro de um método, sem permitir que ele seja acessado diretamente por um usuário do método - porque a variável está disponível para os métodos internos por meio de seus encerramentos, mas não para o código que fica fora do construtor .

Este é um vislumbre do mundo do JavaScript orientado a objetos, que exploraremos com mais profundidade no capítulo 7. Por enquanto, vamos nos concentrar em outro uso comum de encerramentos.

5.2.2 Usando fechamentos com *callbacks*

Outro uso comum de encerramentos ocorre ao lidar com retornos de chamada - quando uma função é chamada em um momento posterior não especificado. Muitas vezes, dentro de tais funções, frequentemente

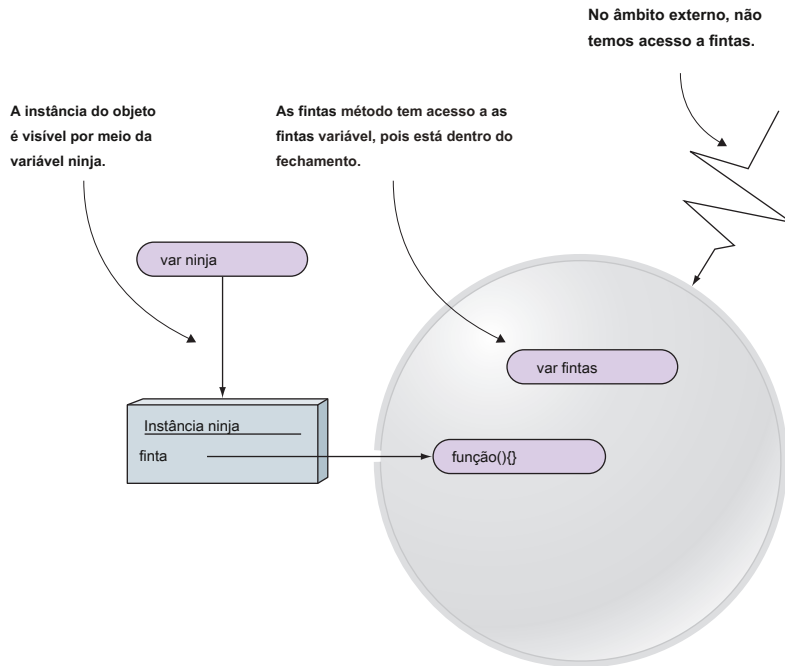


Figura 5.4 Ocultar a variável dentro do construtor a mantém invisível para o escopo externo, mas onde é importante, a variável está viva e bem, protegida pelo fechamento.

precisa acessar dados externos. A lista a seguir mostra um exemplo que cria uma animação simples com cronômetros de retorno de chamada.

Listagem 5.4 Usando um fechamento em um retorno de chamada de intervalo do cronômetro

```

<div id = "box1"> Primeira caixa </div>
<script>
  function animatelt (elementId) {
    var elem = document.getElementById (elementId); var tick = 0;

    var cronômetro = setInterval (function () {
      if (assinale <100) {
        elem.style.left = elem.style.top = tick + "px"; tick ++;
      }
      outro {
        clearInterval (temporizador);
      }
    });
  }

```

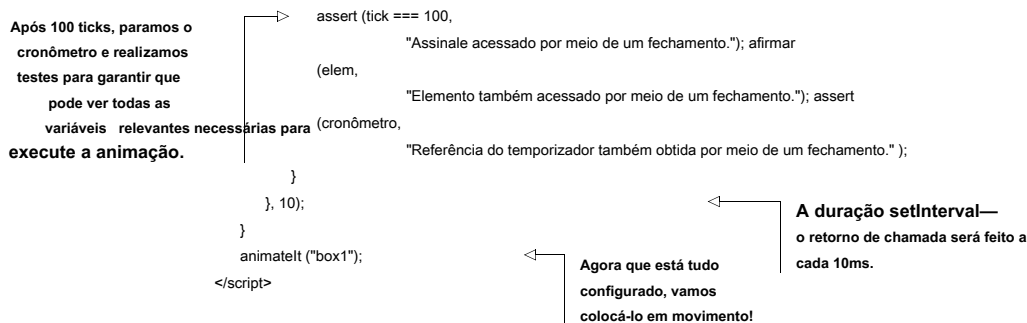
Estabelece um contador para manter rastreo animação ticks (degraus)

O retorno de chamada do cronômetro é invocado a cada 10 milissegundos. Por 100 ticks, ele ajusta a posição do elemento.

Cria o elemento que vamos animar

Dentro de animatelt função, nós temos um referência a esse elemento.

Uma função integrada que cria e inicia um cronômetro de intervalo, dado um ligar de volta



O que é especialmente importante sobre este código é que ele usa uma única função anônima, colocada como um `setInterval` argumento, para realizar a animação do alvo

`div` elemento. Essa função acessa três variáveis: `elem`, `tick`, e `cronômetro`, por meio de um fechamento, para controlar o processo de animação. As três variáveis (a referência ao elemento DOM, `elem`; o contador de carrapatos, `marcação`; e a referência do cronômetro, `cronômetro`) tudo deve ser mantido *entre* as etapas da animação. E precisamos mantê-los fora do escopo global.

Mas o exemplo ainda funcionará bem se movermos as variáveis para fora do `animatelt` função e no âmbito global. Então, por que todo o braço se debatendo sobre não poluir o escopo global?

Vá em frente e mova as variáveis para o escopo global e verifique se o exemplo ainda funciona. Agora modifique o exemplo para animar dois elementos: Adicione outro elemento com um ID exclusivo e chame o `animatelt` função com esse ID logo após a chamada original.

O problema se torna imediatamente óbvio. Se mantivermos as variáveis no escopo global, precisamos de um conjunto de três variáveis para cada animação. Caso contrário, eles vão pisar uns nos outros, tentando usar o mesmo conjunto de variáveis para manter o controle de vários estados.

Definindo as variáveis *lado de dentro* a função, e contando com fechamentos para torná-los disponíveis para as invocações de retorno de chamada do temporizador, cada animação obtém sua própria "bolha" privada de variáveis, conforme mostrado na Figura 5.5.

Sem encerramentos, fazer várias coisas ao mesmo tempo, seja manipulação de eventos, animações ou até mesmo solicitações do servidor, seria incrivelmente difícil. Se você estava esperando por um motivo para se preocupar com os fechamentos, é isso!

Este exemplo é particularmente bom para demonstrar como os encerramentos são capazes de produzir algum código surpreendentemente intuitivo e conciso. Incluindo as variáveis no `animatelt` função, criamos um encerramento implícito sem a necessidade de qualquer sintaxe complexa.

Há outro conceito importante que este exemplo deixa claro. Não apenas podemos ver os valores que essas variáveis tinham no momento em que o encerramento foi criado, mas podemos atualizá-los dentro do encerramento enquanto a função dentro do encerramento é executada. O fechamento não é apenas um instantâneo do estado do escopo no momento da criação, mas um encapsulamento ativo desse estado que podemos modificar enquanto o fechamento existir.

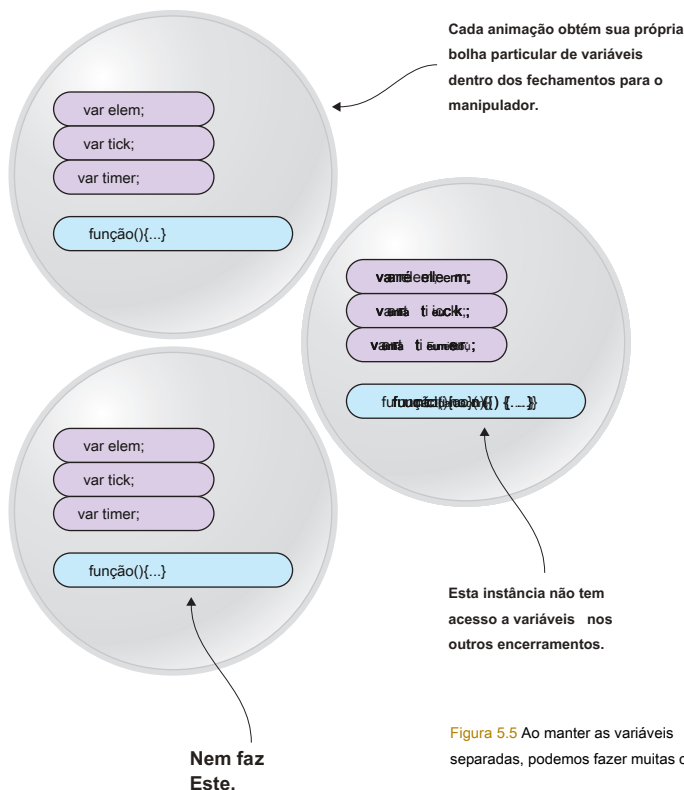


Figura 5.5 Ao manter as variáveis para várias instâncias da função separadas, podemos fazer muitas coisas ao mesmo tempo.

Os fechamentos estão intimamente relacionados aos escopos, portanto, passaremos uma boa parte deste capítulo explorando as regras de escopo em JavaScript. Mas, primeiro, começaremos com os detalhes de como a execução do código é rastreada em JavaScript.

5.3 Acompanhar a execução do código com contextos de execução

Em JavaScript, a unidade fundamental de execução é uma função. Nós os usamos o tempo todo, para calcular algo, para realizar efeitos colaterais, como alterar a IU, para obter a reutilização de código ou para tornar nosso código mais fácil de entender. Para cumprir seu propósito, uma função pode chamar outra função, que por sua vez pode chamar outra função e assim por diante. E quando uma função faz seu trabalho, a execução de nosso programa deve retornar à posição de onde a função foi chamada. Mas você já se perguntou como o mecanismo JavaScript mantém o controle de todas essas funções em execução e posições de retorno?

Como mencionamos no capítulo 2, existem dois tipos principais de código JavaScript: *código global*, colocado fora de todas as funções, e *código de função*, contido em funções. Quando nosso código está sendo executado pelo motor JavaScript, cada instrução é executada em um determinado *contexto de execução*.

E assim como temos dois tipos de código, temos dois tipos de contextos de execução: a *contexto de execução global* e um *contexto de execução da função*. Aqui está a diferença significativa:

Há apenas 1 contexto de execução global, criado quando nosso programa JavaScript começa a ser executado, enquanto um *novo* o contexto de execução da função é criado em *cada* invocação de função.

NOTA Você deve se lembrar do capítulo 4 que *contexto de função* é o objeto no qual nossa função é invocada, que pode ser acessado por meio do esta palavra-chave. Um contexto de execução, embora tenha um nome semelhante, é uma coisa completamente diferente. É um conceito interno de JavaScript que o mecanismo JavaScript usa para rastrear a execução de nossas funções.

Como mencionamos no capítulo 2, o JavaScript é baseado em um modelo de execução de thread único: apenas uma parte do código pode ser executada por vez. Cada vez que uma função é chamada, o contexto de execução atual deve ser interrompido e um novo contexto de execução da função, no qual o código da função será avaliado, deve ser criado. Depois que a função realiza sua tarefa, o contexto de execução da função geralmente é descartado e o contexto de execução do chamador é restaurado. Portanto, é necessário manter o controle de todos esses contextos de execução - tanto o que está em execução quanto os que estão esperando pacientemente. A maneira mais fácil de fazer isso é usando um *pilha*. Chamou o *pilha de contexto de execução* (ou frequentemente chamado de *pilha de chamadas*).

NOTA Uma pilha é uma estrutura de dados fundamental na qual você pode colocar novos itens apenas no topo e pode pegar os itens existentes apenas no topo. Pense em uma pilha de bandejas em uma cafeteria. Quando você quer pegar um, você escolhe um do topo. E um funcionário do refeitório que tem um novo limpo também o coloca no topo.

Isso pode parecer vago, então vamos dar uma olhada no código a seguir, que relata a atividade de dois ninjas furtivos.

Listagem 5.5 A criação de contextos de execução

<pre>function skulk (ninja) { relatório (ninja + "esquiva"); }</pre>	Uma função que chama outra função
<pre>relatório de função (mensagem) { console.log (mensagem); }</pre>	Uma função que relata uma mensagem por meio da função interna console.log
<pre>skulk ("Kuma"); skulk ("Yoshi");</pre>	Duas chamadas de função do código global

Este código é direto; nós definimos o esconder-se função, que chama o relatório função, que produz uma mensagem. Então, a partir do código global, fazemos duas chamadas separadas para o esconder-se função: skulk ("Kuma") e skulk ("Yoshi"). Usando esse código como base, exploraremos a criação de contextos de execução, conforme mostrado na figura 5.6.

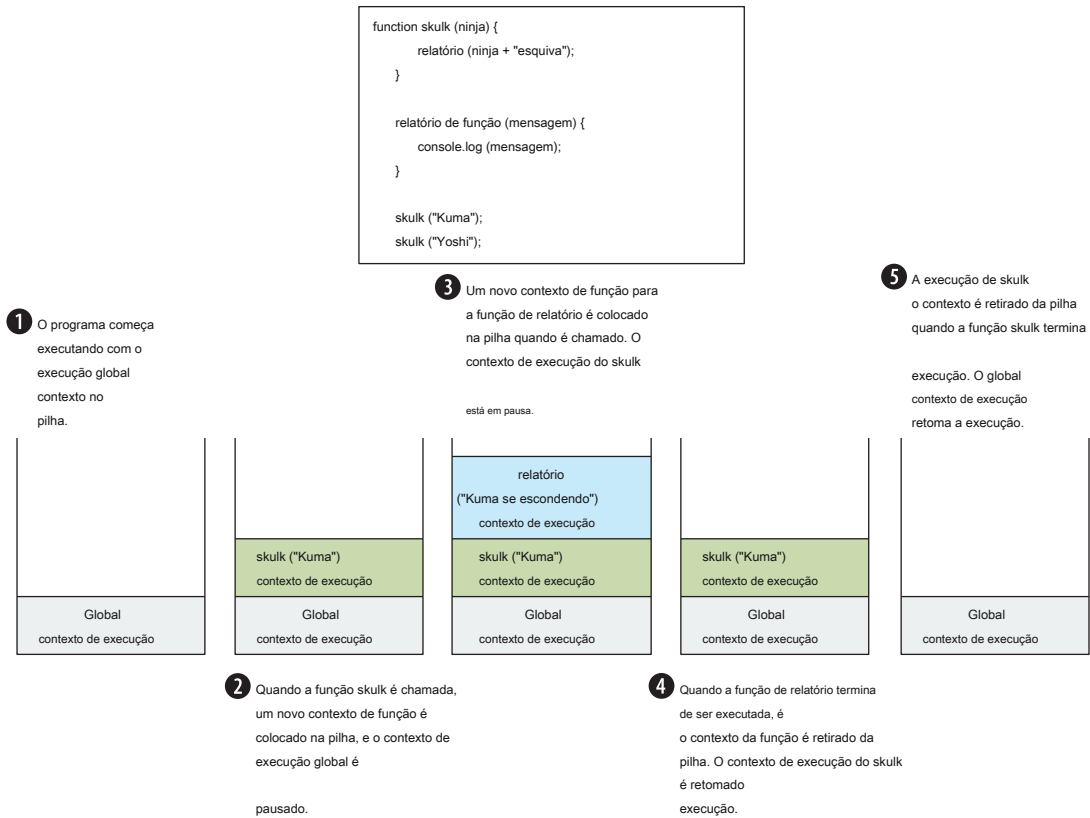


Figura 5.6 O comportamento da pilha de contexto de execução

Ao executar o código de exemplo, o contexto de execução se comporta da seguinte maneira:

- 1 A pilha de contexto de execução começa com o contexto de execução global que é criado apenas uma vez por programa JavaScript (uma vez por página, no caso de páginas da web). O contexto de execução global é o contexto de execução ativo ao executar o código global.
- 2 No código global, o programa primeiro define duas funções: esconder-se e relatório, e então chama o esconder-se funcionar com skulk ("Kuma"). Como apenas uma parte do código pode ser executada de uma vez, o mecanismo JavaScript pausa a execução do código global e vai executar o esconder-se código de função com Kuma como um argumento. Isso é feito criando um *novo* contexto de execução da função e colocá-lo no topo da pilha.
- 3 O esconder-se função, por sua vez, chama o relatório função com o argumento Kuma se escondendo. Novamente, porque apenas um trecho de código pode ser executado por vez, o esconder-se contexto de execução é pausado, e um novo contexto de execução de função para o relatório função, com o argumento Kuma se escondendo, é criado e colocado na pilha.

- 4 Depois de relatório função registra a mensagem usando o integrado console.log função (ver apêndice C) e termina sua execução, temos que voltar ao esconder-se função. Isso é feito abrindo o relatório contexto de execução da função da pilha. O esconder-se contexto de execução da função é então reativado, e a execução do esconder-se a função continua. Algo semelhante acontece quando o esconder-se função termina sua execução: O contexto de execução da
- 5 função do esconder-se a função é removida da pilha e o contexto de execução global, que esperou pacientemente todo esse tempo, é restaurado como o contexto de execução ativo. A execução do código JavaScript global é restaurada.

Todo este processo é repetido de forma semelhante para a segunda chamada para o esconder-se função, agora com o argumento Yoshi. Dois novos contextos de execução de função são criados e empurrado para a pilha, skulk ("Yoshi") e relatório ("Yoshi se escondendo"), quando o respectivas funções são chamadas. Esses contextos de execução também são retirados da pilha quando o programa retorna da função de correspondência.

Mesmo que a pilha de contexto de execução seja um conceito interno de JavaScript, você pode explorá-la em qualquer depurador de JavaScript, onde é referido como um *pilha de chamadas*. A Figura 5.7 mostra a pilha de chamadas no Chrome DevTools.

NOTA O Apêndice C oferece uma visão mais detalhada das ferramentas de depuração disponíveis em vários navegadores.

Além de acompanhar a posição na execução do aplicativo, o contexto de execução é vital em *resolução do identificador*, o processo de descobrir a qual variável um determinado identificador se refere. O contexto de execução faz isso por meio do *ambiente lexical*.

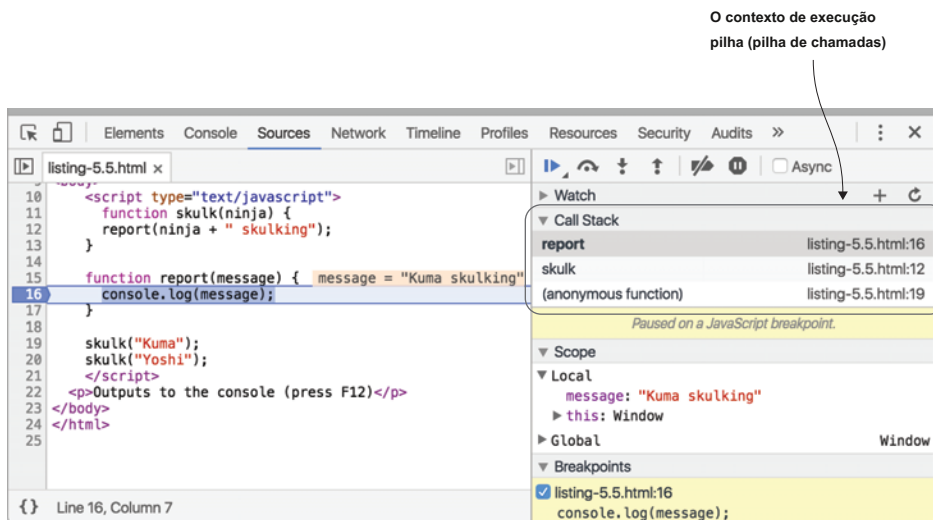


Figura 5.7 O estado atual da pilha de contexto de execução no Chrome DevTools

5,4 Acompanhar identificadores com ambientes lexicais

UMA *ambiente lexical* é uma construção de mecanismo JavaScript interno usada para rastrear o mapeamento de identificadores para variáveis específicas. Por exemplo, considere o seguinte código:

```
var ninja = "Hattori"; console.log (ninja);
```

O ambiente lexical é consultado quando o `ninja` variável é acessada no `console.log` demonstração.

NOTA Ambientes lexicais são uma implementação interna do mecanismo de escopo do JavaScript, e as pessoas costumam se referir a eles como *escopos*.

Normalmente, um ambiente léxico é associado a uma estrutura específica de código JavaScript. Ele pode ser associado a uma função, um bloco de código ou a pegar parte de um tentar pegar demonstração. Cada uma dessas estruturas (funções, blocos e pegar partes) podem ter seus próprios mapeamentos de identificador separados.



NOTA Em versões pré-ES6 de JavaScript, um ambiente léxico pode ser associado apenas a uma função. Variáveis podem ter apenas escopo de função. Isso causou muita confusão. Como o JavaScript é uma linguagem semelhante a C, as pessoas vindas de outras linguagens semelhantes a C (como C ++, C # ou Java) naturalmente esperavam que alguns conceitos de baixo nível, como a existência de escopos de bloco, fossem os mesmos. Com o ES6, isso finalmente foi corrigido.

5.4.1 Aninhamento de código

Ambientes lexicais são fortemente baseados em *aninhamento de código*, que permite que uma estrutura de código seja contida em outra. A Figura 5.8 mostra vários tipos de aninhamento de código.

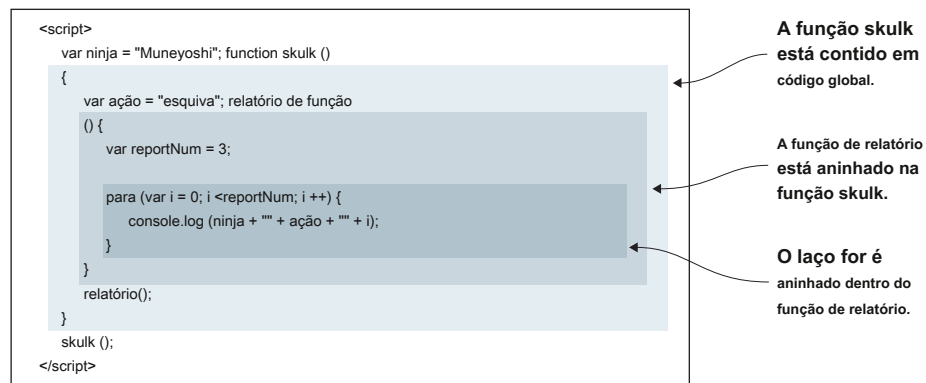


Figura 5.8 Tipos de aninhamento de código

Neste exemplo, podemos ver o seguinte:

- O pra loop está aninhado dentro do relatório função.
- O relatório função está aninhado no esconder-se função.
- O esconder-se função está aninhado no código global.

Em termos de escopos, cada uma dessas estruturas de código obtém um ambiente léxico associado *cada* hora em que o código é avaliado. Por exemplo, em cada invocação do esconder-se função, um novo ambiente léxico de função é criado.

Além disso, é importante enfatizar que uma estrutura de código interna tem acesso às variáveis definidas em estruturas de código externas; por exemplo, o pra loop pode acessar variáveis do relatório função, o esconder-se função e o código global; a relatório função pode acessar variáveis do esconder-se função e o código global; e a esconder-se função pode acessar apenas variáveis adicionais do código global.

Não há nada de especial nessa forma de acessar variáveis; todos nós provavelmente já o fizemos muitas vezes. Mas como o mecanismo JavaScript rastreia todas essas variáveis e o que é acessível de onde? É aqui que entram os ambientes lexicais.

5.4.2 *Aninhamento de código e ambientes lexicais*

Além de manter o controle de variáveis locais, declarações de função e parâmetros de função, cada ambiente léxico deve manter o controle de seus *exterior* (pai) ambiente léxico. Isso é necessário porque temos que ser capazes de acessar variáveis definidas em estruturas de código externas; se um identificador não puder ser encontrado no ambiente atual, o ambiente externo é pesquisado. Isso para quando a variável correspondente é encontrada ou com um erro de referência se alcançamos o ambiente global e não há sinal do identificador procurado. A Figura 5.9 mostra um exemplo; você pode ver como os identificadores introdução, ação, e ninja são resolvidos ao executar o relatório função.

Neste exemplo, o relatório função é chamada pelo esconder-se função, que por sua vez é chamada pelo código global. Cada contexto de execução possui um ambiente léxico associado a ele que contém o mapeamento para todos os identificadores definidos diretamente naquele contexto. Por exemplo, o ambiente global mantém o mapeamento para identificadores ninja e esquivar-se, a esconder-se ambiente mantém o mapeamento para os identificadores ação e relatório, e a relatório ambiente mantém o mapeamento para o introdução identificador (lado direito da figura 5.9).

Em um contexto de execução particular, além de acessar identificadores definidos diretamente no ambiente léxico correspondente, nossos programas frequentemente acessam outras variáveis definidas em ambientes externos. Por exemplo, no corpo do relatório função, acessamos a variável ação do exterior esconder-se função, bem como o global ninja variável. Para fazer isso, precisamos de alguma forma acompanhar esses ambientes externos. JavaScript faz isso aproveitando as funções como objetos de primeira classe.

Sempre que uma função é criada, uma referência ao ambiente léxico no qual a função foi criada é armazenada em uma propriedade interna (o que significa que você não pode acessá-la ou manipulá-la diretamente) chamada `[[ Ambiente]]`; colchetes duplos é a notação que usaremos para marcar essas propriedades internas. No nosso caso, o esconder-se função vai

```
< roteiro >
  var ninja = "Muneyoshi";

  função skulk () {
    var action = "Skulking";

    função relatório() {
      var intro = "Aha!";
      assert (intro === "Aha!", "Local"); assert (action === "Skulking",
        "Outer"); afirmar (ninja === "Muneyoshi", "Global");
    }

    relatório();
  }

  skulk ();
</ roteiro >
```

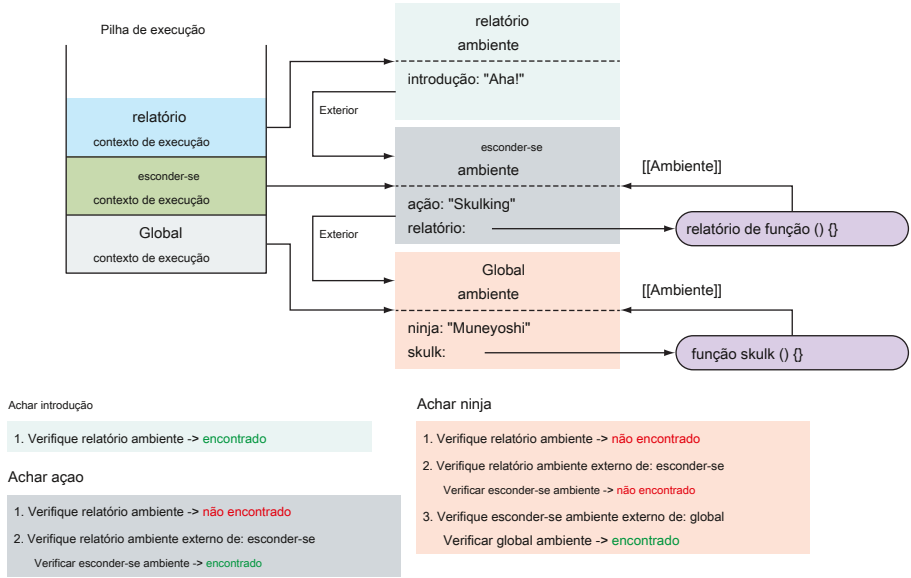


Figura 5.9 Como os mecanismos JavaScript resolvem os valores das variáveis

manter uma referência ao meio ambiente global, e o relatório função manterá uma referência ao esconder-se ambiente, porque esses foram os ambientes em que as funções foram criadas.

NOTA Isso pode parecer estranho à primeira vista. Por que simplesmente não percorremos toda a pilha de contextos de execução e procuramos em seus ambientes correspondentes por mapeamentos de identificadores? Tecnicamente, isso funcionaria em nosso exemplo. Mas lembre-se, uma função JavaScript pode ser passada como qualquer outro objeto, então a posição da definição da função e a posição de onde a função é chamada geralmente não estão relacionadas (lembre-se dos fechamentos).

Sempre que uma função é chamada, um novo contexto de execução de função é criado e colocado na pilha de contexto de execução. Além disso, um novo ambiente léxico associado é criado. Agora vem a parte crucial: para o ambiente externo do ambiente léxico recém-criado, o mecanismo JavaScript coloca o ambiente referenciado pelo interno da função chamada [[Ambiente]] propriedade, o ambiente no qual a função agora chamada foi criada!

No nosso caso, quando o esconder-se função é chamada, o ambiente externo do recém-criado esconder-se ambiente torna-se o ambiente global (porque é o ambiente no qual o esconder-se função foi criada). Da mesma forma, ao chamar o relatório função, o ambiente externo do recém-criado relatório ambiente está definido para o esconder-se ambiente.

Agora vamos dar uma olhada no relatório função:

```
relatório de função () {
  var intro = "Aha!";
  assert (intro === "Aha!", "Local"); assert (action === "Skulking", "Outer");
  afirmar (ação === "Muneyoshi", "Global");
}
```

Quando o primeiro afirmar declaração está sendo avaliada, temos que resolver o introdução identificador. Para fazer isso, o mecanismo JavaScript começa verificando o ambiente do contexto de execução atualmente em execução, o relatório ambiente. Porque o relatório ambiente contém uma referência a introdução, o identificador é resolvido.

Próximo, o segundo afirmar declaração tem que resolver o ação identificador. Novamente, o ambiente do contexto de execução atualmente em execução é verificado. Mas o relatório ambiente não contém uma referência ao ação identificador, portanto, o mecanismo JavaScript deve verificar o ambiente externo do relatório ambiente: o esconder-se ambiente. Felizmente, o esconder-se ambiente contém uma referência ao ação identificador e o identificador é resolvido. Um processo semelhante é seguido ao tentar resolver o ninja identificador (uma pequena dica: o identificador pode ser encontrado no ambiente global).

Agora que você entende os fundamentos da resolução de identificadores, vamos examinar as várias maneiras como uma variável pode ser declarada.

5,5 *Noções básicas sobre tipos de variáveis JavaScript*

Em JavaScript, podemos usar três palavras-chave para definir variáveis: var, deixe, e const. Eles diferem em dois aspectos: *mutabilidade* e sua relação com o ambiente lexical.



NOTA A palavra-chave var faz parte do JavaScript desde o início, enquanto deixe e const são adições ES6. Você pode verificar se o seu navegador suporta deixe e const nos seguintes links:
<http://mng.bz/CGJ6> e <http://mng.bz/uUIT>.

5.5.1 Mutabilidade variável

Se fôssemos dividir palavras-chave de declaração de variável por mutabilidade, colocaríamos `const` de um lado e `var` e `deixei` por outro lado. Todas as variáveis definidas com `const` são imutáveis, o que significa que seu valor pode ser definido apenas uma vez. Por outro lado, variáveis definidas com palavras-chave `var` e `deixei` são variáveis comuns típicas, cujo valor podemos alterar quantas vezes forem necessárias.

Agora, vamos nos aprofundar em como `const` variáveis funcionam e se comportam.

VARIÁVEIS CONST

UMA `const` “Variável” é semelhante a uma variável normal, com a exceção de que temos que fornecer um valor de inicialização quando ela é declarada e não podemos atribuir um valor completamente novo a ela posteriormente. Hmm não muito *variável*, é isso?

`Const` variáveis são frequentemente usadas para dois propósitos ligeiramente diferentes:

- Especificar variáveis que não devem ser reatribuídas (e, no resto do livro, as usaremos principalmente a esse respeito).
- Referenciar um valor fixo, por exemplo, o número máximo de ronins em um esquadrão, `MAX_RONIN_COUNT`, pelo nome, em vez de usar um número literal como 234. Isso torna nossos programas mais fáceis de entender e manter. Nosso código não está cheio de literais aparentemente arbitrários (234), mas com nomes significativos (`MAX_RONIN_COUNT`) cujos valores são especificados em apenas um lugar.

Em qualquer caso, porque `const` as variáveis não devem ser reatribuídas durante a execução do programa, protegemos nosso código contra modificações indesejadas ou acidentais e até possibilitamos que o mecanismo JavaScript faça algumas otimizações de desempenho.

A lista a seguir ilustra o comportamento de `const` variáveis.

Listagem 5.6 O comportamento de `const` variáveis

```
const firstConst = "samurai";
assert (firstConst === "samurai", "firstConst é um samurai");
```

```
tentar{
  firstConst = "ninja";
  fail ("Não deveria estar aqui");
} catch (e) {
  pass ("Ocorreu uma exceção");
}
```

```
assert (firstConst === "samurai",
        "firstConst ainda é um samurai!");
```

```
const secondConst = {};
```

```
secondConst.weapon = "wakizashi";
```

```
assert (secondConst.weapon === "wakizashi", variável secondConst, mas não há nada que pare
        "Podemos adicionar novas propriedades");
```

Define um `const` variável e verifica que o valor era atribuído

Tentando atribuir um novo valor para uma variável `const` lança uma exceção.

← Cria uma nova variável `const` e atribui um novo objeto a ela

← Não podemos atribuir um objeto completamente novo ao `secondConst`, mas não há nada que pare de modificarmos o que já temos.


```
const thirdConst = [];
assert (thirdConst.length === 0, "Nenhum item em nossa matriz");

thirdConst.push("Yoshi");

assert (thirdConst.length === 1, "O array mudou");
```

Exatamente o mesmo
coisa vale para
matrizes.

Aqui, primeiro definimos um `const` variável chamada `firstConst` com um valor `samurai` e teste se a variável foi inicializada, conforme esperado:

```
const firstConst = "samurai";
assert (firstConst === "samurai", "firstConst é um samurai");
```

Continuamos tentando atribuir um valor completamente novo, `ninja`, para nosso `firstConst` variável:

```
tentar{
  firstConst = "ninja";
  fail ("Não deveria estar aqui");
} catch (e) {
  pass ("Ocorreu uma exceção");
}
```

Porque o `firstConst` variável é, bem, uma constante, não podemos atribuir um novo valor a ela, portanto, o mecanismo JavaScript lança uma exceção sem modificar o valor da variável. Observe que estamos usando duas funções que não usamos até agora: `falhou` e `passar`. Esses dois métodos se comportam de forma semelhante ao `afirmar` método, exceto `falhou` sempre falha e `passar` sempre passa. Aqui, nós os usamos para verificar se ocorreu uma exceção: Se ocorrer uma exceção, o `pegar` declaração é ativada e o `passar` método é executado. Se não houver exceção, o `falhou` método é executado e seremos notificados de que algo não está como deveria estar. Podemos verificar para ver-

se acontecer a exceção, conforme mostrado na figura 5.10.

Em seguida, definimos outro `const` variável, desta vez inicializando-a para um objeto vazio:

```
const secondConst = {};
```

Agora vamos discutir um recurso importante do `const` variáveis. Como você já viu, não podemos atribuir um valor completamente novo a um `const` variável. Mas não há nada nos impedindo de *modificando* o atual. Por exemplo, podemos adicionar novas propriedades ao objeto atual:

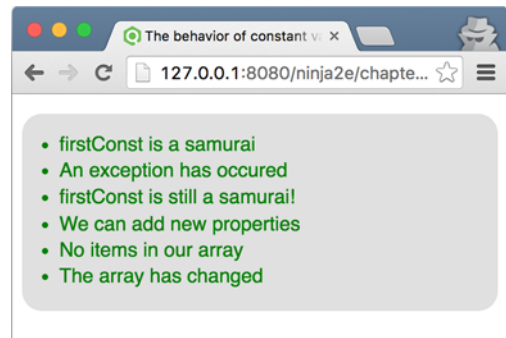


Figura 5.10 Verificando o comportamento de `const` variáveis. Uma exceção ocorre quando tentamos atribuir um valor completamente novo a um `const` variável.

```
secondConst.weapon = "wakizashi";
assert (secondConst.weapon === "wakizashi",
        "Podemos adicionar novas propriedades");
```

Ou, se nosso `const` variável se refere a uma matriz, podemos modificar essa matriz em qualquer grau:

```
const thirdConst = [];
assert (thirdConst.length === 0, "Nenhum item em nossa matriz");

thirdConst.push ("Yoshi");

assert (thirdConst.length === 1, "O array mudou");
```

É só isso. `const` variáveis não são tão complicadas para começar. Você só precisa lembrar que o valor de um `const` variável pode ser definida apenas na inicialização e não podemos atribuir um valor completamente novo posteriormente. Ainda podemos modificar o valor existente; simplesmente não podemos substituí-lo completamente.

Agora que exploramos a mutabilidade de variáveis, vamos considerar os detalhes das relações entre vários tipos de variáveis e ambientes lexicais.

5.5.2 Palavras-chave de definição de variável e ambientes lexicais

Os três tipos de definições de variáveis - `var`, `deixe`, e `const` - também pode ser categorizado por sua relação com o ambiente lexical (em outras palavras, por seu escopo). Nesse caso, podemos colocar `var` de um lado, e `deixe` e `const` no outro.

USANDO A PALAVRA-CHAVE VAR

Quando usamos o `var` palavra-chave, a variável é definida na função mais próxima ou ambiente léxico global. (Observe que os blocos são ignorados!) Este é um detalhe antigo do JavaScript que confundiu muitos desenvolvedores vindos de outras linguagens.

Considere a seguinte lista.

Listagem 5.7 usando o `var` palavra chave

```
var globalNinja = "Yoshi";

function reportActivity () {
    var functionActivity = "pular";

    para (var i = 1; i < 3; i++) {
        var forMessage = globalNinja + "" + functionActivity; assert (forMessage === "Yoshi jumping",
            "Yoshi está pulando dentro do bloco for"); assert (i, "Contador de loop atual:" +
            i);
    }

    assert (i === 3 && forMessage === "Yoshi pulando",
        "Variáveis de loop acessíveis fora do loop");
}

reportActivity ();
```

Define um global variável, usando `var`

Define uma variável local de função, usando `var`

Define dois variáveis no loop `for`, usando `var`

Dentro de `para` loop, nós podemos acessar o bloco variáveis, função variáveis, e variáveis globais - nada surpreendente lá.

Mas as variáveis do loop `for` também estão acessíveis fora do loop `for`.

```
assert (typeof functionActivity === "undefined"
    && typeof i === "undefined" && typeof forMessage === "undefined", "Não podemos ver variáveis de função fora de uma função");
```

Normalmente, nenhuma das variáveis de função são acessíveis fora da função.

Começamos definindo uma variável global, `globalNinja`, que é seguido pela definição de um `reportActivity` função que faz um loop duas vezes e nos notifica sobre a atividade de salto de nosso `globalNinja`. Como você pode ver, dentro do corpo do `pra` loop, podemos normalmente acessar ambas as variáveis de bloco (`eu` e `forMessage`), as variáveis de função (`functionActivity`), e as variáveis globais (`globalNinja`).

Mas o que é estranho com JavaScript, e o que confunde muitos desenvolvedores vindos de outras linguagens, é que podemos acessar as variáveis definidas com blocos de código mesmo fora desses blocos:

```
assert (i === 3 && forMessage === "Yoshi pulando",
    "Variáveis de loop acessíveis fora do loop");
```

Isso decorre do fato de que as variáveis declaradas com a palavra-chave `var` são sempre registrados na função mais próxima ou no ambiente léxico global, sem prestar atenção aos blocos. A Figura 5.11 mostra essa situação, mostrando o estado dos ambientes lexicais após a segunda iteração do `pra` loop no `reportActivity` função.

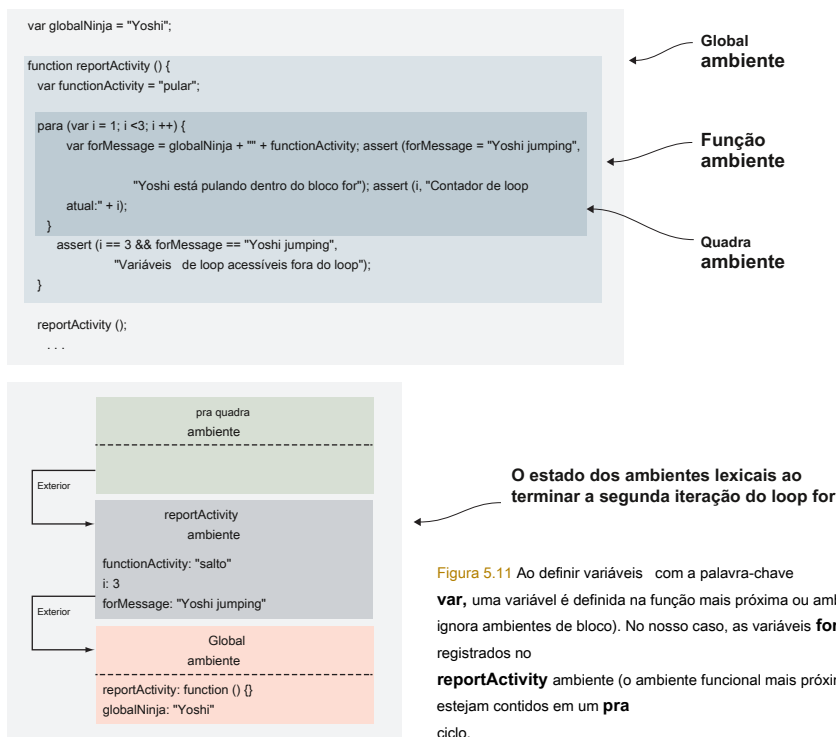


Figura 5.11 Ao definir variáveis com a palavra-chave `var`, uma variável é definida na função mais próxima ou ambiente global (enquanto ignora ambientes de bloco). No nosso caso, as variáveis `forMessage` e `eu` estão registrados no `reportActivity` ambiente (o ambiente funcional mais próximo), mesmo que estejam contidos em um `pra` ciclo.

Aqui temos três ambientes lexicais:

- O ambiente global em que o globalNinja variável é registrada (porque esta é a função mais próxima ou ambiente léxico global) O reportActivity ambiente, criado no reportActivity invocação de função, que contém
- o functionActivity, i, e forMessage variáveis, porque são definidas com a palavra-chave var, e este é o ambiente funcional mais próximo
- O pra ambiente de bloco, que está vazio, porque var- variáveis definidas ignoram blocos (mesmo quando contidos neles)

Como esse comportamento é um pouco estranho, a versão ES6 do JavaScript oferece duas novas palavras-chave de declaração de variável: `deixe` e `const`.

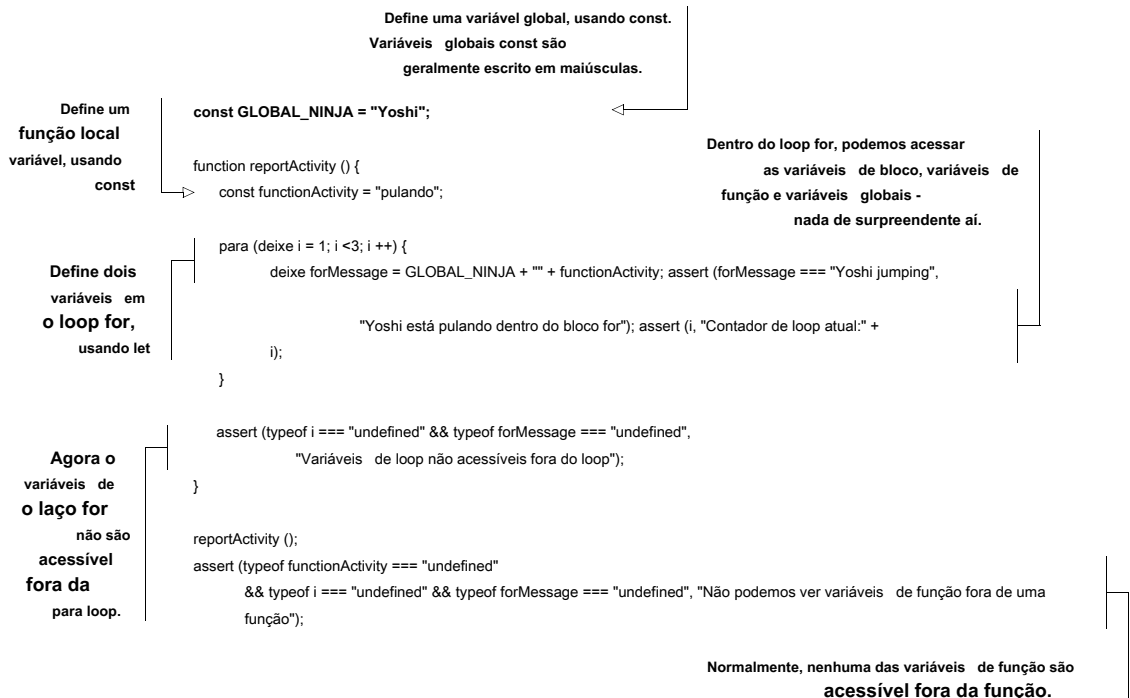
USANDO LET E CONST PARA ESPECIFICAR VARIÁVEIS COM ESCOPO DE BLOCO

diferente `var`, que define a variável na função mais próxima ou ambiente léxico global, o `deixe` e `const` palavras-chave são mais diretas. Eles definem variáveis no ambiente léxico mais próximo (que pode ser um ambiente de bloco, um ambiente de loop, um ambiente de função ou até mesmo o ambiente global). Podemos usar `deixe` e

`const` para definir variáveis com escopo de bloco, escopo de função e escopo global.

Vamos reescrever nosso exemplo anterior para usar `const` e `deixe`.

Listagem 5.8 Usando `const` e `deixe` palavras-chave



A Figura 5.12 ilustra a situação atual, ao finalizar a execução da segunda iteração do `pra` loop no `reportActivity` função. Mais uma vez, temos três ambientes lexicais: o ambiente global (para código global fora de todas as funções e blocos), o `reportActivity` ambiente ligado ao `reportActivity` função e o ambiente de bloco para o `pra` corpo do laço. Mas porque estamos usando `deixe`

e `const` palavras-chave, as variáveis são definidas em seu ambiente lexical mais próximo; a `GLOBAL_NINJA` variável é definida no ambiente global, o `functionActivity` variável no `reportActivity` ambiente, e o `eu` e `forMessage` variáveis no `pra` ambiente de bloco.

Agora isso `const` e `deixe` foram introduzidos, muitos novos desenvolvedores de JavaScript que vieram recentemente de outras linguagens de programação podem ficar em paz. JavaScript

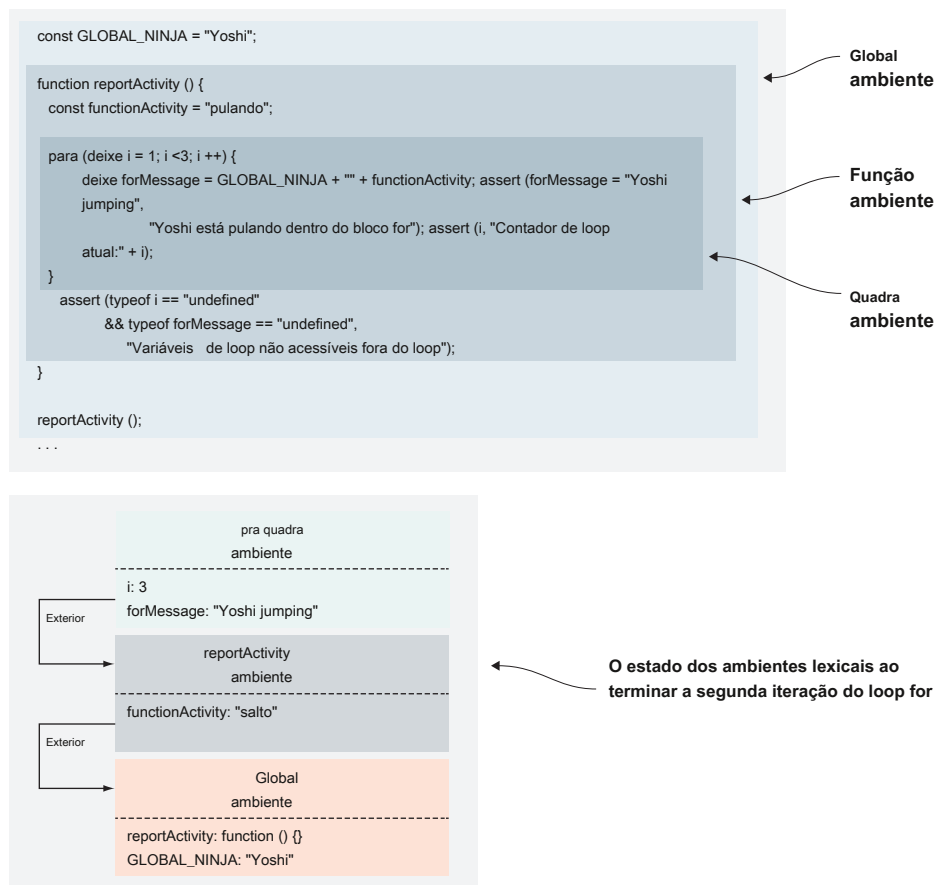


Figura 5.12 Ao definir variáveis com palavras-chave **deixe** e **const**, uma variável é definida no ambiente mais próximo. Em nosso caso, variáveis **forMessage** e **eu** estão registrados no **pra** ambiente de bloco, a variável **functionActivity** no **reportActivity** ambiente, e o

GLOBAL_NINJA variável no ambiente global (em todos os casos, o ambiente mais próximo da respectiva variável).

finalmente suporta as mesmas regras de escopo que outras linguagens semelhantes a C. Por esse motivo, a partir deste ponto deste livro, quase sempre usamos `const` e deixei ao invés de `var`.

Agora que entendemos como os mapeamentos de identificadores são mantidos em ambientes lexicais e como os ambientes lexicais estão vinculados à execução do programa, vamos discutir o processo exato pelo qual os identificadores são definidos em ambientes lexicais. Isso nos ajudará a entender melhor alguns bugs comuns.

5.5.3 Registrando identificadores em ambientes lexicais

Um dos princípios básicos por trás do design de JavaScript como linguagem de programação foi sua facilidade de uso. Esse é um dos principais motivos para não especificar tipos de retorno de função, tipos de parâmetro de função, tipos de variável e assim por diante. E você já sabe que o código JavaScript é executado linha por linha, de maneira direta. Considere o seguinte:

```
firstRonin = "Kiyokawa";  
secondRonin = "Kondo";
```

O valor que `Kiyokawa` é atribuído ao identificador `firstRonin`, e então o valor `Kondo` é atribuído ao identificador `secondRonin`. Não há nada de estranho nisso, certo? Mas dê uma olhada em outro exemplo:

```
const firstRonin = "Kiyokawa"; verificar (firstRonin);  
  
verificação de função (ronin) {  
  assert (ronin === "Kiyokawa", "O ronin foi verificado!");  
}
```

Neste caso, atribuímos o valor `Kiyokawa` para o identificador `firstRonin`, e então chamamos o `Verifica função` com o identificador `firstRonin` como um parâmetro. Mas espere um segundo - se o código for executado linha por linha, devemos ser capazes de chamar o `Verifica função`? A execução do nosso programa não atingiu sua declaração, então o mecanismo JavaScript nem deveria saber disso.

Mas se verificarmos, como mostrado na figura 5.13, você verá que está tudo bem e bem. JavaScript não é muito exigente sobre onde definimos nossas funções. Podemos escolher colocar declarações de função antes ou mesmo depois de suas respectivas chamadas. Isso não é algo que o desenvolvedor precise se preocupar.

O PROCESSO DE REGISTRO DE IDENTIFICADORES

Mas facilidade de uso à parte, se o código é executado linha por linha, como o motor JavaScript sabia que uma função chamada

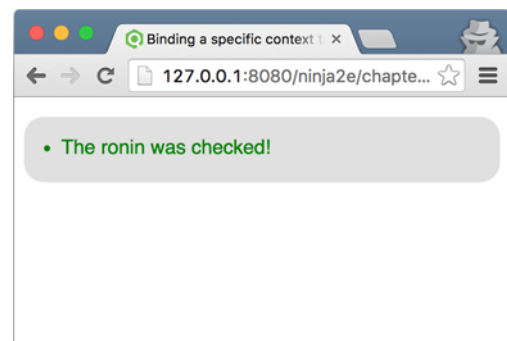


Figura 5.13 A função é de fato visível, mesmo antes que a execução alcance sua definição.

Verifica existe? Acontece que o mecanismo JavaScript “trapaceia” um pouco e que a execução do código JavaScript ocorre em duas fases.

A primeira fase é ativada sempre que um novo ambiente léxico é criado. Nesta fase, o código não é executado, mas o mecanismo JavaScript visita e registra todas as variáveis e funções declaradas no ambiente léxico atual. A segunda fase, a execução do JavaScript, começa depois que isso é realizado; o comportamento exato depende do tipo de variável (let, var, const, declaração de função) e o tipo de ambiente (global, função ou bloco).

O processo é como se segue:

- 1 Se estamos criando um ambiente de função, o implícito argumentos identificador é criado, junto com todos os parâmetros de função formais e seus valores de argumento. Se estivermos lidando com um ambiente sem função, esta etapa será ignorada.
- 2 Se estivermos criando um ambiente global ou de função, o código atual será verificado (sem entrar no corpo de outras funções) em busca de declarações de função (mas não de expressões de função ou funções de seta!). Para cada declaração de função descoberta, uma nova função é criada e associada a um identificador no ambiente com o nome da função. Se esse nome de identificador já existir, seu valor será sobrescrito. Se estivermos lidando com ambientes de bloco, esta etapa será ignorada.
- 3 O código atual é verificado em busca de declarações de variáveis. Em ambientes funcionais e globais, todas as variáveis declaradas com a palavra-chave var e definidas fora de outras funções (mas podem ser colocadas dentro blocos!) são encontrados, e todas as variáveis declaradas com as palavras-chave let e const definidas fora de outras funções e blocos são encontrados. Em ambientes de bloco, o código é verificado apenas para variáveis declaradas com as palavras-chave let e const, diretamente no bloco atual. Para cada variável descoberta, se o identificador não existir no ambiente, o identificador é registrado e seu valor inicializado para Indefinido. Mas se o identificador existir, ele será deixado com seu valor.

Essas etapas estão resumidas na figura 5.14.

Agora examinaremos as implicações dessas regras. Você verá alguns conun-

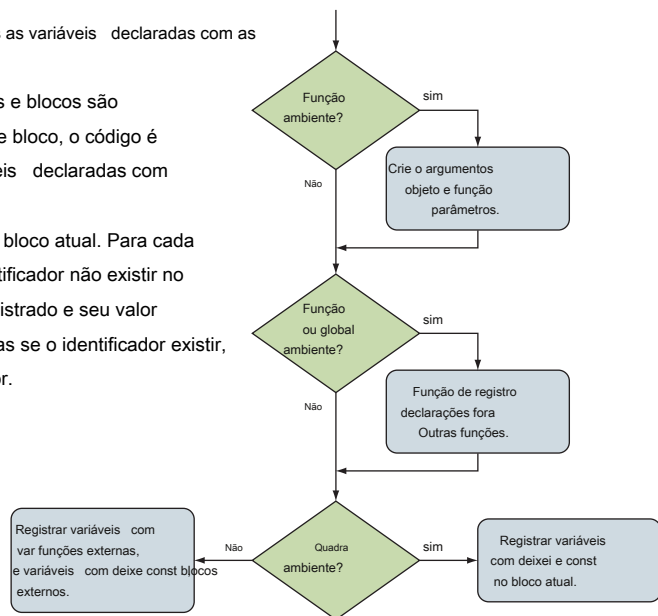


Figura 5.14 O processo de registro de identificadores, dependendo do tipo de ambiente

tambores que podem levar a bugs estranhos que são fáceis de

criar, mas difícil de entender. Vamos começar explicando por que podemos chamar uma função antes mesmo de ela ser declarada.

FUNÇÕES DE CHAMADA ANTES DE SUAS DECLARAÇÕES

Um dos recursos que tornam o JavaScript agradável de usar é que a ordem das definições das funções não importa. Aqueles que usaram Pascal podem não se lembrar com carinho de seus rígidos requisitos estruturais. Em JavaScript, podemos chamar uma função antes mesmo de ela ser declarada formalmente. Confira a lista a seguir.

Listagem 5.9 Acessando uma função antes de sua declaração

```

assert (typeof fun === "função",
    "diversão é uma função, embora sua definição ainda não seja alcançada!");

assert (typeof myFunExpr === "undefined",
    "Mas não podemos acessar expressões de função");

assert (typeof myArrow === "undefined",
    "Nem funções de seta");

função diversão () {}

var myFunExpr = function () {}; var minhaArrow = (x) =>
x;
```

Podemos acessar uma função que ainda não foi definida, se a função for definida como um declaração de função.

Não podemos acessar funções que são definidas como função expressões ou funções de seta.

A função divertida é definida como uma declaração de função.

myFunExpr aponta para uma expressão de função e myArrow para uma função de seta.

Podemos acessar a função Diversão mesmo antes de defini-lo. Podemos fazer isso porque Diversão é definido como uma declaração de função, e a segunda etapa (listada anteriormente nesta seção) indica que as funções criadas com as declarações de função são criadas e seus identificadores registrados quando o ambiente léxico atual é criado, *antes* qualquer código JavaScript é executado. Então, mesmo antes de começarmos a executar nosso afirmar Ligar para Diversão função já existe.

O mecanismo JavaScript faz isso para facilitar as coisas para nós como desenvolvedores, permitindo-nos encaminhar funções de referência e não nos sobrecarregando com uma ordem exata para colocar funções. As funções já existem no momento em que nosso código começa a ser executado.

Observe que isso é válido apenas para declarações de funções. Expressões de função e funções de seta não fazem parte deste processo e são criadas quando a execução do programa atinge suas definições. É por isso que não podemos acessar o myFunExpr e minha flecha funções.

FUNÇÕES DE SUBSTITUIÇÃO

O próximo enigma a ser resolvido é o problema de substituir os identificadores de função. Vamos dar uma olhada em outro exemplo.

Listagem 5.10 Substituindo identificadores de função

```

assert (typeof fun === "função", "Acessamos a função");

var fun = 3;

assert (typeof fun === "number", "Agora vamos acessar o número");

função diversão () {}

assert (typeof fun === "número", "Ainda um número");

```

← Define uma variável divertida e atribui um número a ela

← diversão se refere a uma função.

← Uma declaração de função divertida

← diversão se refere a um número.

← diversão ainda se refere a um número.

Neste exemplo, uma declaração de variável e uma declaração de função têm o mesmo nome: Diversão. Se você executar este código, verá que ambos afirmam passar. Em primeiro lugar, o identificador Diversão refere-se a uma função; e no segundo e terceiro, Diversão refere-se a um número.

Esse comportamento é uma consequência direta das etapas executadas ao registrar os identificadores. Na segunda etapa do processo delineado, funções definidas com declarações de função são criadas e associadas a seus identificadores antes de qualquer código ser avaliado; e na terceira etapa, as declarações de variáveis são processadas, e o valor Indefinido está associado a identificadores que ainda não foram encontrados no ambiente atual.

Neste caso, porque o identificador Diversão foi encontrado na segunda etapa, quando as declarações de função são registradas, o valor Indefinido não é atribuído à variável Diversão. É por isso que a primeira afirmação, testando se Diversão é uma função, passa. Depois disso, temos uma declaração de atribuição, `var fun = 3`, que atribui o número 3 ao identificador Diversão. Ao fazer isso, perdemos a referência à função e, a partir de então, o identificador Diversão refere-se a um número.

Durante a execução do programa real, as declarações de função são ignoradas, de modo que a definição do Diversão função não tem nenhum impacto sobre o valor do Diversão identificador.

Levantação variável

Se você leu vários blogs ou livros sobre JavaScript explicando a resolução de identificadores, provavelmente encontrou o termo *elevação* — Por exemplo, declarações de variáveis e funções são *levantadas*, para o topo de uma função ou escopo global.

Como você viu, porém, essa é uma visão simplista. Variáveis e declarações de funções tecnicamente não são “movidas” para lugar nenhum. Eles são visitados e registrados em ambientes lexicais antes de qualquer código ser executado. Embora *elevação*, como geralmente é definido, é o suficiente para fornecer uma compreensão básica de como o escopo do JavaScript funciona. Nós fomos muito mais fundo do que isso ao observar os ambientes lexicais, dando mais um passo no caminho para nos tornarmos um verdadeiro ninja do JavaScript.

Na próxima seção, todos os conceitos que exploramos até agora neste capítulo o ajudarão a entender melhor os encerramentos.

5,6 Explorando como funcionam os fechamentos

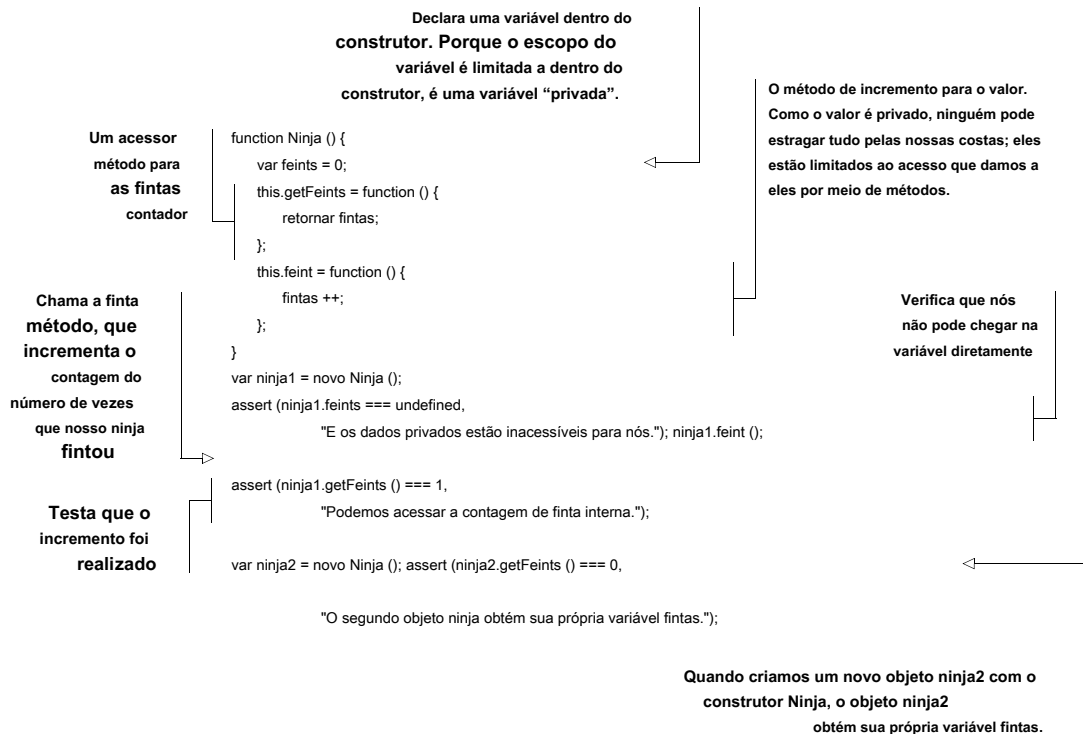
Começamos este capítulo com closures, um mecanismo que permite a uma função acessar todas as variáveis que estão no escopo quando a própria função é criada. Você também viu algumas maneiras pelas quais os encerramentos podem ajudá-lo - por exemplo, permitindo-nos imitar variáveis de objeto privado ou tornando nosso código mais elegante ao lidar com retornos de chamada.

Os fechamentos são irrevogavelmente acoplados aos escopos. Os fechamentos são um efeito colateral direto da maneira como as regras de escopo funcionam em JavaScript. Portanto, nesta seção, revisitaremos os exemplos de fechamento do início do capítulo. Mas desta vez, você aproveitará os contextos de execução e ambientes lexicais que permitirão que você entenda como os fechamentos funcionam nos bastidores.

5.6.1 Revisitando variáveis privadas imitando com fechamentos

Como você já viu, os fechamentos podem nos ajudar a imitar as variáveis privadas. Agora que temos um entendimento sólido de como as regras de escopo funcionam em JavaScript, vamos rever o exemplo de variáveis privadas. Desta vez, vamos nos concentrar em contextos de execução e ambientes lexicais. Só para facilitar, vamos repetir a listagem.

Listagem 5.11 Aproximar variáveis privadas com fechamentos



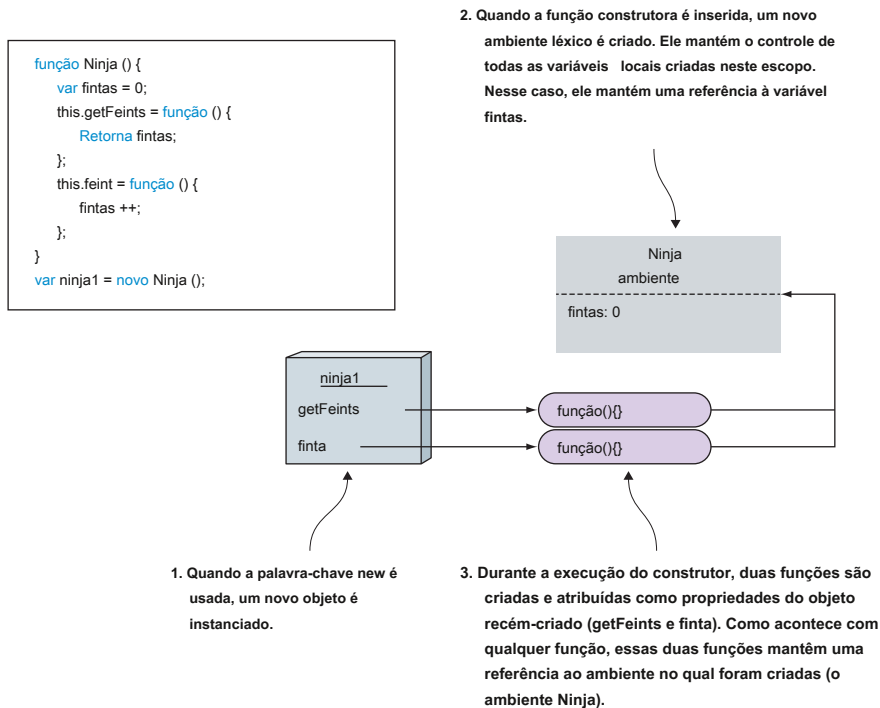


Figura 5.15 Variáveis privadas são realizadas como fechamentos que são criados por métodos de objeto definidos no construtor.

Agora vamos analisar o estado do aplicativo após o primeiro `Ninja` objeto foi criado, conforme mostrado na figura 5.15. Podemos usar nosso conhecimento das complexidades da resolução de identificadores para entender melhor como os fechamentos entram em jogo nessa situação. Construtores de JavaScript são funções invocadas com a palavra-chave `novo`. Portanto, *cada*

vez que invocamos uma função construtora, criamos um *novo* ambiente léxico, que controla as variáveis locais do construtor. Neste exemplo, um novo `Ninja` ambiente que mantém o controle do `fintas` variável é criada.

Além disso, sempre que uma função é criada, ela mantém uma referência ao ambiente léxico no qual foi criada (por meio de um `[[ Ambiente]]` propriedade). Neste caso, dentro do `Ninja` função construtora, criamos duas novas funções: `getFeints` e `finta`, que obtêm uma referência ao `Ninja` ambiente, porque este é o ambiente em que foram criados.

O `getFeints` e `finta` funções são atribuídas como métodos do recém-criado `ninja` objeto (que, se você se lembra do capítulo anterior, é acessível através do esta palavra-chave). Portanto, `getFeints` e `finta` estará acessível de fora do `Ninja` função construtora, que por sua vez leva ao fato de que você efetivamente criou um fechamento em torno do `fintas` variável.

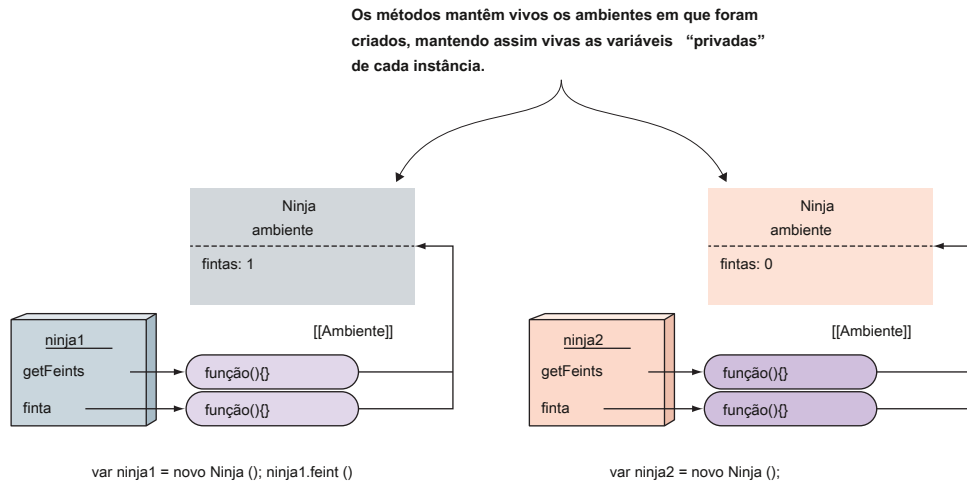


Figura 5.16 Os métodos de cada instância criam fechamentos em torno das variáveis de instância "privadas".

Quando criamos outro Ninja objeto, o ninja2 objeto, todo o processo é repetido. A Figura 5.16 mostra o estado do aplicativo após a criação do segundo

Ninja objeto.

Cada objeto criado com o Ninja construtor obtém seus próprios métodos (o `ninja1.getFeints` método é diferente do `ninja2.getFeints` método) que fecha em torno das variáveis definidas quando o construtor foi chamado. Essas variáveis "privadas" são acessíveis apenas por meio de métodos de objeto criados dentro do construtor, e não diretamente!

Agora vamos ver como as coisas acontecem ao fazer o `ninja2.getFeints ()` chamar. A Figura 5.17 mostra os detalhes.

Antes de fazer o `ninja2.getFeints ()` chamada, nosso mecanismo JavaScript está executando o código global. A execução do nosso programa está no contexto de execução global, que também é o único contexto na pilha de execução. Ao mesmo tempo, o único ambiente léxico ativo é o ambiente global, o ambiente associado ao contexto de execução global.

Ao fazer o `ninja2.getFeints ()` ligar, estamos ligando para o `getFeints` método do `ninja2` objeto. Porque cada chamada de função causa a criação de um novo contexto de execução, um novo `getFeints` o contexto de execução é criado e enviado para a pilha de execução. Isso também leva à criação de um novo `getFeints` Ambiente léxico, que normalmente é usado para rastrear variáveis definidas nesta função. Além disso, o `getFeints` ambiente léxico, como seu ambiente externo, obtém o ambiente no qual o `getFeints` função foi criada, o `Ninja` ambiente que estava ativo quando o `ninja2` objeto foi construído.

```

função Ninja () {
  var fintas = 0;
  this.getFeints = função () {
    Retorna fintas;
  };
  this.feint = função () {
    fintas ++;
  };
}

var ninja1 = novo Ninja ();
var ninja1.feint ();

var ninja2 = novo Ninja (); ninja2.getFeints
();

```

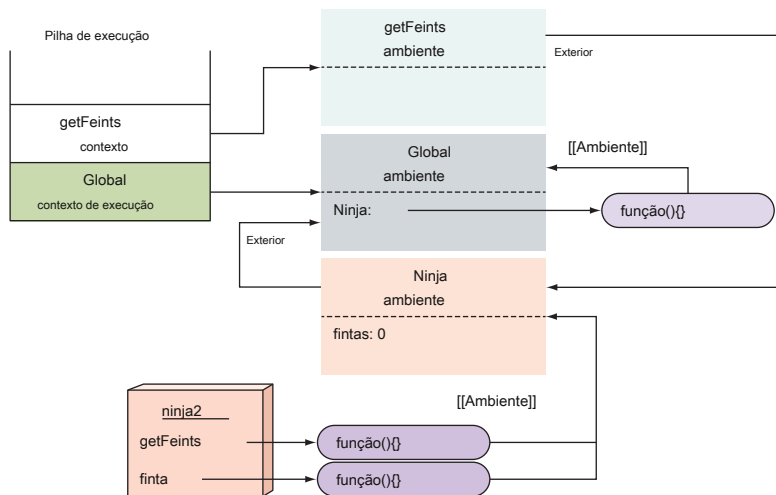


Figura 5.17 O estado dos contextos de execução e ambientes lexicais ao realizar o `ninja2.getFeints ()` chamar. Uma nova `getFeints` ambiente é criado que tem o ambiente da função de construtor em que `ninja2` foi criado como seu ambiente externo. `getFeints` pode acessar o "privado" `fintas` variável.

Agora vamos ver como as coisas acontecem quando tentamos obter o valor do `fintas` variável. Primeiro, o atualmente ativo `getFeints` ambiente léxico é consultado. Porque não definimos nenhuma variável no `getFeints` função, este ambiente léxico está vazio e nosso alvo `fintas` variável não será encontrada lá. Em seguida, a pesquisa continua no `exterior` ambiente do ambiente léxico atual - no nosso caso, o `Ninja` ambiente é ativo durante a construção do `ninja2` objeto. Desta vez, o `Ninja` ambiente tem uma referência ao `fintas` variável, e a pesquisa é feita. É simples assim.

Agora que entendemos o papel que os contextos de execução e ambientes lexicais desempenham ao lidar com fechamentos, gostaríamos de voltar nossa atenção para variáveis "privadas"

e por que continuamos colocando aspas entre eles. Como você já deve ter percebido, essas variáveis “privadas” não são propriedades privadas do objeto, mas são variáveis mantidas vivas pelos métodos de objeto criados no construtor. Vamos dar uma olhada em um efeito colateral interessante disso.

5.6.2 Advertência sobre variáveis privadas

Em JavaScript, não há nada que nos impeça de atribuir propriedades criadas em um objeto para outro objeto. Por exemplo, podemos facilmente reescrever o código da listagem 5.11 em algo como o seguinte.

Listagem 5.12 Variáveis privadas são acessadas por meio de funções, não por meio de objetos!

```
function Ninja () {
  var feints = 0;
  this.getFeints = function () {
    retornar fintas;
  };
  this.feint = function () {
    fintas++;
  };
}
var ninja1 = novo Ninja ();
ninja1.feint ();

var impostor = {};
imposter.getFeints = ninja1.getFeints;

assert (imposter.getFeints () === 1,
        "O impostor tem acesso à variável fintas!");
```

Torna a função getFeints de ninja1 acessível por meio do impostor

Verifica se podemos acessar o suposto variável privada de ninja1

Esta lista modifica o código-fonte de uma forma que atribui o `ninja1.getFeints` método para um completamente novo impostor objeto. Então, quando chamamos o `getFeints` funcionar no impostor objeto, testamos se podemos acessar o valor da variável `fintas` criado quando `ninja1` foi instanciado, provando assim que estamos falsificando toda essa coisa de variável “privada”. Veja a figura 5.18.

Este exemplo ilustra que não existem variáveis de objeto privado em JavaScript, mas que podemos usar closures criados por métodos de objeto para ter uma alternativa “boa o suficiente”. Ainda assim, embora não seja a coisa real, muitos desenvolvedores acham útil essa forma de ocultar informações.

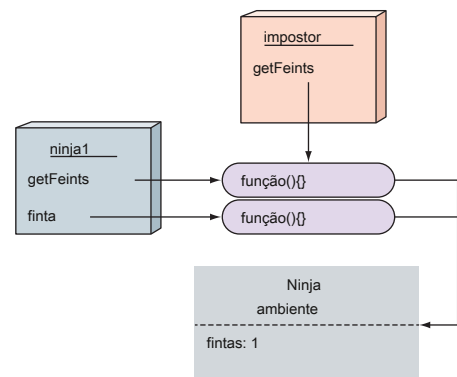


Figura 5.18 Podemos acessar as variáveis “privadas” por meio de funções, mesmo que essa função esteja anexada a outro objeto!

5.6.3 Revisitando o exemplo de encerramentos e retornos de chamada

Vamos voltar ao nosso exemplo de animações simples com cronômetros de retorno de chamada. Desta vez, vamos animar dois objetos, conforme mostrado na lista a seguir.

Listagem 5.13 Usando um fechamento em um cronômetro intervalo de retorno

```
<div id = "box1"> Primeira caixa </div>
<div id = "box2"> Segunda caixa </div>
<script>
  function animatelt (elementId) {
    var elem = document.getElementById (elementId); var tick = 0;

    var timer = setInterval (function () {
      if (assinale <100) {
        elem.style.left = elem.style.top = tick + "px"; tick ++;

      }
      outro {
        clearInterval (temporizador);
        assert (tick === 100,
          "Assinale acessado por meio de um fechamento."); afirmar
        (elem,
          "Elemento também acessado por meio de um fechamento."); assert
        (cronômetro,
          "Referência do temporizador também obtida por meio de um fechamento." );
      }
    }, 10);
  }
  animatelt ("box1");
  animatelt ("box2");
</script>
```

Como você viu no início do capítulo, usamos fechamentos para simplificar a animação de vários objetos em nossas páginas. Mas agora vamos considerar os ambientes lexicais, como mostrado na figura 5.19.

Cada vez que ligamos para o `animatelt` função, um novo ambiente léxico de função é criado **AC** que mantém o controle do conjunto de variáveis importantes para a animação (o `elementId`; `elem`, o elemento que está sendo animado; `marcação`, o número atual de carrapatos; e `cronômetro`, o ID do cronômetro que faz a animação). Esse ambiente será mantido ativo enquanto houver pelo menos uma função que trabalhe com suas variáveis por meio de fechamentos. Neste caso, o navegador manterá vivo o `setInterval` callback até que chamemos o `clearInterval` função. Mais tarde, quando um intervalo expira, o navegador chama o retorno de chamada correspondente - e com ele, por meio dos fechamentos, vêm as variáveis definidas quando o retorno de chamada foi criado. Isso nos permite evitar o problema de

mapeando manualmente o retorno de chamada e as variáveis ativas **DEF**, assim, simplificando significativamente nosso código.

Isso é tudo o que temos a dizer sobre encerramentos e escopos. Agora recapitule este capítulo e nos encontre no próximo, onde exploraremos dois conceitos ES6 completamente novos, geradores e promessas, que podem ajudar ao escrever código assíncrono.

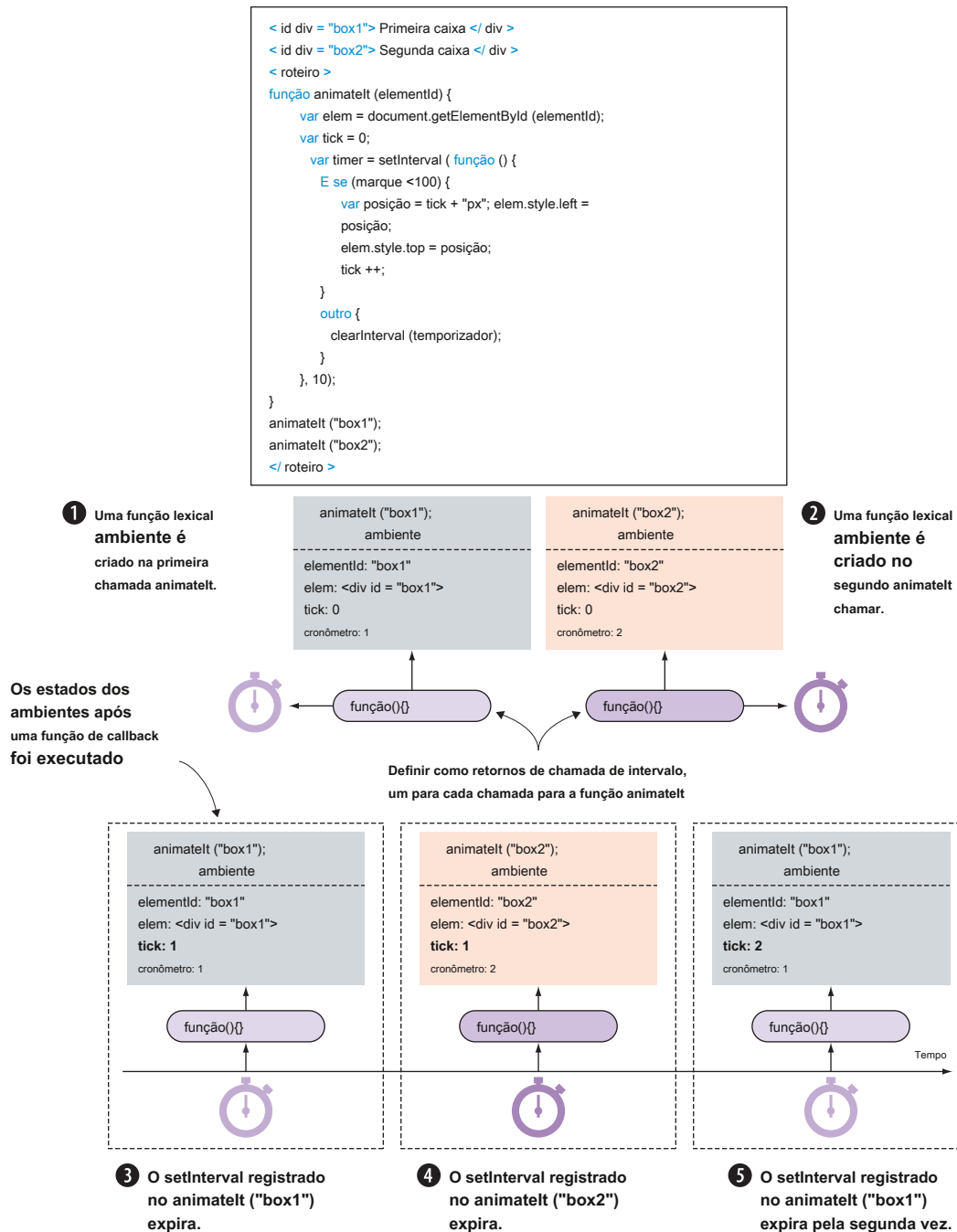


Figura 5.19 Ao criar vários encerramentos, fazemos muitas coisas ao mesmo tempo. Cada intervalo de tempo expira, a função de retorno de chamada reativa o ambiente que estava ativo no momento da criação do retorno de chamada. O encerramento de cada retorno de chamada controla automaticamente seu próprio conjunto de variáveis.

5,7 *Resumo*

- Os fechamentos permitem que uma função acesse todas as variáveis que estão no escopo quando a própria função foi definida. Eles criam uma “bolha de segurança” da função e das variáveis que estão no escopo no ponto de definição da função. Dessa forma, a função tem tudo o que precisa para ser executada, mesmo que o escopo em que a função foi criada já tenha se esgotado.
- Podemos usar encerramentos de função para esses usos avançados:
 - Imitar variáveis de objeto privado, fechando sobre variáveis de construtor por meio de encerramentos de método
 - Lidar com retornos de chamada, de uma forma que simplifica significativamente nosso código
- Os mecanismos JavaScript rastreiam a execução da função por meio de uma pilha de contexto de execução (ou uma pilha de chamadas). Cada vez que uma função é chamada, um novo contexto de execução de função é criado e colocado na pilha. Quando a execução de uma função termina, o contexto de execução correspondente é retirado da pilha.
- Os mecanismos JavaScript rastreiam identificadores com ambientes lexicais (ou coloquialmente, escopos).
- Em JavaScript, podemos definir variáveis com escopo global, com escopo de função e com escopo de bloco par.
- Para definir variáveis, usamos `var`, `deixe`, e `const` palavras-chave:
 - O `var` palavra-chave define uma variável na função mais próxima ou escopo global (enquanto ignora os blocos).
 - `deixe` e `const` palavras-chave definem uma variável no escopo mais próximo (incluindo blocos), permitindo-nos criar variáveis com escopo de bloco, algo que não era possível no JavaScript pré-ES6. Além disso, a palavra-chave `const` nos permite definir “variáveis” cujo valor pode ser atribuído apenas uma vez.
- Os fechamentos são apenas um efeito colateral das regras de escopo do JavaScript. Uma função pode ser chamada mesmo quando o escopo no qual ela foi criada já não existe mais.

5,8 *Exercícios*

- 1 Fechamentos permitem funções para
 - `uma` Acesse variáveis externas que estão no escopo quando a função é definida
 - `b` Acesse variáveis externas que estão no escopo quando a função é chamada
- 2 Fechamentos vêm com
 - `uma` Custos de tamanho de código
 - `b` Custos de memória
 - `c` Custos de processamento
- 3 No exemplo de código a seguir, marque os identificadores acessados por meio de fechamentos:


```
function Samurai (nome) {
  var arma = "katana";
```

```

    this.getWeapon = function () {
        arma de retorno;
    };

    this.getName = function () {
        nome de retorno;
    }

    esta.mensagem = nome + "empunhando uma" + arma;

    this.getMessage = function () {
        return this.message;
    }
}

var samurai = novo Samurai ("Hattori");

samurai.getWeapon ();
samurai.getName ();
samurai.getMessage ();

```

- 4 No código a seguir, quantos contextos de execução são criados e qual é o maior tamanho da pilha de contexto de execução?

```

função executar (ninja) {
    sneak (ninja);
    infiltrar (ninja);
}

função sneak (ninja) {
    retornar ninja + "esquiva";
}

função infiltrar (ninja) {
    retornar ninja + "infiltrando";
}

perforar ("Kuma");

```

- 5 Qual palavra-chave em JavaScript nos permite definir variáveis que não podem ser reassinadas com um valor completamente novo?
- 6 Qual é a diferença entre var e let?
- 7 Onde e por que o código a seguir lançará uma exceção?

```

getNinja ();
getSamurai ();

function getNinja () {
    retornar "Yoshi";
}

var getSamurai = () => "Hattori";

```

6

Funções para o futuro: geradores e promessas

Este capítulo cobre

- Continuando a execução da função com geradores
- Lidando com tarefas assíncronas com promessas
- Alcançar código assíncrono elegante combinando geradores e promessas

Nos três capítulos anteriores, nos concentramos nas funções, especificamente em como definir funções e como usá-las com ótimo efeito. Embora tenhamos introduzido alguns recursos do ES6, como funções de seta e escopos de bloco, estivemos explorando principalmente os recursos que fazem parte do JavaScript há algum tempo. Este capítulo aborda a vanguarda do ES6 apresentando *geradores* e *promessas*, dois recursos de JavaScript completamente novos.



NOTA Geradores e promessas são introduzidos pelo ES6. Você pode verificar o suporte do navegador atual em <http://mng.bz/sOs4> e <http://mng.bz/Du38>.

Geradores são um tipo especial de função. Enquanto uma função padrão produz no máximo um único valor enquanto executa seu código do início ao fim, os geradores produzem vários valores, por solicitação, enquanto suspendem sua execução entre essas solicitações. Embora novos em JavaScript, os geradores já existem há algum tempo em Python, PHP e C #.

Os geradores são frequentemente considerados um recurso de linguagem quase estranho ou marginal que não costuma ser usado pelo programador médio. Embora a maioria dos exemplos deste capítulo sejam projetados para ensinar como funcionam as funções do gerador, também exploraremos alguns aspectos práticos dos geradores. Você verá como usar geradores para simplificar loops complicados e como tirar proveito da capacidade dos geradores de suspender e retomar sua execução, o que pode ajudá-lo a escrever um código assíncrono mais simples e elegante.

Promessas, por outro lado, são um novo tipo de objeto integrado que o ajuda a trabalhar com código assíncrono. Uma promessa é um espaço reservado para um valor que ainda não temos, mas que teremos em algum momento posterior. Eles são especialmente bons para trabalhar com várias etapas assíncronas.

Neste capítulo, você verá como funcionam os geradores e as promessas, e terminaremos explorando como combiná-los para simplificar muito nossas negociações com o código assíncrono. Mas antes de entrar em detalhes, vamos dar uma espiada em como nosso código assíncrono pode ser muito mais elegante.

.....

Quais são alguns usos comuns para uma função de gerador? Por que as promessas são melhores do que simples retornos de chamada para asyn-
código crônico?

Você sabe? Você inicia uma série de tarefas de longa duração com Promise
. *raça*. Quando a promessa se resolve? Quando isso não resolveria?

.....

6,1 *Tornando nosso código assíncrono elegante com geradores e promessas*

Imagine que você seja um desenvolvedor trabalhando no freelanceninja.com, um popular site de recrutamento de ninjas freelance que permite aos clientes contratar ninjas para missões furtivas. Sua tarefa é implementar uma funcionalidade que permita aos usuários obter detalhes sobre a missão mais bem avaliada realizada pelo ninja mais popular. Os dados que representam os ninjas, os resumos de suas missões, bem como os detalhes das missões são armazenados em um servidor remoto, codificado em JSON. Você pode escrever algo assim:

```
tentar {
  var ninjas = syncGetJSON ("ninjas.json");
  var missions = syncGetJSON (ninjas [0] .missionsUrl);
  var missionDetails = syncGetJSON (missões [0] .detailsUrl); // Estude a descrição da missão
}

pegar (e) {
  // Oh não, não conseguimos obter os detalhes da missão
}
```

Este código é relativamente fácil de entender e, se ocorrer um erro em qualquer uma das etapas, podemos detectá-lo facilmente no pegar quadra. Mas, infelizmente, esse código tem um grande problema. Obter dados de um servidor é uma operação de longa duração e, como o JavaScript depende de um modelo de execução de thread único, apenas bloqueamos nossa IU até que as operações de longa duração terminem. Isso leva a aplicativos que não respondem e usuários decepcionados. Para resolver esse problema, podemos reescrevê-lo com retornos de chamada, que serão invocados quando uma tarefa terminar, sem bloquear a IU:

```
getJSON("ninjas.json", function (err, ninjas) {
  if (err) {
    console.log("Erro ao buscar lista de ninjas", err); Retorna;
  }
  getJSON(ninjas[0].missionsUrl, function (err, missões) {
    if (err) {
      console.log("Erro ao localizar missões ninja", err); Retorna;
    }
    getJSON(missions[0].detailsUrl, function (err, missionDetails) {
      if (err) {
        console.log("Erro ao localizar detalhes da missão", err); Retorna;
      }
      // Estude o plano de inteligência));
    });
  });
});
```

Embora esse código seja muito melhor recebido por nossos usuários, você provavelmente concordará que é confuso, adiciona muito código clichê de tratamento de erros e é totalmente feio. É aqui que os geradores e as promessas entram em ação. Ao combiná-los, podemos transformar o código de callback não bloqueador, mas estranho, em algo muito mais elegante:

Uma função geradora é definida colocando um asterisco logo após a palavra-chave da função. Podemos usar a nova palavra-chave yield em funções geradoras.

As promessas estão ocultas no método getJSON.

```
assíncrono (função * () {
  tentar {
    const ninjas = rendimento getJSON("ninjas.json");
    missões const = rendimento getJSON(ninjas[0].missionsUrl);
    const missionDescription = rendimento getJSON(missões[0].detailsUrl); // Estude os detalhes da missão
  }
  pegar (e) {
    // Oh não, não conseguimos obter os detalhes da missão
  }
});
```

Não se preocupe se este exemplo não fizer muito sentido ou se você encontrar alguma sintaxe (como função* ou produzem) desconhecido. No final deste capítulo, você conhecerá todos

os ingredientes necessários. Por enquanto, é suficiente que você compare a elegância (ou a falta dela) do código de retorno de chamada não bloqueador e os geradores não bloqueadores e código de promessas.

Vamos começar devagar explorando as funções do gerador, o primeiro trampolim para um código assíncrono elegante.

6,2 *Trabalhando com funções do gerador*

Os geradores são um tipo de função completamente novo e são significativamente diferentes das funções padrão comuns. Uma *gerador* é uma função que gera uma sequência de valores, mas não todos de uma vez, como faria uma função padrão, mas por solicitação. Temos que pedir explicitamente ao gerador um novo valor, e o gerador responderá com um valor ou nos notificará de que não há mais valores para produzir. O que é ainda mais curioso é que depois que um valor é produzido, uma função geradora não termina sua execução, como faria uma função normal. Em vez disso, um gerador está simplesmente suspenso. Então, quando uma solicitação de outro valor chega, o gerador continua de onde parou.

A lista a seguir fornece um exemplo simples de uso de um gerador para gerar uma sequência de armas.

Listagem 6.1 Usando uma função geradora para gerar uma sequência de valores

```
função* WeaponGenerator () {  
  produzem " Katana ";  
  produzem " Wakizashi ";  
  produzem " Kusarigama ";  
}
```

Define uma função geradora colocando * após a palavra-chave da função

Gera valores individuais usando a nova palavra-chave de rendimento

```
para (deixar arma de WeaponGenerator ()) {  
  afirmar (arma! == indefinido, arma);  
}
```

Consome o gerado sequência com o novo loop for-of

Começamos definindo um gerador que produzirá uma sequência de armas. Criar uma função geradora é simples: acrescentamos um asterisco (*) após o função palavra-chave. Isso nos permite usar o novo `produzem` palavra-chave dentro do corpo do gerador para produzir valores individuais. A Figura 6.1 ilustra a sintaxe.

Neste exemplo, criamos um gerador chamado `WeaponGenerator` que produz uma sequência de armas: `Katana`, `Wakizashi`, e `Kusarigama`. Uma maneira de consumir essa sequência de armas é usar um novo tipo de loop, o `para` de ciclo:

```
para (deixar arma de WeaponGenerator ()) {  
  afirmar (arma, arma);  
}
```

O resultado de invocar este `para` de loop é mostrado na figura 6.2. (Por enquanto, não se preocupe muito com o `para` de loop, como iremos explorar mais tarde.)

Coloque um * após a palavra-chave de função para definir uma função do gerador.

```
função* WeaponGenerator () {
  ...
  produzem "Katana";
  ...
}
```

Dentro das funções do gerador, use o rendimento para produzir valores individuais.

Figura 6.1 Adicione um asterisco (*) após a palavra-chave de função para definir um gerador.

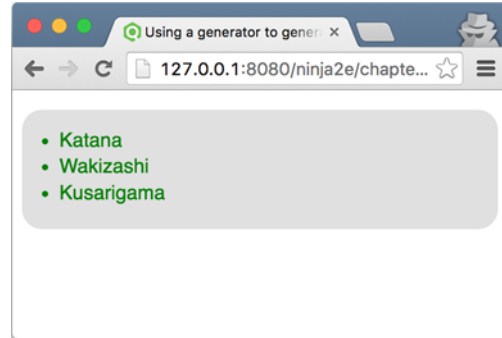


Figura 6.2 O resultado da iteração em nosso **WeaponGenerator**

No lado direito do para de loop, colocamos o resultado de invocar nosso gerador. Mas se você der uma olhada mais de perto no corpo do WeaponGenerator função, você verá que não há Retorna demonstração. O que há com isso? Neste caso, o lado direito do loop for-of avaliar para Indefinido, como seria o caso se estivéssemos lidando com uma função padrão?

A verdade é que os geradores são bastante diferentes das funções padrão. Para começar, chamar um gerador não executa a função do gerador; em vez disso, ele cria um objeto chamado de *iterador*. Vamos explorar esse objeto.

6.2.1 Controlar o gerador por meio do objeto iterador

Fazer uma chamada para um gerador não significa que o corpo da função do gerador será executado. Em vez disso, um objeto iterador é criado, um objeto por meio do qual podemos nos comunicar com o gerador. Por exemplo, podemos usar o iterador para solicitar valores adicionais. Vamos ajustar nosso exemplo anterior para explorar como o objeto iterador funciona.

Listagem 6.2 Controlando um gerador por meio de um objeto iterador

```

Define um gerador que irá produzir
uma sequência de duas armas

função* WeaponGenerator () {
  produzem "Katana ";
  produzem "Wakizashi ";
}

Chamando o
próximo iterador
pedidos de método
um novo valor de
o gerador.

const weaponiterador = WeaponGenerator ();

const result1 = weaponiterador.next ();
assert (typeof result1 === "objeto"
  && result1.value === "Katana" &&! result1.done,
  "Katana recebeu!");

Chamando um gerador
cria um iterador
através do qual nós
controlar o
gerador
execução.

O resultado é um objeto com um valor
retornado e um
indicador que nos diz se o
gerador tem mais valores.
```

```
const result2 = weaponIterator.next (); assert (typeof result2 === "objeto"

    && result2.value === "Wakizashi" &&! result2.done,

    "Wakizashi recebeu!");

const result3 = weaponIterator.next (); assert (typeof result3 === "objeto"

    && result3.value === undefined && result3.done,

    "Não há mais resultados!");
```

Ligando em seguida novamente
obtem outro valor
do gerador.

Quando não há mais código para
executar, o gerador retorna
"indefinido" e
indica que está feito.

Como você pode ver, quando chamamos um gerador, um novo *iterador* é criado:

```
const weaponIterator = WeaponGenerator ();
```

O iterador é usado para controlar a execução do gerador. Uma das coisas fundamentais que o objeto iterador expõe é o *Próximo* método, que podemos usar para controlar o gerador, solicitando um valor dele:

```
const result1 = weaponIterator.next ();
```

Em resposta a essa chamada, o gerador executa seu código até atingir um *prodizer* palavra-chave que produz um resultado intermediário (um item na sequência de itens gerada) e retorna um *novo* objeto que encapsula aquele resultado e nos diz se seu trabalho está concluído.

Assim que o valor atual é produzido, o gerador suspende sua execução sem bloquear e espera pacientemente por outra solicitação de valor. Este é um recurso incrivelmente poderoso que as funções padrão não possuem, um recurso que usaremos posteriormente com grande efeito.

Neste caso, a primeira chamada para o iterador *Próximo* método executa o código do gerador para o primeiro *prodizer* expressão, rendimento "Katana", e retorna um objeto com a propriedade *valor* definido como Katana e a propriedade *feito* definido como falso, sinalizando que há mais valores a serem produzidos.

Mais tarde, solicitamos outro valor do gerador, fazendo outra chamada para o *weaponIterator* de *Próximo* método:

```
const result2 = weaponIterator.next ();
```

Isso tira o gerador da suspensão, e o gerador continua de onde parou, executando seu código até que outro valor intermediário seja alcançado: rendimento "Wakizashi". Isso suspende o gerador e produz um objeto carregando Wakizashi.

Finalmente, quando chamamos o *Próximo* pela terceira vez, o gerador retoma sua execução. Mas desta vez não há mais código para executar, então o gerador retorna um objeto com *valor* definido como *Indefinido*, e *feito* definido como verdade, sinalizando que seu trabalho está concluído.

Agora que você viu como controlar geradores por meio de iteradores, está pronto para aprender como iterar sobre os valores produzidos.

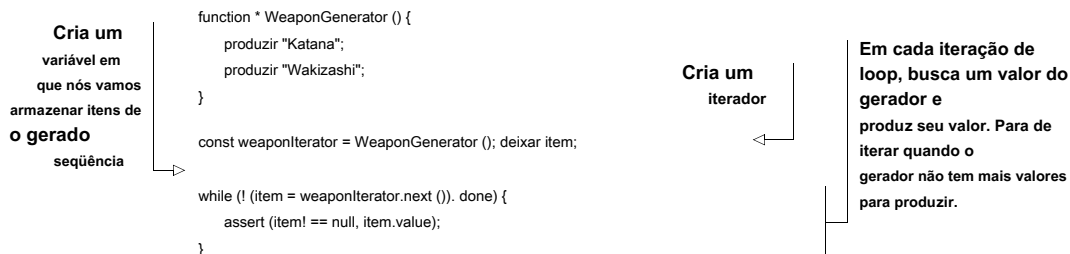
ITERANDO O ITERADOR

O iterador, criado chamando um gerador, expõe um *Próximo* método através do qual podemos solicitar um novo valor do gerador. O *Próximo* método retorna um objeto que carrega o valor produzido pelo gerador, bem como as informações armazenadas no

feito propriedade que nos diz se o gerador tem valores adicionais para produzir.

Agora vamos aproveitar esses fatos para usar um velho e simples enquanto loop para iterar sobre os valores produzidos por um gerador. Veja a lista a seguir.

Listagem 6.3 Iterando sobre os resultados do gerador com um enquanto ciclo



Aqui, criamos novamente um objeto iterador chamando uma função geradora:

```
const weaponIterator = WeaponGenerator ();
```

Nós também criamos um item variável na qual armazenaremos valores individuais produzidos pelo gerador. Continuamos especificando um enquanto loop com uma condição de loop ligeiramente complicada, que vamos quebrar um pouco:

```
while (! (item = weaponIterator.next ()), done) {
  afirmar (item! == null, item.value)
}
```

Em cada iteração de loop, buscamos um valor do gerador chamando o *Próximo* método do nosso `weaponIterator`, e armazenamos esse valor no item variável. Como todos esses objetos, o objeto referenciado pelo item variável tem um valor propriedade que armazena o valor retornado do gerador, e um *feito* propriedade que sinaliza se o gerador terminou de produzir valores. Se o gerador não terminar seu trabalho, vamos para outra iteração do loop; e se *for*, paramos o loop.

E é assim que para de loop, do nosso primeiro exemplo de gerador, funciona. O para de loop é um açúcar sintático para iteração sobre iteradores:

```
para (item var de WeaponGenerator ()) {
  afirmar (item! == nulo, item);
}
```

Em vez de chamar manualmente o *Próximo* método do iterador correspondente e sempre verificando se terminamos, podemos usar o para de loop para fazer exatamente a mesma coisa, apenas nos bastidores.

RENDENDO A OUTRO GERADOR

Assim como frequentemente chamamos uma função padrão de outra função padrão, em certos casos queremos ser capazes de delegar a execução de um gerador a outro. Vamos dar uma olhada em um exemplo que gera guerreiros e ninjas.

Listagem 6.4 usando `produzem*` para delegar a outro gerador

```
function * WarriorGenerator () {  
  produzir "Sun Tzu";  
  produzem* NinjaGenerator ();  
  produzir "Genghis Khan";  
}  
  
function * NinjaGenerator () {  
  produzir "Hattori";  
  produzir "Yoshi";  
}  
  
for (let warrior of WarriorGenerator ()) {  
  afirmar (guerreiro! == null, guerreiro);  
}
```

← **yield * delega para outro gerador.**

Se você executar este código, verá que a saída é Sun Tzu, Hattori, Yoshi, Genghis Khan.

Gerando Sun Tzu provavelmente não o pega desprevenido; é o primeiro valor produzido pelo `WarriorGenerator`. Mas a segunda saída, Hattori, merece uma explicação.

Usando o `produzem*` operador em um iterador, cedemos a outro gerador. Neste exemplo, a partir de `WarriorGenerator` estamos cedendo a um novo `NinjaGenerator`; todas as chamadas para o atual `WarriorGenerator` iterador Próximo método são redirecionados para o `NinjaGenerator`. Isso dura até o `NinjaGenerator` não tem trabalho a fazer. Então, em nosso exemplo, após Sun Tzu, o programa gera Hattori e Yoshi. Somente quando o `NinjaGenerator` é feito com o seu trabalho; a execução do iterador original continuará gerando Genghis Khan. Observe que isso está acontecendo de forma transparente com o código que chama o gerador original. O para de loop não se importa que o `WarriorGenerator`

cede a outro gerador; continua ligando Próximo até que esteja feito.

Agora que você tem uma ideia de como os geradores funcionam em geral e como funciona a delegação para outro gerador, vamos dar uma olhada em alguns exemplos práticos.

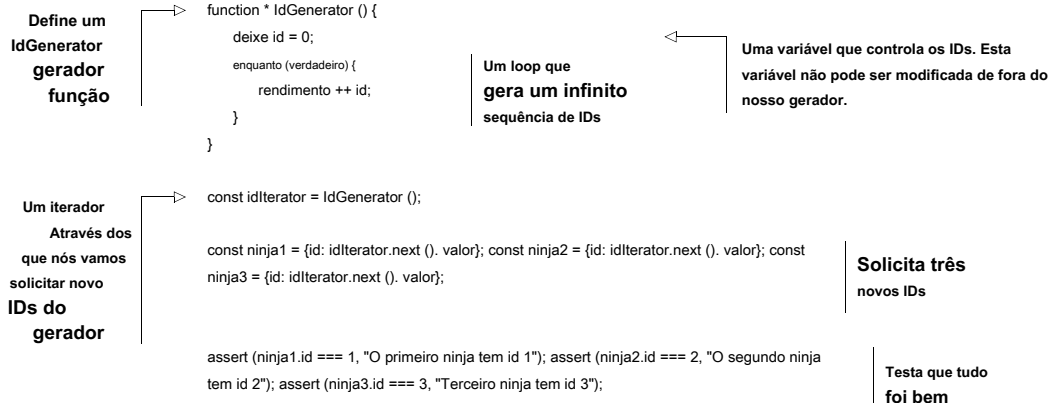
6.2.2 Usando geradores

Gerar sequências de itens é muito bom e elegante, mas vamos ser mais práticos, começando com um caso simples de geração de IDs únicos.

USANDO GERADORES PARA GERAR IDS

Ao criar certos objetos, muitas vezes precisamos atribuir um ID exclusivo para cada objeto. A maneira mais fácil de fazer isso é por meio de uma variável de contador global, mas isso é meio feio porque a variável pode ser acidentalmente bagunçada de qualquer lugar em nosso código. Outra opção é usar um gerador, conforme mostrado na lista a seguir.

Listagem 6.5 Usando geradores para gerar IDs



Este exemplo começa com um gerador que possui uma variável local, `eu ia`, que representa nosso contador de ID. O `eu ia` variável é local para nosso gerador; não há medo de que alguém o modifique acidentalmente em algum outro lugar do código. Isso é seguido por um infinito `enquanto` loop, que a cada iteração produz um novo `eu ia` valor e suspende sua execução até que uma solicitação de outro ID apareça:

```
function * IdGenerator () {
  deixe id = 0;
  enquanto (verdadeiro) {
    rendimento ++ id;
  }
}
```

NOTA Escrever loops infinitos não é algo que geralmente queremos fazer em uma função padrão. Mas com geradores está tudo bem! Sempre que o gerador encontra um `produzem` declaração, a execução do gerador é suspensa até o `Próximo` método é chamado novamente. Então todo `Próximo` chamada executa apenas uma iteração de nosso infinito `enquanto` loop e envia de volta o próximo valor de ID.

Depois de definir o gerador, criamos um objeto iterador:

```
const idIterator = IdGenerator ();
```

Isso nos permite controlar o gerador com chamadas para o `idIterator.next ()` método. Isso executa o gerador até um `produzem` é encontrado, retornando um novo valor de ID que podemos usar para nossos objetos:

```
const ninja1 = {id: idIterator.next (). valor};
```

Veja como isso é simples? Nenhuma variável global confusa cujo valor pode ser alterado acidentalmente. Em vez disso, usamos um iterador para solicitar valores de um gerador. Além disso, se

mais tarde, precisamos de outro iterador para rastrear os IDs de, por exemplo, samurai, podemos inicializar um novo gerador para isso.

USANDO GERADORES PARA TRAVESSAR O DOM

Como você viu no capítulo 2, o layout de uma página da web é baseado no DOM, uma estrutura em forma de árvore de nós HTML, em que cada nó, exceto o raiz, tem exatamente um pai e pode ter zero ou mais filhos. Como o DOM é uma estrutura tão fundamental no desenvolvimento da web, muito do nosso código é baseado em atravessá-lo. Uma maneira relativamente fácil de fazer isso é implementando uma função recursiva que será executada para cada nó visitado. Veja o seguinte código.

Listagem 6.6 Traversal de DOM recursivo

```
<div id = "subTree">
  <form>
    <input type = "text" />
  </form>
  <p> Parágrafo </p>
  <span> Span </span>
</div>
<script>
  function traverseDOM (elemento, retorno de chamada) {
    retorno de chamada (elemento);
    element = element.firstChild;
    while (elemento) {
      traverseDOM (elemento, retorno de chamada);
      element = element.nextElementSibling;
    }
  }
  const subTree = document.getElementById ("subTree"); traverseDOM (subárvore, função
(elemento) {
  assert (element! == null, element.nodeName); });
</script>
```

Processa o atual
nó com um retorno de chamada

Percorre o DOM de cada
elemento filho

Inicia todo o processo
chamando o traverseDOM
função para o nosso elemento raiz

Neste exemplo, usamos uma função recursiva para percorrer todos os descendentes do elemento com o subárvore id, no processo de registro de cada tipo de nó que visitamos. Neste caso, o código produz DIV, FORM, INPUT, P, e PERÍODO.

Já faz um tempo que escrevemos esse código de travessia do DOM e ele nos serviu perfeitamente. Mas agora que temos geradores à nossa disposição, podemos fazer isso de forma diferente; veja o código a seguir.

Listagem 6.7 Iterando em uma árvore DOM com geradores

```
function * DomTraversal (elemento) {
  elemento de rendimento;
  element = element.firstChild;
  while (elemento) {
    produzem* DomTraversal (elemento);
    element = element.nextElementSibling;
  }
}
```

Usa yield * para transferir o controle
de iteração para outra instância do
gerador DomTraversal

```

    }
  }

  const subTree = document.getElementById("subTree"); para (elemento let de DomTraversal
(subárvore)) {
    assert (element! == null, element.nodeName);
  }
}

```

**Itera sobre os nós
usando o loop for-of**

Esta listagem mostra que podemos obter travessias de DOM com geradores, tão facilmente quanto com recursão padrão, mas com o benefício adicional de não ter que usar a sintaxe um pouco estranha de callbacks. Em vez de processar a subárvore de cada nó visitado recorrendo a outro nível, criamos uma função geradora para cada nó visitado e nos rendemos a ela. Isso nos permite escrever o que é código conceitualmente recursivo de maneira iterável. O benefício é que podemos consumir a sequência gerada de nós com um simples `para de loop`, sem recorrer a retornos de chamada desagráveis.

Este exemplo é particularmente bom, porque também mostra como usar geradores para separar o código que está produzindo valores (neste caso, nós HTML) do código que está consumindo a sequência de valores gerados (neste caso, o `para de loop` que registra os nós visitados), sem ter que recorrer a callbacks. Além disso, usar iterações é, em certos casos, muito mais natural do que recursão, por isso é sempre bom ter nossas opções abertas.

Agora que exploramos alguns aspectos práticos dos geradores, vamos voltar a um tópico um pouco mais teórico e ver como trocar dados com um gerador em execução.

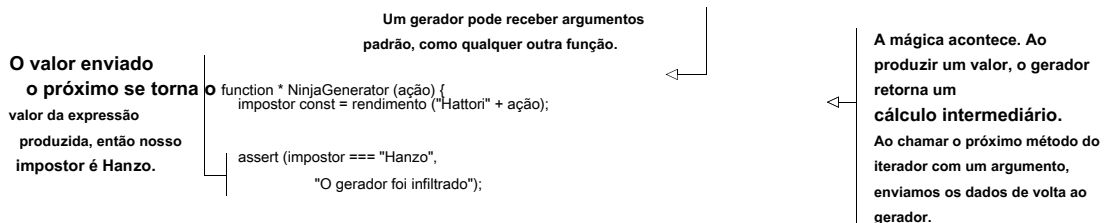
6.2.3 Comunicando-se com um gerador

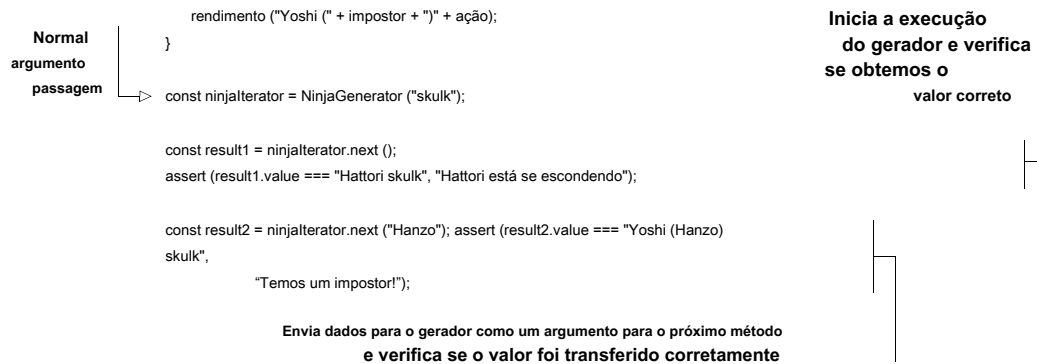
Nos exemplos apresentados até agora, você viu como retornar vários valores *a partir de* um gerador usando produzem expressões. Mas os geradores são ainda mais poderosos do que isso! Nós também podemos enviar dados *para* um gerador, conseguindo assim comunicação bidirecional! Com um gerador, podemos produzir um resultado intermediário, usar esse resultado para calcular outra coisa de fora do gerador e, então, quando estivermos prontos, enviar dados completamente novos de volta para o gerador e retomar sua execução. Usaremos esse recurso com grande efeito no final do capítulo para lidar com o código assíncrono, mas, por enquanto, vamos mantê-lo simples.

ENVIANDO VALORES COMO ARGUMENTOS DE FUNÇÃO DO GERADOR

A maneira mais fácil de enviar dados para um gerador é tratando-os como qualquer outra função e usando argumentos de chamada de função. Dê uma olhada na lista a seguir.

Listagem 6.8 Enviando dados e recebendo dados de um gerador





Uma função que recebe dados não é nada especial; as funções antigas fazem isso o tempo todo. Mas lembre-se, os geradores têm esse poder incrível; eles podem ser suspensos e reiniciados. E acontece que, ao contrário das funções padrão, os geradores podem até receber dados *depois de* sua execução é iniciada, sempre que os retomamos solicitando o próximo valor.

USANDO O PRÓXIMO MÉTODO PARA ENVIAR VALORES PARA UM GERADOR

Além de fornecer dados ao invocar o gerador pela primeira vez, podemos enviar dados *em* um gerador passando argumentos para o Próximo método. Nesse processo, despertamos o gerador da suspensão e retomamos sua execução. Este valor passado é usado pelo gerador como o valor de toda produz expressão, na qual o gerador estava atualmente suspenso, conforme mostrado na figura 6.3.

Neste exemplo, temos duas chamadas para o `ninjaGenerator` de Próximo método. A primeira chamada, `ninjaGenerator.next()`, solicita o primeiro valor do gerador. Como nosso gerador não começou a ser executado, esta chamada inicia o gerador, que calcula o

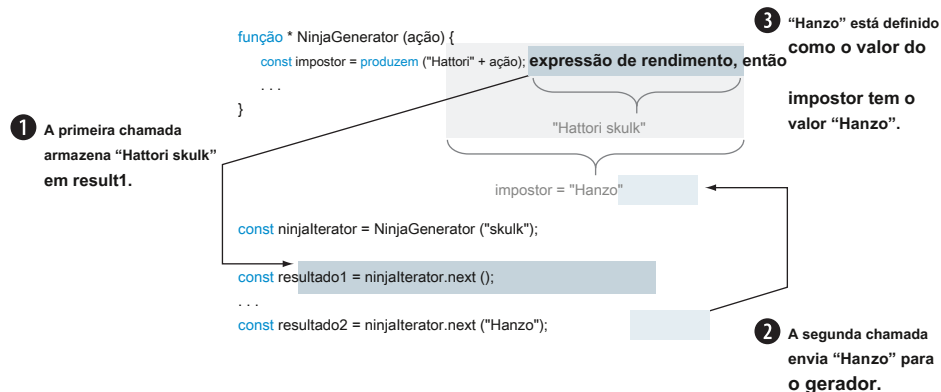


Figura 6.3 A primeira chamada para `ninjaGenerator.next()` solicita um novo valor do gerador, que retorna **Hattori skulk** e suspende a execução do gerador no **produzem** expressão. A segunda chamada para `ninjaGenerator.next("Hanzo")` solicita um novo valor, mas também envia o argumento

Hanzo no gerador. Este valor será usado como o valor de toda **produzem** expressão, e a variável **impostor** agora carregará o valor **Hanzo**.

valor da expressão "Hattori" + ação, produz o Hattori skulk valor e suspende a execução do gerador. Não há nada de especial nisso; fizemos algo semelhante várias vezes ao longo deste capítulo.

O interessante acontece na segunda chamada para o `ninjalterator` de Próximo método: `ninjalterator.next("Hanzo")`. Desta vez, estamos usando o Próximo método para passar os dados de volta para o gerador. Nossa função geradora está esperando pacientemente, suspensa na expressão `rendimento("Hattori" + ação)`, então o valor `Hanzo`, passado como o argumento para `Próximo()`, é usado como o valor do todo produzem expressão. Na nossa caso, isso significa que a variável `impostor no impostor = rendimento("Hattori" + ação)` vai acabar com o valor `Hanzo`.

É assim que alcançamos a comunicação bidirecional com um gerador. Nós usamos `produzem` para retornar dados de um gerador, e o iterador de `Próximo()` método para passar os dados de volta para o gerador.

NOTA O Próximo método fornece o valor para a espera produzem expressão, então se não houver produzem expressão esperando, não há nada para fornecer o valor. Por este motivo, nós *não pode* fornecer valores durante a primeira chamada para o Próximo método. Mas lembre-se, se você precisa fornecer um valor inicial para o gerador, você pode fazê-lo ao chamar o próprio gerador, como fizemos com `NinjaGenerator("skulk")`.

EXCEÇÕES DE LANÇAMENTO

Existe outra maneira, um pouco menos ortodoxa, de fornecer um valor a um gerador: lançando uma exceção. Cada iterador, além de ter um Próximo método, tem um `lançar` método que podemos usar para lançar uma exceção de volta para o gerador. Novamente, vamos ver um exemplo simples.

Listagem 6.9 Lançando exceções para geradores

```
function * NinjaGenerator () {
  tentar{
    produzir "Hattori";
    fail ("A exceção esperada não ocorreu");
  }
  pegar (e) {
    assert (e === "Catch this!", "Aha! Pegamos uma exceção");
  }
}
```

Esta falha não deveria ser alcançado.

Captura exceções e testa se nós recebemos o exceção esperada

```
const ninjalterator = NinjaGenerator ();

const result1 = ninjalterator.next ();
assert (result1.value === "Hattori", "Temos Hattori");

ninjalterator.throw ("Pegue isso!");
```

Puxa um valor de o gerador

Lança uma exceção para o gerador

A Listagem 6.9 começa de forma semelhante à Listagem 6.8, especificando um gerador chamado `NinjaGenerator`.

Mas, desta vez, o corpo do gerador é ligeiramente diferente. Rodeamos todo o código do corpo da função com um `try` bloco:

```
function * NinjaGenerator () {
  tentar{
    produzir "Hattori";
    fail ("A exceção esperada não ocorreu");
  }
  pegar (e) {
    assert (e === "Catch this!", "Aha! Pegamos uma exceção");
  }
}
```

Em seguida, continuamos criando um iterador e obtendo um valor do gerador:

```
const ninjalteator = NinjaGenerator (); const result1 = ninjalteator.next ();
```

Finalmente, usamos o lançar método, disponível em todos os iteradores, para lançar uma exceção de volta ao gerador:

```
ninjalteator.throw ("Pegue isso!");
```

Ao executar essa listagem, podemos ver que nosso lançamento de exceção funciona conforme o esperado, conforme mostrado na figura 6.4.

Esse recurso que nos permite lançar exceções de volta aos geradores pode parecer um pouco estranho no início. Por que faríamos isso? Não se preocupe; não vamos mantê-lo no escuro por muito tempo. No final deste capítulo, usaremos esse recurso para melhorar a comunicação assíncrona do lado do servidor.

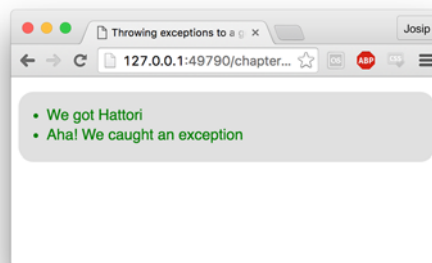


Figura 6.4 Podemos lançar exceções para geradores de fora de um gerador.

Apenas seja paciente um pouco mais.

Agora que você viu vários

aspectos dos geradores, estamos prontos para dar uma olhada nos bastidores para ver como funcionam os geradores.

6.2.4 Explorando geradores sob o capô

Até agora sabemos que chamar um gerador não o executa. Em vez disso, ele cria um novo iterador que podemos usar para solicitar valores do gerador. Depois que um gerador produz (ou fornece) um valor, ele suspende sua execução e aguarda a próxima solicitação. Então, de certa forma, um gerador funciona quase como um pequeno programa, uma máquina de estado que se move entre estados:

- *Início suspenso*—Quando o gerador é criado, ele inicia neste estado. Nenhum código do gerador é executado.
- *Executando*—O estado em que o código do gerador é executado. A execução continua desde o início ou de onde o gerador foi o último

suspenso. Um gerador se move para este estado quando o iterador correspondente **Próximo** método é chamado e existe código a ser executado.

- **Rendimento suspenso**—Durante a execução, quando um gerador atinge um **produzem** expressão, ele cria um novo objeto carregando o valor de retorno, o produz e suspende sua execução. Este é o estado em que o gerador está pausado e está esperando para continuar sua execução.
- **Concluído**—Se durante a execução o gerador funcionar em um **Retorna instrução** ou fica sem código para executar, o gerador passa para este estado.

A Figura 6.5 ilustra esses estados.

Agora vamos complementar isso em um nível ainda mais profundo, vendo como a execução dos geradores é rastreada com contextos de execução.

```
função * NinjaGenerator () {
  produzem "Hattori";
  produzem "Yoshi";
}
```

- 1 `const ninjalterator = NinjaGenerator ();`
Crie um novo gerador no estado inicial suspenso.
- 2 `const result1 = ninjalterator.next ();`
Ative o gerador. Mova do início suspenso para a execução.
Executar até rendimento "Hattori"
e faça uma pausa. Mova para o estado de rendimento suspenso.
Retorne um novo objeto: { valor: "Hattori",
feito: falso}.
- 3 `const result2 = ninjalterator.next ();`
Reative o gerador. Mova de rendimento suspenso para execução.
Executar até rendimento "Yoshi"
e faça uma pausa. Mova para o estado de rendimento suspenso.
Retorne um novo objeto: { valor: "Yoshi",
feito: falso}.
- 4 `const result3 = ninjalterator.next ();`
Reative o gerador. Mova de rendimento suspenso para execução. Não há mais código para executar. Mova para o estado Concluído. Devolver um novo objeto: { valor: indefinido, feito: verdadeiro}.

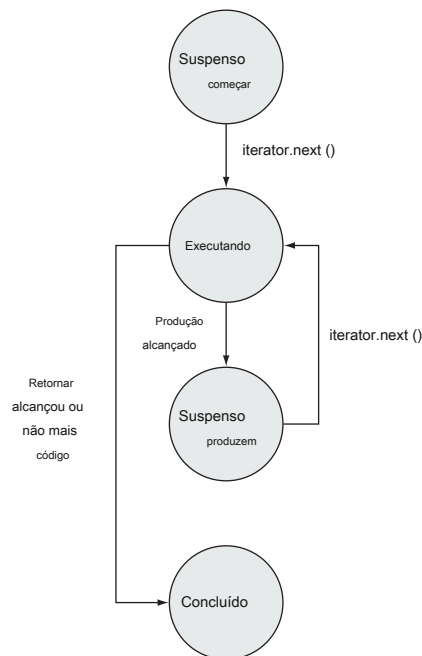


Figura 6.5 Durante a execução, um gerador se move entre os estados acionados por chamadas para o iterador correspondente **Próximo** método.

GERADORES DE RASTREIO COM CONTEXTOS DE EXECUÇÃO

No capítulo anterior, apresentamos o contexto de execução, um mecanismo interno de JavaScript usado para rastrear a execução de funções. Embora um tanto especiais, os geradores ainda são funções, portanto, vamos examinar mais de perto explorando a relação entre eles e os contextos de execução. Começaremos com um fragmento de código simples:

```
function * NinjaGenerator (ação) {
  rendimento "Hattori" + ação; retornar "Yoshi" +
  ação;
}
```

```
const ninjaltorator = NinjaGenerator ("skulk"); const result1 = ninjaltorator.next (); const
result2 = ninjaltorator.next ();
```

Aqui, reutilizamos nosso gerador que produz dois valores: Hattori skulk e Yoshi skulk.

Agora, vamos explorar o estado do aplicativo, a pilha de contexto de execução em vários pontos da execução do aplicativo. A Figura 6.6 fornece um instantâneo em duas posições na execução do aplicativo. O primeiro instantâneo mostra o estado do aplicativo execução *antes* chamando o NinjaGenerator função **B**. Porque estamos executando código global, a pilha de contexto de execução contém apenas o contexto de execução global, que faz referência ao ambiente global no qual nossos identificadores são mantidos. Apenas o NinjaGenerator identificador faz referência a uma função, enquanto os valores de todos os outros identificadores são Indefinido.

Quando fazemos a ligação para o NinjaGenerator função **C**

```
const ninjaltorator = NinjaGenerator ("skulk");
```

o fluxo de controle entra no gerador e, como acontece quando entramos em qualquer outra função, um novo NinjaGenerator O item de contexto de execução é criado (junto com o ambiente léxico correspondente) e colocado na pilha. Mas porque os geradores são especiais, *Nenhum* do código de função é executado. Em vez disso, um novo iterador, ao qual nos referiremos no código como ninjaltorator, é criado e retornado. Como o iterador é usado para controlar a execução do gerador, o iterador obtém uma referência ao contexto de execução no qual foi criado.

Uma coisa interessante acontece quando a execução do programa sai do gerador, como mostra a figura 6.7. Normalmente, quando a execução do programa retorna de uma função padrão, o contexto de execução correspondente é retirado da pilha e completamente descartado. Mas este não é o caso dos geradores.

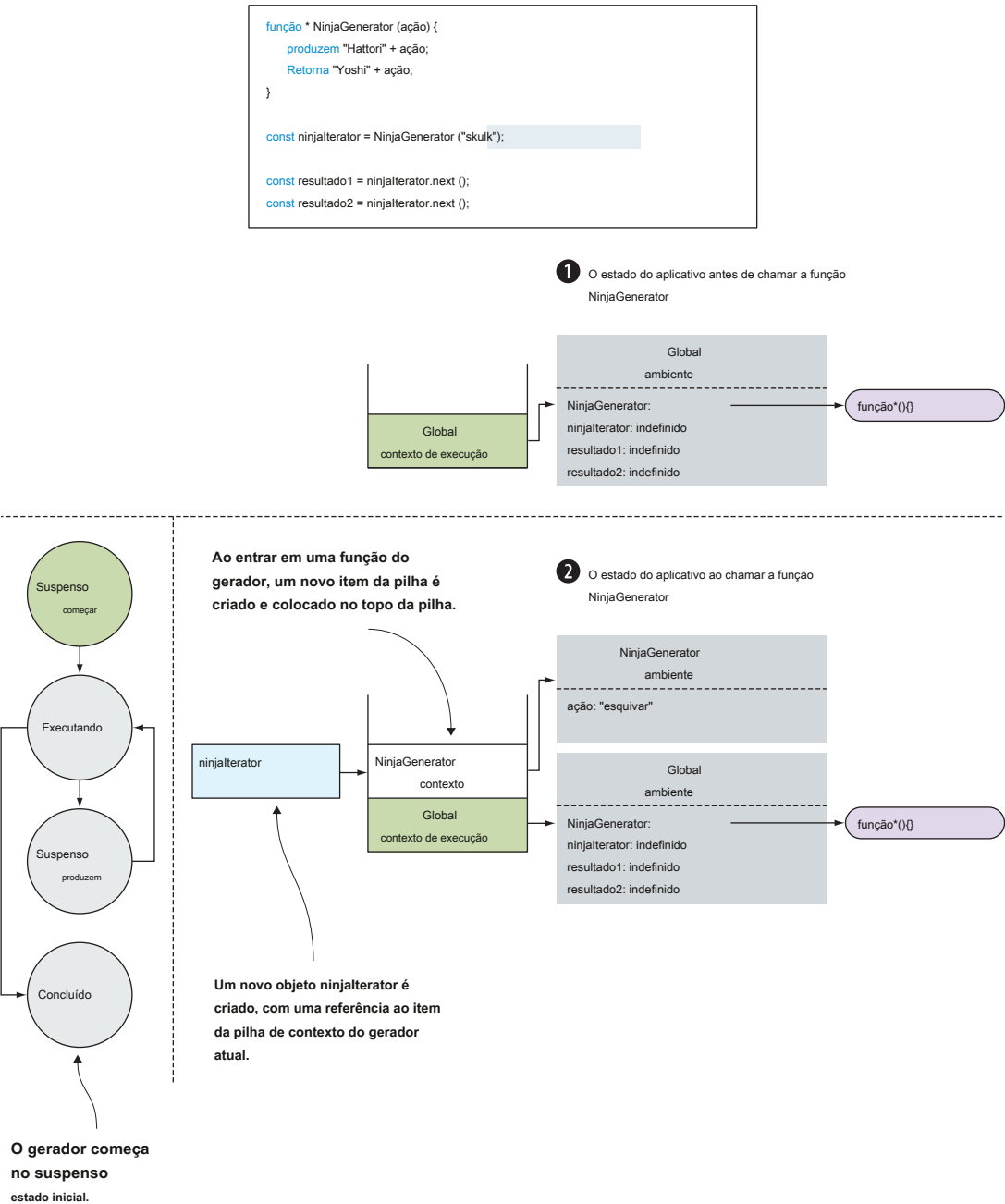


Figura 6.6 O estado da pilha de contexto de execução *antes* chamando o `NinjaGenerator` função `B` , e *quando* chamando o `NinjaGenerator` função `C`

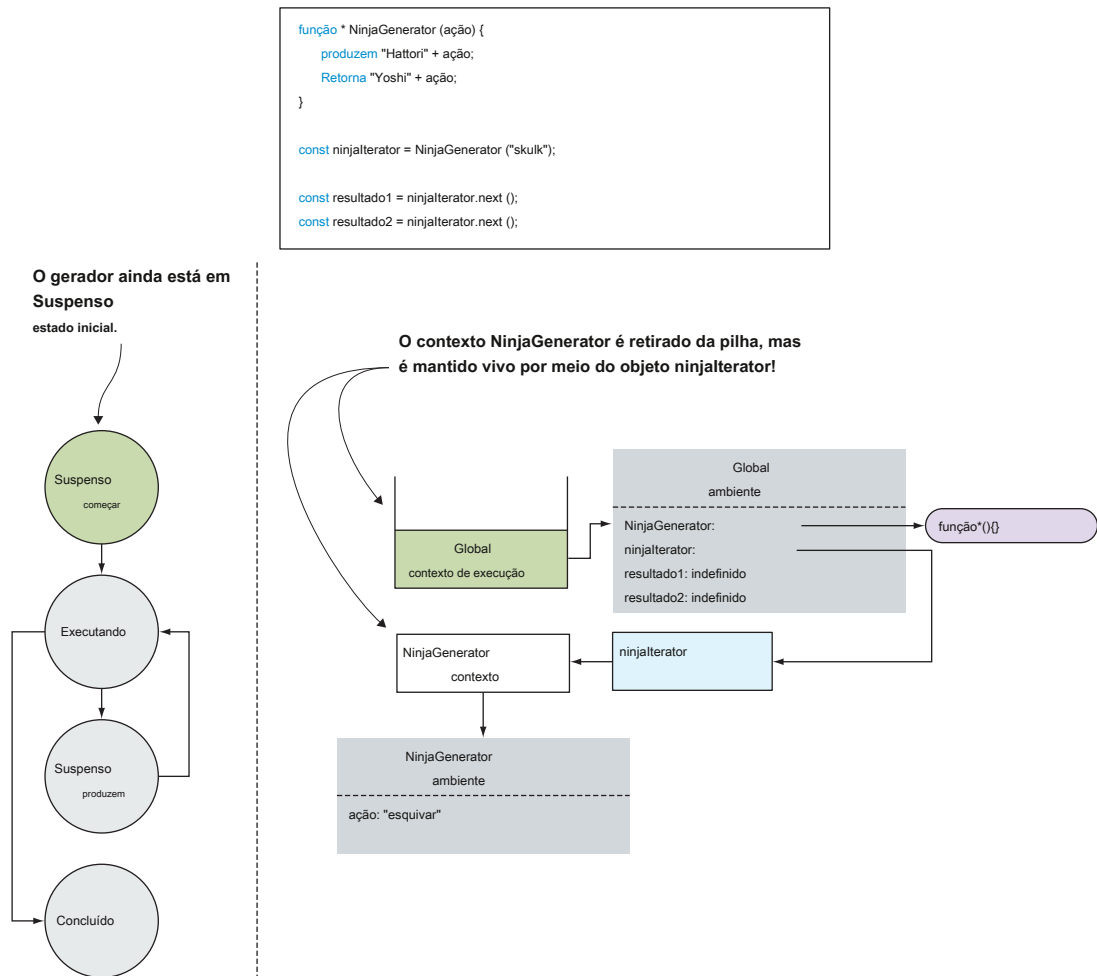


Figura 6.7 O estado do aplicativo ao retornar do **NinjaGenerator** chamar

A correspondência NinjaGenerator item de pilha *é* saiu da pilha, mas *é não* descartado, porque o ninjaltorator mantém uma referência a ele. Você pode vê-lo como um análogo aos fechamentos. Nos encerramentos, precisamos manter vivas as variáveis que estão vivas no momento em que o encerramento da função é criado, para que nossas funções mantenham uma referência ao ambiente no qual foram criadas. Desta forma, garantimos que o ambiente e suas variáveis estão vivos enquanto a própria função. Os geradores, por outro lado, devem ser capazes de retomar sua execução. Como a execução de todas as funções é tratada por contextos de execução, o iterador mantém uma referência ao seu contexto de execução, de modo que esteja vivo pelo tempo que o iterador precisar.

Outra coisa interessante acontece quando chamamos o Próximo método no iterador:

```
const result1 = ninjaltorator.next ();
```

Se esta fosse uma chamada de função direta padrão, isso causaria a criação de um *novo* `Próximo()` item de contexto de execução, que seria colocado na pilha. Mas como você deve ter notado, os geradores são tudo menos padrão, e uma chamada para o `Próximo` método de um iterador se comporta de maneira muito diferente. Ele reativa o contexto de execução correspondente, neste caso, o `NinjaGenerator` contexto e o coloca no topo da pilha, continuando a execução de onde parou, conforme mostrado na figura 6.8.

A Figura 6.8 ilustra uma diferença crucial entre funções padrão e geradores. As funções padrão só podem ser chamadas de novo, e cada chamada cria um *novo* contexto de execução. Em contraste, o contexto de execução de um gerador pode ser temporariamente suspenso e reiniciado à vontade.

Em nosso exemplo, porque esta é a primeira chamada para o `Próximo` método, e o gerador não começou a executar, o gerador inicia sua execução e passa para o estado `Executando`. A próxima coisa interessante acontece quando nossa função de gerador atinge este ponto:

rendimento "Hattori" + ação

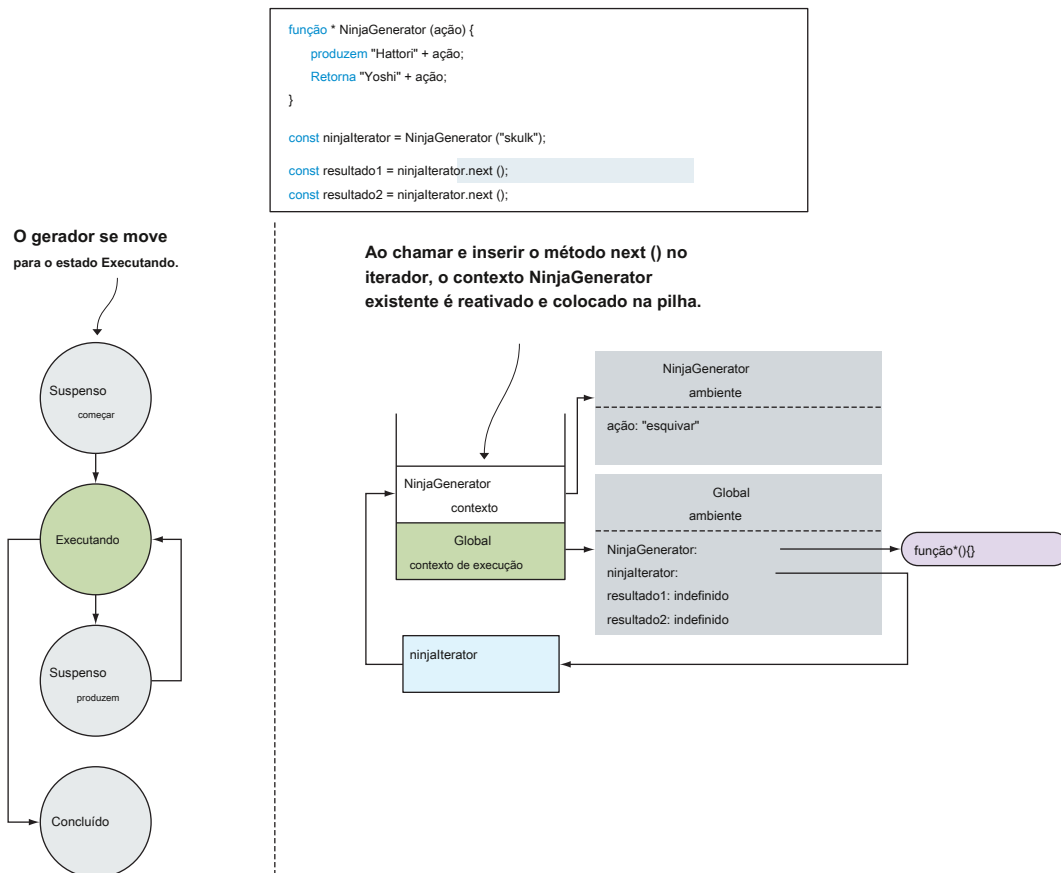


Figura 6.8 Chamando o iterador **Próximo** método reativa o item da pilha de contexto de execução do gerador correspondente, coloca-o na pilha e continua de onde parou da última vez.

O gerador determina que a expressão é igual Hattori skulk, e a avaliação chega ao produzir palavra-chave. Isso significa que Hattori skulk é o primeiro resultado intermediário do nosso gerador e que queremos suspender a execução do gerador e retornar esse valor. Em termos de estado do aplicativo, algo semelhante acontece como antes: o NinjaGenerator contexto é retirado da pilha, mas não é completamente descartado, porque ninjalteator mantém uma referência a ele. O gerador agora está suspenso e mudou para o estado Suspended Yield, sem bloqueio. A execução do programa é retomada em código global, armazenando o valor gerado em resultado1.

O estado atual do aplicativo é mostrado na figura 6.9.

O código continua alcançando outra chamada de iterador:

```
const result2 = ninjalteator.next ();
```

```
função * NinjaGenerator (ação) {
  produzem "Hattori" + ação;
  Retorna "Yoshi" + ação;
}

const ninjalteator = NinjaGenerator ("skulk");

const resultado1 = ninjalteator.next ();
const resultado2 = ninjalteator.next ();
```

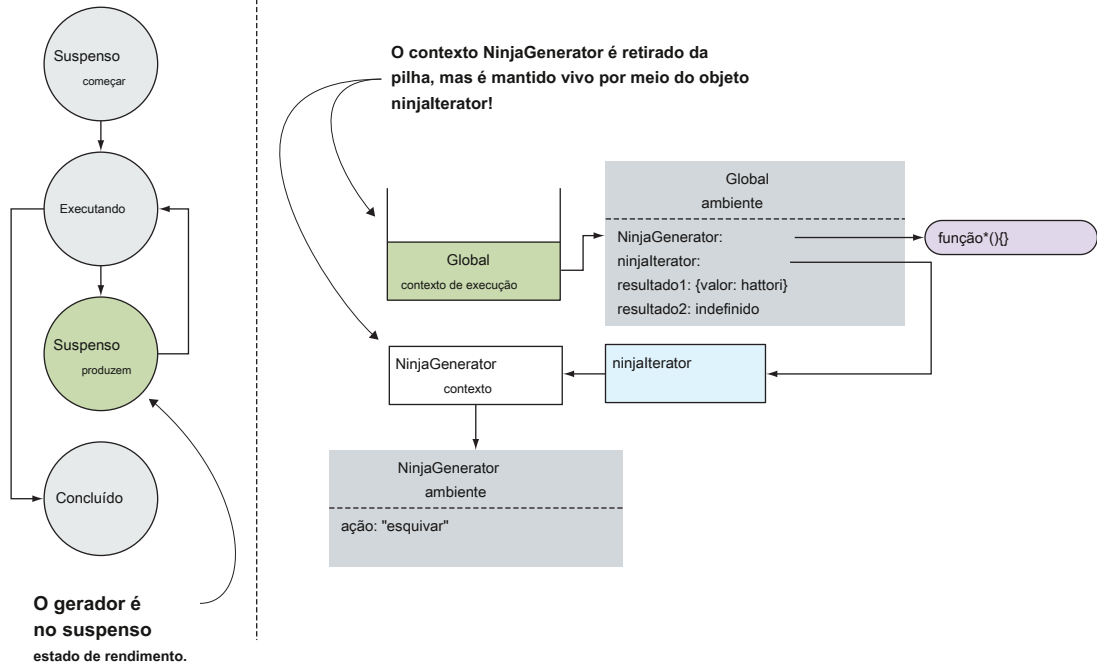


Figura 6.9 Depois de gerar um valor, o contexto de execução do gerador é retirado da pilha (mas não é descartado, porque **ninjalteator** mantém uma referência a ele), e a execução do gerador é suspensa (o gerador passa para o estado de rendimento suspenso).

Neste ponto, passamos por todo o procedimento mais uma vez: reativamos o NinjaGenerator contexto referenciado por `ninjaIteator`, coloque-o na pilha e continue a execução de onde paramos. Neste caso, o gerador avalia a expressão

"Yoshi" + ação. Mas desta vez não há produção de expressão e, em vez disso, o programa encontra uma demonstração de retorno. Isso retorna o valor Yoshi skul e conclui a execução do gerador movendo o gerador para o estado Concluído.

Uff, isso era incrível! Nós nos aprofundamos em como os geradores funcionam sob o capô para mostrar que todos os benefícios maravilhosos dos geradores são um efeito colateral do fato de que o contexto de execução de um gerador é mantido vivo se produzirmos de um gerador, e não destruído como é o caso com valores de retorno e funções padrão.

Agora, recomendamos que você respire fundo antes de continuar com o segundo ingrediente-chave necessário para escrever um código assíncrono elegante: promessas.

6.3 *Trabalhando com promessas*

Em JavaScript, dependemos muito de cálculos assíncronos, cálculos cujos resultados ainda não temos, mas teremos em algum momento posterior. Portanto, o ES6 introduziu um novo conceito que torna o manuseio de tarefas assíncronas mais fácil: promessas.

UMA *promessa* é um espaço reservado para um valor que não temos agora, mas teremos mais tarde; é uma garantia de que, eventualmente, saberemos o resultado de uma computação assíncrona. Se cumprirmos nossa promessa, nosso resultado será um valor. Se ocorrer um problema, nosso resultado será um erro, uma desculpa para não podermos entregar. Um ótimo exemplo de uso de promessas é buscar dados de um servidor; prometemos que eventualmente obteremos os dados, mas sempre há uma chance de ocorrerem problemas.

Criar uma nova promessa é fácil, como você pode ver no exemplo a seguir.

Listagem 6.10 Criar uma promessa simples

Uma promessa é resolvida com sucesso chamando o resolução aprovada função (e rejeitada por ligar a função de rejeição).

```
const ninjaPromise = new Promise ((resolver, rejeitar) => {
  resolver ("Hattori");
  // rejeitar ("Um erro ao resolver uma promessa!"); });
```

Cria uma promessa chamando um construtor `Promise` integrado e passando uma função de retorno de chamada com dois parâmetros: `resolver` e `rejeitar`

E o segundo é chamado se um erro ocorre.

```
ninjaPromise.then (ninja => {
  assert (ninja === "Hattori", "Foi-nos prometido Hattori!"); }, err => {
  fail ("Não deve haver um erro");});
```

Ao usar o método `then` em uma promessa, podemos passar em dois retornos de chamada; o primeiro é chamado se uma promessa foi resolvida com sucesso.

Para criar uma promessa, usamos o novo e integrado Promessa construtor, para o qual passamos uma função, neste caso uma função de seta (mas poderíamos facilmente usar uma expressão de função). Esta função, chamada de *executor* função, tem dois parâmetros:

resolver e rejeitar. O executor é chamado *imediatamente* ao construir o

Promessa objeto com duas funções integradas como argumentos: resolver, que chamamos manualmente se quisermos que a promessa seja resolvida com sucesso, e rejeitar, que chamamos se ocorrer um erro.

Este código usa a promessa chamando o integrado então método no Promessa objeto, um método para o qual passamos duas funções de retorno de chamada: a *sucesso* retorno de chamada e um *fracasso* ligar de volta. O primeiro é chamado se a promessa for resolvida com sucesso (se o resolver função é chamada na promessa), e o último é chamado se houver um problema (ocorre uma exceção não tratada ou o rejeitar função é chamada em uma promessa).

Em nosso código de exemplo, criamos uma promessa e a resolvemos imediatamente chamando o resolver função com o argumento Hattori. Portanto, quando chamamos o então método, o primeiro, sucesso, retorno de chamada é executado e o teste que produz fomos prometeu Hattori! passes.

Agora que temos uma ideia geral do que são as promessas e como funcionam, vamos dar um passo atrás para ver alguns dos problemas que as promessas enfrentam.

6.3.1 Compreender os problemas com retornos de chamada simples

Usamos código assíncrono porque não queremos bloquear a execução de nosso aplicativo (decepcionando assim nossos usuários) durante a execução de tarefas de longa duração. Atualmente, resolvemos este problema com retornos de chamada: Para uma tarefa de longa execução, fornecemos uma função, um retorno de chamada que é invocado quando a tarefa finalmente é concluída.

Por exemplo, buscar um arquivo JSON em um servidor é uma tarefa de longa duração, durante a qual não queremos que o aplicativo pare de responder para nossos usuários. Portanto, fornecemos um retorno de chamada que será invocado quando a tarefa for concluída:

```
getJSON ("data / ninjas.json", function () {
  /* Manipular resultados */
});
```

Naturalmente, durante essa tarefa de longa execução, podem ocorrer erros. E o problema com retornos de chamada é que você não pode usar construções de linguagem integradas, como tentar pegar declarações, da seguinte forma:

```
tentar {
  getJSON ("data / ninjas.json", function () {
    // Manipular resultados
  });
} catch (e) { /* Manipular erros */ }
```

Isso acontece porque o código que invoca o retorno de chamada geralmente não é executado na mesma etapa do loop de evento que o código que inicia a tarefa de longa execução (você verá exatamente o que isso significa quando aprender mais sobre o loop de evento no capítulo 13).

Como consequência, os erros geralmente se perdem. Muitas bibliotecas, portanto, definem suas próprias convenções para relatar erros. Por exemplo, no mundo Node.js, callbacks costumam ter dois argumentos, *erro* e *dados*. Onde *erro* será um valor não nulo se ocorrer um erro em algum lugar ao longo do caminho. Isso leva ao primeiro problema com retornos de chamada: *difícil tratamento de erros*.

Depois de realizar uma tarefa de longa duração, geralmente queremos fazer algo com os dados obtidos. Isso pode levar ao início de outra tarefa de longa execução, que pode eventualmente acionar outra tarefa de longa duração e assim por diante - levando a uma série de etapas interdependentes, assíncronas e processadas por callback. Por exemplo, se quisermos executar um plano sorrateiro para encontrar todos os ninjas à nossa disposição, obter a localização do primeiro ninja e enviar-lhe algumas ordens, acabaremos com algo assim:

```
getJSON("data / ninjas.json", function (err, ninjas) {
  getJSON(ninjas[0].location, function (err, locationInfo) {
    sendOrder(locationInfo, function (err, status) {
      /* Status do processo */
    })
  })
});
```

Você provavelmente acabou, pelo menos uma ou duas vezes, com código estruturado de forma semelhante - um monte de callbacks aninhados que representam uma série de etapas que precisam ser feitas. Você pode notar que esse código é difícil de entender, inserir novas etapas é uma dor e o tratamento de erros complica seu código significativamente. Você obtém essa "pirâmide da desgraça" que continua crescendo e é difícil de administrar. Isso nos leva ao segundo problema com retornos de chamada: *executar sequências de etapas é complicado*.

Às vezes, as etapas que temos que percorrer para chegar ao resultado final não dependem umas das outras, então não temos que executá-las em sequência. Em vez disso, para economizar milissegundos preciosos, podemos fazê-los em paralelo. Por exemplo, se quisermos colocar um plano em movimento que exija que saibamos quais ninjas temos à nossa disposição, o próprio plano e o local onde nosso plano será executado, poderíamos aproveitar as vantagens do jQuery obter método e escreva algo assim:

```
var ninjas, mapInfo, plano;

$.get("data / ninjas.json", function (err, data) {
  if (err) {processError(err); Retorna; } ninjas = dados;

  actionItemArrived ();
});

$.get("data / mapInfo.json", function (err, data) {
  if (err) {processError(err); Retorna; } mapInfo = dados;

  actionItemArrived ();
});
```

```
$.get("plan.json", função (err, dados) {
  if (err) {processError (err); Retorna; }

  plano = dados;
  actionItemArrived ();
});

function actionItemArrived () {
  if (ninjas! = null && mapInfo! = null && plan! = null) {
    console.log ("O plano está pronto para ser colocado em ação!");
  }
}

function processError (err) {
  alert ("Erro", err)
}
```

Neste código, executamos as ações de obter os ninjas, obter as informações do mapa e obter o plano em paralelo, porque essas ações não dependem umas das outras. Só nos preocupamos que, no final, tenhamos todos os dados à nossa disposição. Como não sabemos a ordem em que os dados são recebidos, sempre que obtemos alguns dados, temos que verificar se é a última peça do quebra-cabeça que está faltando. Finalmente, quando todas as peças estiverem no lugar, podemos colocar nosso plano em ação. Observe que temos que escrever muito código clichê apenas para fazer algo tão comum quanto executar uma série de ações em paralelo. Isso nos leva ao terceiro problema com retornos de chamada: *executar uma série de etapas em paralelo também é complicado*.

Ao apresentar o primeiro problema com retornos de chamada - lidar com erros - mostramos como não podemos usar algumas das construções fundamentais da linguagem, como trycatch declarações. Algo semelhante acontece com os loops: se você quiser realizar ações assíncronas para cada item em uma coleção, terá que passar por mais alguns obstáculos para fazê-lo.

É verdade que você pode fazer uma biblioteca para simplificar o tratamento de todos esses problemas (e muitas pessoas fizeram isso). Mas isso geralmente leva a muitas maneiras ligeiramente diferentes de lidar com os mesmos problemas, então as pessoas por trás do JavaScript nos deram *promessas*, uma abordagem padrão para lidar com computação assíncrona.

Agora que você entende a maioria das razões por trás da introdução de promessas, bem como tem um conhecimento básico delas, vamos dar um passo adiante.

6.3.2 Mergulhando em promessas

Uma promessa é um objeto que serve como um marcador para um resultado de uma tarefa assíncrona. Representa um valor que não temos, mas esperamos ter no futuro. Por essa razão, durante sua vida, uma promessa pode passar por alguns estados, como mostra a figura 6.10.

Uma promessa começa no *pendente* estado, no qual não sabemos nada sobre nosso valor prometido. É por isso que uma promessa no estado pendente também é chamada de não resolvido promessa. Durante a execução do programa, se a promessa resolver função é chamada, o

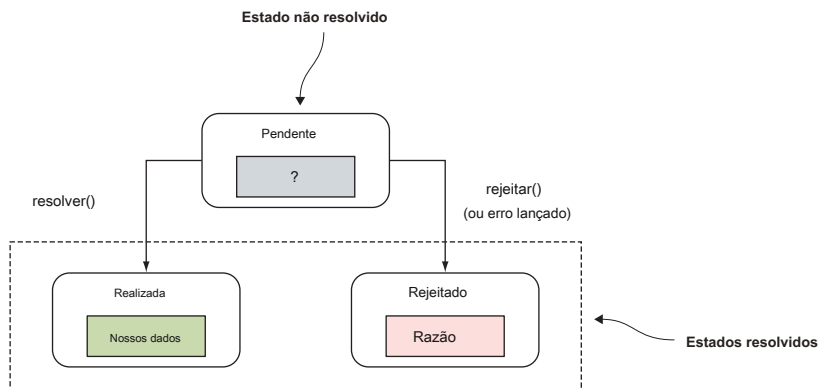


Figura 6.10 Estados de uma promessa

promessa move-se para o *realizada* estado, no qual obtivemos com sucesso o valor prometido. Por outro lado, se a promessa é rejeitar função é chamada, ou se uma exceção não tratada ocorrer durante o tratamento da promessa, a promessa muda para o *rejeitado* estado, no qual não conseguimos obter o valor prometido, mas no qual ao menos sabemos o porquê. Uma vez que uma promessa tenha alcançado o *realizada* estado ou o

rejeitado estado, ele não pode mudar (uma promessa não pode ir de cumprida para rejeitada ou vice-versa) e sempre permanece nesse estado. Dizemos que uma promessa é *resolvida* (com sucesso ou não).

A lista a seguir fornece uma visão mais detalhada do que está acontecendo quando usamos as promessas.

Listagem 6.11 Uma análise mais detalhada da ordem de execução da promessa

```
relatório ("No início do código");
```

```
var ninjaDelayedPromise = new Promise ((resolver, rejeitar) => {
  report ("executor ninjaDelayedPromise");
  setTimeout (() => {
    report ("Resolvendo ninjaDelayedPromise");
    resolver ("Hattori");
  }, 500);
});
```

```
assert (ninjaDelayedPromise! == null, "Depois de criar ninjaDelayedPromise");
```

```
ninjaDelayedPromise.then (ninja => {
  afirmar (ninja === "Hattori",
    "resolução de ninjaDelayedPromise tratada com Hattori");
});
```

Chamando a Promessa construtor imediatamente invoca o passado função.

Resolveremos essa promessa como bem-sucedida depois que um tempo limite de 500 ms expirar.

O método Promise then é usado para configurar um retorno de chamada que será chamado quando a promessa for resolvida, em nosso caso quando o tempo limite expirar.

```
const ninjaImmediatePromise = new Promise ((resolver, rejeitar) => {
  report ("executor ninjaImmediatePromise. Resolução imediata."); resolver ("Yoshi");

});

ninjaImmediatePromise.then (ninja => {
  afirmar (ninja === "Yoshi",
    "resolução de ninjaImmediatePromise tratada com Yoshi");
});

relatório ("No final do código");
```

**Cria um novo
prometa que
Obtém imediatamente
resolvido**

**Configura um retorno de chamada a ser invocado quando a promessa
resolve. Mas nossa promessa já foi cumprida!**

O código na listagem 6.11 produz os resultados mostrados na figura 6.11. Como você pode ver, o código começa registrando a mensagem "No início do código" usando nosso personalizado relatório função (apêndice C) que exibe a mensagem na tela. Isso nos permite rastrear facilmente a ordem de execução.

Em seguida, criamos uma nova promessa chamando o Promessa construtor. Isso invoca imediatamente a função do executor na qual configuramos um tempo limite:

```
setTimeout (() => {
  report ("Resolvendo ninjaDelayedPromise");
  resolver ("Hattori");
}, 500);
```

O tempo limite resolverá a promessa a seguir 500ms. Esta poderia ter sido qualquer outra tarefa assíncrona, mas escolhemos o humilde timeout devido à sua simplicidade.

Depois de `ninjaDelayedPromise` tem sido criado, ele ainda não sabe o valor que eventualmente terá, ou mesmo se terá sucesso. (Lembre-se de que ainda está esperando o tempo limite que o resolverá.) Assim, após a construção

ção, o `ninjaDelayedPromise` está no

primeiro estado de promessa, *pendente*.

Em seguida, usamos o então método no `ninjaDelayedPromise` para agendar um callback a ser executado quando a promessa for resolvida com sucesso:

```
ninjaDelayedPromise.then (ninja => {
  afirmar (ninja === "Hattori",
    "resolução de ninjaDelayedPromise tratada com Hattori");
});
```

Este retorno de chamada irá *sempre* ser chamado de forma assíncrona, independentemente do estado atual da promessa.

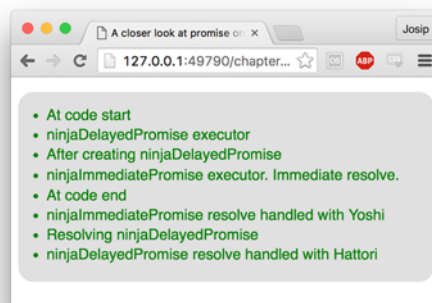


Figura 6.11 O resultado da execução da listagem 6.11

Continuamos criando outra promessa, `ninjalImmediatePromise`, que é resolvido imediatamente durante sua construção, chamando o resolver função. Ao contrário do `ninjaDelayedPromise`, que após a construção está no *pendente* estado, o `ninjalImmediatePromise` termina a construção no *resolvido* estado, e a promessa já tem o valor Yoshi.

Depois, usamos o `ninjalImmediatePromise` de então método para registrar um retorno de chamada que será executado quando a promessa for resolvida com êxito. Mas nossa promessa já foi cumprida; isso significa que o retorno de chamada de sucesso será chamado imediatamente ou que será ignorado? A resposta é *nenhum*.

As promessas são projetadas para lidar com ações assíncronas, portanto, o mecanismo de JavaScript *sempre* recorre ao tratamento assíncrono para tornar previsível o comportamento da promessa. O motor faz isso executando o então callbacks depois que todo o código na etapa atual do loop de eventos for executado (mais uma vez, exploraremos exatamente o que isso significa no capítulo 13). Por este motivo, se estudarmos a saída na figura 6.11, veremos que primeiro registramos "No final do código" e, em seguida, registramos que o `ninjalImmediatePromise` foi resolvido. No final, após o tempo limite de 500 ms expirar, o `ninjaDelayedPromise` é resolvido, o que causa a execução da correspondência então ligar de volta.

Neste exemplo, por uma questão de simplicidade, trabalhamos apenas com o cenário otimista em que tudo vai bem. Mas o mundo real não é só raios de sol e arco-íris, então vamos ver como lidar com todos os tipos de problemas malucos que podem ocorrer.

6.3.3 Rejeitando promessas

Existem duas maneiras de rejeitar uma promessa: *explicitamente*, chamando o aprovado rejeitar método na função executora de uma promessa, e *implicitamente*, se durante o tratamento de uma promessa, ocorre uma exceção não tratada. Vamos começar nossa exploração com a seguinte lista.

Listagem 6.12 Rejeitando promessas explicitamente

```
promessa const = nova promessa ((resolver, rejeitar) => {
  rejeitar ("Rejeitar explicitamente uma promessa!");
});

promessa.então (
  () => falha ("Caminho feliz, não será chamado!"),
  error => pass ("Uma promessa foi rejeitada explicitamente!")
);
```



Uma promessa pode ser rejeitada explicitamente chamando a função de rejeição passada.



Se uma promessa for rejeitada, a segunda, erro, o retorno de chamada é invocado.

Podemos rejeitar explicitamente uma promessa, chamando o aprovado rejeitar método:

`rejeitar ("Rejeitar explicitamente uma promessa!")`. Se uma promessa for rejeitada, ao registrar retornos de chamada através do então método, o segundo, erro, retorno de chamada sempre será invocado.

Além disso, podemos usar uma sintaxe alternativa para lidar com rejeições de promessa, usando o pegar método, conforme mostrado na lista a seguir.

Listagem 6.13 Encadeando um pegar método

```
var promessa = nova promessa ((resolver, rejeitar) => {
  rejeitar ("Rejeitar explicitamente uma promessa"); });

promessa.then (() => falha ("Caminho feliz, não será chamado!"))
  .pegar (() => passar ("A promessa também foi rejeitada"));
```

Em vez de fornecer o segundo, erro, retorno de chamada, podemos encadear o método catch e passar para ele o callback de erro. O resultado final é o mesmo.

Como mostra a listagem 6.13, podemos encadear no pegar método após o então , para também fornecer um retorno de chamada de erro que será invocado quando uma promessa for rejeitada. Neste exemplo, é uma questão de estilo pessoal. Ambas as opções funcionam igualmente bem, mas mais tarde, ao trabalhar com cadeias de promessas, veremos um exemplo em que encadear o pegar método é útil.

Além da rejeição explícita (por meio do rejeitar chamada), uma promessa também pode ser rejeitada implicitamente, se ocorrer uma exceção durante seu processamento. Dê uma olhada no exemplo a seguir.

Listagem 6.14 Exceções rejeitam implicitamente uma promessa

```
promessa const = nova promessa ((resolver, rejeitar) => {
  undeclaredVariable ++;
});

promessa.then (() => falha ("Caminho feliz, não será chamado!"))
  .catch (error => pass ("A terceira promessa também foi rejeitada"));
```

Uma promessa é rejeitada implicitamente se uma exceção não tratada ocorrer quando processamento da promessa.

Se ocorrer uma exceção, o segundo, erro, o retorno de chamada é invocado.

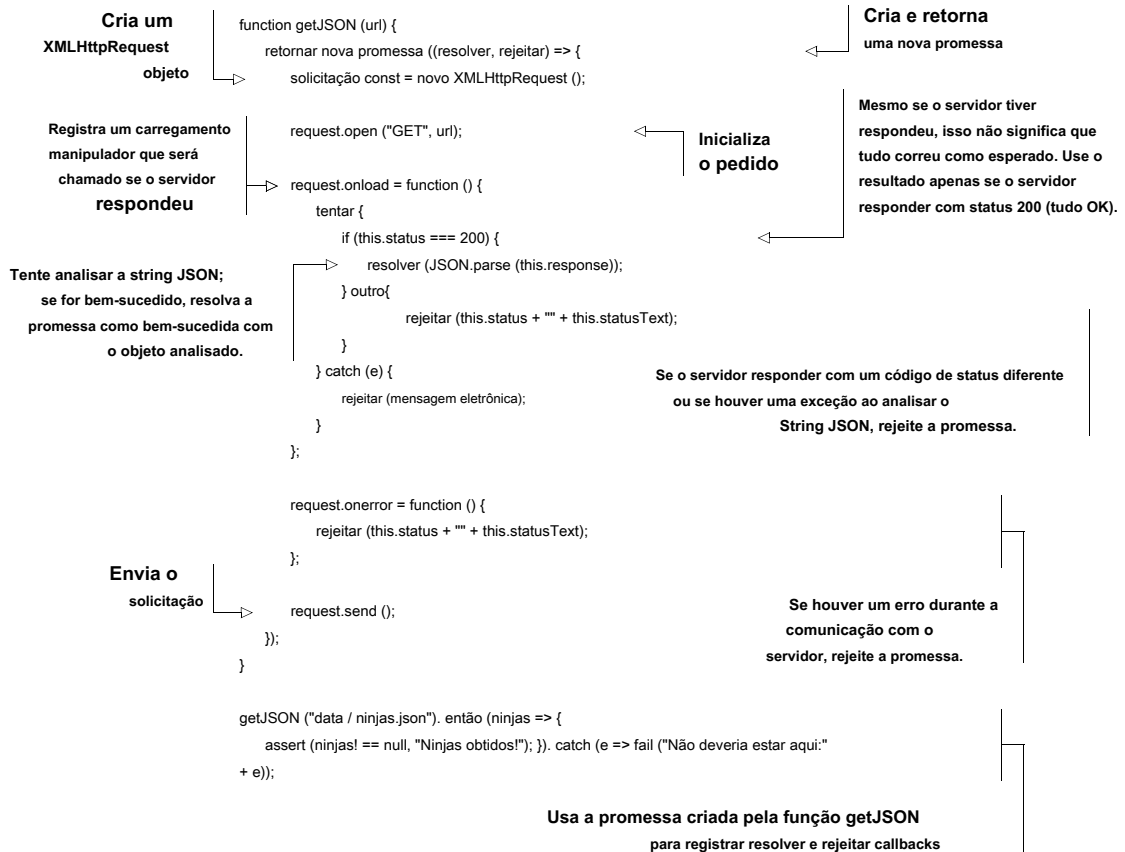
Dentro do corpo do executor da promessa, tentamos incrementar variável não declarada, uma variável que não está definida em nosso programa. Como esperado, isso resulta em uma exceção. Porque não há tentar pegar declaração dentro do corpo do executor, isso resulta em uma rejeição implícita da promessa atual, e o pegar o retorno de chamada é eventualmente invocado. Nesta situação, poderíamos ter facilmente fornecido o segundo retorno de chamada para o então método, e o efeito final seria o mesmo.

Essa maneira de tratar todos os problemas que acontecem ao trabalhar com as promessas de maneira uniforme é extremamente útil. Independentemente de como a promessa foi rejeitada, seja explicitamente chamando o rejeitar método ou mesmo implicitamente, se ocorrer uma exceção, todos os erros e motivos de rejeição são direcionados ao nosso callback de rejeição. Isso torna nossas vidas como desenvolvedores um pouco mais fáceis.

Agora que entendemos como as promessas funcionam e como agendar retornos de chamada de sucesso e falha, vamos pegar um cenário do mundo real, obtendo dados formatados em JSON de um servidor, e "prometê-los".

6.3.4 Criando nossa primeira promessa do mundo real

Uma das ações assíncronas mais comuns no cliente é buscar dados do servidor. Como tal, este é um excelente pequeno estudo de caso sobre o uso de promessas. Para a implementação subjacente, usaremos o integrado XMLHttpRequest objeto.

Listagem 6.15 Criando um `getJSON` promessa

NOTA Executar este exemplo, e todos os exemplos subsequentes que reutilizam esta função, requer um servidor em execução. Você pode, por exemplo, usar [www.npmjs.com/ package / http-server](http://www.npmjs.com/package/http-server) .

Nosso objetivo é criar um `getJSON` função que retorna uma promessa que nos permitirá registrar retornos de chamada de sucesso e falha para obter dados formatados em JSON de maneira assíncrona do servidor. Para a implementação subjacente, usamos o integrado XMLHttpRequest

objeto que oferece dois eventos: carregando e onerror. O carregando evento é acionado quando o navegador recebe uma resposta do servidor, e onerror é acionado quando ocorre um erro na comunicação. Esses manipuladores de eventos serão chamados de forma assíncrona pelo navegador, à medida que ocorrem.

Se ocorrer um erro na comunicação, definitivamente não conseguiremos obter nossos dados do servidor, portanto, a coisa mais honesta a fazer é rejeitar nossa promessa:

```
request.onerror = function () {  
  rejeitar (this.status + "" + this.statusText);  
};
```

Se recebermos uma resposta do servidor, temos que analisar essa resposta e considerar a situação exata. Sem entrar em muitos detalhes, um servidor pode responder com várias coisas, mas neste caso, nos preocupamos apenas se a resposta foi bem-sucedida (status 200). Se não for, rejeitamos novamente a promessa.

Mesmo que o servidor tenha respondido com sucesso com dados, isso ainda não significa que estamos livres. Como nosso objetivo era obter objetos formatados em JSON do servidor, o código JSON sempre poderia ter erros de sintaxe. É por isso que, ao chamar o `JSON.parse` método, envolvemos o código com um `try` para pegar demonstração. Se ocorrer uma exceção ao analisar a resposta do servidor, também rejeitaremos a promessa. Com isso, cuidamos de todos os cenários ruins que podem acontecer.

Se tudo correr conforme o planejado, e obtivermos nossos objetos com sucesso, podemos cumprir a promessa com segurança. Finalmente, podemos usar nosso `getJSON` função para buscar ninjas do servidor:

```
getJSON ("data / ninjas.json"). então (ninjas => {  
  assert (ninjas! == null, "Ninjas obtidos!");  
}). catch (e => fail ("Não deveria estar aqui:"  
+ e));
```

Nesse caso, temos três fontes potenciais de erros: erros no estabelecimento da comunicação entre o servidor e o cliente, o servidor respondendo com dados imprevistos (status de resposta inválido) e código JSON inválido. Mas da perspectiva do código que usa o `getJSON` função, não nos importamos com as especificações das fontes de erro. Nós fornecemos apenas um retorno de chamada que é acionado se tudo correr bem e os dados são recebidos corretamente, e um retorno de chamada que é acionado se ocorrer algum erro. Isso torna nossas vidas como desenvolvedores muito mais fáceis.

Agora vamos subir um degrau e explorar outra grande vantagem das promessas: sua composição elegante. Começaremos encadeando várias promessas em uma série de etapas distintas.

6.3.5 Encadeando promessas

Você já viu como lidar com uma sequência de etapas interdependentes leva à pirâmide da desgraça, uma sequência de callbacks profundamente aninhada e difícil de manter. As promessas são um passo para resolver esse problema, porque têm a capacidade de serem acorrentadas.

No início do capítulo, você viu como, usando o `então` método em uma promessa, podemos registrar um retorno de chamada que será executado se uma promessa for resolvida com sucesso. O que não dissemos é que chamar o `então` método também retorna uma nova promessa. Então

não há nada que nos impeça de acorrentar tantos então métodos como quisermos; veja o código a seguir.

Listagem 6.16 Encadeando promessas com `então`

```
getJSON("data / ninjas.json")
  . então (ninjas => getJSON (ninjas [0] .missionsUrl))
  . então (missões => getJSON (missões [0] .detailsUrl))
  . então (missão => afirmar (missão! == null, "Missão Ninja obtida!"))
  . catch (erro => falha ("Ocorreu um erro"));
```

Capturas promovem rejeições
em qualquer uma das etapas

Especifica vários
etapas sequenciais por
encadeando e depois liga

Isso cria uma sequência de promessas que serão, se tudo correr conforme o planejado, resolvidas uma após a outra. Primeiro, usamos o `getJSON("data / ninjas.json")` método para obter uma lista de ninjas do arquivo no servidor. Após recebermos essa lista, pegamos as informações sobre o primeiro ninja e solicitamos uma lista de missões para as quais o ninja está atribuído:

`getJSON(ninjas[0].missionsUrl)`. Mais tarde, quando essas missões chegam, fazemos mais um pedido para os detalhes da primeira missão: `getJSON(missões[0].detailsUrl)`. Finalmente, registramos os detalhes da missão.

Escrever esse código usando callbacks padrão resultaria em uma sequência profundamente aninhada de callbacks. Identificar a sequência exata de etapas não seria fácil, e Deus nos livre se decidirmos adicionar uma etapa extra em algum lugar no meio.

PEGANDO ERROS EM PROMESSAS EM CADEIA

Ao lidar com sequências de etapas assíncronas, um erro pode ocorrer em qualquer etapa. Já sabemos que podemos fornecer um segundo retorno de chamada de erro para o `então` ligar ou fazer uma corrente em um `pegar` chamada que leva um retorno de chamada de erro. Quando nos preocupamos apenas com o sucesso / falha de toda a sequência de etapas, pode ser entediante fornecer a cada etapa um tratamento especial de erros. Portanto, conforme mostrado na listagem 6.16, podemos tirar proveito do

pegar método que você viu antes:

```
... catch (error => fail ("Ocorreu um erro:" + err));
```

Se ocorrer uma falha em qualquer uma das promessas anteriores, o `pegar` método pega isso. Se nenhum erro ocorrer, o fluxo do programa continuará por ele, sem obstruções.

Lidar com uma sequência de etapas é muito mais agradável com promessas do que com retornos de chamada regulares, você não concorda? Mas ainda não é tão elegante quanto poderia ser. Chegaremos a isso em breve, mas primeiro vamos ver como usar promessas para cuidar das etapas assíncronas paralelas.

6.3.6 Esperando por uma série de promessas

Além de nos ajudar a lidar com sequências de etapas assíncronas interdependentes, as promessas reduzem significativamente a carga de espera por várias tarefas assíncronas independentes. Vamos revisitar nosso exemplo no qual queremos, em paralelo, reunir informações sobre os ninjas à nossa disposição, os meandros do plano e o mapa do

local onde o plano será colocado em movimento. Com promessas, isso é tão simples quanto mostrado na lista a seguir.

Listagem 6.17 Esperando por uma série de promessas com **Promise.all**

O resultado é uma série de sucessos valores, no ordem de aprovação em promessas.

```
Promise.all ([ getJSON ("data / ninjas.json"),
               getJSON ("data / mapInfo.json"),
               getJSON ("data / plan.json")]). então (resultados => {
  const ninjas = resultados [0], mapInfo = resultados [1], plano = resultados [2];

  afirmar (ninjas! == undefined
    && mapInfo! == undefined && plan! == undefined,
    "O plano está pronto para ser posto em prática!"); e cria uma nova promessa
}). catch (erro => {
  falha ("Um problema em realizar nosso plano!"); });
```

O método **Promise.all** leva uma série de promessas, que tem sucesso se tudo as promessas são bem-sucedidas e fracassam se uma única promessa falhar.

Como você pode ver, não precisamos nos preocupar com a ordem em que as tarefas são executadas e se algumas delas foram concluídas e outras não. Afirmamos que queremos esperar por uma série de promessas usando o integrado **Promise.all** método. Este método aceita uma série de promessas e cria um *nova* promessa que é resolvida com sucesso quando todas as promessas passadas são resolvidas, e rejeita se até mesmo uma das promessas falhar. O retorno de chamada de sucesso recebe uma matriz de valores de sucesso, um para cada uma das promessas passadas, em ordem. Reserve um minuto para apreciar a elegância do código que processa várias tarefas assíncronas paralelas com promessas.

O **Promise.all** método espera por todas as promessas em uma lista. Mas às vezes temos inúmeras promessas, mas nos preocupamos apenas com a primeira que é bem-sucedida (ou falha). Conheça o **Promise.race** método.

PROMESSAS DE CORRIDA

Imagine que temos um grupo de ninjas à nossa disposição e que queremos dar uma missão ao primeiro ninja que atender à nossa chamada. Ao lidar com promessas, podemos escrever algo como a lista a seguir.

Listagem 6.18 Promessas de corrida com **Promise.race**

```
Promise.race ([ getJSON ("data / yoshi.json"),
               getJSON ("data / hattori.json"),
               getJSON ("data / hanzo.json")])
. então (ninja => {
  assert (ninjal == null, ninja.name + "respondeu primeiro"); }). catch (error => fail ("Failure!"));
```

É simples assim. Não há necessidade de rastrear tudo manualmente. Nós usamos o **Promise.race** método para pegar uma série de promessas e retornar um método completo *nova* promessa que resolve ou rejeita assim que a primeira das promessas resolve ou rejeita.

Até agora, você viu como as promessas funcionam e como podemos usá-las para simplificar muito o tratamento de uma série de etapas assíncronas, em série ou em paralelo. Embora

as melhorias, quando comparadas aos retornos de chamada antigos em termos de tratamento de erros e elegância do código, são ótimas, o código prometido ainda não está no mesmo nível de elegância que o código síncrono simples. Na próxima seção, os dois grandes conceitos que introduzimos neste capítulo, *geradores* e *promessas*, vêm juntos para fornecer a simplicidade do código síncrono com a natureza não bloqueadora do código assíncrono.

6.4 Combinando geradores e promessas

Nesta seção, combinaremos geradores (e sua capacidade de pausar e retomar sua execução) com promessas, a fim de obter um código assíncrono mais elegante. Usaremos o exemplo de uma funcionalidade que permite aos usuários obter detalhes da missão de alto nível realizada pelo ninja mais popular. Os dados que representam os ninjas, os resumos de suas missões, bem como os detalhes das missões são armazenados em um servidor remoto, codificado em JSON.

Todas essas subtarefas são de longa duração e mutuamente dependentes. Se tivéssemos que implementá-los de maneira síncrona, obteríamos o seguinte código simples:

```
tentar {  
  const ninjas = syncGetJSON ("data / ninjas.json"); missões const = syncGetJSON (ninjas [0]  
    .missionsUrl);  
  const missionDetails = syncGetJSON (missões [0].detailsUrl); // Estude a descrição da missão  
  
} catch (e) {  
  // Oh não, não conseguimos obter os detalhes da missão  
}
```

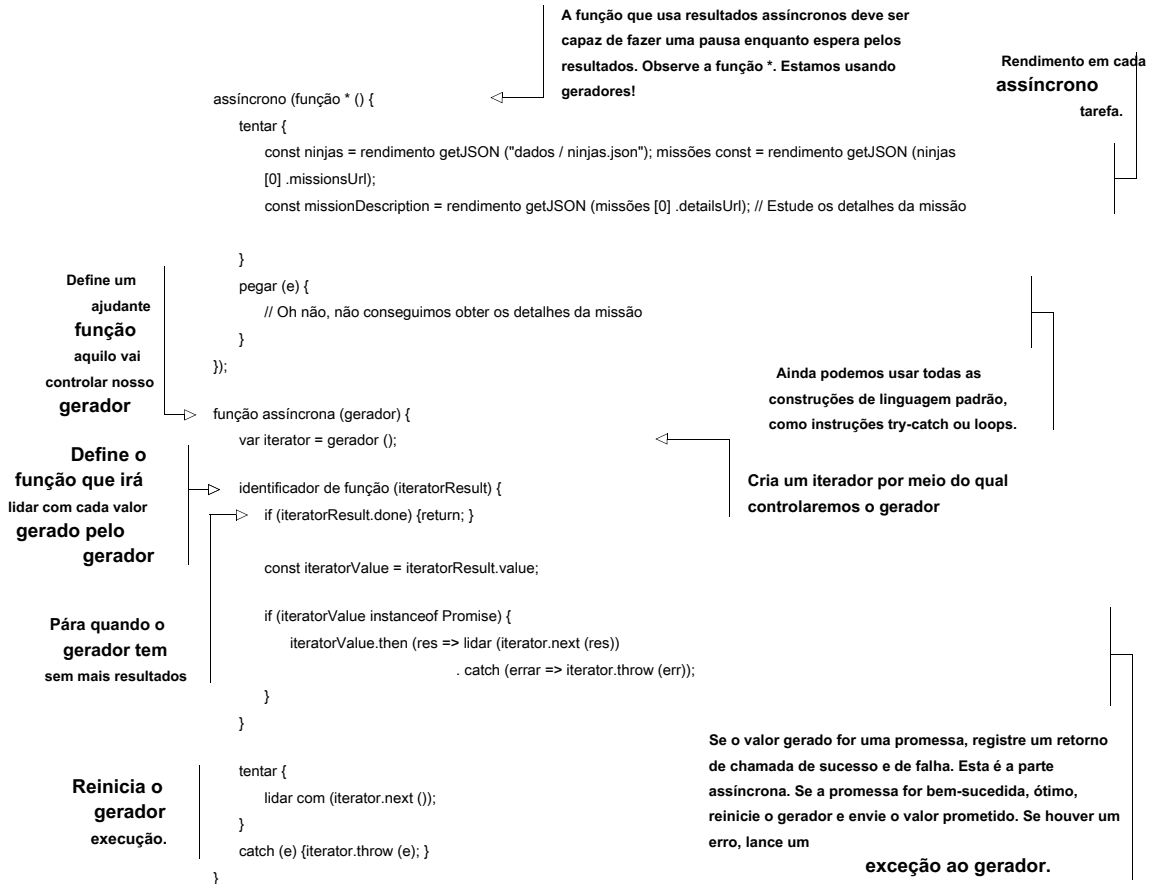
Embora esse código seja ótimo por sua simplicidade e tratamento de erros, ele bloqueia a IU, o que resulta em usuários insatisfeitos. Idealmente, gostaríamos de alterar esse código para que nenhum bloqueio ocorra durante uma tarefa de longa execução. Uma maneira de fazer isso é combinando geradores e promessas.

Como sabemos, ceder de um gerador suspende a execução do gerador sem bloqueio. Para acordar o gerador e continuar sua execução, temos que chamar o Próximo método no iterador do gerador. Promises, por outro lado, nos permitem especificar um callback que será acionado caso possamos obter o valor prometido e um callback que será acionado caso ocorra um erro.

A ideia, então, é combinar geradores e promessas da seguinte maneira: Colocamos o código que usa tarefas assíncronas em um gerador e executamos essa função do gerador. Quando alcançamos um ponto na execução do gerador que chama uma tarefa assíncrona, criamos uma promessa que representa o valor dessa tarefa assíncrona. Porque não temos ideia de quando essa promessa será resolvida (ou mesmo se será resolvida), neste ponto de execução do gerador, nós rendemos do gerador, para não causarmos bloqueio. Depois de um tempo, quando a promessa é cumprida, continuamos a execução de nosso gerador chamando o iterador Próximo método. Fazemos isso quantas vezes forem necessárias. Veja a lista a seguir para um exemplo prático.

Listagem 6.19

Combinando geradores e promessas



O assíncrono A função pega um gerador, o chama e cria um iterador que será usado para retomar a execução do gerador. Dentro de assíncrono função, nós declaramos um lidar função que lida com um valor de retorno do gerador - uma "iteração" de nosso iterador. Se o resultado do gerador é uma promessa que é resolvida com sucesso, usamos o iterador Próximo método para enviar o valor prometido de volta ao gerador e retomar a execução do gerador. Se ocorrer um erro e a promessa for rejeitada, lançamos esse erro para o gerador usando o iterador lançar método (disse que seria útil). Continuamos fazendo isso até que o gerador diga que está pronto.

NOTA Este é um esboço, uma quantidade mínima de código necessária para combinar geradores e promessas. Não recomendamos que você use esse código na produção.

Agora vamos dar uma olhada mais de perto no gerador. Na primeira invocação do iterador Próximo método, o gerador executa até o primeiro getJSON ("data / ninjas.json")

chamar. Esta chamada cria uma promessa que acabará por conter a lista de informações sobre o nosso ninjas. Como esse valor é buscado de forma assíncrona, não temos ideia de quanto tempo o navegador levará para obtê-lo. Mas sabemos uma coisa: não queremos bloquear a execução do aplicativo enquanto aguardamos. Por esse motivo, neste momento de execução, o gerador cede o controle, que pausa o gerador, e devolve o fluxo de controle à invocação do lidar função. Porque o valor gerado é um `getJSON` promessa, no lidar função, usando o `então` e

pegar métodos da promessa, registramos um sucesso e um retorno de chamada de erro e continuamos a execução. Com isso, o fluxo de controle sai da execução do lidar função e o corpo do assíncrono função, e continua após a chamada para o assíncrono função (em nosso caso, não há mais código depois, então ele fica inativo). Durante este tempo, nossa função gerador espera pacientemente suspensão, sem bloquear a execução do programa.

Muito, muito mais tarde, quando o navegador recebe uma resposta (positiva ou negativa), um dos retornos de chamada de promessa é invocado. Se a promessa foi resolvida com sucesso, o retorno de chamada de sucesso é invocado, o que, por sua vez, causa a execução do iterador Próximo método, que pede ao gerador outro valor. Isso tira o gerador da suspensão e envia para ele o valor passado pelo callback. Isso significa que entramos novamente no corpo do nosso gerador, após o primeiro produzir expressão, cujo valor se torna o ninjas lista que foi obtida de forma assíncrona do servidor. A execução da função do gerador continua, e o valor é atribuído ao

plano variável.

Na próxima linha do gerador, usamos alguns dos dados obtidos, `ninjas [0]` `.missionUrl`, para fazer outro `getJSON` chamada que cria outra promessa que deve conter uma lista de missões feitas pelo ninja mais popular. Novamente, como essa é uma tarefa assíncrona, não temos ideia de quanto tempo vai demorar, então, novamente entregamos a execução e repetimos todo o processo.

Este processo é repetido enquanto houver tarefas assíncronas no gerador. Isso era um pouco complexo, mas gostamos deste exemplo porque ele combina muitas coisas que você aprendeu até agora:

- *Funciona como objetos de primeira classe* - Enviamos uma função como um argumento para o assíncrono função.
- *Funções geradoras* — Usamos sua capacidade de suspender e retomar a execução.
- *Promessas* — Eles nos ajudam a lidar com o código assíncrono.
- *Callbacks* — Nós registramos retornos de chamada de sucesso e fracasso em nossas promessas.
- *Funções de seta* - Devido à sua simplicidade, para retornos de chamada, usamos as funções de seta.
- *Fechamentos* — O iterador, por meio do qual controlamos o gerador, é criado no assíncrono função, e a acessamos, por meio de fechamentos, nos retornos de chamada de promessa.

Agora que já passamos por todo o processo, vamos parar um minuto para avaliar como é muito mais elegante o código que implementa nossa lógica de negócios. Considere isto:

```

getJSON ("data / ninjas.json", (err, ninjas) => {
  if (err) {console.log ("Erro ao buscar ninjas", err); Retorna; }

  getJSON (ninjas [0] .missionsUrl, (err, missões) => {
    if (err) {console.log ("Erro ao localizar missões ninja", err); Retorna; } console.log (missões);

  })
});

```

Em vez de fluxo de controle e tratamento de erros mistos e código um pouco confuso, terminamos com algo assim:

```

assíncrono (função * () {
  tentar {
    const ninjas = rendimento getJSON ("dados / ninjas.json"); missões const = rendimento getJSON (ninjas
    [0] .missionsUrl);

    // Todas as informações recebidas
  }
  pegar (e) {
    //Ocorreu um erro
  }
});

```

Este resultado final combina as vantagens do código síncrono e assíncrono. Do código síncrono, temos a facilidade de compreensão e a capacidade de usar todos os mecanismos de controle de fluxo e tratamento de exceção padrão, como loops e tentar pegar declarações. Do código assíncrono, obtemos a natureza não bloqueadora; a execução de nosso aplicativo não é bloqueada enquanto espera por tarefas assíncronas de longa duração.

6.4.1 Olhando para o futuro - a função assíncrona

Observe que ainda tivemos que escrever algum código clichê; tivemos que desenvolver um assíncrono função que se encarrega de lidar com promessas e solicitar valores do gerador. Embora possamos escrever essa função apenas uma vez e, em seguida, reutilizá-la em todo o nosso código, seria ainda melhor se não tivéssemos que pensar sobre isso. As pessoas encarregadas de JavaScript estão bem cientes da utilidade da combinação de geradores e promessas, e querem tornar nossas vidas ainda mais fáceis construindo um suporte de linguagem direto para combinar geradores e promessas.

Para essas situações, o plano atual é incluir duas novas palavras-chave, `assíncrono` e `aguardam`, isso cuidaria desse código clichê. Em breve, poderemos escrever algo assim:

```

( assíncrono function () {
  tentar {
    const ninjas = aguardam getJSON ("data / ninjas.json"); missões const = aguardam getJSON (missões [0]
    .missionsUrl);

    console.log (missões);
  }
}

```

```
pegar (e) {  
  console.log ("Erro:", e);  
}  
}) ()
```

Nós usamos o assíncrono palavra-chave na frente da palavra-chave de função para especificar que esta função depende de valores assíncronos, e em cada lugar onde chamamos uma tarefa assíncrona, colocamos o `await` palavra-chave que diz ao mecanismo JavaScript, aguarde este resultado sem bloquear. No fundo, tudo acontece como discutimos anteriormente ao longo do capítulo, mas agora não precisamos nos preocupar com isso.

NOTA As funções assíncronas aparecerão na próxima parte do JavaScript. Atualmente nenhum navegador oferece suporte, mas você pode usar transpiladores como Babel ou Traceur se desejar usar `async` em seu código hoje.

6.5 *Resumo*

- Geradores são funções que geram sequências de valores - não todos de uma vez, mas por solicitação.
- Ao contrário das funções padrão, os geradores podem suspender e retomar sua execução. Depois que um gerador gera um valor, ele suspende sua execução sem bloquear o thread principal e espera pacientemente pela próxima solicitação.
- Um gerador é declarado colocando um asterisco (*) após a palavra-chave da função. Dentro do corpo do gerador, podemos usar o novo `yield` palavra-chave que produz um valor e suspende a execução do gerador. Se quisermos ceder a outro gerador, usamos o `yield` operador.
- Chamar um gerador cria um objeto iterador por meio do qual controlamos a execução do gerador. Solicitamos novos valores do gerador usando o iterador `next` método, e podemos até lançar exceções no gerador chamando o método do iterador `throw` método. Além disso, o `next` método pode ser usado para enviar valores para o gerador.
- Uma promessa é um espaço reservado para os resultados de um cálculo; é uma garantia de que eventualmente saberemos o resultado da computação, na maioria das vezes uma computação assíncrona. Uma promessa pode ter sucesso ou falhar e, depois de cumprida, não haverá mais mudanças.
- As promessas simplificam significativamente nossas negociações com tarefas assíncronas. Podemos facilmente trabalhar com sequências de etapas assíncronas interdependentes usando o `then` método para encadear promessas. O tratamento paralelo de várias etapas assíncronas também é bastante simplificado; nós usamos o `Promise.all` método.
- Podemos combinar geradores e promessas para lidar com tarefas assíncronas com a simplicidade do código síncrono.

6,6 Exercícios

- 1 Depois de executar o código a seguir, quais são os valores das variáveis a1 para a4?

```
function * EvenGenerator () {
  deixe num = 2;
  enquanto (verdadeiro) {
    rendimento num;
    num = num + 2;
  }
}

let generator = EvenGenerator ();

deixe a1 = generator.next (). value; deixe a2 = generator.next
(). value;
deixe a3 = EvenGenerator (). next (). valor; deixe a4 = generator.next ().
value;
```

- 2 Qual é o conteúdo do ninjas array depois de executar o seguinte código? (Dica: pense sobre como o para de loop pode ser implementado com um enquanto ciclo.)

```
function * NinjaGenerator () {
  produzir "Yoshi";
  retornar "Hattori";
  produzir "Hanzo";
}

var ninjas = [];
para (deixe ninja de NinjaGenerator ()) {
  ninjas.push (ninja);
}

ninjas;
```

- 3 Quais são os valores das variáveis a1 e a2, depois de executar o seguinte código?

```
function * Gen (val) {
  val = rendimento val * 2; rendimento
  val;
}

deixe gerador = Gen (2);
deixe a1 = generator.next (3) .value; deixe a2 = generator.next
(4) .value;
```

- 4 Qual é a saída do código a seguir?

```
promessa const = nova promessa ((resolver, rejeitar) => {
  rejeitar ("Hattori");
});

promessa.então (val => alert ("Sucesso:" + val))
  . catch (e => alert ("Erro:" + e));
```


5 Qual é a saída do código a seguir?

```
promessa const = nova promessa ((resolver, rejeitar) => {  
  resolver ("Hattori");  
  setTimeout (() => rejeitar ("Yoshi"), 500);  
});  
  
promessa.então (val => alert ("Sucesso:" + val))  
  . catch (e => alert ("Erro:" + e));
```

Cavando em objetos e fortalecendo seu código

N Como você aprendeu os meandros das funções, continuaremos nossa exploração do JavaScript examinando mais de perto os fundamentos dos objetos, no capítulo 7.

No capítulo 8, estudaremos como controlar o acesso e monitorar nossos objetos com getters e setters, e com proxies, um tipo completamente novo de objeto em JavaScript.

Vamos dar uma olhada nas coleções no capítulo 9 - as tradicionais, como matrizes, bem como tipos completamente novos, como mapas e conjuntos.

A partir daí, passaremos para as expressões regulares no capítulo 10. Você aprenderá que muitas tarefas que costumavam exigir resmas de código para serem realizadas podem ser condensadas em um simples punhado de instruções por meio do uso adequado de expressões regulares JavaScript.

Finalmente, no capítulo 11, mostraremos como estruturar seus aplicativos JavaScript em unidades menores e bem organizadas de funcionalidade, chamadas de módulos.



Orientação do objeto com protótipos

Este capítulo cobre

- Explorando protótipos
- Usando funções como construtores
- Estendendo objetos com protótipos
- Evitando pegadinhas comuns
- Construindo classes com herança

Você aprendeu que as funções são objetos de primeira classe em JavaScript, que os encerramentos os tornam incrivelmente versáteis e úteis e que você pode combinar funções geradoras com promessas para resolver o problema do código assíncrono. Agora estamos prontos para lidar com outro aspecto importante do JavaScript: protótipos de objetos.

UMA *protótipo* é um objeto ao qual a pesquisa de uma propriedade específica pode ser delegada. Protótipos são um meio conveniente de definir propriedades e funcionalidades que estarão automaticamente acessíveis a outros objetos. Os protótipos têm um propósito semelhante ao das classes nas linguagens orientadas a objetos clássicas. Na verdade, o principal uso de protótipos em JavaScript é na produção de código escrito em um formato orientado a objetos

forma, semelhante, mas não exatamente como, código em linguagens baseadas em classe mais convencionais, como Java ou C#.

Neste capítulo, vamos nos aprofundar em como os protótipos funcionam, estudar sua conexão com funções construtoras e ver como imitar alguns dos recursos orientados a objetos frequentemente usados em outras linguagens orientadas a objetos mais convencionais. Também exploraremos uma nova adição ao JavaScript, o *await* palavra-chave, que não traz exatamente classes completas para o JavaScript, mas nos permite simular classes e herança facilmente. Vamos começar a explorar.

.....

Como você testa se um objeto tem acesso a um determinado
propriedade?

Você sabe? Por que uma cadeia de protótipos é importante para trabalhar com
objetos em JavaScript?

As classes ES6 mudam a forma como o JavaScript funciona com objetos?

.....

7.1 *Compreendendo os protótipos*

Em JavaScript, os objetos são coleções de propriedades nomeadas com valores. Por exemplo, podemos criar facilmente novos objetos com notação literal de objeto:

```
let obj = {
  prop1: 1,
  prop2: function () {},
  prop3: {}
}
```

Atribui um valor simples

Atribui uma função

Atribui outro objeto

Como podemos ver, as propriedades do objeto podem ser valores simples (como números ou strings), funções e até mesmo outros objetos. Além disso, JavaScript é uma linguagem altamente dinâmica e as propriedades atribuídas a um objeto podem ser facilmente alteradas, modificando e excluindo as propriedades existentes:

```
obj.prop1 = 1;
obj.prop1 = [];
delete obj.prop2;
```

prop1 armazena um número simples.

Atribui um valor de um completamente diferente tipo, aqui uma matriz

Remove a propriedade do objeto

Podemos até adicionar propriedades completamente novas:

```
obj.prop4 = "Olá";
```

Adiciona uma propriedade completamente nova

No final, todas essas modificações deixaram nosso objeto simples no seguinte estado:

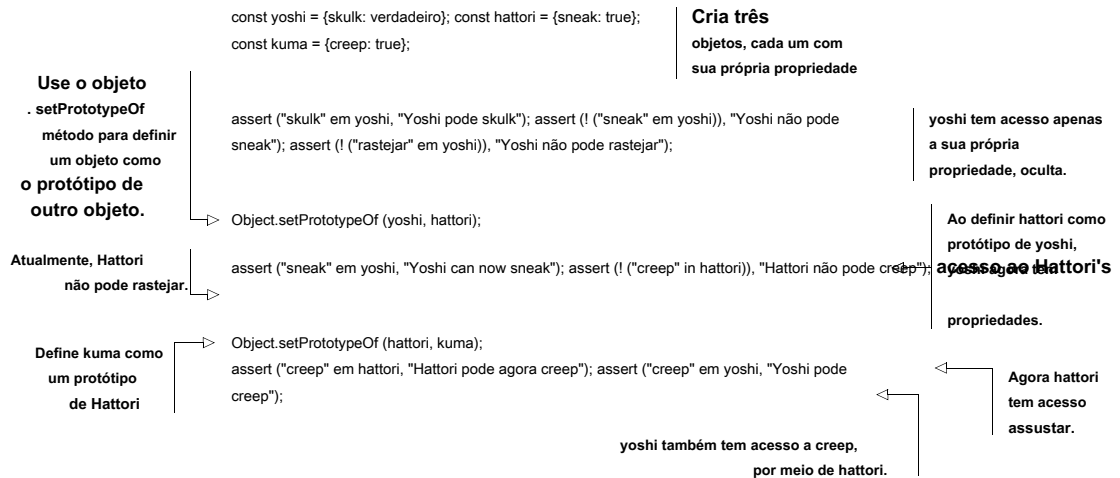
```
{
  prop1: [],
  prop3: {},
  prop4: "Olá"
};
```

Ao desenvolver software, nos esforçamos para não reinventar a roda, portanto, queremos reutilizar o máximo de código possível. Uma forma de reutilização de código que também ajuda a organizar nossos programas é *herança*, estendendo as características de um objeto para outro. Em JavaScript, a herança é implementada com prototipagem.

A ideia de prototipagem é simples. Cada objeto pode ter uma referência ao seu *protótipo*, um objeto ao qual a busca por uma propriedade particular pode ser delegada, se o próprio objeto não tiver essa propriedade. Imagine que você está participando de um teste de jogo com um grupo de pessoas e que o apresentador do programa faz uma pergunta. Se você sabe a resposta, você a dá imediatamente; se não souber, pergunte à pessoa ao seu lado. É simples assim.

Vamos dar uma olhada na lista a seguir.

Listagem 7.1 Com protótipos, objetos podem acessar propriedades de outros objetos



Neste exemplo, começamos criando três objetos: yoshi, hattori, e kuma. Cada um tem uma propriedade específica acessível apenas para aquele objeto: Apenas yoshi posso esquivar-se, só Hattori posso esgueirar-se, e somente Kuma posso rastejar. Veja a figura 7.1.

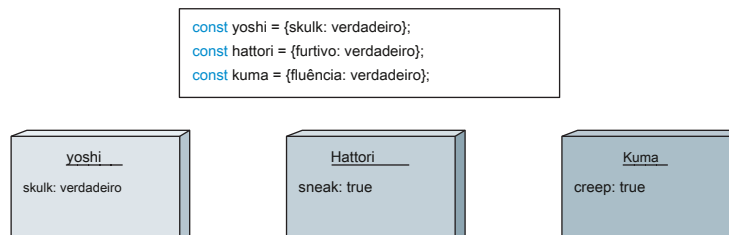


Figura 7.1 Inicialmente, cada objeto tem acesso apenas às suas próprias propriedades.

Para testar se um objeto tem acesso a uma propriedade particular, podemos usar o `in` operador. Por exemplo, executando `skulk` em `yoshi` retorna verdade, Porque `yoshi` tem acesso ao `esconder-se` propriedade; enquanto executando `esgueirar-se` em `yoshi` retorna falso.

Em JavaScript, a propriedade `prototype` do objeto é uma propriedade interna que não pode ser acessada diretamente (então a marcamos com `[[protótipo]]`). Em vez disso, o método integrado `Object.setPrototypeOf` recebe dois argumentos de objeto e define o segundo objeto como o protótipo do primeiro. Por exemplo, ligando `Object.setPrototypeOf(yoshi, hattori)`; estabelece `Hattori` como um protótipo de `yoshi`.

Como resultado, sempre que perguntamos `yoshi` por uma propriedade que não tem, `yoshi` delegados que procuram `hattori`. Podemos acessar `Hattori` de `esgueirar-se` propriedade através de `yoshi`.

Veja a figura 7.2.

Podemos fazer algo semelhante com `Hattori` e `kuma`. Usando o `Object.setPrototypeOf` método, podemos definir `Kuma` como o protótipo de `hattori`. Se então perguntarmos `Hattori` por uma propriedade que ele não possui, essa pesquisa será delegada a `kuma`. Nesse caso, `Hattori` agora tem acesso a `Kuma` de `rastejar` propriedade. Veja a figura 7.3.

É importante enfatizar que todo objeto pode ter um protótipo, e o protótipo de um objeto também pode ter um protótipo, e assim por diante, formando um *cadeia de protótipo*. A delegação de pesquisa para uma propriedade específica ocorre em toda a cadeia e só para quando não há mais protótipos para explorar. Por exemplo, como mostrado na figura 7.3, perguntando `yoshi` pelo valor do `rastejar` propriedade aciona a pesquisa pela propriedade primeiro em `yoshi`. Como a propriedade não foi encontrada, `yoshi` protótipo de, `hattori`, é pesquisado. De novo, `Hattori` não tem uma propriedade chamada `rastejar`, então `Hattori` protótipo de, `kuma`, é pesquisado e a propriedade é finalmente encontrada.

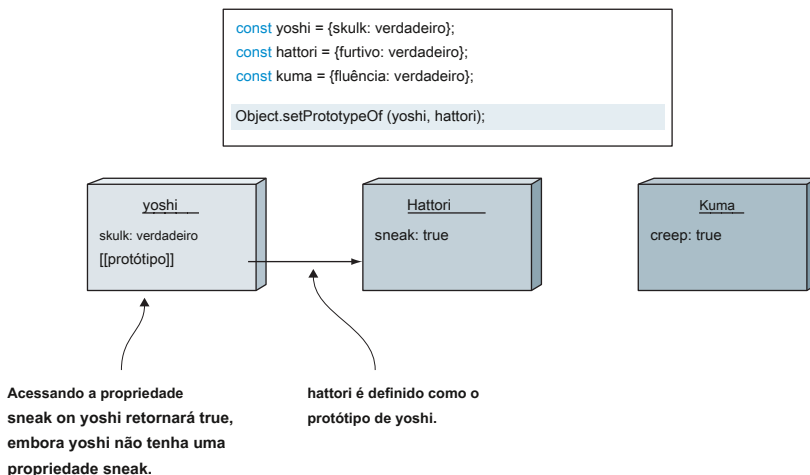


Figura 7.2 Quando acessamos uma propriedade que o objeto não possui, o protótipo do objeto é pesquisado por essa propriedade. Aqui, podemos acessar **Hattori** de **esgueirar-se** propriedade através de **yoshi**, Porque **yoshi** é **Hattori** protótipo de.

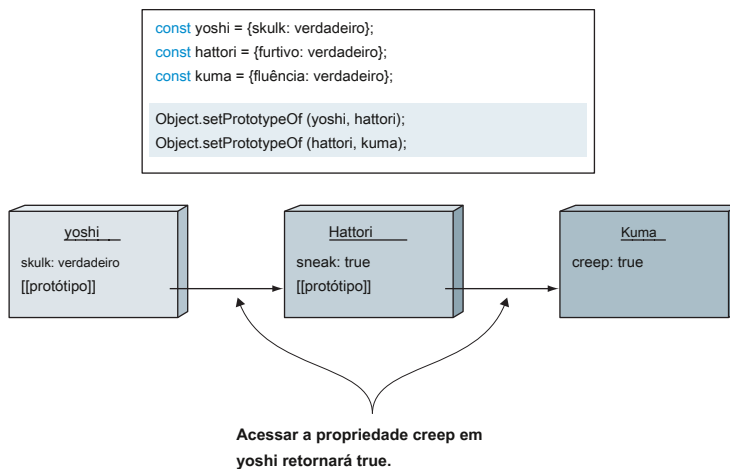


Figura 7.3 A busca por uma determinada propriedade pára quando não há mais protótipos para explorar. Acessando **yoshi.creep** aciona a pesquisa primeiro em **yoshi**, então em **hattori**, e finalmente em **kuma**.

Agora que temos uma ideia básica de como a busca por uma propriedade específica ocorre por meio da cadeia de protótipos, vamos ver como os protótipos são usados na construção de novos objetos com funções construtoras.

7.2 Construção de objetos e protótipos

A maneira mais simples de criar um novo objeto é com uma instrução como esta:

```
guerreiro const = {};
```

Isso cria um objeto novo e vazio, que podemos preencher com propriedades por meio de instruções de atribuição:

```
guerreiro const = {}; warrior.name = 'Saito';
```

```
warrior.occupation = 'atirador';
```

Mas aqueles que vêm de um background orientado a objetos podem perder o encapsulamento e a estruturação que vem com um construtor de classe, uma função que serve para inicializar um objeto a um estado inicial conhecido. Afinal, se vamos criar várias instâncias do mesmo tipo de objeto, atribuir as propriedades individualmente não é apenas tedioso, mas também altamente sujeito a erros. Gostaríamos de poder consolidar o conjunto de propriedades e métodos para uma classe de objetos em um só lugar.

JavaScript fornece esse mecanismo, embora em uma forma diferente da maioria das outras linguagens. Como as linguagens orientadas a objetos, como Java e C++, JavaScript emprega o novo operador para instanciar novos objetos por meio de construtores, mas não há definição de classe verdadeira em JavaScript. Em vez disso, o novo operador, aplicado a uma função construtora (como você viu no capítulo 3), dispara a criação de um objeto recém-alocado.

O que não aprendemos nos capítulos anteriores é que cada função tem um objeto protótipo que é definido automaticamente como o protótipo dos objetos criados com aquela função. Vamos ver como isso funciona na lista a seguir.

Listagem 7.2

Criando um **novo** instância com um método prototipado

```
função Ninja () {}
Ninja.prototype.swingSword = function () {
    return true;
};

const ninja1 = Ninja (); assert (ninja1 === undefined,

    "Nenhuma instância de Ninja criada.");

const ninja2 = novo Ninja (); afirmar (ninja2 &&

    ninja2.swingSword &&
    ninja2.swingSword (),
    "A instância existe e o método pode ser chamado.");
```

Define uma função que não faz nada e não retorna nada

Cada função tem um objeto de protótipo embutido, que podemos modificar livremente.

Chama a função como uma função. O teste confirma que nada parece acontecer.

Chama a função como um construtor. O teste confirma que não apenas uma nova instância de objeto é criada, mas possui o método do protótipo da função.

Neste código, definimos uma função aparentemente não fazer nada chamada `Ninja` que invocaremos de duas maneiras: como uma função "normal", `const ninja1 = Ninja ();` e como uma construção `tor, const ninja2 = novo Ninja () ;`.

Quando a função é criada, ela imediatamente obtém um novo objeto atribuído ao seu objeto protótipo, um objeto que podemos estender como qualquer outro objeto. Neste caso, adicionamos um `swingSword` método para isso:

```
Ninja.prototype.swingSword = function () {
    return true;
};
```

Em seguida, testamos a função. Primeiro chamamos a função normalmente e armazenamos seu resultado na variável `ninja1`. Olhando para o corpo da função, vemos que ela não retorna nenhum valor, então esperamos `ninja1` para testar como `Indefinido`, que afirmamos ser verdade. Como uma função simples, `Ninja` não parece ser tão útil.

Em seguida, chamamos a função por meio do novo operador, invocando-o como um *construtor*, e algo completamente diferente acontece. A função é chamada mais uma vez, mas desta vez um objeto recém-alocado foi criado e definido como o contexto da função (e está acessível através do esta palavra-chave). O resultado retornou do novo operador é uma referência a este novo objeto. Nós então testamos isso `ninja2` tem uma referência ao objeto recém-criado, e que esse objeto tem um `swingSword` método que podemos chamar. Veja a figura 7.4 para uma visão geral do estado atual do aplicativo.

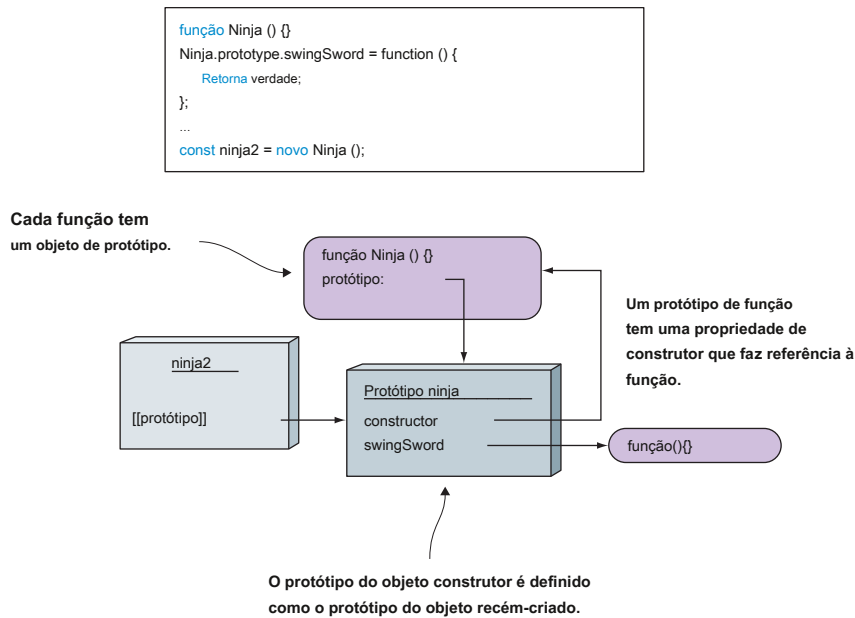


Figura 7.4 Cada função, quando criada, obtém um novo objeto de protótipo. Quando usamos uma função como construtor, o protótipo do objeto construído é definido como o protótipo da função.

Como você pode ver, uma função, quando criada, obtém um novo objeto que é atribuído à sua propriedade prototype. O objeto de protótipo inicialmente tem apenas uma propriedade, construtor, que faz referência à função (vamos revisar o construtor propriedade mais tarde).

Quando usamos uma função como construtor (por exemplo, chamando novo Ninja ()), o protótipo do objeto recém-construído é definido como o objeto referenciado pelo protótipo da função construtora.

Neste exemplo, estendemos o Ninja.prototype com o swingSword método, e quando o ninja2 objeto é criado, seu protótipo propriedade está definida para Ninja protótipo de. Portanto, quando tentamos acessar o swingSword propriedade em ninja2, a busca por essa propriedade é delegada ao Ninja objeto de protótipo. Notar que *todo* objetos criados com o Ninja construtor terá acesso ao swingSword método. Agora isso é reutilização de código!

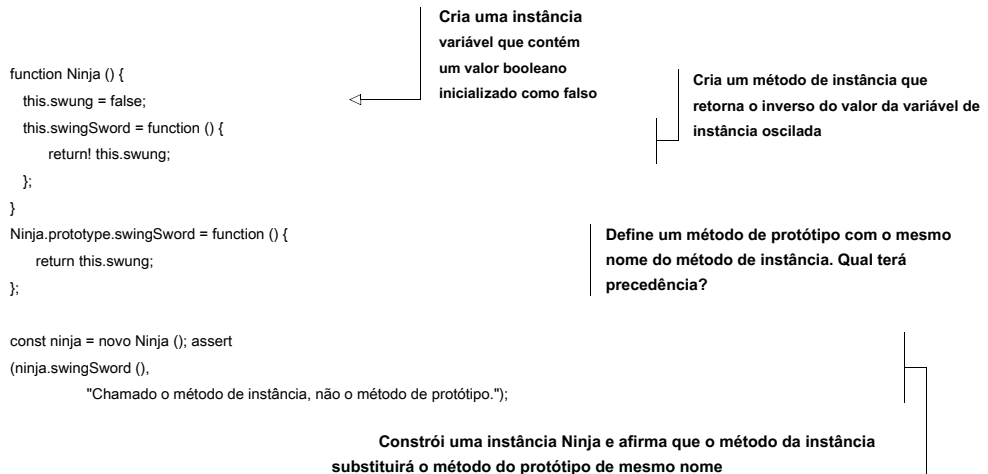
O swingSword método é uma propriedade do Ninja protótipo de, e não uma propriedade de ninja instâncias. Vamos explorar essa diferença entre propriedades de instância e propriedades de protótipo.

7.2.1 Propriedades da instância

Quando a função é chamada como um construtor por meio do novo operador, seu contexto é definido como a nova instância do objeto. Além de expor propriedades por meio do protótipo, podemos

inicializar valores dentro da função construtora por meio do este parâmetro. Vamos examinar a criação de tais propriedades de instância na próxima listagem.

Listagem 7.3 Observando a precedência das atividades de inicialização



A Listagem 7.3 é semelhante ao exemplo anterior em que definimos um `swingSword` método adicionando-o ao protótipo propriedade do construtor:

```
Ninja.prototype.swingSword = function () {
  return this.swung;
};
```

Mas também adicionamos um método com nome idêntico dentro da própria função construtora:

```
function Ninja () {
  this.swung = false;
  this.swingSword = function () {
    return! this.swung;
  };
}
```

Os dois métodos são definidos para retornar resultados opostos para que possamos dizer qual será chamado.

NOTA Isso não é algo que aconselharíamos fazer no código do mundo real; muito pelo contrário. Estamos fazendo isso aqui apenas para demonstrar a precedência das propriedades.

Quando você executa o teste, vê que ele passa! Isso mostra que os membros da instância irão ocultar propriedades com o mesmo nome definido no protótipo. Veja a figura 7.5.

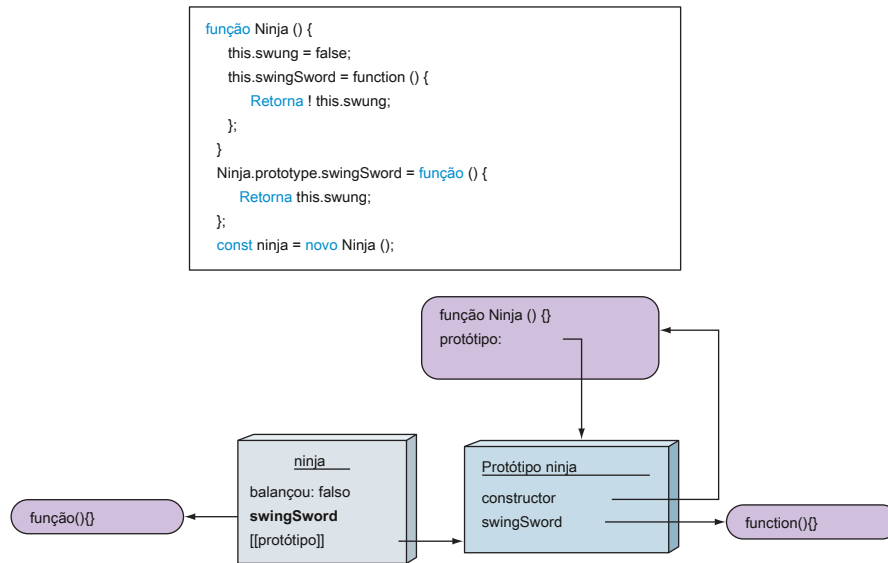


Figura 7.5 Se uma propriedade puder ser encontrada na própria instância, o protótipo nem mesmo é consultado!

Dentro da função construtora, o esta palavra-chave refere-se ao objeto recém-criado, portanto, as propriedades adicionadas dentro do construtor são criadas diretamente no novo ninja instância. Mais tarde, quando acessarmos a propriedade swingSword sobre ninja, não há necessidade de atravessar a cadeia de protótipo (conforme mostrado na figura 7.4); a propriedade criada no construtor é imediatamente encontrada e retornada (consulte a figura 7.5).

Isso tem um efeito colateral interessante. Dê uma olhada na figura 7.6, que mostra o estado do aplicativo se criarmos três ninja instâncias.

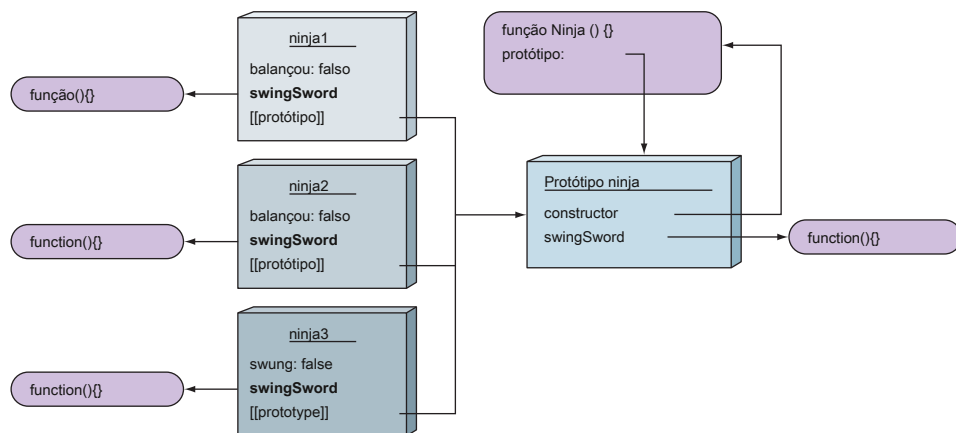


Figura 7.6 Cada instância obtém sua própria versão das propriedades criadas no construtor, mas todas elas têm acesso às mesmas propriedades do protótipo.

Como você pode ver, cada ninja instância obtém sua própria versão das propriedades que foram criadas dentro do construtor, enquanto todas têm acesso às mesmas propriedades do protótipo. Isso é bom para propriedades de valor (por exemplo, `balançado`) que são específicos para cada instância do objeto. Mas, em certos casos, pode ser problemático para métodos.

Neste exemplo, teríamos três versões do `swingSword` método que todos executam a mesma lógica. Isso não é um problema se criarmos alguns objetos, mas é algo a que prestar atenção se planejamos criar um grande número de objetos. Como cada cópia de método se comporta da mesma forma, a criação de várias cópias geralmente não faz sentido, porque apenas consome mais memória. Claro, em geral, o mecanismo JavaScript pode realizar algumas otimizações, mas isso não é algo em que se confiar. Dessa perspectiva, faz sentido colocar métodos de objeto apenas no protótipo da função, pois dessa forma temos um único método compartilhado por todas as instâncias de objeto.

NOTA Lembre-se do capítulo 5 sobre encerramentos: os métodos definidos nas funções construtoras nos permitem imitar variáveis de objetos privados. Se isso for algo de que precisamos, especificar métodos dentro dos construtores é o único caminho a percorrer.

7.2.2 Efeitos colaterais da natureza dinâmica do JavaScript

Você já viu que JavaScript é uma linguagem dinâmica na qual as propriedades podem ser facilmente adicionadas, removidas e modificadas à vontade. A mesma coisa vale para protótipos, tanto protótipos de função quanto protótipos de objeto. Veja a lista a seguir.

Listagem 7.4 Com protótipos, tudo pode ser alterado em tempo de execução

<pre>function Ninja () { this.swing = true; }</pre>	Define um construtor que cria um Ninja com uma única propriedade booleana	Cria uma instância de Ninja chamando a função do construtor via o “novo” operador
<pre>const ninja1 = novo Ninja ();</pre>		
<pre>Ninja.prototype.swingSword = function () { return this.swing; }; assert (ninja1.swingSword (), "O método existe, mesmo fora de serviço.");</pre>	Adiciona um método ao protótipo após o objeto foi criado	Mostra que o método existe no objeto
<pre>Ninja.prototype = { pierce: function () { return true; } }</pre>	Completamente substitui o do Ninja protótipo com um novo objeto através do método de perfuração	Embora tenhamos substituído completamente o protótipo do construtor Ninja, nosso Ninja ainda pode brandir uma espada, porque mantém uma referência ao antigo protótipo Ninja.
<pre>assert (ninja1.swingSword (), "Nosso ninja ainda pode balançar!");</pre>		
<pre>const ninja2 = novo Ninja (); assert (ninja2.pierce (), "Ninjas recém-criados podem perfurar"); assert (!ninja2.swingSword, "Mas eles não podem balançar!");</pre>		Ninjas recém-criados referenciar o novo protótipo, para que possam perfurar, mas não balançar.

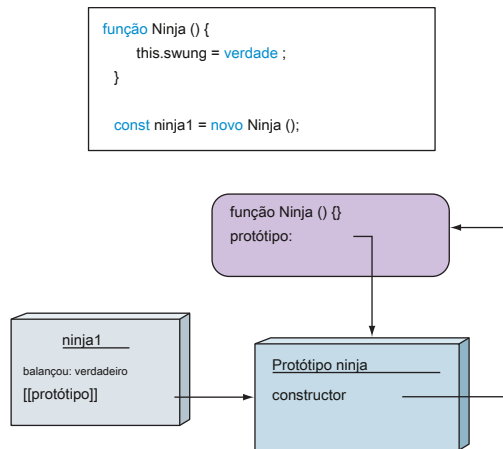


Figura 7.7 Após a construção, **ninja1** tem a propriedade **balançado**, e seu protótipo é o **Ninja** protótipo que tem apenas um **constructor** propriedade.

Aqui, novamente definimos um Ninja construtor e continue a usá-lo para criar uma instância de objeto. O estado da aplicação neste momento é mostrado na figura 7.7.

Depois de a instância foi criada, nós adicionamos um swingSword método para o protótipo. Em seguida, executamos um teste para mostrar que a alteração que fizemos no protótipo depois que o objeto foi construído entra em vigor. O estado atual do aplicativo é mostrado na figura 7.8.

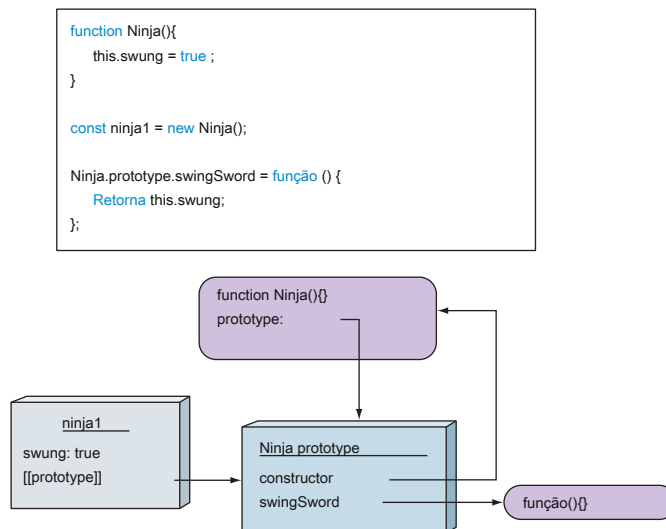


Figura 7.8 Porque o **ninja1** instância faz referência a **Ninja** protótipo, mesmo as alterações feitas depois que a instância foi construída são acessíveis.

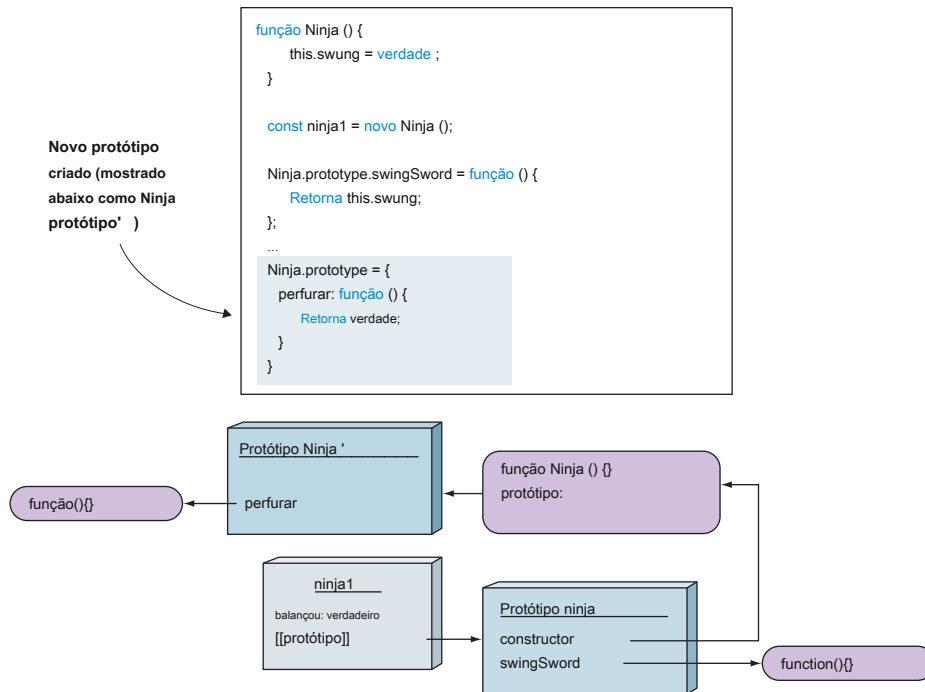


Figura 7.9 O protótipo da função pode ser substituído à vontade. As instâncias já construídas fazem referência ao antigo protótipo!

Mais tarde, substituímos o Ninja protótipo da função atribuindo-o a um objeto completamente novo que tem um pierce método. Isso resulta no estado do aplicativo mostrado na figura 7.9.

Como você pode ver, embora o Ninja função não faz referência ao antigo Ninja protótipo, o antigo protótipo ainda é mantido vivo pelo ninja1 instância, que ainda pode, através da cadeia de protótipos, acessar o swingSword método. Mas se criarmos novos objetos após essa troca de protótipo, o estado da aplicação será como mostrado na figura 7.10.

A referência entre um objeto e o protótipo da função é estabelecida no momento da instanciação do objeto. Objetos recém-criados terão uma referência ao novo protótipo e terão acesso ao perfurar , enquanto os objetos antigos pré-mudança de protótipo mantêm seu protótipo original, balançando suas espadas alegremente.

Exploramos como os protótipos funcionam e como estão relacionados à instanciação de objetos. Bem feito! Agora, respire rapidamente, para que possamos continuar aprendendo mais sobre a natureza desses objetos.

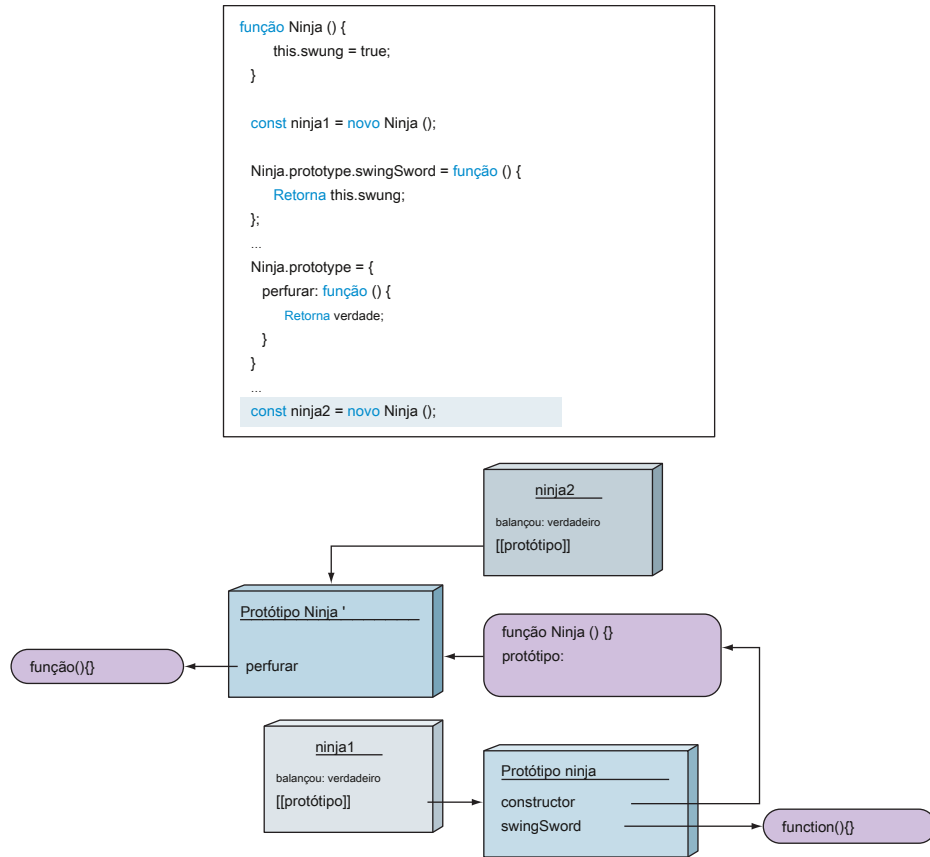


Figura 7.10 Todas as instâncias recém-criadas fazem referência ao novo protótipo.

7.2.3 Digitação de objetos por meio de construtores

Embora seja ótimo saber como JavaScript usa o protótipo para encontrar as referências de propriedade corretas, também é útil saber qual função construiu uma instância de objeto. Como você viu anteriormente, o construtor de um objeto está disponível por meio da propriedade constructor do protótipo da função do construtor. Por exemplo, figura

7.11 mostra o estado do aplicativo quando instanciamos um objeto com o Ninja construtor.

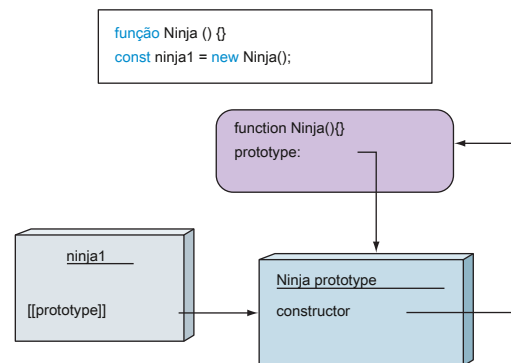


Figura 7.11 O objeto de protótipo de cada função tem uma propriedade de construtor que faz referência à função.

Usando o construtor propriedade, podemos acessar a função que foi usada para criar o objeto. Essas informações podem ser usadas como forma de verificação de tipo, conforme mostrado na listagem a seguir.

Listagem 7.5 Examinando o tipo de uma instância e seu construtor

```
função Ninja () {}
const ninja = novo Ninja ();

assert (typeof ninja === "objeto",
  "O tipo da instância é objeto."); assert (instância ninja de Ninja,
  "instanceof identifica o construtor."); assert (ninja.constructor === Ninja,
  "O objeto ninja foi criado pela função Ninja.");
```

Testa o tipo de ninja por meio de **typeof**. Isso nos diz que é um objeto, mas não muito mais.

Testa o tipo de ninja por meio de **instanceof**. Isso fornece **mais informações - que foi construído a partir de Ninja**.

Testa o tipo de ninja por meio da referência do construtor. Isso fornece uma referência à função do construtor.

Definimos um construtor e criamos uma instância de objeto usando-o. Em seguida, examinamos o tipo da instância usando o tipo de operador. Isso não revela muito, pois todas as instâncias serão objetos, portanto, sempre retornando objeto como resultado. Muito mais interessante é o **instanceof** de operador, que nos dá uma maneira de determinar se uma instância foi criada por um construtor de função particular. Você aprenderá mais sobre como o **instanceof** de operador trabalhará posteriormente neste capítulo.

Além disso, podemos usar o construtor propriedade, que agora sabemos que é acessível a todas as instâncias, como uma referência à função original que a criou. Podemos usar isso para verificar a origem da instância (tanto quanto podemos com o **instanceof** de operador).

Além disso, como esta é apenas uma referência ao construtor original, podemos instanciar um novo Ninja objeto usando-o, conforme mostrado na próxima lista.

Listagem 7.6 Instanciar um novo objeto usando uma referência a um construtor

```
função Ninja () {}

const ninja = novo Ninja ();
const ninja2 = novo ninja.constructor ();

assert (instância ninja2 de Ninja, "É um Ninja!"); afirmar (ninja! == ninja2, "Mas não o mesmo Ninja!");
```

Constrói um segundo Ninja desde o primeiro

Prova o novo Ninja-ness do objeto

Eles não são o mesmo objeto, mas duas instâncias distintas.

Aqui, definimos um construtor e criamos uma instância usando esse construtor. Então usamos o construtor propriedade da instância criada para construir uma segunda instância. O teste mostra que um segundo Ninja foi construído e que a variável não aponta meramente para a mesma instância.

O que é especialmente interessante é que podemos fazer isso sem nem mesmo ter acesso à função original; podemos usar a referência completamente nos bastidores, mesmo se o construtor original não estiver mais no escopo.

NOTA Apesar de construtor propriedade de um objeto pode ser alterada, fazer isso não tem nenhum propósito construtivo imediato ou óbvio (embora possamos pensar em alguns maliciosos). A razão de ser da propriedade é indicar de onde o objeto foi construído. Se o

construtor propriedade é substituída, o valor original é perdido.

Tudo isso é útil, mas acabamos de arranhar a superfície dos superpoderes que os protótipos nos conferem. Agora as coisas ficam interessantes.

7,3 Atingindo a herança

Herança é uma forma de reutilização na qual novos objetos têm acesso às propriedades dos objetos existentes. Isso nos ajuda a evitar a necessidade de repetir código e dados em nossa base de código. Em JavaScript, a herança funciona de maneira um pouco diferente do que em outras linguagens orientadas a objetos populares. Considere a seguinte lista, na qual tentamos obter herança.

Listagem 7.7 Tentando obter herança com protótipos	
<pre>função Person () {} Person.prototype.dance = function () {};</pre>	Define uma pessoa que dança por meio de um construtor e seu protótipo
<pre>função Ninja () {} Ninja.prototype = {dança: Person.prototype.dance};</pre>	Define um Ninja
<pre>const ninja = novo Ninja (); assert (instância ninja de Ninja,</pre>	Tentativas de fazer do Ninja uma pessoa dançarina copiando a dança
<pre>"ninja recebe funcionalidade do protótipo Ninja"); assert (instância ninja de Pessoa, "... e o protótipo da Pessoa"); assert (ninja instanceof Object, "... and the Object prototype");</pre>	método do Protótipo de pessoa

Como o protótipo de uma função é um objeto, existem várias maneiras de copiar a funcionalidade (como propriedades ou métodos) para efetuar a herança. Neste código, definimos um Pessoa e então Ninja. E porque

uma Ninja é claramente uma pessoa, nós queremos Ninja para herdar os atributos de Pessoa. Tentamos fazer isso copiando o dança propriedade do Pessoa método do protótipo para uma propriedade com nome semelhante no Ninja protótipo.

A execução de nosso teste revela que, embora possamos ter ensinado o ninja a dançar, não conseguimos fazer o Ninja uma Pessoa, como mostrado na figura 7.12. Nós ensinamos o Ninja imitar a dança

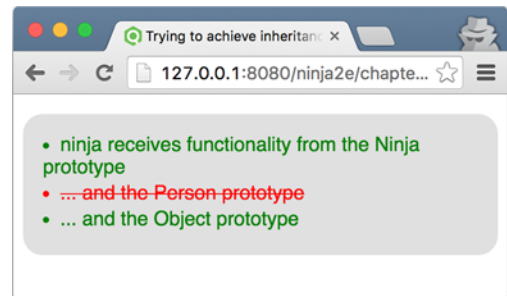


Figura 7.12 Nosso **Ninja** não é realmente um **Pessoa**. Sem dança feliz!

de uma pessoa, mas isso não *fez* a Ninja uma Pessoa. Isso não é herança - é apenas cópia.

Além do fato de que essa abordagem não está exatamente funcionando, também precisaríamos copiar cada propriedade de Pessoa ao Ninja protótipo individualmente. Isso não é maneira de fazer herança. Vamos continuar explorando.

O que realmente queremos alcançar é um *cadeia de protótipo* para que um Ninja posso *estar* uma Pessoa, e um Pessoa pode ser um Mamífero, e um Mamífero pode ser um Animal, e assim por diante, todo o caminho para Objeto. A melhor técnica para criar essa cadeia de protótipo é usar uma instância de um objeto como o protótipo do outro objeto:

```
SubClass.protótipo = novo SuperClass ();
```

Por exemplo:

```
Ninja.protótipo = nova pessoa ();
```

Isso preserva a cadeia de protótipo, porque o protótipo do Subclasse instância será uma instância do SuperClass, que tem um protótipo com todas as propriedades de SuperClass, e que, por sua vez, terá umotótipo que aponta para uma instância de *Está* superclasse e assim por diante. Na próxima listagem, alteramos ligeiramente a listagem 7.7 para usar essa técnica.

Listagem 7.8 Obtendo herança com protótipos

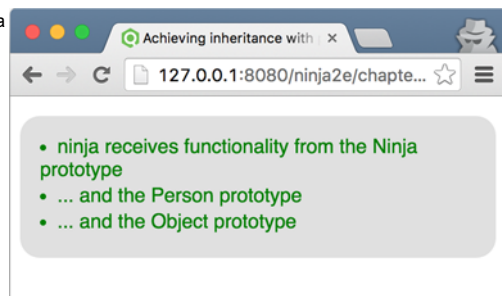
```
função Person () {}  
Person.prototype.dance = function () {};
```

```
função Ninja () {}  
Ninja.prototype = nova pessoa();
```

```
const ninja = novo Ninja (); assert (instância ninja de  
Ninja,  
    "ninja recebe funcionalidade do protótipo Ninja"); assert (instância ninja de Pessoa, "... e o protótipo da  
Pessoa"); assert (ninja instanceof Object, "... and the Object prototype"); assert (typeof ninja.dance === "function", "... and  
can dance!")
```

Torna um Ninja uma Pessoa,
tornando o protótipo Ninja uma
instância de Pessoa

A única mudança no código é usar uma instância de Pessoa como o protótipo para Ninja. A execução dos testes mostra que fomos bem-sucedidos, conforme mostrado na figura 7.13. Agora vamos dar uma olhada mais de perto no funcionamento interno, observando o estado do aplicativo depois de criarmos o novo ninja objeto, conforme mostrado na figura 7.14.



A Figura 7.14 mostra que quando definimos um Pessoa função, um Pessoa

Figura 7.13 Nosso **Ninja** é um **Pessoa**! Deixe a dança da vitória começar.

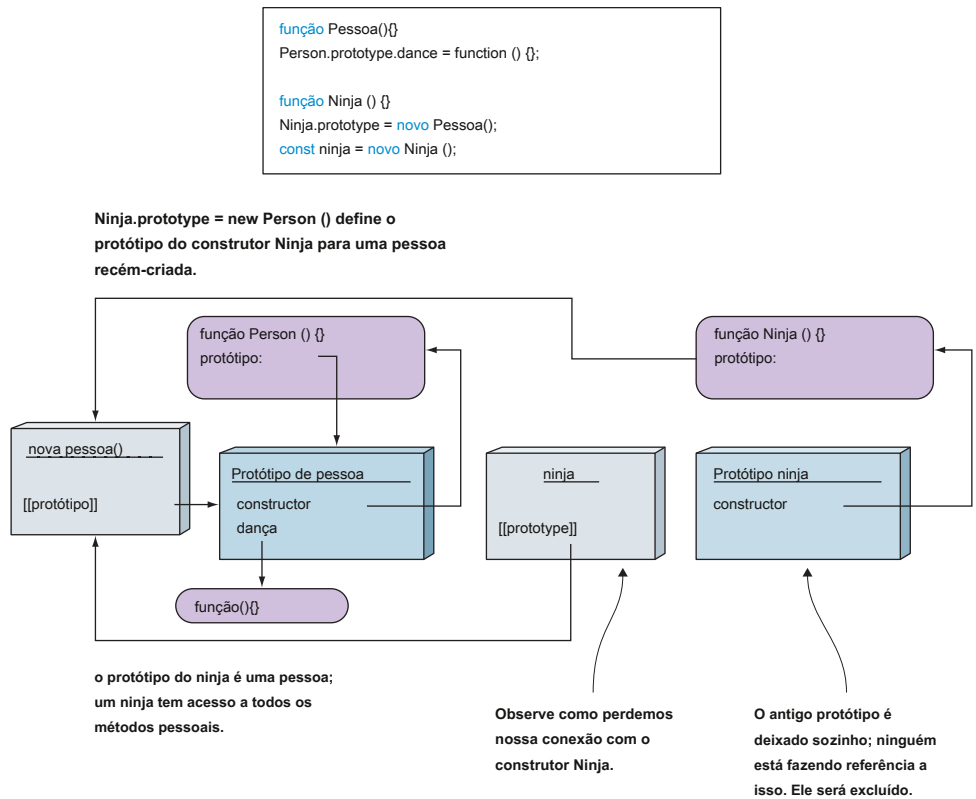


Figura 7.14 Conseguimos herança definindo o protótipo do **Ninja** construtor para uma nova instância de um **Pessoa** objeto.

protótipo também é criado que faz referência ao Pessoa função através de seu constructor propriedade. Normalmente, podemos estender o Person protótipo com propriedades adicionais e, neste caso, especificamos que cada pessoa, criada com o Person construtor, tem acesso ao dança método:

```

função Person () {}
Person.prototype.dance = function () {};

```

Também definimos um Ninja função que obtém seu próprio objeto de protótipo com um constructor propriedade referenciando o Ninja função: função Ninja () {}.

Em seguida, a fim de alcançar a herança, substituímos o protótipo do Ninja funcionar com um novo Pessoa instância. Agora, quando criamos um novo Ninja objeto, a propriedade protótipo interno do recém-criado ninja objeto será definido como o objeto para o qual o Ninja propriedade de protótipo aponta para, o previamente construído Pessoa instância:

```
função Ninja () {}  
Ninja.prototype = nova pessoa (); var ninja = novo Ninja ();
```

Quando tentamos acessar o `dança` método através do `ninja` objeto, o tempo de execução do JavaScript primeiro verificará o `ninja` objeto em si. Porque não tem o `dança` propriedade, seu protótipo, o `pessoa` objeto, é pesquisado. O `pessoa` objeto também não tem o `dança` propriedade, então seu protótipo é pesquisado e a propriedade finalmente encontrada. É assim que se obtém herança em JavaScript!

Aqui está a implicação importante: quando executamos um instancia de operação, podemos determinar se a função herda a funcionalidade de qualquer objeto em sua cadeia de protótipo.

NOTA Outra técnica que pode ter ocorrido com você, e que recomendamos fortemente contra, é usar o `Pessoa` objeto de protótipo diretamente como o `Ninja` protótipo, assim: `Ninja.prototype = Person.prototype`. Quaisquer mudanças para a `Ninja` protótipo também mudará o `Pessoa` protótipo (porque eles são o mesmo objeto), e isso provavelmente terá efeitos colaterais indesejáveis.

Um efeito colateral adicional feliz de fazer a herança do protótipo dessa maneira é que todos os protótipos de função herdada continuarão a atualizar ao vivo. Os objetos que herdaram do protótipo sempre têm acesso às propriedades do protótipo atual.

7.3.1 *O problema de substituir a propriedade do construtor*

Se olharmos mais de perto a figura 7.14, veremos isso definindo o novo `Pessoa` objeto como um protótipo do `Ninja` construtor, perdemos nossa conexão com o `Ninja` construtor que foi anteriormente mantido pelo original `Ninja` protótipo. Isso é um problema, porque o construtor propriedade pode ser usada para determinar a função com a qual o objeto foi criado. Alguém usando nosso código pode fazer uma suposição perfeitamente razoável de que o seguinte teste será aprovado:

```
assert (ninja.constructor === Ninja,  
        "O objeto ninja foi criado pelo construtor Ninja");
```

Mas, no estado atual do aplicativo, esse teste falha. Como mostra a figura 7.14, se pesquisarmos o `ninja` objeto para o construtor propriedade, não vamos encontrar. Então vamos ao seu protótipo, que também não tem uma propriedade `constructor`, e novamente, seguimos o protótipo e terminamos no objeto de protótipo de `Pessoa`, que tem um construtor

propriedade referenciando o `Pessoa` função. Na verdade, recebemos a resposta errada: se perguntarmos o `ninja` objeto cuja função o construiu, vamos obter `Pessoa` como a resposta. Isso pode ser a origem de alguns bugs graves.

Cabe a nós consertar essa situação! Mas antes de fazermos isso, precisamos fazer um desvio e ver como o JavaScript nos permite configurar propriedades.

CONFIGURANDO PROPRIEDADES DOS OBJETOS

Em JavaScript, cada propriedade de objeto é descrita com um *descritor de propriedade* através do qual podemos configurar as seguintes chaves:

- **configurável** —Se definido como verdade, o descritor da propriedade pode ser alterado e a propriedade pode ser excluída. Se definido para falso, não podemos fazer nenhuma dessas coisas.
- **enumerável** —Se definido como verdade, a propriedade aparece durante um `for` dentro `loop` sobre as propriedades do objeto (chegaremos ao `for` dentro `loop` em breve).
- **valor** —Especifica o valor da propriedade. Padrões para `Indefinido`. **gravável** —Se definido como verdade, o valor da propriedade pode ser alterado usando uma atribuição.
- **obter** —Define o *getter* função, que será chamada quando acessarmos a propriedade. Não pode ser definido em conjunto com **valor** e **gravável**. **definir** —Define o *normalizador*, que será chamada sempre que for feita uma atribuição à propriedade. Também não pode ser definido em conjunto com **valor** e **gravável**.

gravável.

Digamos que criamos uma propriedade por meio de uma atribuição simples, por exemplo:

```
ninja.name = "Yoshi";
```

Esta propriedade será configurável, enumerável e gravável, seu valor será definido como Yoshi, e funções obter e definir seria `Indefinido`.

Quando quisermos ajustar a configuração de nossa propriedade, podemos usar o `Object.defineProperty` método, que pega um objeto no qual a propriedade será definida, o nome da propriedade e um objeto descritor de propriedade. Como exemplo, dê uma olhada no código a seguir.

Listagem 7.9 Configurando propriedades

```
var ninja = {};  
ninja.name = "Yoshi";  
ninja.weapon = "kusarigama";
```

Cria um objeto vazio; usa atribuições para adicionar duas propriedades

```
Object.defineProperty (ninja, "sorrateiro", {  
  configurável: falso,  
  enumerável: falso,  
  valor: verdadeiro,  
  gravável: verdadeiro  
});
```

O `Object.defineProperty` integrado método é usado para ajustar os detalhes de configuração da propriedade.

```
assert ("sorrateiro" em ninja, "Podemos acessar a nova propriedade");
```

```
para (deixe prop no ninja) {  
  assert (prop! == undefined, "Uma propriedade enumerada:" + prop);  
}
```

Usa o `loop for-in` para iterar propriedades enumeráveis do `ninja`

Começamos com a criação de um objeto vazio, ao qual adicionamos duas propriedades: nome e arma, à boa e velha maneira, usando atribuições. Em seguida, usamos o integrado Objeto

.defineProperty método para definir o propriedade `sorradeira`, que não é configurável, não é enumerável, e tem o seu valor definido como verdade. Esta valor pode ser mudado porque é gravável.

Finalmente, testamos se podemos acessar o recém-criado `sorradeira` propriedade, e usamos o `para dentro` loop para percorrer todas as propriedades enumeráveis do objeto. A Figura 7.15 mostra o resultado.

Pela configuração enumerável para falso, podemos ter certeza de que a propriedade não aparecerá ao usar o `para dentro` ciclo. Para entender por que queremos fazer algo assim, vamos voltar ao problema original.

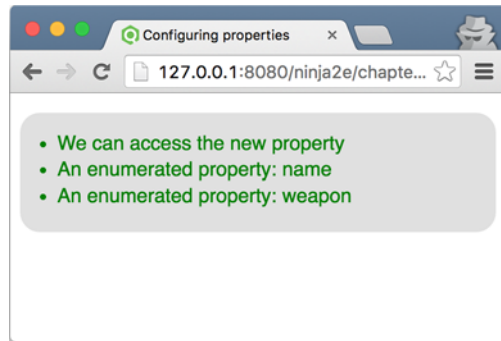


Figura 7.15 Propriedades **nome** e **arma** será visitado no **para dentro** loop, enquanto nosso especialmente adicionado **sorradeira** propriedade não (embora possamos acessá-la normalmente).

RESOLVENDO FINALMENTE O PROBLEMA DE SUBSTITUIR A PROPRIEDADE DO CONSTRUTOR

Ao tentar estender `Pessoa` com `Ninja` (ou para fazer `Ninja` uma subclasse de `Pessoa`), encontramos o seguinte problema: quando definimos um novo `Pessoa` objeto como um protótipo para o `Ninja` construtor, perdemos o original `Ninja` protótipo que mantém nosso construtor propriedade. Não queremos perder o construtor , porque é útil para determinar a função usada para criar nossas instâncias de objeto e pode ser esperada por outros desenvolvedores que trabalham em nossa base de código.

Podemos resolver esse problema usando o conhecimento que acabamos de obter. Vamos definir um novo construtor propriedade no novo `Ninja.prototype` usando o `Object.defineProperty` método. Veja a lista a seguir.

Listagem 7.10 Corrigindo o problema da propriedade do construtor

```
função Person () {}
Person.prototype.dance = function () {};
```

```
função Ninja () {}
Ninja.prototype = nova pessoa();
```

```
Object.defineProperty (Ninja.prototype, "construtor", {
  enumerável: falso,
  valor: Ninja,
  gravável: verdadeiro
});
```

```
var ninja = novo Ninja ();
```

Definimos um novo
construtor não enumerável
propriedade apontando de volta para `Ninja`.

Nós temos restabelecido a conexão.

```

assert (ninja.constructor === Ninja,
        "Conexão de instâncias ninja ao construtor Ninja
         restabelecido! ");
for (let prop in Ninja.prototype) {
    assert (prop === "dance", "A única propriedade enumerável é dance!");
}

```

Não adicionamos nenhuma propriedade enumerável ao `Ninja.prototype`.

Agora, se executarmos o código, veremos que está tudo ótimo. Nós restabelecemos a conexão entre `ninja` instâncias e o `Ninja` função, para que possamos saber que eles foram construídos pela `Ninja` função. Além disso, se alguém tentar percorrer as propriedades do `Ninja.prototype` objeto, certificamo-nos de que nossa propriedade corrigida construtor não será visitado. Essa é a marca de um verdadeiro `ninja`; entramos, fizemos nosso trabalho e saímos, sem ninguém perceber nada de fora!

7.3.2 O operador *instanceof*

Na maioria das linguagens de programação, a abordagem direta para verificar se um objeto faz parte de uma hierarquia de classes é usar o instância de operador. Por exemplo, em Java, o instância de operador funciona verificando se o objeto do lado esquerdo é a mesma classe ou uma subclasse do tipo de classe à direita.

Embora certos paralelos possam ser feitos com a forma como o instância de operador funciona em JavaScript, há uma pequena diferença. Em JavaScript, o instância de operador trabalha na cadeia de protótipo do objeto. Por exemplo, digamos que temos a seguinte expressão:

```
instância ninja de Ninja
```

O instância de operador trabalha verificando se o *atual* protótipo do

`Ninja` função está na cadeia de protótipo do `ninja` instância. Voltemos às nossas pessoas e ninjas, para um exemplo mais concreto.

Listagem 7.11 Estudando o instância de operador

```

função Person () {}
função Ninja () {}

Ninja.prototype = nova pessoa ();

const ninja = novo Ninja ();

assert (instância ninja de Ninja, "Nosso ninja é um Ninja!"); assert (instância ninja de Pessoa, "Um ninja também é uma
pessoa.");

```

Uma instância `ninja` é tanto um `Ninja` e uma `pessoa`.

Como esperado, um `ninja` é, ao mesmo tempo, um `Ninja` e um `Pessoa`. Mas, para esclarecer esse ponto, a figura 7.16 mostra como a coisa toda funciona nos bastidores.

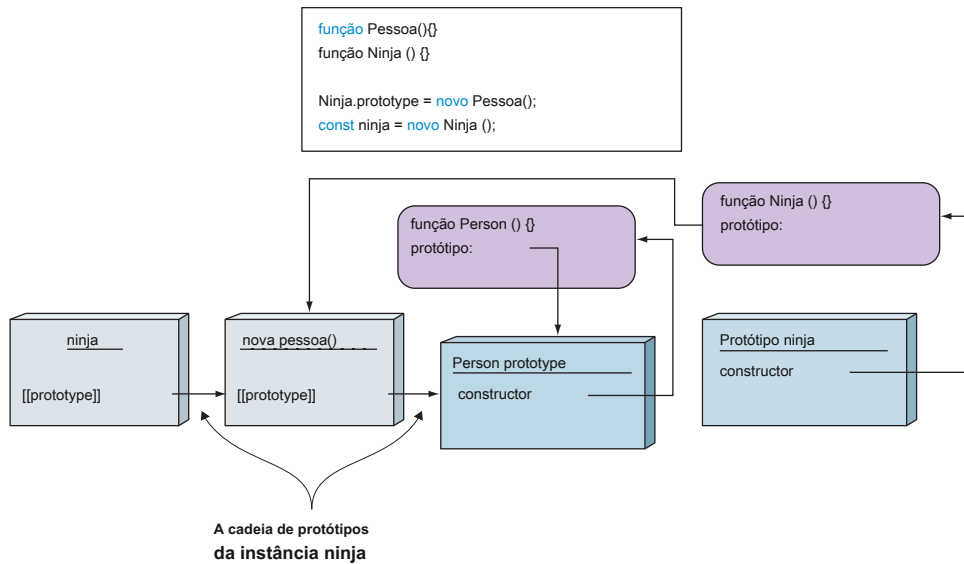


Figura 7.16 A cadeia de protótipo de um **ninja** instância é composta por um **nova pessoa()** objeto e o **Protótipo de pessoa**.

A cadeia de protótipo de uma instância **ninja** é composta por um `new Person()` objeto, através do qual alcançamos a herança, e o Protótipo de pessoa. Ao avaliar a expressão instância **ninja** do **Ninja**, o motor JavaScript pega o protótipo do

Ninja função, o `new Person()` objeto e verifica se ele está na cadeia de protótipos do **ninja** instância. Porque o **nova pessoa()** objeto é um protótipo direto do **ninja** exemplo, o resultado é verdade.

No segundo caso, onde verificamos instância **ninja** de **Pessoa**, o motor JavaScript pega o protótipo do **Pessoa** função, o Protótipo de pessoa, e verifica se ele pode ser encontrado na cadeia de protótipos do **ninja** instância. Mais uma vez, pode, porque é o protótipo do nosso **nova pessoa()** objeto, que, como já vimos, é o protótipo do **ninja** instância.

E isso é tudo que há para saber sobre o instância de operador. Embora seu uso mais comum seja fornecer uma maneira clara de determinar se uma instância foi criada por um construtor de função específico, ele não funciona exatamente assim. Em vez disso, ele verifica se o protótipo da função do lado direito está na cadeia de protótipos do objeto à esquerda. Portanto, há uma advertência com a qual devemos ter cuidado.

A INSTÂNCIA DA CAVEAT

Como você viu várias vezes ao longo deste capítulo, JavaScript é uma linguagem dinâmica na qual podemos modificar um *muito* de coisas durante a execução do programa. Por exemplo, não há nada que nos impeça de alterar o protótipo de um construtor, conforme mostrado na listagem a seguir.

Listagem 7.12 Cuidado com as mudanças nos protótipos do construtor

Nós mudamos o protótipo do Ninja construtor função.

```

função Ninja () {}

const ninja = new Ninja();

assert (instância ninja de Ninja, "Nosso ninja é um Ninja!");

Ninja.prototype = {};

afirmar (! (instância do ninja Ninja), "O ninja agora não é um Ninja !?");

```

Mesmo que nossa instância ninja tenha sido criada pelo construtor Ninja, o operador instanceof agora diz aquele ninja não é mais uma instância de Ninja!

Neste exemplo, repetimos novamente todas as etapas básicas para fazer um ninja instância, e nosso primeiro teste vai bem. Mas se mudarmos o protótipo do Ninja função construtora *depois de* a criação do ninja instância, e novamente teste se ninja é um instância de Ninja, veremos que a situação mudou. Isso nos surpreenderá apenas se nos apegarmos à suposição imprecisa de que o instanceof operador nos diz se uma instância foi criada por um construtor de função particular. Se, por outro lado, tomarmos a semântica real do instanceof operador - que verifica apenas se o protótipo da função do lado direito está na cadeia de protótipos do objeto do lado esquerdo - não ficaremos surpresos. Essa situação é mostrada na figura 7.17.

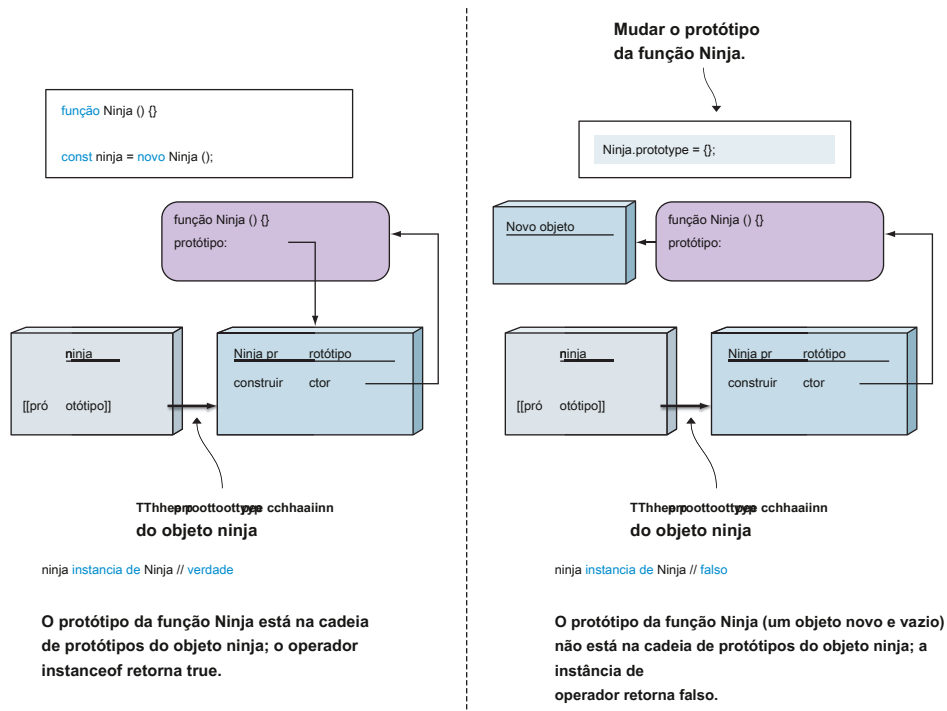


Figura 7.17 O **instancia de** operador verifica se o protótipo da função à direita está na cadeia de protótipos do objeto à esquerda. Tome cuidado; o protótipo da função pode ser alterado a qualquer momento!

Agora que entendemos como os protótipos funcionam em JavaScript e como usar protótipos em conjunto com funções de construtor para implementar herança, vamos passar para uma nova adição na versão ES6 de JavaScript: classes.

7.4 Usando “classes” de JavaScript no ES6

É ótimo que o JavaScript nos permita usar uma forma de herança por meio de protótipos. Mas muitos desenvolvedores, especialmente aqueles com experiência clássica em orientação a objetos, prefeririam uma simplificação ou abstração do sistema de herança do JavaScript em um com o qual estejam mais familiarizados.

Isso leva inevitavelmente ao reino das classes, embora o JavaScript não suporte herança clássica nativamente. Em resposta a essa necessidade, surgiram várias bibliotecas JavaScript que simulam a herança clássica. Como cada biblioteca implementa classes em sua própria maneira, o comitê ECMAScript padronizou a sintaxe para simular a herança baseada em classes. Observe como dissemos *simulando*. Mesmo que agora possamos usar o `class` palavra-chave em JavaScript, a implementação subjacente ainda é baseada na herança do protótipo!



NOTA O aula a palavra-chave foi adicionada à versão ES6 do JavaScript e nem todos os navegadores a implementam (consulte <http://mng.bz/3ykA> para suporte atual).

Vamos começar estudando a nova sintaxe.

7.4.1 Usando a palavra-chave class

ES6 apresenta um novo **aula** palavra-chave que fornece uma maneira muito mais elegante de criar objetos e implementar herança do que implementá-la manualmente com protótipos. Usando o **aula** palavra-chave é fácil, conforme mostrado na lista a seguir.

The diagram is divided into two main sections by a horizontal line. The top section is titled 'Usa a classe' and shows a code snippet for a `Ninja` class. A callout box explains that the `new` keyword is used to create a new instance of the class. The bottom section is titled 'Instancia' and shows a code snippet for creating a `ninja` object. A callout box explains that the `new` keyword is used to create a new instance of the class.

```
class Ninja {
  constructor(nome) {
    this.name = nome;
  }

  swingSword () {
    return true;
  }
}
```

Define uma função construtora que será chamada quando chamarmos a classe com a palavra-chave `new`

```
var ninja = novo Ninja ("Yoshi");

assert (instância ninja de Ninja, "Nosso ninja é um Ninja"); assert (ninja.name === "Yoshi", "nomeado Yoshi"); assert (ninja.swingSword (), "e ele pode brandir uma espada");
```

Define um adicional método acessível para todas as instâncias `Ninja`

Testes para o comportamento esperado

Listagem 7.13 mostra que podemos criar um Ninja classe usando o `class` palavra-chave. Ao criar classes ES6, podemos definir explicitamente um construtor função que será invocada ao instanciar um Ninja instância. No corpo do construtor, podemos acessar a instância recém-criada com o `this` palavra-chave, e podemos facilmente adicionar novas propriedades, como a `nome` propriedade. Dentro do corpo da classe, também podemos definir métodos que serão acessíveis a todos Ninja instâncias. Neste caso, definimos um `swingSword`

método que retorna verdade:

```
class Ninja {  
  constructor (nome) {  
    this.name = nome;  
  }  
  
  swingSword () {  
    return true;  
  }  
}
```

Em seguida, podemos criar um Ninja instância chamando o Ninja classe com a palavra-chave `new`, assim como faríamos se Ninja era uma função construtora simples (como no início do capítulo):

```
var ninja = new Ninja ("Yoshi");
```

Finalmente, podemos testar se o `ninja` instância se comporta conforme o esperado, que é um instância de Ninja, tem um `nome` propriedade, e tem acesso ao `swingSword` método:

```
assert (instância ninja de Ninja, "Nosso ninja é um Ninja");  
assert (ninja.name === "Yoshi", "nomeado Yoshi");  
assert (ninja.swingSword (), "e ele pode brandir uma espada");
```

AS AULAS SÃO AÇÚCAR SINTÁTICO

Conforme mencionado anteriormente, embora ES6 tenha introduzido o `class` palavra-chave, sob o capô ainda estamos lidando com os bons e velhos protótipos; classes são açúcares sintáticos projetados para tornar nossas vidas um pouco mais fáceis ao imitar classes em JavaScript.

Nosso código de classe da listagem 7.13 pode ser traduzido para código ES5 funcionalmente idêntico:

```
function Ninja (nome) {  
  this.name = nome;  
}  
Ninja.prototype.swingSword = function () {  
  return true;  
};
```

Como você pode ver, não há nada de especialmente novo nas classes ES6. O código é mais elegante, mas os mesmos conceitos são aplicados.

MÉTODOS ESTÁTICOS

Nos exemplos anteriores, você viu como definir métodos de objeto (métodos de protótipo), acessíveis a todas as instâncias de objeto. Além de tais métodos, as linguagens clássicas orientadas a objetos, como Java, usam métodos estáticos, métodos definidos em nível de classe. Veja o seguinte exemplo.

Listagem 7.14 Métodos estáticos em ES6

```
class Ninja {
  construtor (nome, nível) {
    this.name = nome;
    this.level = level;
  }

  swingSword () {
    return true;
  }

  estático compare (ninja1, ninja2) {
    retornar ninja1.level - ninja2.level;
  }
}
```

Usa a **estática**
palavra-chave para fazer
um método estático

```
var ninja1 = novo Ninja ("Yoshi", 4); var ninja2 = novo Ninja ("Hattori",
3);
```

instâncias ninja
não tem acesso
comparar.

```
assert (! ("compare" no ninja1) &&! ("compare" no ninja2),
  "Uma instância ninja não sabe como comparar");
```

```
assert (Ninja.compare (ninja1, ninja2)> 0,
  "A classe Ninja pode fazer a comparação!");
```

A classe **Ninja**
tem acesso ao método
de comparação.

```
assert (! ("swingSword" em Ninja),
  "A classe Ninja não pode brandir uma espada");
```

Nós novamente criamos um Ninja classe que tem um swingSword método acessível de todos ninja instâncias. Também definimos um método estático, comparar, prefixando o nome do método com a palavra-chave estático.

```
estático compare (ninja1, ninja2) {
  retornar ninja1.level - ninja2.level;
}
```

O comparar método, que compara os níveis de habilidade de dois ninjas, é definido no nível da classe, e não no nível da instância! Mais tarde, testamos se isso significa efetivamente que o comparar método não é acessível de ninja instâncias, mas está acessível a partir do Ninja aula:

```
assert (! ("compare" no ninja1) &&! ("compare" no ninja2),
  "A instância ninja não sabe como comparar"); assert (Ninja.compare (ninja1, ninja2)> 0,
  "A classe Ninja pode fazer a comparação!");
```

Também podemos ver como os métodos “estáticos” podem ser implementados no código pré-ES6. Para isso, devemos lembrar apenas que as classes são implementadas por meio de funções. Como os métodos estáticos são métodos de nível de classe, podemos implementá-los tirando proveito de funções como objetos de primeira classe e adicionando uma propriedade de método à nossa função de construtor, como no exemplo a seguir:

```
função Ninja () {}  
Ninja.compare = função (ninja1, ninja2) {...}
```

Estende a função do construtor com um método para imitar métodos estáticos no código pré-ES6

Agora, vamos prosseguir para a herança.

7.4.2 Implementando herança

Para ser honesto, executar a herança no código pré-ES6 pode ser uma dor. Vamos voltar aos nossos ninjas confiáveis, exemplo de Pessoas:

```
função Person () {}  
Person.prototype.dance = function () {};  
  
função Ninja () {}  
Ninja.prototype = nova pessoa();  
  
Object.defineProperty (Ninja.prototype, "construtor", {  
  enumerável: falso,  
  valor: Ninja,  
  gravável: verdadeiro  
});
```

Há muito para se manter em mente aqui: métodos acessíveis a todas as instâncias devem ser adicionados diretamente ao protótipo da função do construtor, como fizemos com o dança método e o Pessoa construtor. Se quisermos implementar herança, temos que definir o protótipo da “classe” derivada para a instância da “classe” base. Neste caso, atribuímos uma nova instância de Pessoa para Ninja.prototype. Infelizmente, isso bagunça o construtor propriedade, então temos que restaurá-la manualmente com o

Object.defineProperty método. É muito importante ter isso em mente ao tentar obter um recurso relativamente simples e comumente usado (herança). Felizmente, com o ES6, tudo isso é significativamente simplificado.

Vamos ver como isso é feito na lista a seguir.

Listagem 7.15 Herança em ES6

```
class Person {  
  construtor (nome) {  
    this.name = nome;  
  }  
  
  dança(){  
    return true;  
  }  
}
```

```

    }

    classe Ninja extends Person {
        construtor (nome, arma) {
            super (nome);
            this.weapon = arma;
        }

        wieldWeapon () {
            return true;
        }
    }

    var pessoa = nova pessoa ("Bob");

    assert (person instanceof Person, "Uma pessoa é uma pessoa"); assert (person.dance (), "Uma pessoa pode dançar."); assert
    (person.name === "Bob", "Podemos chamá-lo pelo nome."); assert (! (person instanceof Ninja), "Mas não é um Ninja"); assert (!
    ("wieldWeapon" pessoalmente), "E não pode empunhar uma arma");

    var ninja = novo Ninja ("Yoshi", "Wakizashi"); assert (instância ninja de Ninja, "A ninja's a ninja"); assert
    (ninja.wieldWeapon (), "Isso pode empunhar uma arma"); assert (instância ninja de Person, "Mas também é
    uma pessoa"); assert (ninja.name === "Yoshi", "Isso tem um nome"); assert (ninja.dance (), "E gosta de
    dançar");

```

Usa a palavra-chave **extends** para herdar de outra classe

Usa a palavra-chave **super** para chamar o construtor da classe base

A Listagem 7.15 mostra como obter herança no ES6; nós usamos o estende palavra-chave para herdar de outra classe:

```

classe Ninja estende Pessoa

```

Neste exemplo, criamos um Pessoa classe com um construtor que atribui um nome para cada Pessoa instância. Também definimos um dança método que será acessível a todos Pessoa instâncias:

```

class Person {
    construtor (nome) {
        this.name = nome;
    }
    dança(){
        return true;
    }
}

```

Em seguida, definimos um Ninja classe que estende o Pessoa aula. Tem um adicional arma propriedade, e um empunhar arma método:

```

classe Ninja extends Person {
    construtor (nome, arma) {
        super (nome);
    }
}

```

```

        this.weapon = arma;
    }

    wieldWeapon () {
        return true;
    }
}

```

No construtor do derivado, Ninja classe, há uma chamada para o construtor da base, Pessoa classe, por meio da palavra-chave `super`. Isso deve ser familiar, se você já trabalhou com qualquer linguagem baseada em classe.

Continuamos criando um pessoa instância e verificando se é uma instância do Pessoa classe que tem um nome e pode dança. Só para ter certeza, também verificamos se uma pessoa que *não é* um Ninja não pode empunhar uma arma:

```

var pessoa = nova pessoa ("Bob");

assert (person instanceof Person, "Uma pessoa é uma pessoa"); assert (person.dance (), "Uma pessoa pode dançar."); assert
(person.name === "Bob", "Podemos chamá-lo pelo nome."); assert (! (person instanceof Ninja), "Mas não é um Ninja"); assert (!
("wieldWeapon" pessoalmente), "E não pode empunhar uma arma");

```

Nós também criamos um ninja instância e verifique se é uma instância de Ninja e pode empunhar uma arma. Porque todo ninja também é um Pessoa, verificamos se um ninja é uma instância de Pessoa, que tem um nome, e que também, no íterim da luta, gosta de dançar:

```

var ninja = novo Ninja ("Yoshi", "Wakizashi"); assert (instância ninja de Ninja, "A ninja's a ninja"); assert
(ninja.wieldWeapon (), "Isso pode empunhar uma arma"); assert (instância ninja de Person, "Mas também é
uma pessoa"); assert (ninja.name === "Yoshi", "Isso tem um nome"); assert (ninja.dance (), "E gosta de
dançar");

```

Veja como isso é fácil? Não há necessidade de pensar sobre protótipos ou os efeitos colaterais de certas propriedades substituídas. Definimos classes e especificamos seu relacionamento usando o estende palavra-chave. Finalmente, com o ES6, hordas de desenvolvedores vindos de linguagens como Java ou C # podem ficar em paz.

E é isso. Com o ES6, construímos hierarquias de classes quase tão facilmente quanto em qualquer outra linguagem orientada a objetos mais convencional.

7,5 *Resumo*

- Os objetos JavaScript são coleções simples de propriedades nomeadas com valores. JavaScript usa
- protótipos.
- Cada objeto pode ter uma referência a um *protótipo*, um objeto ao qual delegamos a busca por uma propriedade particular, se o próprio objeto não tiver a propriedade procurada. O protótipo de um objeto pode ter seu próprio protótipo, e assim por diante, formando um *cadeia de protótipo*.

- Podemos definir o protótipo de um objeto usando o `Object.setPrototypeOf` método.
- Os protótipos estão intimamente ligados às funções do construtor. Cada função tem um protótipo propriedade que é definida como o protótipo dos objetos que ela instancia. Uma função protótipo objeto
- tem um construtor propriedade apontando de volta para a própria função. Esta propriedade é acessível a todos os objetos instanciados com aquela função e, com certas limitações, pode ser usada para descobrir se um objeto foi criado por uma função específica.
- Em JavaScript, quase tudo pode ser alterado em tempo de execução, incluindo os protótipos de um objeto e os protótipos de uma função!
- Se quisermos as instâncias criadas por um Ninja função de construtor para "herdar" (mais precisamente, ter acesso a) propriedades acessíveis a instâncias criadas pelo Pessoa função de construtor, defina o protótipo do Ninja construtor para uma nova instância do Pessoa aula.
- Em JavaScript, as propriedades têm atributos (configuráveis, enumeráveis, graváveis). Essas propriedades podem ser definidas usando o built-in `Object.defineProperty` método. JavaScript ES6 adiciona suporte para uma aula
- palavra-chave que nos permite simular classes mais facilmente. Nos bastidores, os protótipos ainda estão em jogo!
- O estende palavra-chave habilita herança elegante.

7,6 *Exercícios*

- 1 Qual das seguintes propriedades aponta para um objeto que será pesquisado se o objeto de destino não tem a propriedade procurada?

- a) `aula`
- b) `instância`
- c) `protótipo`
- d) `aponta para`

- 2 Qual é o valor da variável `a1` depois que o código a seguir é executado?

```
função Ninja () {}
Ninja.prototype.talk = function () {
  retornar "Olá";
};

const ninja = novo Ninja (); const a1 = ninja.talk ();
```

- 3 Qual é o valor de `a1` depois de executar o seguinte código?

```
função Ninja () {}
Ninja.message = "Olá";

const ninja = novo Ninja ();

const a1 = ninja.mensagem;
```

- 4 Explique a diferença entre o `getFullName` método nestes dois códigos fragmentos:

```
// Primeiro fragmento
função Pessoa (primeiroNome, últimoNome) {
  this.firstName = primeiroNome;
  this.lastName = últimoNome;

  this.getFullName = function () {
    retorna this.firstName + " " + this.lastName;
  }
}

// segundo fragmento
função Pessoa (primeiroNome, últimoNome) {
  this.firstName = primeiroNome;
  this.lastName = últimoNome;
}

Person.prototype.getFullName = function () {
  retorna this.firstName + " " + this.lastName;
}
```

- 5 Depois de executar o código a seguir, o que `ninja.constructor` aponta para?

```
function Person () {} function Ninja () {}

const ninja = novo Ninja ();
```

- 6 Depois de executar o código a seguir, o que `ninja.constructor` aponta para?

```
function Person () {} function Ninja () {}

Ninja.prototype = nova pessoa (); const ninja = novo Ninja ();
```

- 7 Explique como o instancia de operador funciona no exemplo a seguir.

```
função Guerreiro () {}

função Samurai () {}
Samurai.prototype = novo Guerreiro ();

var samurai = novo Samurai ();

exemplo de samurai do guerreiro; //Explicar
```

- 8 Traduza o seguinte código ES6 para o código ES5.

```
class Warrior {
  construtor (arma) {
```

```
        this.weapon = arma;
    }

    manejar() {
        return "Wielding" + this.weapon;
    }

    duelo estático (guerreiro1, guerreiro2) {
        return warrior1.wield () + "" + warrior2.wield ();
    }
}
```

8

Controle de acesso a objetos

Este capítulo cobre

- Usando getters e setters para controlar o acesso às propriedades do objeto
- Controlar o acesso a objetos por meio de proxies
- Usando proxies para questões transversais

No capítulo anterior, você viu que os objetos JavaScript são coleções dinâmicas de propriedades. Podemos facilmente adicionar novas propriedades, alterar os valores das propriedades e, eventualmente, remover completamente as propriedades existentes. Em muitas situações (por exemplo, ao validar valores de propriedade, registrar ou exibir dados na IU), precisamos ser capazes de monitorar exatamente o que está acontecendo com nossos objetos. Portanto, neste capítulo, você aprenderá técnicas para controlar o acesso e monitorar todas as mudanças que ocorrem em seus objetos.

Começaremos com getters e setters, métodos que controlam o acesso a propriedades específicas do objeto. Você viu esses métodos em ação pela primeira vez nos capítulos 5 e 7. Neste capítulo, você verá alguns de seus suportes de linguagem embutidos e como usá-los para registro, execução de validação de dados e definição de propriedades computadas.

Continuaremos com proxies, um tipo de objeto completamente novo introduzido no ES6. Esses objetos controlam o acesso a outros objetos. Você aprenderá como eles funcionam e como usá-los com grande eficácia para expandir facilmente seu código com questões transversais

como medição de desempenho ou registro e como evitar exceções nulas ao preencher automaticamente as propriedades do objeto. Vamos começar a jornada com algo que já sabemos até certo ponto: getters e setters.

.....

Quais são alguns dos benefícios de acessar uma propriedade
valor por meio de getters e setters?

Você sabe? Qual é a principal diferença entre proxies e getters
e setters?

O que são armadilhas de proxy? Cite três tipos de armadilha.

.....

8,1 *Controle de acesso a propriedades com getters e setters*

Em JavaScript, os objetos são coleções relativamente simples de propriedades. A principal forma de acompanhar o estado do nosso programa é modificando essas propriedades. Por exemplo, considere o seguinte código:

```
função Ninja (nível) {
  this.skillLevel = nível;
}
const ninja = novo Ninja (100);
```

Aqui nós definimos um Ninja construtor que cria ninja objetos com uma propriedade nível de habilidade. Mais tarde, se quisermos alterar o valor dessa propriedade, podemos escrever o seguinte código ing: `ninja.skillLevel = 20`.

Tudo isso é bom e conveniente, mas o que acontece nos casos a seguir?

- Queremos nos proteger contra erros acidentais, como atribuição de dados imprevistos. Por exemplo, queremos nos impedir de fazer algo como atribuir um valor de um tipo errado: `ninja.skillLevel = "alto"`.
- Queremos registrar todas as alterações no nível de habilidade propriedade. Precisamos mostrar o valor do nosso nível de
- habilidade propriedade em algum lugar na interface do usuário de nossa página da web. Naturalmente, queremos apresentar o último valor atualizado da propriedade, mas como podemos fazer isso facilmente?

Podemos lidar com todos esses casos elegantemente com os métodos `getter` e `setter`.

No capítulo 5, você viu os getters e setters como um meio de imitar propriedades de objetos privados em JavaScript por meio de fechamentos. Vamos revisar o material que você aprendeu até agora, trabalhando com ninjas que têm um nível de habilidade propriedade acessível apenas por meio de getters e setters, conforme mostrado na listagem a seguir.

Listagem 8.1 Usando getters e setters para proteger propriedades privadas

```
function Ninja () {
  let skillLevel;
```

← Define uma variável privada `skillLevel`

```
this.getSkillLevel = () => skillLevel;
```



O método **getter** controla o acesso à nossa variável particular `skillLevel`.

```
this.setSkillLevel = value => {
  skillLevel = value;
};
```

O método **setter** controla os valores que podemos atribuir a `skillLevel`.

```
}
```

```
const ninja = novo Ninja (); ninja.setSkillLevel
(100);
assert (ninja.getSkillLevel () === 100,
  "Nosso ninja está no nível 100!");
```



Define um novo valor de `skillLevel` por meio do método **setter**

Recupera o valor de `skillLevel` com o método **getter**

Nós definimos um `Ninja` construtor que cria ninjas com um "privado" nível de habilidade variável acessível apenas através de nosso `getSkillLevel` e `setSkillLevel` métodos: O valor da propriedade pode ser obtido apenas através do `getSkillLevel` método, enquanto um novo valor de propriedade pode ser definido apenas por meio do `setSkillLevel` método (lembra do capítulo 5 sobre fechamentos?).

Agora, se quisermos registrar todas as tentativas de leitura do nível de habilidade propriedade, nós expandimos o `getSkillLevel` método; e se quisermos reagir a todas as tentativas de gravação, expandimos o `setSkillLevel` método, como no seguinte snippet:

```
function Ninja () {
  let skillLevel;

  this.getSkillLevel = () => {
    relatório ("Obtendo valor do nível de habilidade");
    return skillLevel;
  };

  this.setSkillLevel = value => {
    report ("Modificando a propriedade skillLevel de:",
      nível de habilidade, " para: ", valor); skillLevel =
    value;
  }
}
```



Usando **getters**, podemos saber sempre que o código acessa uma propriedade.

Usando **setters**, podemos saber sempre que o código deseja definir um novo valor para uma propriedade.

Isso é ótimo. Podemos reagir facilmente a todas as interações com nossas propriedades, conectando, por exemplo, registro, validação de dados ou outros efeitos colaterais, como modificações da IU.

Mas uma preocupação persistente pode estar surgindo em sua mente. O nível de habilidade propriedade é uma propriedade de valor; faz referência a dados (o número 100) e não a uma função. Infelizmente, para tirar proveito de todos os benefícios do acesso controlado, todas as nossas interações com a propriedade devem ser feitas chamando explicitamente os métodos associados, o que é, para ser honesto, um pouco estranho.

Felizmente, o JavaScript tem suporte integrado para getters e setters verdadeiros: propriedades que são acessadas como propriedades de dados normais (por exemplo, `ninja.skillLevel`), mas esses são métodos que podem calcular o valor de uma propriedade solicitada, validar o valor passado ou qualquer outra coisa que precisemos que eles façam. Vamos dar uma olhada neste suporte integrado.

8.1.1 Definindo getters e setters

Em JavaScript, os métodos getter e setter podem ser definidos de duas maneiras:

- Especificando-os em literais de objeto ou nas definições de classe ES6
- Usando o integrado `Object.defineProperty` método

O suporte explícito para getters e setters já existe há algum tempo, desde os dias de ES5. Como sempre, vamos explorar a sintaxe por meio de um exemplo. Nesse caso, temos um objeto que armazena uma lista de ninjas e queremos ser capazes de obter e definir o primeiro ninja da lista.

Listagem 8.2 Definindo getters e setters em literais de objeto

```
const ninjaCollection = {
  ninjas: ["Yoshi", "Kuma", "Hattori"],
  obter firstNinja () {
    report ("Getting firstNinja");
    retornar this.ninjas [0];
  },
  definir firstNinja (valor) {
    report ("Configurando firstNinja");
    this.ninjas [0] = valor;
  }
};

assert (ninjaCollection.firstNinja === "Yoshi",
  "Yoshi é o primeiro ninja");

ninjaCollection.firstNinja = "Hachi";

assert (ninjaCollection.firstNinja === "Hachi"
  && ninjaCollection.ninjas [0] === "Hachi",
  "Agora Hachi é o primeiro ninja");
```

Define um método getter para a propriedade `firstNinja` que retorna o primeiro ninja em nossa coleção e registra uma mensagem

Define um método setter para a propriedade `firstNinja` que modifica o primeiro ninja em nossa coleção e registra uma mensagem

Acessa a propriedade `firstNinja` como se fosse uma propriedade de objeto padrão

Modifica a propriedade `firstNinja` como se fosse uma propriedade de objeto padrão

Testa se a modificação da propriedade é armazenada

Este exemplo define um `ninjaCollection` objeto que possui uma propriedade padrão, `ninjas`, que faz referência a uma matriz de ninjas e um getter e um setter para a propriedade `firstNinja`. A sintaxe geral para getters e setters é mostrada na figura 8.1.

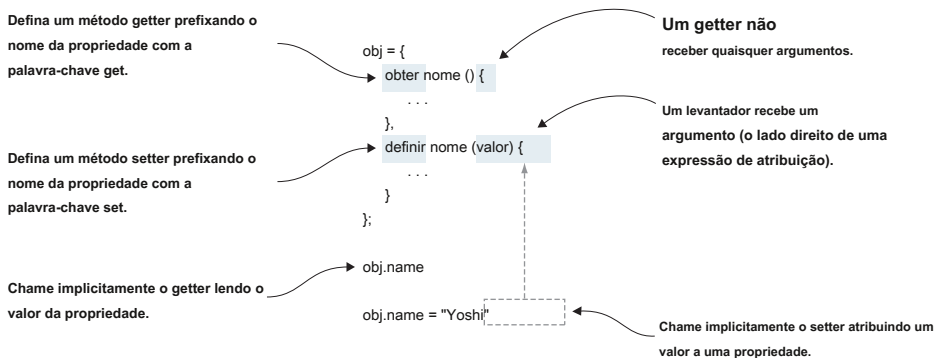


Figura 8.1 A sintaxe para definir getters e setters. Prefixe o nome da propriedade com o **obter** ou o **definir** palavra-chave.

Como você pode ver, definimos uma propriedade getter prefixando o nome com um obter palavra-chave e uma propriedade setter com um definir palavra-chave.

Na Listagem 8.2, tanto o getter quanto o setter registram uma mensagem. Além disso, o getter retorna o valor do ninja no índice 0, e o setter atribui um novo valor ao ninja no mesmo índice:

```
obter firstNinja () {  
    report ("Getting firstNinja");  
    retornar this.ninjas [0];  
},  
definir firstNinja (valor) {  
    report ("Configurando firstNinja");  
    this.ninjas [0] = valor;  
}
```

Em seguida, testamos se o acesso à propriedade getter retorna o primeiro ninja, Yoshi:

```
afirmar( ninjaCollection.firstNinja === "Yoshi ",  
        "Yoshi é o primeiro ninja");
```

Observe que a propriedade getter é acessada como se fosse uma propriedade de objeto padrão (e não como o método que é).

Depois de acessar uma propriedade getter, o método getter associado é implicitamente chamado, a mensagem Pegando o primeiro Ninja é registrado, e o valor do ninja no índice 0 é devolvido.

Continuamos tirando proveito de nosso método setter e escrevendo para o firstNinja propriedade, novamente, da mesma forma que atribuíríamos um novo valor a uma propriedade de objeto normal:

```
ninjaCollection.firstNinja = "Hachi";
```

Semelhante ao caso anterior, porque o firstNinja A propriedade tem um método setter, sempre que atribuímos um valor a essa propriedade, o método setter é chamado implicitamente. Isso registra a mensagem Configurando o firstNinja e modifica o valor do ninja no índice 0

Finalmente, podemos testar se nossa modificação fez o trabalho e se o novo valor do ninja no índice 0 pode ser acessado através do ninjas coleção e por meio de nosso método getter:

```
assert (ninjaCollection.firstNinja === "Hachi"  
        && ninjaCollection.ninjas [0] === "Hachi",  
        "Agora Hachi é o primeiro ninja");
```

A Figura 8.2 mostra a saída gerada pela Listagem 8.2. Quando acessamos uma propriedade com um getter (por exemplo, por meio de `ninjaCollection.firstNinja`), o método getter é chamado imediatamente e, neste caso, a mensagem Pegando o primeiro Ninja é registrado. Mais tarde, testamos se a saída é Yoshi e que a mensagem Yoshi é o primeiro ninja é o primeiro ninja é registrado. Procedemos de forma semelhante, atribuindo um novo valor ao firstNinja propriedade,

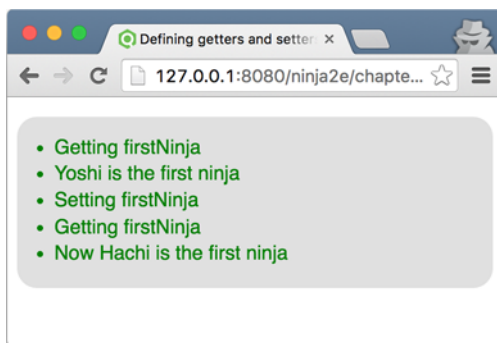


Figura 8.2 A saída da listagem 8.2: se uma propriedade tem um método getter e um método setter, o método getter é implicitamente chamado sempre que lemos o valor da propriedade, e o método setter é chamado sempre que atribuímos um novo valor à propriedade.

e como podemos ver na saída, isso aciona implicitamente a execução do método setter, que gera a mensagem Configurando firstNinja.

Um ponto importante a tirar de tudo isso é que getters e setters nativos nos permitem especificar propriedades que são acessadas como propriedades padrão, mas que são métodos cuja execução é acionada imediatamente quando a propriedade é acessada. Isso é enfatizado ainda mais na figura 8.3.

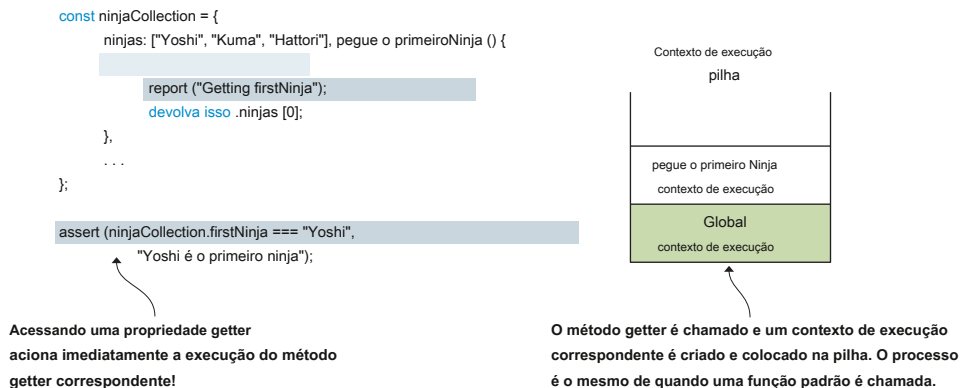


Figura 8.3 Acessar uma propriedade com um método getter chama implicitamente o getter correspondente. O processo é o mesmo como se fosse uma chamada de método padrão e o método getter é executado. Algo semelhante acontece quando atribuímos um valor a uma propriedade por meio de um método setter.

Essa sintaxe para definir um getter e um setter é direta, portanto, não é de se admirar que possamos usar exatamente a mesma sintaxe para definir getters e setters em outras situações. O exemplo a seguir usa classes ES6.

Listagem 8.3 Usando getters e setters com classes ES6

```
class NinjaCollection {
  constructor(){
    this.ninjas = ["Yoshi", "Kuma", "Hattori"];
  }
}
```

```
obter firstNinja () {
    report ("Getting firstNinja");
    retornar this.ninjas [0];
}

definir firstNinja (valor) {
    report ("Configurando firstNinja");
    this.ninjas [0] = valor;
}

}

const ninjaCollection = new NinjaCollection ();

assert (ninjaCollection.firstNinja === "Yoshi",
    "Yoshi é o primeiro ninja");

ninjaCollection.firstNinja = "Hachi";

assert (ninjaCollection.firstNinja === "Hachi"
    && ninjaCollection.ninjas [0] === "Hachi",
    "Agora Hachi é o primeiro ninja");
```

Define um getter e um setter dentro de uma classe ES6

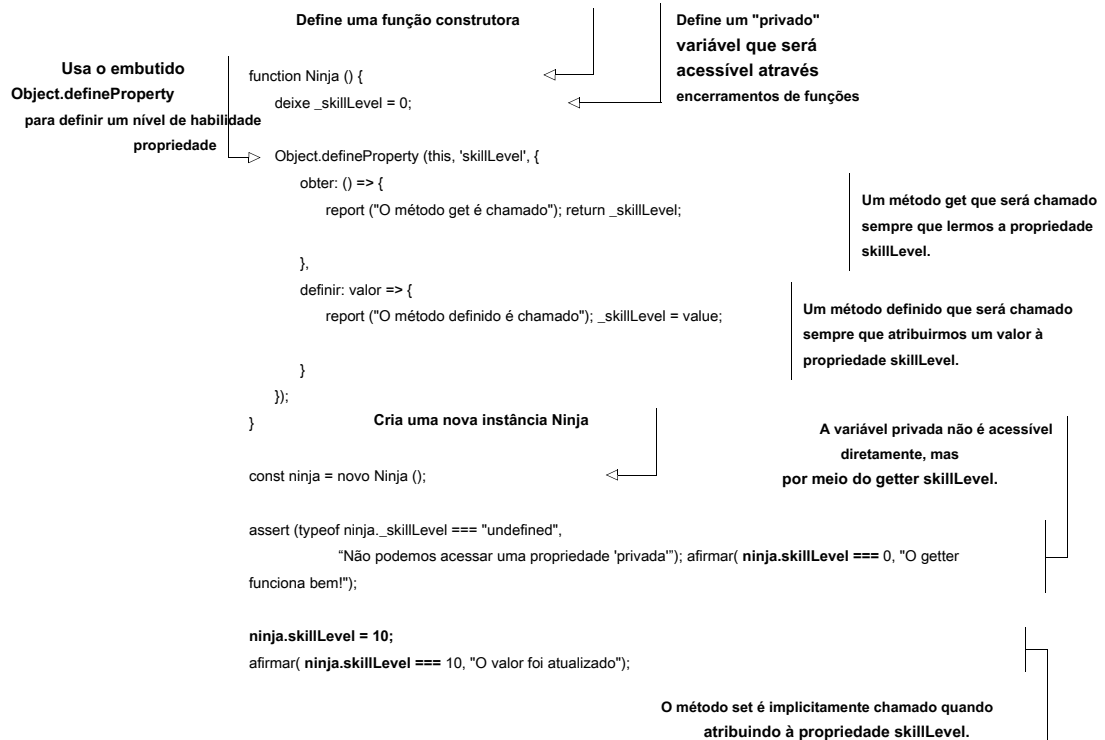
Isso modifica o código da listagem 8.2 para incluir classes ES6. Guardamos todos os testes para verificar se o exemplo ainda funciona conforme o esperado.

NOTA Nem sempre precisamos definir um getter e um setter para uma determinada propriedade. Por exemplo, muitas vezes queremos fornecer apenas um getter. Se, nesse caso, ainda tentarmos escrever um valor para essa propriedade, o comportamento exato depende se o código está no modo estrito ou não estrito. Se o código estiver no modo não estrito, atribuir um valor a uma propriedade com apenas um getter não leva a nada; o mecanismo JavaScript irá ignorar silenciosamente nossa solicitação. Se, por outro lado, o código estiver no modo estrito, o mecanismo JavaScript emitirá um erro de tipo, indicando que estamos tentando atribuir um valor a uma propriedade que possui um getter, mas nenhum setter.

Embora especificar getters e setters por meio de classes ES6 e literais de objeto seja fácil, você provavelmente percebeu que algo está faltando. Tradicionalmente, getters e setters são usados para controlar o acesso às propriedades do objeto privado, como na Listagem 8.1. Infelizmente, como já sabemos no capítulo 5, o JavaScript não tem propriedades de objeto privado. Em vez disso, podemos imitá-los por meio de fechamentos, definindo variáveis e especificando métodos de objeto que fecharão sobre essas variáveis. Como com literais e classes de objeto, nossos métodos getter e setter não são criados no mesmo escopo de função que as variáveis que poderíamos usar para propriedades de objeto privado, não podemos fazer isso. Felizmente, há uma maneira alternativa, por meio do `Object.defineProperty` método.

No capítulo 7, você viu que o `Object.defineProperty` O método pode ser usado para definir novas propriedades passando em um objeto descritor de propriedade. Entre outras coisas, o descritor de propriedade pode incluir um `obter` e um `definir` propriedade que define os métodos `getter` e `setter` da propriedade.

Usaremos esse recurso para modificar a listagem 8.1 para implementar getters e setters integrados que controlam o acesso a uma propriedade de objeto "privada", conforme mostrado na listagem a seguir.

Listagem 8.4 Definindo getters e setters com `Object.defineProperty`

Neste exemplo, primeiro definimos um `Ninja` função construtora com `_` nível de habilidade variável que usaremos como uma variável privada, assim como na listagem 8.1.

Em seguida, no objeto recém-criado, referenciado pelo esta palavra-chave, definimos um nível de habilidade propriedade usando o embutido `Object.defineProperty` método:

```
Object.defineProperty (this, 'skillLevel', {
  obter: () => {
    report ("O método get é chamado"); return _skillLevel;
  },
  definir: valor => {
    report ("O método definido é chamado"); _skillLevel = value;
  }
});
```

Porque queremos o nível de habilidade propriedade para controlar o acesso a uma variável privada, especificamos um `obter` e um `definir` método que será chamado sempre que a propriedade for acessada.

Observe que, ao contrário de getters e setters especificados em literais e classes de objeto, o `obter` e `definir` métodos definidos através de `Object.defineProperty` são criados no mesmo escopo que o "privado" nível de habilidade variável. Ambos os métodos criam um fechamento

em torno da variável privada, e podemos acessar essa variável privada apenas por meio desses dois métodos.

O resto do código funciona exatamente como nos exemplos anteriores. Nós criamos um novo Ninja instância e verifique se não podemos acessar a variável privada diretamente. Em vez disso, todas as interações devem passar por getter e setter, que agora usamos como se fossem propriedades de objeto padrão:

```
ninja.skillLevel === 0
ninja.skillLevel = 10
```

Ativa o método getter

Ativa o método setter

Como você pode ver, a abordagem com `Object.defineProperty` é mais detalhado e complicado do que getters e setters em literais e classes de objeto. Mas em certos casos, quando precisamos de propriedades de objetos privados, vale a pena.

Independentemente da forma como os definimos, getters e setters nos permitem definir propriedades de objeto que são usadas como propriedades de objeto padrão, mas são métodos que podem executar código adicional sempre que lemos ou gravamos em uma propriedade particular. Este é um recurso incrivelmente útil que nos permite realizar o registro, validar valores de atribuição e até mesmo notificar outras partes do código quando certas alterações ocorrem. Vamos explorar alguns desses aplicativos.

8.1.2 Usando getters e setters para validar valores de propriedade

Como estabelecemos até agora, um setter é um método executado sempre que gravamos um valor na propriedade correspondente. Podemos tirar proveito dos setters para executar uma ação sempre que o código tentar atualizar o valor de uma propriedade. Por exemplo, uma das coisas que podemos fazer é validar o valor passado. Dê uma olhada no código a seguir, que garante que nosso nível de habilidade propriedade pode ser atribuída apenas a valores inteiros.

Listagem 8.5: Validando atribuições de valor de propriedade com setters

```
function Ninja () {
  deixe _skillLevel = 0;

  Object.defineProperty (this, 'skillLevel', {
    get: () => _skillLevel, set: value => {

      if (! Number.isInteger (value)) {
        lance novo TypeError ("Nível de habilidade deve ser um número");
      }
      _skillLevel = value;
    }
  });
}

const ninja = novo Ninja ();

ninja.skillLevel = 10;
afirmar( ninja.skillLevel === 10, "O valor foi atualizado");
```

Verifica se o valor passado é um número inteiro. Se não for, uma exceção é lançada.

Podemos atribuir um valor inteiro para a propriedade.

```
tentar {
  ninja.skillLevel = "Ótimo";
  fail ("Não deveria estar aqui"); } catch (e) {

  pass ("Definir um valor não inteiro lança uma exceção");
}
```

Tentando atribuir um valor não inteiro (em neste caso, uma string) resulta em uma exceção lançada do método `setter`.

Este exemplo é uma extensão direta da listagem 8.4. A única grande diferença é que agora, sempre que um novo valor é atribuído ao nível de habilidade, verificamos se o valor passado é um número inteiro. Se não for, uma exceção é lançada e o `_nível de habilidade` variável não será modificada. Se tudo correr bem e um valor inteiro for recebido, terminaremos com um novo valor de `_nível de habilidade` variável:

```
definir: valor => {
  if (! Number.isInteger (value)) {
    lance novo TypeError ("Nível de habilidade deve ser um número");
  }
  _skillLevel = value;
}
```

Ao testar este código, primeiro verificamos se tudo vai bem se atribuímos um inteiro:

```
ninja.skillLevel = 10;
assert (ninja.skillLevel === 10, "O valor foi atualizado");
```

E então testamos a situação em que atribuímos erroneamente um valor de outro tipo, como uma string. Nesse caso, devemos terminar com uma exceção.

```
tentar {
  ninja.skillLevel = "Ótimo";
  fail ("Não deveria estar aqui"); } catch (e) {

  pass ("Definir um valor não inteiro lança uma exceção");
}
```

É assim que você evita todos aqueles bichinhos bobos que acontecem quando um valor do tipo errado acaba em uma determinada propriedade. Claro, isso adiciona sobrecarga, mas esse é um preço que às vezes temos que pagar para usar com segurança uma linguagem altamente dinâmica como JavaScript.

Este é apenas um exemplo da utilidade dos métodos `setter`; há muitos mais que não exploraremos explicitamente. Por exemplo, você pode usar o mesmo princípio para rastrear o histórico de valores, fazer o registro, fornecer notificação de mudança e muito mais.

8.1.3 Usando *getters* e *setters* para definir propriedades computadas

Além de ser capaz de controlar o acesso a certas propriedades do objeto, `getters` e `setters` podem ser usados para definir *propriedades computadas*, propriedades cujo valor é calculado por solicitação. Propriedades computadas não armazenam um valor; eles fornecem um `obter` e / ou um `definir` método para recuperar e definir outras propriedades indiretamente. No exemplo a seguir, o objeto tem duas propriedades, `nome` e `clã`, que usaremos para calcular a propriedade `título completo`.

Listagem 8.6 Definindo propriedades computadas

```
const shogun = {
  nome: "Yoshiaki",
  clã: "Ashikaga",
  get fullTitle () {
    return this.name + " " + this.clan;
  },
  set fullTitle (value) {
    const segmentos = valor.split(" "); this.name = segmentos [0];

    this.clan = segmentos [1];
  }
};
```

Define um método getter em uma propriedade fullTitle de um literal de objeto que calcula o valor concatenando duas propriedades de objeto

Define um método setter em uma propriedade fullTitle de um literal de objeto que divide o valor passado e atualiza duas propriedades padrão

```
assert (shogun.name === "Yoshiaki", "Nosso shogun Yoshiaki"); assert (shogun.clan === "Ashikaga", "Do clã Ashikaga"); assert (shogun.fullTitle === "Yoshiaki Ashikaga",
```

```
    "O nome completo agora é Yoshiaki Ashikaga");
```

```
shogun.fullTitle = "Ieyasu Tokugawa";
```

```
assert (shogun.name === "Ieyasu", "Nosso shogun Ieyasu"); assert (shogun.clan === "Tokugawa", "Do clã Tokugawa"); assert (shogun.fullTitle === "Ieyasu Tokugawa",
```

```
    "O nome completo agora é Ieyasu Tokugawa");
```

Atribuir um valor à propriedade fullTitle chama o método set, que calcula e atribui novos valores às propriedades de nome e clã.

O nome e as propriedades do clã são normais propriedades cujo valores são diretamente obtido. Acessando a propriedade fullTitle chama o método get, que calcula o valor.

Aqui nós definimos um Shogun objeto, com duas propriedades padrão, nome e clã. Além disso, especificamos um método getter e um método setter para uma propriedade computada, título completo:

```
const shogun = {
  nome: "Yoshiaki",
  clã: "Ashikaga",
  get fullTitle () {
    return this.name + " " + this.clan;
  },
  set fullTitle (value) {
    const segmentos = valor.split(" "); this.name = segmentos [0];

    this.clan = segmentos [1];
  }
};
```

O obter método calcula o valor do título completo propriedade, a pedido, concatenando o nome e clã propriedades.

O definir método, por outro lado, usa o embutido dividir método, disponível para todas as strings, para dividir a string atribuída em segmentos pelo caractere de espaço. O primeiro segmento representa o nome e é atribuído ao nome propriedade, enquanto o segundo segmento representa o clã e é atribuído ao clã propriedade.

Isso cuida de ambas as rotas: Lendo o título completo propriedade calcula seu valor, e a gravação no título completo propriedade modifica as propriedades que constituem o valor da propriedade.

Para ser honesto, não precisamos usar propriedades computadas. Um método chamado `getFullTitle` poderia ser igualmente útil, mas as propriedades computadas podem melhorar a clareza conceitual de nosso código. Se um determinado valor (neste caso, o título completo valor) depende só no estado interno do objeto (neste caso, no nome e clã propriedades), faz todo o sentido representá-lo como um campo de dados, uma propriedade, em vez de uma função.

Isso conclui nossa exploração de getters e setters. Você viu que eles são uma adição útil à linguagem que pode nos ajudar a lidar com o registro, validação de dados e detecção de mudanças nos valores das propriedades. Infelizmente, às vezes isso não é suficiente. Em certos casos, precisamos controlar todos os tipos de interações com nossos objetos e, para isso, podemos usar um tipo de objeto completamente novo: a *proxy*.

8,2

Usando proxies para controlar o acesso

UMA *procuração* é um substituto por meio do qual controlamos o acesso a outro objeto. Ele nos permite definir ações personalizadas que serão executadas quando um objeto está sendo interagido - por exemplo, quando um valor de propriedade é lido ou definido, ou quando um método é chamado. Você pode pensar em proxies como quase uma generalização de getters e setters; mas com cada getter e setter, você controla o acesso a apenas uma única propriedade do objeto, enquanto os proxies permitem que você manipule genericamente todas as interações com um objeto, incluindo até mesmo chamadas de método.

Podemos usar proxies quando tradicionalmente usamos getters e setters, como para registro, validação de dados e propriedades computadas. Mas os proxies são ainda mais poderosos. Eles nos permitem adicionar facilmente perfis e medições de desempenho ao nosso código, preencher automaticamente as propriedades do objeto para evitar exceções nulas incômodas e agrupar objetos de host, como o DOM, a fim de reduzir incompatibilidades entre navegadores.



NOTA Os proxies são introduzidos pelo ES6. Para suporte de navegador atual, consulte <http://mng.bz/9uEM>.

Em JavaScript, podemos criar proxies usando o built-in Proxy construtor. Vamos começar de forma simples, com um proxy que intercepta todas as tentativas de leitura e gravação nas propriedades de um objeto.

Listagem 8.7 Criando proxies com o **Proxy** construtor

```

O imperador é nosso objeto alvo.
imperador const = {nome: "Komei"}; representante const = novo Proxy ({imperador,
{

```

← Cria um proxy com o construtor Proxy que recebe o objeto que o proxy envolve ...

Acessa o
nome da propriedade
ambos através
o Imperador
objeto e
através de
objeto proxy

```
obter: (alvo, chave) => {
    relatório ("Leitura" + tecla + "através de um proxy"); chave de retorno no alvo? alvo
    [chave]

    : "Não incomode o imperador!"

},
definir: (alvo, chave, valor) => {
    relatório ("Escrita" + tecla + "através de um proxy"); alvo [chave] = valor;

}
});
```

... e um objeto
com armadilhas que
será chamado
ao ler
(obter) e
escrever (definir) para
propriedades.

```
assert (emperor.name === "Komei", "O nome do imperador é Komei"); afirmar (representante.name === "Komei",
    "Podemos obter a propriedade do nome por meio de um proxy"); propriedade existente
```

Acessando um não
diretamente no objeto
retorna indefinido.

```
assert (emperor.nickname === undefined,
    "O imperador não tem apelido");
assert (representante.nickname === "Não incomode o imperador!",
    "O proxy salta quando fazemos pedidos inadequados");
```

Acessando um
propriedade
através de um proxy
detecta que o
objeto não
existe em nosso
objeto alvo, então
um aviso
mensagem é
devolvida.

```
representante.nickname = "Tenno";
assert (emperor.nickname === "Tenno",
    "O imperador agora tem um apelido");
assert (representante.nickname === "Tenno",
    "O apelido também pode ser acessado pelo proxy");
```

Adiciona uma propriedade por meio do proxy. A propriedade é acessível
tanto por meio do objeto de destino quanto por meio do proxy.

Primeiro criamos nossa base imperador objeto que tem apenas um nome propriedade. Em seguida, usando o integrado Proxy construtor, nós envolvemos nosso imperador objeto (ou *alvo* objeto, como é comumente chamado) em um objeto proxy denominado representante. Durante a construção do proxy, como segundo argumento, também enviamos um objeto que especifica *armadilhas*, funções que serão chamadas quando certas ações forem realizadas em um objeto:

```
representante const = novo Proxy (imperador, {
    obter: (alvo, chave) => {
        relatório ("Leitura" + tecla + "através de um proxy"); chave de retorno no alvo? alvo
        [chave]

        : "Não incomode o imperador!"

    },
    definir: (alvo, chave, valor) => {
        relatório ("Escrita" + tecla + "através de um proxy"); alvo [chave] = valor;

    }
});
```

Nesse caso, especificamos duas armadilhas: a obter armadilha que será chamada sempre que tentarmos ler um valor de uma propriedade por meio do proxy, e um definir trap que será chamado sempre que definirmos um valor de propriedade por meio do proxy. O *obter* trap executa a seguinte funcionalidade: Se o objeto de destino tiver uma propriedade, essa propriedade será retornada; e se o

objeto não tem uma propriedade, retornamos uma mensagem avisando nosso usuário para não incomodar o imperador com detalhes frívolos.

```
obter: (alvo, chave) => {
  relatório ("Leitura" + tecla + "através de um proxy"); chave de retorno no alvo? alvo
  [chave]
    : "Não incomode o imperador!"
}
```

Em seguida, testamos se podemos acessar o nome propriedade tanto diretamente através do alvo imperador objeto, bem como por meio de nosso objeto proxy:

```
assert (emperor.name === "Komei", "O nome do imperador é Komei"); afirmar (representante.name === "Komei",
  "Podemos obter a propriedade do nome por meio de um proxy");
```

Se acessarmos o nome propriedade diretamente através do imperador objeto, o valor Komei é devolvido. Mas se acessarmos o nome propriedade através do procuração objeto, o obter trap é implicitamente chamado. Porque o nome propriedade é encontrada no alvo imperador objeto, o valor Komei também é retornado. Veja a figura 8.4.

NOTA É importante enfatizar que as armadilhas de proxy são ativadas da mesma maneira que getters e setters. Assim que executamos uma ação (por exemplo, acessar um valor de propriedade em um proxy), a armadilha correspondente é chamada implicitamente e o mecanismo JavaScript passa por um processo semelhante, como se tivéssemos chamado explicitamente uma função.

Por outro lado, se acessarmos um não existente apelido propriedade diretamente no alvo imperador objeto, obteremos, sem surpresa, um Indefinido valor. Mas se tentarmos acessá-lo por meio de nosso procuração objeto, o obter manipulador será ativado. Porque o alvo

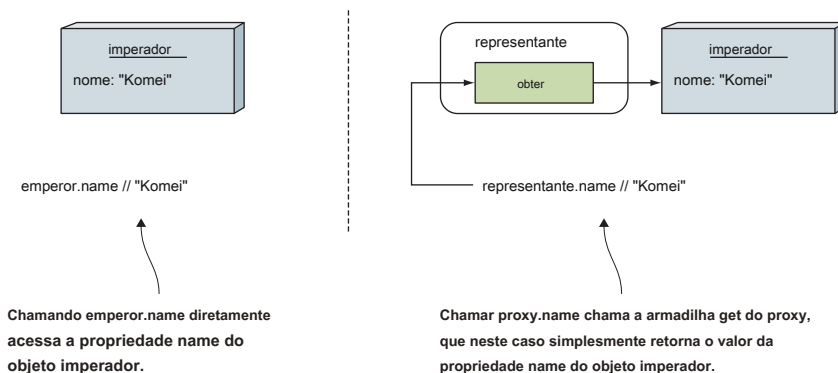


Figura 8.4 Acessando o **nome** propriedade direta (à esquerda) e indiretamente, por meio de um proxy (à direita)

imperador objeto não tem um apelido propriedade, o proxy obter armadilha irá retornar o Não incomode o imperador! mensagem.

Continuaremos o exemplo atribuindo uma nova propriedade por meio de nosso objeto proxy: representante.nickname = "Tenno". Como a atribuição é feita por meio de um proxy, e não diretamente, o definir trap, que registra uma mensagem e atribui uma propriedade ao nosso alvo imperador objeto, está ativado:

```
definir: (alvo, chave, valor) => {  
    relatório ("Escrita" + tecla + "através de um proxy"); alvo [chave] = valor;  
}
```

Naturalmente, a propriedade recém-criada pode ser acessada por meio do objeto proxy e do objeto de destino:

```
assert (emperor.nickname === "Tenno",  
        "O imperador agora tem um apelido");  
assert (representante.nickname === "Tenno",  
        "O apelido também pode ser acessado pelo proxy");
```

Esta é a essência de como usar proxies: por meio do Proxy construtor, criamos um objeto proxy que controla o acesso ao objeto de destino ativando certas armadilhas, sempre que uma operação é realizada diretamente em um proxy.

Neste exemplo, usamos o obter e definir armadilhas, mas muitas outras armadilhas integradas nos permitem definir manipuladores para várias ações de objeto (consulte <http://mng.bz/ba55>) Por exemplo:

- O *Aplique* trap será ativado ao chamar uma função, e o *construir* armadilha ao usar o novo operador.
- O *obter* e *definir* armadilhas serão ativadas ao ler / gravar em uma propriedade. O *enumerar* armadilha
- será ativada para dentro declarações.
- getPrototypeOf e setPrototypeOf será ativado para obter e definir o valor do protótipo.

Podemos interceptar muitas operações, mas passar por todas elas está fora do escopo deste livro. Por enquanto, voltamos nossa atenção para algumas operações que não podemos substituir: igualdade (== ou ===), instancia de, e a tipo de operador.

Por exemplo, a expressão `x == y` (ou um mais estrito `x === y`) é usado para verificar se `x` e `y` referem-se a objetos idênticos (ou têm o mesmo valor). Este operador de igualdade tem algumas suposições. Por exemplo, comparar dois objetos deve sempre retornar o mesmo valor para os mesmos dois objetos, o que não é algo que possamos garantir se esse valor for determinado por uma função especificada pelo usuário. Além disso, o ato de comparar dois objetos não deve dar acesso a um desses objetos, o que seria o caso se a igualdade pudesse ser interceptada. Por razões semelhantes, o instancia de e a tipo de os operadores não podem ser presos.

Agora que sabemos como os proxies funcionam e como criá-los, vamos explorar alguns de seus aspectos práticos, como usar proxies para registro, medição de desempenho, propriedades de preenchimento automático e implementação de matrizes que podem ser acessadas com índices negativos. Começaremos com o registro.

8.2.1 Usando proxies para registro

Uma das ferramentas mais poderosas ao tentar descobrir como o código funciona ou ao tentar chegar à raiz de um bug desagradável é *exploração madeireira*, o ato de enviar informações que consideramos úteis em um determinado momento. Podemos, por exemplo, querer saber quais funções são chamadas, há quanto tempo estão em execução, quais propriedades são lidas ou gravadas e assim por diante.

Infelizmente, ao implementar o log, geralmente espalhamos as instruções de log por todo o código. Dê uma olhada no Ninja exemplo usado anteriormente neste capítulo.

Listagem 8.8 Log sem proxies

```
function Ninja () {
  deixe _skillLevel = 0;

  Object.defineProperty (this, 'skillLevel', {
    obter: () => {
      report ("o método de obtenção skillLevel é chamado"); return _skillLevel;
    },
    definir: valor => {
      report ("o método de conjunto skillLevel é chamado"); _skillLevel = value;
    }
  });
}

const ninja = novo Ninja (); ninja.skillLevel;

ninja.skillLevel = 4;
```

Registramos sempre que o A propriedade skillLevel foi lida ...

... skillLevel é gravado.

Lê o skillLevel propriedade e gatilhos o método get

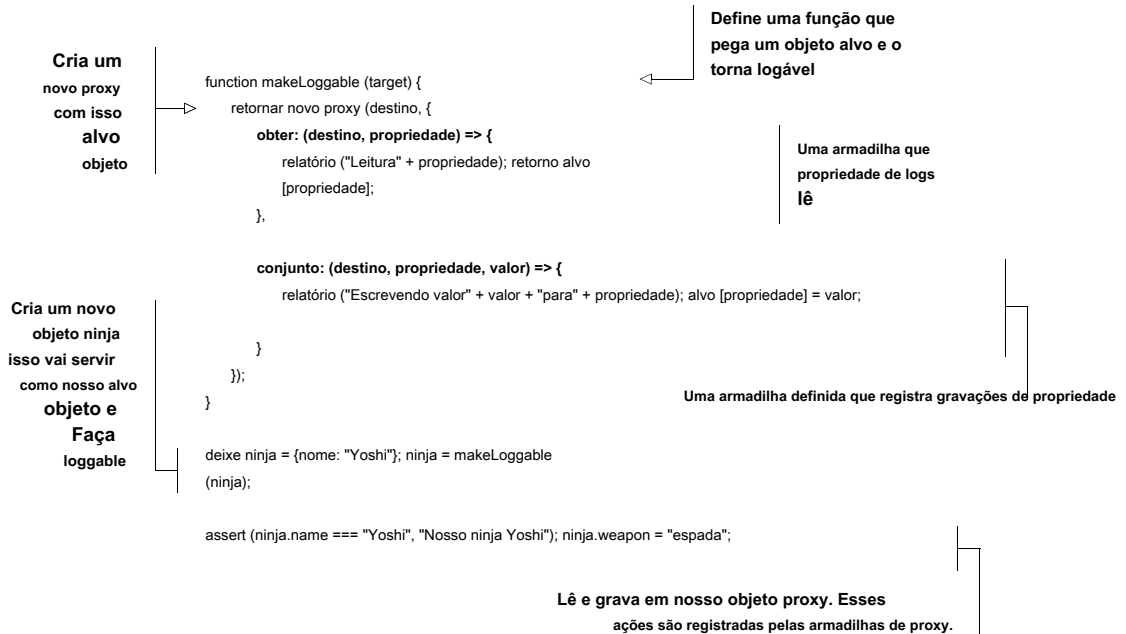
Grava na propriedade skillLevel e aciona o método set

Nós definimos um Ninja função construtora que adiciona um getter e um setter ao nível de habilidade , que registra todas as tentativas de leitura e gravação nessa propriedade.

Observe que esta não é uma solução ideal. Desordenamos nosso código de domínio que lida com a leitura e gravação de uma propriedade de objeto com código de registro. Além disso, se no futuro precisarmos de mais propriedades no ninja objeto, temos que ter cuidado para não esquecer de adicionar instruções de registro adicionais a cada nova propriedade.

Felizmente, um dos usos diretos dos proxies é habilitar o registro sempre que lermos ou escrevermos em uma propriedade, mas de uma maneira muito mais agradável e limpa. Considere o seguinte exemplo.

Listagem 8.9 O uso de proxies torna mais fácil adicionar registro a objetos



Aqui nós definimos um makeLoggable função que leva um alvo objeto e retorna um novo Proxy que tem um manipulador com um obter e um definir armadilha. Essas armadilhas, além de ler e gravar na propriedade, registram as informações sobre a propriedade lida ou gravada.

Em seguida, criamos um ninja objeto com um nome propriedade, e passamos para o makeLoggable, na qual será usado como destino para um proxy recém-criado. Em seguida, atribuímos o proxy de volta ao ninja identificador, substituindo-o. (Não se preocupe, nosso original ninja objeto é mantido ativo como o objeto de destino do nosso proxy.)

Sempre que tentamos ler uma propriedade (por exemplo, com `ninja.name`), a obter trap será chamado e as informações sobre qual propriedade foi lida serão registradas. Algo semelhante acontecerá ao gravar em uma propriedade: `ninja.weapon = "espada"`.

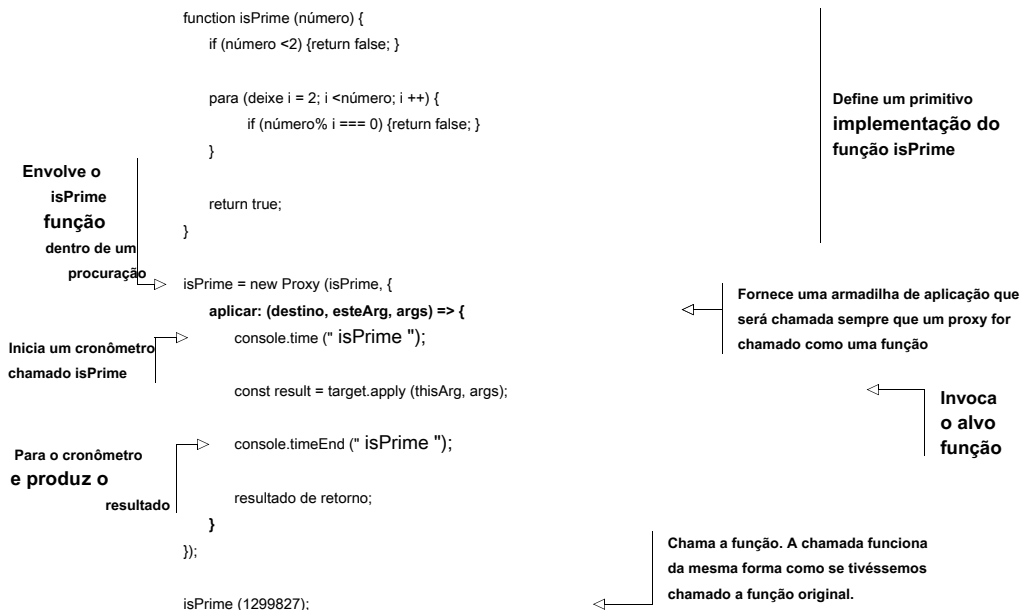
Observe como isso é muito mais fácil e transparente quando comparado à maneira padrão de usar getters e setters. Não precisamos misturar nosso código de domínio com nosso código de registro e não há necessidade de adicionar registros separados para cada propriedade de objeto. Em vez disso, todas as leituras e gravações de propriedades passam por nossos métodos de armadilha de objeto proxy. O registro foi especificado em apenas um lugar e é reutilizado quantas vezes forem necessárias, em quantos objetos forem necessários.

8.2.2 Usando proxies para medir o desempenho

Além de ser usado para registrar acessos de propriedade, os proxies podem ser usados para medir o desempenho de invocações de função, sem mesmo modificar o código-fonte de um

função. Digamos que queremos medir o desempenho de uma função que calcula se um número é primo, conforme mostrado na lista a seguir.

Listagem 8.10 Medindo o desempenho com proxies



Neste exemplo, temos um simples isPrime função. (A função exata não importa; estamos usando-a como um exemplo de uma função cuja execução pode durar um período de tempo não trivial.)

Agora imagine que precisamos medir o desempenho do isPrime função, mas sem modificar seu código. Poderíamos envolver a função em um proxy que tenha uma armadilha que será chamada sempre que a função for chamada:

```

isPrime = new Proxy (isPrime, {
  aplicar: (destino, esteArg, args) => {
    ...
  }
});

```

Nós usamos o isPrime functionas o objeto de destino de um proxy recém-construído. Além disso, fornecemos um manipulador com um Aplique trap que será executado na invocação da função.

Da mesma forma, como no exemplo anterior, atribuímos o proxy recém-criado ao isPrime identificador. Dessa forma, não temos que alterar nenhum código que chama a função cujo tempo de execução queremos medir; o resto do código do programa é completamente alheio às nossas mudanças. (O que acha disso para uma ação furtiva ninja?)

Sempre que isPrime função é chamada, essa chamada é redirecionada para o nosso proxy Aplique armadilha, que iniciará um cronômetro com o console.time método (lembre-se do capítulo 1), chame o original isPrime função, registre o tempo decorrido e, finalmente, retorne o resultado do isPrime invocação.

8.2.3 Usando proxies para preencher automaticamente as propriedades

Além de simplificar o registro, os proxies podem ser usados para preencher automaticamente as propriedades. Por exemplo, imagine que você precise modelar a estrutura de pastas do seu computador, na qual um objeto de pasta pode ter propriedades que também podem ser pastas. Agora imagine que você precise modelar um arquivo no final de um longo caminho, como este:

```
rootFolder.ninjasDir.firstNinjaDir.ninjaFile = "yoshi.txt";
```

Para criar isso, você pode escrever algo com as seguintes linhas:

```
const rootFolder = nova pasta (); rootFolder.ninjasDir = nova pasta ();
```

```
rootFolder.ninjasDir.firstNinjaDir = nova pasta (); rootFolder.ninjasDir.firstNinjaDir.ninjaFile = "yoshi.txt";
```

Parece um pouco mais tedioso do que o necessário, não é? É aqui que as propriedades de preenchimento automático entram em jogo; basta dar uma olhada no exemplo a seguir.

Listagem 8.11 Preenchendo automaticamente propriedades com proxies

```
function Folder () {
  retornar novo Proxy ({}, {
    obter: (destino, propriedade) => {
      relatório ("Leitura" + propriedade);

      if (! (propriedade no destino)) {
        destino [propriedade] = nova pasta ();
      }

      retorno alvo [propriedade];
    }
  });
}

const rootFolder = nova pasta ();

tentar {
  rootFolder.ninjasDir.firstNinjaDir.ninjaFile = "yoshi.txt"; pass ("Uma exceção não foi levantada");
}
pegar (e) {
  fail ("Ocorreu uma exceção");
}
```

Registra todas as leituras para o nosso objeto

Se o acessado propriedade não existir, nós o criamos.

Sempre que uma propriedade é acessada, o get trap, que cria uma propriedade se não existe, está ativado.

Nenhuma exceção será levantada.

Normalmente, se considerarmos apenas o código a seguir, esperaríamos que uma exceção fosse levantada:

```
const rootFolder = nova pasta ();
rootFolder.ninjasDir.firstNinjaDir.ninjaFile = "yoshi.txt";
```

Estamos acessando uma propriedade, `firstNinjaDir`, de uma propriedade indefinida, `ninjasDir`, do pasta raiz objeto. Mas se executarmos o código, você verá que está tudo bem, como mostra a figura 8.5.

Isso acontece porque estamos usando proxies. Cada vez que acessamos uma propriedade, o proxy obter armadilha é ativada. Se nosso objeto de pasta já contém a propriedade solicitada, seu valor é retornado e, caso não contenha, uma nova pasta é criada e atribuída à propriedade. É assim que duas de nossas propriedades, `ninjasDir` e

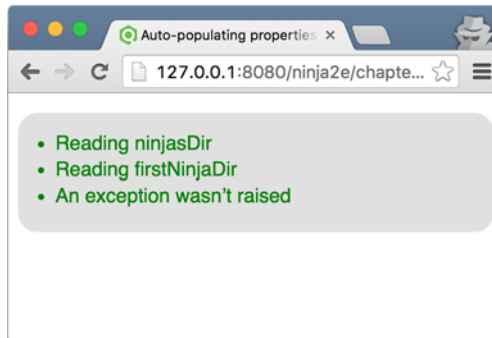


Figura 8.5 A saída da execução do código da listagem 8.11

`firstNinjaDir`, são criados. Solicitando um valor de uma propriedade não inicializada aciona sua criação.

Finalmente, temos uma ferramenta para nos livrar de alguns casos da incômoda exceção nula!

8.2.4 Usando proxies para implementar índices de array negativos

Em nossa programação do dia a dia, geralmente trabalhamos com *muito* de matrizes. Vamos explorar como tirar proveito dos proxies para tornar o tratamento com arrays um pouco mais agradável.

Se o seu histórico de programação vem de linguagens como Python, Ruby ou Perl, você pode estar acostumado com índices de array negativos, que permitem usar índices negativos para acessar itens de array na parte de trás, conforme mostrado no seguinte snippet:

```
const ninjas = ["Yoshi", "Kuma", "Hattori"];
```

```
ninjas[0]; // "Yoshi"
ninjas[1]; // "Kuma"
ninjas[2]; // "Hattori"
```

```
ninjas[-1]; // "Hattori"
ninjas[-2]; // "Kuma"
ninjas[-3]; // "Yoshi"
```

Acesso padrão a itens da matriz, com índices de matriz

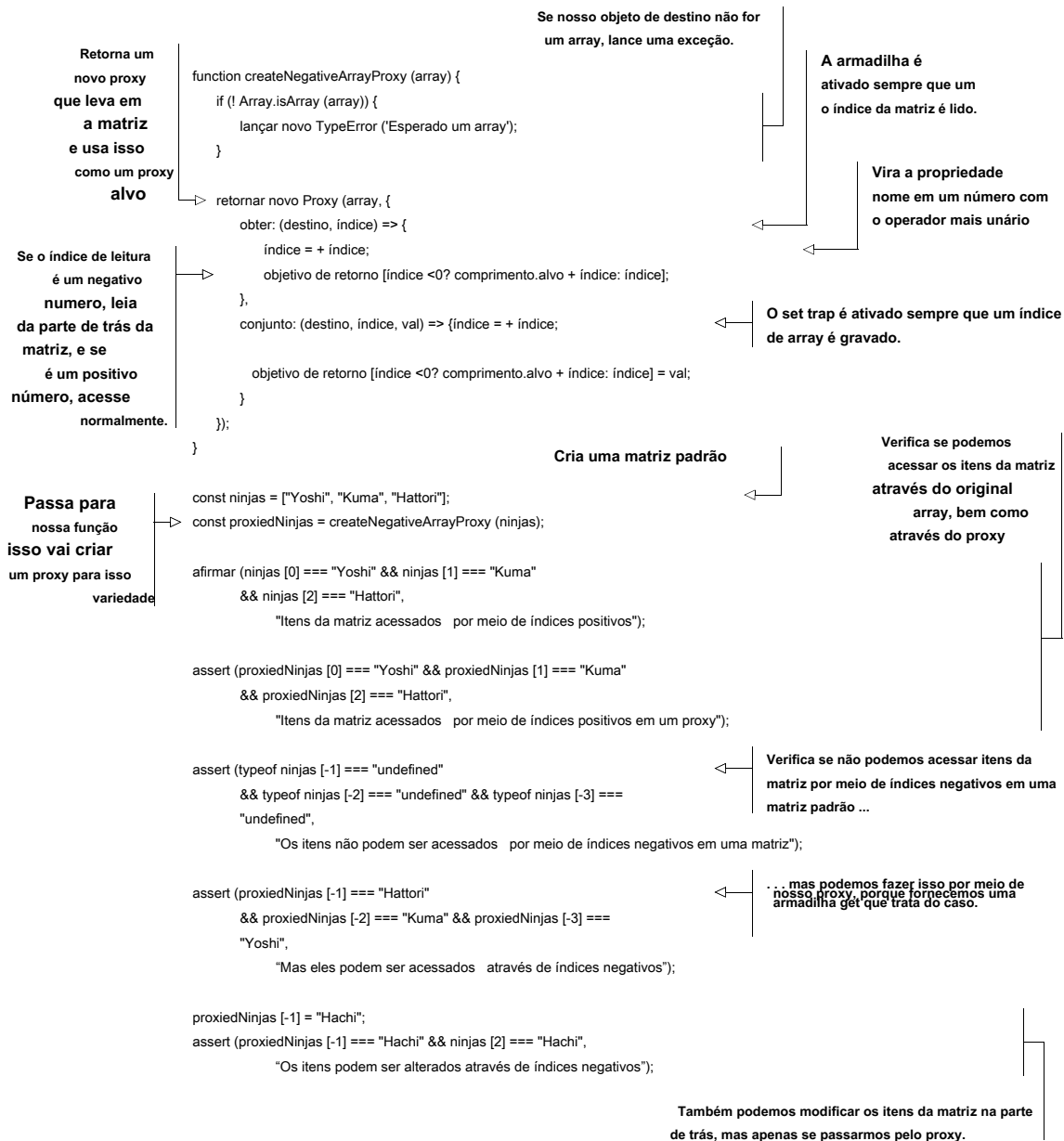
Os índices de array negativos nos permitem acessar itens de array de trás, começando com `-1`, que acessa o último item de array.

Agora compare o código que normalmente usamos para acessar o último item da matriz, `ninjas[ninjas.length-1]`, com o código que podemos usar se a nossa linguagem de escolha suportar índices de array negativos, `ninjas[-1]`. Veja como isso é muito mais elegante?

Infelizmente, o JavaScript não oferece suporte integrado para índices de array negativos, mas podemos imitar essa capacidade por meio de proxies. Para explorar esse conceito, veremos um

versão ligeiramente simplificada do código escrito por Sindre Sorhus (<https://github.com/sindresorhus/negative-array>), conforme mostrado na lista a seguir.

Listagem 8.12 Índices de array negativos com proxies



Neste exemplo, definimos uma função que criará um proxy para um array passado. Como não queremos que nosso proxy funcione com outros tipos de objetos, lançamos uma exceção caso o argumento não seja um array:

```
if (!Array.isArray (array)) {  
    lançar novo TypeError ('Esperado um array');  
}
```

Continuamos criando e retornando um novo proxy com duas armadilhas, um obter armadilha que será ativada sempre que tentarmos ler um item do array, e um definir armadilha que será ativada sempre que escrevermos em um item da matriz:

```
retornar novo Proxy (array, {  
    obter: (destino, índice) => {  
        índice = + índice;  
        objetivo de retorno [índice < 0? comprimento.alvo + índice: índice];  
    },  
    conjunto: (alvo, índice, val) => {  
        índice = + índice;  
        objetivo de retorno [índice < 0? comprimento.alvo + índice: índice] = val;  
    }  
});
```

Os corpos das armadilhas são semelhantes. Primeiro, transformamos a propriedade em um número usando o operador unário mais (`índice = + índice`). Então, se o índice solicitado for menor que 0, acessamos os itens da matriz na parte de trás, ancorando ao comprimento da matriz, e se for maior ou igual a 0, acessamos o item do array de uma maneira padrão.

Por fim, realizamos vários testes para verificar se em arrays normais só podemos acessar itens de array por meio de índices de array positivos e se, se usarmos um proxy, podemos acessar e modificar itens de array por meio de índices negativos.

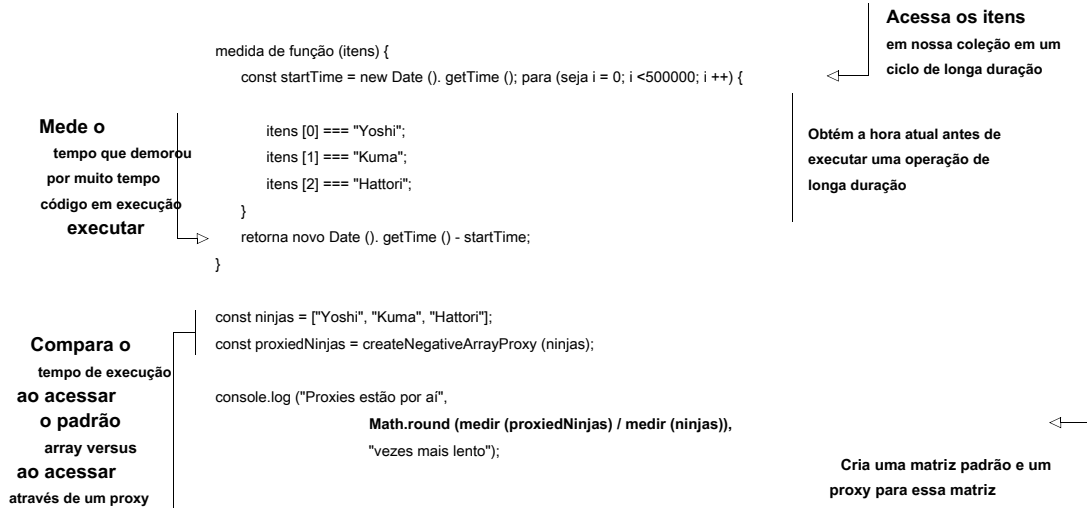
Agora que você viu como usar proxies para obter alguns recursos interessantes, como autopreencher propriedades de objetos e acessar índices de array negativos, que são totalmente impossíveis sem proxies, vamos explorar a desvantagem mais significativa dos proxies: problemas de desempenho.

8.2.5 *Custos de desempenho de proxies*

Como já sabemos, um proxy é um objeto substituto por meio do qual controlamos o acesso a outro objeto. Um proxy pode definir traps, funções que serão executadas sempre que uma determinada operação for realizada em um proxy. E, como você também viu, podemos usar essas armadilhas para implementar funcionalidades úteis, como registro, medições de desempenho, propriedades de preenchimento automático, índices de array negativos e assim por diante. Infelizmente, também há uma desvantagem. O fato de todas as nossas operações terem que passar pelo proxy adiciona uma camada de indireção que nos permite implementar todos esses recursos interessantes, mas ao mesmo tempo introduz uma quantidade significativa de processamento adicional que afeta o desempenho.

Para testar esses problemas de desempenho, podemos construir sobre o exemplo de índices de array negativos da listagem 8.12 e comparar o tempo de execução ao acessar itens em um array normal versus acessar itens por meio de um proxy, conforme mostrado na listagem a seguir.

Listagem 8.13 Verificando as limitações de desempenho de proxies



Como uma única operação do código acontece muito rapidamente para ser medida de maneira confiável, o código deve ser executado muitas vezes para obter um valor mensurável. Frequentemente, essa contagem pode chegar a dezenas de milhares, ou mesmo milhões, dependendo da natureza do código que está sendo medido. Uma pequena tentativa e erro nos permite escolher um valor razoável: neste caso, 500.000.

Também precisamos agrupar a execução do código com dois `new Date (). getTime ()` timestamps: um antes de começarmos a executar o código de destino e um depois. A diferença deles nos diz quanto tempo o código levou para ser executado. Finalmente, podemos comparar os resultados chamando o `medir` função na matriz com proxy e na matriz padrão.

Em nossa máquina, os resultados não são bons para proxies. Acontece que, no Chrome, os proxies são cerca de 50 vezes mais lentos; no Firefox, eles são cerca de 20 vezes mais lentos.

Por enquanto, recomendamos que você tome cuidado ao usar proxies. Embora eles permitam que você seja criativo ao controlar o acesso aos objetos, essa quantidade de controle vem com problemas de desempenho. Você pode usar proxies com código que não é sensível ao desempenho, mas tenha cuidado ao usá-los em código que é executado muito. Como sempre, recomendamos que você teste exaustivamente o desempenho do seu código.

8,3 Resumo

- Podemos monitorar nossos objetos com getters, setters e proxies.
- Usando métodos de acesso (getters e setters), podemos controlar o acesso às propriedades do objeto.

- As propriedades do acessador podem ser definidas usando o `Object.defineProperty` método ou com um especial obter e definir sintaxe como partes de literais de objeto ou classes ES6.
- Uma obter método é implicitamente chamado sempre que tentamos ler, enquanto um definir método é chamado sempre que atribuímos um valor à propriedade do objeto correspondente. Os métodos
- getter podem ser usados para definir propriedades computadas, propriedades cujo valor é calculado por solicitação, enquanto os métodos setter podem ser usados para obter validação de dados e registro.
- Proxies são uma adição ES6 ao JavaScript e são usados para controlar outros objetos.
 - Os proxies nos permitem definir as opções que serão executadas quando um objeto for interagido com (por exemplo, quando uma propriedade é lida ou uma função é chamada).
 - Todas as interações devem passar pelo proxy, que possui armadilhas que são acionadas quando ocorre uma ação específica.
- Use proxies para obter elegância
 - ~~Exemplo de elegância~~
 - Medidas de desempenho
 - Data de validade
 - Preenchimento automático de propriedades de objetos (evitando assim exceções nulas incômodas)
 - Índices de matriz negativos
- Os proxies não são rápidos, então tome cuidado ao usá-los em códigos que são muito executados. Recomendamos que você faça testes de desempenho.

8,4 Exercícios

- 1 Depois de executar o código a seguir, qual das seguintes expressões lançará uma exceção e por quê?

```
const ninja = {
  obter nome () {
    retornar "Akiyama";
  }
}

uma ninja.name ();
b nome const = ninja.name;
```

- 2 No código a seguir, qual mecanismo permite que os getters acessem um objeto privado variável?

```
function Samurai () {
  const _weapon = "katana";
  Object.defineProperty (this, "weapon", {
    obter: () => _weapon});
}

Const samurai = novo Samurai ();
assert (samurai.weapon === "katana", "Um samurai empunhando uma katana");
```

3 Qual das afirmações a seguir será aprovada?

```
const daimyo = {nome: "Matsu", clã: "Takasu"}; const proxy = new Proxy(daimyo, {

  obter: (alvo, chave) => {
    if (chave === "clã") {
      retornar "Tokugawa";
    }
  }
});

assert(daimyo.clan === "Takasu", "Matsu do clã Takasu"); assert(proxy.clan === "Tokugawa", "Matsu do clã Tokugawa?");

proxy.clan = "Tokugawa";

assert(daimyo.clan === "Takasu", "Matsu do clã Takasu"); assert(proxy.clan === "Tokugawa", "Matsu do clã Tokugawa?");
```

4 Qual das afirmações a seguir será aprovada?

```
const daimyo = {nome: "Matsu", clã: "Takasu", armySize: 10000}; const proxy = new Proxy(daimyo, {

  definir: (alvo, chave, valor) => {
    if (key === "armySize") {
      número const = Number.parseInt(valor); if (!Number.isNaN(number)) {

        alvo[chave] = número;
      }
    }
    } outro {
      alvo[chave] = valor;
    }
  },
});

assert(daimyo.armySize === 10000, "Matsu tem 10.000 homens armados"); assert(proxy.armySize === 10000, "Matsu tem 10.000 homens armados");

proxy.armySize = "grande";
assert(daimyo.armySize === "grande", "Matsu tem um grande exército");

daimyo.armySize = "grande";
assert(daimyo.armySize === "grande", "Matsu tem um grande exército");
```

Lidando com coleções



Este capítulo cobre

- Criação e modificação de matrizes
- Usando e reutilizando funções de array
- Criação de dicionários com mapas
- Criação de coleções de objetos únicos com conjuntos

Agora que passamos algum tempo discutindo as particularidades da orientação a objetos em JavaScript, passaremos para um tópico intimamente relacionado: coleções de itens. Começaremos com matrizes, o tipo mais básico de coleção em JavaScript, e examinaremos algumas peculiaridades de matrizes que você não pode esperar se sua experiência em programação estiver em outra linguagem de programação. Continuaremos explorando alguns dos métodos de array integrados que o ajudarão a escrever um código de manipulação de array mais elegante.

A seguir, discutiremos duas novas coleções ES6: mapas e conjuntos. Usando mapas, você pode criar dicionários de um tipo que carregam mapeamentos entre chaves e valores - uma coleção que é extremamente útil em certas tarefas de programação. Conjuntos, por outro lado, são coleções de itens exclusivos em que cada item não pode ocorrer mais de uma vez. Vamos começar nossa exploração com a mais simples e mais comum de todas as coleções: arrays.

.....

Quais são algumas das armadilhas comuns de usar objetos como dicionários ou mapas?

Você sabe?

Quais tipos de valor um par chave / valor pode ter em um Mapa ?

Os itens devem estar em um Definir ser do mesmo tipo?

.....

9,1 Arrays

Arrays são um dos tipos de dados mais comuns. Usando-os, você pode lidar com coleções de itens. Se o seu histórico de programação está em uma linguagem fortemente tipada como C, você provavelmente pensa em matrizes como pedaços sequenciais de memória que hospedam itens do mesmo tipo, onde cada pedaço de memória é de tamanho fixo e tem um índice associado através do qual você pode acessá-lo facilmente.

Mas, como acontece com muitas coisas em JavaScript, os arrays têm uma peculiaridade: eles são apenas objetos. Embora isso leve a alguns efeitos colaterais desagradáveis, principalmente em termos de desempenho, também tem alguns benefícios. Por exemplo, arrays podem acessar métodos, como outros objetos - métodos que tornarão nossas vidas muito mais fáceis.

Nesta seção, veremos primeiro as maneiras de criar arrays. Em seguida, exploraremos como adicionar e remover itens de diferentes posições em um array. Finalmente, examinaremos os métodos de array integrados que tornarão nosso código de manipulação de array muito mais elegante.

9.1.1 Criação de matrizes

Existem duas maneiras fundamentais de criar novos arrays:

- Usando o integrado Variedade constructor
- Usando literais de matriz []

Vamos começar com um exemplo simples em que nós crie uma série de ninjas e uma série de samurais.

Listagem 9.1 Criando arrays

... OU O Array embutido constructor.

O comprimento propriedade diz nós do tamanho de a matriz.

Lendo itens fora da matriz limites resulta em Indefinido.

```
const ninjas = ["Kuma", "Hattori", "Yagyu"]; const samurai = novo Array ("Oda", "Tomoe");

assert (ninjas.length === 3, "Existem três ninjas"); assert (samurai.length === 2, "E apenas dois samurais");

assert (ninjas [0] === "Kuma", "Kuma é o primeiro ninja"); assert (samurai [samurai.length-1] === "Tomoe", "Tomoe é o último samurai");

afirmar (ninjas [4] === indefinido, "Ficamos indefinidos se tentamos acessar um índice fora dos limites");
```

Para criar uma matriz, podemos usar um literal de matriz [] ...

Nós acessamos a matriz itens com índice notação: o primeiro o item está indexado com 0, e o último com array.length - 1.

```
ninjas[4] = "Ishi";
assert (ninjas.length === 5,
  "As matrizes são expandidas automaticamente");
```

Gravar em índices fora dos limites do array estende o array.

```
ninjas.length = 2;
assert (ninjas.length === 2, "Existem apenas dois ninjas agora"); assert (ninjas[0] === "Kuma" && ninjas[1] ===
  "Hattori",
  "Kuma e Hattori");
assert (ninjas[2] === undefined, "Mas nós perdemos Yagyu");
```

Substituir manualmente a propriedade de comprimento por um valor inferior exclui os itens em excesso.

Na Listagem 9.1, começamos criando dois arrays. O ninjas array é criado com um literal de array simples:

```
const ninjas = ["Kuma", "Hattori", "Yagyu"];
```

É imediatamente preenchido com três ninjas: Kuma, Hattori e Yagyu. O samurai array é criado usando o Variedade construtor:

```
const samurai = novo Array ("Oda", "Tomoe");
```

Literais de matriz versus o construtor de matriz

Usar literais de array para criar arrays é preferível em vez de criar arrays com o Variedade construtor. O principal motivo é a simplicidade: `[]` versus `novo Array ()` (2 personagens contra 11 personagens - dificilmente um concurso justo). Além disso, como o JavaScript é altamente dinâmico, nada impede que alguém substitua o código integrado Variedade construtor, o que significa chamar `novo Array ()` não precisa necessariamente criar um array. Portanto, recomendamos que você geralmente se atenha aos literais de array.

Independentemente de como o criamos, cada array tem um comprimento propriedade que especifica o tamanho da matriz. Por exemplo, o comprimento do ninjas array é 3, e contém 3 ninjas. Podemos testar isso com as seguintes afirmações:

```
assert (ninjas.length === 3, "Existem três ninjas"); assert (samurai.length === 2, "E apenas dois
samurais");
```

Como você provavelmente sabe, você acessa itens de matriz usando notação de índice, onde o primeiro item é posicionado no índice 0 e o último item no índice `array.length - 1`. Mas se tentarmos acessar um índice fora desses limites - por exemplo, com `ninjas[4]` (lembre-se de que temos apenas três ninjas!), não obteremos a assustadora exceção "Array index out of bounds" que recebemos na maioria das outras linguagens de programação. Em vez de, Indefinido é retornado, sinalizando que não há nada lá:

```
afirmar (ninjas[4] === indefinido,
  "Ficamos indefinidos se tentarmos acessar um índice fora dos limites");
```

Esse comportamento é uma consequência do fato de que os arrays JavaScript são objetos. Assim como nós obteríamos `Indefinido` se tentamos acessar uma propriedade de objeto inexistente, obtemos `Indefinido` ao acessar um índice de matriz inexistente.

Por outro lado, se tentarmos escrever para uma posição fora dos limites da matriz, como em

```
ninjas [4] = "Ishi";
```

a matriz se expandirá para acomodar a nova situação. Por exemplo, veja a figura 9.1: Essencialmente, criamos uma lacuna na matriz e o item no índice 3 é `Indefinido`.

Isso também muda o valor do comprimento propriedade, que agora relata um valor de 5, mesmo que um item da matriz seja `Indefinido`.

```
var ninjas = ["Kuma", "Hattori", "Yagyu"]
```

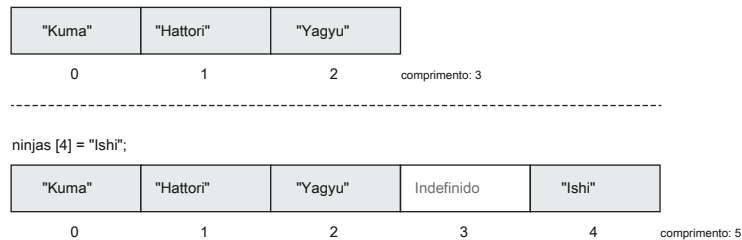


Figura 9.1 Gravar em um índice de array fora dos limites de array expande o array.

Ao contrário da maioria das outras linguagens, em JavaScript, arrays também exibem uma característica peculiar relacionada ao comprimento propriedade: Nada nos impede de alterar manualmente seu valor. Definir um valor maior do que o comprimento atual irá expandir a matriz com `Indefinido` itens, ao passo que definir o valor com um valor inferior irá cortar a matriz, como em

```
ninjas.length = 2 ;
```

Agora que passamos pelos fundamentos da criação de array, vamos examinar alguns dos métodos de array mais comuns.

9.1.2 Adicionar e remover itens nas extremidades de uma matriz

Vamos começar com os seguintes métodos simples que podemos usar para adicionar e remover itens de uma matriz:

- `Empurre` adiciona um item ao final da matriz.
- `não mudar` adiciona um item ao início da matriz.
- `pop` remove um item do final da matriz.
- `mudança` remove um item do início da matriz.

Você provavelmente já usou esses métodos, mas apenas no caso, vamos garantir que estamos na mesma página explorando a lista a seguir.

Listagem 9.2 Adicionando e removendo itens de array

```
const ninjas = [];
assert (ninjas.length === 0, "Um array começa vazio");
```

**Cria um novo,
array vazio**

```
ninjas.push("Kuma");
afirmar (ninjas [0] === "Kuma",
    "Kuma é o primeiro item da matriz");
assert (ninjas.length === 1, "Temos um item na matriz");
```

**Empurra um novo
item até o fim
da matriz**

```
ninjas.push("Hattori");
afirmar (ninjas [0] === "Kuma",
    "Kuma ainda é o primeiro");
assert (ninjas [1] === "Hattori",
    "Hattori é adicionado ao final da matriz"); assert (ninjas.length === 2,
    "Temos dois itens no array!");
```

**Empurra outro
item até o fim
da matriz**

```
ninjas.unshift("Yagyu");
assert (ninjas [0] === "Yagyu",
    "Agora Yagyu é o primeiro item"); afirmar (ninjas [1] ===
    "Kuma",
    "Kuma passou para o segundo lugar"); assert (ninjas [2] ===
    "Hattori",
    "E Hattori para o terceiro lugar"); assert (ninjas.length === 3,
    "Temos três itens no array!");
```

**Usa o método integrado
unshift para inserir o item no
início da matriz. Outros itens
são ajustados de acordo.**

```
const lastNinja = ninjas.pop (); assert (lastNinja ===
    "Hattori",
    "Removemos o Hattori do final da matriz"); assert (ninjas [0] === "Yagyu",
    "Agora Yagyu ainda é o primeiro item"); afirmar (ninjas [1] === "Kuma",
    "Kuma ainda está em segundo lugar"); assert (ninjas.length ===
    2,
    "Agora, existem dois itens na matriz");
```

**Retira o último item
do array**

```
const firstNinja = ninjas.shift (); assert (firstNinja === "Yagyu",
    "Removemos Yagyu do início da matriz"); afirmar (ninjas [0] === "Kuma",
    "Kuma mudou para o primeiro lugar"); assert (ninjas.length === 1,
    "Há apenas um ninja na matriz");
```

**Remove o primeiro
item da matriz. Outros
itens são
movido para a
esquerda em conformidade.**

Neste exemplo, primeiro criamos um novo e vazio ninjas variedade:

```
ninjas = [] // ninjas: []
```

Em cada array, podemos usar o embutido Empurre método para anexar um item ao final da matriz, alterando seu comprimento no processo:

```
ninjas.push("Kuma"); // ninjas: ["Kuma"];
ninjas.push("Hattori"); // ninjas: ["Kuma", "Hattori"];
```

Também podemos adicionar novos itens ao início da matriz usando o `unshift` método:

```
ninjas.unshift("Yagyu"); // ninjas: ["Yagyu", "Kuma", "Hattori"];
```

Observe como os itens da matriz existentes são ajustados. Por exemplo, antes de chamar o `unshift` método, "Kuma" estava no índice 0, e depois está no índice 1

Também podemos remover elementos do final ou do início do array. Chamando o integrado `pop` método remove um elemento do final da matriz, reduzindo o comprimento da matriz no processo:

```
var lastNinja = ninjas.pop(); // ninjas: ["Yagyu", "Kuma"]
// lastNinja: "Hattori"
```

Também podemos remover um item do início da matriz usando o `shift` método:

```
var firstNinja = ninjas.shift(); // ninjas: ["Kuma"]
// firstNinja: "Yagyu"
```

A Figura 9.2 mostra como `empurre`, `estale`, `mude`, e `não mudar` modificar matrizes.

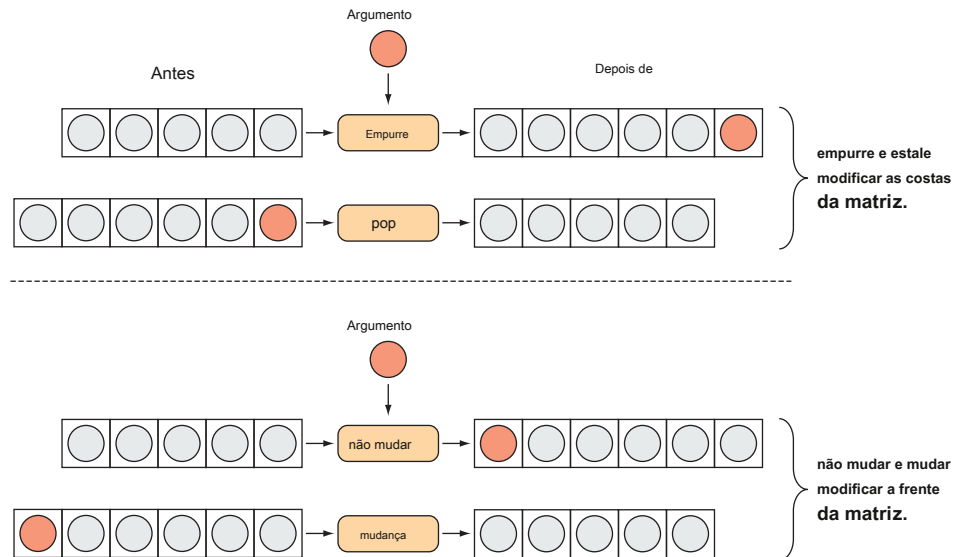


Figura 9.2 O **Empurre** e **pop** métodos modificam o final de uma matriz, enquanto **mudança** e **não mudar** modificar o início do array.

Considerações de desempenho: pop e push versus shift e unshift

O pop e Empurre métodos afetam apenas o último item em uma matriz: pop removendo o último item, e Empurre inserindo um item no final da matriz. Por outro lado, o mudança e não mudar métodos alteram o primeiro item na matriz. Isso significa que os índices de quaisquer itens subsequentes do array devem ser ajustados. Por esta razão, Empurre e pop são operações significativamente mais rápidas do que mudança e sem mudança, e recomendamos usá-los, a menos que você tenha um bom motivo para fazer o contrário.

9.1.3 Adicionar e remover itens em qualquer localização da matriz

O exemplo anterior removeu itens do início e do final da matriz. Mas isso é muito restritivo - em geral, devemos ser capazes de remover itens de qualquer localização de array. Uma abordagem direta para fazer isso é mostrada na lista a seguir.

Listagem 9.3 Maneira ingênua de remover um item de array

```
const ninjas = ["Yagyu", "Kuma", "Hattori", "Fuma"];
```

```
exclua ninjas [1];
```

← Usa o comando delete para deletar um item

```
assert (ninjas.length === 4,
      "Comprimento ainda relata que existem 4 itens");
```

```
assert (ninjas [0] === "Yagyu", "O primeiro item é Yagyu"); assert (ninjas [1] === undefined, "Nós simplesmente criamos um buraco"); assert (ninjas [2] === "Hattori", "Hattori ainda é o terceiro item"); assert (ninjas [3] === "Fuma", "E Fuma é o último item");
```

Excluimos um item, mas a matriz ainda relata que possui 4 itens. Nós apenas criamos um buraco na matriz.

Esta abordagem para excluir um item de uma matriz não funciona. Efetivamente, apenas criamos uma lacuna na matriz. O array ainda informa que tem quatro itens, mas um deles - o que queríamos excluir - é Indefinido (veja a figura 9.3).

```
var ninjas = ["Yagyu", "Kuma", "Hattori", "Fuma"]
```

"Yagyu"	"Kuma"	"Hattori"	"Fuma"
---------	--------	-----------	--------

```
excluir ninjas [1]
```

"Yagyu"	Indefinido	"Hattori"	"Fuma"
---------	------------	-----------	--------

Figura 9.3 Excluir um item de uma matriz cria um buraco na matriz.

Da mesma forma, se quiséssemos inserir um item em uma posição arbitrária, por onde começaríamos? Como uma resposta a esses problemas, todas as matrizes de JavaScript têm acesso ao emenda método: a partir de um determinado índice, este método remove e insere itens. Veja o seguinte exemplo.

Listagem 9.4 Removendo e adicionando itens em posições arbitrárias

```
const ninjas = ["Yagyu", "Kuma", "Hattori", "Fuma"];

var removedItems = ninjas.splice(1, 1);

assert(removedItems.length === 1, "Um item foi removido"); assert(removedItems[0] === "Kuma");

assert(ninjas.length === 3,
  "Existem agora três itens na matriz"); assert(ninjas[0] === "Yagyu",
  "O primeiro item ainda é Yagyu"); assert(ninjas[1] ===
  "Hattori",
  "Hattori está agora em segundo lugar"); afirmar(ninjas[2] === "Fuma",
  "E Fuma está em terceiro lugar");

removedItems = ninjas.splice(1, 2, "Mochizuki", "Yoshi", "Momochi"); assert(removedItems.length === 2, "Agora, removemos dois
itens"); assert(removedItems[0] === "Hattori", "Hattori foi removido"); assert(removedItems[1] === "Fuma", "Fuma foi removido");
assert(ninjas.length === 4, "Inserimos alguns itens novos"); assert(ninjas[0] === "Yagyu", "Yagyu ainda está aqui");

afirmam(ninjas[1] === "Mochizuki", "Mochizuki também"); assert(ninjas[2] === "Yoshi", "Yoshi
também"); afirmam(ninjas[3] === "Momochi", "e Momochi");
```

Cria uma nova matriz com quatro itens

Usa o embutido método de emenda para remove um elemento, começando no índice 1

splice retorna uma matriz dos itens removidos. Neste caso, nós removeu um item.

A matriz ninja não contém mais Kuma; itens subsequentes foram mudou automaticamente.

Podemos inserir um elemento em uma posição adicionando argumentos à chamada de splice.

Começamos criando uma nova matriz com quatro itens:

```
var ninjas = ["Yagyu", "Kuma", "Hattori", "Fuma"];
```

Então chamamos o embutido emenda método:

```
var removedItems = ninjas.splice(1,1); // ninjas: ["Yagyu", "Hattori", "Fuma"];
// removedItems: ["Kuma"]
```

Nesse caso, emenda recebe dois argumentos: o índice a partir do qual a emenda começa e o número de elementos a serem removidos (se omitirmos este argumento, todos os elementos até o final da matriz serão removidos). Neste caso, o elemento com índice 1 é removido da matriz e todos os elementos subsequentes são deslocados de acordo.

Além disso, o método `emenda` retorna uma matriz de itens que foram removidos. Nesse caso, o resultado é uma matriz com um único item: "Kuma".

Usando o método `emenda`, também podemos inserir itens em posições arbitrárias em uma matriz. Por exemplo, considere o seguinte código:

```
removedItems = ninjas.splice(1, 2, "Mochizuki", "Yoshi", "Momochi"); // ninjas: ["Yagyu", "Mochizuki", "Yoshi", "Momochi"]

// removedItems: ["Hattori", "Fuma"]
```

A partir do índice 1, primeiro remove dois itens e depois adiciona três itens: "Mochizuki", "Yoshi", e "Momochi".

Agora que relembramos como funcionam os arrays, vamos continuar estudando algumas operações comuns que costumam ser realizadas em arrays. Isso o ajudará a escrever um código de manipulação de array mais elegante.

9.1.4 Operações comuns em matrizes

Nesta seção, exploraremos algumas das operações mais comuns em matrizes:

- *Iterando* (ou *atravessando*) através de matrizes
- *Mapeamento* itens da matriz existente para criar uma nova matriz com base neles
- *Testando* itens de matriz para verificar se eles satisfazem certas condições
- *Encontrando* itens específicos da matriz
- *Agregando* matrizes e computar um único valor com base em itens de matriz (por exemplo, calcular a soma de uma matriz)

Bem, comece com o básico: iterações de array.

ITERANDO SOBRE ARRAYS

Uma das operações mais comuns é a iteração em uma matriz. Voltando à Ciência da Computação 101, uma iteração é mais frequentemente realizada da seguinte maneira:

```
const ninjas = ["Yagyu", "Kuma", "Hattori"];

para (seja i = 0; i < ninjas.length; i++) {
  afirmar (ninjas[i] !== null, ninjas[i]);
}
```

Informa o valor
de cada ninja

Este exemplo é tão simples quanto parece. Usa um `for` loop para verificar todos os itens na matriz; os resultados são mostrados na figura 9.4.

Você provavelmente já escreveu algo assim tantas vezes que nem precisa pensar mais nisso. Mas, por precaução, vamos dar uma olhada mais de perto no `for` loop.

Para percorrer uma matriz, temos que configurar uma variável de contador, eu, especificar o número até o qual queremos contar (`ninjas.length`), e

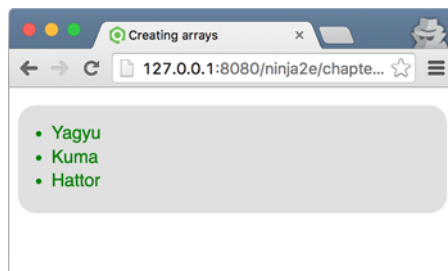


Figura 9.4 A saída da verificação dos ninjas com um `for` loop

definir como o contador será modificado (`i++`). É uma quantidade enorme de contabilidade para realizar uma ação tão comum e pode ser uma fonte de pequenos bugs irritantes. Além disso, torna nosso código mais difícil de ler. Os leitores devem olhar atentamente para cada parte do `for` loop, apenas para ter certeza de que passa por todos os itens e não pula nenhum.

Para tornar a vida mais fácil, todos os arrays de JavaScript têm um built-in para cada método que podemos usar em tais situações. Veja o exemplo a seguir.

Listagem 9.5 Usando o `para cada` método

```
const ninjas = ["Yagyu", "Kuma", "Hattori"];
```

```
ninjas. para cada( ninja => {  
  afirmar (ninja! == nulo, ninja);  
});
```

Usa o embutido
método `forEach` para
iterar sobre a matriz

Fornecemos um retorno de chamada (neste caso, uma função de seta) que é chamado *imediatamente*, para cada item da matriz. É isso - não precisa mais se preocupar com o índice inicial, a condição final ou a natureza exata do incremento. O mecanismo JavaScript cuida de tudo isso para nós, nos bastidores. Observe como esse código é muito mais fácil de entender e como ele tem menos pontos de geração de bug.

Continuaremos elevando as coisas e vendo como podemos mapear arrays para outros arrays.

ARRAYS DE MAPEAMENTO

Imagine que você tem uma série de ninja objetos. Cada ninja tem um nome e uma arma favorita, e você deseja extrair uma variedade de armas do ninjas variedade. Armado com o conhecimento do `para cada` método, você pode escrever algo como a lista a seguir.

Listagem 9.6 Extração ingênua de uma matriz de armas

```
const ninjas = [  
  {nome: "Yagyu", arma: "shuriken"}, {nome: "Yoshi", arma: "katana"},  
  {nome: "Kuma", arma: "wakizashi"}  
];
```

```
armas const = [];  
ninjas.forEach (ninja => {  
  weapons.push (ninja.weapon);  
});
```

Cria uma nova matriz e usa um loop
`forEach` sobre ninjas para extrair armas
ninja individuais

```
afirmar (armas [0] === "shuriken"  
  && armas [1] === "katana"  
  && armas [2] === "wakizashi" && weapons.length ===  
  3,  
  "O novo array contém todas as armas");
```

Isso não é tão ruim: criamos um novo array vazio e usamos o `para` para cada método para iterar sobre o ninjas variedade. Então, para cada ninja objeto, adicionamos a arma atual ao armas variedade.

Como você pode imaginar, criar novos arrays com base nos itens de um array existente é surpreendentemente comum - tão comum que tem um nome especial: *mapeamento* uma matriz. A ideia é mapear cada item de um array para um novo item de um novo array. Convenientemente, o JavaScript tem um mapa função que faz exatamente isso, conforme mostrado na lista a seguir.

Listagem 9.7 Mapeando um array

```
const ninjas = [
  {nome: "Yagyu", arma: "shuriken"}, {nome: "Yoshi", arma: "katana"},
  {nome: "Kuma", arma: "wakizashi"}
];

armas const = ninjas.map(ninja => ninja.weapon);

afirmar (armas[0] === "shuriken"
  && armas[1] === "katana"
  && armas[2] === "wakizashi"
  && weapons.length === 3, "O novo array contém todas as armas");
```

O método de mapa embutido usa uma função que é chamada para cada item da matriz.

O embutido mapa método constrói um array completamente novo e então itera sobre o array de entrada. Para cada item na matriz de entrada, mapa coloca exatamente um item na matriz recém-construída, com base no resultado do retorno de chamada fornecido para mapa. O funcionamento interno do mapa função são mostradas na figura 9.5.

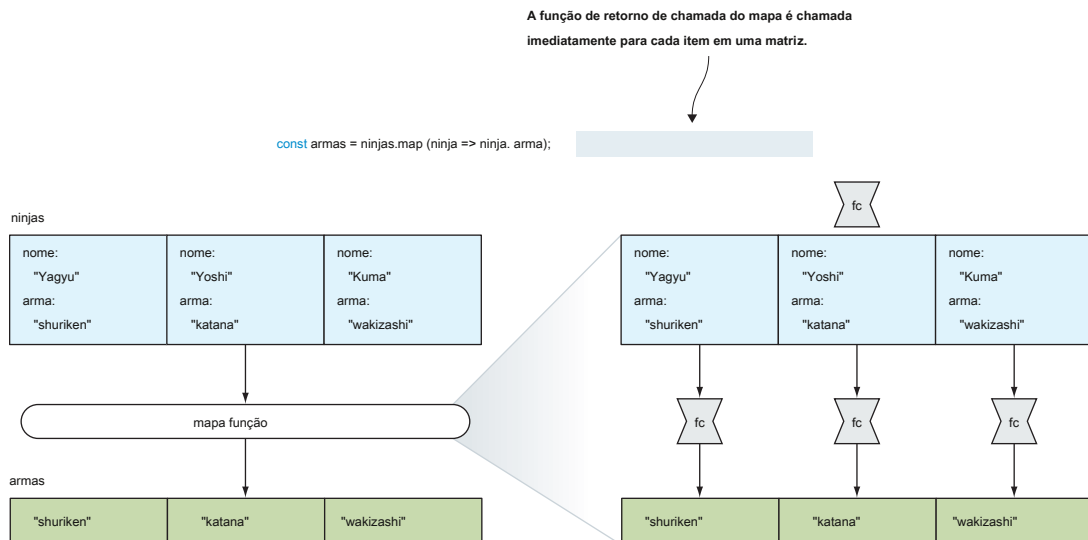


Figura 9.5 O `mapa` function chama a função de retorno de chamada fornecida (fc) em cada item da matriz e cria uma nova matriz com valores de retorno de chamada.

Agora que sabemos como mapear arrays, vamos ver como testar itens de array para certas condições.

ARRAY ITEMS DE TESTE

Ao trabalhar com coleções de itens, muitas vezes nos deparamos com situações em que precisamos saber se *todo* ou pelo menos *alguns* dos itens da matriz satisfazem certas condições. Para escrever este código da forma mais eficiente possível, todos os arrays JavaScript têm acesso ao *cada* e alguns métodos, mostrados a seguir.

Listagem 9.8 Testando matrizes com o *cada* e *alguns* métodos

```
const ninjas = [
  {nome: "Yagyu", arma: "shuriken"}, {nome: "Yoshi"},

  {nome: "Kuma", arma: "wakizashi"}
];
```

O método *every* integrado recebe um retorno de chamada que é chamado para cada item do array. Ele retorna verdadeiro se o retorno de chamada retornar um valor verdadeiro para *toda* variedade de itens, ou falso caso contrário.

```
const allNinjasAreNamed = ninjas.every (ninja => "nome" em ninja); const allNinjasAreArmed = ninjas.every (ninja => "arma" em
ninja);
```

```
assert (allNinjasAreNamed, "Todo ninja tem um nome"); assert (! allNinjasAreArmed, "Mas nem todo ninja está
armado");
```

```
const someNinjasAreArmed = ninjas.some (ninja => "arma" em ninja); assert (someNinjasAreArmed, "Mas alguns ninjas estão
armados");
```

O método embutido também recebe um retorno de chamada. Ele retorna verdadeiro se o retorno de chamada retornar um valor verdadeiro para em pelo menos um item da matriz, ou falso caso contrário.

A Listagem 9.8 mostra um exemplo onde temos uma coleção de ninja objetos, mas não têm certeza de seus nomes e se todos eles estão armados. Para chegar à raiz deste problema, primeiro aproveitamos *cada*:

```
var allNinjasAreNamed = ninjas. cada( ninja => "nome" em ninja);
```

O *cada* O método recebe um retorno de chamada que, para cada ninja da coleção, verifica se sabemos o nome do ninja. *cada* retorna verdade apenas se o retorno de chamada passado retornar verdade pra *cada* item na matriz. A Figura 9.6 mostra como *cada* funciona.

Em outros casos, só nos importamos se *alguns* os itens da matriz satisfazem uma determinada condição. Para essas situações, podemos usar o método integrado *alguns*:

```
const someNinjasAreArmed = ninjas.some (ninja => "arma" em ninja);
```

Começando com o primeiro item da matriz, *alguns* chama o retorno de chamada em cada item da matriz até que um item seja encontrado para o qual o retorno de chamada retorna um verdade valor. Se tal item for encontrado, o valor de retorno é verdade; se não, o valor de retorno é falso.

Cada função de retorno de chamada é chamada imediatamente para cada item em uma matriz, até que `false` seja retornado.

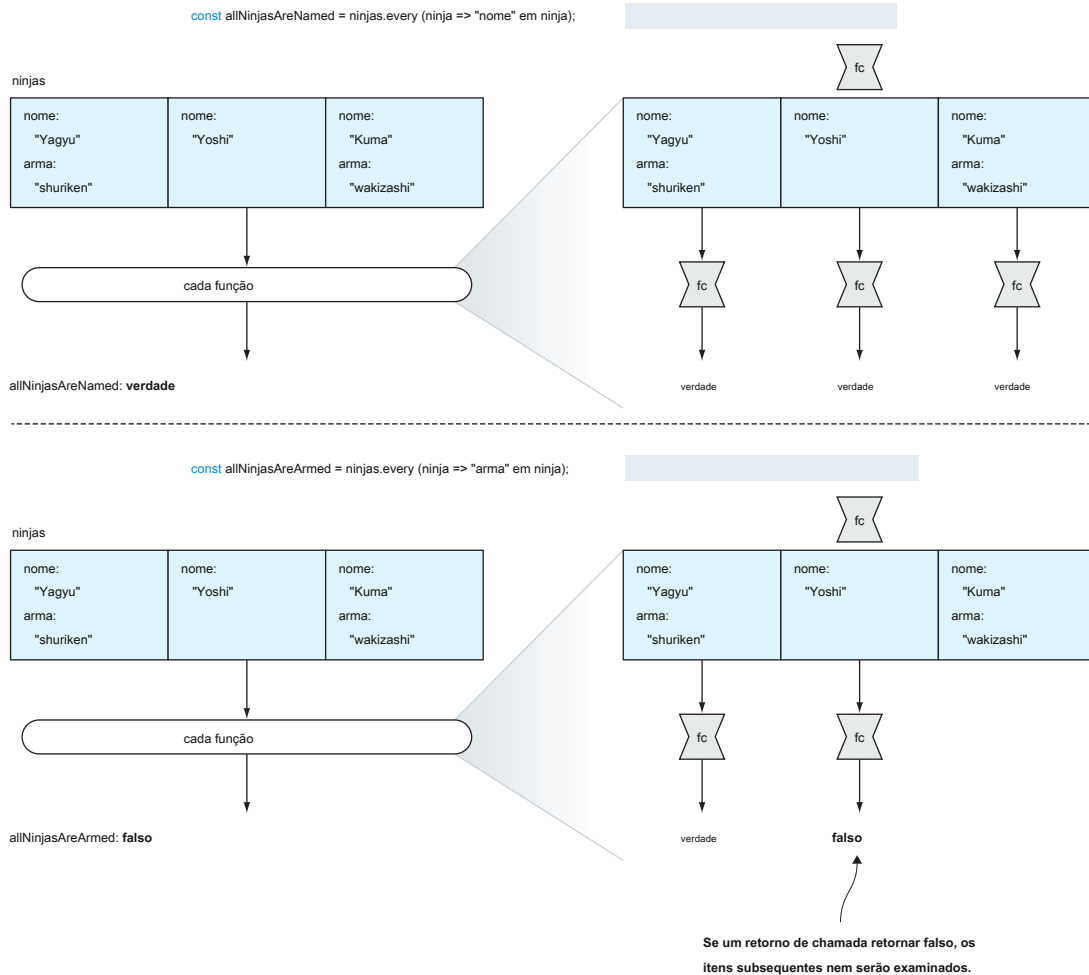


Figura 9.6 O `cada` método testa se todos os itens em uma matriz satisfazem uma determinada condição representada por um retorno de chamada.

A Figura 9.7 mostra como alguns funciona nos bastidores: Pesquisamos uma matriz para descobrir se alguns ou todos os seus itens satisfazem uma determinada condição. A seguir, exploraremos como pesquisar um array para encontrar um item específico.

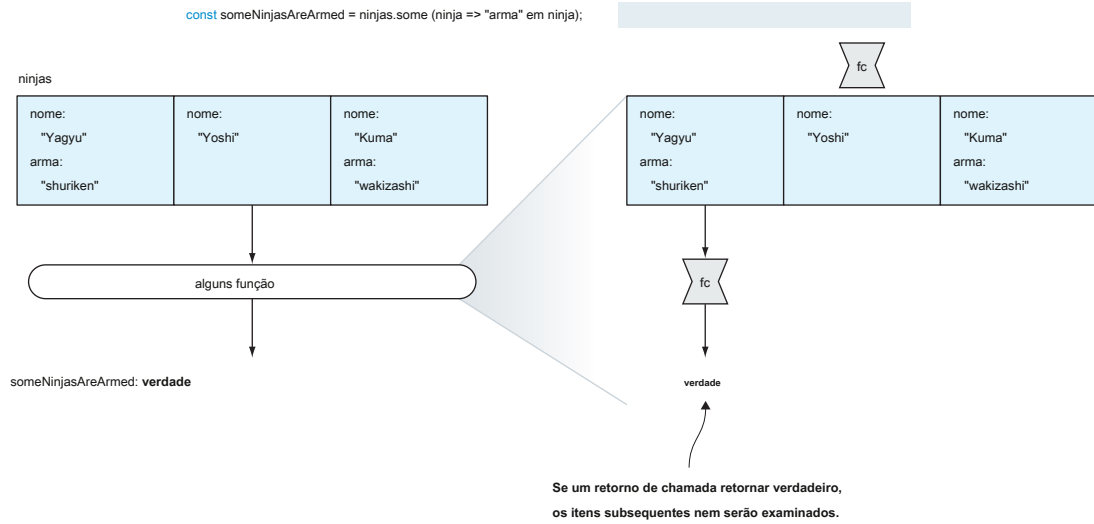


Figura 9.7 O **alguns** O método verifica se pelo menos um item da matriz satisfaz uma condição representada pelo retorno de chamada passado.

PESQUISANDO ARRAYS

Outra operação comum que você deve usar, mais cedo ou mais tarde, é localizar itens em um array. Novamente, essa tarefa é bastante simplificada com outro método de array integrado: **achar**. Vamos estudar a seguinte lista.



NOTA O embutido **achar** método faz parte do padrão ES6. Para compatibilidade do navegador atual, consulte <http://mng.bz/U532>.

Listagem 9.9 Encontrando itens do array

```
const ninjas = [
  {nome: "Yagyu", arma: "shuriken"}, {nome: "Yoshi"},
  {nome: "Kuma", arma: "wakizashi"}
];
```

```
const ninjaWithWakizashi = ninjas.find (ninja => {
  return ninja.weapon === "wakizashi"; });
```

```
assert (ninjaWithWakizashi.name === "Kuma"
  && ninjaWithWakizashi.weapon === "wakizashi", "Kuma está empunhando um
  wakizashi");
```

Usa o método **find** para encontrar o primeiro item da matriz que satisfaça uma certa condição, representado por um retorno de chamada passado.

```
const ninjaWithKatana = ninjas.find (nina => {
  return ninja.weapon === "katana"; });
```

```
assert (ninjaWithKatana === undefined,
  "Não conseguimos encontrar um ninja que empunhava uma katana");
```

```
const armadoNinjas = ninjas.filter (ninja => "arma" em ninja);
```

```
assert (armadoNinjas.length === 2, "Existem dois ninjas armados."); assert (armadoNinjas [0] .name === "Yagyu"
```

```
&& armadoNinjas [1] .nome === "Kuma", "Yagyu e Kuma");
```

← O método `find` retorna `undefined` se um item não puder ser encontrado.

Use o método de filtro para encontrar vários itens que satisfaçam uma determinada condição.

É fácil encontrar um item de matriz que satisfaça uma determinada condição: Usamos o `achar`, passando a ele um retorno de chamada que é invocado para cada item na coleção até que o item de destino seja encontrado. Isso é indicado pelo retorno de chamada verdade. Por exemplo, a expressão

```
ninjas.find (ninja => ninja.weapon === "wakizashi");
```

encontra Kuma, o primeiro ninja do `ninjas` matriz que está empunhando um `wakizashi`.

Se nós examinamos toda a matriz sem um único item retornando verdade, o resultado final da pesquisa é Indefinido. Por exemplo, o código

```
ninjaWithKatana = ninjas.find (ninja => ninja.weapon === "katana");
```

retorna Indefinido, porque não há um ninja empunhando uma katana. A Figura 9.8 mostra o funcionamento interno do `achar` função.

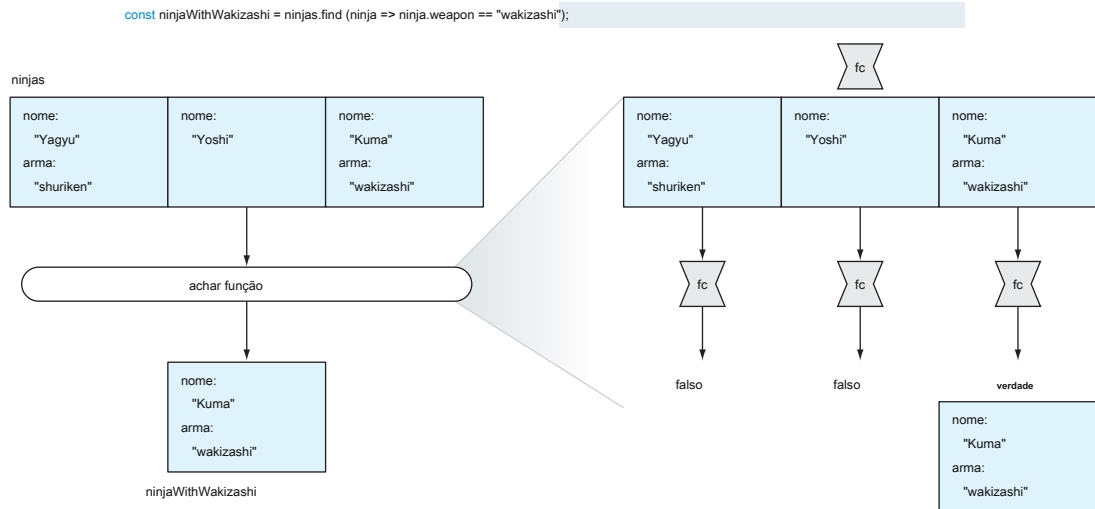


Figura 9.8 O `achar` função encontra um item em uma matriz: o primeiro item para o qual o `achar` retorno de chamada **verdade**.

Se precisarmos encontrar vários itens que satisfaçam um determinado critério, podemos usar o filtro método, que cria um *novo* array contendo todos os itens que atendem a esse critério. Por exemplo, a expressão

```
const armadoNinjas = ninjas.filter(ninja => "arma" em ninja);
```

cria um novo Armado Ninjas array que contém apenas ninjas com uma arma. Nesse caso, o pobre Yoshi desarmado fica de fora. A Figura 9.9 mostra como o filtro função funciona.

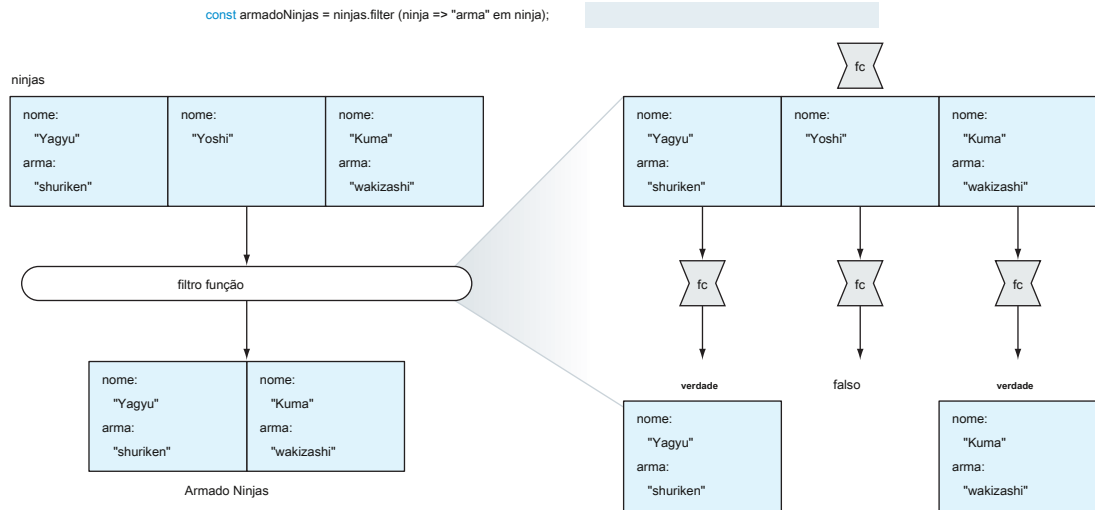


Figura 9.9 O **filtro** função cria uma nova matriz que contém todos os itens para os quais o retorno de chamada retorna **verdade**.

Ao longo deste exemplo, você viu como encontrar itens específicos em uma matriz, mas em muitos casos também pode ser necessário encontrar o índice de um item. Vamos dar uma olhada mais de perto, com o exemplo a seguir.

Listagem 9.10 Encontrando índices de array

```
const ninjas = ["Yagyu", "Yoshi", "Kuma", "Yoshi"];
```

```
assert(ninjas.indexOf("Yoshi") === 1, "Yoshi está no índice 1"); assert(ninjas.lastIndexOf("Yoshi") === 3, "e no índice 3");
```

```
const yoshiIndex = ninjas.findIndex(ninja => ninja === "Yoshi");
```

```
assert(yoshiIndex === 1, "Yoshi ainda está no índice 1");
```

Para encontrar o índice de um item específico, usamos a `índice de método`, passando para ele o item cujo índice queremos encontrar:

```
ninjas.indexOf("Yoshi")
```

Nos casos em que um determinado item pode ser encontrado várias vezes em uma matriz (como é o caso de "Yoshi" e a ninjas matriz), também podemos estar interessados em encontrar o último índice em que Yoshi aparece. Para isso, podemos usar o `lastIndexOf` método:

```
ninjas.lastIndexOf("Yoshi")
```

Finalmente, no caso mais geral, quando não temos uma referência ao item exato cujo índice queremos pesquisar, podemos usar o `findIndex` método:

```
const yoshiIndex = ninjas.findIndex(ninja => ninja === "Yoshi");
```

O `findIndex` método recebe um retorno de chamada e retorna o índice do primeiro item para o qual o retorno de chamada retorna verdade. Em essência, funciona muito como o `achar` método, a única diferença é que `achar` retorna um item específico, enquanto `findIndex` retorna o índice desse item.

SORTING ARRAYS

Uma das operações de array mais comuns é *Ordenação*- organizar os itens sistematicamente em alguma ordem. Infelizmente, implementar algoritmos de classificação corretamente não é a mais fácil das tarefas de programação: temos que selecionar o melhor algoritmo de classificação para a tarefa, implementá-lo e adaptá-lo às nossas necessidades, enquanto, como sempre, tomamos cuidado para não introduzir bugs sutis. Para tirar esse fardo de nossas costas, como você viu no capítulo 3, todos os arrays de JavaScript têm acesso ao `organizar` método, cujo uso é semelhante a este:

```
array.sort((a, b) => a - b);
```

O mecanismo JavaScript implementa o algoritmo de classificação. A única coisa que temos a fornecer é um retorno de chamada que informa o algoritmo de classificação sobre a relação entre dois itens do array. Os resultados possíveis são os seguintes:

- Se um retorno de chamada retornar um valor menor que 0, então item uma deve vir antes do item b.
- Se um retorno de chamada retornar um valor igual a 0, então itens uma e b estão em pé de igualdade (no que diz respeito ao algoritmo de classificação, eles são iguais).
- Se um retorno de chamada retornar um valor maior que 0, então item uma deve vir depois do item b.

A Figura 9.10 mostra as decisões tomadas pelo algoritmo de classificação dependendo do valor de retorno de chamada.

	...uma...b	...b...uma
returnValue < 0 (uma deveria vir antes b)	Deixe como está	uma deve ser movido antes b
returnValue == 0 (uma e b estão em pé de igualdade)	Deixe como está	Deixe como está
returnValue > 0 (b deveria vir antes a)	b deve ser movido antes uma	Deixe como está

Figura 9.10 Se a chamada `back` retorna um valor menor que 0, o primeiro item deve vir antes do segundo. Se o retorno de chamada retornar 0, Ambas os itens devem ser deixados como estão. E se o valor de retorno for maior que 0, o primeiro item deve vir após o segundo item.

E isso é tudo que você precisa saber sobre o algoritmo de classificação. A classificação real é realizada nos bastidores, sem a necessidade de mover manualmente os itens do array. Vejamos um exemplo simples.

Listagem 9.11 Classificando um array

```
const ninjas = [{nome: "Yoshi"}, {nome: "Yagyu"}, {nome: "Kuma"}];

ninja.sort (função (ninja1, ninja2) {
  if (ninja1.nome <ninja2.nome) {retorno -1; } if (ninja1.nome> ninja2.nome) {return 1; }

  return 0;
});

assert (ninjas [0] .name === "Kuma", "Kuma é o primeiro"); assert (ninjas [1] .name === "Yagyu", "Yagyu
é o segundo"); assert (ninjas [2] .name === "Yoshi", "Yoshi é o terceiro");
```

Passa um retorno de chamada para o método de classificação integrado para especificar uma ordem de classificação

Na listagem 9.11, temos uma matriz de ninja objetos, onde cada ninja tem um nome. Nosso objetivo é classificar esse array lexicograficamente (em ordem alfabética), de acordo com os nomes dos ninjas. Para isso, naturalmente usamos o organizar função:

```
ninjas.sort (função (ninja1, ninja2) {
  if (ninja1.nome <ninja2.nome) {retorno -1; } if (ninja1.nome> ninja2.nome) {return 1; }

  return 0;
});
```

Ao organizar só precisamos passar um retorno de chamada usado para comparar dois itens do array. Porque queremos fazer uma comparação lexical, afirmamos que se ninja1 o nome de é "menor" que ninja2 nome de, o retorno de chamada retorna - 1 (lembre-se, isso significa ninja1 deveria vir antes ninja2, na ordem final de classificação); se for maior, o retorno de chamada retorna 1

(ninja1 deveria vir depois ninja2); se forem iguais, o retorno de chamada retorna 0 Observe que podemos usar operadores simples menor que (<) e maior que (>) para comparar dois nomes de ninja.

É sobre isso! O resto dos detalhes essenciais da classificação são deixados para o mecanismo JavaScript, sem que tenhamos que nos preocupar com eles.

AGREGANDO ARRAY ITMS

Quantas vezes você escreveu um código como o seguinte?

```
números const = [1, 2, 3, 4]; soma const = 0;

number.forEach (number => {
  soma + = número;
});

assert (sum === 10, "A soma dos quatro primeiros números é 10");
```

Esse código deve visitar cada item de uma coleção e agregar algum valor, basicamente reduzindo todo o array a um único valor. Não se preocupe - o JavaScript também tem algo para ajudar nessa situação: o `reduzir` método, conforme mostrado no exemplo a seguir.

Listagem 9.12 Agregando itens com `reduzir`

```
números const = [1, 2, 3, 4];
```

```
const sum = number.reduce ((agregado, número) =>
    agregado + número, 0);
```

Usa `reduzir` para acumular um único valor de uma matriz

```
assert (sum === 10, "A soma dos quatro primeiros números é 10");
```

O `reduzir` método funciona pegando o valor inicial (neste caso, 0) e chamar a função de retorno de chamada em cada item da matriz com o resultado da chamada de retorno anterior (ou o valor inicial) e o item da matriz atual como argumentos. O resultado do `reduzir`

invocação é o resultado do último retorno de chamada, chamado no último item do array. A Figura 9.11 lança mais luz sobre o processo.

Esperamos ter convencido você de que o JavaScript contém alguns métodos de array úteis que podem tornar nossas vidas significativamente mais fáceis e seu código mais elegante, sem ter que recorrer a métodos incômodos pra rotações. Se você quiser saber mais sobre esses e outros métodos de array, recomendamos a explicação da rede do desenvolvedor Mozilla em <http://mng.bz/CS21>. Agora, levaremos as coisas um pouco mais adiante e mostraremos como reutilizar esses métodos de array em seus próprios objetos personalizados.

9.1.5 Reutilizar funções de matriz integradas

Às vezes, podemos querer criar um objeto que contenha uma coleção de dados. Se a coleção fosse tudo que nos preocupasse, poderíamos usar um array. Mas, em certos casos, pode haver mais estado para armazenar do que apenas a coleção em si - talvez precisemos armazenar algum tipo de metadados sobre os itens coletados.

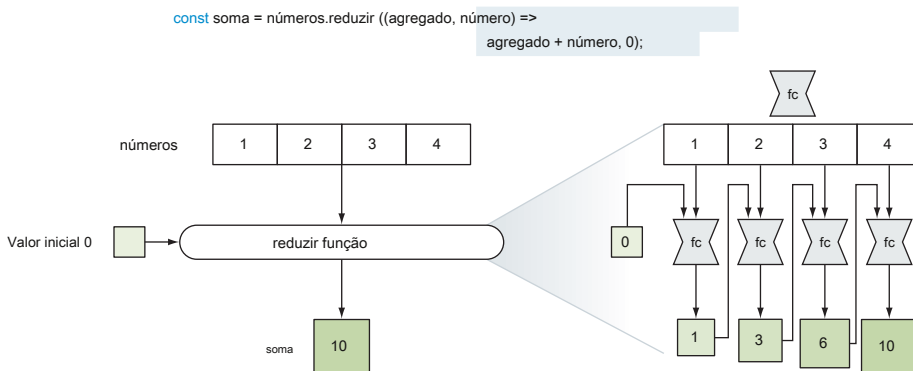


Figura 9.11 O `reduzir` A função aplica um retorno de chamada a um valor agregado e a cada item em uma matriz para reduzir a matriz a um único valor.

Uma opção pode ser criar um novo array sempre que desejar criar uma nova versão de tal objeto e adicionar as propriedades e métodos de metadados a ele. Lembre-se de que podemos adicionar propriedades e métodos a um objeto como desejarmos, incluindo matrizes. Geralmente, entretanto, isso pode ser lento, para não mencionar tedioso.

Vamos examinar a possibilidade de usar um objeto normal e *dando* é a funcionalidade que desejamos. Métodos que sabem como lidar com coleções já existem no Variedade objeto; podemos enganá-los para trabalhar em nossos próprios objetos? Acontece que podemos, conforme mostrado na lista a seguir.

Listagem 9.13 Simulando métodos semelhantes a array

```
<body>
<input id = "first" />
<input id = "second" />
<script>
  const elems = {
    comprimento: 0,
    add: function (elem) {
      Array.prototype.push.call (this, elem);
    },
    reunir: função (id) {
      this.add (document.getElementById (id));
    },
    find: function (callback) {
      return Array.prototype.find.call (this, callback);
    }
  };

  elems.gather ("primeiro");
  assert (elems.length === 1 && elems [0] .nodeType,
    "Verifique se temos um elemento em nosso estoque");

  elems.gather ("segundo");
  assert (elems.length === 2 && elems [1] .nodeType,
    "Verifique a outra inserção");

  elems.find (elem => elem.id === "segundo"); assert (found && found.id ===
    "segundo",
    "Encontramos nosso elemento");
</script>
</body>
```

Armazena a contagem de elementos. A matriz precisa de um local para armazenar o número de itens que está armazenando.

Implementa o método para adicionar elementos a uma coleção. O protótipo do Array tem um método para fazer isso, então por que não usá-lo em vez de reinventar a roda?

Implementa o método de coleta para encontrar elementos por seus valores de id e adicioná-los à coleção

Implementa o método para encontrar elementos na coleção. Semelhante ao método add, ele reutiliza o método find existente acessível a matrizes.

Neste exemplo, criamos um objeto "normal" e o instrumentamos para imitar alguns dos comportamentos de um array. Primeiro, definimos um comprimento propriedade para registrar o número de elementos que são armazenados, como um array. Em seguida, definimos um método para adicionar um elemento ao final do array simulado, chamando este método adicionar:

```
add: function (elem) {
  Array.prototype.push.call (this, elem);
}
```


Em vez de escrever nosso próprio código, podemos usar um método nativo de matrizes JavaScript:

```
Array.prototype.push.
```

Normalmente, o `Array.prototype.push` método operaria em seu próprio array por meio de seu contexto de função. Mas aqui, estamos enganando o método a usar *nosso* objeto como seu contexto usando o `chamar` método (lembre-se do capítulo 4) e forçando nosso objeto a ser o contexto do `Empurre` método. (Observe como poderíamos ter usado facilmente o `Aplique` método.) `Empurre` método, que incrementa o comprimento propriedade (pensando que é o comprimento propriedade de uma matriz), adiciona uma propriedade numerada ao objeto que faz referência ao elemento passado. De certa forma, esse comportamento é quase subversivo (como apropriado para ninjas!), Mas exemplifica o que podemos fazer com contextos de objetos mutáveis.

O `adicionar` método espera que uma referência de elemento seja passada para armazenamento. Embora às vezes possamos ter essa referência por perto, na maioria das vezes não o fazemos, então também definimos um método de conveniência, `reunir`, que procura o elemento por seu `id` e o adiciona ao armazenamento:

```
reunir: função (id) {
    this.add (document.getElementById (id));
}
```

Finalmente, também definimos um `achar` método que nos permite encontrar um item arbitrário em nosso objeto personalizado, tirando proveito do array integrado `achar` método:

```
find: function (callback) {
    return Array.prototype.find.call (this, callback);
}
```

O comportamento nefasto limítrofe que demonstramos nesta seção não apenas revela o poder que os contextos de função maleável nos dão, mas também mostra como podemos ser inteligentes em reutilizar código já escrito, em vez de reinventar constantemente a roda.

Agora que já passamos algum tempo com matrizes, vamos passar para dois novos tipos de coleções introduzidos pelo ES6: mapas e conjuntos.

9,2 Mapas

Imagine que você é um desenvolvedor do `freelanceninja.com`, um site que deseja atender a um público mais internacional. Para cada texto no site - por exemplo, "Ninjas de aluguel" - você gostaria de criar um mapeamento para cada idioma de destino, como "レンタル用の忍者" em japonês, "忍者 出租" "Em chinês, ou" 고용 남자 "Em coreano (esperemos que o Google Translate tenha feito um trabalho adequado). Essas coleções, que mapeiam uma chave para um valor específico, são chamadas por nomes diferentes em linguagens de programação diferentes, mas na maioria das vezes são conhecidas como *dicionários* ou *mapas*.

Mas como você gerencia com eficiência essa localização em JavaScript? Uma abordagem tradicional é aproveitar as vantagens do fato de que os objetos são coleções de propriedades e valores nomeados e criar algo como o seguinte dicionário:

```
dicionário const = {
  "ja": {
    "Ninjas para alugar": " レンタル用の忍者 "
  },
  "zh": {
    "Ninjas para alugar": " 忍者 出租 "
  },
  "ko": {
    "Ninjas para alugar": " 고용 닌자 "
  }
}
assert(dictionary.ja["Ninjas para alugar"] === " レンタル用の忍者 ");
```

À primeira vista, isso pode parecer uma abordagem perfeitamente adequada para esse problema e, para este exemplo, não é nada ruim. Mas, infelizmente, em geral, você não pode confiar nisso.

9.2.1 Não use objetos como mapas

Imagine que em algum lugar do nosso site precisamos acessar a tradução da palavra *construtor*, portanto, estendemos o exemplo do dicionário para o código a seguir.

Listagem 9.14 Objetos têm acesso a propriedades que não foram definidas explicitamente

```
dicionário const = {
  "ja": {
    "Ninjas para alugar": " レンタル用の忍者 "
  },
  "zh": {
    "Ninjas para alugar": " 忍者 出租 "
  },
  "ko": {
    "Ninjas para alugar": " 고용 닌자 "
  }
};

assert(dictionary.ja["Ninjas para alugar"] === " レンタル用の忍者 ",
  "Nós sabemos dizer 'Ninjas de aluguel' em japonês!");

assert (typeof dictionary.ja["constructor"] === "undefined",
  dicionário.ja["constructor"]);
```

Tentamos acessar a tradução da palavra *construtor* - uma palavra que esquecemos totalmente de definir em nosso dicionário. Normalmente, nesse caso, esperamos que o dicionário retorne Indefinido. Mas esse não é o resultado, como você pode ver na figura 9.12.

Como você pode ver, acessando o construtor propriedade, obtemos a seguinte string:

```
"function Object () {[código nativo]}"
```

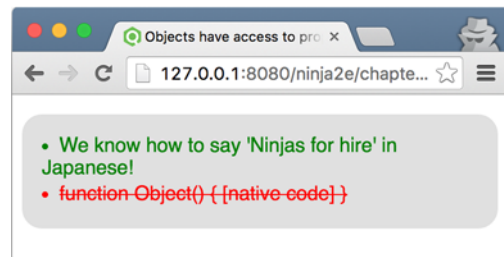


Figura 9.12 Executar a listagem 9.14 mostra que os objetos não são bons mapas, porque eles têm acesso a propriedades que não foram definidas explicitamente (por meio de seus protótipos).

O que há com isso? Como você aprendeu no capítulo 7, todos os objetos têm protótipos; mesmo se definirmos objetos novos e vazios como nossos mapas, eles ainda terão acesso às propriedades dos objetos de protótipo. Uma dessas propriedades é construtor (lembre-se disso construtor é a propriedade do objeto de protótipo que aponta de volta para a função do construtor), e é o culpado por trás da bagunça que agora temos em nossas mãos.

Além disso, com objetos, as chaves podem ser apenas valores de string; se você quiser criar um mapeamento para qualquer outro valor, esse valor será convertido silenciosamente em uma string sem que ninguém pergunte nada! Por exemplo, imagine que queremos rastrear algumas informações sobre nós HTML, como na lista a seguir.

Listagem 9.15 Mapeando valores para nós HTML com objetos



Na listagem 9.15, criamos dois elementos HTML, `firstElement` e `secondElement`, que então buscamos do DOM usando o `document.getElementById` método. Para criar um mapeamento que armazenará informações adicionais sobre cada elemento, definimos um objeto JavaScript simples e antigo:

```
mapa const = {};
```

Em seguida, usamos o elemento HTML como uma chave para nosso objeto de mapeamento e associamos alguns dados a ele:

```
mapa [firstElement] = {data: "firstElement"}
```

E verificamos se podemos recuperar esses dados. Como isso funciona como deveria, repetimos todo o processo para o segundo elemento:

```
mapa [secondElement] = {data: "secondElement"};
```

Novamente, tudo parece ótimo; associamos com sucesso alguns dados ao nosso elemento HTML. Mas ocorre um problema se decidirmos revisitar o primeiro elemento:

```
map [firstElement] .data
```

Seria normal supor que obteríamos novamente as informações sobre o primeiro elemento, mas não é o caso. Em vez disso, como mostra a figura 9.13, as informações sobre o segundo elemento são retornadas.

Isso acontece porque, com objetos, as chaves são armazenadas como strings. Isso significa que quando tentamos usar qualquer valor não string, como um elemento HTML, como uma propriedade de um objeto, esse valor é silenciosamente convertido em uma string chamando seu

método. Aqui, isso retorna a string "[object HTMLDivElement]", e a

informações sobre o primeiro elemento são armazenadas como o valor do [objeto HTMLDivElement] propriedade.

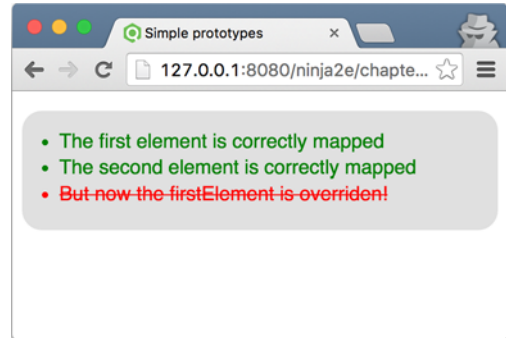


Figura 9.13 Executar o código da listagem 9.15 mostra que os objetos são convertidos em strings se tentarmos usá-los como propriedades do objeto.

Em seguida, quando tentamos criar um mapeamento para o segundo elemento, algo semelhante acontece. O segundo elemento, que também é um elemento HTML div, também é convertido em uma string e seus dados adicionais também são atribuídos ao [objeto HTMLDivElement] , substituindo o valor que definimos para o primeiro elemento.

Por esses dois motivos - propriedades herdadas por meio de protótipos e suporte para chaves somente de string - objetos simples geralmente não são úteis como mapas. Devido a esta limitação, o comitê ECMAScript especificou um tipo completamente novo: Mapa.



NOTA Os mapas são parte do padrão ES6. Para compatibilidade do navegador atual, consulte: <http://mng.bz/JYYM> .

9.2.2 Criando nosso primeiro mapa

Criar mapas é fácil: usamos um novo e integrado Mapa construtor. Veja o exemplo a seguir.

Listagem 9.16 Criando nosso primeiro mapa

```

const ninjalslandMap = new Map ();

const ninja1 = {nome: "Yoshi"}; const ninja2 = {nome: "Hattori"};
const ninja3 = {nome: "Kuma"};

ninjalslandMap.set (ninja1, {homelsland: "Honshu"}); ninjalslandMap.set (ninja2, {homelsland:
"Hokkaido"});

assert (ninjalslandMap.get (ninja1) .homelsland === "Honshu",
        "O primeiro mapeamento funciona");
assert (ninjalslandMap.get (ninja2) .homelsland === "Hokkaido",
        "O segundo mapeamento funciona");

assert (ninjalslandMap.get (ninja3) === undefined,
        "Não há mapeamento para o terceiro ninja");

assert (ninjalslandMap.size === 2,
        "Criamos dois mapeamentos");

afirmar (ninjalslandMap.has (ninja1)
        && ninjalslandMap.has (ninja2),
        "Temos mapeamentos para os dois primeiros ninjas"); assert (! ninjalslandMap.has
(ninja3),
        "Mas não para o terceiro ninja");

ninjalslandMap.delete (ninja1);
assert (! ninjalslandMap.has (ninja1)
        && ninjalslandMap.size () === 1,
        "Não há mais mapeamento do primeiro ninja");

ninjalslandMap.clear ();
assert (ninjalslandMap.size === 0,
        "Todos os mapeamentos foram limpos");

```

Usa o construtor de mapa para criar um mapa

Define três objetos ninja

Cria um mapeamento para os dois primeiros objetos ninja usando o método map set

Obtém o mapeamento para os primeiros dois objetos ninja por usando o método map get

Verifica se não há mapeamento para o terceiro ninja

Verifica se o mapa contém mapeamentos para os dois primeiros ninjas, mas não para o terceiro!

Usa o método has para verificar se existe um mapeamento para uma determinada chave

Usa o método delete para deletar uma chave do mapa

Usa o método clear para limpar completamente o mapa

Neste exemplo, criamos um novo mapa chamando o Mapa construtor:

```
const ninjalslandMap = new Map ();
```

Em seguida, criamos três objetos ninja, habilmente chamados `ninja1`, `ninja2`, e `ninja3`. Em seguida, usamos o mapa integrado definir método:

```
ninjalslandMap.set (ninja1, {homelsland: "Honshu"});
```

Isso cria um mapeamento entre uma chave - neste caso, o `ninja1` objeto - e um valor - neste caso, um objeto que carrega as informações sobre a ilha natal do ninja. Fazemos isso para os dois primeiros ninjas, `ninja1` e `ninja2`.

Na próxima etapa, obtemos o mapeamento para os dois primeiros ninjas usando outro método de mapa integrado, obter:

```
assert (ninjalMap.get(ninja1).homeland == "Honshu",
        "O primeiro mapeamento funciona");
```

O mapeamento, é claro, existe para os dois primeiros ninjas, mas não existe para o terceiro ninja, porque não usamos o terceiro ninja como um argumento para o definir método. O estado atual do mapa é mostrado na figura 9.14.

Além de obter e definir métodos, cada mapa também tem um tamanho propriedade e tem e excluir métodos. O tamanho propriedade nos diz quantos mapeamentos criamos. Nesse caso, criamos apenas dois mapeamentos.

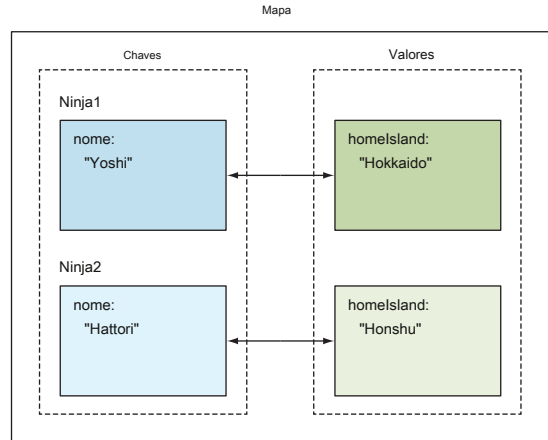


Figura 9.14 Um mapa é uma coleção de pares de valores-chave, onde uma chave pode ser qualquer coisa - até mesmo outro objeto.

O tem método, por outro lado, nos notifica se um mapeamento para uma determinada chave já existe:

```
ninjalMap.has(ninja1); //verdade
ninjalMap.has(ninja3); //falso
```

O excluir método nos permite remover itens de nosso mapa:

```
ninjalMap.delete(ninja1);
```

Um dos conceitos fundamentais ao lidar com mapas é determinar quando duas chaves de mapa são iguais. Vamos explorar esse conceito.

IGUALDADE CHAVE

Se você vem de uma formação um pouco mais tradicional, como C #, Java ou Python, pode se surpreender com o próximo exemplo.

Listagem 9.17 Igualdade de chave em mapas

```
mapa const = novo Mapa ();
const currentLocation = location.href;
```

```
const firstLink = novo URL (currentLocation); const secondLink = novo URL
(currentLocation);
```

```
map.set (firstLink, {descrição: "firstLink"}); map.set (secondLink, {descrição: "secondLink"});
```

Usa a propriedade interna location.href para obter o URL da página atual

Cria dois links para a página atual

Adiciona um mapeamento para ambos os links

```
assert (map.get (firstLink) .description === "firstLink",
        "Mapeamento do primeiro link");
assert (map.get (secondLink) .description === "secondLink",
        "Segundo mapeamento de link");
assert (map.size === 2, "Existem dois mapeamentos");
```

Cada link obtém seu próprio mapeamento, embora eles apontam para a mesma página.

Na Listagem 9.17, usamos o integrado `location.href` propriedade para obter o URL da página atual. Em seguida, usando o construtor de URL integrado, criamos dois novos objetos de URL que vinculam à página atual. Em seguida, associamos um objeto de descrição a cada link. Por fim, verificamos se os mapeamentos corretos foram criados, conforme mostrado na figura 9.15.

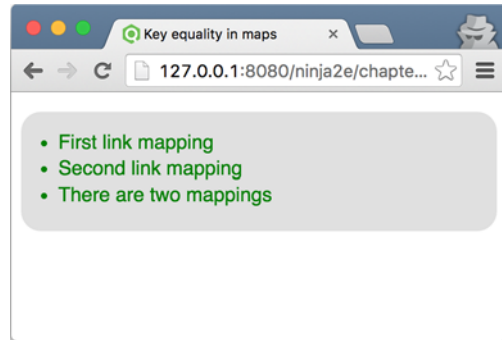


Figura 9.15 Se executarmos o código da Listagem 9.17, podemos ver que a igualdade de chave nos mapas é baseada na igualdade do objeto.

Pessoas que trabalharam principalmente em JavaScript podem não achar este resultado inesperado: Temos dois objetos diferentes para os quais criamos dois

mapeamentos ent. Mas observe que os dois objetos URL, mesmo que sejam objetos separados, ainda apontam para o mesmo local do URL: o local da página atual. Poderíamos argumentar que, ao criar mapeamentos, esses dois objetos devem ser considerados iguais. Mas em JavaScript, não podemos sobrecarregar o operador de igualdade, e os dois objetos, embora tenham o mesmo conteúdo, são sempre considerados diferentes. Este não é o caso com outras linguagens, como Java e C #, então tome cuidado!

9.2.3 Iterando em mapas

Até agora, você viu algumas das vantagens dos mapas: Você pode ter certeza de que eles contêm apenas os itens que você colocou neles e pode usar qualquer coisa como chave. Mas tem mais!

Como os mapas são coleções, não há nada que nos impeça de iterar sobre eles com `for` de rotações. (Lembre-se, usamos o `for` de loop para iterar sobre os valores criados pelos geradores no capítulo 6.) Você também tem a garantia de que esses valores serão visitados na ordem em que foram inseridos (algo em que não podemos confiar ao iterar objetos usando o `for ... in` ciclo). Vejamos o seguinte exemplo.

Listagem 9.18 Iterando sobre mapas

```
diretório const = novo Map ();
```

```
directory.set ("Yoshi", "+81 26 6462"); directory.set ("Kuma", "+81 52 2378 6462");
directory.set ("Hiro", "+81 76 277 46");
```

← Cria um novo mapa, assim como fizemos até agora

Cria um diretório ninja que armazena o número de telefone de cada ninja

```
para (deixar item do diretório) {
    assert (item [0]! == null, "Chave:" + item [0]); assert (item [1]! == null, "Value:" + item
    [1]);
}
```

Itera sobre cada item em um dicionário usando o loop for ... of. Cada item é uma matriz de dois itens: uma chave e um valor.

```
para (deixe a chave do diretório.keys ()) {
    assert (key! == null, "Key:" + key); assert (directory.get (key)! = null,

    "Valor:" + diretório.get (chave));
}
```

Também podemos iterar sobre as chaves usando o método de chaves integradas ...

```
para (valor var de diretório.values ()) {
    assert (valor! == null, "Valor:" + valor);
}
```

... E sobre os valores usando o método de valores embutidos.

Como mostra a lista anterior, depois de criar um mapeamento, podemos facilmente iterar sobre ele usando o para de ciclo:

```
para (var item do diretório) {
    assert (item [0]! == null, "Chave:" + item [0]); assert (item [1]! == null, "Value:" + item
    [1]);
}
```

Em cada iteração, isso fornece uma matriz de dois itens, onde o primeiro item é uma chave e o segundo item é o valor de um item de nosso mapa de diretório. Também podemos usar o `chaves` e `valores` métodos para iterar sobre, bem, chaves e valores contidos em um mapa.

Agora que vimos os mapas, vamos visitar outro novato em JavaScript: *conjuntos*, que são coleções de itens exclusivos.

9,3

Jogos

Em muitos problemas do mundo real, temos que lidar com coleções de *distinto* itens (o que significa que cada item não pode aparecer mais de uma vez) chamados *conjuntos*. Até o ES6, isso era algo que você mesmo precisava implementar imitando conjuntos com objetos padrão. Para um exemplo bruto, consulte a próxima lista.

Listagem 9.19 Conjuntos de imitação com objetos

```
function Set () {
    this.data = {};
    this.length = 0;
}
```

Usa um objeto para armazenar itens

```
Set.prototype.has = function (item) {
    return typeof this.data [item]! == "Indefinido";
};
```

Verifica se o item já está armazenado

```
Set.prototype.add = function (item) {
    if (! this.has (item)) {
        this.data [item] = verdadeiro;
        this.length ++;
    }
};
```

Adiciona um item apenas se ainda não estiver contido no conjunto


```
Set.prototype.remove = function (item) {
  if (this.has (item)) {
    delete this.data [item];
    this.length--;
  }
};
```

**Remove um item se já
estiver contido
no set**

```
const ninjas = new Set (); ninjas.add ("Hattori");
```

```
ninjas.add ("Hattori");
```

**Tenta adicionar
Hattori duas vezes**

```
assert (ninjas.has ("Hattori") && ninjas.length === 1,
  "Nosso conjunto contém apenas um Hattori");
```

**Verifica que Hattori
foi adicionado apenas uma vez**

```
ninjas.remove ("Hattori");
assert (! ninjas.has ("Hattori") && ninjas.length === 0,
  "Nosso conjunto agora está vazio");
```

**Remove Hattori e
verifica se ele foi removido
do conjunto**

A Listagem 9.19 mostra um exemplo simples de como os conjuntos podem ser imitados com objetos. Usamos um objeto de armazenamento de dados, dados, para acompanhar nossos itens definidos, e expomos três métodos: tem, que verifica se um item já está contido no conjunto; adicionar, que adiciona um item apenas se o mesmo item ainda não estiver contido no conjunto; e retirar, que remove um item já contido do conjunto.

Mas este é um pobre doppelganger. Porque com mapas, você não pode realmente armazenar objetos - apenas strings e números - e sempre há o risco de acessar objetos de protótipo. Por essas razões, o comitê ECMAScript decidiu introduzir um tipo completamente novo de coleção: *conjuntos*.



NOTA Os conjuntos são parte do padrão ES6. Para compatibilidade do navegador atual, consulte <http://mng.bz/QRTS>.

9.3.1 Criando nosso primeiro conjunto

A pedra angular da criação de conjuntos é a função construtora recém-introduzida, convenientemente chamada Definir. Vejamos um exemplo.

Listagem 9.20 Criando um conjunto

```
const ninjas = novo Conjunto (["Kuma", "Hattori", "Yagyu", "Hattori"]);

assert (ninjas.has ("Hattori"), "Hattori está em nosso conjunto");
assert (ninjas.size === 3, "Existem apenas três ninjas em nosso conjunto!");
```

**O construtor Set pode ter uma matriz de
itens com os quais o conjunto é inicializado.**

Descarta todos os itens duplicados

```
afirmar (! ninjas.has ("Yoshi"), "Yoshi não está, ainda .."); ninjas.add ("Yoshi");
```

```
assert (ninjas.has ("Yoshi"), "Yoshi é adicionado");
```

```
assert (ninjas.size === 4, "Existem quatro ninjas em nosso conjunto!"); no conjunto.
```

```
assert (ninjas.has ("Kuma"), "Kuma já foi adicionado"); ninjas.add ("Kuma");
```

```
assert (ninjas.size === 4, "Adicionar Kuma novamente não tem efeito");
```

```
para (vamos ninja de ninjas) {
    afirmar (ninja! == nulo, ninja);
}
```

Podemos adicionar novos itens que não são já contido

Adicionando existente itens não tem efeito.

Itera através do

definido com um for ... de loop

Aqui, usamos o integrado Definir construtor para criar um novo ninjas conjunto que conterá ninjas distintos. Se não passarmos nenhum argumento, um conjunto vazio é criado. Também podemos passar uma matriz, como esta, que preenche previamente o conjunto:

```
novo conjunto (["Kuma", "Hattori", "Yagy", "Hattori"]);
```

Como já mencionamos, conjuntos são coleções de itens únicos e seu objetivo principal é nos impedir de armazenar várias ocorrências do mesmo objeto. Neste caso, isso significa " Hattori ", que tentamos adicionar duas vezes, é adicionado apenas uma vez.

Vários métodos são acessíveis a partir de cada conjunto. Por exemplo, o tem método verifica se um item está contido no conjunto:

```
ninjas.has ("Hattori")
```

e a adicionar método é usado para adicionar itens únicos ao conjunto:

```
ninjas.add ("Yoshi");
```

Se você está curioso sobre como muitos itens estão em um conjunto, você sempre pode usar o Tamanho propriedade.

Semelhante a mapas e matrizes, conjuntos são coleções, então podemos iterar sobre eles com um para de ciclo. Como você pode ver na figura 9.16, os itens são sempre iterados na ordem em que foram inseridos.

Agora que passamos pelos fundamentos dos conjuntos, vamos visitar algumas operações comuns nos conjuntos: uniões, interseções e diferenças.

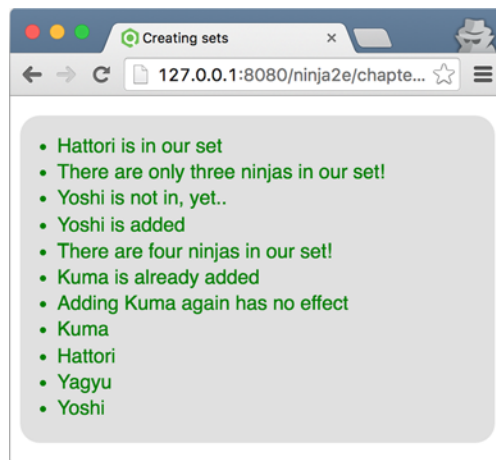


Figura 9.16 Executar o código da listagem 9.20 mostra que os itens em um conjunto são iterados na ordem em que foram inseridos.

9.3.2 União de conjuntos

Uma união de dois conjuntos, UMA e B, cria um novo conjunto que contém todos os elementos de ambos UMA e B. Naturalmente, cada item não pode ocorrer mais de uma vez no novo conjunto.

Listagem 9.21 Usando conjuntos para realizar uma união de coleções

```
const ninjas = ["Kuma", "Hattori", "Yagyu"]; const samurai = ["Hattori", "Oda", "Tomoe"];

guerreiros const = novo conjunto ([... ninjas, ... samurai]);

afirmar (warriors.has ("Kuma"), "Kuma está aqui"); afirme (warriors.has ("Hattori"), "E Hattori");
assert (warriors.has ("Yagyu"), "And Yagyu");
afirmar (warriors.has ("Oda"), "E Oda");
assert (warriors.has ("Tomoe"), "Tomoe, por último, mas não menos importante");

afirme (warriors.size === 5, "Há 5 guerreiros no total");
```

Cria uma série de ninjas e samurais. Observe que Hattori é um ninja e um samurai.

Cria um novo conjunto de guerreiros espalhando ninjas e samurai

Todos os ninjas e samurais são incluído em novo conjunto de guerreiros.

Não há duplicatas no novo conjunto. Mesmo que Hattori esteja nos conjuntos de ninjas e samurai, ele é incluído apenas uma vez.

Primeiro criamos uma matriz de ninjas e uma série de samurai. Observe que Hattori leva uma vida agitada: samurai de dia, ninja à noite. Agora imagine que precisamos criar um grupo de pessoas que possamos chamar às armas se um daimyo vizinho decidir que sua província está um pouco apertada. Criamos um novo conjunto, guerreiros, isso inclui todos os ninjas e todos os samurais. Hattori está em ambas as coleções, mas queremos incluí-lo apenas uma vez - não é como se dois Hattoris respondessem ao nosso chamado.

Nesse caso, um conjunto é perfeito! Não precisamos controlar manualmente se um item já foi incluído: o conjunto cuida disso sozinho, automaticamente. Ao criar este novo conjunto, usamos o operador spread [... ninjas, ... samurai] (lembre-se do capítulo 3) para criar um novo array que contenha todos os ninjas e todos os samurais. Caso você esteja se perguntando, o Hattori está presente duas vezes nesta nova matriz. Mas quando finalmente passamos essa matriz para o Definir construtor, Hattori é incluído apenas uma vez, conforme mostrado na figura 9.17.

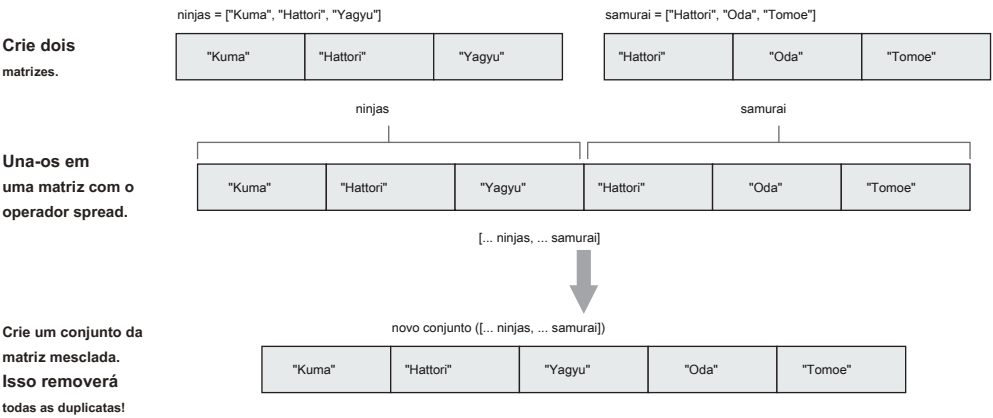


Figura 9.12 Uma união de dois conjuntos mantém os itens de ambas as coleções (sem duplicatas).

9.3.3 Interseção de conjuntos

O *interseção* de dois conjuntos, UMA e B, cria um conjunto que contém elementos de UMA que também estão em B. Por exemplo, podemos encontrar ninjas que também são samurais, como mostrado a seguir.

Listagem 9.22 Intersecção de conjuntos

```
const ninjas = novo conjunto (["Kuma", "Hattori", "Yagyu"]);
const samurai = new Set (["Hattori", "Oda", "Tomoe"]);

const ninjaSamurais = novo conjunto (
  [... ninjas].filter (ninja => samurai.has (ninja))
);

assert (ninjaSamurais.size === 1, "Há apenas um samurai ninja");
assert (ninjaSamurais.has ("Hattori"), "Hattori é o nome dele");
```

Usa o operador spread para virar nosso conjunto em uma matriz para que possamos usar o método de filtro da matriz para manter apenas ninjas que são contido no conjunto de samurai

A ideia por trás da listagem 9.22 é criar um novo conjunto que contenha apenas ninjas que também são samurais. Fazemos isso aproveitando as vantagens do array filtro método, que, como você deve se lembrar, cria uma nova matriz que contém apenas os itens que correspondem a um determinado critério. Nesse caso, o critério é que o ninja também seja um samurai (está contido no conjunto de samurai). Porque o filtro método só pode ser usado em matrizes, temos que transformar o ninjas definido em uma matriz usando o operador spread:

```
[... ninjas]
```

Por fim, verificamos que encontramos apenas um ninja que também é samurai: o pau para toda obra, Hattori.

9.3.4 Diferença de conjuntos

A diferença de dois conjuntos, UMA e B, contém todos os elementos que estão no conjunto UMA mas são *não* em conjunto B. Como você pode imaginar, isso é semelhante à interseção de conjuntos, com uma diferença pequena, mas significativa. Na próxima lista, queremos encontrar apenas ninjas verdadeiros (não aqueles que também fazem luar como samurais).

Listagem 9.23 Diferença de conjuntos

```
const ninjas = novo conjunto (["Kuma", "Hattori", "Yagyu"]);
const samurai = new Set (["Hattori", "Oda", "Tomoe"]);

const pureNinjas = new Set (
  [... ninjas].filter (ninja => !samurai.has (ninja))
);

assert (pureNinjas.size === 2, "Há apenas um samurai ninja");
assert (pureNinjas.has ("Kuma"), "Kuma é um verdadeiro ninja");
assert (pureNinjas.has ("Yagyu"), "Yagyu é um verdadeiro ninja");
```

Com diferença definida, nos preocupamos apenas com ninjas que não são samurais!

A única mudança é especificar que nos preocupamos apenas com os ninjas que são *não* também samurai, colocando um ponto de exclamação (!) antes do samurai.has (ninja) expressão.

9,4 *Resumo*

- Arrays são um tipo especial de objeto com um comprimento propriedade e Array.prototype como seu protótipo.
- Podemos criar novos arrays usando a notação literal de array (`[]`) ou chamando o Variedade construtor.
- Podemos modificar o conteúdo de uma matriz usando vários métodos acessíveis a partir de objetos de matriz:
 - O embutido `Empurre` e `pop` métodos adicionam itens e removem itens do final da matriz.
 - O embutido `mudança` e `não mudar` métodos adicionam itens e removem itens do início da matriz.
 - O embutido `emenda` O método pode ser usado para remover itens e adicionar itens a posições arbitrárias de array.
- Todos os arrays têm acesso a vários métodos úteis:
 - O `mapa` método cria uma nova matriz com os resultados de chamar um retorno de chamada em cada elemento.
 - O `cada` e alguns métodos determinam se todos ou alguns itens da matriz satisfazem um determinado critério.
 - O `achar` e `filtro` métodos encontram itens de array que satisfazem uma certa condição.
 - O `organizar` método classifica uma matriz. O `reduzir` método agrega todos os itens em uma matriz em um único valor.
- Você pode reutilizar os métodos de matriz integrados ao implementar seus próprios objetos, definindo explicitamente o contexto de chamada do método com o `chamar` ou `Aplique método`. Mapas e dicionários são objetos que contêm mapeamentos entre uma chave e um valor.
- Objetos em JavaScript são mapas ruins porque você só pode usar valores de string como chaves e porque sempre há o risco de acessar propriedades de protótipo. Em vez disso, use o novo Mapa coleção.
- Os mapas são coleções e podem ser iterados usando o `para de ciclo`. Conjuntos são coleções de itens exclusivos.

9,5 *Exercícios*

1 Qual será o conteúdo do `samurai` array, depois de executar o seguinte código?

```
const samurai = ["Oda", "Tomoe"]; samurai[3] = "Hattori";
```

2 Qual será o conteúdo do `ninjas` array, depois de executar o seguinte código?

```
const ninjas = [];  
  
ninjas.push("Yoshi");
```

```
ninjas.unshift("Hattori");

ninjas.length = 3;

ninjas.pop();
```

- 3 Qual será o conteúdo do samurai array, depois de executar o seguinte código?

```
const samurai = [];

samurai.push("Oda");
samurai.unshift("Tomoe");
samurai.splice(1, 0, "Hattori", "Takeda"); samurai.pop();
```

- 4 O que será armazenado nas variáveis primeiro segundo, e terceiro, depois de executar o código abaixo?

```
const ninjas = [{nome: "Yoshi", idade: 18},
                 {nome: "Hattori", idade: 19},
                 {nome: "Yagyu", idade: 20}];

const primeiro = pessoas.map(ninja => ninja.age); const segundo = primeiro.filtro(idade =>
idade% 2 == 0);
const third = first.reduce((agregado, item) => agregado + item, 0);
```

- 5 O que será armazenado nas variáveis primeiro e segundo, depois de executar o seguinte código?

```
const ninjas = [{nome: "Yoshi", idade: 18},
                 {nome: "Hattori", idade: 19}, {nome: "Yagyu", idade:
20}];

const primeiro = ninjas.some(ninja => ninja.age% 2 == 0); segundo const = ninjas.every(ninja => ninja.age%
2 == 0);
```

- 6 Qual das afirmações a seguir será aprovada?

```
const samuraiClanMap = new Map();

const samurai1 = {nome: "Toyotomi"}; const samurai2 = {nome:
"Takeda"}; const samurai3 = {nome: "Akiyama"};

const oda = {clã: "Oda"};
const tokugawa = {clã: "Tokugawa"}; const takeda = {clã: "Takeda"};

samuraiClanMap.set(samurai1, oda);
samuraiClanMap.set(samurai2, tokugawa);
samuraiClanMap.set(samurai2, takeda);
```

```
assert (samuraiClanMap.size === 3, "Existem três mapeamentos"); assert (samuraiClanMap.has (samurai1), "O primeiro samurai tem um mapeamento"); assert (samuraiClanMap.has (samurai3), "O terceiro samurai tem um mapeamento");
```

7 Qual das afirmações a seguir será aprovada?

```
const samurai = novo Conjunto ("Toyotomi", "Takeda", "Akiyama", "Akiyama"); assert (samurai.size === 4, "Existem quatro samurais no conjunto");
```

```
samurai.add ("Akiyama");  
assert (samurai.size === 5, "Existem cinco samurais no conjunto");
```

```
assert (samurai.has ("Toyotomi", "Toyotomi está dentro!"); assert (samurai.has ("Hattori", "Hattori está dentro!");
```

10

Wrangling expressões regulares

Este capítulo cobre

- Uma atualização sobre expressões regulares
- Compilando expressões regulares
- Captura com expressões regulares
- Trabalhar com expressões idiomáticas frequentemente encontradas

As expressões regulares são uma necessidade do desenvolvimento moderno. Lá, nós dissemos isso. Embora muitos desenvolvedores da Web possam passar a vida ignorando as expressões regulares, alguns problemas que precisam ser resolvidos no código JavaScript não podem ser resolvidos com elegância sem as expressões regulares.

Claro, pode haver outras maneiras de resolver os mesmos problemas. Mas frequentemente, algo que pode ocupar uma meia tela de código pode ser destilado em uma única instrução com o uso adequado de expressões regulares. Todos os ninjas do JavaScript precisam de expressões regulares como parte essencial de seus kits de ferramentas.

As expressões regulares trivializam o processo de separar strings e procurar informações. Onde quer que você olhe nas principais bibliotecas JavaScript, você verá o uso predominante de expressões regulares para várias tarefas pontuais:

- Manipulando strings de nós HTML
- Localizando seletores parciais em uma expressão de seletor CSS
- Determinar se um elemento tem um nome de classe específico

- Validação de entrada
- E mais

Vamos começar vendo um exemplo.

GORJETA Tornar-se fluente em expressões regulares exige muita prática. Você pode encontrar um site como JS Bin (<http://jsbin.com>) útil para brincar com exemplos. Alguns sites são dedicados a testes de expressão regular, como a página de teste de expressão regular para JavaScript (www.regexplanet.com/advanced/javascript/index.html)

e regex101 (www.regex101.com/#javascript) .

regex101 é um site especialmente útil para iniciantes, porque também gera automaticamente explicações para a expressão regular de destino.

.....

Quando você prefere usar um literal RegExp em vez de um Objeto RegExp?

Você sabe?

O que é correspondência aderente e como você a habilita?

Como a correspondência difere ao usar um global versus um expressão regular não global?

.....

10.1 Por que as expressões regulares são demais

Digamos que desejamos validar que uma string, talvez inserida em um formulário por um usuário de site, segue o formato de um código postal dos EUA de nove dígitos. Todos nós sabemos que o Serviço Postal dos EUA tem pouco senso de humor e insiste que um código postal (também conhecido como CEP) siga este formato específico:

99999-9999

Cada 9 representa um dígito decimal, e o formato é 5 dígitos decimais, seguidos por um hífen, seguido por 4 dígitos decimais. Se você usar qualquer outro formato, seu pacote ou carta será desviado para o buraco negro do departamento de classificação manual e boa sorte ao prever quanto tempo levará para emergir novamente.

Vamos criar uma função que, dada uma string, verifique se o US Postal Service ficará feliz. Poderíamos recorrer à comparação de cada personagem, mas somos um ninja e isso é uma solução muito deselegante, resultando em muitas repetições desnecessárias. Em vez disso, considere a seguinte solução.

Listagem 10.1 Testando um padrão específico em uma string

```
function isThisAZipCode (candidate) {
  if (typeof candidate! == "string" ||
      candidato.comprimento! = 10) retorna falso; para (seja n = 0; n <comprimento
  candidato; n ++) {
    seja c = candidato [n];
```

**Curto-circuitos obviamente
candidatos falsos**

```

switch (n) {
  caso 0: caso 1: caso 2: caso 3: caso 4: caso 6: caso 7: caso 8: caso 9:

    if (c <'0' || c > '9') retorna falso; intervalo;

  caso 5:
    if (c != '-') retorna falso; intervalo;

}
}
return true;
}

```

Executa testes com base no índice de personagem

Se tudo der certo, foram bons!

Este código tira vantagem do fato de que temos apenas duas verificações a fazer, dependendo da posição do caractere dentro da string. Ainda precisamos realizar até nove comparações em tempo de execução, mas temos que escrever cada comparação apenas uma vez.

Mesmo assim, alguém consideraria esta solução *elegante*? É mais elegante do que a abordagem de força bruta e não literária, mas ainda parece uma quantidade enorme de código para uma verificação tão simples. Agora considere esta abordagem:

```

function isThisAZipCode (candidate) {
  return /^d{5}-d{4}$/.test(candidate);
}

```

Exceto por alguma sintaxe esotérica no corpo da função, ela é muito mais sucinta e elegante, não? Esse é o poder das expressões regulares e é apenas a ponta do iceberg. Não se preocupe se essa sintaxe parecer que a iguana de estimação de alguém atravessou o teclado; estamos prestes a recapitular as expressões regulares antes de você aprender a usá-las de maneira ninja em suas páginas.

10.2 Uma atualização de expressão regular

Por mais que gostaríamos, não podemos oferecer um tutorial exaustivo sobre expressões regulares no espaço que temos. Afinal, livros inteiros foram dedicados a expressões regulares. Mas faremos o nosso melhor para atingir todos os pontos importantes.

Para mais detalhes do que podemos oferecer neste capítulo, os livros *Dominando Expressões Regulares* por Jeffrey EF Friedl, *Apresentando Expressões Regulares* por Michael Fitzgerald, e *Livro de receitas de expressões regulares* por Jan Goyvaerts e Steven Levithan, todos da O'Reilly, são escolhas populares.

Vamos cavar.

10.2.1 Expressões regulares explicadas

O termo *expressão regular* deriva da matemática de meados do século, quando um matemático chamado Stephen Kleene descreveu modelos de autômatos computacionais como "conjuntos regulares". Mas isso não nos ajudará a entender nada sobre expressões regulares, então vamos simplificar as coisas e dizer que uma expressão regular é uma forma de expressar uma *padronizar* para combinar sequências de texto. A própria expressão consiste em termos e

operadores que nos permitem definir esses padrões. Veremos em que consistem esses termos e operadores em breve.

Em JavaScript, como na maioria dos outros tipos de objeto, temos duas maneiras de criar uma expressão regular:

- Por meio de uma expressão regular literal
- Construindo uma instância de um RegExp objeto

Por exemplo, se quisermos criar uma expressão regular mundana (ou *regex*, para breve) que corresponde à string teste exatamente, poderíamos fazer isso com um literal regex:

```
padrão const = / teste /;
```

Isso pode parecer estranho, mas os literais regex são delimitados por barras da mesma forma que literais de string são delimitados por aspas.

Alternativamente, poderíamos construir um RegExp instância, passando o regex como uma string:

```
padrão const = novo RegExp ("teste");
```

Ambos os formatos resultam na mesma regex sendo criada na variável padronizar.

GORJETA A sintaxe literal é preferida quando a regex é conhecida no tempo de desenvolvimento e a abordagem do construtor é usada quando a regex é construída em tempo de execução, construindo-a dinamicamente em uma string.

Um dos motivos pelos quais a sintaxe literal é preferível a expressar regexes em uma string é que (como você verá em breve) o caractere de barra invertida desempenha um papel importante nas expressões regulares. Mas o caractere barra invertida é *Além disso* o caractere de escape para literais de string, portanto, para expressar uma barra invertida dentro de um literal de string, precisamos usar uma barra invertida dupla (\). Isso pode tornar as expressões regulares, que já possuem uma sintaxe enigmática, ainda mais estranhas quando expressas em strings.

Além da própria expressão, cinco sinalizadores podem ser associados a um regex:

- **eu** - Torna o regex insensível a maiúsculas e minúsculas, então `/ teste / i` corresponde não só *teste*, mas também *Teste*, *teste*, *teste*, e assim por diante.
- **g** - Corresponde a todas as instâncias do padrão, em oposição ao padrão de *local*, que corresponde apenas à primeira ocorrência. Mais sobre isso mais tarde.
- **m** —Permite correspondências em várias linhas, como pode ser obtido a partir do valor de um área de texto elemento.
- **y** —Ativa a correspondência fixa. Uma expressão regular executa a correspondência fixa na string de destino, tentando corresponder a partir da última posição de correspondência.
- **você** —Ativa o uso de pontos de escape Unicode (`\ você{...}`).

Esses sinalizadores são anexados ao final do literal (por exemplo, `/ test / ig`) ou passado em uma string como o segundo parâmetro para o RegExp construtor (`novo RegExp ("teste", "ig")`).

Combinando a string exata *teste* (mesmo que não faça distinção entre maiúsculas e minúsculas) não é interessante - afinal, podemos fazer essa verificação específica com uma comparação de string simples. Portanto, vamos dar uma olhada nos termos e operadores que fornecem às expressões regulares seu imenso poder para corresponder a padrões mais atraentes.

10.2.2 Termos e operadores

As expressões regulares, como a maioria das outras expressões com as quais estamos familiarizados, são compostas de termos e operadores que qualificam esses termos. Nas seções a seguir, você verá como esses termos e operadores podem ser usados para expressar padrões.

CORRESPONDÊNCIA EXATA

Qualquer caractere que não seja um caractere ou operador especial (que apresentaremos à medida que prosseguirmos) deve aparecer literalmente na expressão. Por exemplo, em nosso / teste/ regex, quatro termos representam caracteres que devem aparecer literalmente em uma string para corresponder ao padrão expresso.

Colocar esses caracteres um após o outro denota implicitamente uma operação que significa *seguido pela*. Então / teste/ meios *t* seguido pela *e* seguido pela *s* seguido pela *t*.

CORRESPONDÊNCIA DE UMA CLASSE DE PERSONAGENS

Muitas vezes, não queremos corresponder a um caractere literal específico, mas a um caractere de um conjunto finito de caracteres. Podemos especificar isso com o operador set (também chamado de *classe de personagem* operador) colocando o conjunto de caracteres que queremos corresponder entre colchetes: [abc].

O exemplo anterior significa que queremos corresponder a qualquer um dos caracteres a, b, ou c. Observe que, embora essa expressão tenha cinco caracteres (três letras e dois colchetes), ela corresponde a apenas um único caractere na string candidata.

Outras vezes, queremos combinar qualquer coisa *mas* um conjunto finito de caracteres. Podemos especificar isso colocando um caractere circunflexo (^) logo após o colchete de abertura do operador de conjunto:

```
[^ abc]
```

Isso muda o significado de qualquer caractere *mas* a, b, ou c.

Há mais uma variação inestimável para a operação de conjunto: a capacidade de especificar uma faixa de valores. Por exemplo, se quisermos combinar qualquer um dos caracteres minúsculos entre *uma* e *m*, poderíamos escrever [abcdefghijklm]. Mas podemos expressar isso muito mais sucintamente da seguinte forma:

```
[sou]
```

O traço indica que todos os personagens de uma Através dos m inclusivos (e lexicograficamente) estão incluídos no conjunto.

ESCAPANDO

Nem todos os caracteres representam seu equivalente literal. Certamente, todos os caracteres alfabéticos e decimais representam a si mesmos, mas, como você verá, os caracteres especiais como \$ e o ponto (.) Representam correspondências com algo diferente de si mesmos,

ou operadores que qualificam o termo anterior. Na verdade, você já viu como o `[.]`, `-`, e `^` caracteres são usados para representar algo diferente de seus eus literais.

Como especificamos que desejamos corresponder a um literal [ou `$` ou `^` otherou outro caractere especial? Em uma regex, o caractere de barra invertida escapa de qualquer caractere que o segue, tornando-o um termo de correspondência literal. Portanto, `\` [especifica uma correspondência literal com o caractere `[`, em vez da abertura de uma expressão de classe de caractere. Uma barra invertida dupla (`\\`) corresponde a uma única barra invertida.

COMEÇA E TERMINA

Freqüentemente, podemos querer garantir que um padrão corresponda ao início de uma string ou talvez ao final de uma string. O caractere circunflexo, quando usado como o primeiro caractere da regex, ancora a correspondência no início da string, de modo que `/^ teste/` corresponde apenas se a substring teste aparece no início da string que está sendo correspondida. (Observe que isso é uma sobrecarga do caractere `^`, porque também é usado para negar um conjunto de classes de caracteres.)

Da mesma forma, o cifrão (`$`) significa que o padrão deve aparecer no final da string:

`/ test $ /`

O uso de `^` e `$` indica que o padrão especificado deve abranger toda a string candidata:

`/^ teste $ /`

OCORRÊNCIAS REPETIDAS

Se quisermos combinar uma série de quatro uma personagens, podemos expressar isso com `/aaaa/`, mas e se quisermos combinar *nenhum* número do mesmo personagem? As expressões regulares nos permitem especificar várias opções de repetição:

- Para especificar que um caractere é opcional (ele pode aparecer uma vez ou não), siga-o com `?`. Por exemplo, `/ teste/` combina com ambos *teste* e *Husa*.
- Para especificar que um caractere deve aparecer uma ou várias vezes, use `+`, como em `/ t + est /`, que combina *test*, *ttest*, e *tttest*, mas não *Husa*.
- Para especificar que o personagem aparece *Zero um*, ou *vários* vezes, use `*`, como em `/ teste/`, que combina *test*, *ttest*, *tttest* e *est*.
- Para especificar um número fixo de repetições, indique o número de repetições permitidas entre colchetes. Por exemplo, `/ um {4} /` indica uma partida em quatro consecutivas uma personagens.
- Para especificar um intervalo para a contagem de repetição, indique o intervalo com um separador de vírgula. Por exemplo, `/ a {4,10} /` corresponde a qualquer sequência de 4 a 10 consecutivos uma personagens.
- Para especificar um intervalo aberto, omita o segundo valor do intervalo (mas deixe a vírgula). O regex `/ a {4,} /` corresponde a qualquer sequência de quatro ou mais consecutivas uma personagens.

Qualquer um desses operadores de repetição pode ser *ambicioso* ou *não melancólico*. Por padrão, eles são gananciosos: eles vão consumir todos os personagens possíveis que compõem uma combinação. Anotando o operador com um? personagem (uma sobrecarga do operador?), como em a + ?, torna a operação não séria: vai consumir *só* caracteres suficientes para fazer uma correspondência.

Por exemplo, se estivermos combinando com a string aaa, a expressão regular / a + / corresponderia a todos os três uma caracteres, enquanto a expressão não agitada / a +? / corresponderia a apenas um uma personagem, porque um único uma caráter é tudo o que é necessário para satisfazer o a + prazo.

CLASSES DE CARÁTER PREDEFINIDAS

Alguns caracteres que podemos desejar corresponder são impossíveis de especificar com caracteres literais (por exemplo, caracteres de controle, como um retorno de carro). Além disso, muitas vezes podemos querer combinar classes de caracteres, como um conjunto de dígitos decimais ou um conjunto de caracteres de espaço em branco. A sintaxe da expressão regular fornece termos predefinidos que representam esses caracteres ou classes comumente usadas para que possamos usar a correspondência de caracteres de controle em nossas expressões regulares e não precisemos recorrer ao operador de classe de caracteres para conjuntos de caracteres comumente usados.

A Tabela 10.1 lista esses termos e o caractere ou conjunto que eles representam. Esses conjuntos predefinidos ajudam a evitar que nossas expressões regulares pareçam excessivamente enigmáticas.

Tabela 10.1 Classes de personagens e termos de personagens predefinidos

Termo predefinido	Partidas
\ t	Aba horizontal
\ b	Backspace
\ v	Aba vertical
\ f	Feed de formulário
\ r	Retorno de carruagem
\ n	Nova linha
\ cA: \ cZ	Personagens de controle
\ u0000: \ uFFFF	Hexadecimal Unicode
\ x00: \ xFF	Hexadecimal ASCII
.	Qualquer caractere, exceto para caracteres de espaço em branco (\ s)
\ d	Qualquer dígito decimal; equivalente a [0-9]
\ D	Qualquer caractere, exceto um dígito decimal; equivalente a [^ 0-9]
\ C	Qualquer caractere alfanumérico incluindo sublinhado; equivalente a [A-Za-z0-9_]
\ C	Qualquer caractere, exceto caracteres alfanuméricos e sublinhados; equivalente a [^ A-Za-z0-9_]

Tabela 10.1 Classes de caracteres e termos de caracteres predefinidos (*contínuo*)

Termo predefinido	Partidas
<code>\s</code>	Qualquer caractere de espaço em branco (espaço, tabulação, alimentação de formulário e assim por diante)
<code>\S</code>	Qualquer caractere, exceto um caractere de espaço em branco
<code>\b</code>	Um limite de palavra
<code>\B</code>	Sem limite de palavra (dentro de uma palavra)

AGRUPAMENTO

Até agora, você viu que os operadores (como `+` e `*`) afetam apenas o termo anterior. Se quisermos aplicar o operador a um grupo de termos, podemos usar parênteses para grupos, assim como em uma expressão matemática. Por exemplo, `/(ab)+/` corresponde a uma ou mais ocorrências consecutivas da substring `ab`.

Quando uma parte de uma regex é agrupada com parênteses, ela tem uma dupla função, criando também o que é conhecido como *capturar*. Há muitas capturas e as discutiremos com mais detalhes na seção 10.4.

ALTERNAÇÃO (OU)

As alternativas podem ser expressas usando o caractere barra vertical (`|`). Por exemplo: `/a|b/` combina com o `a` ou `b` personagem, e `/(ab)+|(cd)+/` corresponde a uma ou mais ocorrências de qualquer `ab` ou `cd`.

BACKREFERENCES

Os termos mais complexos que podemos expressar em expressões regulares são referências anteriores a *captura* definido no regex. Abordamos as capturas detalhadamente na seção 10.4, mas por enquanto apenas pense nelas como as partes de uma string candidata que são correspondidas com êxito aos termos da expressão regular. A notação para tal termo é a barra invertida seguida pelo número da captura a ser referenciada, começando com 1, como `\1`, `\2`, e assim por diante.

Um exemplo é `/^([dtn])a\1/`, que corresponde a uma string que começa com qualquer um dos `d`, `t`, ou `n` caracteres, seguidos por um `a`, seguido por qualquer caractere que corresponda à primeira captura. Este último ponto é importante! Este não é o mesmo que `/[dtn]a[dtn]/`. O personagem seguindo o `a` não pode ser nenhum de `d`, ou `t`, ou `n`, mas deve ser aquele que acionar a correspondência para o primeiro caractere. Como tal, qual personagem o `\1` irá corresponder não pode ser conhecido até a hora da avaliação.

Um bom exemplo de onde isso pode ser útil é na correspondência de elementos de marcação do tipo XML. Considere o seguinte regex:

```
/<(\w+)>(.+)<\1>/
```

Isso nos permite combinar elementos simples, como `< forte>` tanto faz `</ forte>`. Sem a capacidade de especificar uma referência anterior, isso não seria possível, porque não teríamos como saber com antecedência qual tag de fechamento corresponderia à tag de abertura.

GORJETA Foi uma espécie de curso intensivo sobre expressões regulares. Se eles ainda estão fazendo você puxar seus cabelos e você se encontrar atolado no material a seguir, recomendamos o uso de um dos recursos mencionados anteriormente neste capítulo.

Agora que você conhece as expressões regulares, está pronto para ver como usá-las sabiamente em seu código.

10.3 Compilando expressões regulares

As expressões regulares passam por várias fases de processamento e entender o que acontece durante cada fase pode nos ajudar a otimizar o código JavaScript que usa expressões regulares. As duas fases principais são compilação e execução.

Compilação ocorre quando a expressão regular é criada pela primeira vez. *Execução* ocorre quando usamos a expressão regular compilada para combinar padrões em uma string.

Durante a compilação, a expressão é analisada pelo mecanismo JavaScript e convertida em sua representação interna (seja ela qual for). Esta fase de análise e conversão deve ocorrer toda vez que uma expressão regular é criada (descontando quaisquer otimizações internas realizadas pelo navegador).

Freqüentemente, navegadores *estamos* inteligente o suficiente para determinar quando expressões regulares idênticas estão sendo usadas e para armazenar em cache os resultados da compilação para aquela expressão específica. Mas não podemos contar com isso sendo o caso em todos os navegadores. Para expressões complexas, em particular, podemos começar a obter algumas melhorias de velocidade perceptíveis predefinindo (e, portanto, pré-compilando) nossas expressões regulares para uso posterior.

Como aprendemos em nossa visão geral das expressões regulares na seção anterior, há duas maneiras de criar uma expressão regular compilada em JavaScript: por meio de um literal e por meio de um construtor. Vejamos um exemplo na lista a seguir.

Listagem 10.2 Duas maneiras de criar uma expressão regular compilada

```
const re1 = / teste / i;
const re2 = novo RegExp ("teste", "i"); assert (re1.toString () === "/
test / i",
    "Verifique o conteúdo da expressão.");
assert (re1.test ("TesT"), "Sim, não faz distinção entre maiúsculas e minúsculas."); assert (re2.test ("TesT"),
    "Este também é."); assert (re1.toString () === re2.toString (),
    "As expressões regulares são iguais.");
assert (re1 !== re2, "Mas eles são objetos diferentes.");
```

← Cria um regex por meio de um literal

← Cria um regex por meio do construtor

Neste exemplo, ambas as expressões regulares estão em seu estado compilado após a criação. Se substituíssemos todas as referências a `re1` com o literal `/ teste / i`, é possível que o mesmo regex seja compilado repetidamente, portanto, compilar um regex *uma vez* e armazená-lo em uma variável para referência posterior pode ser uma otimização importante.

Observe que cada regex tem uma representação de objeto exclusiva: Cada vez que uma expressão regular é criada (e, portanto, compilada), um novo objeto de expressão regular é criado.

Isso é diferente de outros tipos primitivos (como string, número e assim por diante), porque o resultado sempre será único.

De particular importância é o uso do construtor (novo `RegExp (...)`) para criar uma expressão regular. Essa técnica nos permite construir e compilar uma expressão a partir de uma string que podemos criar dinamicamente em tempo de execução. Isso pode ser extremamente útil para construir expressões complexas que serão amplamente reutilizadas.

Por exemplo, digamos que desejamos determinar quais elementos em um documento têm um nome de classe específico, cujo valor não saberemos até o tempo de execução. Como os elementos podem ter vários nomes de classes associados a eles (armazenados de forma inconveniente em uma string delimitada por espaço), isso serve como um exemplo interessante de tempo de execução, compilação de expressão regular (consulte a lista a seguir).

Listagem 10.3 Compilando uma expressão regular de tempo de execução para uso posterior

```
<div class = "samurai ninja"> </div>
<div class = "ninja samurai"> </div>
<div> </div>
<span class = "samurai ninja ronin"> </span> <script>

    function findClassInElements (className, type) {
        const elems =
            document.getElementsByTagName (digite || "");
        const regex =
            new RegExp ("(^ | \\s)" + className + "(\\s | $)"); resultados const = [];

        para (seja i = 0, comprimento = elems.length; i < comprimento; i++)
            if (regex.test (elems [i].className)) {
                resultados.push (elems [i]);
            }
        resultados de retorno;
    }
    assert (findClassInElements ("ninja", "div"). length === 2,
        "A quantidade certa de ninjas div foi encontrada.");
    assert (findClassInElements ("ninja", "span"). length === 1,
        "A quantidade certa de ninjas span foi encontrada."); assert (findClassInElements
        ("ninja"). length === 3,
        "A quantidade certa de ninjas foi encontrada.");
</script>
```

Armazena o resultados →

Cria assuntos de teste de vários elementos com vários nomes de classe

← **Coleta elementos por tipo**

← **Compila um regex usando o nome de classe passado**

← **Testes para correspondências de regex**

Podemos aprender várias coisas interessantes na Listagem 10.3. Para começar, configuramos uma série de assuntos de teste `<div>` e `<span>` elementos com várias combinações de nomes de classes. Em seguida, definimos nossa função de verificação de nome de classe, que aceita como parâmetros o nome da classe para a qual verificaremos e o tipo de elemento a ser verificado.

Em seguida, coletamos todos os elementos do tipo especificado usando o `getElementsByTagName` método integrado e configure nossa expressão regular:

```
const regex = new RegExp ("(^ | \\s)" + className + "(\\s | $)");
```

Observe o uso do novo RegExp () construtor para compilar uma expressão regular com base no nome da classe passado para a função. Esta é uma instância em que não podemos usar um literal de regex, pois o nome da classe que pesquisaremos não é conhecido de antemão.

Construímos (e, portanto, compilamos) esta expressão uma vez para evitar recompilações frequentes e desnecessárias. Porque o conteúdo da expressão é dinâmico (com base na entrada nome da classe argumento), podemos obter grandes economias de desempenho manipulando a expressão dessa maneira.

A própria regex corresponde ao início da string ou a um caractere de espaço em branco, seguido pelo nome da classe de destino, seguido por um caractere de espaço em branco ou o final da string. Observe o uso de um escape duplo (\\) no novo regex: \\ s.

Ao criar expressões regulares literais com termos incluindo a barra invertida, temos que fornecer a barra invertida apenas uma vez. Mas, como estamos escrevendo essas barras invertidas em uma string, devemos fazer um duplo escape delas. Isso é um incômodo, com certeza, mas devemos estar atentos ao construir expressões regulares em strings em vez de literais.

Depois que o regex é compilado, usá-lo para coletar os elementos correspondentes é fácil por meio do teste método:

```
regex.test (elems [i].className)
```

A pré-construção e pré-compilação de expressões regulares para que possam ser reutilizadas (executadas) vezes sem conta é uma técnica recomendada que fornece ganhos de desempenho que não podem ser ignorados. Praticamente todas as situações complexas de expressão regular podem se beneficiar com o uso dessa técnica.

Anteriormente neste capítulo, mencionamos que o uso de parênteses em expressões regulares não serve apenas para agrupar termos para a aplicação do operador, mas também cria *captura*. Vamos descobrir mais sobre isso.

10.4 Captura de segmentos correspondentes

A altura da utilidade em relação às expressões regulares é percebida quando nós *capturar* os resultados que são encontrados para que possamos fazer algo com eles. Determinar se uma string corresponde a um padrão é um primeiro passo óbvio e muitas vezes tudo de que precisamos, mas determinar *o que* foi correspondido também é útil em muitas situações.

10.4.1 Execução de capturas simples

Digamos que queremos extrair um valor embutido em uma string complexa. Um bom exemplo de tal string é o valor da propriedade de transformação do CSS, por meio do qual podemos modificar a posição visual de um elemento HTML.

Listagem 10.4 Uma função simples para capturar um valor incorporado

```
<div id = "square" style = "transform: translateY (15px);"> </div> <script>
```

```
function getTranslateY (elem) {
  const transformValue = elem.style.transform;
```

Define o
cobaia

```

    if (transformValue) {
      const match = transformValue.match (/ translateY \ (((^ \)|) +) \) /); jogo de volta? correspondência [1]: "";

    }

    Retorna "";
  }

  const square = document.getElementById ("square");

  assert (getTranslateY (square) === "15px",
    "Extraímos o valor translateY");
</script>

```

**Extraí o translateY
valor da string**

Definimos um elemento que especifica o estilo que traduzirá sua posição em 15 px:

```
"transform: translateY (15px);"
```

Infelizmente, o navegador não oferece uma API para buscar facilmente a quantidade pela qual o elemento é traduzido. Então, criamos nossa própria função:

```

function getTranslateY (elem) {
  const transformValue = elem.style.transform; if (transformValue) {

    const match = transformValue.match (/ translateY \ (((^ \)|) +) \) /); jogo de volta? correspondência [1]: "";

  }

  Retorna "";
}

```

O código de análise de transformação pode parecer confuso no início:

```
const match = transformValue.match (/ translateY \ (((^ \)|) +) \) /); jogo de volta? correspondência [1]: "";
```

Mas não é tão ruim quando o quebramos. Para começar, precisamos determinar se um transformar até mesmo existe propriedade para analisarmos. Caso contrário, retornaremos uma string vazia. Se o transformar propriedade é residente, podemos descer para a extração do valor de opacidade. O Combine método de uma expressão regular retorna uma matriz de valores capturados se uma correspondência for encontrada, ou nulo se nenhuma correspondência for encontrada.

A matriz retornada por Combine inclui toda a correspondência no primeiro índice e, em seguida, cada captura subsequente. Portanto, a entrada zero seria toda a sequência combinada de translateY (15px), e a entrada na próxima posição seria 15px.

Lembre-se de que as capturas são definidas por parênteses na expressão regular. Assim, quando combinamos o valor de transformação, o valor está contido no [1] posição da matriz, porque a única captura que especificamos em nossa regex foi criada pelos parênteses que incorporamos após o traduzir Y parte do regex.

Este exemplo usa uma expressão regular local e o Combine método. As coisas mudam quando usamos expressões globais. Vamos ver como.

10.4.2 Correspondência usando expressões globais

Como vimos na seção anterior, usando uma expressão regular local (uma sem o sinalizador global) com o `Fragmento` do objeto `Combine` métodos retorna uma matriz contendo toda a string correspondida, junto com quaisquer capturas correspondidas na operação.

Mas quando fornecemos uma expressão regular global (uma com a `g` bandeira incluída), `Combine` retorna algo diferente. Ainda é uma matriz de resultados, mas no caso de uma expressão regular global, que corresponde a todas as possibilidades na string candidata, e não apenas à primeira correspondência, a matriz retornada contém as correspondências globais; captura *dentro de* cada correspondência não é retornada neste caso.

Podemos ver isso em ação no código e nos testes a seguir.

Listagem 10.5 Diferenças entre pesquisas globais e locais com `Combine`

```
const html = "<div class = 'test'> <b> Olá </b> <i>world!</i> </div>"; resultados const = html.match (/ <(\/? ) (\ w +) ([^>] *)?> /);

assert (results [0] === "<div class = 'test'>", "A correspondência inteira."); assert (results [1] === "", "A barra (ausente).");

assert (resultados [2] === "div", "O nome da tag.");
assert (resultados [3] === "classe = 'teste'", "Os atributos.");

const all = html.match (/ <(\/? ) (\ w +) ([^>] *)?> / g); assert (all [0] === "<div class = 'test'>", "Abrindo tag div.");
assert (all [1] === "<b>", "Abrindo tag b.");

assert (all [2] === "</b>", "Fechando tag b."); assert (all [3] === "<i>", "Abrindo tag i."); assert
(all [4] === "</i>", "Fechando a tag i."); assert (all [5] === "</div>", "Fechando tag div.");
```

Jogos usando
um regex local

Jogos usando
um regex global

Podemos ver que quando fazemos uma partida local, `html.match (/ <(\/? ) (\ w +) ([^>] *)?> /)`, uma única instância é correspondida e as capturas nessa correspondência também são retornadas. Mas quando usamos uma correspondência global, `html.match (/ <(\/? ) (\ w +) ([^>] *)?> / g)`, o que é retornado é a lista de correspondências.

Se as capturas forem importantes para nós, podemos recuperar essa funcionalidade enquanto ainda fazemos uma pesquisa global usando a expressão regular `exec` método. Este método pode ser repetidamente chamado em uma expressão regular, fazendo com que ela retorne o próximo conjunto de informações correspondentes toda vez que for chamada. Um padrão típico de uso é mostrado na lista a seguir.

Listagem 10.6 Usando o `exec` método para fazer a captura e uma pesquisa global

```
const html = "<div class = 'test'> <b> Olá </b> <i>world!</i> </div>"; tag const = / <(\/? ) (\ w +) ([^>] *)?> / g;

deixe corresponder, num = 0;
while ((match = tag.exec (html))! == null) {
    assert (match.length === 4,
        "Cada correspondência encontra cada tag e 3 capturas.");
    num ++;
}
assert (num === 6, "3 tags de abertura e 3 tags de fechamento encontradas.");
```

Repetidamente
chama `exec`

Neste exemplo, chamamos repetidamente o `exec` método:

```
while ((match = tag.exec (html)) != null) {...}
```

Isso retém o estado da chamada anterior para que cada chamada subsequente avance para a próxima correspondência global. Cada chamada retorna a próxima partida e suas capturas.

Usando qualquer `Combine` ou `exec`, sempre podemos encontrar as correspondências exatas (e capturas) que estamos procurando. Mas precisamos ir mais longe se quisermos nos referir às próprias capturas dentro da regex.

10.4.3 Capturas de referência

Podemos nos referir a partes de uma correspondência que capturamos de duas maneiras: uma dentro da própria correspondência e outra dentro de uma string de substituição (quando aplicável). Por exemplo, vamos revisitar a correspondência na listagem 10.6 (na qual combinamos uma tag HTML de abertura ou fechamento) e modificá-la na lista a seguir para também corresponder ao conteúdo interno da própria tag.

Listagem 10.7 Usando referências anteriores para corresponder ao conteúdo de uma tag HTML

```
const html = "<b class = 'hello'> Olá </b> <i> mundo! </i>"; padrão const = /<(w+)([^\>]*)(.*)</\1>/g; deixe
corresponder = pattern.exec (html);

assert (match [0] === "<b class = 'hello'> Olá </b>",
      "A tag inteira, do início ao fim."); assert (match [1] === "b", "O nome
da tag.");
assert (match [2] === "class = 'hello'", "Os atributos da tag."); assert (match [3] === "Olá", "O conteúdo da tag.");

match = pattern.exec (html);
assert (match [0] === "<i> mundo! </i>",
      "A tag inteira, do início ao fim."); assert (match [1] === "i", "O nome da tag."); assert (match [2] ===
"", "Os atributos da tag."); assert (match [3] === "mundo!", "O conteúdo da tag.");
```

Usa captura de referência anterior

Executa o padrão na string de teste

Testa vários captura que são capturado pelo padrão definido

Nós usamos `\ 1` para se referir à primeira captura dentro da expressão, que neste caso é o nome da tag. Usando essas informações, podemos combinar a tag de fechamento apropriada, referindo-se a qualquer que seja a captura correspondida. (Isso tudo pressupõe, é claro, que não existem tags incorporadas com o mesmo nome na tag atual, portanto, este dificilmente é um exemplo completo de correspondência de tag.)

Além disso, podemos obter referências de captura dentro da string de substituição de uma chamada para o `substituir` método. Em vez de usar os códigos de referência anterior, como na listagem 10.7, usamos a sintaxe de `$ 1`, `$ 2`, `$ 3`, através de cada número de captura. Aqui está um exemplo:

```
assert ("fontFamily".replace (/ ([AZ]) / g, "- $ 1").toLowerCase () ===
      "font-family", "Converta o camelCase em notação tracejada.");
```

Neste código, o valor da primeira captura (neste caso, a letra maiúscula F) é referenciado no *substituir string* (via \$ 1). Isso nos permite especificar uma string de substituição, mesmo sem saber qual será seu valor até o momento da correspondência. Essa é uma arma poderosa do tipo ninja para empunhar.

A capacidade de fazer referência a capturas de expressão regular ajuda a tornar bastante fácil muitos códigos que de outra forma seriam difíceis. A natureza expressiva que ele fornece acaba permitindo algumas declarações concisas que de outra forma poderiam ser um tanto obtusas, complicadas e extensas.

Como as capturas e o agrupamento de expressões são especificados usando parênteses, não há como o processador de expressões regulares saber quais conjuntos de parênteses adicionamos ao regex para agrupamento e quais deveriam indicar capturas. Ele trata todos os conjuntos de parênteses como grupos e capturas, o que pode resultar na captura de mais informações do que realmente pretendíamos, porque precisávamos especificar algum agrupamento na regex. O que podemos fazer nesses casos?

10.4.4 Grupos de não captura

Como observamos, os parênteses têm uma dupla função: eles não apenas agrupam termos para operações, mas também especificam capturas. Isso geralmente não é um problema, mas em expressões regulares nas quais muitos agrupamentos estão ocorrendo, isso pode causar muitas capturas desnecessárias, o que pode tornar a classificação das capturas resultantes entediante.

Considere o seguinte regex:

```
padrão const = / ((ninja -) +) espada /;
```

Aqui, a intenção é criar um regex que permita o prefixo ninja- aparecer uma ou mais vezes antes da palavra espada, e queremos capturar o prefixo inteiro. Este regex requer dois conjuntos de parênteses:

- Os parênteses que definem a captura (tudo antes da string espada)
- Os parênteses que agrupam o texto ninja- para o operador +

Tudo isso funciona bem, mas resulta em mais do que a única captura pretendida por causa do conjunto interno de parênteses de agrupamento.

Para indicar que um conjunto de parênteses não deve resultar em captura, a sintaxe da expressão regular permite colocar a notação?: Imediatamente após o parêntese de abertura. Isso é conhecido como *subexpressão passiva*.

Alterando esta expressão regular para

```
padrão const = / ((?: ninja -) +) espada /;
```

faz com que apenas o conjunto externo de parênteses crie uma captura. Os parênteses internos foram convertidos em uma subexpressão passiva.

Para testar isso, dê uma olhada no código a seguir.

Listagem 10.8 Agrupando sem capturar

```
padrão const = /((?: ninja -) +) espada /;
const ninjas = "espada-ninja-ninja" .match (padrão);
```

← Usa um passivo
subexpressão

```
assert (ninjas.length === 2, "Apenas uma captura foi retornada."); assert (ninjas [1] === "ninja-ninja-",
```

```
"Combinou ambas as palavras, sem qualquer captura extra.");
```

Executando esses testes, podemos ver que a subexpressão passiva / ((?: ninja -) +) espada / evita capturas desnecessárias.

Sempre que possível em nossas expressões regulares, devemos nos esforçar para usar grupos de não captura (passivos) no lugar da captura quando a captura for desnecessária, de modo que o mecanismo de expressão tenha muito menos trabalho a fazer para lembrar e retornar as capturas. Se não precisarmos dos resultados capturados, não há necessidade de solicitá-los! O preço que pagamos é que expressões regulares já complexas podem se tornar um pouco mais enigmáticas.

Agora vamos voltar nossa atenção para outra maneira que as expressões regulares nos dão poderes ninja: usando funções com o `Fragmento` do objeto `substituir` método.

10.5 Substituindo usando funções

O `substituir` método do `Fragmento` object é um método poderoso e versátil, que vimos usado brevemente em nossa discussão sobre capturas. Quando uma expressão regular é fornecida como o primeiro parâmetro para `substituir`, irá causar uma substituição em uma partida (ou *partidas* se a `regex` for global) para o padrão em vez de em uma string fixa.

Por exemplo, digamos que desejamos substituir todos os caracteres maiúsculos em uma string por X. Poderíamos escrever o seguinte:

```
"ABCDEfg" .replace (/ [AZ] / g, "X")
```

Isso resulta em um valor de XXXXXfg. Legal.

Mas talvez o recurso mais poderoso apresentado por `substituir` é a capacidade de fornecer uma função como valor de substituição, em vez de uma string fixa.

Quando o valor de substituição (o segundo argumento) é uma função, é invocado para cada correspondência encontrada (lembre-se de que uma pesquisa global irá corresponder a todas as instâncias do padrão na string de origem) com uma lista variável de parâmetros:

- O texto completo da partida
- As capturas da correspondência, um parâmetro para cada O índice da
- correspondência dentro da string original A string de origem
-

O valor retornado da função serve como valor de substituição.

Isso fornece uma enorme margem de manobra para determinar qual deve ser a string de substituição em tempo de execução, com muitas informações sobre a natureza da correspondência ao nosso alcance. Por exemplo, na lista a seguir, usamos a função para

fornece um valor de substituição dinâmica para converter uma string com palavras separadas por traços em seu equivalente em caixa de camelo.

Listagem 10.9: Convertendo uma string tracejada em caixa de camelo

```
função superior (tudo, letra) {return letter.toUpperCase (); } assert ("border-bottom-width".replace (/ - (\w) / g,
superior)
    === "borderBottomWidth",
    "Camel envolve uma string hifenizada.");
```

← **Converte para
maiúsculas**

← **Partidas tracejadas
personagens**

Aqui, fornecemos uma regex que corresponde a qualquer caractere precedido por um traço. Uma captura na regex global identifica o caractere que foi correspondido (sem o traço). Cada vez que a função é chamada (duas vezes neste exemplo), é passada a string de correspondência completa como o primeiro argumento e a captura (apenas uma para esta regex) como o segundo argumento. Não estamos interessados no restante dos argumentos, então não os especificamos.

Na primeira vez que a função é chamada, ela é passada - b e b; e na segunda vez que é chamado, já passou - C e C. Em cada caso, a letra capturada é maiúscula e retornada como a string de substituição. Acabamos com - b substituído por B e com - C substituído por C.

Como uma regex global fará com que essa função de substituição seja executada para cada correspondência em uma string de origem, essa técnica pode até ser estendida além de fazer substituições rotineiras. Podemos usar a técnica como um meio de atravessar string, em vez de fazer o `exec()` - técnica de loop interno que vimos anteriormente neste capítulo.

Por exemplo, digamos que estamos procurando uma string de consulta e convertê-la em um formato alternativo que atenda aos nossos objetivos. Viraríamos uma string de consulta como

```
foo = 1 & foo = 2 & blah = a & blah = b & foo = 3
```

em um que se parece com este

```
foo = 1,2,3 & blah = a, b "
```

Uma solução usando expressões regulares e substituir pode resultar em algum código especialmente conciso, conforme mostrado na próxima listagem.

Listagem 10.10 Uma técnica para compactar uma string de consulta

```
função compress (fonte) {
```

```
    chaves const = {};
```

```
    source.replace (
```

```
        / ([^ = &] +) = ([^ &] *) / g,
```

```
        função (completa, chave, valor) {
```

```
            chaves [chave] =
```

```
                (teclas [tecla]? teclas [tecla] + ";": "") + valor; Retorna "";
```

```
    }
```

← **Armazena chaves localizadas**

← **Extrai informações de valor-chave**


```

);
resultado const = [];
para (deixe digitar as chaves) {
    result.push (key + "=" + keys [key]);
}
return result.join("&");
}

```

**Coleta
informação chave**

← **Une resultados com &**

```

assert (compress ("foo = 1 & foo = 2 & blah = a & blah = b & foo = 3") ===
    "foo = 1,2,3 & blah = a, b",
    "A compressão está OK!");

```

O aspecto mais interessante deste exemplo é o uso da string substituir método como um meio de percorrer uma string para obter valores, em vez de um mecanismo de pesquisa e substituição. O truque é duplo: passar uma função como o argumento do valor de substituição e, em vez de retornar um valor, usá-lo como meio de pesquisa.

O código de exemplo primeiro declara um hash chave em que armazenamos as chaves e os valores que encontramos na string de consulta de origem. Então chamamos o substituir na string de origem, passando um regex que corresponderá aos pares de valores-chave e capturará a chave e o valor. Também passamos uma função que receberá a correspondência completa, a captura de chave e a captura de valor. Esses valores capturados são armazenados no hash para referência posterior. Observe que retornamos a string vazia porque não nos importamos com quais substituições acontecem na string de origem - estamos apenas usando os efeitos colaterais em vez do resultado.

Depois de substituir retorna, declaramos uma matriz na qual agregaremos os resultados e iteraremos por meio das chaves que encontramos, adicionando cada uma à matriz. Por fim, juntamos cada um dos resultados que armazenamos na matriz usando & como delimitador e retornamos o resultado:

```

resultado const = [];
para (deixe digitar as chaves) {
    result.push (key + "=" + keys [key]);
}
return result.join("&");

```

Usando esta técnica, podemos cooptar o Fragmento do objeto substituir método como nosso próprio mecanismo de pesquisa de string. O resultado não é apenas rápido, mas também simples e eficaz. O nível de poder que essa técnica fornece, especialmente em vista da pequena quantidade de código necessária, não deve ser subestimado.

Todas essas técnicas de expressão regular podem ter um grande impacto em como escrevemos o script em nossas páginas. Vamos ver como aplicar o que você aprendeu para resolver alguns problemas comuns que podemos encontrar.

10.6 Resolvendo problemas comuns com expressões regulares

Em JavaScript, alguns idiomas tendem a ocorrer repetidamente, mas suas soluções nem sempre são óbvias. O conhecimento das expressões regulares pode definitivamente vir em nosso auxílio e, nesta seção, veremos alguns problemas comuns que podemos resolver com um regex ou dois.

10.6.1 Novas linhas correspondentes

Ao realizar uma pesquisa, às vezes é desejável que o termo ponto (.), que corresponde a qualquer caractere, exceto para nova linha, inclua também caracteres de nova linha. Implementações de expressões regulares em outras linguagens frequentemente incluem um sinalizador para tornar isso possível, mas a implementação de JavaScript não.

Vejamos algumas maneiras de contornar essa omissão no JavaScript, conforme mostrado na próxima listagem.

Listagem 10.11 Correspondência *todo* personagens, incluindo novas linhas

```

const html = "<b> Olá </b> \n <i> mundo! </i>"; assert (/.*/. exec (html) [0] === "<b>
Olá </b>",
    "Uma captura normal não trata das linhas finais.");
assert (/ [\ S \ s] */. exec (html) [0] ===
    "<b> Olá </b> \n <i> mundo! </i>",
    "Combinando tudo com um conjunto de caracteres.");
assert (/ (?.: | \ s) */. exec (html) [0] ===
    "<b> Olá </b> \n <i> mundo! </i>",
    "Usando um grupo sem captura para combinar tudo.");
  
```

Define um assunto de teste (points to the test string)

Mostra essas novas linhas não combinam (points to the first assertion failure)

Corresponde a todos usando correspondência de espaço em branco (points to the second assertion success)

Corresponde a todos usando alteração (points to the third assertion success)

Este exemplo define uma string de assunto de teste: " Olá \n <i> mundo! </i> ", contendo uma nova linha. Em seguida, tentamos várias maneiras de combinar todos os caracteres da string.

No primeiro teste, `/.*/. exec (html) [0] === "<b> Olá </b>"`, verificamos que as novas linhas não são correspondidas pelo operador. Os ninjas não serão negados, portanto, no próximo teste, vamos usar uma regex alternativa, `/[\ WL]*/`, em que definimos uma classe de personagem que corresponde a qualquer coisa que seja *não* um caractere de espaço em branco e qualquer coisa que *é* um caractere de espaço em branco. Essa união é o conjunto de todos os personagens.

Outra abordagem é feita no próximo teste:

```
/[\S\s]*/.exec(html)[0] === "<b> Olá </b> \n <i> mundo! </i>"
```

Aqui, usamos uma regex de alternância, `/ (?.: | \ s) */`, em que nós combinamos tudo com correspondência por., que é tudo menos nova linha, e tudo considerado espaço em branco, o que inclui nova linha. A união resultante é o conjunto de todos os caracteres, incluindo novas linhas. Observe o uso de uma subexpressão passiva para evitar qualquer captura não intencional. Por causa de sua simplicidade (e benefícios implícitos de velocidade), a solução fornecida por `/[\ WL]*/` geralmente é considerado ótimo.

A seguir, vamos dar um passo para ampliar nossa visão para um âmbito mundial.

10.6.2 Unicode correspondente

Freqüentemente, no uso de expressões regulares, desejamos corresponder caracteres alfanuméricos, como um seletor de ID em uma implementação de mecanismo seletor CSS. Mas assumir que os caracteres alfabéticos virão apenas do conjunto de caracteres ASCII em inglês é uma visão curta.

Expandir o conjunto para incluir caracteres Unicode às vezes é desejável, suportando explicitamente vários idiomas não cobertos pelo conjunto de caracteres alfanuméricos tradicional (consulte a lista a seguir).

Listagem 10.12 Correspondência de caracteres Unicode

```
const text = "\u5FCD\u8005\u30D1\u30EF\u30FC";
const matchAll = /\w\u0080-\uFFFF_/i;
assert(text.match(matchAll), "Nosso regex corresponde a não ASCII!");
```

Corresponde a tudo,
incluindo Unicode

Esta lista inclui toda a gama de caracteres Unicode na correspondência, criando uma classe de caracteres que inclui o `\C` termo, para corresponder a todos os caracteres de palavra "normais", mais um intervalo que abrange todo o conjunto de caracteres Unicode acima de `U + 0080`. Começar em 128 nos dá alguns caracteres ASCII altos junto com todos os caracteres Unicode no plano multilíngue básico.

Os astutos entre vocês podem notar que, adicionando toda a gama de caracteres Unicode acima `\u0080`, combinamos não apenas caracteres alfabéticos, mas também todas as pontuações Unicode e outros caracteres especiais (setas, por exemplo). Mas tudo bem, porque o objetivo do exemplo é mostrar como combinar caracteres Unicode em geral. Se você tem um intervalo específico de caracteres que deseja combinar, você pode usar a lição deste exemplo para adicionar o intervalo que deseja à classe de caracteres.

Antes de prosseguirmos em nosso exame das expressões regulares, vamos abordar mais um problema comum.

10.6.3 Caracteres de escape correspondentes

É comum que os autores das páginas usem nomes que estejam em conformidade com os identificadores do programa ao atribuir eu ia valores para os elementos da página, mas isso é apenas uma convenção; eu ia os valores podem conter caracteres diferentes de caracteres de "palavra", incluindo pontuação. Por exemplo, um desenvolvedor da web pode usar o eu ia valor formulário: atualização para um elemento.

Um desenvolvedor de biblioteca, ao escrever uma implementação para, digamos, um mecanismo seletor de CSS, gostaria de oferecer suporte a caracteres de escape. Isso permite que o usuário especifique nomes complexos que não estão em conformidade com as convenções de nomenclatura típicas. Portanto, vamos desenvolver um regex que permitirá a correspondência de caracteres de escape. Considere o seguinte código.

Listagem 10.13 Caracteres de escape correspondentes em um seletor CSS

```
const pattern = /^(\\w+)((\\.|\\$))+$/; testes const = [
  "formUpdate",
  "formulário \\ atualizar \\ seja o que for",
  "formulário \\: atualização",
  "\\ f \\ o \\ r \\ m \\ u \\ p \\ d \\ a \\ t \\ e",
  "formulário: atualização"
];
```

Configura vários
assuntos de teste.
Todos deveriam passar
mas o último,
que falha em
escapar dela
não palavra
personagem (:).

Esta expressão regular permite qualquer
sequência composta de caracteres de
palavras, uma barra invertida seguida por
qualquer caractere (mesmo um
barra invertida) ou ambos.

```

para (deixe n = 0; n < tests.length; n++) {
    assert (pattern.test (tests [n]),
            tests [n] + "é um identificador válido");
}

```

← **Passa por todos os assuntos de teste**

Essa expressão em particular funciona permitindo a correspondência de uma sequência de caracteres de palavra ou de uma barra invertida seguida por qualquer caractere.

Observe que é necessário mais trabalho para oferecer suporte total a todos os caracteres de escape. Para mais detalhes, visite <https://mathiasbynne>

10,7 Resumo

- Expressões regulares são uma ferramenta poderosa que permeia o desenvolvimento JavaScript moderno; praticamente todos os aspectos de qualquer tipo de combinação dependem de seu uso. Com um bom entendimento dos conceitos avançados de regex abordados neste capítulo, você deve se sentir confortável para lidar com qualquer parte desafiadora de código que possa se beneficiar das expressões regulares.
- Podemos criar expressões regulares com literais de expressão regular (/ teste/) e com o RegExp construtor (novo RegExp (" teste")). Os literais são preferidos quando a regex é conhecida no tempo de desenvolvimento e o construtor quando a regex é construída no tempo de execução.
- Com cada expressão regular, podemos associar cinco sinalizadores: eu torna a regex insensível a maiúsculas e minúsculas, g corresponde a todas as instâncias do padrão, m permite correspondências em várias linhas, y permite a correspondência pegajosa, enquanto você permite o uso de escapes Unicode. Sinalizadores são adicionados no final de um literal regex: / test / ig, ou como um segundo parâmetro para o RegExp construtor: novo RegExp ("teste", "i").
- Use [] (como em [abc]) para especificar um conjunto de caracteres que desejamos corresponder.
- Use ^ para significar que o padrão deve aparecer no início de uma string e \$ para significar que o padrão deve aparecer no final de uma string.
- Usar ? para especificar que um termo é opcional, + que um termo deve aparecer uma ou várias vezes e * para especificar que um termo aparece zero, uma ou várias vezes. Usar . para combinar com qualquer personagem.
-
- Podemos usar barra invertida (\) para escapar de caracteres regex especiais (como. [\$ ^). Use parênteses () para agrupar vários termos e barra vertical (|) para especificar a alternância.
- As partes de uma string que são correspondidas com sucesso aos termos podem ser referenciadas de volta com uma barra invertida seguida pelo número da captura (\ 1, \ 2, e assim por diante).
- Cada string tem acesso ao Combine , que recebe uma expressão regular e retorna uma matriz contendo toda a string correspondida junto com todas as capturas correspondidas. Também podemos usar o substituir função, que causa uma substituição em correspondências de padrão em vez de em uma string fixa.

10.8 Exercícios

- 1 Em JavaScript, as expressões regulares podem ser criadas com qual das seguintes opções?

uma Literais de expressão regular
b O embutido RegExp constructor
c The built-in Expressão regular constructor

- 2 Qual das opções a seguir é um literal de expressão regular?

a / teste/
b \ teste\
c novo RegExp ("teste");

- 3 Escolha os sinalizadores de expressão regular corretos:

a / teste / g
b g / test /
c novo RegExp ("teste", "gi");

- 4 A expressão regular / def / corresponde a qual das seguintes strings?

uma Uma das cordas d, e, f
b def
c de

- 5 A expressão regular / [^ abc]/ corresponde a qual das seguintes?

uma Uma das cordas a, b, c
b Uma das cordas d, e, f
c Corresponde à string ab

- 6 Qual das seguintes expressões regulares corresponde à string Olá?

uma / Olá/
b / Olá/
c / Olá/
d /[Olá]/

- 7 A expressão regular / (cd) + (de) * / corresponde a qual das seguintes strings?

uma CD
b de
c cdde
d cdcd
e ce
f cdcdededede

8 Em expressões regulares, podemos expressar alternativas com qual das seguintes?

a #

b E

c |

9 Na expressão regular / ([0-9]) 2 /, podemos fazer referência ao primeiro dígito correspondido com qual dos seguintes?

a 0

b 1

c \ 0

d \ 1

10 A expressão regular / ([0-5]) 6 \ 1 / corresponderá a qual dos seguintes?

a 060

b 16

c 261

d 565

11 A expressão regular / (? : ninja) - (truque)? - \ 1 / irá corresponder a qual dos seguintes mugindo?

a ninja-

b ninja-truque-ninja

c truque-truque-ninja

12 Qual é o resultado de executar " 012675 ".substituir (/ 0-5 / g," a ")?

a aaa67a

b a12675

c a1267a

11

Código

técnicas de modularização

Este capítulo cobre

- Usando o padrão de módulo
- Usando os padrões atuais para escrever código modular: AMD e CommonJS
- Trabalhando com módulos ES6

Até agora, exploramos os primitivos básicos do JavaScript, como funções, objetos, coleções e expressões regulares. Temos mais do que algumas ferramentas em nosso cinto para resolver problemas específicos com nosso código JavaScript. Mas, à medida que nossos aplicativos começam a crescer, outro conjunto de problemas, relacionados a como estruturamos e gerenciamos nosso código, começa a surgir. Repetidamente, foi provado que as bases de código grandes e monolíticas são muito mais difíceis de entender e manter do que as menores e bem organizadas. Portanto, é natural que uma maneira de melhorar a estrutura e a organização de nossos programas seja dividi-los em segmentos menores, relativamente fracamente acoplados, chamados *módulos*.

Módulos são unidades maiores de organização de nosso código do que objetos e funções; eles nos permitem dividir programas em clusters que pertencem juntos. Ao criar

módulos, devemos nos esforçar para formar abstrações consistentes e encapsular detalhes de implementação. Isso torna mais fácil raciocinar sobre nosso aplicativo, porque não nos importamos com vários detalhes frívolos ao usar a funcionalidade de nosso módulo. Além disso, ter módulos significa que podemos facilmente reutilizar a funcionalidade do módulo em diferentes partes de nossos aplicativos e até mesmo em diferentes aplicativos, acelerando significativamente nosso processo de desenvolvimento.

Como você viu anteriormente neste livro, o JavaScript é grande em variáveis globais: sempre que definimos uma variável no código da linha principal, essa variável é automaticamente considerada global e pode ser acessada de qualquer outra parte do nosso código. Isso pode não ser um problema para programas pequenos, mas à medida que nossos aplicativos começam a crescer e incluímos código de terceiros, a chance de ocorrer conflito de nomes começa a aumentar significativamente. Na maioria das outras linguagens de programação, esse problema é resolvido com namespaces (C++ e C#) ou pacotes (Java), que envolvem todos os nomes incluídos em outro nome, reduzindo significativamente os conflitos potenciais.

Até o ES6, o JavaScript não oferecia um recurso integrado de nível superior que nos permitisse agrupar variáveis relacionadas em um módulo, namespace ou pacote. Portanto, para resolver esse problema, os programadores de JavaScript desenvolveram técnicas avançadas de modularização de código que tiram proveito de construções JavaScript existentes, como objetos, funções imediatas e encerramentos. Neste capítulo, exploraremos algumas dessas técnicas.

Felizmente, é apenas uma questão de tempo até que sejamos capazes de abandonar completamente essas técnicas alternativas, porque o ES6 finalmente introduz os módulos nativos. Infelizmente, os navegadores não pegaram, então vamos explorar como os módulos devem funcionar no ES6, embora não tenhamos uma implementação de navegador nativo específica para testá-los.

Vamos começar com técnicas de modularização que podemos usar hoje.

.....

Você sabe?

Que mecanismo existente você usa para aproximar módulos em JavaScript pré-ES6?

Qual é a diferença entre o AMD e CommonJS especificações do módulo?

Usando ES6, quais duas instruções você precisaria para usar o `tryThisOut ()` função de um módulo chamado teste de dentro de outro módulo chamado porquinho da índia ?

.....

11.1 Modularizando o código em JavaScript pré-ES6

O JavaScript pré-ES6 tem apenas dois tipos de escopos: escopo global e escopo de função. Não tem algo intermediário, um namespace ou um módulo que nos permitiria agrupar certas funcionalidades. Para escrever código modular, os desenvolvedores de JavaScript são forçados a ser criativos com os recursos existentes da linguagem JavaScript.

Ao decidir quais recursos usar, devemos ter em mente que, no mínimo, cada sistema de módulo deve ser capaz de fazer o seguinte:

- *Defina uma interface* através do qual podemos acessar a funcionalidade oferecida pelo módulo.
- *Ocultar componentes internos do módulo* para que os usuários de nossos módulos não sejam sobrecarregados com uma série de detalhes de implementação sem importância. Além disso, ao ocultar os componentes internos do módulo, protegemos esses componentes internos do mundo externo, evitando, assim, modificações indesejadas que podem levar a todos os tipos de efeitos colaterais e bugs.

Nesta seção, veremos primeiro como criar módulos usando recursos JavaScript padrão que exploramos até agora no livro, recursos como objetos, fechamentos e funções imediatas. Continuaremos com essa via de modularização explorando a Asynchronous Module Definition (AMD) e o CommonJS, os dois padrões de especificação de módulo mais populares, construídos em bases ligeiramente diferentes. Você aprenderá como definir módulos usando esses padrões, bem como seus prós e contras.

Mas vamos começar com algo para o qual já definimos o cenário nos capítulos anteriores.

11.1.1 Usando objetos, fechamentos e funções imediatas para especificar módulos

Vamos voltar aos nossos requisitos mínimos de sistema de módulo, *escondendo detalhes de implementação e definindo interfaces de módulo*. Agora pense em quais recursos da linguagem JavaScript podemos aproveitar para implementar esses requisitos mínimos:

- *Escondendo o interior do módulo* — Como já sabemos, chamar uma função JavaScript cria um novo escopo no qual podemos definir variáveis que são visíveis apenas na função atual. Portanto, uma opção para ocultar as partes internas do módulo é usar funções como módulos. Desta forma, todas as variáveis de função se tornam variáveis de módulo interno que estão ocultas do mundo externo.
- *Definindo interfaces de módulo* — Implementar os componentes internos do módulo por meio de variáveis de função significa que essas variáveis só podem ser acessadas dentro do módulo. Mas se nossos módulos devem ser usados a partir de outro código, temos que ser capazes de definir uma interface limpa por meio da qual podemos expor a funcionalidade oferecida pelo módulo. Uma maneira de conseguir isso é aproveitando objetos e fechos. A ideia é que, a partir de nosso módulo de função, retornemos um objeto que representa a interface pública de nosso módulo. Esse objeto deve conter métodos oferecidos pelo módulo, métodos que irão, por meio de fechamentos, manter vivas nossas variáveis de módulo internas, mesmo depois que nossa função de módulo terminar sua execução.

Agora que fornecemos uma descrição de alto nível de como implementar módulos em JavaScript, vamos examiná-lo lentamente, passo a passo, começando com o uso de funções para ocultar o interior do módulo.

FUNÇÕES COMO MÓDULOS

Chamar uma função cria um novo escopo que podemos usar para definir variáveis que não serão visíveis de fora da função atual. Vamos dar uma olhada no seguinte snippet de código que conta o número de cliques em uma página da web:

```
(function countClicks () {
  deixe numClicks = 0;
  document.addEventListener ("click", () => {
    alerta (++ numClicks);
  });
})();
```

Define uma variável local que armazenará contagens de cliques

Sempre que um usuário clica, o contador é incrementado e o valor atual é relatado.

Neste exemplo, criamos uma função chamada `countClicks` isso cria uma variável `numClicks` e registra um manipulador de eventos de clique em todo o documento. Sempre que um clique é feito, o `numClicks` variável é incrementada e o resultado é exibido ao usuário por meio de uma caixa de alerta. Há duas coisas importantes a serem observadas aqui:

- O `numClicks` variável, interna ao `countClicks` função, é mantida ativa através do encerramento da função de tratamento de clique. A variável pode ser referenciada apenas dentro do manipulador, e *em nenhum outro lugar!* Nós protegemos o `numClicks` variável do código fora do `countClicks` função. Ao mesmo tempo, não poluímos o namespace global do nosso programa com uma variável que provavelmente não é tão importante para o resto do nosso código.
- Nosso `countClicks` função é chamada apenas neste lugar, portanto, em vez de definir uma função e, em seguida, chamá-la em uma instrução separada, usamos uma função imediata, ou um IIFE (apresentado no capítulo 3), para definir e invocar imediatamente o `countClicks` função.

Também podemos dar uma olhada no estado atual do aplicativo, com relação a como nossa variável de função interna (ou módulo) é mantida viva por meio de fechamentos, conforme mostrado na Figura 11.1.

Agora que entendemos como ocultar os detalhes do módulo interno e como os fechamentos podem manter esses detalhes internos vivos pelo tempo que for necessário, vamos passar ao nosso segundo requisito mínimo para módulos: definir interfaces de módulo.

O PADRÃO DO MÓDULO: FUNÇÕES DE AUMENTAÇÃO COMO MÓDULOS COM OBJETOS COMO INTERFACES

A interface do módulo é geralmente composta de um conjunto de variáveis e funções que nosso módulo fornece para o mundo exterior. A maneira mais fácil de criar tal interface é usar o humilde objeto JavaScript.

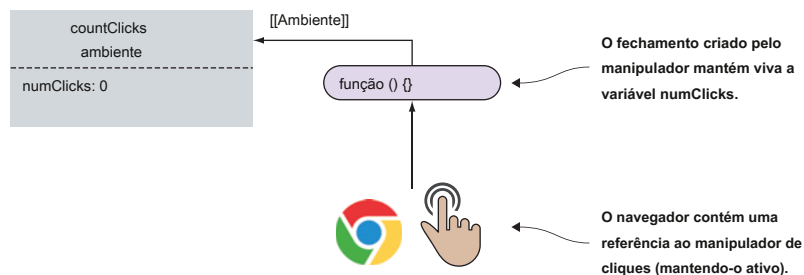
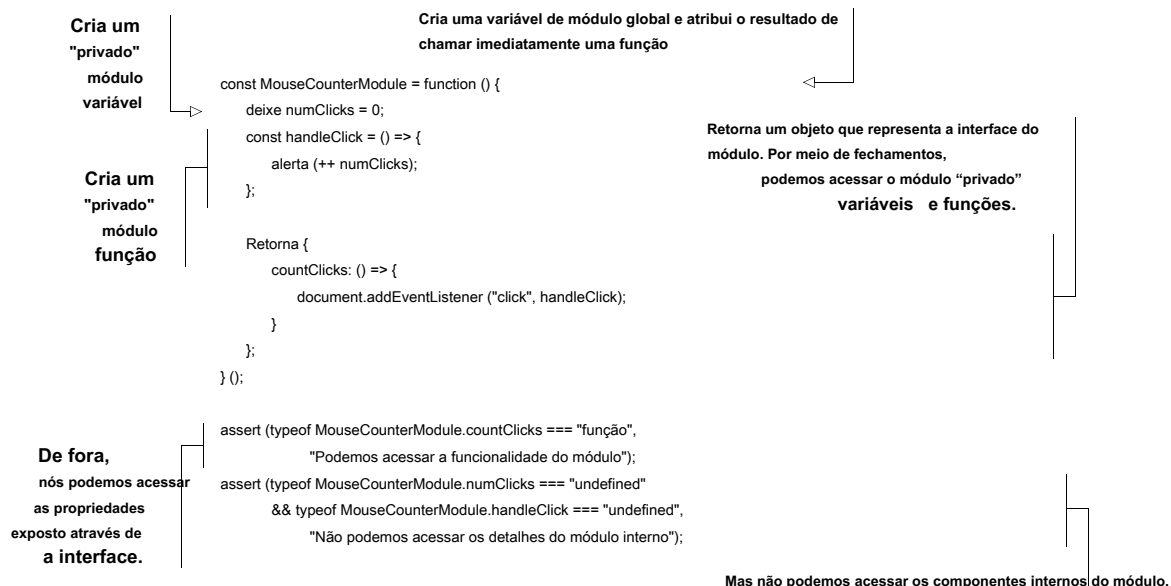


Figura 11.1 O manipulador de eventos click, por meio de fechamentos, mantém vivo o local `numClicks` variável.

Por exemplo, vamos criar uma interface para nosso módulo que conta os cliques em nossa página web, conforme mostrado na listagem a seguir.

Listagem 11.1 O padrão do módulo



Aqui, usamos uma função imediata para implementar um módulo. Dentro da função imediata, definimos nossos detalhes de implementação do módulo interno: uma variável local, `numClicks`, e uma função local, `handleClick`, que são acessíveis apenas dentro do módulo. Em seguida, criamos e retornamos imediatamente um objeto que servirá como a "interface pública" do módulo. Esta interface contém um `countClicks` método que podemos usar de fora do módulo para acessar a funcionalidade do módulo.

Ao mesmo tempo, como expusemos uma interface de módulo, os detalhes de nosso módulo interno são mantidos vivos por meio de fechamentos criados pela interface. Por exemplo, neste caso, o `countClicks` método da interface mantém variáveis do módulo interno vivas `numClicks` e `handleClick`, conforme mostrado na figura 11.2.

Por fim, armazenamos o objeto que representa a interface do módulo, retornado pela função imediata, em uma variável chamada `MouseCounterModule`, por meio do qual podemos consumir facilmente a funcionalidade do módulo, escrevendo o seguinte código:

```
MouseCounterModule.countClicks ()
```

E é basicamente isso.

Aproveitando as funções imediatas, podemos ocultar certos detalhes de implementação do módulo. Então, adicionando objetos e fechamentos à mistura, podemos especificar uma interface de módulo que expõe a funcionalidade fornecida por nosso módulo para o mundo externo.

```
const MouseCounterModule = função () {
  deixe numClicks = 0;
  const handleClick = () => {
    alerta (++ numClicks);
  };

  Retorna {
    countClicks: () => {
      document.addEventListener ("click", handleClick);
    }
  };
} ();
```

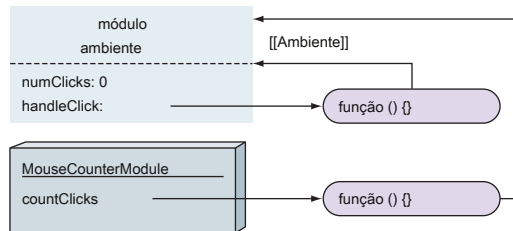


Figura 11.2 Exponha o interface pública de um módulo por meio de um objeto retornado. A implementação do módulo interno ("privado" variáveis e funções) é mantida ativa por meio de fechamentos criados por métodos de interface pública.

Este padrão de usar funções imediatas, objetos e encerramentos para criar módulos em JavaScript é chamado de *padrão de módulo*. Foi popularizado por Douglas Crockford e foi uma das primeiras formas massivamente populares de modularizar o código JavaScript.

Assim que tivermos a capacidade de definir módulos, é sempre bom poder dividi-los em vários arquivos (para gerenciá-los mais facilmente), ou ser capaz de definir funcionalidades adicionais em módulos existentes, sem modificar seu código-fonte.

Vamos ver como isso pode ser feito.

MÓDULOS DE AUMENTAÇÃO

Vamos aumentar nosso MouseCounterModule do exemplo anterior com um recurso adicional de contagem do número de rolagens do mouse, mas sem modificar o original MouseCounterModule código. Veja a lista a seguir.

Listagem 11.2 Módulos de aumento

```
const MouseCounterModule = function () {
  deixe numClicks = 0;
  const handleClick = () => {
    alerta (++ numClicks);
  };

  Retorna {
    countClicks: () => {
      document.addEventListener ("click", handleClick);
    }
  };
} ();

(função (módulo) {
```

O original
MouseCounterModule

Invoca imediatamente uma função que
aceita o módulo que queremos estender
como argumento

Passa no módulo como um argumento

```

deixe numScrolls = 0;
const handleScroll = () => {
  alerta (++ numScrolls);
}

module.countScrolls = () => {
  document.addEventListener ("wheel", handleScroll);
};
})(MouseCounterModule);

assert (typeof MouseCounterModule.countClicks === "função",
        "Podemos acessar a funcionalidade do módulo inicial");

assert (typeof MouseCounterModule.countScrolls === "função",
        "Podemos acessar a funcionalidade do módulo aumentado");

```

Define novo variáveis privadas e funções

Estende o interface do módulo

Ao aumentar um módulo, geralmente seguimos um procedimento semelhante à criação de um novo módulo. Chamamos imediatamente uma função, mas desta vez, passamos a ela o módulo que queremos estender como argumento:

```

(função (módulo) {
  ...
  módulo de retorno;
})(MouseCounterModule);

```

Dentro da função, realizamos nosso trabalho e criamos todas as variáveis e funções privadas necessárias. Neste caso, definimos uma variável privada e uma função privada para contar e relatar o número de pergaminhos:

```

deixe numScrolls = 0;
const handleScroll = () => {
  alerta (++ numScrolls);
}

```

Finalmente, estendemos nosso módulo, disponível através da função imediata módulo parâmetro, da mesma forma que estenderíamos qualquer outro objeto:

```

module.countScrolls = () => {
  document.addEventListener ("wheel", handleScroll);
};

```

Depois de realizar esta operação simples, nosso MouseCounterModule também pode countScrolls.

Nossa interface de módulo público agora tem dois métodos e podemos usar o módulo das seguintes maneiras:

```

MouseCounterModule.countClicks ();
MouseCounterModule.countScrolls ();

```

Um método que faz parte da interface do módulo desde o início

Um novo método de módulo que adicionamos ao estender o módulo

Como já mencionamos, estendemos o módulo de uma maneira semelhante à criação de um novo módulo, por meio de uma função imediatamente chamada que estende o módulo. Isso tem alguns efeitos colaterais interessantes em termos de encerramentos, então vamos dar uma olhada mais de perto no estado do aplicativo depois de aumentar o módulo, conforme mostrado na Figura 11.3.

Se você olhar com atenção, a figura 11.3 também mostra uma das deficiências do padrão de módulo: a incapacidade de compartilhar variáveis de módulo privadas entre extensões de módulo. Por exemplo, o `countClicks` função mantém um fechamento em torno do `numClicks` e `handleClick` variáveis, e poderíamos acessar essas partes internas do módulo privado através a `countClicks` método.

```
const MouseCounterModule = função () {
  deixe numClicks = 0;
  const handleClick = () => {
    alerta (++ numClicks);
  };
  Retorna {
    countClicks: () => {
      document.addEventListener ("click", handleClick);
    }
  };
} ();

( função (módulo){
  deixe numScrolls = 0;
  const handleScroll = () => {
    alerta (++ numScrolls);
  }

  module.countScrolls = () => {
    document.addEventListener ("roda", handleClick);
  };
}) (MouseCounterModule);
```

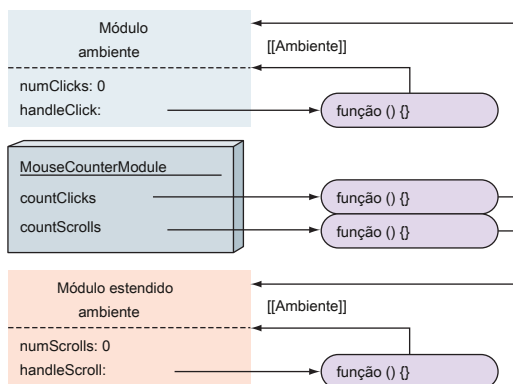


Figura 11.3 Ao aumentar um módulo, estendemos sua interface externa com novas funcionalidades, geralmente passando o módulo para outra função imediata. Neste exemplo, adicionamos a capacidade de `countScrolls` para nosso **MouseCounterModule**. Observe que duas funções separadas são definidas em ambientes diferentes e não podem acessar as variáveis internas uma da outra.

Infelizmente, nossa extensão, o `countScrolls` função, é criado em um escopo completamente separado, com um conjunto completamente novo de variáveis privadas: `numScrolls` e `handleScroll`. O `countScrolls` função cria um fechamento apenas em torno `numScrolls` e `handleScroll` variáveis e, portanto, não pode acessar o `numClicks` e `handleClick` variáveis.

NOTA As extensões de módulo, quando executadas por meio de funções imediatas separadas, não podem compartilhar internos de módulo privados, porque cada chamada de função cria um novo escopo. Embora isso seja uma deficiência, não é um obstáculo e ainda podemos usar o padrão de módulo para manter nossos aplicativos JavaScript modulares.

Observe que, no padrão de módulo, os módulos são objetos como qualquer outro e podemos estendê-los da maneira que acharmos apropriada. Por exemplo, podemos adicionar novas funcionalidades estendendo o objeto de módulo com novas propriedades:

```
MouseCounterModule.newMethod = () => {...}
```

Também podemos usar o mesmo princípio para criar submódulos facilmente:

```
MouseCounterModule.newSubmodule = () => {  
  Retorna {...};  
} ();
```

Observe que todas essas abordagens sofrem da mesma deficiência fundamental do padrão do módulo: as extensões subsequentes do módulo não podem acessar os detalhes internos do módulo definidos anteriormente.

Infelizmente, existem mais problemas com o padrão do módulo. Quando começamos a construir aplicativos modulares, os próprios módulos muitas vezes dependem de outros módulos para sua funcionalidade. Infelizmente, o padrão do módulo não cobre o gerenciamento dessas dependências. Nós, como desenvolvedores, temos que cuidar da ordem de dependência certa para que nosso código de módulo tenha tudo de que precisa para ser executado. Embora isso não seja um problema em aplicativos pequenos e médios, pode apresentar problemas sérios em aplicativos grandes que usam muitos módulos interdependentes.

Para lidar com esses problemas, surgiram alguns padrões concorrentes, a saber, Asynchronous Module Definition (AMD) e CommonJS.

11.1.2 *Modularizando aplicativos JavaScript com AMD e CommonJS*

AMD e CommonJS são padrões concorrentes de especificação de módulo que nos permitem especificar módulos JavaScript. Além de algumas diferenças na sintaxe e filosofia, a principal diferença é que o AMD foi projetado explicitamente com o navegador em mente, enquanto o CommonJS foi projetado para um ambiente JavaScript de uso geral (como servidores, com Node.js), sem estar vinculado às limitações de o navegador. Esta seção fornece uma visão geral relativamente curta das especificações do módulo; configurando-os e incluindo

em seus projetos está além do escopo deste livro. Para mais informações, recomendamos *Design de aplicativo JavaScript* por Nicolas G. Bevacqua (Manning, 2015).

AMD

AMD cresceu a partir do kit de ferramentas Dojo (<https://dojotoolkit.org/>), um dos kits de ferramentas JavaScript populares para construir aplicativos da Web do lado do cliente. A AMD nos permite especificar facilmente os módulos e suas dependências. Ao mesmo tempo, foi desenvolvido do zero para o navegador. Atualmente, a implementação AMD mais popular é RequireJS (<http://requirejs.org/>)

Vamos ver um exemplo de definição de um pequeno módulo que tem uma dependência do jQuery.

Listagem 11.3 Usando AMD para especificar um módulo dependente de jQuery

```
definir(' MouseCounterModule ', [' jQuery '], $ => {
  deixe numClicks = 0;
  const handleClick = () => {
    alerta (++ numClicks);
  };

  Retorna {
    countClicks: () => {
      $ (document) .on ("click", handleClick);
    }
  };
});
```

← Usa a função define para especificar um módulo, suas dependências e a função de fábrica do módulo que criará o módulo

← A interface pública do nosso módulo

AMD fornece uma função chamada definir que aceita o seguinte:

- O ID do módulo recém-criado. Este ID pode ser usado posteriormente para solicitar o módulo de outras partes do nosso sistema.
- Uma lista de IDs de módulo dos quais nosso módulo atual depende (os módulos necessários).
- Uma função de fábrica que irá inicializar o módulo e que aceita os módulos necessários como argumentos.

Neste exemplo, usamos definir função para criar um módulo com um ID MouseCounterModule isso depende do jQuery. Por causa dessa dependência, a AMD primeiro solicita o módulo jQuery, o que pode levar algum tempo se o arquivo tiver que ser solicitado de um servidor. Esta ação é realizada de forma assíncrona, a fim de evitar o bloqueio. Depois que todas as dependências foram baixadas e avaliadas, a função de fábrica do módulo é chamada com um argumento para cada módulo solicitado. Nesse caso, haverá um argumento, porque nosso novo módulo requer apenas jQuery. Dentro da função de fábrica, criamos nosso módulo da mesma forma que faríamos com o padrão de módulo padrão: retornando um objeto que expõe a interface pública do módulo.

Como você pode ver, a AMD oferece vários benefícios interessantes, como estes:

- Resolução automática de dependências, para que não tenhamos que pensar na ordem em que incluímos nossos módulos.
- Os módulos podem ser carregados de forma assíncrona, evitando assim o bloqueio. Vários
- módulos podem ser definidos em um arquivo.

Agora que você tem uma ideia básica de como o AMD funciona, vamos dar uma olhada em outro padrão de definição de módulo muito popular.

COMMONJS

Enquanto o AMD foi desenvolvido explicitamente para o navegador, CommonJS é uma especificação de módulo projetada para um ambiente JavaScript de propósito geral. Atualmente, ele tem o maior número de seguidores na comunidade Node.js.

CommonJS usa módulos baseados em arquivo, portanto, podemos especificar um módulo por arquivo. Para cada módulo, CommonJS expõe uma variável, `módulo`, com uma propriedade, exportações, que podemos facilmente estender com propriedades adicionais. No final, o conteúdo de `module.exports` é exposta como a interface pública do módulo.

Se quisermos usar um módulo de outras partes do aplicativo, podemos solicitá-lo. O arquivo será carregado de forma síncrona e teremos acesso à sua interface pública. Esta é a razão pela qual CommonJS é muito mais popular no servidor, onde a busca do módulo é relativamente rápida porque requer apenas uma leitura do sistema de arquivos, do que no cliente, onde o módulo deve ser baixado de um servidor remoto e onde sincronizado carregar geralmente significa bloqueio.

Vejamos um exemplo que define nossa recorrência `MouseCounterModule`, desta vez em CommonJS.

Listagem 11.4 Usando CommonJS para definir um módulo

```
//MouseCounterModule.js
const $ = require("jQuery"); deixe numClicks = 0;

const handleClick = () => {
  alerta(++ numClicks);
};

module.exports = {
  countClicks: () => {
    $(document).on("click", handleClick);
  }
};
```

← Requer sincronicamente
um módulo jQuery

← Modifica a propriedade `module.exports` para especificar
a interface pública de um módulo

Para incluir nosso módulo em um arquivo diferente, podemos escrever isto:

```
const MouseCounterModule = require("MouseCounterModule.js"); MouseCounterModule.countClicks ();
```

Veja como isso é simples?

Como a filosofia do CommonJS determina um módulo por arquivo, qualquer código que colocarmos em um módulo de arquivo fará parte desse módulo. Portanto, não há necessidade de agrupar variáveis em funções imediatas. Todas as variáveis definidas em um módulo estão seguramente contidas no escopo do módulo atual e não vazam para o escopo global. Por exemplo, todas as três variáveis do nosso módulo (\$, numClicks, e

handleClick) têm escopo de módulo, embora sejam definidos no código de nível superior (fora de todas as funções e blocos), o que os tornaria tecnicamente variáveis globais em arquivos JavaScript padrão.

Mais uma vez, é importante observar que apenas as variáveis e funções expostas por meio do `module.exports` objeto estão disponíveis fora do módulo. O procedimento é semelhante ao padrão de módulo, só que em vez de retornar um objeto completamente novo, o ambiente já fornece um que podemos estender com nossos métodos e propriedades de interface.

O CommonJS tem algumas vantagens:

- Possui sintaxe simples. Precisamos especificar apenas o `module.exports` propriedades, enquanto o resto do código do módulo permanece praticamente o mesmo, como se estivéssemos escrevendo JavaScript padrão. Requerer módulos também é simples; nós apenas usamos o `require` função.
- CommonJS é o formato de módulo padrão para Node.js, portanto, temos acesso a milhares de pacotes que estão disponíveis por meio do npm, o gerenciador de pacotes do nó.

A maior desvantagem do CommonJS é que ele não foi criado explicitamente com o navegador em mente. Em JavaScript no navegador, não há suporte para o módulo variável e o exportar propriedade; temos que empacotar nossos módulos CommonJS em um formato legível por navegador. Podemos conseguir isso com o Browserify (<http://browserify.org/>) ou RequireJS (<http://requirejs.org/docs/commonjs.html>)

Ter dois padrões concorrentes para especificar módulos, AMD e CommonJS, levou a uma daquelas situações em que as pessoas tendem a se dividir em dois, às vezes até mesmo campos opostos. Se você trabalha em projetos relativamente fechados, isso pode não ser um problema; você escolhe o padrão que melhor se adapta a você. Podem surgir problemas, no entanto, quando precisamos reutilizar o código do campo oposto e somos forçados a saltar por todos os tipos de obstáculos. Uma solução é usar a definição de módulo universal, ou UMD (<https://github.com/umdjs/umd>), um padrão com uma sintaxe um tanto complicada que permite que o mesmo arquivo seja usado por AMD e CommonJS. Isso está além do escopo deste livro, mas se você estiver interessado, muitos recursos de qualidade estão disponíveis online.

Felizmente, o comitê ECMAScript reconheceu a necessidade de uma sintaxe de módulo unificada com suporte em todos os ambientes JavaScript, então o ES6 define um novo padrão de módulo que deve finalmente colocar essas diferenças de lado.

11.2 Módulos ES6

Os módulos ES6 são projetados para combinar as vantagens de CommonJS e AMD:

- Semelhante ao CommonJS, os módulos ES6 têm uma sintaxe relativamente simples e os módulos ES6 são baseados em arquivo (um módulo por arquivo).
- Semelhante ao AMD, os módulos ES6 fornecem suporte para carregamento de módulo assíncrono.



NOTA Os módulos integrados fazem parte do padrão ES6. Como você verá em breve, a sintaxe do módulo ES6 inclui semântica e palavras-chave adicionais (como o exportar e importar palavras-chave) que não são compatíveis com os navegadores atuais. Se quisermos usar módulos hoje, temos que transpilar nosso código de módulo com Traceur (<https://github.com/google/traceur-compiler>), Babel (<http://babeljs.io/>) ou TypeScript (www.typescriptlang.org/) Também podemos usar a biblioteca SystemJS (<https://github.com/systemjs/systemjs>), que fornece suporte para carregar todos os padrões de módulo disponíveis atualmente: AMD, CommonJS e até módulos ES6. Você pode encontrar instruções sobre como usar SystemJS no repositório do projeto (<https://github.com/systemjs/systemjs>)

A ideia principal por trás dos módulos ES6 é que apenas os identificadores explicitamente exportados de um módulo são acessíveis de fora desse módulo. Todos os outros identificadores, mesmo aqueles definidos no escopo de nível superior (o que seria o escopo global em JavaScript padrão), são acessíveis apenas de dentro do módulo. Isso foi inspirado por CommonJS.

Para fornecer essa funcionalidade, o ES6 apresenta duas novas palavras-chave:

- exportar —Para tornar certos identificadores disponíveis de fora do módulo
- importar —Para importar identificadores de módulo exportados

A sintaxe para exportar e importar a funcionalidade do módulo é simples, mas tem muitas nuances sutis que exploraremos lentamente, passo a passo.

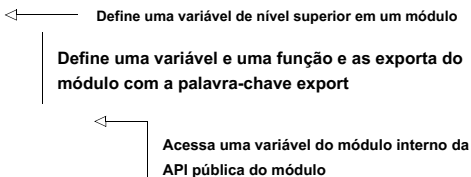
11.2.1 Funcionalidade de exportação e importação

Vamos começar com um exemplo simples que mostra como exportar funcionalidade de um módulo e importá-lo para outro.

Listagem 11.5 Exportando de um módulo Ninja.js

```
const ninja = "Yoshi";
export const message = "Olá";

função de exportação sayHiToNinja () {
  mensagem de retorno + "" + ninja;
}
```



Primeiro definimos uma variável, `ninja`, uma variável de módulo que estará acessível apenas dentro deste módulo, embora seja colocada no código de nível superior (o que a tornaria uma variável global no código pré-ES6).

Em seguida, definimos outra variável de nível superior, `mensagem`, que tornamos acessível de fora do módulo usando o novo exportar palavra-chave. Por fim, também criamos e exportamos o diga `HiToNinja` função.

E é isso! Esta é a sintaxe mínima que precisamos saber para definir nossos próprios módulos. Não precisamos usar funções imediatas ou lembrar de qualquer sintaxe esotérica para exportar a funcionalidade de um módulo. Escrevemos nosso código como escreveríamos código JavaScript padrão, com a única diferença de que prefixamos alguns dos identificadores (como variáveis, funções ou classes) com um exportar palavra-chave.

Antes de aprender a importar essa funcionalidade exportada, vamos dar uma olhada em uma forma alternativa de exportar identificadores: Listamos tudo o que queremos exportar no final do módulo, conforme mostrado na lista a seguir.

Listagem 11.6 Exportando no final de um módulo

```
const ninja = "Yoshi"; const mensagem =
"Olá";
```

Especifica todos os identificadores de módulo

```
function sayHiToNinja () {
  mensagem de retorno + "" + ninja;
}
```

Exporta alguns dos
identificadores de módulo

```
exportar {mensagem, digaHiToNinja};
```

Esta forma de exportar identificadores de módulo tem alguma semelhança com o padrão do módulo, pois uma função imediata retorna um objeto que representa a interface pública do nosso módulo, e especialmente para CommonJS, conforme expandimos o `module.exports` objeto com a interface do módulo público.

Independentemente de como exportamos os identificadores de um determinado módulo, se precisarmos importá-los para outro módulo, temos que usar o importar palavra-chave, como no exemplo a seguir.

Listagem 11.7 Importando do módulo `Ninja.js`

```
importar {mensagem, digaHiToNinja} de "Ninja.js";
```

Usa a palavra-chave `import` para importar uma
ligação de identificador de um módulo

```
assert (mensagem === "Olá",
  "Podemos acessar a variável importada"); assert (sayHiToNinja () === "Olá
Yoshi",
  "Podemos dizer olá ao Yoshi de fora do módulo");
```

Agora podemos acessar a variável
importada e chamar a função
importada.

```
assert (typeof ninja === "undefined",
  "Mas não podemos acessar Yoshi diretamente");
```

Não podemos acessar variáveis de
módulo não exportadas diretamente.

Usamos o novo importar palavra-chave para importar uma variável, mensagem e uma função, diga HiToNinja de ninja módulo:

```
importar {mensagem, digaHiToNinja} de "Ninja.js";
```

Fazendo isso, ganhamos acesso a esses dois identificadores definidos no ninja módulo. Finalmente, podemos testar se podemos acessar o mensagem variável e chame o diga HiToNinja função:

```
assert (mensagem === "Olá",
        "Podemos acessar a variável importada"); assert (sayHiToNinja () === "Olá
Yoshi",
        "Podemos dizer olá ao Yoshi de fora do módulo");
```

O que não podemos fazer é acessar as variáveis não exportadas e não importadas. Por exemplo, não podemos acessar o ninja variável porque não está marcada com exportar:

```
assert (typeof ninja === "undefined",
        "Mas não podemos acessar Yoshi diretamente");
```

Com os módulos, estamos finalmente um pouco mais seguros do uso indevido de variáveis globais. Tudo o que não marcamos explicitamente para exportação fica bem isolado dentro de um módulo.

Neste exemplo, usamos um *exportação nomeada*, o que nos permite exportar vários identificadores de um módulo (como fizemos com mensagem e digaHiToNinja). Como podemos exportar um grande número de identificadores, listar todos eles em uma instrução de importação pode ser entediante. Portanto, uma notação abreviada nos permite trazer todos os identificadores exportados de um módulo, conforme mostrado na lista a seguir.

Listagem 11.8 Importando todas as exportações nomeadas do módulo Ninja.js

```
importar * como ninjaModule de "Ninja.js";
```

```
assert (ninjaModule.message === "Olá",
        "Podemos acessar a variável importada");
assert (ninjaModule.sayHiToNinja () === "Olá Yoshi",
        "Podemos dizer olá ao Yoshi de fora do módulo");
```

```
assert (typeof ninjaModule.ninja === "undefined",
        "Mas não podemos acessar Yoshi diretamente");
```

Usa * notação para importar todos os identificadores exportados

Refere-se às exportações nomeadas por meio de notação de propriedade

Ainda não conseguimos acessar identificadores não exportados.

Como mostra a listagem 11.8, para importar todos os identificadores exportados de um módulo, usamos o importar

* notação em combinação com um identificador que usaremos para se referir a todo o módulo (neste caso, o ninjaModule identificador). Depois de fazer isso, podemos acessar os identificadores exportados por meio da notação de propriedade; por exemplo, ninjaModule.message, ninjaModule.sayHiToNinja. Observe que ainda não podemos acessar variáveis de nível superior que não foram exportadas, como é o caso com o ninja variável.

EXPORTAÇÕES PADRÃO

Freqüentemente, não queremos exportar um conjunto de identificadores relacionados de um módulo, mas sim representar o módulo inteiro por meio de uma única exportação. Uma situação bastante comum em que isso ocorre é quando nossos módulos contêm uma única classe, como na listagem a seguir.

Listagem 11.9 Uma exportação padrão de Ninja.js

```
exportação padrão class Ninja {
  construtor (nome) {
    this.name = nome;
  }
}

função de exportação compareNinjas (ninja1, ninja2) {
  return ninja1.name === ninja2.name;
}
```

← Usa as palavras-chave padrão de exportação para especificar a ligação do módulo padrão

← Ainda podemos usar exportações nomeadas junto com a exportação padrão.

Aqui nós adicionamos o predefinição palavra-chave após o exportar palavra-chave, que especifica a ligação padrão para este módulo. Neste caso, a ligação padrão para este módulo é a classe chamada Ninja. Mesmo que tenhamos especificado uma ligação padrão, ainda podemos usar exportações nomeadas para exportar identificadores adicionais, como fizemos com o compareNinjas função.

Agora, podemos usar uma sintaxe simplificada para importar funcionalidades do Ninja.js, conforme mostrado na lista a seguir.

Listagem 11.10 Importando uma exportação padrão

```
Ao importar uma exportação padrão, não há
precisamos de colchetes, e podemos usar qualquer
nome que queremos.

importar ImportedNinja de "Ninja.js";
importar {compareNinjas} de "Ninja.js";

const ninja1 = novo ImportedNinja ("Yoshi"); const ninja2 = novo ImportedNinja
("Hattori");

afirmar (ninja1! == indefinido
  && ninja2! == undefined, "Podemos criar alguns Ninjas");

assert (! compareNinjas (ninja1, ninja2),
  "Podemos comparar ninjas");
```

← Ainda podemos importar exportações nomeadas.

← Também podemos acessar as exportações nomeadas.

Cria um casal de ninjas, e testa que eles existir

Começamos este exemplo importando uma exportação padrão. Para isso, usamos uma sintaxe de importação menos confusa, eliminando as chaves que são obrigatórias para importar exportações nomeadas. Além disso, observe que podemos escolher um nome arbitrário para se referir à exportação padrão; não estamos obrigados a usar o que usamos na exportação. Neste exemplo,

ImportedNinja refere-se a Ninja classe definida no arquivo Ninja.js.

Continuamos o exemplo importando uma exportação nomeada, como nos exemplos anteriores, apenas para ilustrar que podemos ter uma exportação padrão e uma série de exportações nomeadas em um único módulo. Por fim, instanciamos alguns ninjas objetos e chamar o `compareNinjas` função, para confirmar que todas as importações funcionam como deveriam.

Nesse caso, ambas as importações são feitas a partir do mesmo arquivo. ES6 oferece uma sintaxe abreviada:

```
import ImportedNinja, {compareNinjas} de "Ninja.js";
```

Aqui, usamos o operador vírgula para importar as exportações padrão e nomeadas do arquivo `Ninja.js`, em uma única instrução.

RENAMING EXPORTS AND IMPORTS

Se necessário, também podemos renomear exportações e importações. Vamos começar renomeando as exportações, conforme mostrado no código a seguir (os comentários indicam em qual arquivo o código está localizado):

```
// ***** Greetings.js ***** /
function sayHi () {
  retornar "Olá";
}
```

Define uma função
chamado `sayHi`

```
assert (typeof sayHi === "função"
  && typeof sayHello === "undefined",
  "Dentro do módulo podemos acessar apenas sayHi");
```

Testa se podemos acessar apenas
a função `sayHi`, mas não o alias!

```
export { diga olá como diga olá }
```

Fornece um alias de identificador
com a palavra-chave `as`

```
// ***** main.js ***** /
import {sayHello} de "Greetings.js";
```

```
assert (typeof sayHi === "undefined"
  && typeof sayHello === "função",
  "Ao importar, só podemos acessar o alias");
```

Ao importar,
apenas dizer `olá`
alias está disponível.

No exemplo anterior, definimos uma função chamada `diga oi`, e testamos se podemos acessar a função apenas embora o `diga oi` identificador, e não através do `diga olá` alias que fornecemos no final do módulo por meio do `como` palavra-chave:

```
exportar {sayHi as sayHello}
```

Podemos realizar uma renomeação de exportação apenas neste formulário de exportação, e não prefixando a variável ou declaração de função com o `exportar` palavra-chave.

Então, quando realizamos uma importação da exportação renomeada, referenciamos a importação por meio do alias fornecido:

```
import {sayHello} de "Greetings.js";
```

Finalmente, testamos se temos acesso ao identificador com alias, mas não ao original:

```
assert (typeof sayHi === "undefined"
  && typeof sayHello === "função",
  "Ao importar, só podemos acessar o alias");
```

A situação é semelhante ao renomear importações, conforme mostrado no seguinte segmento de código:

<pre> / ***** Hello.js ***** / função de exportação greet () { retornar "Olá"; }</pre>	Exporta uma função com o nome greet do módulo Hello.js.
<pre> / ***** Salute.js ***** / função de exportação greet () { retornar "Saudação"; }</pre>	Exporta uma função com o mesmo nome saudação de Salute.js
<pre> / ***** main.js ***** / import { cumprimente dizendo olá } de "Hello.js"; import { cumprimentar como saudação } de "Salute.js"; assert (typeof greet === "undefined", "Não podemos acessar saudar"); assert (sayHello () === "Hello" && salute () === "Salute", "Podemos acessar identificadores com alias");</pre>	Usa a palavra-chave as para importações de alias, evitando conflitos de nome Não podemos acessar o nome da função original. Mas podemos acessar os aliases.

Da mesma forma que exportar identificadores, também podemos usar o como palavra-chave para criar aliases ao importar identificadores de outros módulos. Isso é útil quando precisamos fornecer um nome melhor que seja mais adequado ao contexto atual, ou quando queremos evitar conflitos de nomenclatura, como é o caso neste pequeno exemplo.

Com isso, concluímos nossa exploração da sintaxe dos módulos ES6, que é recapitulada na tabela 11.1.

Tabela 11.1 Visão geral da sintaxe do módulo ES6

Código	Significado
<pre>export const ninja = "Yoshi"; função de exportação compare () {} exportar classe Ninja {}</pre>	Exporte uma variável nomeada. Exporte uma função nomeada. Exporte uma classe nomeada.
<pre>exportar a classe padrão Ninja {} exportar a função padrão Ninja () {}</pre>	Exporte a exportação de classe padrão. Exporte a exportação de função padrão.
<pre>const ninja = "Yoshi"; função compare () {}; exportar {ninja, comparar}; exportar {ninja como samurai, compare};</pre>	Exporte variáveis existentes. Exporte uma variável por meio de um novo nome.
<pre>importar Ninja de "Ninja.js"; importar {ninja, Ninja} de "Ninja.js";</pre>	Importe uma exportação padrão. Importar exportações nomeadas.
<pre>importar * como Ninja de "Ninja.js"; importar {ninja como iNinja} de "Ninja.js";</pre>	Importe todas as exportações nomeadas de um módulo. Importe uma exportação nomeada por meio de um novo nome.

11.3 Resumo

- Bases de código grandes e monolíticas têm muito mais probabilidade de serem difíceis de entender e manter do que as menores e bem organizadas. Uma forma de melhorar a estrutura e a organização de nossos programas é dividi-los em segmentos ou módulos menores e relativamente fracamente acoplados.
- Módulos são unidades maiores de código de organização do que objetos e funções e nos permitem dividir programas em clusters que pertencem uns aos outros.
- Em geral, os módulos promovem a compreensibilidade, facilitam a manutenção e melhoram a capacidade de reutilização do código.
- O JavaScript pré-ES6 não tem módulos embutidos e os desenvolvedores precisam ser criativos com os recursos de linguagem existentes para permitir a modularização do código. Uma das maneiras mais populares de criar módulos é combinando funções imediatamente invocadas com fechamentos.
 - As funções imediatas são usadas porque criam um novo escopo para definir variáveis de módulo que não são visíveis de fora desse escopo.
 - Os fechamentos são usados porque nos permitem manter as variáveis do módulo vivas. O padrão mais popular é o padrão de módulo, que geralmente combina uma função imediata com o retorno de um novo objeto que representa a interface pública do módulo.
- Além do padrão de módulo, existem dois padrões de módulo populares: Definição de Módulo Assíncrono, projetado para habilitar módulos no navegador; e CommonJS, que é mais popular em JavaScript do lado do servidor.
 - A AMD pode resolver dependências automaticamente e os módulos são carregados de forma assíncrona, evitando assim o bloqueio.
 - CommonJS tem uma sintaxe simples, carrega módulos de forma síncrona (e, portanto, é mais apropriado para o servidor) e tem muitos pacotes disponíveis através do gerenciador de pacotes do nó (npm).
- Os módulos ES6 são projetados para levar em consideração os recursos do AMD e CommonJS. Esses módulos têm uma sintaxe simples influenciada pelo CommonJS e fornecem carregamento de módulo assíncrono como no AMD.
 - Módulos ES6 são baseados em arquivo, um módulo por arquivo.
 - Exportamos identificadores para que possam ser referenciados por outros módulos usando o novo exportar palavra-chave.
 - Importamos identificadores exportados de outros módulos usando o importar palavra-chave.
 - Um módulo pode ter um único predefinição exportação, que usamos se quisermos representar todo o módulo por meio de uma única exportação.
 - Tanto as importações quanto as exportações podem ser renomeadas com o como palavra-chave.

11.4 Exercícios

- 1 Qual mecanismo habilita variáveis de módulo privadas no padrão de módulo?

uma Protótipos

b Fechamentos

c Promessas

- 2 No código a seguir que usa módulos ES6, quais identificadores podem ser acessados se o módulo for importado?

```
const spy = "Yagyu"; comando de
função () {
    retorno geral + "ordena que você faça guerra!";
}
export const general = "Minamoto";
```

uma espião

b comando

c em geral

- 3 No código a seguir que usa módulos ES6, quais identificadores podem ser acessados quando o módulo é importado?

```
const ninja = "Yagyu"; comando de função
() {
    retorno geral + "ordena que você faça guerra!";
}
const general = "Minamoto";

exportar {ninja como espião};
```

uma espião

b comando

c em geral

d ninja

- 4 Quais das seguintes importações são permitidas?

```
// Arquivo: staff.js
const ninja = "Yagyu"; comando de função
() {
    retorno geral + "ordena que você faça guerra!";
}
const general = "Minamoto";

exportar {ninja como espião};
```

uma import {ninja, spy, general} de "staff.js"

b import * as Personnel de "staff.js"

c importar {spy} de "staff.js"

- 5 Se tivermos o seguinte código de módulo, qual instrução importará o Ninja aula?

```
//Ninja.js  
exportar classe padrão Ninja {  
  skulk () {return "skulking"; }  
}
```

- uma importar Ninja de "Ninja.js"
- b importar * como Ninja de "Ninja.js"
- c importar * de "Ninja.js"

Reconhecimento de navegador

Nomo que exploramos os fundamentos da linguagem JavaScript, passaremos para os navegadores, o ambiente no qual a maioria dos aplicativos JavaScript é executada.

No capítulo 12, examinaremos mais de perto o DOM, explorando técnicas eficientes para modificá-lo e obter aplicativos da Web rápidos e altamente dinâmicos.

No capítulo 13, você aprenderá sobre eventos, com foco especial no loop de eventos e sua influência no desempenho percebido do aplicativo da web.

Finalmente, o livro conclui com um tópico não tão agradável, mas necessário: desenvolvimento de crossbrowser. Embora a situação tenha melhorado muito nos últimos anos, ainda não podemos presumir que nosso código funcionará da mesma maneira em todos os navegadores disponíveis. Portanto, o capítulo 14 apresenta estratégias para o desenvolvimento de aplicativos da web entre navegadores.

12

Trabalhando o DOM

Este capítulo cobre

- Inserindo HTML no DOM
- Noções básicas sobre atributos e propriedades DOM
- Descobrindo estilos computados
- Lidando com a alteração do layout

Até agora, você aprendeu principalmente sobre a linguagem JavaScript e, embora haja muitas nuances no JavaScript puro, o desenvolvimento de aplicativos da web definitivamente não fica mais fácil quando colocamos o Document Object Model (DOM) do navegador na mistura. Um dos principais meios para alcançar aplicativos da web altamente dinâmicos que respondem às ações do usuário é modificar o DOM. Mas se abríssemos uma biblioteca JavaScript, você notaria o comprimento e a complexidade do código por trás de operações simples do DOM. Mesmo operações presumivelmente simples, como `cloneNode` e `removeChild` têm implementações relativamente complexas.

Isso levanta duas questões:

- Por que esse código é tão complexo?
- Por que você precisa entender como funciona se a biblioteca vai cuidar disso para você?

O motivo mais convincente é *atuação*. Entender como a modificação DOM funciona em bibliotecas pode permitir que você escreva um código melhor e mais rápido que usa a biblioteca ou, alternativamente, permite que você use essas técnicas em seu próprio código.

Portanto, começaremos este capítulo vendo como criar novas partes de nossas páginas, sob demanda, injetando HTML arbitrário. Continuaremos examinando todos os enigmas que os navegadores nos lançam com respeito às propriedades e atributos dos elementos, e descobriremos por que os resultados nem sempre são exatamente os que esperamos.

O mesmo vale para Cascading Style Sheets (CSS) e o estilo dos elementos. Muitas das dificuldades que encontraremos ao construir um aplicativo da web dinâmico resultam das complicações de configuração e obtenção de estilo de elemento. Este livro não pode cobrir tudo o que se sabe sobre como lidar com o estilo dos elementos (isso é o suficiente para preencher outro livro inteiro), mas os fundamentos básicos são discutidos.

Concluiremos o capítulo dando uma olhada em algumas das dificuldades de desempenho que podem surgir se você não prestar atenção à maneira como modifica e lê as informações do DOM. Vamos começar vendo como injetar HTML arbitrário em nossas páginas.

.....

Por que você precisa preparar elementos de fechamento automático em uma página antes de injetar HTML nela?

Quais são os benefícios de trabalhar com fragmentos DOM ao inserir HTML?

Como você determina as dimensões de um elemento escondido em uma página?

.....

12.1 Injetando HTML no DOM

Nesta seção, veremos uma maneira eficiente de inserir HTML em um documento em qualquer local, considerando esse HTML como uma string. Apresentamos essa técnica específica porque ela é frequentemente usada para criar páginas da web altamente dinâmicas nas quais a interface do usuário é modificada como uma resposta às ações do usuário ou dados recebidos do servidor. Isso é particularmente útil para os seguintes cenários:

- Injetar HTML arbitrário em uma página e manipular e inserir modelos do lado do cliente
- Recuperando e injetando HTML enviado de um servidor

Pode ser tecnicamente desafiador implementar essa funcionalidade corretamente (especialmente quando comparada à construção de uma API de construção DOM de estilo orientado a objetos, que certamente é mais fácil de implementar, mas requer uma camada extra de abstração do que injetar o HTML). Considere este exemplo de criação de elementos HTML a partir de uma string HTML que podemos usar com jQuery:

```
$ (document.body) .append ("<div> <h1> Saudações </h1> <p> Yoshi aqui </p> </div>")
```

E compare isso com uma abordagem que usa apenas a API DOM:

```
const h1 = document.createElement("h1"); h1.textContent = "Saudações";
```

```
const p = document.createElement("p");  
p.textContent = "Yoshi aqui";
```

```
const div = document.createElement("div");
```

```
div.appendChild(h1);  
div.appendChild(p);
```

```
document.body.appendChild(div);
```

Qual você prefere usar?

Por esses motivos, implementaremos nossa própria maneira de fazer a manipulação limpa do DOM do zero. A implementação requer as seguintes etapas:

- 1 Converta uma string HTML arbitrária, mas válida, em uma estrutura DOM.
- 2 Injete essa estrutura DOM em qualquer local no DOM da maneira mais eficiente possível.

Essas etapas fornecem aos autores de páginas uma maneira inteligente de injetar HTML em um documento. Vamos começar.

12.1.1 Convertendo HTML em DOM

Converter uma string HTML em uma estrutura DOM não envolve muita mágica. Na verdade, ele usa uma ferramenta com a qual você provavelmente já está familiarizado: o `innerHTML` propriedade dos elementos DOM.

Usá-lo é um processo de várias etapas:

- 1 Certifique-se de que a string HTML contém um código HTML válido.
- 2 Envolve a string em qualquer marcação envolvente exigida pelas regras do navegador. Insira a
- 3 string HTML, usando `innerHTML`, em um elemento DOM fictício. Extraia os nós DOM de volta.
- 4

As etapas não são excessivamente complexas, mas a inserção real tem alguns truques que precisaremos levar em consideração. Vamos dar uma olhada em cada etapa em detalhes.

PREPROCESSANDO A CADEIA DE FONTE HTML

Para começar, precisaremos limpar o código-fonte HTML para atender às nossas necessidades. Por exemplo, vamos dar uma olhada em um esqueleto HTML que nos permite escolher um ninja (por meio do opção) e que mostra os detalhes do ninja escolhido em uma tabela, detalhes que devem ser adicionados posteriormente:

```
<option> Yoshi </option>  
<option> Kuma </option>  
<mesa />
```


Esta string HTML tem dois problemas. Primeiro, os elementos de opção não devem se sustentar por si próprios. Se você seguir a semântica HTML adequada, eles devem estar contidos em um selecionar elemento. Em segundo lugar, embora as linguagens de marcação geralmente nos permitam fechar automaticamente os elementos sem filhos, como `< tabela />`, em HTML, o fechamento automático funciona apenas para um pequeno subconjunto de elementos (tabela não sendo um deles). Tentar usar essa sintaxe em outros casos pode causar problemas em alguns navegadores.

Vamos começar resolvendo o problema dos elementos de fechamento automático. Para oferecer suporte a esse recurso, podemos fazer uma preparação rápida na string HTML para converter elementos como `<mesa />` para `< tabela> </table>` (que será tratado uniformemente em todos os navegadores), conforme mostrado na lista a seguir.

Listagem 12.1 Certificando-se de que os elementos de fechamento automático são interpretados corretamente

```
tags const =
  ↪ / ^ ( area | base | br | col | embed | hr | img | input | keygen | link | menuitem | meta | param |
  ↪      fonte | faixa | wbr ) $ / i ;
function convert (html) {
  ↪      return html.replace (
      / (<(\w +) [^>]*?) \ /> / g, (all, front, tag) => {return tags.test (tag)? todo :
      front + ">" + tag + ">";
  });
}
assert (convert ("<a/>") === "<a> </a>", "Verifique a conversão âncora."); assert (convert ("<hr />") === "<hr />", "Verifique a
conversão de hr.");
```

Usa uma expressão regular para corresponder ao nome da tag de quaisquer elementos com os quais não precisamos nos preocupar

Uma função que usa expressões regulares para converter auto-fechamento tags para a forma "normal"

Quando aplicamos o `converter` a esta string HTML de exemplo, terminamos com a seguinte string HTML:

```
<option> Yoshi </option>
<option> Kuma </option>
<table> </table>
```

<tabela /> expandida

Feito isso, ainda temos que resolver o problema que nosso opção elementos não estão contidos em um selecionar elemento. Vamos ver como determinar se um elemento precisa ser encapsulado.

WRAPPING HTML

De acordo com a semântica do HTML, alguns elementos HTML devem estar dentro de certos elementos de contêiner antes de serem injetados. Por exemplo, um `< opção>` elemento deve estar contido em um `< seleccione>`.

Podemos resolver esse problema de duas maneiras, sendo que ambas requerem a construção de um mapeamento entre os elementos problemáticos e seus contêineres:

- A string pode ser injetada diretamente em um pai específico usando innerHTML, onde o pai foi construído anteriormente usando o integrado documento createElement. Embora isso possa funcionar em alguns casos e em alguns navegadores, não é universalmente garantido.
- A string pode ser envolvida com a marcação necessária apropriada e, em seguida, injetada diretamente em qualquer elemento de contêiner (como um <div>). Isso é mais infalível, mas também dá mais trabalho.

A segunda técnica é preferida; envolve pouco código específico do navegador, em contraste com a primeira abordagem, que requer uma boa quantidade de código principalmente específico do navegador.

O conjunto de elementos problemáticos que precisam ser incluídos em elementos de contêiner específicos é felizmente um conjunto de sete bastante gerenciável. Na tabela 12.1, as reticências (...) indicam os locais onde os elementos devem ser injetados.

Tabela 12.1 Elementos que precisam estar contidos em outros elementos

Nome do elemento	Elemento ancestral
<option>, <optgroup>	<selecionar vários> ... </select>
<legenda>	<fieldset> ... </fieldset>
<thead>, <tbody>, <tfoot>, <table> ... </table>	<colgroup>, <caption>
<tr>	<table><thead>...</thead> </table> <table><tbody>...</tbody> </table> <table><tfoot>...</tfoot> </table>
<td>, <th>	<table><tbody><tr>...</tr></tbody> </table>
<col>	<table> <tbody> </tbody> <colgroup> ... </colgroup> </table>

Quase todos eles são diretos, exceto pelos seguintes pontos, que requerem um pouco de explicação:

- A <selecionar> elemento com o múltiplo atributo é usado (ao contrário de uma seleção não múltipla) porque ele não verificará automaticamente nenhuma das opções que são colocadas dentro dele (enquanto uma única seleção verificará automaticamente a primeira opção). O <col> correção inclui um <tbody>, sem o qual o <colgroup> não será gerado corretamente.

Com os elementos devidamente mapeados para seus requisitos de agrupamento, vamos começar a gerar.

Com as informações da tabela 12.1, podemos gerar o HTML que precisamos inserir em um elemento DOM, conforme mostrado na lista a seguir.

Listagem 12.2 Gerando uma lista de nós DOM de alguma marcação

```
function getNodes (htmlString, doc) {
  const map = {
    "<td>": [3, "<table> <tbody> <tr>", "</tr> </tbody> </table>"], "<th>": [3, "<table> <tbody> <tr> ", "</tr> </tbody> </table>"], "<tr>": [2, "<table> <thead> ", "</thead> </table>"],
    "<option>": [1, "<select multiple>", "</select>"],
    "<optgroup>": [1, "<select multiple>", "</select>"],
    "<legend>": [1, "<fieldset>", "</fieldset>"],
    "<thead>": [1, "<table>", "</table>"],
    "<tbody>": [1, "<table>", "</table>"],
    "<tfoot>": [1, "<table>", "</table>"],
    "<colgroup>": [1, "<table>", "</table>"],
    "<caption>": [1, "<table>", "</table>"],
    "<col>": [2, "<table> <tbody> </tbody> <colgroup>", "</colgroup> </table>"],
  };
  const tagName = htmlString.match (/ <\w + /);
  let mapEntry = tagName? map [tagName [0]]: null; if (! mapEntry) {mapEntry = [0, "", ""].;}

  let div = (doc || document) .createElement ("div");
  div.innerHTML = mapEntry [1] + htmlString + mapEntry [2]; while (mapEntry [0] -) {div = div.lastChild;}
  return div.childNodes;
}

assert (getNodes ("<td> test </td> <td> test2 </td>").length === 2,
  "Obtenha dois nós de volta do método.");
assert (getNodes ("<td> test </td>") [0] .nodeName === "TD",
  "Verifique se estamos obtendo o nó correto.");
```

Partidas
a
abertura
suporte
e marcar
nome

Mapa de tipos de elementos que precisam de contêineres pai especiais. Cada entrada tem a profundidade do novo nó, abrindo HTML para os pais e fechando o HTML para os pais.

Se estiver no mapa, captura a entrada; caso contrário, constrói uma entrada falsa com marcação "pai" vazia e profundidade zero.

Retorna o elemento recém-criado

Cria um elemento <div> no qual criar os novos nós. Observe que usamos um documento aprovado, se existir, ou o padrão para o documento atual, caso não exista.

Desce a árvore recém-criada até a profundidade indicada pela entrada do mapa. Este deve ser o pai do nó desejado

criado a partir da marcação.

Envolve a entrada marcação com o pais da entrada do mapa e o injeta como o HTML interno do <div> recém-criado

Criamos um mapa de todos os tipos de elementos que precisam ser colocados em contêineres pai especiais, um mapa que contém a profundidade do nó, bem como o HTML que o contém. Em seguida, usamos uma expressão regular para combinar o colchete de abertura e o nome da tag do elemento que queremos inserir:

```
const tagName = htmlString.match (/ <\w + /);
```

Em seguida, selecionamos uma entrada do mapa e, caso não haja uma, criamos uma entrada fictícia com uma marcação de elemento pai vazia:

```
let mapEntry = tagName? map [tagName [0]]: null; if (! mapEntry) {mapEntry = [0, "", ""].; }
```

Seguimos isso criando um novo div elemento, envolvendo-o com o HTML mapeado e inserindo o HTML recém-criado no criado anteriormente div elemento:

```
let div = (doc || document) .createElement ("div");
div.innerHTML = mapEntry [1] + htmlString + mapEntry [2]
```

Finalmente, encontramos o pai do nó desejado criado a partir de nossa string HTML e retornamos o nó recém-criado:

```
while (mapEntry [0] -) {div = div.lastChild;} return div.firstChild;
```

Depois de tudo isso, temos um conjunto de nós DOM que podemos começar a inserir no documento.

Se voltarmos ao nosso exemplo motivador e aplicar o `getNodes` função, vamos acabar com algo nas seguintes linhas:

```
<selecionar vários>
  <option> Yoshi </option>
  <option> Kuma </option>
</select>
<table> </table>
```

Elementos de opção têm
foi embalado dentro de um
elemento de seleção.

12.1.2 Inserindo elementos no documento

Depois de termos os nós DOM, é hora de inseri-los no documento. Algumas etapas são necessárias, e as trabalharemos nesta seção.

Como temos uma matriz de elementos que precisamos inserir - potencialmente em qualquer número de locais dentro do documento - queremos tentar manter o número de operações realizadas ao mínimo. Podemos fazer isso usando *Fragmentos DOM*. Os fragmentos DOM fazem parte da especificação W3C DOM e são suportados em todos os navegadores. Esse recurso útil nos dá um contêiner para armazenar uma coleção de nós DOM.

Isso em si é bastante útil, mas também tem a vantagem de que o fragmento pode ser injetado e clonado em uma única operação, em vez de ter que injetar e clonar cada nó individual repetidamente. Isso tem o potencial de reduzir drasticamente o número de operações necessárias para uma página.

Antes de usarmos esse mecanismo em nosso código, vamos revisar o `getNodes()` código da listagem 12.2 e ajuste-o um pouco para usar fragmentos DOM. As mudanças são pequenas e consistem em adicionar um fragmento parâmetro para a lista de parâmetros da função, como segue.

Listagem 12.3 Expandindo o `getNodes` função com fragmentos

```
function getNodes (htmlString, doc, fragmento){
  const map = {
    "<td>": [3, "<table> <tbody> <tr> ", "</tr> </tbody> </table>"], "<th>": [3, "<table> <tbody> <tr> ", "</tr> </tbody> </table> "], "<tr>": [2, "<table> <thead> ", "</thead> </table> "],
    "<option>": [1, "<select multiple>", "</select>"],
    "<optgroup>": [1, "<select multiple>", "</select>"],
    "<legend>": [1, "<fieldset>", "</fieldset>"],
    "<thead>": [1, "<table>", "</table>"],
    "<tbody>": [1, "<table>", "</table>"],
    "<tfoot>": [1, "<table>", "</table>"],
    "<colgroup>": [1, "<table>", "</table>"],
    "<caption>": [1, "<table>", "</table>"],
```

←
Adiciona um novo
fragmento
parâmetro para
a função

```

    "<col": [2, "<table> <tbody> </tbody> <colgroup>", "</colgroup> </table>"],
  };
  const tagName = htmlString.match (/ <\w + /);
  let mapEntry = tagName? map [tagName [0]]: null; if (! mapEntry) {mapEntry = [0, "", ""].;}

  let div = (doc || document) .createElement ("div");
  div.innerHTML = mapEntry [1] + htmlString + mapEntry [2]; while (mapEntry [0] -) {div = div.lastChild;}; if
  (fragmento) {

    while (div.firstChild) {
      fragment.appendChild (div.firstChild);
    }
  }

  return div.childNodes;
}

```

Se o fragmento
existe, injeta o
nós nele.

Neste exemplo, fazemos algumas alterações. Primeiro, modificamos a assinatura da função adicionando outro parâmetro, `fragmento`:

```
function getNodes (htmlString, doc, fragmento) {...}
```

Este parâmetro, se for passado, deve ser um fragmento DOM no qual queremos que os nós sejam injetados para uso posterior.

Para fazer isso, adicionamos o seguinte fragmento antes do Retorna declaração da função para adicionar os nós ao fragmento passado:

```

if (fragmento) {
  while (div.firstChild) {
    fragment.appendChild (div.firstChild);
  }
}

```

Agora, vamos ver em uso. Na lista a seguir, que assume que o `getNodes` função está no escopo, um fragmento é criado e passado para essa função (que, você deve se lembrar, converte a string HTML recebida em elementos DOM). Este DOM agora está anexado ao fragmento.

Listagem 12.4 Inserindo um fragmento DOM em vários locais no DOM

```
<div id = "test"> <b> Olá </b>, sou um ninja! </div> <div id = "test2"> </div>
```

Configura alguns
nós de teste

```

<script>
  document.addEventListener ("DOMContentLoaded", () => {
    inserção de função (elems, args, callback) {
      if (elems.length) {
        const doc = elems [0] .ownerDocument || elems [0].
        fragment = doc.createDocumentFragment (),
        scripts = getNodes (args, doc, fragmento), primeiro = fragment.firstChild;

```

Cria HTML
nós do
String HTML

Cria um documento
fragmento no qual
vamos inserir nós

```

    if (primeiro) {
      para (deixe i = 0; elems [i]; i++) {
        callback.call (root (elems [i], primeiro),
          i > 0? fragment.cloneNode (true): fragment);
      }
    }
  }
}

const divs = document.querySelectorAll ("div"); inserir (divs, "<b> Nome: </b>", função
(fragmento) {
  this.appendChild (fragmento);
});

insert (divs, "<span> primeiro </span> <span> último </span>",
  função (fragmento) {
    this.parentNode.insertBefore (fragmento, isso);
  });
});
</script>

```

Se precisarmos inserir os
nós em mais de um elemento,
temos que clonar o fragmento a
cada vez.

Há outro ponto importante aqui: se estivermos inserindo esse elemento em mais de um local no documento, precisaremos clonar esse fragmento várias vezes. Se não estivéssemos usando um fragmento, teríamos que clonar cada nó individual todas as vezes, em vez de todo o fragmento de uma vez.

Com isso, desenvolvemos uma maneira de gerar e inserir elementos DOM arbitrários de maneira intuitiva. Vamos continuar esta exploração do DOM vendo a diferença entre os atributos e propriedades do DOM.

12.2 Usando atributos e propriedades DOM

Ao acessar os valores dos atributos do elemento, temos duas opções: usando os métodos tradicionais de DOM de `getAttribute` e `setAttribute`, ou usando propriedades dos objetos DOM que correspondem aos atributos.

Por exemplo, para obter o `eu` de um elemento cuja referência é armazenada na variável `e`, podemos usar um dos seguintes:

```

e.getAttribute ('id')
e.id

```

Qualquer um nos dará o valor do `eu` `ia`.

Vamos examinar o código a seguir para entender melhor como os valores dos atributos e suas propriedades correspondentes se comportam.

Listagem 12.5 Acessando valores de atributo por meio de métodos e propriedades DOM

```

<div> </div>
<script>
  document.addEventListener ("DOMContentLoaded", () => {
    const div = document.querySelector ("div");

```

Obtém um elemento
referência

Usando
setAttribute
também muda
O valor que
obtido
através de
propriedade.

```
div.setAttribute("id", "ninja-1");
assert (div.getAttribute('id') === "ninja-1",
        "Atributo alterado com sucesso");

div.id = "ninja-2";
assert (div.id === "ninja-2",
        "Propriedade alterada com sucesso");

assert (div.getAttribute('id') === "ninja-2",
        "Atributo alterado com sucesso via propriedade");
div.setAttribute("id", "ninja-3");
assert (div.id === "ninja-3",
        "Propriedade alterada com sucesso via atributo"); assert (div.getAttribute('id') ===
        "ninja-3",
        "Atributo alterado com sucesso");

});
</script>
```

Altera o valor do atributo id com o método setAttribute e testa se o valor mudou

Altera o valor da propriedade e testa se o valor mudou

Alterando a propriedade também muda o valor obtido com getAttribute.

Este exemplo mostra um comportamento interessante com relação aos atributos e propriedades do elemento. Ele começa definindo um <div> elemento que usaremos como assunto de teste. Dentro do documento DOMContentLoaded handler (para garantir que o DOM seja totalmente construído), obtemos uma referência ao único <div> elemento, const div = document

.querySelector("div"), e, em seguida, execute alguns testes.

No primeiro teste, definimos o eu ia Atribua ao valor ninja-1 através do setAttribute () método. Então afirmamos que getAttribute () retorna o mesmo valor para esse atributo. Não deve ser surpresa que este teste funcione bem quando carregamos a página:

```
div.setAttribute("id", "ninja-1");
assert (div.getAttribute('id') === "ninja-1",
        "Atributo alterado com sucesso");
```

Da mesma forma, no próximo teste, definimos o eu ia propriedade para o valor ninja-2 e então verifique se o valor da propriedade realmente mudou. Sem problemas.

```
div.id = "ninja-2";
assert (div.id === "ninja-2",
        "Propriedade alterada com sucesso");
```

O próximo teste é quando as coisas ficam interessantes. Nós novamente definimos o eu ia propriedade para um novo valor, neste caso ninja-3, e verifique novamente se o valor da propriedade mudou. Mas então também afirmamos que não só deve mudar o valor da propriedade, mas também o valor do eu ia *atributo*. Ambas as afirmações passam. Com isso aprendemos que o eu ia propriedade e o eu ia atributo estão de alguma forma ligados entre si. Alterando a eu ia o valor da propriedade também altera o eu ia Valor do atributo:

```
div.id = "ninja-3";
assert (div.id === "ninja-3",
        "Propriedade alterada com sucesso");
assert (div.getAttribute('id') === "ninja-3",
        "Atributo alterado com sucesso via propriedade");
```

O próximo teste prova que também funciona ao contrário: definir um valor de atributo também altera o valor da propriedade correspondente.

```
div.setAttribute("id", "ninja-4");
assert (div.id === "ninja-4",
    "Propriedade alterada com sucesso via atributo");
assert (div.getAttribute('id') === "ninja-4", "Atributo alterado");
```

Mas não se deixe enganar pensando que a propriedade e o atributo compartilham o mesmo valor - não estão. Veremos mais adiante neste capítulo que o atributo e a propriedade correspondente, embora vinculados, nem sempre são idênticos.

É importante observar que nem todos os atributos são representados por propriedades de elementos. Embora seja geralmente verdadeiro para atributos que são nativamente especificados pelo HTML DOM, *atributos personalizados* que podemos colocar nos elementos em nossas páginas não são automaticamente representados pelas propriedades do elemento. Para acessar o valor de um atributo personalizado, precisamos usar os métodos DOM `getAttribute()` e `setAttribute()`.

Se você não tiver certeza se uma propriedade para um atributo existe, você sempre pode testá-la e voltar aos métodos DOM se ela não existir. Aqui está um exemplo:

```
const value = element.someValue? element.someValue
    : element.getAttribute('someValue');
```

GORJETA Em HTML5, use o prefixo `data-` para todos os atributos personalizados para mantê-los válidos de acordo com a especificação HTML5. É uma boa convenção que separa claramente os atributos personalizados dos atributos nativos.

12.3 Dores de cabeça de atributo de estilo

Tal como acontece com os atributos gerais, obter e definir atributos de estilo pode ser uma dor de cabeça. Tal como acontece com os atributos e propriedades na seção anterior, novamente temos duas abordagens para lidar com estilo valores: o valor do atributo e a propriedade do elemento criada a partir dele.

O mais comumente usado é o estilo propriedade do elemento, que não é uma string, mas um objeto que contém propriedades correspondentes aos valores de estilo especificados na marcação do elemento. Além disso, você verá que há um método para acessar as informações de estilo computado de um elemento, onde *estilo computado* significa o estilo que será aplicado ao elemento após avaliar todas as informações de estilo herdadas e aplicadas.

Esta seção descreve o que você precisa saber ao trabalhar com estilos em navegadores. Vamos começar examinando onde as informações de estilo são registradas.

12.3.1 Onde estão meus estilos?

As informações de estilo localizadas no estilo propriedade de um elemento DOM é inicialmente definida a partir do valor especificado para o estilo atributo na marcação do elemento. Por exemplo, `estilo = "cor: vermelho;"` resulta na inserção dessas informações de estilo no objeto de estilo. Durante a execução da página, o script pode definir ou modificar valores no objeto de estilo e essas alterações afetarão ativamente a exibição do elemento.

Muitos autores de script ficam desapontados ao descobrir que nenhum valor da página <estilo> elementos ou folhas de estilo externas estão disponíveis no estilo objeto. Mas não ficaremos desapontados por muito tempo - em breve você verá uma maneira de obter essas informações.

Por enquanto, vamos ver como o estilo propriedade obtém seus valores. Examine o seguinte código.

Listagem 12.6 Examinando o **estilo** propriedade

```

<style>
  div {tamanho da fonte: 1.8em; borda: 0 ouro maciço; } </style>

<div style = "color: # 000;" title = "Poder Ninja">
  忍者 パワー
</div>
<script>
  document.addEventListener ("DOMContentLoaded", () => {
    const div = document.querySelector ("div"); assert (div.style.color === 'rgb (0, 0, 0)' ||

      div.style.color === '# 000',
      'a cor foi registrada');
    assert (div.style.fontSize === '1.8em',
      'fontSize foi gravado');
    assert (div.style.borderWidth === '0',
      'borderWidth foi gravado');
    div.style.borderWidth = "4px";
    assert (div.style.borderWidth === '4px',
      'borderWidth foi substituído');

  });
</script>

```

Declara uma folha de estilo in-page que se aplica tamanho da fonte e informações de borda

Este elemento de teste deve receber vários estilos de vários lugares, incluindo seu próprio atributo de estilo e a folha de estilo.

Testa que o estilo de cor embutido foi gravado

Testa se o estilo de tamanho de fonte herdado foi registrado

Testa se o estilo de largura de borda herdado foi registrado

Substitui a fronteira estilo de largura

Testa o estilo da largura da borda mudança foi registrada

Neste exemplo, configuramos um <estilo> elemento para estabelecer uma folha de estilo interna cujos valores serão aplicados aos elementos da página. A folha de estilo especifica que todos <div> os elementos aparecerão em um tamanho de fonte 1,8 vezes maior do que o padrão, com uma borda de ouro sólido 0 largura. Quaisquer elementos aos quais isso seja aplicado possuirão uma borda, mas não serão visíveis porque têm uma largura de 0

```

<style>
  div {tamanho da fonte: 1.8em; borda: 0 ouro maciço; } </style>

```

Em seguida, criamos um <div> elemento com um atributo de estilo embutido que colore o texto do elemento em preto:

```

<div style = "color: # 000;" title = "Poder Ninja">
  忍者 パワー
</div>

```

Em seguida, começamos o teste. Depois de obter uma referência ao <div> elemento, testamos que o estilo atributo recebe um cor propriedade que representa a cor atribuída ao elemento. Observe que embora o cor é especificado como # 000 no estilo inline, é normalizado para notação RGB quando definido no estilo na maioria dos navegadores (por isso verificamos os dois formatos).

```
assert (div.style.color === 'rgb (0, 0, 0)' ||
        div.style.color === '# 000',
        'a cor foi registrada');
```

Olhando para frente, na figura 12.1, vemos que esse teste foi aprovado.

Em seguida, testamos ingenuamente se o estilo do tamanho da fonte e a largura da borda especificada na folha de estilo embutida foram registrados no objeto de estilo. Porém, embora possamos ver na figura 12.1 que o estilo de tamanho da fonte foi aplicado ao elemento, o teste falha. Isso ocorre porque o objeto de estilo não reflete nenhuma informação de estilo herdada das folhas de estilo CSS:

```
assert (div.style.fontSize === '1.8em',
        'fontSize foi gravado');
assert (div.style.borderWidth === '0',
        'borderWidth foi gravado');
```

Continuando, usamos uma atribuição para alterar o valor do borderWidth propriedade no objeto de estilo para 4 pixels de largura e teste se a alteração é aplicada. Podemos ver na figura 12.1 que o teste passa e que a borda anteriormente invisível é aplicada ao elemento. Esta atribuição causa um borderWidth propriedade para aparecer no estilo propriedade do elemento, conforme comprovado pelo teste.

```
div.style.borderWidth = "4px";
assert (div.style.borderWidth === '4px',
        'borderWidth foi substituído');
```

Deve-se notar que quaisquer valores em um elemento estilo A propriedade tem precedência sobre qualquer coisa herdada por uma folha de estilo (mesmo se a regra da folha de estilo usar o ! importante anotação).

Uma coisa que você deve ter notado na listagem 12.6 é que o CSS especifica a propriedade do tamanho da fonte como tamanho da fonte, mas no script você o referencia como tamanho da fonte. Por que é que?

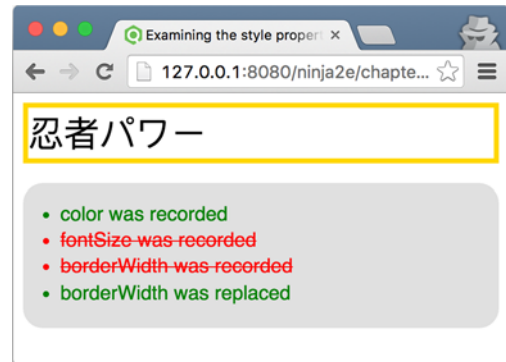


Figura 12.1 Ao executar este teste, podemos ver que os estilos embutidos e atribuídos são registrados, mas os estilos herdados não.

12.3.2 Nomenclatura de propriedade de estilo

Os atributos CSS causam relativamente poucas dificuldades entre navegadores quando se trata de acessar os valores fornecidos pelo navegador. Mas existem diferenças entre como os estilos de nomes CSS e como os acessamos no script, e alguns nomes de estilo diferem entre os navegadores.

Os atributos CSS que abrangem mais de uma palavra separam as palavras com um hífen; exemplos são `font-weight`, `font-size`, e `cor de fundo`. Você deve se lembrar que nomes de propriedades em JavaScript *podem* conter um hífen, mas a inclusão de um hífen impede que a propriedade seja acessada por meio do operador ponto.

Considere este exemplo:

```
const fontSize = element.style ['font-size'];
```

O anterior é perfeitamente válido. Mas o seguinte não é:

```
const fontSize = element.style.font-size;
```

O analisador JavaScript veria o hífen como um operador de subtração e ninguém ficaria feliz com o resultado. Em vez de forçar os desenvolvedores de página a sempre usar o formulário geral para acesso à propriedade, *nomes de estilo CSS com várias palavras são convertidos em caixa de camelo quando usados como um nome de propriedade*. Como um resultado, tamanho da fonte torna-se tamanho da fonte, e fundo-cor torna-se cor de fundo.

Podemos nos lembrar de fazer isso ou escrever uma API simples para definir ou obter estilos que manipulam automaticamente a caixa do camelo, conforme mostrado na lista a seguir.

Listagem 12.7 Um método simples para acessar estilos

Define a função de estilo que atribuirá um valor a uma propriedade de estilo caso um valor seja fornecido e, caso não seja, simplesmente retornará o valor da propriedade de estilo. Podemos usar esta função para definir e obter o valor de uma propriedade de estilo.

```
<div style = "color: red; font-size: 10px; background-color: #eee;"> </div> <script>
```

**Sempre retorna
O valor que
do
estilo
propriedade**

```

estilo da função (elemento, nome, valor) {
  nome = nome.substituir (/ - ([az]) / ig, (todos, letra) => {
    return letra.toUpperCase ();
  });
  if (typeof value! == 'undefined') {
    element.style [nome] = valor;
  }
  return element.style [nome];
}

document.addEventListener ("DOMContentLoaded", () => {
  const div = document.querySelector ("div");
  assert (style (div, 'color') === "red", style (div, 'color')); assert (style (div, 'font-size') === "10px", style (div, 'font-size')); assert (style (div, 'background-color') ===
    "rgb (238, 238, 238)", estilo (div, 'cor de fundo'));
});
</script>
```

**Converte o nome
para a caixa de camelo**

**O novo valor da propriedade
de estilo é definido, se um
valor for fornecido.**

A função de estilo tem duas características importantes:

- Ele usa uma expressão regular para converter o nome parâmetro para notação camel-case. (Se a operação de conversão orientada por regex o deixou coçando a cabeça, você pode querer revisar o material no capítulo 10.)
- Ele pode ser usado como setter e getter, inspecionando sua própria lista de argumentos. Por exemplo, podemos obter o valor da propriedade font-size com estilo (div, 'font-size'), e podemos definir um novo valor com estilo (div, 'font-size', '5px').

Considere o seguinte código:

```
estilo da função (elemento, nome, valor) {  
    ...  
    if (typeof value! == 'undefined') {  
        element.style [nome] = valor;  
    }  
  
    return element.style [name];  
}
```

Se um valor argumento é passado para a função, a função atua como um setter, definindo o valor passado como o valor do atributo. Se o valor argumento é omitido e apenas os dois primeiros argumentos são passados, ele atua como um getter, recuperando o valor do atributo especificado. Em qualquer um dos casos, o valor do atributo é retornado, o que facilita o uso da função em qualquer um de seus modos em uma cadeia de chamada de função.

O estilo propriedade de um elemento não inclui nenhuma informação de estilo que um elemento herda de folhas de estilo no escopo do elemento. Muitas vezes seria útil saber todo o estilo computado que foi aplicado a um elemento, então vamos ver se há uma maneira de obtê-lo.

12.3.3 Buscando estilos calculados

A qualquer momento, o *estilo computado* de um elemento é uma combinação de todos os estilos embutidos fornecidos pelo navegador, todos os estilos aplicados a ele por meio de folhas de estilo, o elemento estilo atributo, e quaisquer manipulações do estilo propriedade por script. A Figura 12.2 mostra como as ferramentas de desenvolvedor de navegador diferenciam os estilos.

O método padrão, implementado por todos os navegadores modernos, é o `getComputedStyle` método. Este método aceita um elemento cujos estilos devem ser calculados e retorna uma interface por meio da qual as consultas de propriedade podem ser feitas. A interface retornada fornece um método chamado `getPropertyValue` para recuperar o estilo calculado de uma propriedade de estilo específica.

Ao contrário das propriedades de um elemento estilo objeto, o `getPropertyValue` método aceita nomes de propriedade CSS (como tamanho da fonte e cor de fundo) em vez das versões em caixa de camelo desses nomes.

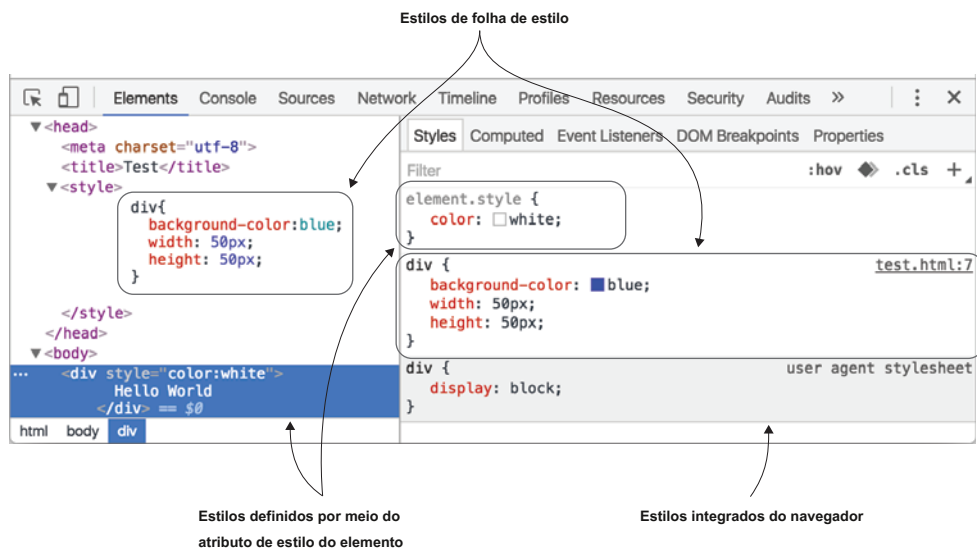


Figura 12.2 O estilo final associado a um elemento pode vir de muitas coisas: os estilos internos do navegador (folha de estilo do agente do usuário), os estilos atribuídos por meio da propriedade de estilo e os estilos das regras CSS definidas no código CSS.

A lista a seguir mostra um exemplo simples.

Listagem 12.8 Buscando valores de estilo computados

```

<style>
  div {
    cor de fundo: #ffc; display: inline; tamanho da fonte: 1.8em; borda: 1px carmesim sólido; cor verde;
  }
</style>
<div style="color: crimson;" id="testSubject" title="Poder Ninja">
  忍者 パワー
</div>
<script>
  function fetchComputedStyle (elemento, propriedade) {
    const computedStyles = getComputedStyle (elemento); if (computedStyles) {

      property = property.replace (/ ([AZ]) / g, '- $1'). toLowerCase (); return computedStyles.getPropertyValue
      (propriedade);
    }
  }
  document.addEventListener ("DOMContentLoaded", () => {
    const div = document.querySelector ("div"); relatório ("cor de fundo:" +
      fetchComputedStyle (div, 'background-color');
      relatório ("display:" +
        fetchComputedStyle (div, 'display'));
  });
</script>

```

Define um função que vai pegar o valor de um estilo computado

Testes que nós pode obter o valores de vários propriedades de estilo, usando diferente notações

Cria um assunto de teste com um atributo de estilo

Usa o método getComputedStyle integrado para obter um objeto descritor

Substitui caixa de camelo notação com travessões

```

relatório ("tamanho da fonte:" +
    fetchComputedStyle (div, 'fontSize'));
relatório ("cor:" +
    fetchComputedStyle (div, 'color'));
relatório ("border-top-color:" +
    fetchComputedStyle (div, 'borderTopColor'));
relatório ("border-top-width:" +
    fetchComputedStyle (div, 'border-top-width'));
});
</script>

```

Para testar a função que iremos criar, configuramos um elemento que especifica informações de estilo em sua marcação e uma folha de estilo que fornece regras de estilo a serem aplicadas ao elemento. Esperamos que os estilos calculados sejam o resultado da aplicação dos estilos imediato e aplicado ao elemento.

Em seguida, definimos a nova função, que aceita um elemento e a propriedade de estilo para a qual queremos encontrar o valor calculado. E para ser especialmente amigável (afinal, somos ninjas - tornar as coisas mais fáceis para aqueles que usam nosso código faz parte do trabalho), permitiremos que nomes de propriedades com várias palavras sejam especificados em qualquer formato: tracejado ou camelcased. Em outras palavras, aceitaremos ambos cor de fundo e cor de fundo.

Veremos como fazer isso em breve.

A primeira coisa que queremos fazer é obter a interface de estilo computado, que armazenamos em uma variável, `computedStyles`, para referência posterior. Queremos fazer as coisas dessa maneira porque não sabemos o quanto caro pode ser essa ligação e é melhor evitar repeti-la desnecessariamente.

```

const computedStyles = getComputedStyle (elemento); if (computedStyles) {

    property = property.replace (/ ([AZ]) / g, '- $ 1'). toLowerCase (); return computedStyles.getPropertyValue
    (propriedade);
}

```

Se isso for bem-sucedido (e não conseguimos pensar em nenhuma razão para não conseguir, mas frequentemente vale a pena ser cauteloso), chamamos o `getPropertyValue ()` método da interface para obter o valor de estilo calculado. Mas primeiro ajustamos o nome da propriedade para acomodar a versão com caixa de camelo ou tracejado do nome da propriedade. O `getPropertyValue` método espera a versão tracejada, então usamos o `Fragmento de substituir()` método, com uma expressão regular simples, mas inteligente, para inserir um hífen antes de cada caractere maiúsculo e depois minúsculo tudo. (Aposto que foi mais fácil do que você pensava.)

Para testar a função, fazemos chamadas para a função, passando vários nomes de estilo em vários formatos, e exibimos os resultados, conforme mostrado na Figura 12.3.

Observe que os estilos são buscados independentemente de serem declarados explicitamente no elemento ou herdados da folha de estilo. Observe também que o `cor` propriedade, especificada na folha de estilo e diretamente no elemento, retorna o valor explícito. Estilos especificados por um elemento estilo atributo sempre tem precedência sobre estilos herdados, mesmo se marcado! importante.

Precisamos estar cientes de mais um tópico ao lidar com propriedades de estilo: *amálgama* propriedades. CSS nos permite usar uma notação de atalho para o amálgama de propriedades como o `fronteira-`

propriedades. Em vez de nos forçar a especificar cores, larguras e estilos de borda individualmente e para todas as quatro bordas, podemos usar uma regra como esta:

borda: 1px carmesim sólido;

Usamos essa regra exata na listagem 12.8. Isso economiza muita digitação, mas precisamos estar cientes de que, quando recuperamos as propriedades, precisamos buscar as propriedades individuais de nível inferior. Não podemos buscar `fronteira-` Figura 12.3 Os estilos calculados incluem todos os estilos especificados com o elemento, bem como aqueles herdados das folhas de estilo.



Pode ser um pouco complicado, especialmente quando todos os quatro estilos têm os mesmos valores, mas essa é a mão que recebemos.

12.3.4 Conversão de valores de pixel

Um ponto importante a considerar ao definir valores de estilo é a atribuição de valores numéricos que representam pixels. Ao definir um valor numérico para uma propriedade de estilo, devemos especificar a unidade para que funcione de forma confiável em todos os navegadores. Por exemplo, digamos que queremos definir o altura valor de estilo de um elemento para 10 pixels. Uma das opções a seguir é uma maneira segura de fazer isso em vários navegadores:

```
element.style.height = "10px";
element.style.height = 10 + "px";
```

O seguinte não é seguro em navegadores:

```
element.style.height = 10;
```

Você pode pensar que seria fácil adicionar um pouco de lógica ao `estilo()` função da listagem 12.7 para unir um px ao final de um valor numérico que entra na função. Mas não tão rápido! Nem todos os valores numéricos representam pixels! Algumas propriedades de estilo assumem valores numéricos que não representam uma dimensão de pixel. A lista inclui o seguinte:

- `z-index`
- espessura da fonte
- opacidade
- ampliação
- altura da linha

Para estes (e quaisquer outros que você possa imaginar), vá em frente e estenda a função da listagem 12.6 para lidar automaticamente com valores não pixelados. Além disso, ao tentar ler um valor de pixel de um atributo de estilo, o `parseFloat` deve ser usado para garantir que você obtenha o valor pretendido em todas as circunstâncias.

Agora, vamos dar uma olhada em um conjunto de propriedades de estilo importantes que podem ser difíceis de manusear.

12.3.5 Medindo alturas e larguras

Propriedades de estilo, como altura e largura representam um problema especial, porque seus valores padrão para auto quando não especificado, para que o elemento se dimensione de acordo com seu conteúdo. Como resultado, não podemos usar o altura e largura propriedades de estilo para obter valores precisos, a menos que valores explícitos sejam fornecidos na string de atributo.

Felizmente, o `offsetHeight` e `offsetWidth` As propriedades fornecem exatamente isso: um meio bastante confiável de acessar a altura e a largura de um elemento. Mas esteja ciente de que os valores atribuídos a essas duas propriedades incluem o preenchimento do elemento. Essas informações geralmente são exatamente o que desejamos se estivermos tentando posicionar um elemento sobre o outro. Mas às vezes podemos querer obter informações sobre as dimensões do elemento com e sem bordas e preenchimento.

Algo a se observar, no entanto, é que em sites altamente interativos, os elementos provavelmente passarão parte de seu tempo em um estado não exibido (com o exibição estilo sendo definido para Nenhum), e quando um elemento não faz parte da exibição, ele não tem dimensões. Qualquer tentativa de buscar o `offsetWidth` ou `offsetHeight` propriedades de um elemento não exibido resultarão em um valor de 0

Para esses elementos ocultos, se quisermos obter as dimensões não ocultas, podemos empregar um truque para revelar momentaneamente o elemento, capturar os valores e ocultá-lo novamente. Claro, queremos fazer isso de uma forma que não deixemos nenhuma pista visível de que isso está acontecendo nos bastidores. Como podemos tornar um elemento oculto não oculto sem torná-lo visível?

Empregando nossas habilidades ninja, podemos fazer isso! Veja como:

- 1 Mudar o exibição propriedade para quadra.
- 2 Definir visibilidade para escondido.
- 3 Definir posição para absoluto.
- 4 Pegue os valores de dimensão.
- 5 Restaure as propriedades alteradas.

Alterando a exibição propriedade para quadra nos permite agarrar os valores de `offsetHeight` e `offsetWidth`, mas isso torna o elemento parte da tela e, portanto, visível. Para tornar o elemento invisível, vamos definir o visibilidade propriedade para escondido. Mas (sempre há outro *mas*) isso vai deixar um grande buraco onde o elemento é posicionado, então também definimos o posição propriedade para absoluto para tirar o elemento do fluxo normal de exibição.

Tudo isso parece mais complicado do que a implementação, que é mostrada na lista a seguir.

Listagem 12.9 Capturando as dimensões de elementos ocultos

<div>

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Suspendisse congue facilisis dignissim. Fusce sodales, odio commodo accumsan commodo, lacus odio aliquet purus,

 <img src = "../ images / ninja-with-shuriken.png"

id = estilo "withShuriken" = "display: nenhum" alt = "ninja shuriken" /> vel rhoncus elit sem quis libero. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Em hac habitasse platea dictumst. Donec adipiscing urna ut nibh vestibulum vitae mattis leo rutrum. Etiam a lectus ut nunc mattis laoreet em

placerat nulla. Aenean tincidunt lorem eu dolor commodo ornare.

</div>

<script

(função(){

const PROPERTIES = {

posição: "absoluta",

visibilidade: "oculto",

display: "bloquear"

};

window.getDimensions = element => {

const anterior = {};

para (deixe digitar PROPERTIES) {

anterior [chave] = element.style [chave];

element.style [chave] = PROPRIEDADES [chave];

}

const result = {

largura: element.offsetWidth,

height: element.offsetHeight

};

para (deixe entrar PROPERTIES) {

element.style [chave] = anterior [chave];

}

resultado de retorno;

};

})();

document.addEventListener ("DOMContentLoaded", () => {

setTimeout (() => {

const withPole = document.getElementById ('withPole'),

withShuriken = document.getElementById ('withShuriken'); assert (withPole.offsetWidth === 41,

"Largura da imagem do pólo obtida; real:" + withPole.offsetWidth + ", esperado: 41"); assert (withPole.offsetHeight === 48,

"Altura da imagem do pólo obtida; real:" + withPole.offsetHeight + ", esperado 48"); assert (withShuriken.offsetWidth === 36,

"Largura da imagem Shuriken obtida; real:" + withShuriken.offsetWidth + ", esperado: 36"); assert (withShuriken.offsetHeight === 48,

"Altura da imagem Shuriken obtida; real:" +

← Cria um escopo privado Define

← as propriedades de destino

← Cria a nova função Lembra

← configurações

← Substitui as configurações

← Coleta dimensões

← Restaura as configurações

← Testa elemento visível

← Testa elemento oculto

```

        withShuriken.offsetHeight + ", esperado 48"); dimensões const = getDimensions (com
Shuriken); afirmar (dimensões.largura === 36,

        "Largura da imagem Shuriken obtida: real:" + dimensões.largura + ", esperada:
36"); afirmar (dimensões.altura === 48,

        "Altura da imagem Shuriken obtida: real:" + dimensões.altura + ", esperado 48");

    }, 3000);
});
</script>

```

← Usa nova função

← Reteste o elemento oculto

É uma longa lista, mas a maior parte é código de teste; a implementação da nova função de busca de dimensão abrange apenas uma dúzia ou mais de linhas de código.

Vamos dar uma olhada peça por peça. Primeiro, configuramos os elementos para testar: a <div> elemento que contém um monte de texto com duas imagens incorporadas, justificado à esquerda por estilos em uma folha de estilo externa. Esses elementos de imagem serão os objetos de nossos testes; um está visível e o outro está oculto.

Antes de executar qualquer script, os elementos aparecem conforme mostrado na figura 12.4. Se a segunda imagem não estivesse escondida, ela apareceria como um segundo ninja logo à direita do visível.

Em seguida, começamos a definir nossa nova função. Vamos usar um hash para algumas informações importantes, mas não queremos poluir o namespace global com esse hash; queremos que ele esteja disponível para a função em seu escopo local, mas não além disso.

Conseguimos isso encerrando a definição de hash e a função de declaração dentro de uma função imediata, que cria um escopo local. O hash não está acessível fora da função imediata, mas o `getDimensions` A função que também definimos na função imediata tem acesso ao hash por meio de seu encerramento. Legal, hein?

```

(função(){
    const PROPERTIES = {
        posição: "absoluta",
        visibilidade: "oculto",
        display: "bloquear"
    };
    window.getDimensions = element => {
        const anterior = {};
        para (deixe digitar PROPERTIES) {
            anterior [chave] = element.style [chave];
            element.style [chave] = PROPRIEDADES [chave];
        }
    }
}

```

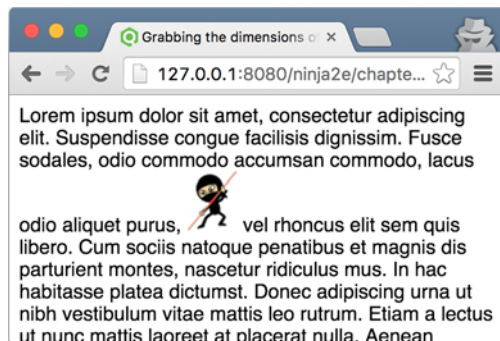


Figura 12.4 Usaremos duas imagens - uma visível, uma oculta - para testar a obtenção de dimensões de elementos ocultos.

```

const result = {
  largura: element.offsetWidth,
  height: element.offsetHeight
};
para (deixe digitar PROPERTIES) {
  element.style [chave] = anterior [chave];
}
}
resultado de retorno;
};
}) ();

```

Nossa nova função de busca de dimensão é então declarada, aceitando o elemento a ser medido. Dentro dessa função, primeiro criamos um hash chamado anterior no qual registraremos os valores anteriores das propriedades de estilo que iremos piscar, para que possamos restaurá-los mais tarde. Fazendo um loop nas propriedades de substituição, registramos cada um de seus valores anteriores e substituímos esses valores pelos novos.

Feito isso, estamos prontos para medir o elemento, que agora faz parte do layout de exibição, invisível e posicionado absolutamente. As dimensões são registradas em um hash atribuído à variável local resultado.

Agora que roubamos o que buscamos, apagamos nossos rastros, restaurando os valores originais das propriedades de estilo que modificamos e retornamos os resultados como um hash contendo largura e altura propriedades.

Tudo bem, mas funciona? Vamos descobrir.

Em um gerenciador de carga, realizamos os testes em um retorno de chamada para um temporizador de 3 segundos. Porque você pergunta? O gerenciador de carga garante que não realizemos o teste até que saibamos que o DOM foi construído, e o cronômetro nos permite observar a tela enquanto o teste está em execução, para garantir que nenhuma falha de exibição ocorra enquanto mexemos nas propriedades do elemento oculto. Afinal, se a tela for perturbada de alguma forma quando executamos nossa função, é um fracasso.

No retorno de chamada do cronômetro, primeiro obtemos uma referência aos nossos assuntos de teste (as duas imagens) e afirmamos que podemos obter as dimensões da imagem visível usando as propriedades de deslocamento. Esse teste passa, o que podemos ver se dermos uma olhada na figura 12.5.

Em seguida, fazemos o mesmo teste no elemento oculto, assumindo incorretamente que as propriedades de deslocamento funcionarão com uma imagem oculta. Não é de surpreender que, como já reconhecemos que isso não vai funcionar, o teste falha.

Em seguida, chamamos nossa nova função na imagem oculta e, em seguida, testamos novamente com esses resultados. Sucesso! Nosso teste passa, conforme mostrado na figura 12.5.

Se observarmos a exibição da página enquanto o teste está sendo executado - lembre-se, atrasamos a execução do teste até 3 segundos depois que o DOM é carregado - podemos ver que a exibição não é perturbada de nenhuma forma por nossos bastidores ajustes das propriedades do elemento oculto.

GORJETA Verificando o `offsetWidth` e `offsetHeight` propriedades de estilo para zeros podem servir como um meio incrivelmente eficiente de determinar a visibilidade de um elemento.

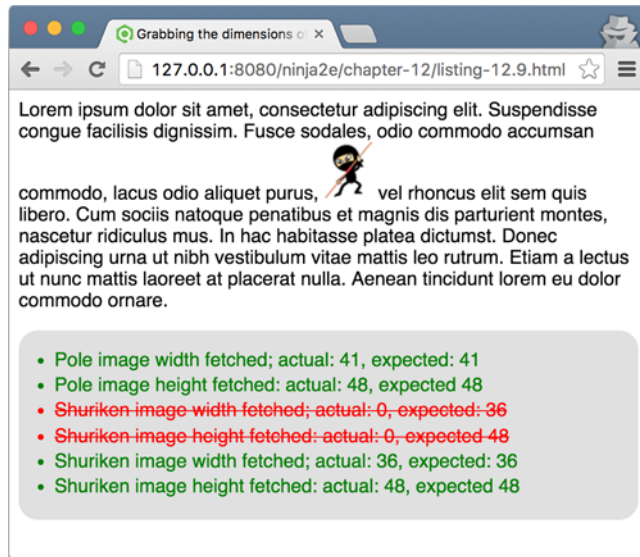


Figura 12.5 Temporariamente ajustando as propriedades de estilo dos elementos ocultos, podemos obter com sucesso suas dimensões.

12.4 Minimizando o thrashing do layout

Até agora neste capítulo, você aprendeu como modificar o DOM com relativa facilidade: criando e inserindo novos elementos, removendo elementos existentes ou modificando seus atributos. Modificar o DOM é uma das ferramentas fundamentais para obter aplicativos da web altamente dinâmicos.

Mas essa ferramenta também vem com asteriscos de uso, sendo um dos mais importantes, esteja ciente de *layout thrashing*. O thrashing de layout ocorre quando executamos uma série de leituras e gravações consecutivas no DOM, no processo, não permitindo que o navegador execute otimizações de layout.

Antes de nos aprofundarmos, considere que alterar os atributos de um elemento (ou modificar seu conteúdo) não afeta necessariamente apenas aquele elemento; em vez disso, pode causar uma cascata de mudanças. Por exemplo, definir a largura de um elemento pode levar a alterações nos filhos, irmãos e pais do elemento. Portanto, sempre que uma alteração é feita, o navegador deve calcular o impacto dessas alterações. Em certos casos, não há nada que possamos fazer a respeito; precisamos que essas mudanças ocorram. Mas, ao mesmo tempo, não há necessidade de colocar peso adicional sobre os ombros de nossos navegadores fracos, fazendo com que o desempenho de nosso aplicativo da web diminua.

Como recalculer o layout é caro, os navegadores tentam ser o mais preguiçosos possível, atrasando o trabalho com o layout o máximo que podem; eles tentam colocar em lote tantas operações de gravação quanto possível no DOM em uma fila para que essas operações possam ser executadas de uma vez. Então, quando surge uma operação que requer um layout atualizado, o navegador obedece a contragosto, executa todas as operações em lote e, finalmente, atualiza o layout. Mas às vezes, a maneira como vamos escrever nosso código não dá ao navegador espaço suficiente para realizar esses tipos de otimizações, e forçamos o

navegador para realizar muitos recálculos (possivelmente desnecessários). É disso que trata o layout; ocorre quando nosso código executa uma série de leituras e gravações consecutivas (muitas vezes desnecessárias) no DOM, não permitindo que o navegador otimize as operações de layout. O problema é que, sempre que modificamos o DOM, o navegador precisa recalcular o layout antes que qualquer informação de layout seja lida. Essa ação é cara, em termos de desempenho. Vamos dar uma olhada em um exemplo.

Listagem 12.10 Séries consecutivas de leituras e gravações causam problemas de layout

```
<div id = "ninja"> Sou um ninja </div> <div id = "samurai"> Sou um  
samurai </div> <div id = "ronin"> Sou um ronin </div> <script>
```

Define alguns elementos HTML

```
const ninja = document.getElementById ("ninja"); const samurai = document.getElementById  
("samurai"); const ronin = document.getElementById ("ronin");
```

Coleta os elementos
do DOM

```
const ninjaWidth = ninja.clientWidth; ninja.style.width = ninjaWidth / 2 + "px";
```

```
const samuraiWidth = samurai.clientWidth; samurai.style.width = samuraiWidth / 2 +  
"px";
```

```
const roninWidth = ronin.clientWidth; ronin.style.width = roninWidth / 2 + "px";  
</script>
```

Executa uma série de leituras
consecutivas e
escreve. Modificações DOM
invalidar o layout.

Lendo o valor do elemento `clientWidth` propriedade é uma daquelas ações que exige que o navegador tenha um layout atualizado. Ao realizar leituras e gravações consecutivas na propriedade `width` de diferentes elementos, não permitimos que o navegador seja preguiçoso de maneira inteligente. Em vez disso, como lemos as informações de layout após cada modificação de layout, o navegador precisa recalcular o layout todas as vezes, apenas para ter certeza de que ainda obteremos as informações corretas.

Uma forma de minimizar o thrashing de layout é criar um código de maneira que não cause recálculos de layout desnecessários. Por exemplo, podemos reescrever a listagem 12.10 no seguinte.

Listagem 12.11 Leituras e gravações Batch DOM para evitar sobrecarga de layout

```
<div id = "ninja"> Sou um ninja </div> <div id = "samurai"> Sou um  
samurai </div> <div id = "ronin"> Sou um ronin </div> <script>
```

```
const ninja = document.getElementById ("ninja"); const samurai = document.getElementById  
("samurai"); const ronin = document.getElementById ("ronin");
```

```
const ninjaWidth = ninja.clientWidth; const samuraiWidth = samurai.clientWidth;  
const roninWidth = ronin.clientWidth;
```

Coloca em lote todas as leituras
para propriedades de layout juntas

```
ninja.style.width = ninjaWidth / 2 + "px"; samurai.style.width = samuraiWidth / 2 +  
"px"; ronin.style.width = roninWidth / 2 + "px"; </script>
```

Coloca em lote todas as gravações nas propriedades de layout juntas

Aqui, agrupamos todas as leituras e gravações, porque sabemos que não existem dependências entre as dimensões de nossos elementos; definindo a largura do ninja elemento não influencia a largura do samurai elemento. Isso permite que o navegador preguiçosamente em lote operações que modificam o DOM.

A alteração do layout não é algo que você notaria em páginas menores e mais simples, mas é algo para se manter em mente ao desenvolver aplicativos da web complexos, especialmente em dispositivos móveis. Por esse motivo, é sempre bom ter em mente os métodos e propriedades que requerem um layout atualizado, mostrados na tabela a seguir (obtida em <http://ricostacruz.com/cheatsheets/layout-thrashing.html>)

Tabela 12.2 APIs e propriedades que causam invalidação de layout

Interface	Nome da propriedade
Elemento	clientHeight, clientLeft, clientTop, clientWidth, focus, getBoundingClientRect, getClientRects, innerText, offsetHeight, offsetLeft, offsetParent, offsetTop, offsetWidth, outerText, scrollByLines, scrollByPages, scrollHeight, scrollIntoView, scrollIntoViewIfNeeded, scrollLeft, scrollTop, scrollWidth
MouseEvent	layerX, layerY, offsetX, offsetY
Janela	getComputedStyle, scrollBy, scrollTo, scroll, scrollY height, width
Quadro, Documento, Imagem	

Várias bibliotecas que tentam minimizar o thrashing de layout foram desenvolvidas. Um dos mais populares é o FastDom (<https://github.com/wilsonpage/fastdom>) O repositório da biblioteca inclui exemplos que mostram claramente os ganhos de desempenho que podem ser alcançados por lote de operações de leitura / gravação DOM (<https://wilsonpage.github.io/fastdom/examples/aspect-ratio.html>)

DOM virtual do React

Uma das bibliotecas mais populares do lado do cliente é o React do Facebook (<https://facebook.github.io/react/>) O React alcança um ótimo desempenho usando um DOM virtual, um conjunto de objetos JavaScript que imitam o DOM real. Quando desenvolvemos aplicações no React, realizamos todas as modificações no DOM virtual, sem qualquer preocupação com a alteração do layout. Então, em um momento apropriado, o React usa o DOM virtual para descobrir quais alterações devem ser feitas no DOM real, a fim de manter a IU em sincronia. Esse lote de atualizações aumenta o desempenho dos aplicativos.

12.5 Resumo

- A conversão de uma string HTML em elementos DOM inclui as seguintes etapas:
 - Certificando-se de que a string HTML é um código HTML válido
 - Envolvendo-o em marcação envolvente, exigido pelas regras do navegador
 - Inserir o HTML em um elemento DOM fictício por meio do `innerHTML` propriedade de um elemento DOM
 - Extrair os nós DOM criados de volta
- Para inserção rápida de nós DOM, use fragmentos DOM, porque um fragmento pode ser injetado em uma única operação, reduzindo drasticamente o número de operações.
- Os atributos e propriedades do elemento DOM, embora vinculados, nem sempre são idênticos! Podemos ler e gravar em atributos DOM usando o `getAttribute` e `setAttribute` métodos, enquanto gravamos nas propriedades do DOM usando a notação de propriedade do objeto.
- Ao trabalhar com atributos e propriedades, devemos estar cientes de *atributos personalizados*. Os atributos que decidimos colocar em elementos HTML para transportar informações úteis para nossos aplicativos não são apresentados automaticamente como propriedades de elementos.
- O estilo propriedade do elemento é um objeto que contém propriedades correspondentes aos valores de estilo especificados na marcação do elemento. Para obter os estilos calculados, que também levam em consideração os estilos definidos nas folhas de estilo, use o recurso integrado `getComputedStyle` método.
- Para obter as dimensões dos elementos HTML, use `offsetWidth` e `offsetHeight` propriedades.
- O thrashing do layout ocorre quando o código executa uma série de leituras e gravações consecutivas no DOM, sempre forçando o navegador a recalculas as informações do layout. Isso leva a aplicativos da web mais lentos e menos responsivos.
- Reúna suas atualizações de DOM!

12.6 Exercícios

- 1 No código a seguir, qual das afirmações a seguir será aprovada?

```
<div id = "samurai"> </div>
<script>
    elemento const = document.querySelector ("# samurai");

    assert (element.id === "samurai", "id da propriedade é samurai"); assert (element.getAttribute ("id") === "samurai",

        "id do atributo é samurai");

    element.id = "novoSamurai";
```

```
assert (element.id === "newSamurai", "id da propriedade é newSamurai"); assert (element.getAttribute ("id") ===  
"newSamurai",  
"id do atributo é newSamurai");  
</script>
```

2 Dado o código a seguir, como podemos acessar o elemento largura da borda estilo

propriedade?

```
<div id = "element" style = "border-width: 1px;  
estilo de borda: sólido; border-color: red ">  
</div>  
<script>  
elemento const = document.querySelector ("# elemento"); </script>
```

- a `element.border-width`
- b `element.getAttribute ("border-width");`
- c `element.style ["border-width"];`
- d `element.style.borderWidth;`

3 Qual método embutido pode obter todos os estilos aplicados a um determinado elemento (estilos pro
fornecidos pelo navegador, estilos aplicados por meio de folhas de estilo e propriedades definidas por meio do atributo style)?

- a `getStyle`
- b `getAllStyles`
- c `getComputedStyle`

4 Quando ocorre a alteração do layout?

13

Sobrevivendo eventos

Este capítulo cobre

- Compreender o ciclo de eventos
- Processando tarefas complexas com cronômetros
- Gerenciando animações com temporizadores
- Usando bolha de evento e delegação
- Usando eventos personalizados

O Capítulo 2 incluiu uma breve discussão sobre o modelo de execução de thread único JavaScript e introduziu o loop de eventos e a fila de eventos, nos quais os eventos aguardam sua vez de serem processados. Essa discussão foi particularmente útil ao apresentar as etapas do ciclo de vida de uma página da web, especialmente ao discutir a ordem em que certas partes do código JavaScript são executadas. Ao mesmo tempo, é uma simplificação; portanto, para obter uma imagem mais completa de como o navegador funciona, passaremos uma parte significativa deste capítulo explorando os cantos e recantos do loop de eventos. Isso nos ajudará a entender melhor algumas das limitações de desempenho inerentes ao JavaScript e ao navegador. Por sua vez, usaremos esse conhecimento para desenvolver aplicativos de execução mais suave.

Durante esta exploração, colocaremos um foco especial em cronômetros, um recurso JavaScript que nos permite atrasar a execução de um trecho de código de forma assíncrona por um determinado período de tempo. À primeira vista, pode não parecer muito, mas mostraremos como usar cronômetros para dividir tarefas de longa execução que tornam os aplicativos lentos e sem resposta em tarefas menores que não obstruem o navegador. Isso ajuda a desenvolver aplicativos com melhor desempenho.

Continuaremos esta exploração de eventos, mostrando como os eventos são propagados através da árvore DOM e como usar esse conhecimento para escrever código mais simples e com menos memória intensiva. Por fim, concluiremos o capítulo criando eventos personalizados, que podem ajudar a reduzir o acoplamento entre as diferentes partes do aplicativo. Sem mais delongas, vamos começar a percorrer o loop de eventos.

.....

Por que o tempo nos retornos de chamada do temporizador não é garantido? Se um `setInterval` cronômetro dispara a cada 3 milissegundos enquanto

outro manipulador de eventos está sendo executado por 16 ms, quantas vezes a função

Você sabe?

de retorno de chamada do temporizador será adicionada à fila de microtarefa?

Por que o contexto da função para um manipulador de eventos é
vezes diferente do alvo do evento?

.....

13.1 Mergulhando no ciclo de eventos

Como você deve ter percebido, o loop de eventos é mais complicado do que sua apresentação no capítulo 2. Para começar, em vez de uma única fila de eventos, que contém apenas eventos, o loop de eventos tem pelo menos duas filas que, além dos eventos, retêm outras ações realizadas pelo navegador. Essas ações são chamadas *tarefas* e são agrupados em duas categorias: *macrotarefas* (ou muitas vezes apenas chamados de tarefas) e *microtarefas*.

Exemplos de macrotarefas incluem a criação do objeto de documento principal, análise de HTML, execução de código JavaScript da linha principal (ou global), alteração da URL atual, bem como vários eventos, como carregamento de página, entrada, eventos de rede e eventos de cronômetro. Da perspectiva do navegador, uma macrotarefa representa uma unidade de trabalho independente e independente. Depois de executar uma tarefa, o navegador pode continuar com outras atribuições, como renderizar novamente a IU da página ou executar a coleta de lixo.

Microtarefas, por outro lado, são tarefas menores que atualizam o estado do aplicativo e devem ser executadas antes que o navegador continue com outras atribuições, como renderizar novamente a IU. Os exemplos incluem retornos de chamada de promessa e alterações de mutação DOM. As microtarefas devem ser executadas o mais rápido possível, de forma assíncrona, mas sem o custo de executar uma macrotarefa totalmente nova. Microtarefas nos permitem executar certas ações *antes* a IU é renderizada novamente, evitando assim a renderização desnecessária da IU que pode mostrar o estado inconsistente do aplicativo.

NOTA A especificação ECMAScript não menciona loops de eventos. Em vez disso, o loop de eventos é detalhado na especificação HTML (<https://html.spec.whatwg.org/#event-loops>), que também discute o conceito de macrotarefas e microtarefas. A especificação ECMAScript menciona *empregos*

(que são análogos a microtarefas) em relação ao tratamento de retornos de chamada de promessa (<http://mng.bz/fOIK>). Mesmo que o loop de eventos seja definido na especificação HTML, outros ambientes, como Node.js, também o usam.

A implementação de um loop de eventos *deve* usar pelo menos uma fila para macrotarefas e pelo menos uma fila para microtarefas. Implementações de loop de evento geralmente vão além disso e têm *vários* filas para diferentes tipos de macro e microtarefas. Isso permite que o loop de eventos priorize tipos de tarefas; por exemplo, dar prioridade a tarefas sensíveis ao desempenho, como entrada do usuário. Por outro lado, como existem muitos navegadores e ambientes de execução de JavaScript na selva, você não deve se surpreender se encontrar loops de eventos com apenas uma única fila para os dois tipos de tarefas juntos.

O loop de eventos é baseado em dois princípios fundamentais:

- As tarefas são tratadas uma de cada vez.
- Uma tarefa é executada até a conclusão e não pode ser interrompida por outra tarefa.

Vamos dar uma olhada na figura 13.1, que descreve esses dois princípios.

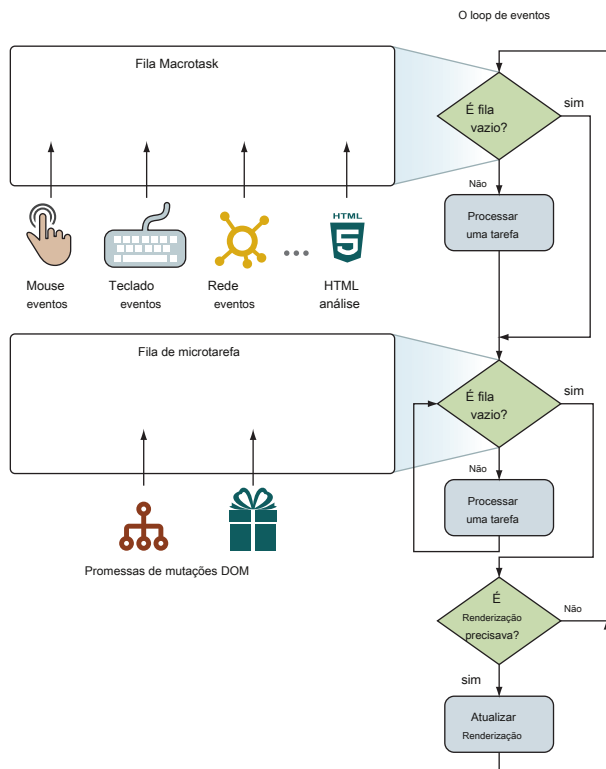


Figura 13.1 O loop de eventos geralmente tem acesso a pelo menos duas filas de tarefas: fila amicrotask e fila amacrotask. Ambos os tipos de tarefas são realizados um de cada vez.

Em um alto nível, a figura 13.1 mostra que em uma única iteração, o loop de eventos primeiro verifica a fila de macro tarefa e, se houver uma macro tarefa esperando para ser executada, inicia sua execução. Somente depois que a tarefa for totalmente processada (ou se não houver tarefas na fila), o loop de eventos passa para o processamento da fila de micro tarefas. Se houver uma tarefa esperando nessa fila, o loop de eventos a pega e a executa até a conclusão. Isso é executado para todas as micro tarefas na fila. Observe a diferença entre lidar com as filas de macro tarefa e micro tarefa: em uma iteração de loop único, uma macro tarefa é processada no máximo (outras são deixadas esperando na fila), enquanto todas as micro tarefas são processadas.

Quando a fila de micro tarefa está finalmente vazia, o loop de eventos verifica se uma atualização de renderização da IU é necessária e, se for, a IU é renderizada novamente. Isso termina a iteração atual do loop de eventos, que volta ao início e verifica a fila de macro tarefa novamente.

Agora que temos um entendimento de alto nível do loop de eventos, vamos verificar alguns dos detalhes interessantes mostrados na figura 13.1:

- *Ambas as filas de tarefas são colocadas fora do loop de eventos*, para indicar que o ato de adicionar tarefas às filas correspondentes ocorre fora do loop de eventos. Se não fosse esse o caso, quaisquer eventos que ocorressem enquanto o código JavaScript estivesse sendo executado seriam ignorados. Como definitivamente não queremos fazer isso, os atos de detectar e adicionar tarefas são feitos separadamente do loop de eventos.
- *Ambos os tipos de tarefas são executados um de cada vez*, porque o JavaScript é baseado em um modelo de execução de thread único. Quando uma tarefa começa a ser executada, ela é executada até sua conclusão, sem ser interrompida por outra tarefa. Apenas o navegador pode interromper a execução de uma tarefa; por exemplo, se a tarefa começa a ser muito egoísta por ocupar muito tempo ou memória.
- *Todas as micro tarefas devem ser executadas antes da próxima renderização*, porque seu objetivo é atualizar o estado do aplicativo antes de ocorrer a renderização.
- *O navegador geralmente tenta renderizar a página 60 vezes por segundo*, para atingir 60 quadros por segundo (60 fps), uma taxa de quadros geralmente considerada ideal para movimentos suaves, como animações— *ou seja, o navegador tenta renderizar um quadro a cada 16 ms.*
Observe como a ação “Atualizar renderização”, mostrada na figura 13.1, acontece dentro do loop de eventos, porque o conteúdo da página não deve ser modificado por outra tarefa enquanto a página está sendo renderizada. Isso tudo significa que, se quisermos obter aplicativos de execução suave, não temos muito tempo para processar tarefas em uma única iteração de loop de evento. *Idealmente, uma única tarefa e todas as micro tarefas geradas por ela devem ser concluídas em 16 ms.*

Agora, vamos considerar três situações que podem ocorrer na próxima iteração do loop de evento, depois que o navegador tiver concluído uma renderização de página:

- O loop de eventos atinge “É necessário renderizar?” ponto de decisão antes que outros 16 ms tenham decorrido. Como atualizar a IU é uma operação complexa, se não houver uma necessidade explícita de renderizar a página, o navegador pode optar por não realizar a renderização da IU nesta iteração de loop.

- O loop de eventos atinge “É necessário renderizar?” ponto de decisão cerca de 16 ms após a última renderização. Nesse caso, o navegador atualiza a IU e os usuários experimentam um aplicativo de execução suave.
- A execução da próxima tarefa (e todas as microtarefas relacionadas) leva muito mais do que 16 ms. Nesse caso, o navegador não será capaz de renderizar novamente a página na taxa de quadros desejada e a IU não será atualizada. Se a execução do código da tarefa não levar muito tempo (mais do que algumas centenas de milissegundos), esse atraso pode nem ser perceptível, especialmente se não houver muito movimento acontecendo na página. Por outro lado, se demormos muito ou se houver animações em execução na página, os usuários provavelmente perceberão a página da Web como lenta e sem resposta. Na pior das hipóteses, em que uma tarefa é executada por mais de alguns segundos, o navegador do usuário mostra a temida mensagem “Script sem resposta”. (Não se preocupe, mais adiante neste capítulo mostraremos a você uma técnica para dividir tarefas complexas em tarefas menores que não obstruem o loop de eventos.)

NOTA Tenha cuidado com os eventos que você decide tratar, com que frequência eles ocorrem e quanto tempo de processamento um tratador de eventos leva. Por exemplo, você deve ser extremamente cuidadoso ao lidar com eventos de movimento do mouse. Mover o mouse faz com que um grande número de eventos sejam enfileirados, portanto, realizar qualquer operação complexa nesse manipulador de movimento do mouse é um caminho certo para a construção de um aplicativo da Web lento e irregular.

Agora que descrevemos como funciona o loop de eventos, você está pronto para explorar alguns exemplos em detalhes.

13.1.1 *Um exemplo com apenas macrotarefas*

O resultado inevitável do modelo de execução de thread único do JavaScript é que apenas uma tarefa pode ser executada por vez. Isso, por sua vez, significa que todas as tarefas criadas precisam esperar em uma fila até que chegue sua vez de execução.

Vamos concentrar nossa atenção em uma página da web simples que contém o seguinte:

- Código JavaScript de linha principal não trivial (global)
- Dois botões e dois manipuladores de clique não triviais, um para cada botão

A lista a seguir mostra o código de amostra.

Listagem 13.1 Pseudocódigo para nossa demonstração de loop de evento com uma fila de tarefas

```
<button id = "firstButton"> </button>
<button id = "secondButton"> </button>
<script>
    const firstButton = document.getElementById ("firstButton"); const secondButton = document.getElementById
    ("secondButton"); firstButton.addEventListener ("click", function firstHandler () {/ * Algum código de identificador de
    clique executado por 8 ms * /
    });
```

Registra um manipulador de eventos para um evento de clique de botão no primeiro botão

```
secondButton.addEventListener("clique", função secondHandler () {  
  /* Clique no código do identificador que é executado por 5ms * /});  
  
/* Código que funciona por 15ms * / </script>
```

Registra outro manipulador de eventos de clique,
desta vez para o segundo botão

Este exemplo requer um pouco de imaginação, então, em vez de bagunçar o fragmento de código com código desnecessário, pedimos que você imagine o seguinte:

- Nosso código JavaScript de linha principal leva 15 ms para ser executado.
- O primeiro manipulador de eventos de clique é executado por 8 ms.
- O segundo manipulador de eventos de clique é executado por 5 ms.

Agora, vamos continuar a ser criativos e dizer que temos um usuário super rápido que clica no primeiro botão 5 ms após o início da execução do nosso script e no segundo botão 12 ms depois. A Figura 13.2 mostra essa situação.

Há muitas informações para digerir aqui, mas entendê-las completamente dará uma ideia melhor de como o loop de eventos funciona. Na parte superior da figura, o tempo

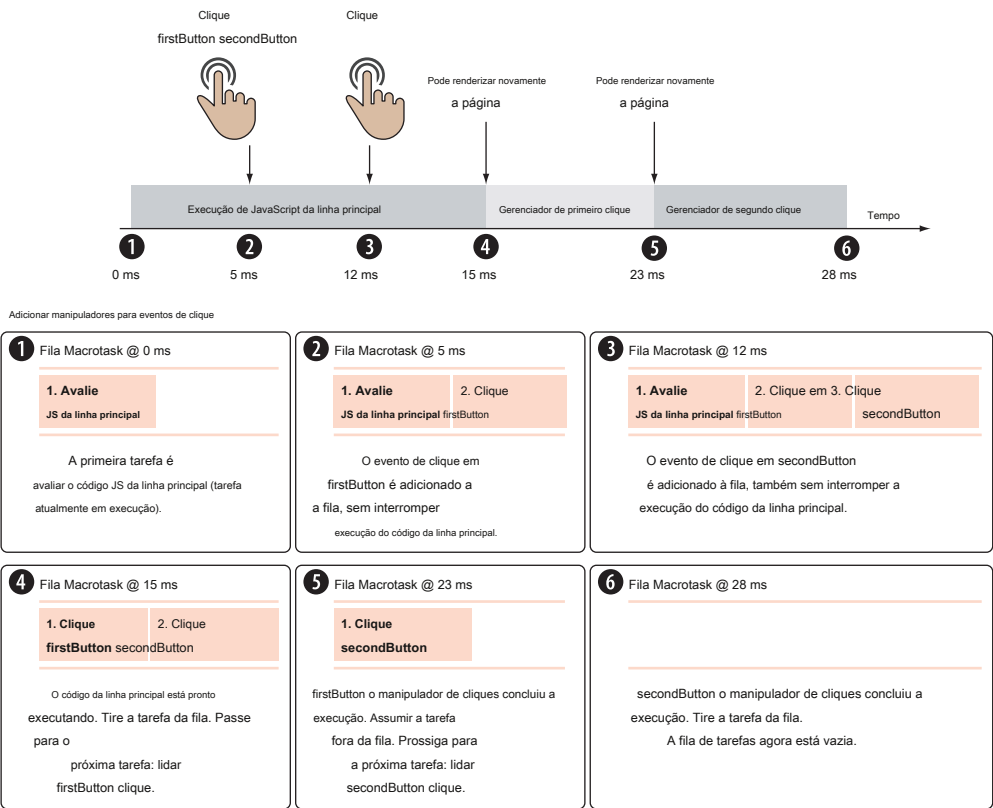


Figura 13.2 Este diagrama de tempo mostra como os eventos são adicionados à fila de tarefas à medida que ocorrem. Quando a execução de uma tarefa termina, o loop de eventos a tira da fila e continua executando a próxima tarefa.

(em milissegundos) é executado da esquerda para a direita ao longo do eixo x. Os retângulos abaixo dessa linha do tempo representam partes do código JavaScript em execução, estendendo-se pelo tempo de execução. Por exemplo, o primeiro bloco de código JavaScript de linha principal é executado por aproximadamente 15 ms, o primeiro manipulador de clique por aproximadamente 8 ms e o segundo manipulador de clique por aproximadamente 5 ms. O diagrama de tempo também mostra quando certos eventos ocorrem; por exemplo, o primeiro clique do botão ocorre 5 ms na execução do aplicativo e o segundo clique no botão 12 ms. A parte inferior da figura mostra o estado da fila de macrotarefa em vários pontos de execução do aplicativo.

O programa começa executando o código JavaScript da linha principal. Imediatamente, dois elementos, `firstButton` e `secondButton`, são buscados no DOM e duas funções, `firstHandler` e `secondHandler`, são registrados como manipuladores de eventos de clique:

```
firstButton.addEventListener("clique", função firstHandler () {...}); secondButton.addEventListener("clique", função secondHandler () {...});
```

Isso é seguido por um código que é executado por mais 15 ms. Durante esta execução, nosso usuário clica rapidamente `firstButton` 5 ms após o programa começar a ser executado e clicar `secondButton` 12 ms depois.

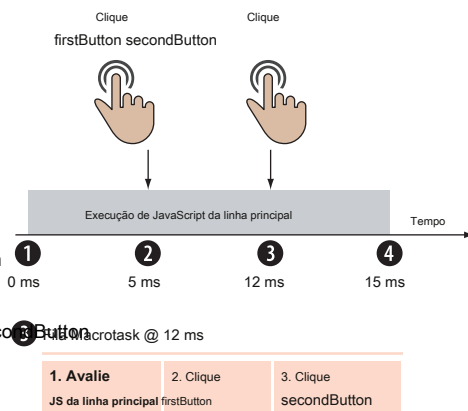
Como o JavaScript é baseado em um modelo de execução de thread único, clicar `firstButton` não significa que o manipulador de cliques seja executado imediatamente. (Lembre-se, se uma tarefa já estiver sendo executada, ela não pode ser interrompida por outra tarefa.) Em vez disso, o evento de clique relacionado a `firstButton` é colocado na fila de tarefas, onde espera pacientemente sua vez de ser executado. A mesma coisa acontece quando um clique de `secondButton`

ocorre: um evento correspondente é colocado na fila de tarefas e aguarda a execução. Observe que é importante que a detecção de eventos e adição à fila de tarefas ocorram fora do loop de eventos; as tarefas são

adicionado à fila de tarefas, mesmo enquanto o código JavaScript da linha principal está sendo executado.

Se tirarmos um instantâneo de nossa fila de tarefas 12 ms na execução de nosso script, veremos as três tarefas a seguir:

- 1 Avalie o código JavaScript da linha principal— a tarefa atualmente em execução. Clique `firstButton`
- 2 —O evento criado quando `firstButton` é clicado. Clique `secondButton` —O evento criado quando `secondButton` é clicado.
- 3



Essas tarefas também são mostradas na figura 13.3.

O próximo ponto interessante na execução do aplicativo acontece aos 15 ms, quando o código JavaScript da linha principal termina seu

Figura 13.3 12 ms na execução do aplicativo, a fila de tarefas tem três tarefas: uma para avaliar o código JavaScript da linha principal (a tarefa atualmente em execução) e uma para cada evento de clique de botão.

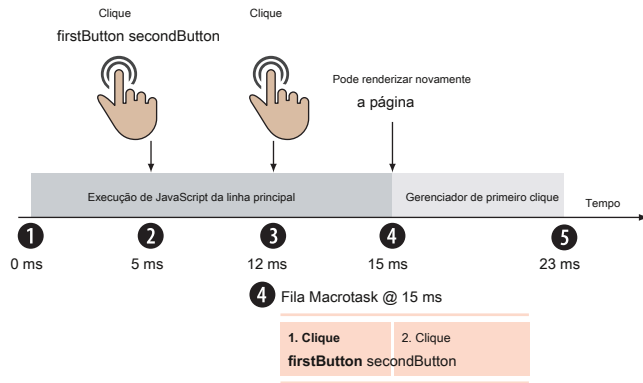


Figura 13.4 A fila de tarefas
15 ms na execução do aplicativo contém duas tarefas para eventos de clique. A primeira tarefa está sendo executada.

execução. Conforme mostrado na figura 13.1, após o término da execução de uma tarefa, o loop de eventos segue para o processamento da fila de microtarefa. Como nessa situação não temos nenhuma microtarefa (nem mesmo mostramos a fila de microtarefas no diagrama, porque ela está sempre vazia), pulamos esta etapa e passamos a atualizar a IU. Neste exemplo, embora a atualização aconteça e leve algum tempo, para simplificar, nós a mantemos fora de nossa discussão. Com isso, o loop de eventos termina a primeira iteração e inicia a segunda iteração, passando para a tarefa seguinte na fila.

A seguir, o firstButton a tarefa click inicia sua execução. A Figura 13.4 ilustra a fila de tarefas 15 ms na execução do aplicativo. A execução de firstHandler, associado com o firstButton clique, leva cerca de 8 ms, e o manipulador é executado até sua conclusão, sem interrupção, enquanto o evento de clique relacionado a secondButton está esperando na fila.

Em seguida, aos 23 ms, o firstButton O evento click é totalmente processado e a tarefa correspondente é removida da fila de tarefas. Novamente, o navegador verifica a fila de microtarefas, que ainda está vazia, e renderiza novamente a página, se necessário.

Finalmente, na terceira iteração do loop, o secondButton o evento click está sendo tratado, conforme mostrado na figura 13.5. O secondHandler leva cerca de 5 ms para ser executado e, depois de executado, a fila de tarefas está finalmente vazia, em 28 ms.

Este exemplo enfatiza que um evento deve aguardar sua vez para ser processado, se outras tarefas já estiverem sendo realizadas. Por exemplo, embora o secondButton o clique aconteceu 12 ms na execução do aplicativo, o manipulador correspondente é chamado em torno de 23 ms na execução do aplicativo.

Agora, vamos estender esse código para incluir microtarefas.

13.1.2 Um exemplo com macro e microtarefas

Agora que você viu como o loop de eventos funciona em uma fila de tarefas, estenderemos nosso exemplo para incluir também uma fila de microtarefa. A maneira mais limpa de fazer isso é incluir uma promessa no primeiro manipulador de clique de botão e o código que trata a promessa depois que ela é resolvida. Como você deve se lembrar do capítulo 6, *uma promessa é um espaço reservado para um valor que ainda não temos, mas que teremos mais tarde*; é uma garantia de que eventualmente saberemos

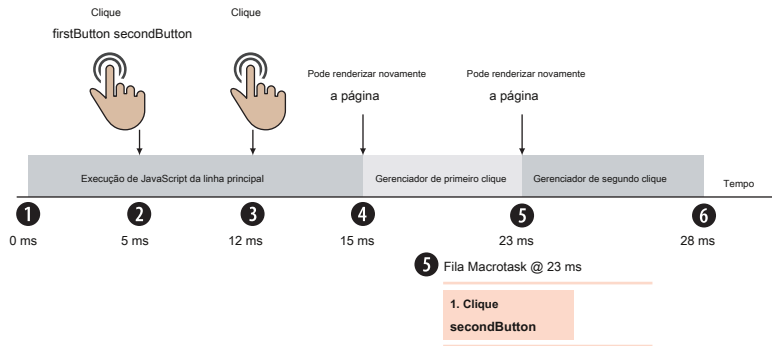


Figura 13.5 23 ms após o início da execução do aplicativo, apenas uma tarefa, tratando do **secondButton** clique no evento, continua a ser executado.

o resultado de uma computação assíncrona. Por esse motivo, manipuladores de promessa, os retornos de chamada que anexamos por meio do então método, são sempre chamados de forma assíncrona, mesmo se os anexarmos a promessas já resolvidas.

A lista a seguir mostra o código modificado para este exemplo de duas filas.

Listagem 13.2 Pseudocódigo para nossa demonstração de loop de evento com duas filas

```
<button id = "firstButton"> </button>
<button id = "secondButton"> </button>
<script>
  const firstButton = document.getElementById("firstButton"); const secondButton = document.getElementById(
    "secondButton"); firstButton.addEventListener("click", function firstHandler () {

    Promise.resolve().Then (() => {
      /* Algum código de tratamento de promessa executado por 4 ms * /});

      /* Alguns códigos de identificador de clique que funcionam por 8 ms * /});

    secondButton.addEventListener("clique", função secondHandler () {
      /* Clique no código do identificador que é executado por 5ms * /});

  /* Código que funciona por 15ms * / </script>
```

Resolve imediatamente um promessa e passa um retorno de chamada para o método then

Neste exemplo, presumimos que as mesmas ações ocorram como no primeiro exemplo:

- `firstButton` é clicado após 5 ms.
- `secondButton` é clicado após 12 ms.
- `firstHandler` lida com o evento de clique de `firstButton` e funciona por 8 ms.
- `secondHandler` lida com o evento de clique de `secondButton` e funciona por 5 ms.

A única diferença é que desta vez, dentro do `firstHandler` código, também criamos uma promessa resolvida imediatamente para a qual passamos um retorno de chamada que será executado por 4 ms. Como uma promessa representa um valor futuro que geralmente não sabemos imediatamente, os manipuladores de promessa sempre são tratados de forma assíncrona.

Para ser honesto, neste caso, onde criamos uma promessa resolvida imediatamente, o mecanismo JavaScript poderia invocar imediatamente o retorno de chamada, porque já sabemos que a promessa foi resolvida com sucesso. Mas, para fins de consistência, o mecanismo JavaScript não faz isso e, em vez disso, chama todos os retornos de chamada de promessa de forma assíncrona, após o resto do `firstHandler` o código (que funciona por 8 ms) termina a execução. Ele faz isso criando uma nova microtarefa e empurrando-a para a fila de microtarefas. Vamos explorar o diagrama de tempo dessa execução na Figura 13.6.

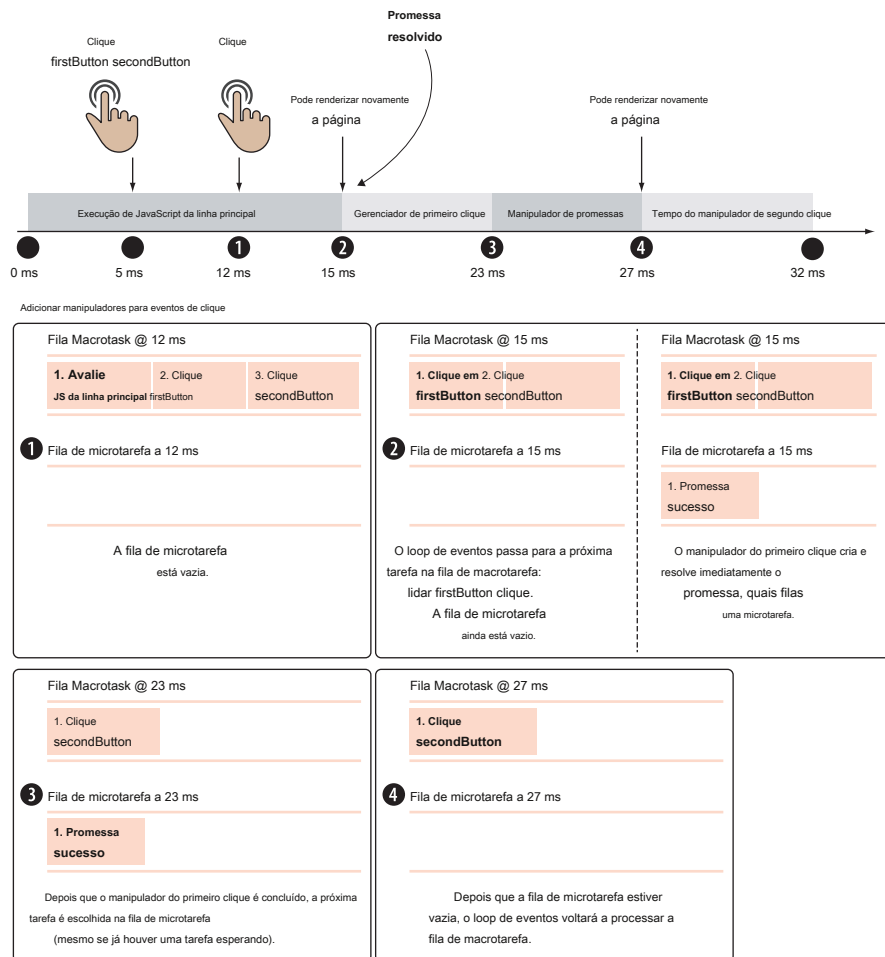


Figura 13.6 Se uma microtarefa estiver enfileirada na fila de microtarefas, ela terá prioridade e será processada mesmo se uma tarefa mais antiga já estiver esperando na fila. Neste caso, a microtarefa de sucesso da promessa tem prioridade sobre o `secondButton` clique na tarefa de evento.

Este diagrama de tempo é semelhante ao diagrama do exemplo anterior. Se tirarmos um instantâneo da fila de tarefas 12 ms na execução do aplicativo, veremos exatamente as mesmas tarefas na fila: O código JavaScript da linha principal está sendo processado enquanto as tarefas para lidar com o `firstButton` clique e o `secondButton` click estão aguardando sua vez (como na figura 13.3). Mas, além da fila de tarefas, neste exemplo também vamos nos concentrar na fila de microtarefas, que ainda está vazia 12 ms na execução do aplicativo.

O próximo ponto interessante na execução do aplicativo acontece aos 15 ms, quando a execução do JavaScript da linha principal termina. Como a execução de uma tarefa foi concluída, o loop de eventos verifica a fila de microtarefas, que está vazia, e segue para a renderização da página, se necessário. Novamente, para simplificar, não incluímos um fragmento de renderização em nosso diagrama de tempo.

Na próxima iteração do loop de eventos, a tarefa associada ao `firstButton` o clique está sendo processado:

```
firstButton.addEventListener("click", function firstHandler () {
    Promise.resolve().Then (() => {
        /* Alguns códigos de manipulação de promessa que são executados por 4ms */
    });
    /* Algum código de identificador de clique que roda por 8ms */});
```

O `firstHandler` função cria uma promessa já resolvida, chamando `Promessa` `.resolver()` com uma função de callback que com certeza será invocada, porque a promessa já foi resolvida. Isso cria uma nova microtarefa para executar o código de retorno de chamada. A microtarefa é colocada na fila de microtarefa e o manipulador de cliques continua a ser executado por mais 8 ms. O estado atual das filas de tarefas é mostrado na figura 13.7.

Revisitamos as filas de tarefas novamente 23 ms na execução do aplicativo, após o `firstButton` click foi completamente administrado e sua tarefa retirada da fila de tarefas.

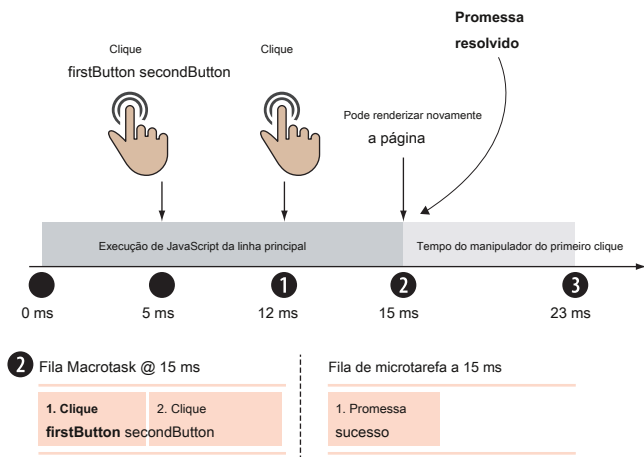


Figura 13.7 Durante a execução do manipulador do primeiro clique, uma promessa resolvida é criada. Isso coloca na fila uma promessa de sucesso de microtarefa nas microtarefas fila que será executada o mais rápido possível, mas sem interromper a tarefa em execução no momento.

Nesse ponto, o loop de eventos deve escolher qual tarefa processar em seguida. Temos uma macrotask para lidar com o `secondButton` clique que foi colocado na fila de tarefas 12 ms na execução do aplicativo e a tarefa onômica para lidar com o sucesso da promessa que foi colocada na fila de tarefas em algum lugar em torno de 15 ms na execução do aplicativo.

Se considerarmos coisas assim, seria justo que o `secondButton` a tarefa de clique é tratada primeiro, mas como já mencionamos, microtarefas são tarefas menores que devem ser executadas o mais rápido possível. As microtarefas têm prioridade e se você olhar para trás na figura 13.1, verá que cada vez que uma tarefa é processada, o loop de eventos verifica primeiro a fila de microtarefas, com o objetivo de processar todas as microtarefas antes de continuar com a renderização ou outras tarefas.

Por este motivo, a tarefa de sucesso da promessa é executada *imediatamente* depois de `firstButton` clique, mesmo com o “mais antigo” `secondButton` clique na tarefa ainda aguardando na fila de tarefas, conforme mostrado na figura 13.8.

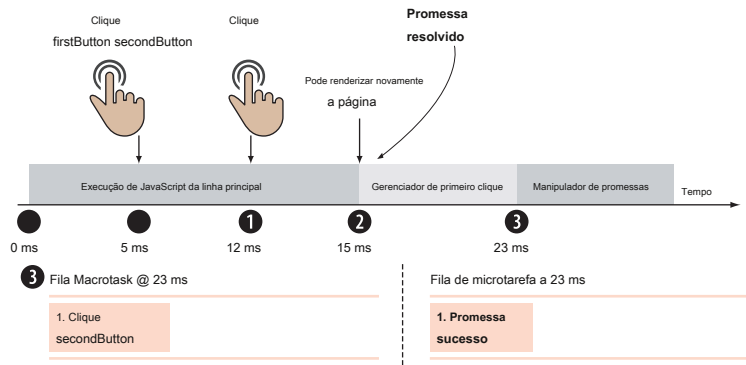


Figura 13.8 Depois que uma tarefa é executada, o loop de eventos processa todas as tarefas na fila de microtarefa. Neste caso, antes de passar para o `secondButton` clique na tarefa, a tarefa de sucesso da promessa é tratada.

Há um ponto importante que precisamos enfatizar. Depois que uma macrotask é executada, o loop de eventos imediatamente passa a lidar com a fila de microtarefas, sem permitir a renderização até que a fila de microtarefas esteja vazia. Basta dar uma olhada no diagrama de tempo na figura 13.9.

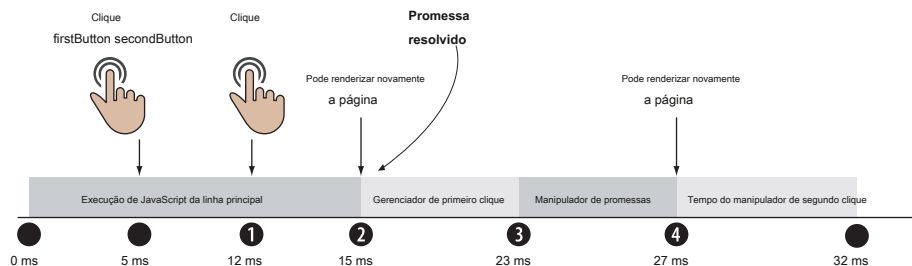


Figura 13.9 Uma página pode ser renderizada novamente entre duas macrotasks (execução de JavaScript da linha principal e manipulador de primeiro clique), enquanto *não* pode ser renderizado antes de uma microtarefa ser executada (antes do manipulador de promessa).

A Figura 13.9 mostra que uma nova renderização pode ocorrer entre duas macrotarefas, apenas se não houver microtarefas entre elas. Em nosso caso, a página pode ser renderizada entre a execução do JavaScript de linha principal e o manipulador do primeiro clique, mas não pode ser renderizada imediatamente após o manipulador do primeiro clique, porque microtarefas, como manipuladores de promessa, têm prioridade sobre a renderização.

Uma renderização também pode ocorrer após uma microtarefa, mas apenas se nenhuma outra microtarefa estiver esperando na fila de microtarefa. Em nosso exemplo, depois que o manipulador de promessa ocorre, mas antes que o loop de eventos passe para o manipulador de segundo clique, o navegador pode renderizar novamente a página.

Observe que não há nada que impeça a promessa de microtarefa de sucesso de enfileirar outras microtarefas, e que todas essas microtarefas terão prioridade sobre o `secondButton` clique na tarefa. O loop de eventos irá renderizar novamente a página e mover para o `secondButton` tarefa apenas quando a fila de microtarefa estiver vazia, então tome cuidado!

Agora que você entende como o loop de eventos funciona, vamos dar uma olhada em um grupo especial de eventos: temporizadores.

13.2 *Cronômetros de domesticação: tempos limite e intervalos*

Timers, um recurso frequentemente mal utilizado e mal compreendido em JavaScript, podem aprimorar o desenvolvimento de aplicativos complexos se usados corretamente. Os cronômetros nos permitem atrasar a execução de um trecho de código por *por pelo menos* um certo número de milissegundos. Usaremos esse recurso para dividir tarefas de longa execução em tarefas menores que não obstruirão o loop de eventos, interrompendo a renderização do navegador e, no processo, tornando nossos aplicativos da web lentos e sem resposta.

Mas primeiro, começaremos examinando as funções que podemos usar para construir e manipular temporizadores. O navegador oferece dois métodos para a criação de cronômetros: `setTimeout` e `setInterval`. O navegador também fornece dois métodos correspondentes para limpá-los (ou removê-los): `clearTimeout` e `clearInterval`. Todos são métodos do `global` objeto. De maneira semelhante ao loop de eventos, os temporizadores não são definidos no próprio JavaScript; em vez disso, eles são fornecidos pelo ambiente de `host` (como o navegador no cliente ou `Node.js` no servidor). A Tabela 13.1 lista os métodos para criar e limpar temporizadores.

Tabela 13.1 Métodos de manipulação de temporizador do JavaScript (métodos do `global` **janela** objeto)

Método	Formato	Descrição
<code>setTimeout</code>	<code>id = setTimeout (fn, atraso)</code>	Inicia um cronômetro que executará o retorno de chamada passado uma vez após o atraso especificado ter decorrido. Um valor que identifica exclusivamente o cronômetro é retornado.
<code>clearTimeout</code>	<code>clearTimeout (id)</code>	Cancela (limpa) o cronômetro identificado pelo valor passado se o cronômetro ainda não tiver disparado.

Tabela 13.1 Métodos de manipulação de temporizador do JavaScript (métodos do global **janela** objeto) (*continua*)

Método	Formato	Descrição
setInterval	id = setInterval (fn, atraso)	Inicia um temporizador que tentará continuamente executar o retorno de chamada passado no intervalo de atraso especificado, até ser cancelado. Um valor que identifica exclusivamente o cronômetro é retornado.
clearInterval	clearInterval (id)	Cancela (limpa) o temporizador de intervalo identificado pelo valor passado.

Esses métodos nos permitem definir e limpar temporizadores que disparam uma única vez ou disparam periodicamente em um intervalo especificado. Na prática, a maioria dos navegadores permite que você use ambos `clearTimeout` e `clearInterval` para cancelar os dois tipos de temporizadores, mas é recomendado que os métodos sejam usados em pares correspondentes, pelo menos para maior clareza.

NOTA É importante entender que *o atraso de um temporizador não é garantido*. Isso tem muito a ver com o loop de eventos, como veremos na próxima seção.

13.2.1 Execução do cronômetro dentro do loop de evento

Você já examinou exatamente o que acontece quando um evento ocorre. Mas os cronômetros são diferentes dos eventos padrão, então vamos explorar um exemplo semelhante aos que você viu até agora. A lista a seguir mostra o código usado para este exemplo.

Listagem 13.3 Pseudocódigo para nosso tempo limite e demonstração de intervalo

```
<button id = "myButton"> </button>
<script>
  setTimeout (function timeoutHandler () {
    /* Algum código de controle de tempo limite que é executado por 6 ms */}, 10);

  setInterval (function intervalHandler () {
    /* Algum código de manipulação de intervalo que é executado por 8 ms */}, 10);

  const myButton = document.getElementById ("myButton"); myButton.addEventListener ("click", function
  clickHandler () {
    /* Alguns códigos de identificador de clique que são executados por 10 ms */
  });

  /* Código que funciona por 18 ms */ </script>
```

Registra um tempo limite
que expira após 10 ms

Registra um intervalo
que expira a cada 10 ms

Registra um evento
manipulador para um
evento de clique de botão

Desta vez, temos apenas um botão, mas também registramos dois temporizadores. Primeiro, registramos um tempo limite que expira após 10 ms:

```
setTimeout (function timeoutHandler () {
  /* Algum código de manipulador de tempo limite que é executado por 6 ms */}, 10);
```

Como manipulador, esse tempo limite tem uma função que leva 6 ms para ser executada. Em seguida, também registramos um intervalo que expira a cada 10 ms:

```
setInterval (function intervalHandler () {
  /* Algum código de manipulador de intervalo que é executado por 8ms */, 10);
```

O intervalo possui um manipulador que leva 8 ms para ser executado. Continuamos registrando um manipulador de eventos de clique de botão que leva 10 ms para ser executado:

```
const myButton = document.getElementById ("myButton"); myButton.addEventListener ("click", function
clickHandler () {
  /* Algum código de manipulador de cliques que é executado por 10 ms */
});
```

Este exemplo termina com um bloco de código que é executado por cerca de 18 ms (novamente, faça um pouco de humor e imagine um código complexo aqui).

Agora, digamos que temos novamente um usuário rápido e impaciente que clica no botão 6 ms na execução do aplicativo. A Figura 13.10 mostra um diagrama de tempo dos primeiros 18 ms de execução.

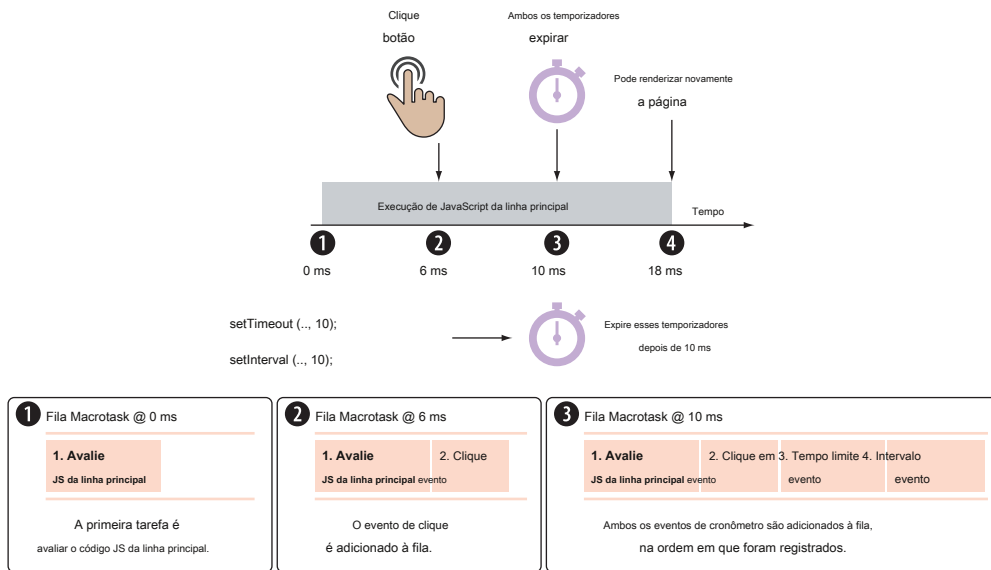


Figura 13.10 Um diagrama de tempo que mostra 18ms de execução no programa de exemplo. A primeira tarefa atualmente em execução é avaliar o código JavaScript da linha principal. Demora 18 ms para executar. Durante essa execução, três eventos ocorrem: um clique do mouse, uma expiração do cronômetro e um disparo de intervalo.

Como nos exemplos anteriores, a primeira tarefa na fila é executar o código JavaScript da linha principal. Durante essa execução, que leva cerca de 18 ms para ser concluída, três coisas importantes ocorrem:

- 1 A 0 ms, um temporizador de tempo limite é iniciado com um atraso de 10 ms e um temporizador de intervalo também é iniciado com um atraso de 10 ms. Suas referências são mantidas pelo navegador. Em 6 ms, o mouse é clicado.
- 2
- 3 Em 10 ms, o temporizador de tempo limite expira e o primeiro intervalo é disparado.

Como já sabemos por nossa exploração do loop de eventos, uma tarefa sempre é executada até a conclusão e não pode ser interrompida por outra tarefa. Em vez disso, todas as tarefas recém-criadas são colocadas em uma fila, onde pacientemente aguardam sua vez de serem processadas. Quando o usuário clica no botão 6 ms na execução do aplicativo, essa tarefa é adicionada à fila de tarefas. Algo semelhante acontece por volta dos 10 ms, quando o cronômetro expira e o intervalo é disparado. Eventos de cronômetro, assim como eventos de entrada (como eventos de mouse), são colocados na fila de tarefas. Observe que tanto o cronômetro quanto o intervalo são iniciados com um atraso de 10 ms, e que após este período, suas tarefas correspondentes são

colocado na fila de tarefas. Voltaremos a isso mais tarde, mas por agora é suficiente que você observe que as tarefas são adicionadas à fila na ordem em que os manipuladores são registrados: primeiro o manipulador de tempo limite e, em seguida, o manipulador de intervalo.

O bloco inicial de código conclui a execução após 18 ms, e como não há microtarefas nesta execução, o navegador pode renderizar novamente a página (novamente, deixada de fora de nossas discussões de tempo, devido à simplicidade) e passar para a segunda iteração de o loop de eventos. O estado da fila de tarefas neste momento é mostrado na figura 13.11.

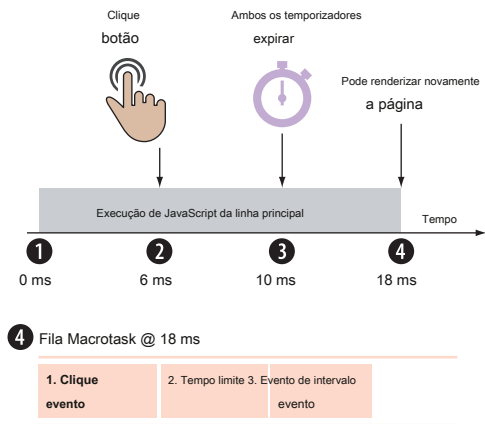


Figura 13.11 Os eventos de cronômetro são colocados na fila de tarefas à medida que expiram.

Quando o bloco inicial de código termina a execução em 18 ms, três blocos de código são enfileirados para execução: o manipulador de clique, o manipulador de tempo limite e a primeira chamada do manipulador de intervalo. Isso significa que o manipulador de cliques em espera (que presumimos que leva 10 ms para ser executado) começa a execução. A Figura 13.12 mostra outro diagrama de tempo.

Ao contrário do `setTimeout` função, que expira apenas uma vez, o `setInterval` A função é disparada até que a limpemos explicitamente. Então, por volta de 20 ms, outro intervalo é disparado. Normalmente, isso criaria uma nova tarefa e a adicionaria à fila de tarefas. Mas desta vez, como uma instância de uma tarefa de intervalo já está enfileirada e aguardando execução, essa chamada é descartada. *O navegador não enfileira mais de uma instância de um manipulador de intervalo específico.*

O manipulador de cliques é concluído em 28 ms, e o navegador pode novamente renderizar a página antes que o loop de eventos entre em outra iteração. Na próxima iteração do loop de evento, a 28 ms, a tarefa de tempo limite é processada. Mas pense no início de

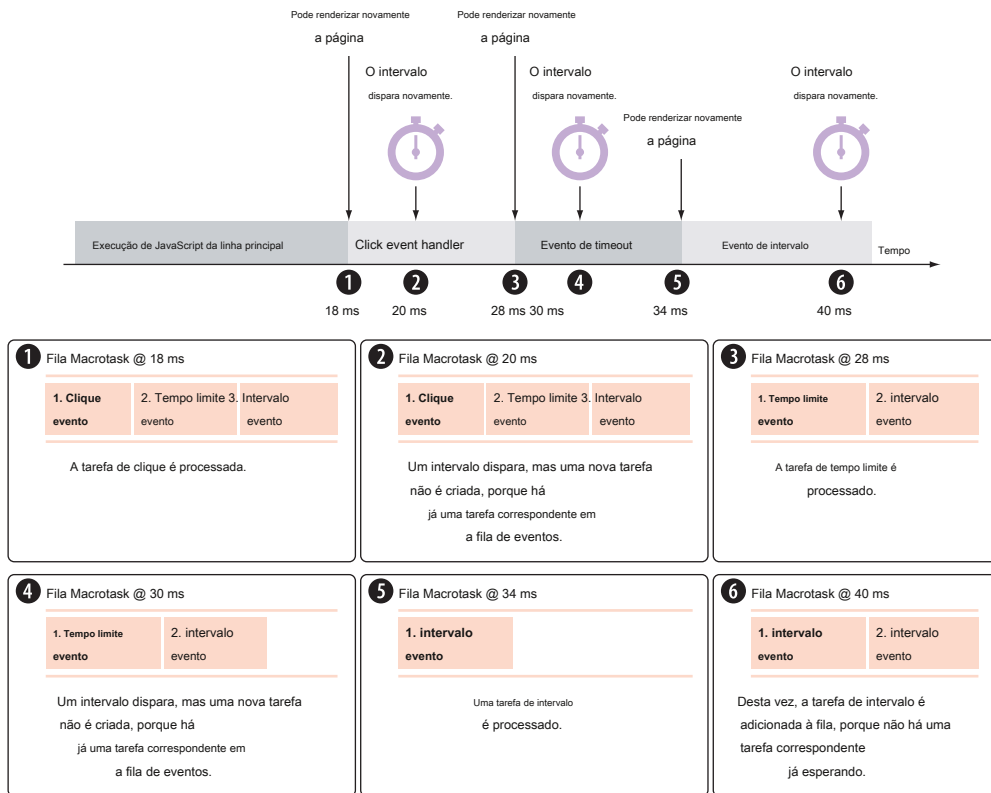


Figura 13.12 Se um evento de intervalo for disparado e uma tarefa já estiver associada a esse intervalo de espera na fila, uma nova tarefa não será adicionada. Em vez disso, nada acontece, como é mostrado para as filas de 20 ms e 30 ms.

este exemplo. Usamos a seguinte chamada de função para definir um tempo limite que deve expirar após 10 ms:

```
setTimeout (function timeoutHandler () {
  /* Algum código de controle de tempo limite que é executado por 6 ms * 10);
```

Como esta é a primeira tarefa em nosso aplicativo, não é estranho esperar que o manipulador de tempo limite seja executado exatamente após 10 ms. Mas, como você pode ver na figura 13.11, o tempo limite começa na marca de 28 ms!

É por isso que fomos extremamente cuidadosos ao dizer que um cronômetro fornece a capacidade de atrasar de forma assíncrona a execução de um trecho de código por *pelo menos* um certo número de milissegundos. Devido à natureza de thread único do JavaScript, podemos controlar apenas quando a tarefa do cronômetro é adicionada à fila, e não quando é finalmente executada! Agora que esclarecemos esse pequeno enigma, vamos continuar com o restante da execução do aplicativo.

A tarefa de time-out leva 6 ms para ser executada, portanto, deve terminar 34 ms na execução do aplicativo. Durante esse período, a 30 ms, outro intervalo é disparado, porque agendamos sua execução a cada 10 ms. Mais uma vez, nenhuma tarefa adicional é enfileirada, porque uma tarefa correspondente para a execução do manipulador de intervalo já está esperando na fila. Em 34 ms, o manipulador de tempo limite termina e o navegador novamente tem a chance de renderizar novamente a página e entrar em outra iteração do loop de eventos.

Finalmente, o manipulador de intervalo começa sua execução em 34 ms, 24 ms *depois de* a marca de 10 ms em que foi adicionado à fila de eventos. Isso novamente enfatiza que o atraso que passamos em como um argumento para as funções `setTimeout(fn, atraso)` e `setInterval(fn, atraso)` especifica apenas o atraso após o qual a tarefa correspondente é adicionada à fila, e não o tempo exato de execução.

O manipulador de intervalo leva 8 ms para ser executado, portanto, enquanto está em execução, outro intervalo expira na marca de 40 ms. Desta vez, como o gerenciador de intervalo está sendo executado (e não esperando na fila), uma nova tarefa de intervalo é finalmente adicionada à fila de tarefas e a execução de nosso aplicativo continua, conforme mostrado na figura 13.13. Configurando um `setInterval` o atraso de 10 ms não significa que terminaremos com nosso manipulador executando a cada 10 ms. Por exemplo, como as tarefas são enfileiradas e a duração da execução de uma única tarefa pode variar, os intervalos podem ser executados um após o outro, como é o caso com intervalos nas marcas de 42 e 50 ms.

Finalmente, após 50 ms, nossos intervalos se estabilizam e são executados a cada 10 ms. O conceito importante a ser aprendido é que o loop de eventos pode processar apenas uma tarefa por vez e que nunca podemos ter certeza de que os manipuladores de cronômetro serão executados exatamente quando esperamos. Isso é especialmente verdadeiro para manipuladores de intervalo. Vimos neste exemplo que, embora tenhamos programado um intervalo esperado para disparar em marcas de 10, 20, 30, 40, 50, 60 e 70 ms, os retornos de chamada foram executados em marcas de 34, 42, 50, 60 e 70 ms. Neste caso, perdemos completamente dois deles ao longo do caminho, e alguns não foram executados no tempo esperado.

Como podemos ver, os intervalos têm considerações especiais que não se aplicam a tempos limite. Vamos examiná-los mais de perto.

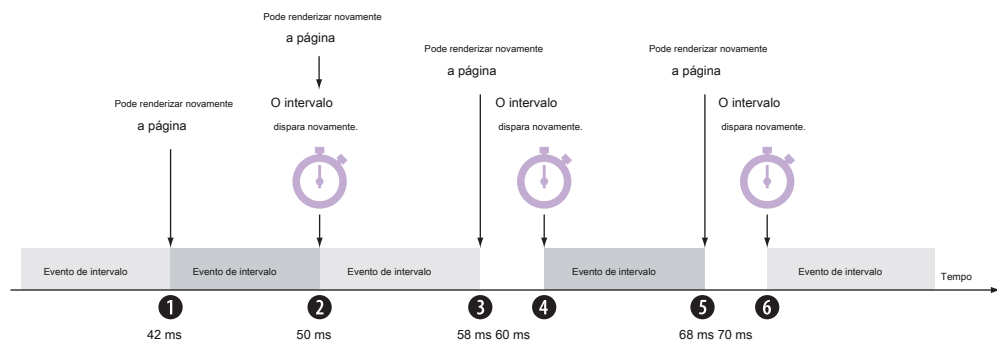


Figura 13.13 Por causa dos contratempos causados pelo clique do mouse e pelo manipulador de tempo limite, leva algum tempo para que os manipuladores de intervalo comecem a ser executados a cada 10 ms.

DIFERENÇAS ENTRE TEMPO LIMITADO E INTERVALOS

À primeira vista, um intervalo pode parecer um tempo limite que se repete periodicamente. Mas as diferenças são um pouco mais profundas. Vejamos um exemplo para ilustrar melhor a diferença ferências entre `setTimeout` e `setInterval`:

```
setTimeout (function repeatMe () {
  /* Algum bloco longo de código ... */ / setTimeout (repeatMe,
    10);
}, 10);
setInterval (() => {
  /* Algum bloco longo de código ... */ /}, 10);
```

Define um tempo limite que se reprograma a cada 10 milissegundos

Define um intervalo que dispara a cada 10 milissegundos

As duas partes do código podem *aparecer* para ser funcionalmente equivalente, mas não são. Notavelmente, o `setTimeout` variante do código sempre terá um atraso de pelo menos 10 ms após a execução do retorno de chamada anterior (dependendo do estado da fila de eventos, pode acabar sendo mais, mas nunca menos), enquanto `setInterval` tentará executar um retorno de chamada a cada 10 ms, independentemente de quando o último retorno de chamada foi executado. E, como você viu no exemplo da seção anterior, os intervalos podem ser disparados imediatamente um após o outro, independentemente do atraso.

Sabemos que o retorno de chamada de tempo limite nunca tem garantia de execução exata quando disparado. Em vez de ser disparado a cada 10 ms, conforme o intervalo, ele se reprogramará por 10 ms depois de começar a ser executado.

Tudo isso é um conhecimento extremamente importante. Saber como um mecanismo JavaScript lida com o código assíncrono, especialmente com o grande número de eventos assíncronos que normalmente ocorrem na página média, cria uma grande base para a construção de partes avançadas de código de aplicativo.

Com tudo isso sob controle, vamos ver como nossa compreensão dos temporizadores e do loop de eventos pode ajudar a evitar algumas armadilhas de desempenho.

13.2.2 Lidar com processamento computacionalmente caro

A natureza de thread único do JavaScript é provavelmente a maior pegadinha no desenvolvimento de aplicativos JavaScript complexos. Enquanto o JavaScript está ocupado em execução, a interação do usuário no navegador pode se tornar, na melhor das hipóteses, lenta e, na pior, sem resposta. O navegador pode engasgar ou parecer travar, porque todas as atualizações na renderização de uma página são suspensas enquanto o JavaScript está sendo executado.

Reduzir todas as operações complexas que levam mais do que algumas centenas de milissegundos em porções gerenciáveis se torna uma necessidade se quisermos manter a interface responsiva. Além disso, a maioria dos navegadores produzirá uma caixa de diálogo avisando ao usuário que um script ficou “sem resposta” se tiver sido executado sem interrupção por pelo menos 5 segundos, enquanto alguns outros navegadores irão até mesmo matar silenciosamente qualquer script em execução por mais de 5 segundos.

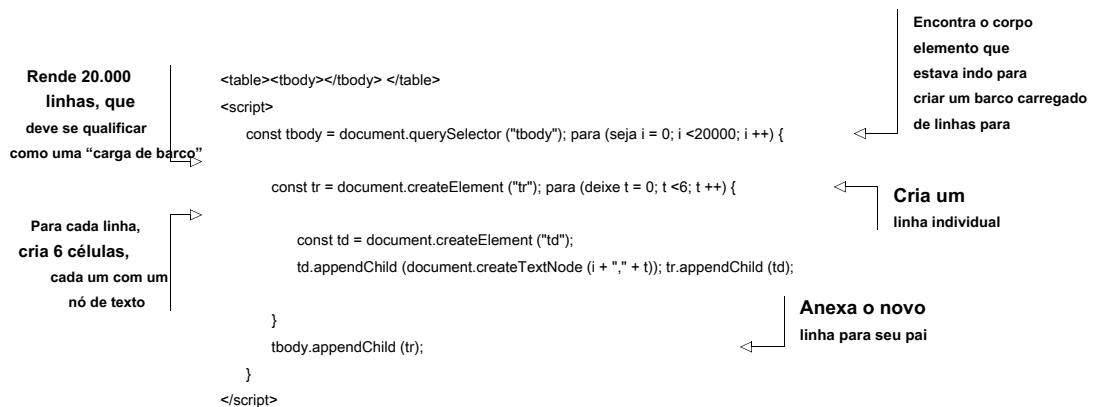
Você pode ter participado de uma reunião de família em que um tio tagarela não para de falar e insiste em contar as mesmas histórias indefinidamente. Se ninguém mais tiver a chance de interromper e dizer uma palavra, a conversa não será agradável para

ninguém (exceto o tio Bruce). Da mesma forma, o código que consome todo o tempo de processamento resulta em um resultado inferior ao desejável; produzir uma interface de usuário que não responde nunca é bom. Mas quase certamente surgirão situações que exigem que processemos uma quantidade significativa de dados, situações como a manipulação de alguns milhares de elementos DOM, por exemplo.

Nessas ocasiões, os cronômetros podem ajudar e se tornar especialmente úteis. Como os temporizadores são capazes de suspender efetivamente a execução de um trecho de JavaScript até um momento posterior, eles também podem quebrar trechos individuais de código em fragmentos que não são longos o suficiente para fazer o navegador travar. Levando isso em consideração, podemos converter loops e operações intensivas em operações não bloqueadoras.

Vejamos o seguinte exemplo de uma tarefa que provavelmente levará muito tempo.

Listagem 13.4 Uma tarefa de longa duração



Neste exemplo, estamos criando um total de 240.000 nós DOM, preenchendo uma tabela com 20.000 linhas de 6 células, cada uma contendo um nó de texto. Isso é incrivelmente caro e provavelmente irá travar o navegador por um período perceptível durante a execução, impedindo o usuário de realizar interações normais (da mesma forma que o tio Bruce domina a conversa na reunião familiar).

O que precisamos fazer é calar o tio Bruce em intervalos regulares para que outras pessoas possam ter a chance de entrar na conversa. No código, podemos introduzir temporizadores para criar apenas essas "interrupções na conversa", conforme mostrado na próxima listagem.

Listagem 13.5 Usando um cronômetro para interromper uma tarefa de longa execução

```

const rowCount = 20000; const divideInto = 4;
const chunkSize = rowCount / divideInto; deixe iteração = 0;

const table = document.getElementsByTagName ("tbody") [0]; setTimeout (function generateRows () {

    const base = chunkSize * iteração;

```

As anotações no código indicam:

- Configura os dados**: Aponta para a definição de `rowCount`, `divideInto` e `chunkSize`.
- Calcula onde paramos da última vez**: Aponta para a definição de `base`.

Horários nas próximas fase

```

para (deixe i = 0; i < chunkSize; i++) {
  const tr = document.createElement("tr"); para (deixe t = 0; t < 6; t++) {

    const td = document.createElement("td"); td.appendChild (

      document.createTextNode ((i + base) + "," + t +

        "," + iteração));

    tr.appendChild (td);
  }
  table.appendChild (tr);
}
iteração ++;
if (iteração < divideInto)
  setTimeout (generateRows, 0);
}, 0);

```

Define o atraso de tempo limite para 0 para indicar que a próxima iteração deve ser executada "assim que possível", mas depois que a IU foi atualizada

Nessa modificação do exemplo, dividimos a longa operação em quatro operações menores, cada uma criando seu próprio compartilhamento de nós DOM. Essas operações menores têm muito menos probabilidade de interromper o fluxo do navegador, conforme mostrado na figura 13.14. Observe que nós o configuramos para que os valores dos dados que controlam a operação sejam coletados em variáveis facilmente ajustáveis (rowCount, divideInto, e tamanho do pedaço), se precisarmos dividir as operações em, digamos, dez partes em vez de quatro.

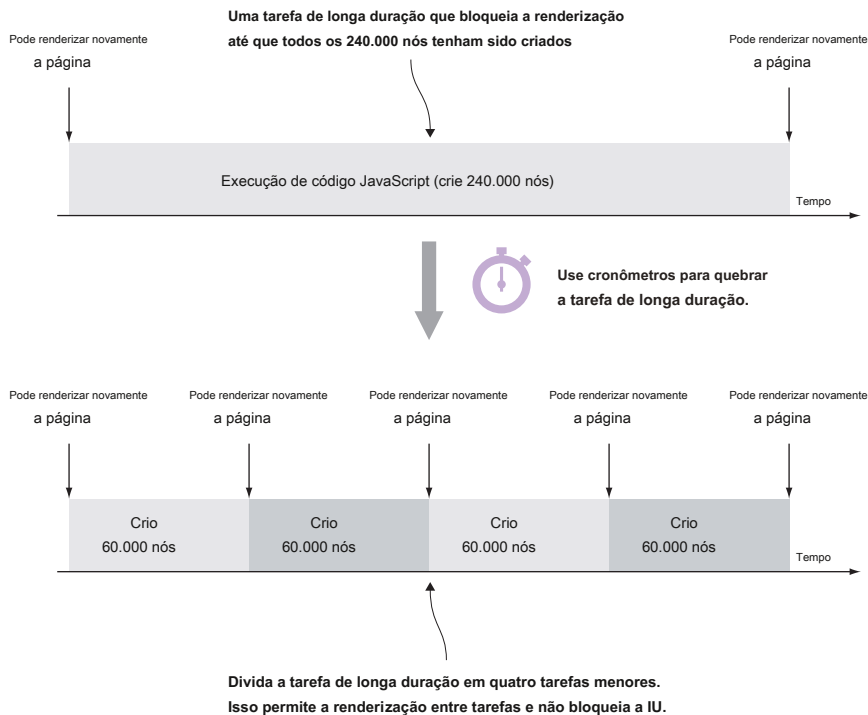


Figura 13.14 Use temporizadores para dividir tarefas de longa duração em tarefas menores que não obstruam o loop de eventos.

Também é importante observar o pouco de matemática necessária para manter o controle de onde paramos na iteração anterior, $base = chunkSize * iteration$, e como agendamos automaticamente as próximas iterações até determinarmos que terminamos:

```
if (iteração < divideInto)
  setTimeout (generateRows, 0);
```

O que é impressionante é quão pouco o código precisa mudar para acomodar essa nova abordagem assíncrona. Temos que fazer um *pouco* mais trabalho para acompanhar o que está acontecendo, para garantir que a operação seja conduzida corretamente e para programar as peças de execução. Mas, além disso, o núcleo do código é semelhante ao que começamos.

NOTA Neste caso, usamos 0 para o nosso atraso de tempo limite. Se você prestou muita atenção em como funciona o loop de eventos, sabe que isso não significa que o retorno de chamada será executado em 0 ms. Em vez disso, é uma forma de dizer ao navegador, execute este retorno de chamada o mais rápido possível; mas, ao contrário das microtarefas, você tem permissão para fazer a renderização da página no meio. Isso permite que o navegador atualize a IU e torne nossos aplicativos da web mais responsivos.

A mudança mais perceptível resultante dessa técnica, da perspectiva do usuário, é que um longo travamento do navegador é substituído por quatro (ou quantas quisermos) atualizações visuais da página. Embora o navegador tente executar os segmentos de código o mais rápido possível, ele também renderizará as alterações do DOM após cada etapa do cronômetro. Na versão original do código, ele precisava esperar por uma grande atualização em massa.

Na maior parte do tempo, esses tipos de atualizações são imperceptíveis para o usuário, mas é importante lembrar que eles ocorrem. Devemos nos esforçar para garantir que qualquer código que introduzimos na página não interrompa perceptivelmente a operação normal do navegador.

Muitas vezes é surpreendente como essa técnica pode ser útil. Ao compreender como o loop de eventos funciona, podemos contornar as restrições do ambiente do navegador de thread único, ao mesmo tempo que oferecemos uma experiência agradável ao usuário.

Agora que entendemos o loop de eventos e as funções que os temporizadores podem desempenhar ao lidar com operações complexas, vamos dar uma olhada mais de perto em como os próprios eventos funcionam.

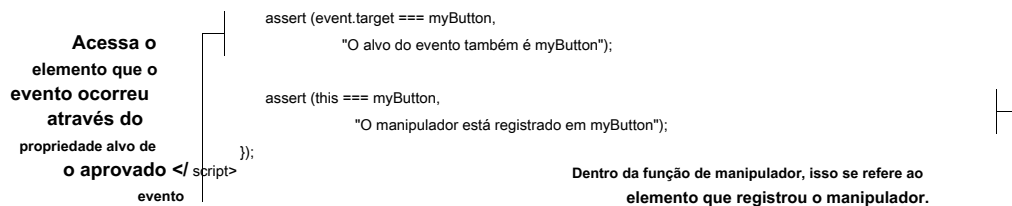
13.3 Trabalho com eventos

Quando um determinado evento ocorre, podemos tratá-lo em nosso código. Como você viu muitas vezes ao longo deste livro, uma maneira comum de registrar manipuladores de eventos é usando o sistema integrado `addEventListener` método, conforme mostrado na lista a seguir.

Listagem 13.6 Registrando manipuladores de eventos

<pre><button id = "myButton"> Clique </button> <script> const myButton = document.getElementById ("myButton"); myButton.addEventListener ("click", function myHandler (event) {</pre>	Registra um manipulador de eventos usando o método <code>addEventListener</code>
---	--





Neste snippet, definimos um botão chamado `myButton` e registrar um manipulador de eventos de clique usando o built-in `addEventListener` método que é acessível a partir de todos os elementos.

Depois que um evento de clique ocorre, o navegador chama o manipulador associado, neste caso o `myHandler` função. Para esse manipulador, o navegador passa um objeto de evento que contém propriedades que podemos usar para descobrir mais informações sobre o evento, como a posição do mouse ou o botão do mouse que foi clicado, caso se trate de um evento de clique do mouse ou a tecla pressionada se estivermos lidando com um evento de teclado.

Uma das propriedades do objeto de evento passado é o `target` propriedade, que faz referência ao elemento no qual o evento ocorreu.

NOTA Como na maioria das outras funções, no manipulador de eventos, podemos usar o esta palavra-chave. As pessoas costumam dizer coloquialmente que, em um manipulador de eventos, o esta palavra-chave se refere ao objeto no qual o evento ocorreu, mas como logo descobriremos, isso não é exatamente verdade. Em vez disso, o esta palavra-chave refere-se ao elemento no qual o manipulador de eventos foi *registrado*. Para ser honesto, na maioria dos casos, o elemento no qual o manipulador de eventos foi registrado *é* o elemento no qual o evento ocorreu, mas há exceções. Exploraremos essas situações em breve.

Antes de explorar mais esse conceito, vamos definir o cenário para que você possa ver como os eventos podem ser propagados por meio do DOM.

13.3.1 Propagar eventos através do DOM

Como já sabemos no capítulo 2, em documentos HTML, os elementos são organizados em uma árvore. Um elemento pode ter zero ou mais filhos, e todos os elementos (exceto a raiz `html` elemento) tem exatamente um pai. Agora, suponha que estejamos trabalhando com uma página que possui um elemento dentro de outro elemento, e ambos os elementos possuem um manipulador de cliques, como na listagem a seguir.

Listagem 13.7 Elementos aninhados e manipuladores de clique

```

<html>
  <head>
    <style>
      # outerContainer {width: 100px; altura: 100px; cor de fundo: azul;}
      # innerContainer {width: 50px; altura: 50px; cor de fundo: vermelho;} </style>
    </head>
  <body>
  
```

```

<div id = "outerContainer">
  <div id = "innerContainer"> </div>
</div>
<script>
  const outerContainer = document.getElementById ("outerContainer"); const innerContainer = document.getElementById
  ("innerContainer");

  outerContainer.addEventListener ("click", () => {
    relatório ("clique do recipiente externo");
  });

  innerContainer.addEventListener ("click", () => {
    relatório ("Clique no recipiente interno");
  });

  document.addEventListener ("click", () => {
    relatório ("clique no documento");
  });
</script>
</body>
</html>

```

Cria dois elementos aninhados



Registra um manipulador de cliques para o contêiner externo



Registra um manipulador de cliques para o contêiner interno



Registra um manipulador de cliques para todo o documento

Aqui temos dois elementos HTML, `outerContainer` e `innerContainer`, que são, como todos os outros elementos HTML, contidos em nosso global documento. E em todos os três objetos, registramos um manipulador de cliques.

Agora vamos supor que um usuário clique no `innerContainer` elemento. Porque `innerContainer` está contido no `outerContainer` elemento, e ambos os elementos estão contidos no documento, é óbvio que isso deve acionar a execução de todos os três manipuladores de eventos, gerando três mensagens. O que não está aparente é a ordem em que os manipuladores de eventos devem ser executados.

Devemos seguir a ordem em que os eventos foram registrados? Devemos começar com o elemento no qual o evento ocorre e mover para cima? Ou devemos começar do topo e descer em direção ao elemento-alvo? No passado, quando os navegadores estavam tomando essas decisões pela primeira vez, os dois principais concorrentes, Netscape e Microsoft, fizeram escolhas opostas.

No modelo de evento do Netscape, a manipulação de eventos começa com o elemento superior e desce até o elemento de destino do evento. Em nosso caso, os manipuladores de eventos seriam executados na seguinte ordem: documento manipulador de cliques, `outerContainer` manipulador de clique e finalmente `innerContainer` clique no manipulador. Isso é chamado *captura de eventos*.

A Microsoft optou por fazer o contrário: comece a partir do elemento-alvo e borbulhe na árvore DOM. Em nosso caso, os eventos seriam executados na seguinte ordem: `innerContainer` manipulador de cliques, `outerContainer` manipulador de clique e documento clique no manipulador. Isso é chamado *borbulhamento do evento*.

O padrão definido pelo Consórcio W3 (www.w3.org/TR/DOM-Level-3-Events/), que é implementado por todos os navegadores modernos, abraça ambas as abordagens. Um evento é tratado em duas fases:

- 1 *Fase de captura*—Um evento é primeiro capturado no elemento superior e arrastado para o elemento de destino.
- 2 *Fase de bolhas*—Depois que o elemento de destino é alcançado na fase de captura, o tratamento de eventos muda para bubbling e o evento borbulha novamente do elemento de destino para o elemento superior.

Essas duas fases são mostradas na figura 13.15.

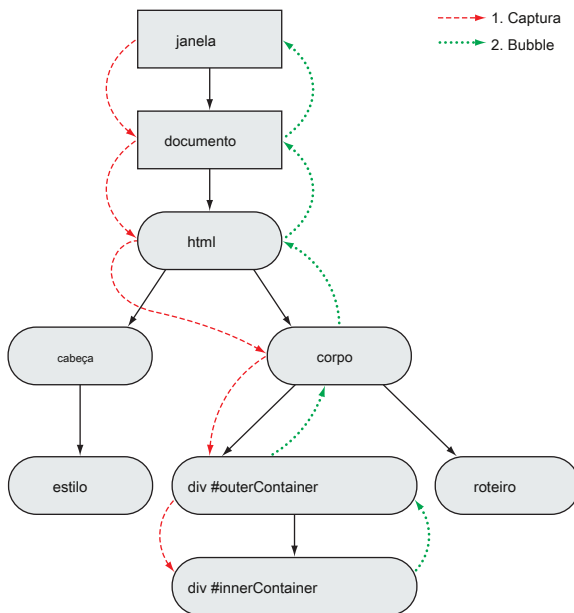


Figura 13.15 Com a captura, o evento desce até o elemento de destino. Com o borbulhamento, o evento borbulha do elemento de destino.

Podemos facilmente decidir qual ordem de manipulação de eventos queremos usar, adicionando outro argumento booleano ao `addEventListener` método. Se usarmos `true` como o terceiro argumento, o evento será capturado, enquanto se usarmos `false` (ou omita o valor), o evento irá borbulhar. Então, de certa forma, o padrão W3C prefere o bubbling de evento um pouco mais do que a captura de evento, porque o bubbling se tornou a opção padrão.

Agora, vamos voltar à listagem 13.7 e olhar atentamente como registramos os eventos:

```
outerContainer.addEventListener("click", () => {
    relatório("clique do recipiente externo");
});

innerContainer.addEventListener("click", () => {
    relatório("Clique no recipiente interno");
});

document.addEventListener("click", () => {
    relatório("clique no documento");
});
```

Como você pode ver, em todos os três casos, chamamos o `addEventListener` método com apenas dois argumentos, o que significa que o método padrão, *borbulhando*, é escolhido. Então, neste caso, se clicarmos no `innerContainer`, os manipuladores de eventos seriam executados nesta ordem: `innerContainer` manipulador de cliques, `outerContainer` manipulador de cliques,

documento clique no manipulador.

Vamos modificar o código na Listagem 13.7 da seguinte maneira.

Listagem 13.8 Captura versus borbulhamento

```
const outerContainer = document.getElementById("outerContainer"); const innerContainer = document.getElementById("innerContainer");
```

```
document.addEventListener("click", () => {  
    relatório("clique no documento");  
});
```

← Ao não especificar o terceiro argumento, o modo padrão, bubbling, é ativado.

```
outerContainer.addEventListener("click", () => {  
    relatório("clique do recipiente externo");  
}, verdadeiro);
```

← Passar verdadeiro como terceiro argumento permite a captura.

```
innerContainer.addEventListener("click", () => {  
    relatório("Clique no recipiente interno");  
}, falso);
```

← Passando em falso permite borbulhar.

Desta vez, definimos o manipulador de eventos do `outerContainer` para o modo de captura (passando verdade como o terceiro argumento), e os manipuladores de eventos de `innerContainer` (passando em falso como o terceiro argumento) e documento para o modo bubbling (deixando de fora o terceiro argumento escolhe o modo padrão, bubbling).

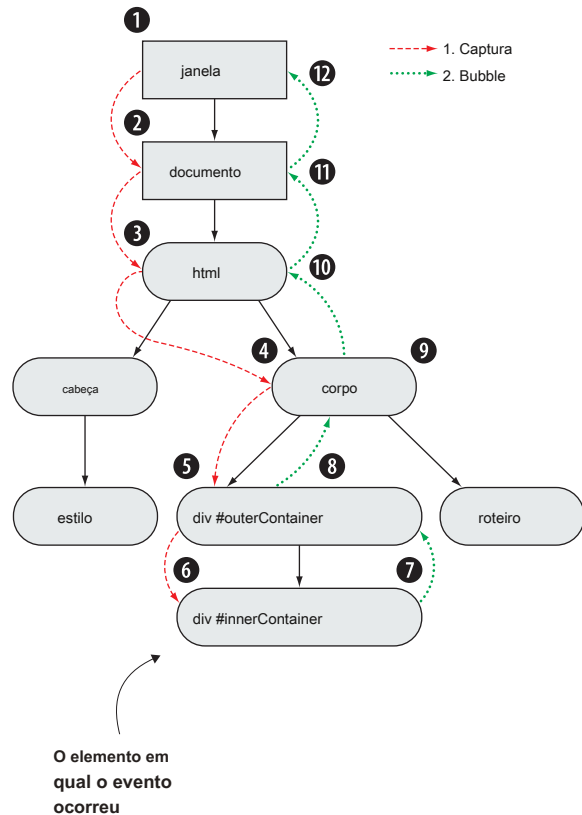
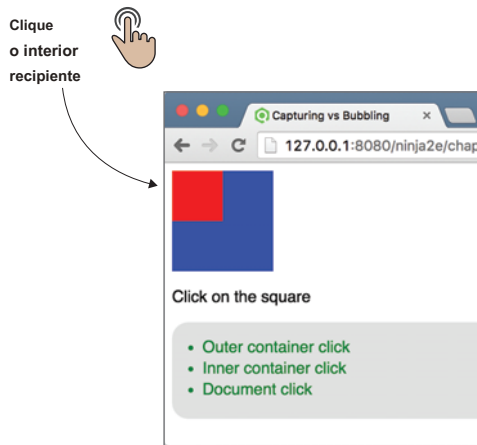
Como você sabe, um único evento pode acionar a execução de vários manipuladores de eventos, onde cada manipulador pode estar no modo de captura ou bolha. Por esse motivo, o evento primeiro passa pela captura, começando do elemento superior e descendo até o elemento de destino do evento. Quando o elemento de destino é alcançado, o modo de bolhas é ativado e o evento borbulha do elemento de destino de volta ao topo.

Em nosso caso, a captura começa do topo, janela objeto e goteja até o `innerContainer`, com o objetivo de encontrar todos os elementos que possuem um manipulador de eventos para este evento de clique no modo de captura. Apenas um elemento, `outerContainer`, é encontrado e seu manipulador de cliques correspondente é executado como o primeiro manipulador de eventos.

O evento continua viajando pelo caminho de captura, mas nenhum manipulador de eventos com captura é encontrado. Depois que o evento atinge o elemento de destino do evento, o `innerContainer` elemento, o evento segue para a fase de bolhas, onde vai do elemento de destino de volta ao topo, executando todos os manipuladores de eventos de bolhas nesse caminho.

No nosso caso, o `innerContainer` o manipulador de cliques será executado como o segundo manipulador de eventos, e o documento clique no manipulador como o terceiro. A saída gerada clicando no `innerContainer` elemento, bem como o caminho percorrido, é mostrado na figura 13.16.

- 11** Borbulhante
- ```
document.addEventListener("click", () => {
 relatório("clique no documento");
});
```
- 5** Capturando
- ```
outerContainer.addEventListener("click", () => {
  relatório("clique do recipiente externo");
}, verdade);
```
- 7** Borbulhante
- ```
innerContainer.addEventListener("click", () => {
 relatório("Clique no recipiente interno");
}, falso);
```



**Figura 13.16** Primeiro, o evento desce do topo, executando todos os manipuladores de eventos no modo de captura. Quando o elemento de destino é alcançado, o evento borbulha no topo, executando todos os manipuladores de eventos no modo de bolhas.

Uma das coisas que este exemplo mostra é que o elemento no qual o evento é tratado não precisa ser o elemento no qual o evento ocorre. Por exemplo, em nosso caso, o evento ocorre no innerContainer elemento, mas podemos tratá-lo em elementos superiores na hierarquia DOM, como no outerContainer ou o documento elemento.

Isso nos leva de volta ao esta palavra-chave em manipuladores de eventos e por que afirmamos explicitamente que o esta palavra-chave refere-se ao elemento no qual o manipulador de eventos está registrado, e não necessariamente ao elemento no qual o evento ocorre.

Novamente, vamos modificar nosso exemplo de execução, conforme mostrado na lista a seguir.

#### Listagem 13.9 A diferença entre *esta* e *event.target* em manipuladores de eventos

```
const outerContainer = document.getElementById("outerContainer"); const innerContainer = document.getElementById("innerContainer");
```

```
innerContainer.addEventListener("click", function (event) {
```

```

report ("manipulador innerContainer");
assert (this === innerContainer,
 "Isso se refere ao innerContainer"); assert (event.target ===
innerContainer,
 "event.target refere-se ao innerContainer");
});

```

Dentro do innerContainer handler, this e event.target apontam para o elemento innerContainer.

```

outerContainer.addEventListener ("click", function (event) {
 report ("manipulador innerContainer");
 assert (this === outerContainer,
 "Isso se refere ao outerContainer"); assert (event.target ===
innerContainer,
 "event.target refere-se ao innerContainer");
});

```

**No manipulador outerContainer, se estivermos manipulando o evento originado no innerContainer, isso fará referência ao outerContainer e event.target ao innerContainer.**

Novamente, vamos olhar para a execução do aplicativo quando ocorre um clique em innerContainer.

Porque ambos os manipuladores de eventos usam bolhas de eventos (não há um terceiro argumento definido para

verdade no addEventListener métodos), primeiro o innerContainer o manipulador de cliques é invocado. Dentro do corpo do manipulador, verificamos se o esta palavra-chave e o event.target propriedade refere-se ao innerContainer elemento:

```

assert (this === innerContainer,
 "Isso se refere ao innerContainer"); assert (event.target ===
innerContainer,
 "event.target refere-se ao innerContainer");

```

O esta a palavra-chave aponta para o innerContainer elemento porque esse é o elemento no qual o manipulador atual foi *registrado*. Considerando que a event.target propriedade aponta para o innerContainer elemento porque é o elemento no qual o evento tem *ocorreu*.

Em seguida, o evento borbulha até o outerContainer manipulador. Desta vez, o esta palavra-chave e o event.target apontar para diferentes elementos:

```

afirmar (este === outerContainer,
 "Isso se refere ao outerContainer"); assert (event.target === innerContainer,
 "event.target refere-se ao innerContainer");

```

Como esperado, o esta palavra-chave refere-se ao outerContainer , porque este é o elemento no qual o manipulador atual foi registrado. Por outro lado, o event.target propriedade aponta para o innerContainer elemento, porque este é o elemento no qual o evento ocorreu.

Agora que entendemos como um evento é propagado por meio da árvore DOM e como acessar o elemento no qual o evento ocorreu originalmente, vamos ver como aplicar esse conhecimento para escrever código com menos memória intensiva.

## DELEGANDO EVENTOS PARA UM ANCESTOR

Digamos que desejamos indicar visualmente se uma célula de uma tabela foi clicada pelo usuário, inicialmente exibindo um fundo branco para cada célula e, em seguida, alterando a cor de fundo para amarelo após o clique na célula. Parece muito fácil. Podemos iterar por todas as células e estabelecer um manipulador em cada uma que altera a propriedade da cor de fundo:

```
células const = document.querySelectorAll('td'); para (seja n = 0; n < comprimento das
células; n++) {
 células[n].addEventListener('click', function () {
 this.style.backgroundColor = 'amarelo';
 });
}
```

Claro que funciona, mas é elegante? Não. Estamos estabelecendo exatamente o mesmo manipulador de eventos em potencialmente centenas de elementos, e todos eles *exatamente a mesma coisa*.

Uma abordagem muito mais elegante é estabelecer um único manipulador em um nível mais alto do que as células que podem manipular todos os eventos usando o bubbling de eventos. Sabemos que todas as células serão descendentes de sua tabela envolvente e sabemos que podemos obter uma referência para o elemento que foi clicado por meio de `event.target`. É muito mais suave para

*delegar* o tratamento de eventos para a mesa, da seguinte maneira:

```
const table = document.getElementById('someTable'); table.addEventListener('click', function
(event) {
 if (event.target.tagName.toLowerCase() === 'td')
 event.target.style.backgroundColor = 'amarelo';
});
```

← Executa uma ação apenas se o clique acontecer em um elemento de célula (e não em um descendente aleatório)

Aqui, estabelecemos um manipulador que facilmente manipula o trabalho de alterar a cor de fundo de todas as células da tabela que foram clicadas. Isso é muito mais eficiente e elegante.

Com a delegação de eventos, temos que nos certificar de que ela é aplicada apenas a elementos que são ancestrais dos elementos que são os alvos do evento. Dessa forma, temos certeza de que os eventos irão eventualmente borbulhar até o elemento ao qual o manipulador foi delegado.

Até agora, lidamos com eventos fornecidos pelo navegador, mas você nunca desejou ferverosamente a capacidade de acionar o seu próprio *personalizadas* eventos?

### 13.3.2 Eventos personalizados

Imagine um cenário no qual você deseja executar uma ação, mas deseja acioná-la sob uma variedade de condições a partir de diferentes partes do código, talvez até mesmo de um código que esteja em arquivos de script compartilhados. Um novato repetiria o código em todos os lugares necessários. Um jornalista criaria uma função global e a chamaria de qualquer lugar que fosse necessário. Um ninja usaria eventos personalizados. Mas por que?

## ACOPLAMENTO SOLTO

Digamos que estejamos fazendo operações a partir do código compartilhado e queremos que o código da página saiba quando é hora de reagir a uma condição específica. Se usarmos a abordagem de função global do jornaleiro, apresentaremos a desvantagem de que nosso código compartilhado precisa definir um nome fixo para a função e todas as páginas que usam o código compartilhado precisam usar tal função.

Além disso, e se houver várias coisas a serem feitas quando a condição de acionamento ocorrer? Fazer concessões para várias notificações seria árduo e necessariamente confuso. Essas desvantagens são resultado de *acoplamento próximo*, em que o código que detecta as condições deve conhecer os detalhes do código que reagirá a essa condição.

*Acoplamento solto*, por outro lado, ocorre quando o código que dispara a condição não sabe nada sobre o código que vai reagir à condição, ou mesmo se há algo que vai reagir a ela. Uma das vantagens dos manipuladores de eventos é que podemos estabelecer quantos quisermos, e esses manipuladores são completamente independentes uns dos outros. Portanto, o tratamento de eventos é um bom exemplo de acoplamento fraco. Quando um evento de clique de botão é disparado, o código que dispara o evento não tem conhecimento de quais manipuladores estabelecemos na página, ou mesmo se houver algum. Em vez disso, o evento click é enviado para a fila de tarefas pelo navegador, e o que quer que tenha causado o disparo do evento não se importa com o que acontece depois disso. Se os manipuladores tiverem sido estabelecidos para o evento de clique, eles serão chamados individualmente de maneira completamente independente.

Há muito a ser dito sobre o acoplamento fraco. Em nosso cenário, o código compartilhado, quando detecta uma condição interessante, dispara um sinal de algum tipo que diz: “Esta coisa interessante aconteceu; qualquer pessoa interessada pode lidar com isso”, e não dá a mínima se alguém está interessado. Vamos examinar um exemplo concreto.

## UM EXEMPLO AJAX-Y

Vamos fingir que escrevemos algum código compartilhado que realizará uma solicitação Ajax. As páginas nas quais esse código será usado desejam ser notificadas quando uma solicitação Ajax começa e quando termina; cada página tem suas próprias coisas que precisa fazer quando esses “eventos” ocorrem.

Por exemplo, em uma página usando este pacote, queremos exibir um catavento giratório quando uma solicitação Ajax começa e queremos ocultá-lo quando a solicitação for concluída, a fim de fornecer ao usuário algum feedback visual de que uma solicitação está sendo processada. Se imaginarmos a condição inicial como um evento denominado *ajax-start*, e a condição de parada como *ajax-complete*, não seria ótimo se pudéssemos estabelecer manipuladores de eventos na página para esses eventos que mostram e ocultam a imagem conforme apropriado?

Considere isto:

```
document.addEventListener('ajax-start', e => {
 document.getElementById('whirlyThing').style.display = 'bloco embutido'; });

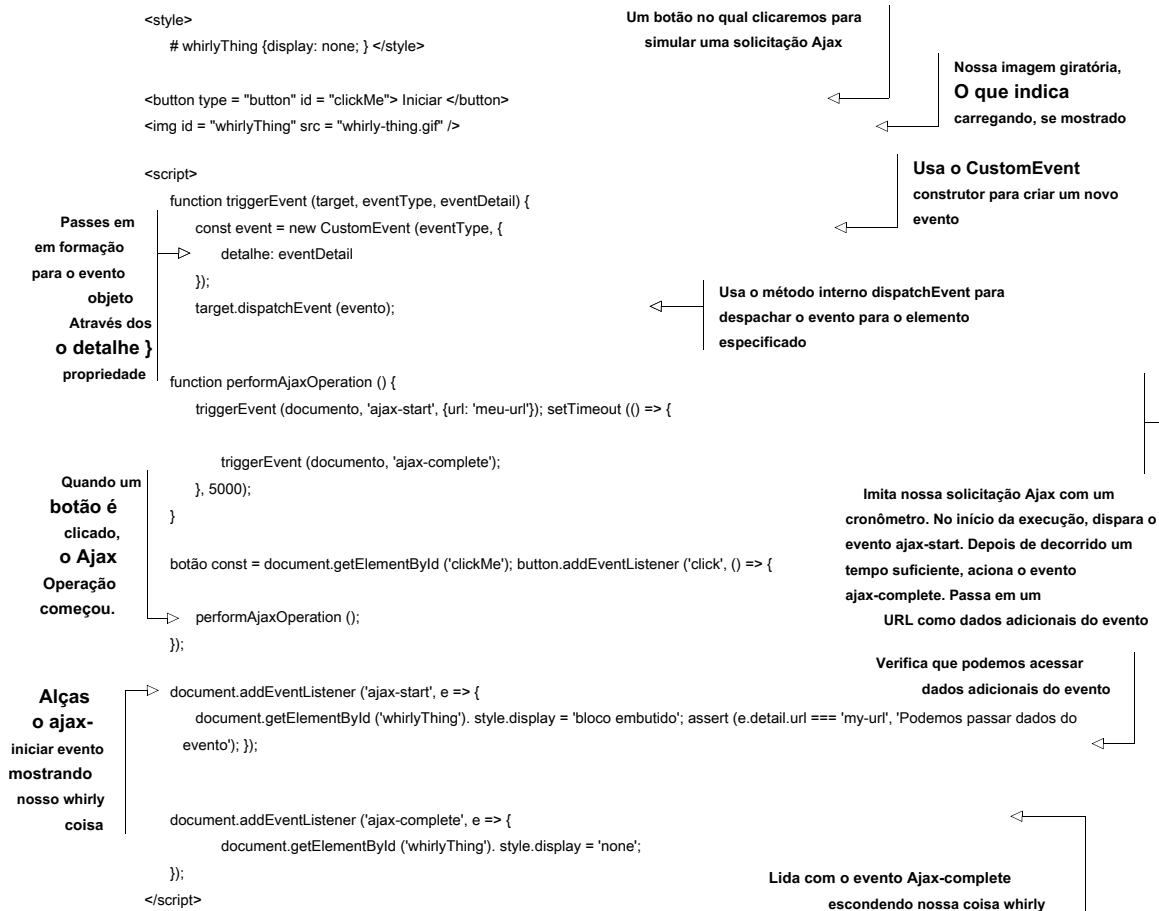
document.addEventListener('ajax-complete', e => {
 document.getElementById('whirlyThing').style.display = 'none'; });
```

Infelizmente, esses eventos não existem, mas nada nos impede de trazê-los à existência.

#### CRIANDO EVENTOS PERSONALIZADOS

Os eventos personalizados são uma forma de simular (para o usuário do nosso código compartilhado) a experiência de um evento real, mas um evento que tem sentido comercial dentro do contexto de nossa aplicação. A lista a seguir mostra um exemplo de como disparar um evento personalizado.

Listagem 13.10 Usando eventos personalizados



Neste exemplo, exploramos eventos personalizados estabelecendo o cenário descrito na seção anterior: Uma imagem de cata-vento animada é exibida enquanto uma operação Ajax está em andamento. A operação é acionada com o clique de um botão.

De uma forma completamente dissociada, um manipulador para um evento personalizado chamado ajaxstart é estabelecido, como é o do ajax-complete evento personalizado. Os manipuladores para esses eventos mostram e ocultam a imagem do cata-vento, respectivamente:

```
button.addEventListener ('click', () => {
 performAjaxOperation ();
});

document.addEventListener ('ajax-start', e => {
 document.getElementById ('whirlyThing'). style.display = 'bloco embutido'; assert (e.detail.url === 'my-url', 'Podemos passar dados do
evento'); });

document.addEventListener ('ajax-complete', e => {
 document.getElementById ('whirlyThing'). style.display = 'none';
});
```

Observe que os três manipuladores nada sabem da existência um do outro. Em particular, o manipulador de clique de botão não tem responsabilidades com relação a mostrar e ocultar a imagem.

A própria operação Ajax é simulada com o seguinte código:

```
function performAjaxOperation () {
 triggerEvent (documento, 'ajax-start', {url: 'meu-url'}); setTimeout (() => {

 triggerEvent (documento, 'ajax-complete');
 }, 5000);
}
```

A função aciona o ajax-start evento e envia dados sobre o evento (o url), fingindo que uma solicitação Ajax está prestes a ser feita. A função então emite um tempo limite de 5 segundos, simulando uma solicitação Ajax que se estende por 5 segundos. Quando o cronômetro expira, fingimos que a resposta foi retornada e acionamos um ajaxcomplete evento para indicar que a operação Ajax foi concluída.

Observe o alto grau de desacoplamento em todo este exemplo. O código de operação Ajax compartilhado não tem conhecimento do que o código da página fará quando os eventos forem disparados, ou mesmo se houver código da página para disparar. O código da página é modularizado em pequenos manipuladores que não conhecem uns aos outros. Além disso, o código da página não tem ideia de como o código compartilhado está funcionando; ele apenas reage a eventos que podem ou não ser acionados.

Esse nível de desacoplamento ajuda a manter o código modular, mais fácil de escrever e muito mais fácil de depurar quando algo dá errado. Também torna mais fácil compartilhar partes do código e movê-las sem medo de violar uma dependência acoplada entre os fragmentos de código. O desacoplamento é uma vantagem fundamental ao usar eventos customizados no código e nos permite desenvolver aplicativos de uma maneira muito mais expressiva e flexível.



## 13,4 Resumo

- Uma tarefa de loop de evento representa uma ação executada pelo navegador. As tarefas são agrupadas em duas categorias:
  - As macrotarefas são ações discretas e independentes do navegador, como criar o objeto do documento principal, manipular vários eventos e fazer alterações de URL.
  - Microtarefas são tarefas menores que devem ser executadas o mais rápido possível. Os exemplos incluem retornos de chamada de promessa e alterações de mutação DOM.
- Por causa do modelo de execução de thread único, as tarefas são processadas uma por vez e, depois que uma tarefa começa a ser executada, ela não pode ser interrompida por outra tarefa. O loop de eventos geralmente tem pelo menos duas filas de eventos: uma fila de macrotarefa e uma fila de microtarefa.
- Timers fornecem a capacidade de atrasar de forma assíncrona a execução de um trecho de código por *pelo menos* algum número de milissegundos.
  - Use o `setTimeout` função para executar um retorno de chamada uma vez após o atraso especificado ter decorrido.
  - Use o `setInterval` para iniciar um cronômetro que tentará executar o retorno de chamada no intervalo de atraso especificado, até ser cancelado.
  - Ambas as funções retornam um ID do temporizador que podemos usar para cancelar um temporizador através de `clearTimeout` e `clearInterval` funções.
  - Use temporizadores para dividir códigos caros computacionalmente em pedaços gerenciáveis que não obstruam o navegador.
- O DOM é uma árvore hierárquica de elementos e um evento que ocorre em um elemento (o destino) geralmente é propagado por meio do DOM. Existem dois mecanismos de propagação:
  - Na captura de evento, o evento desce do elemento superior até o elemento de destino.
  - No evento borbulhando, o evento borbulha do elemento de destino até o elemento superior.
- Ao chamar manipuladores de eventos, o navegador também passa um objeto de evento. Acesse o elemento no qual o evento ocorreu por meio do alvo propriedade. Dentro do manipulador, use o esta palavra-chave para se referir ao elemento no qual o manipulador foi registrado.
- Use eventos personalizados, criados por meio do `CustomEvent` construtor, e despachado com o `dispatchEvent` método, para reduzir o acoplamento entre as diferentes partes de seu aplicativo.

## 13.5 Exercícios

- 1 Por que é importante que a adição de tarefas à fila de tarefas aconteça fora do loop de eventos?
- 2 Por que é importante que cada iteração do loop de eventos não leve muito mais de cerca de 16 ms?

3 Qual é o resultado da execução do código a seguir por 2 segundos?

```
setTimeout (function () {
 console.log ("Tempo limite");
}, 1000);
```

```
setInterval (function () {
 console.log ("Intervalo");
}, 500);
```

uma Timeout Interval Interval Interval Interval

b Intervalo Tempo Limite Intervalo Intervalo Intervalo

c Tempo Limite do Intervalo

4 Qual é o resultado da execução do código a seguir por 2 segundos?

```
const timeoutId = setTimeout (function () {
 console.log ("Tempo limite");
}, 1000);
```

```
setInterval (function () {
 console.log ("Intervalo");
}, 500);
```

```
clearTimeout (timeoutId);
```

uma Intervalo Tempo Limite Intervalo Intervalo Intervalo

b Intervalo

c Intervalo Intervalo Intervalo Intervalo

5 Qual é o resultado de executar o código a seguir e clicar no elemento com o ID interno?

```
<body>
 <div id = "outer">
 <div id = "inner"> </div>
 </div>
 <script>
 const innerElement = document.querySelector ("# inner"); const outerElement = document.querySelector ("# outer"); const bodyElement = document.querySelector ("body");
```

```
 innerElement.addEventListener ("click", function () {
 console.log ("Interno");
 });
```

```
 outerElement.addEventListener ("click", function () {
 console.log ("Externo");
 }, verdade);
```

```
 bodyElement.addEventListener ("click", function () {
 console.log ("Corpo");
```

```
 })
 </script>
</body>
```

uma Corpo Externo Interno

b Corpo Externo Interno

c Corpo Interno Externo

# 14

## *Em desenvolvimento estratégias para vários navegadores*

---

### *Este capítulo cobre*

- Desenvolvimento de código JavaScript reutilizável em vários navegadores
- Analisando problemas entre navegadores que precisam ser resolvidos
- Lidando com esses problemas de maneira inteligente

Qualquer pessoa que tenha passado um tempo desenvolvendo código JavaScript na página sabe que existe uma ampla gama de pontos problemáticos quando se trata de garantir que o código funcione perfeitamente em um conjunto de navegadores compatíveis. Essas considerações vão desde o fornecimento de desenvolvimento básico para necessidades imediatas até o planejamento de futuras versões do navegador, até a reutilização de código em páginas da web que ainda não foram criadas.

Codificar para vários navegadores é uma tarefa não trivial que deve ser balanceada de acordo com as metodologias de desenvolvimento que você possui, bem como os recursos disponíveis para o seu projeto. Por mais que adoraríamos que nossas páginas funcionassem perfeitamente em todos os navegadores que já existiram ou que existirão, a realidade vai mostrar sua cara feia e devemos perceber que temos recursos de desenvolvimento finitos. Devemos planejar para aplicar esses recursos de forma adequada e cuidadosa, obtendo o maior retorno para nosso investimento.

Por isso, começamos este capítulo com conselhos sobre como escolher quais navegadores oferecer suporte. Isso é seguido por uma discussão das principais preocupações com relação ao desenvolvimento de crossbrowser, bem como estratégias eficazes para lidar com esses problemas. Vamos começar a escolher cuidadosamente os navegadores compatíveis.

.....

Quais são algumas maneiras comuns de lidar com inconsistências  
cres no comportamento com navegadores diferentes?

Você sabe? Qual é a melhor maneira de tornar seu código utilizável em outros  
as páginas das pessoas?

Por que polyfills são úteis em scripts entre navegadores?

.....

### 14.1 Considerações entre navegadores

Aperfeiçoar nossas habilidades de programação em JavaScript nos levará muito longe, especialmente agora que o JavaScript escapou dos limites do navegador e está sendo usado no servidor com Node.js. Mas ao desenvolver aplicativos JavaScript baseados em navegador (que é o foco deste livro), mais cedo ou mais tarde, vamos enfrentar primeiro *Os navegadores* e seus vários problemas e inconsistências.

Em um mundo perfeito, todos os navegadores estariam livres de erros e ofereceriam suporte aos padrões da web de maneira consistente, mas como todos sabemos, não vivemos nesse mundo. Embora a qualidade dos navegadores tenha melhorado muito nos últimos tempos, todos ainda têm alguns bugs, APIs ausentes e peculiaridades específicas do navegador com os quais precisaremos lidar. O desenvolvimento de uma estratégia abrangente para lidar com esses problemas do navegador e se familiarizar intimamente com suas diferenças e peculiaridades não é menos importante do que a proficiência no próprio JavaScript.

Ao escrever aplicativos de navegador, é importante escolher quais navegadores dar suporte. Provavelmente gostaríamos de oferecer suporte aos pequenos, mas as limitações de desenvolvimento e recursos de teste ditam o contrário. Então, como decidimos quais apoiar e em que nível?

Uma abordagem que podemos empregar foi livremente emprestada de um antigo Yahoo! aproximação, *suporte de navegador classificado*. Nessa técnica, criamos uma matriz de suporte ao navegador que serve como um instantâneo da importância de um navegador e sua plataforma para nossas necessidades. Nesta tabela, listamos as plataformas de destino em um eixo e os navegadores no outro. Então, nas células da tabela, damos uma “nota” (A a F, ou qualquer outro sistema de notas que atenda às nossas necessidades) para cada combinação de navegador / plataforma. A Tabela 14.1 mostra um exemplo hipotético.

Observe que não preenchemos nenhuma nota. As notas que você atribui a uma combinação específica de plataforma e navegador dependem inteiramente das necessidades e requisitos de seu projeto, bem como de outros fatores importantes, como a composição do público-alvo. Podemos usar essa abordagem para chegar a notas que medem a importância do suporte para essa plataforma / navegador e combinar essas informações com o custo desse suporte para tentar chegar ao conjunto ideal de navegadores suportados.

Tabela 14.1 Uma matriz hipotética de suporte ao navegador

	janelas	OS X	Linux	iOS	Android
IE 9		N / D	N / D	N / D	N / D
IE10		N / D	N / D	N / D	N / D
IE11		N / D	N / D	N / D	N / D
Beira		N / D	N / D	N / D	N / D
Raposa de fogo				N / D	
cromada					
Ópera					
Safári			N / D		N / D

Quando optamos por oferecer suporte a um navegador, normalmente fazemos as seguintes promessas:

- Testaremos ativamente esse navegador com nosso conjunto de testes.
- Vamos consertar bugs e regressões associados a esse navegador.
- O navegador executará nosso código com um nível de desempenho razoável.

Como é impraticável desenvolver em várias combinações de plataforma / navegador, devemos pesar os custos e os benefícios de oferecer suporte aos vários navegadores. Esta análise deve levar em consideração várias considerações, e as principais são as seguintes:

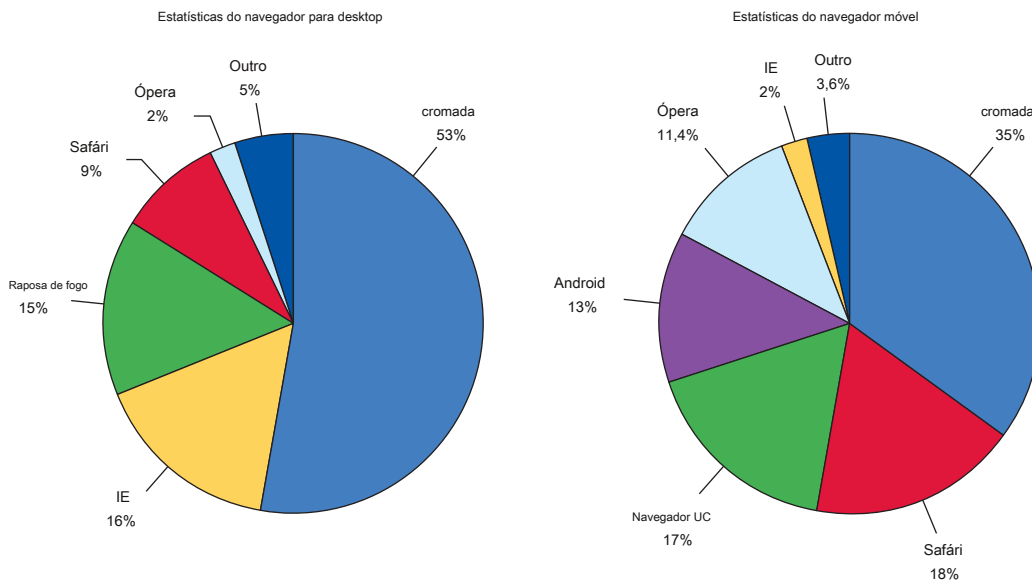
- As expectativas e necessidades do público-alvo
- A participação de mercado do navegador
- A quantidade de esforço necessária para suportar o navegador

O primeiro ponto é subjetivo que somente o seu projeto pode determinar. A participação de mercado, por outro lado, pode frequentemente ser medida usando as informações disponíveis. E uma estimativa aproximada do esforço envolvido no suporte de cada navegador pode ser determinada considerando os recursos dos navegadores e sua adesão aos padrões modernos.

A Figura 14.1 mostra um gráfico de amostra que representa informações sobre o uso do navegador (obtido a partir de <http://gs.statcounter.com> para abril de 2016). Qualquer pedaço de código JavaScript reutilizável, seja uma biblioteca JavaScript de consumo em massa ou nosso próprio código na página, deve ser desenvolvido para funcionar em tantos ambientes quanto possível, concentrando-se nos navegadores e plataformas que são importantes para o usuário final. Para bibliotecas de consumo em massa, esse é um grande conjunto; para aplicativos mais direcionados, o conjunto necessário pode ser mais estreito.

Mas é de vital importância não morder mais do que você pode mastigar, e a qualidade nunca deve ser sacrificada pela cobertura. Isso é importante o suficiente para repetir; na verdade, recomendamos que você leia em voz alta:

*A qualidade nunca deve ser sacrificada pela cobertura.*



**Figura 14.1** Observar as estatísticas de uso de navegadores em desktops e dispositivos móveis nos dá uma ideia de quais navegadores devemos concentrar nossa atenção.

Neste capítulo, examinaremos as situações em que o código JavaScript se encontrará em relação ao suporte para navegadores cruzados. Em seguida, examinaremos algumas das melhores maneiras de escrever esse código com o objetivo de aliviar quaisquer problemas potenciais que essas situações representam. Isso deve ajudar muito a decidir quais dessas técnicas vale a pena adotar e deve ajudá-lo a preencher seu próprio gráfico de suporte do navegador.

#### 14.2 As cinco principais preocupações de desenvolvimento

Qualquer pedaço de código não trivial carrega uma miríade de preocupações de desenvolvimento com que se preocupar. Mas cinco pontos principais representam os maiores desafios para nosso código JavaScript reutilizável, conforme ilustrado na Figura 14.2.

Estes são os cinco pontos:

- Bugs do navegador
- Correções de bug do navegador
- Código externo
- Regressões de navegador
- Recursos ausentes nos navegadores

Bem queremos equilibrar o valor que gastamos

Tempo em cada ponto com

os benefícios resultantes. Em última análise, estes



**Figura 14.2** Os cinco principais pontos de preocupação para o desenvolvimento de JavaScript reutilizável

são perguntas que você terá que responder, aplicando-as à sua situação. Uma análise de seu público-alvo, recursos de desenvolvimento e cronograma são fatores que influenciam sua decisão.

Ao nos esforçarmos para desenvolver código JavaScript reutilizável, devemos levar todos os pontos em consideração, mas prestar mais atenção aos navegadores mais populares que existem no momento, porque eles são mais prováveis de serem usados por nosso público-alvo. Com outros navegadores menos populares, devemos pelo menos nos certificar de que nosso código seja degradado normalmente. Por exemplo, se um navegador não oferece suporte a uma determinada API, no mínimo, devemos ter cuidado para que nosso código não lance nenhuma exceção para que o resto do código possa ser executado.

Nas seções a seguir, analisaremos essas várias preocupações para entender melhor os desafios que enfrentamos e como combatê-los.

#### 14.2.1 *Bugs e diferenças do navegador*

Uma das preocupações com a qual precisaremos lidar ao desenvolver o código JavaScript reutilizável é lidar com os vários bugs e diferenças de API associadas ao conjunto de navegadores que decidimos oferecer suporte. Embora os navegadores sejam muito mais uniformes atualmente, quaisquer recursos que fornecemos em nosso código devem ser completamente e *verificável* correto em *todo* navegadores que escolhemos oferecer suporte.

A maneira de conseguir isso é direta: precisamos de um conjunto abrangente de testes para cobrir os casos de uso comuns e marginais do código. Com uma boa cobertura de teste, podemos nos sentir seguros em saber que o código que desenvolvemos funcionará no conjunto de navegadores com suporte. E presumindo que nenhuma alteração subsequente do navegador rompa a compatibilidade com versões anteriores, teremos uma sensação calorosa e confusa de que nosso código funcionará até mesmo em versões futuras desses navegadores. Veremos estratégias específicas para lidar com bugs e diferenças do navegador na seção 14.3.

Um ponto complicado em tudo isso é implementar correções para bugs atuais do navegador de forma que sejam resistentes a quaisquer correções para esses bugs que sejam implementados em versões futuras do navegador.

#### 14.2.2 *Correções de bug do navegador*

Assumir que um navegador apresentará para sempre um bug específico é temerário - a maioria dos bugs do navegador eventualmente são corrigidos e *contando* na presença do bug é uma estratégia de desenvolvimento perigosa. É melhor usar as técnicas da seção 14.3 para certificar-se de que quaisquer soluções alternativas de bug sejam preparadas para o futuro, tanto quanto possível.

Ao escrever um trecho de código JavaScript reutilizável, queremos ter certeza de que ele pode durar muito tempo. Assim como acontece com a escrita de qualquer aspecto de um site (CSS, HTML e assim por diante), não queremos voltar e corrigir o código que está quebrado por um novo lançamento do navegador.

Fazer suposições sobre bugs do navegador causa uma forma comum de quebra de site: hacks específicos colocados em prática para contornar bugs apresentados por um navegador que quebram quando o navegador corrige os bugs em versões futuras.



O problema de lidar com bugs do navegador é duplo:

- Nosso código pode quebrar quando a correção do bug for instituída.
- Podemos acabar treinando fornecedores de navegadores para *não* consertar bugs por medo de causar falhas em sites.

Um exemplo interessante da segunda situação ocorreu recentemente, com o

scrollTop erro ( <https://dev.opera.com/articles/fixing-the-scrolltop-bug/> ) Ao lidar com elementos no HTML DOM, podemos usar o scrollTop e

scrollLeft propriedades para acessar e modificar a posição de rolagem atual do elemento. Mas se usarmos essas propriedades na raiz, html elemento, essas propriedades devem, de acordo com a especificação, em vez disso relatar (e influenciar) a posição de rolagem da janela de visualização. O IE 11 e o Firefox seguem de perto essa especificação. Infelizmente, Safari, Chrome e Opera não. Em vez disso, se você tentar modificar essas propriedades da raiz, html elemento, nada acontece. Para obter o mesmo efeito nesses navegadores, temos que usar o scrollTop e scrollLeft propriedades no corpo elemento.

Quando confrontados com essa inconsistência, os desenvolvedores da web frequentemente recorrem à detecção do nome atual do navegador (por meio da string do agente do usuário, mais sobre isso posteriormente) e, em seguida, modificando o scrollTop e scrollLeft do html elemento se o nosso código JavaScript estiver sendo executado no IE ou Firefox, e do corpo elemento se o código estiver sendo executado no Safari, Chrome ou Opera. Infelizmente, essa maneira de contornar esse bug provou ser desastrosa. Como muitas páginas agora codificam explicitamente "se este for Safari, Chrome ou Opera", modifique o corpo, esses navegadores não podem realmente corrigir esse bug, porque a correção do bug, ironicamente, causaria falhas em muitas páginas da web.

Isso traz outro ponto importante sobre os bugs: ao determinar se uma parte da funcionalidade é potencialmente um bug, sempre verifique-o com a especificação!

Um bug do navegador também é diferente de uma API não especificada. É importante consultar as especificações do navegador, porque elas fornecem os padrões exatos que os navegadores usam para desenvolver e melhorar seu código. Em contraste, a implementação de uma API não especificada pode mudar a qualquer momento (especialmente se a implementação tentar se tornar padronizada). No caso de inconsistências em APIs não especificadas, você deve sempre testar a saída esperada. Esteja sempre ciente de que mudanças futuras podem ocorrer nessas APIs à medida que se solidificam.

Além disso, há uma distinção entre correções de bugs e alterações de API. Considerando que correções de bugs são facilmente previstas - um navegador eventualmente corrigirá os bugs em sua implementação, mesmo que leve muito tempo - as alterações de API são muito mais difíceis de detectar. É improvável que as APIs padrão mudem (embora não seja totalmente inédito); as alterações são muito mais prováveis de ocorrer com APIs não especificadas.

Felizmente, isso raramente acontece de uma forma que prejudique massivamente a maioria dos aplicativos da web. Mas se isso acontecer, é efetivamente indetectável com antecedência (a menos, é claro, que testemos cada API que já tocamos - mas a sobrecarga incorrida em tal processo seria ridícula). Mudanças de API desse tipo devem ser tratadas como qualquer outra regressão.

Para nosso próximo ponto de preocupação, sabemos que nenhum homem é uma ilha e nem o nosso código. Vamos explorar as ramificações.

### 14.2.3 Código externo e marcação

Qualquer código reutilizável deve coexistir com o código que o cerca. Quer estejamos esperando que nosso código funcione em páginas que escrevemos ou em sites desenvolvidos por terceiros, precisamos garantir que ele possa existir na página com qualquer outro código aleatório.

Esta é uma faca de dois gumes: nosso código não apenas deve ser capaz de suportar a convivência com código externo potencialmente mal escrito, mas também deve tomar cuidado para não ter efeitos adversos no código com o qual vive.

Exatamente quanto precisamos estar vigilantes sobre esse ponto de preocupação depende muito do ambiente no qual esperamos que o código seja usado. Por exemplo, se estamos escrevendo um código reutilizável para um único ou um número limitado de sites sobre os quais temos algum nível de controle, pode ser seguro nos preocupar menos com os efeitos do código externo porque sabemos onde o código irá operar, e nós podemos, até certo ponto, corrigir quaisquer problemas por nós mesmos.

**GORJETA** Esta é uma preocupação importante o suficiente para justificar um livro inteiro sobre o assunto.

Se você gostaria de se aprofundar mais, recomendamos altamente *JavaScript de terceiros* por Ben Vinegar e Anton Kovalyov (Manning, 2013, <https://www.manning.com/books/third-party-javascript>)

Se estivermos desenvolvendo código que terá um amplo nível de aplicabilidade em ambientes desconhecidos (e incontrolláveis), precisaremos ter a certeza de que nosso código é robusto. Vamos discutir algumas estratégias para conseguir isso.

#### ENCAPSULANDO NOSSO CÓDIGO

Para evitar que nosso código afete outras partes do código nas páginas onde é carregado, é melhor praticar *encapsulamento*. Em geral, refere-se ao ato de colocar algo dentro ou como se estivesse dentro de uma cápsula. Uma definição mais focada no domínio é “um mecanismo de linguagem para restringir o acesso a alguns dos componentes do objeto”. Sua tia Mathilda pode resumir de forma mais sucinta como “Mantenha o seu nariz no seu próprio negócio!”

Manter uma pegada global incrivelmente pequena ao introduzir nosso código em uma página pode ajudar muito a deixar tia Mathilda feliz. Na verdade, manter nossa pegada global para um punhado de variáveis globais, ou melhor ainda, `1`, é bastante fácil.

Como você viu no capítulo 12, jQuery, a biblioteca JavaScript mais popular do lado do cliente, é um bom exemplo disso. Ele apresenta uma variável global (uma função) chamada `jQuery`, e um alias para essa variável global, `$`. Ele ainda possui um meio suportado de devolver o `$` alias a qualquer outro código na página ou outra biblioteca que queira usá-lo.

Quase todas as operações em jQuery são feitas por meio do jQuery função. E quaisquer outras funções que ele forneça (chamadas *funções utilitárias*) são definidos como propriedades de jQuery (lembre-se do capítulo 3 como é fácil definir funções que são propriedades de outras funções), usando assim o nome jQuery como um *namespace* para todas as suas definições.

Podemos usar a mesma estratégia. Digamos que estejamos definindo um conjunto de funções para nosso próprio uso, ou para uso de outros, que agruparemos em um namespace de nossa própria escolha - digamos, `ninja`.

Poderíamos, como jQuery, definir uma função global chamada `ninja` () que executa várias operações com base no que passamos para a função. Por exemplo:

```
var ninja = function () { /* o código de implementação vai aqui */ }
```

Definir nossas próprias funções de utilitário que usam esta função como seu namespace é fácil:

```
ninja.hitsuke = function () { /* código para distrair os guardas com fogo aqui */ }
```

Se não quisermos ou precisarmos `ninja` para ser uma função, mas apenas para servir como um namespace, poderíamos defini-la da seguinte maneira:

```
var ninja = {};
```

Isso cria um objeto vazio no qual podemos definir propriedades e funções para evitar a adição desses nomes ao namespace global.

Outras práticas que queremos evitar, a fim de manter nosso código encapsulado, são a modificação de quaisquer variáveis existentes, protótipos de função ou mesmo elementos DOM. Qualquer aspecto da página que nosso código modifica, fora dele, é uma área potencial de colisão e confusão.

O outro lado da rua de mão dupla é que, mesmo que *nós* seguir as práticas recomendadas e encapsular cuidadosamente nosso código, não podemos ter certeza de que o código que não escrevemos será tão bem comportado.

#### LIDANDO COM CÓDIGO MENOS DO QUE EXEMPLAR

Há uma velha piada que circula desde que Grace Hopper removeu aquela mariposa de um retransmissor no período Cretáceo: “O único código que não é uma merda é o código que você mesmo escreve”. Isso pode parecer cínico, mas quando nosso código coexiste com código que não podemos controlar, devemos presumir o pior, apenas por segurança.

Algum código, mesmo se bem escrito, pode *intencionalmente* estar fazendo coisas como modificar protótipos de funções, propriedades de objetos e métodos de elementos DOM. Essa prática, bem-intencionada ou não, pode nos colocar em armadilhas.

Nessas circunstâncias, nosso código poderia estar fazendo algo inócuo, como usar arrays JavaScript, e ninguém poderia nos culpar por fazer a simples suposição de que os arrays JavaScript vão agir como arrays JavaScript. Mas se algum outro código na página modificar a maneira como os arrays funcionam, nosso código pode acabar não funcionando como planejado, sem absolutamente nenhuma falha nossa.

Infelizmente, não existem muitas regras constantes ao lidar com situações dessa natureza, mas podemos tomar algumas medidas atenuantes. As próximas seções apresentam essas etapas defensivas.

#### LIDANDO COM IDS GOSTOSOS

A maioria dos navegadores exibe um *anti-recurso* (não podemos chamá-lo de *erro* porque o comportamento é absolutamente intencional) que pode fazer com que nosso código tropece e caia inesperadamente. Este recurso faz com que referências de elemento sejam adicionadas a outros elementos usando o `eu` ou `ia` ou nome atributos do elemento original. E quando isso `eu` ou `ia` ou nome conflita com propriedades que já fazem parte do elemento, coisas ruins podem acontecer.

Dê uma olhada no seguinte trecho de HTML para observar a maldade que pode resultar desses *IDs gananciosos*:

```
<form id = "form" action = "/" ocultar">
 <input type = "text" id = "action" />
 <input type = "submit" id = "submit" />
</form>
```

Agora, nos navegadores, vamos chamar isso de:

```
var what = document.getElementById ('form'). action;
```

Corretamente, esperamos que este seja o valor do formulário ação atributo. E na maioria dos casos, seria. Mas se inspecionarmos o valor da variável o que, descobrimos que, em vez disso, é uma referência ao input # action elemento! Huh?

Vamos tentar outra coisa:

```
document.getElementById ('formulário'). submit ();
```

Essa declaração deve fazer com que o formulário seja enviado, mas em vez disso, obtemos um erro de script:

TypeError não capturado: a propriedade 'submit' do objeto # <HTMLFormElement> não é uma função

O que está acontecendo?

Os navegadores adicionaram propriedades ao < formulário> elemento para cada um dos elementos de entrada dentro do formulário que faz referência ao elemento. Isso pode parecer útil no início, até percebermos que o nome da propriedade adicionada é tirado do eu ia ou nome valores dos elementos de entrada. E se esse valor for apenas uma propriedade já usada do elemento do formulário, como ação ou enviar, essas propriedades originais são substituídas pela nova propriedade. Isso geralmente é conhecido como *DOM clobbering*.

Então, antes do input # submit elemento é criado, a referência form.action aponta para o valor do ação atributo para o < formulário>. Depois, ele aponta para o input # submit elemento. A mesma coisa acontece com form.submit. Sim!

Isso é um resquício de muito tempo atrás, de uma época em que os navegadores não tinham um rico conjunto de métodos API para buscar elementos do DOM. Os fornecedores de navegadores adicionaram esse recurso para fornecer acesso fácil aos elementos do formulário. Hoje em dia, podemos acessar facilmente qualquer elemento no DOM, portanto, ficamos apenas com os efeitos colaterais infelizes do recurso.

Em qualquer caso, esse "recurso" específico dos navegadores pode causar inúmeros e confusos problemas em nosso código, e devemos mantê-lo em mente ao depurar. Quando encontramos propriedades que aparentemente foram inexplicavelmente transformadas em algo diferente do que esperamos, a destruição do DOM é um provável culpado.

Felizmente, podemos evitar esse problema em nossa própria marcação, evitando eu ia e nome valores que podem entrar em conflito com nomes de propriedade padrão e podemos incentivar outros a fazer o mesmo. O valor que enviar deve ser evitado principalmente, pois é uma fonte comum de comportamento problemático e frustrante.

## CARREGANDO A ORDEM DE FOLHAS DE ESTILO E SCRIPTS

Freqüentemente, esperamos que as regras CSS já estejam disponíveis quando nosso código for executado. Uma das melhores maneiras de garantir que as regras CSS fornecidas pelas folhas de estilo sejam definidas quando nosso código JavaScript é executado é incluir as folhas de estilo externas *prévio* para incluir os arquivos de script externos.

Não fazer isso pode causar resultados inesperados, porque o script tenta acessar as informações de estilo ainda indefinidas. Infelizmente, esse problema não pode ser corrigido facilmente com JavaScript puro e, em vez disso, deve ser tratado com a documentação do usuário.

Essas últimas seções cobriram alguns exemplos básicos de como as externalidades podem afetar o funcionamento do nosso código, frequentemente de maneiras não intencionais e confusas. Problemas com nosso código costumam aparecer quando outros usuários tentam integrá-lo *deles* locais, ponto em que devemos ser capazes de diagnosticar os problemas e construir testes apropriados para lidar com eles. Em outras ocasiões, descobriremos esses problemas ao integrar o código de outras pessoas em nossas páginas e, esperançosamente, as dicas nessas seções ajudarão a identificar as causas.

É uma pena que não haja soluções melhores e deterministas para lidar com esses problemas de integração, a não ser dar alguns primeiros passos inteligentes e escrever nosso código de forma defensiva. Vamos agora passar para o próximo ponto de preocupação.

### 14.2.4 Regressões

*Regressões* são um dos problemas mais difíceis que encontraremos na criação de código JavaScript reutilizável e sustentável. Esses são bugs, ou alterações de API não compatíveis com versões anteriores (principalmente para APIs não especificadas), que os navegadores introduziram e que fazem com que o código seja interrompido de maneiras imprevisíveis.

**NOTA** Aqui estamos usando o termo *regressão* em sua definição clássica: um recurso que costumava funcionar, mas não funciona mais como esperado. Isso geralmente não é intencional, mas às vezes é causado por alterações deliberadas que quebram o código existente.

## ANTECIPANDO MUDANÇAS

Lá *estamos* algumas mudanças de API que, com alguma previsão, podemos detectar e tratar proativamente, conforme mostrado na Listagem 14.1. Por exemplo, com o Internet Explorer 9, a Microsoft introduziu suporte para manipuladores de eventos DOM nível 2 (vinculados usando o `addEventListener` método), enquanto as versões anteriores do IE usavam o método interno específico do IE `attachEvent` método. Para código escrito antes do IE 9, a detecção de recurso simples foi capaz de lidar com essa mudança.

Listagem 14.1 Antecipando uma mudança de API futura

```
function bindEvent (element, type, handle) {
 if (element.addEventListener) {
 element.addEventListener (tipo, identificador, falso);
 }
 else if (element.attachEvent) {
```

← Vincula-se usando o  
API padrão

```
 element.attachEvent ("on" + tipo, identificador);
 }
}
```



Vincula-se usando um  
API proprietária

Neste exemplo, preparamos nosso código para o futuro, sabendo (ou tendo esperanças) de que algum dia a Microsoft colocará o Internet Explorer em linha com os padrões DOM. Se o navegador for compatível com a API compatível com os padrões, usamos a detecção de recursos para inferir isso e usamos a API padrão, o `addEventListener` método. Caso contrário, verificamos se o método proprietário do IE `attachEvent` está disponível e use-o. Se tudo mais falhar, não fazemos nada.

A maioria das mudanças futuras da API, infelizmente, não são fáceis de prever e não há como prever os próximos bugs. Esse é apenas um dos motivos importantes pelos quais enfatizamos os testes ao longo deste livro. Em face das mudanças imprevisíveis que afetarão nosso código, o melhor que podemos esperar é ser diligentes no monitoramento de nossos testes para cada versão do navegador e resolver rapidamente os problemas que as regressões podem apresentar.

Ter um bom conjunto de testes e acompanhar de perto os próximos lançamentos do navegador é absolutamente a melhor maneira de lidar com futuras regressões dessa natureza. Não precisa ser desgastante para o seu ciclo normal de desenvolvimento, que já deve incluir testes de rotina. A execução desses testes em novas versões do navegador deve sempre ser considerada no planejamento de qualquer ciclo de desenvolvimento.

Você pode obter informações sobre os próximos lançamentos de navegador nos seguintes locais:

- Microsoft Edge (um sucessor do IE): <http://blogs.windows.com/msedgedev/>
- Raposa de fogo: <http://ftp.mozilla.org/pub/firefox/nightly/latest-trunk/>
- WebKit (Safari): <https://webkit.org/nightly/>
- Ópera: <https://dev.opera.com/>
- Cromada: <http://chrome.blogspot.hr/>

Diligência é importante. Como nunca podemos prever totalmente os bugs que serão introduzidos por um navegador, é melhor garantir que permaneçamos no controle de nosso código e evitemos rapidamente quaisquer crises que possam surgir.

Felizmente, os fornecedores de navegadores estão fazendo muito para garantir que regressões dessa natureza não ocorram, e os navegadores costumam ter suítes de teste de várias bibliotecas JavaScript integradas em sua suíte de teste de navegador principal. Isso garante que nenhuma regressão futura será introduzida que afete essas bibliotecas diretamente. Embora isso não capture todas as regressões (e certamente não o fará em todos os navegadores), é um ótimo começo e mostra um bom progresso por parte dos fornecedores de navegadores na prevenção do maior número de problemas possível.

Nesta seção, examinamos quatro pontos principais de preocupação para o desenvolvimento de JavaScript reutilizável: bugs do navegador, correções de bugs do navegador, código externo e regressões do navegador. O quinto ponto - recursos ausentes nos navegadores - merece uma menção especial, portanto, o cobriremos na próxima seção, junto com outras estratégias de implementação relevantes para aplicativos da web entre navegadores.

### 14.3 Estratégias de implementação

Saber de quais questões estar atento é apenas metade da batalha. Descobrir soluções eficazes e usá-las para implementar código robusto para vários navegadores é outra questão.

Uma ampla variedade de estratégias está disponível e, embora nem todas as estratégias funcionem em todas as situações, a variedade apresentada nesta seção cobre a maioria das preocupações que precisaremos abordar em nossas robustas bases de código. Vamos começar com algo que seja fácil e quase sem problemas.

#### 14.3.1 Correções seguras para navegadores diferentes

As classes mais simples (e mais seguras) de correções entre navegadores são aquelas que exibem duas características importantes:

- Eles não têm efeitos negativos ou colaterais em outros navegadores.
- Eles não usam nenhuma forma de navegador ou detecção de recursos.

As instâncias para aplicar essas correções podem ser raras, mas elas são uma tática pela qual devemos sempre nos empenhar em nossos aplicativos.

Vejamos um exemplo. O seguinte snippet de código representa uma mudança (retirada do jQuery) que ocorreu ao trabalhar com o Internet Explorer:

```
// ignore os valores negativos de largura e altura
if ((key == 'width' || key == 'height') && parseFloat (value) <0)
 valor = indefinido;
```

Algumas versões do IE lançam uma exceção quando um valor negativo é definido no altura ou largura propriedades de estilo. Todos os outros navegadores ignoram a entrada negativa. Esta solução alternativa ignora todos os valores negativos em *todo* navegadores. Essa alteração evita que uma exceção seja lançada no Internet Explorer e não tem efeito em nenhum outro navegador. Esta é uma mudança indolor que fornece uma API unificada para o usuário (porque lançar exceções inesperadas nunca é desejado).

Outro exemplo desse tipo de correção (também do jQuery) aparece no código de manipulação de atributos. Considere isto:

```
if (nome == "tipo" &&
 elem.nodeName.toLowerCase () == "input" &&
 elem.parentNode)
 lançar "atributo de tipo não pode ser alterado";
```

O Internet Explorer não nos permite manipular o modelo atributo de elementos de entrada que já fazem parte do DOM; tentativas de alterar esse atributo resultam em uma exceção proprietária lançada. jQuery chegou a uma solução intermediária: não permite *todo* tentativas de manipular o modelo atributo em elementos de entrada injetados em todos os navegadores, lançando uma exceção informativa uniforme.

Esta mudança na base de código jQuery não requer navegador ou detecção de recurso; ele unifica a API em todos os navegadores. A ação ainda resulta em uma exceção, mas essa exceção é uniforme em todos os tipos de navegador.

Esta abordagem particular pode ser considerada controversa. Limita propositalmente os recursos da biblioteca em todos os navegadores por causa de um bug que existe em apenas um. A equipe da jQuery avaliou essa decisão com cuidado e decidiu que era melhor ter uma API unificada que funcionasse de forma consistente do que uma API que quebrasse inesperadamente ao desenvolver código para vários navegadores. Você pode se deparar com situações como essa ao desenvolver suas próprias bases de código reutilizáveis e precisará considerar cuidadosamente se uma abordagem de limitação como essa é apropriada para seu público.

O importante a ser lembrado para esses tipos de alterações de código é que eles fornecem uma solução que funciona perfeitamente em todos os navegadores sem a necessidade de detecção de navegador ou recurso, tornando-os efetivamente imunes a alterações futuras. Você deve sempre buscar soluções que funcionem dessa maneira, mesmo que as instâncias aplicáveis sejam poucas e distantes entre si.

### 14.3.2 Detecção de recursos e polyfills

Como discutimos anteriormente, *detecção de recursos* é uma abordagem comumente usada ao escrever código para vários navegadores. Essa abordagem não é apenas simples, mas também geralmente eficaz. Ele funciona determinando se um determinado objeto ou propriedade de objeto existe e, em caso afirmativo, presumindo que ele fornece a funcionalidade implícita. (Na próxima seção, veremos o que fazer nos casos em que essa suposição falha.)

Mais comumente, a detecção de recursos é usada para escolher entre várias APIs que fornecem partes duplicadas de funcionalidade. Por exemplo, o capítulo 10 explorou o `Array.prototype.find`, acessível a todos os arrays, um método que podemos usar para encontrar o primeiro item do array que satisfaça uma determinada condição. Infelizmente, o método é acessível apenas em navegadores que suportam totalmente ES6. Então, o que fazemos quando estamos presos a navegadores que ainda não oferecem suporte a esse recurso? Em geral, como lidamos com os recursos ausentes nos navegadores?

A resposta é polyfilling! Um polyfill é um substituto do navegador. Se um navegador não oferece suporte a uma funcionalidade específica, fornecemos nossa própria implementação. Por exemplo, a Mozilla Developer Network (MDN) fornece polyfills para uma ampla gama de funcionalidades ES6. Entre outras coisas, isso inclui a implementação de JavaScript do `Array.prototype.find` método ( <http://mng.bz/d9IU> ), conforme mostrado na lista a seguir.

Listagem 14.2 Um polyfill para o `Array.prototype.find` método

```
if (! Array.prototype.find) {
 Array.prototype.find = function (predicado) {
 if (this === null) {
 lance new TypeError ('find chamado em null ou undefined');
 }
 if (predicado typeOf! == 'function') {
 // ...
 }
 }
}
```

**Especifica**  
**nosso próprio**  
**implementação**

← **Fornecer um polyfill apenas se o navegador atual não implementar o método**



```

 lance novo TypeError ('predicado deve ser uma função');
 }
 var list = Object (this);
 var length = list.length >>> 0; var thisArg = argumentos [1];
 valor var;

 para (var i = 0; i < comprimento; i++) {
 valor = lista [i];
 if (predicate.call (thisArg, value, i, list)) {
 valor de retorno;
 }
 }
 retornar indefinido;
};
}

```

← Certifica-se de que o comprimento é um número inteiro não negativo

Encontra o primeiro item da matriz que satisfaz um predicado

Neste exemplo, primeiro usamos a detecção de recursos para verificar se o navegador atual tem suporte integrado para o `achar método`:

```

if (! Array.prototype.find) {
 ...
}

```

Sempre que possível, devemos adotar a forma padrão de realizar qualquer ação. Conforme mencionado antes, isso ajudará a tornar nosso código o mais à prova de futuro possível. Por este motivo, se o navegador já suporta o método, não fazemos nada. Se estamos lidando com um navegador que ainda não alcançou o ES6, fornecemos nossa própria implementação.

Acontece que o núcleo do método é direto. Percorremos o array, chamando a função de predicado passada, que verifica se um item do array satisfaz nossos critérios. Se isso acontecer, nós o devolvemos.

Uma técnica interessante é apresentada nesta listagem:

```
var length = list.length >>> 0;
```

O operador `>>>` é o *operador zero-fill shift right*, que desloca o primeiro operando o número especificado de bits para a direita, enquanto descarta os bits em excesso. Neste caso, este operador é usado para converter o comprimento propriedade para um número inteiro não negativo. Isso é feito porque os índices de array em JavaScript devem ser inteiros sem sinal.

Um uso importante da detecção de recursos é descobrir os recursos fornecidos pelo ambiente do navegador no qual o código está sendo executado. Isso nos permite fornecer recursos que usam esses recursos em nosso código ou determinar se precisamos fornecer um fallback.

O snippet de código a seguir mostra um exemplo básico de detecção da presença de um recurso do navegador usando a detecção de recurso, para determinar se devemos fornecer a funcionalidade completa do aplicativo ou um fallback de experiência reduzida:

```
if (typeof document! == "undefined" &&
 document.addEventListener &&
 document.querySelector &&
 document.querySelectorAll) {
 // Temos API suficiente para trabalhar para construir nosso aplicativo
}
outro {
 // Provide Fallback
}
```

Aqui, testamos se

- O navegador tem um documento carregado
- O navegador fornece um meio de vincular manipuladores de eventos
- O navegador pode encontrar elementos com base em um seletor

A falha em qualquer um desses testes nos leva a recorrer a uma posição de recuo. O que é feito no fallback depende das expectativas dos consumidores do código e dos requisitos colocados no código. Algumas opções podem ser consideradas:

- Poderíamos realizar mais detecção de recursos para descobrir como fornecer uma experiência reduzida que ainda usa algum JavaScript.
- Poderíamos optar por não executar nenhum JavaScript, voltando para o HTML sem script na página.
- Poderíamos redirecionar o usuário para uma versão mais simples do site. O Google faz isso com o Gmail, por exemplo.

Como a detecção de recursos tem pouca sobrecarga (é apenas uma pesquisa de propriedade / objeto) e é relativamente simples em sua implementação, é uma boa maneira de fornecer níveis básicos de fallback, tanto no nível da API quanto do aplicativo. É uma boa escolha para a primeira linha de defesa em sua criação de código reutilizável.

### 14.3.3 Problemas não testáveis do navegador

Infelizmente, o JavaScript e o DOM têm várias áreas de problemas possíveis que são impossíveis ou proibitivamente caras para testar. Felizmente, essas situações são raras, mas quando as encontramos, sempre vale a pena gastar tempo investigando para ver se há algo que podemos fazer a respeito.

As seções a seguir discutem alguns problemas conhecidos que são impossíveis de testar usando qualquer interação JavaScript convencional.

#### EVENT HANDLER BINDINGS

Um dos lapsos irritantes dos navegadores é a incapacidade de determinar programaticamente se um manipulador de eventos foi vinculado. Os navegadores não fornecem nenhuma maneira de determinar se alguma função foi associada a um ouvinte de evento em um elemento. Não há como remover todos os manipuladores de eventos vinculados de um elemento, a menos que tenhamos mantido referências a todos os manipuladores vinculados à medida que os criamos.

## EVENT FIRING

Outro agravante é determinar se um evento será disparado. Embora seja possível determinar se um navegador oferece suporte a um meio de vincular um evento, é *não* possível saber se um navegador irá disparar um evento. Isso se torna problemático em alguns lugares.

Primeiro, se um script for carregado dinamicamente após a página já ter sido carregada, o script pode tentar vincular um ouvinte para aguardar o carregamento da janela quando, na verdade, esse evento já aconteceu. Como não há como determinar se o evento já ocorreu, o código pode ficar esperando uma eternidade para ser executado.

A segunda situação ocorre se um script deseja usar eventos personalizados fornecidos por um navegador como alternativa. Por exemplo, o Internet Explorer fornece `mouseenter` e `mouseleave` eventos, que simplificam o processo de determinar quando o mouse de um usuário entra ou sai dos limites de um elemento. Estes são frequentemente usados como alternativas para o `mouseover` e `mouseout` eventos, porque agem um pouco mais intuitivamente do que os eventos padrão. Mas, como não há como determinar se esses eventos serão disparados sem primeiro vinculá-los e aguardar a interação do usuário com eles, é difícil usá-los em código reutilizável.

## EFEITOS DE PROPRIEDADE CSS

Outro ponto problemático é determinar se a alteração de certas propriedades CSS afeta a apresentação. Algumas propriedades CSS afetam apenas a representação visual da tela e nada mais; eles não mudam os elementos circundantes ou afetam outras propriedades no elemento. Exemplos são `color`, `backgroundColor`, e `opacity`.

Não há como determinar programaticamente se a alteração dessas propriedades de estilo gerará os efeitos desejados. A única maneira de verificar o impacto é por meio de um exame visual da página.

## BROWSER CRASHES

Testar script que faz com que o navegador trave é outro aborrecimento. O código que causa o travamento de um navegador é especialmente problemático, porque, ao contrário das exceções que podem ser facilmente capturadas e tratadas, elas sempre farão com que o navegador falhe.

Por exemplo, em versões mais antigas do Safari (consulte <http://bugs.jquery.com/ticket/1331>), criar uma expressão regular que usasse intervalos de caracteres Unicode sempre faria com que o navegador travasse, como no exemplo a seguir:

```
novo RegExp ("[\ w \ u0128- \ uFFFF * _] +");
```

O problema com isso é que não é possível testar se esse problema existe, porque o próprio teste sempre produzirá uma falha no navegador mais antigo.

Além disso, os bugs que causam travamentos para sempre ficam envolvidos em dificuldades, porque embora possa ser aceitável que o JavaScript seja desabilitado em algum segmento da população que usa seu navegador, nunca é aceitável travar completamente o navegador desses usuários.

## APIS INCONGRUOSOS

Um tempo atrás, vimos que o jQuery decidiu proibir a capacidade de alterar o modelo atributo em todos os navegadores devido a um bug no Internet Explorer. Nós *poderia* teste esse recurso e desative-o apenas no IE, mas isso configuraria uma incongruência, já que a API funcionaria de maneira diferente de navegador para navegador. Nessas situações, quando um bug é tão grave que causa a quebra de uma API, a única opção é contornar a área afetada e fornecer uma solução diferente.

Além de problemas impossíveis de testar, alguns problemas são *possível* para testar, mas são proibitivamente difíceis de testar com eficácia. Vejamos alguns deles.

## DESEMPENHO API

Às vezes, APIs específicas são mais rápidas ou mais lentas em navegadores diferentes. Ao escrever código reutilizável e robusto, é importante tentar usar as APIs que fornecem bom desempenho. Mas nem sempre é óbvio qual API é.

A realização eficaz da análise de desempenho de um recurso geralmente envolve o lançamento de uma grande quantidade de dados nele, e isso normalmente leva um tempo relativamente longo. Portanto, não é algo que podemos fazer sempre que nossa página é carregada.

Recursos não testáveis são um aborrecimento significativo que atrapalham a escrita de JavaScript reutilizável, mas frequentemente podemos contorná-los com um pouco de esforço e inteligência. Usando técnicas alternativas ou construindo nossas APIs para evitar esses problemas em primeiro lugar, provavelmente seremos capazes de construir um código eficaz, apesar das probabilidades contra nós.

## 14.4 Reduzindo suposições

Escrever código reutilizável em vários navegadores é uma batalha de suposições, mas usando detecção e criação inteligentes, podemos reduzir o número de suposições que fazemos em nosso código. Quando fazemos suposições sobre o código que escrevemos, podemos encontrar problemas mais adiante.

Por exemplo, supor que um problema ou bug sempre existirá em um navegador específico é uma suposição enorme e perigosa. Em vez disso, testar o problema (como fizemos ao longo deste capítulo) prova ser muito mais eficaz. Em nossa codificação, devemos sempre nos esforçar para reduzir o número de suposições, reduzindo efetivamente a margem de erro e a probabilidade de que algo vai voltar e nos deixar por trás.

A área mais comum para fazer suposições em JavaScript é a detecção do agente do usuário - especificamente, analisar o agente do usuário fornecido por um navegador ( navegador

. agente de usuário) e usá-lo para fazer uma suposição sobre como o navegador se comportará (em outras palavras, detecção do navegador). Infelizmente, a maioria das análises de string de agente de usuário prova ser uma excelente fonte de erros induzidos no futuro. Assumir que um bug, problema ou recurso proprietário sempre estará vinculado a um navegador específico é uma receita para o desastre.

Mas a realidade intervém quando se trata de minimizar suposições: é virtualmente impossível remover todas elas. Em algum ponto, teremos que assumir que um navegador fará o que deve fazer. Descobrir onde encontrar esse equilíbrio depende totalmente do desenvolvedor, e é o que “separa os homens dos meninos”, como eles dizem (com desculpas às nossas leitoras).

Por exemplo, vamos reexaminar o código de anexação de evento que já vimos neste capítulo:

```
function bindEvent (element, type, handle) {
 if (element.addEventListener) {
 element.addEventListener (tipo, identificador, falso);
 }
 else if (element.attachEvent) {
 element.attachEvent ("on" + tipo, identificador);
 }
}
```

Sem olhar para frente, veja se você consegue identificar três suposições feitas por este código. Vá em frente, vamos esperar. ( *Perigo* o tema joga ...)

Como você está? O código anterior tem pelo menos estas três suposições:

- As propriedades que estamos verificando são funções que podem ser chamadas.
- São as funções corretas, realizando as ações que esperamos.
- Esses dois métodos são as únicas maneiras possíveis de vincular um evento.

Poderíamos facilmente nos livrar da primeira suposição adicionando verificações para ver se as propriedades são, de fato, funções. Lidar com os dois pontos restantes é muito mais difícil.

Neste código, sempre precisamos decidir quantas premissas são ideais para nossos requisitos, nosso público-alvo e nós. Frequentemente, reduzir o número de suposições também aumenta o tamanho e a complexidade da base de código. É totalmente possível, e bastante fácil, tentar reduzir as suposições ao ponto da completa insanidade, mas em algum ponto temos que parar e fazer um balanço do que temos, dizer “bom o suficiente” e trabalhar a partir daí. Lembre-se de que mesmo o código que menos supõe ainda está sujeito a regressões introduzidas por um navegador.

## 14.5 Resumo

- Embora a situação tenha melhorado consideravelmente, os navegadores infelizmente não estão isentos de bugs e geralmente não suportam os padrões da web de forma consistente.
- Ao escrever aplicativos JavaScript, a escolha de quais navegadores e plataformas suportar é uma consideração importante.
- Como não é possível oferecer suporte a todas as combinações, a qualidade nunca deve ser sacrificada pela cobertura!
- Os maiores desafios para escrever código JavaScript que pode ser executado em vários navegadores são correções de bugs, regressões, bugs do navegador, recursos ausentes e código externo.
- O desenvolvimento reutilizável em vários navegadores envolve vários fatores:
  - *Tamanho do código* —Mantendo o tamanho do arquivo pequeno
  - *Sobrecarga de desempenho* —Mantendo o nível de desempenho acima de um mínimo palatável
  - *Qualidade API* —Certificando-se de que as APIs funcionam uniformemente em todos os navegadores

- Não existe uma fórmula mágica para determinar o equilíbrio correto desses fatores. Os fatores de
- desenvolvimento são algo que deve ser equilibrado por cada desenvolvedor em seus esforços de desenvolvimento individual.
- Usando técnicas inteligentes, como detecção de recursos, podemos nos defender contra as inúmeras direções a partir das quais o código reutilizável será atacado sem fazer sacrifícios indevidos.

## 14.6 Exercícios

- 1 O que devemos levar em consideração ao decidir quais navegadores oferecer suporte?
- 2 Explique o problema de IDs gananciosos.
- 3 O que é detecção de recurso?
- 4 O que é um polyfill do navegador?



# apêndice A

## Recursos ES6 adicionais

---

### *Este apêndice cobre*

- Literais de modelo
- Destruição
- Aprimoramentos literais de objeto

Este apêndice cobre alguns dos recursos “menores” do ES6 que não se encaixam perfeitamente nos capítulos anteriores. *Literais de modelo* permitir a interpolação de strings e strings de várias linhas, *desestruturação* nos permite extrair facilmente dados de objetos e matrizes, e

*literals de objeto aprimorados* melhorar o relacionamento com, bem, literais de objeto.

### *Literais de modelo*

Literais de modelo são um novo recurso do ES6 que torna a manipulação de strings muito mais agradável do que antes. Basta pensar no passado; quantas vezes você foi forçado a escrever algo tão feio quanto isso?



```
const ninja = {
 nome: "Yoshi",
 ação: "subterfúgio"
};

const concatMessage = "Nome:" + ninja.name + ""
 + "Ação:" + ninja.action;
```

Neste exemplo, temos que construir uma string com dados inseridos dinamicamente. Para conseguir isso, temos que recorrer a algumas concatenações complicadas. Mas não mais! No ES6, podemos obter o mesmo resultado com literais de modelo; basta dar uma olhada na lista a seguir.

#### Listagem A.1 Literais de modelo

```
const ninja = {
 nome: "Yoshi",
 ação: "subterfúgio"
};

const concatMessage = "Nome:" + ninja.name + ""
 + "Ação:" + ninja.action;

const templateMessage = `Nome: ${ninja.name} Ação: ${ninja.action}`;

assert (concatMessage === templateMessage,
 "Nossas mensagens correspondem");
```

Usa crases para criar literais de modelo que podem conter expressões JavaScript encapsuladas em \$ {}

Como você pode ver, o ES6 fornece um novo tipo de string que usa crases (`), uma string que pode conter marcadores de posição, denotados com a sintaxe \$ {}. Dentro desses espaços reservados, podemos colocar qualquer expressão JavaScript: uma variável simples, um acesso de propriedade de objeto (como fizemos com `ninja.action`), e até mesmo chamadas de função.

Quando uma string de modelo é avaliada, os marcadores de posição são substituídos pelo resultado da avaliação da expressão JavaScript contida nesses marcadores.

Além disso, os literais de modelo não estão limitados a uma única linha (assim como os padrões entre aspas duplas e simples), e não há nada que nos impeça de torná-los multilinhas, como mostrado na lista a seguir.

#### Listagem A.2 Literais de modelo multilinha

```
nome const = "Yoshi", ação = "subterfúgio"; const multilineString =
`Nome: ${name}
Yoshi: ${action}`;
```

Strings de modelo não são limitado a uma única linha.

Agora que fornecemos uma breve introdução aos literais de template, vamos dar uma olhada em outro recurso do ES6: desestruturação.

## Destruição

A desestruturação nos permite extrair facilmente dados de objetos e arrays usando padrões. Por exemplo, imagine que você tem um objeto cujas propriedades deseja atribuir a algumas variáveis, como na listagem a seguir.

### Listagem A.3 Destruturando objetos

```
const ninja = {nome: "Yoshi", ação: "skulk", arma: "shuriken"};
```

```
const nameOld = ninja.name; const actionOld = ninja.action;
const weaponOld = ninja.weapon;
```

**Método antigo:** temos que atribuir explicitamente cada propriedade do objeto a uma variável.

```
const {nome, ação, arma} = ninja;
```

```
afirmar (nome === "Yoshi", "Nosso ninja Yoshi"); assert (action === "skulk", "is skulk"); afirmar
(arma === "shuriken", "com uma shuriken");
```

**Desestruturação de objetos:** nós pode atribuir cada propriedade a uma variável com o mesmo nome, tudo de uma vez.

```
const {nome: meuNome, ação: minhaAção, arma: minhaArme} = ninja;
```

```
assert (myName === "Yoshi", "Nosso ninja Yoshi"); assert (myAction === "skulk", "is skulk"); assert
(myWeapon === "shuriken", "com um shuriken");
```

**Se necessário, podemos nomear explicitamente as variáveis para as quais queremos atribuir valores.**

Como mostra a listagem A.3, com a desestruturação do objeto, podemos facilmente extrair várias variáveis de um literal de objeto, tudo de uma vez. Considere a seguinte declaração:

```
const {nome, ação, arma} = ninja;
```

Isso cria três novas variáveis ( nome, ação, e arma) cujos valores são os valores das propriedades correspondentes do objeto no lado direito da instrução ( ninja.name, ninja.action, e ninja.weapon, respectivamente).

Quando não queremos usar os nomes das propriedades do objeto, podemos ajustá-los, como na seguinte instrução:

```
const {nome: meuNome, ação: minhaAção, arma: minhaArme} = ninja;
```

Aqui, criamos três variáveis ( myName, myAction, e minha arma) e atribuir a eles valores das propriedades do objeto especificado.

Anteriormente, mencionamos que também podemos desestruturar arrays, pois os arrays são apenas um tipo especial de objeto. Dê uma olhada na lista a seguir.

### Listagem A.4 Destructuring arrays

```
const ninjas = ["Yoshi", "Kuma", "Hattori"];
```

```
const [primeiroNinja, segundoNinja, terceiroNinja] = ninjas;
```

```
assert (firstNinja === "Yoshi", "Yoshi é nosso primeiro ninja"); assert (secondNinja === "Kuma", "Kuma o
segundo");
```

**Os itens da matriz são, em ordem, atribuídos a variáveis especificadas.**

```
assert (thirdNinja === "Hattori", "E Hattori the terceiro");
```

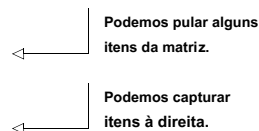
```
const [, terceiro] = ninjas;
```

```
assert (terceiro === "Hattori", "Podemos pular itens");
```

```
const [primeiro, ... restantes] = ninjas;
```

```
afirmar (primeiro === "Yoshi", "Yoshi é novamente nosso primeiro ninja"); assert (restante.length === 2, "Existem dois ninjas restantes");
```

```
afirmar (restante [0] === "Kuma", "Kuma é o primeiro ninja restante"); afirmar (restante [1] === "Hattori", "Hattori o segundo ninja restante");
```



Destruir matrizes é um pouco diferente de desestruturar objetos, principalmente na sintaxe, porque as variáveis são colocadas entre colchetes (ao contrário de colchetes, que são usados para desestruturar objetos), conforme mostrado no fragmento a seguir:

```
const [primeiroNinja, segundoNinja, terceiroNinja] = ninjas;
```

Nesse caso, Yoshi, o primeiro ninja, é atribuído à variável `firstNinja`. Kuma é atribuído à variável `secondNinja`. Hattori é atribuído à variável `thirdNinja`.

A desestruturação de array também tem alguns usos avançados. Por exemplo, se quisermos pular certos itens, podemos omitir os nomes das variáveis, mantendo as vírgulas, como na seguinte instrução:

```
const [, , terceiro] = ninjas;
```

Neste caso, os dois primeiros ninjas serão ignorados, enquanto o valor do terceiro ninja, Hattori, será atribuído à variável `terceiro`.

Além disso, podemos extrair apenas alguns itens, ao atribuir os itens restantes a uma nova matriz:

```
const [primeiro, ... restantes] = ninjas;
```

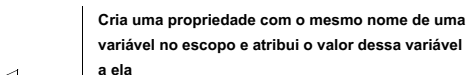
O primeiro item, Yoshi, é atribuído à variável `primeiro`, e os ninjas restantes, Kuma e Hattori, são atribuídos à nova matriz, remanescente. Observe que, neste caso, os itens restantes são marcados da mesma forma que os parâmetros restantes (o operador `...`).

### Líterais de objeto aprimorados

Uma das grandes coisas sobre JavaScript é sua facilidade de criar objetos com literais de objeto: definimos algumas propriedades e as envolvemos entre chaves e voilà, criamos um novo objeto. No ES6, a sintaxe literal do objeto ganhou algumas novas extensões. Vejamos um exemplo. Digamos que queremos criar um ninja objeto e atribua a ele uma propriedade com base no valor de uma variável que está no escopo, uma propriedade cujo nome é calculado dinamicamente e um método, conforme mostrado na lista a seguir.

#### Listagem A.5 Literais de objeto aprimorados

```
const name = "Yoshi";
const oldNinja = {
 nome: nome,
```



```

 getName: function () {
 return this.name;
 }
};

oldNinja ["antigo" + nome] = verdadeiro;
assert (oldNinja.name === "Yoshi", "Yoshi aqui");
assert (typeof oldNinja.getName === "function", "with a method"); assert ("oldYoshi" em oldNinja, "e uma propriedade
dinâmica");

const newNinja = {
 nome,
 getName () {
 return this.name;
 },
 ["novo" + nome]: verdadeiro
};

assert (newNinja.name === "Yoshi", "Yoshi aqui, novamente"); assert (typeof newNinja.getName === "function", "with a
method"); assert ("newYoshi" em newNinja, "e uma propriedade dinâmica");

```

Define um método em um objeto

Cria uma propriedade cujo nome é calculado dinamicamente

Sintaxe abreviada do valor da propriedade; atribui o valor da mesma variável nomeada à propriedade

Abreviação da definição do método; não há necessidade de adicionar dois pontos e a palavra-chave de função. Usar parênteses após o nome da propriedade sinaliza que estamos lidando com um método.

Um nome de propriedade computado

Este exemplo começa criando um oldNinja objeto usando a antiga sintaxe literal de objeto pré-ES6:

```

nome const = "Yoshi"; const oldNinja = {

 nome nome,
 getName: function () {
 return this.name;
 }
};

oldNinja ["antigo" + nome] = verdadeiro;

```

Comparamos isso com literais de objeto aprimorados que alcançam exatamente o mesmo efeito, com menos confusão sintática:

```

const newNinja = {
 nome,
 getName () {
 return this.name;
 },
 ["novo" + nome]: verdadeiro
};

```

Isso completa nossa exploração de novos conceitos importantes introduzidos pelo ES6.

# Apêndice B

## Armando com teste e depuração

---

### *Este apêndice cobre*

- Ferramentas para depurar código JavaScript
- Técnicas para gerar testes
- Construindo um conjunto de testes
- Pesquisando alguns dos frameworks de teste populares

Este apêndice apresenta algumas técnicas fundamentais no desenvolvimento de aplicativos da Web do lado do cliente: depuração e teste. Construir suítes de teste eficazes para nosso código é sempre importante. Afinal, se não testarmos nosso código, como saberemos se ele faz o que pretendemos? O teste nos dá um meio de garantir que nosso código não apenas execute, mas execute *corretamente*.

Além disso, tão importante quanto uma estratégia de teste sólida é para *todo* código, pode ser crucial quando fatores externos têm o potencial de afetar a operação do nosso código, que

é *exatamente* o caso que enfrentamos no desenvolvimento de JavaScript em vários navegadores. Não temos apenas os problemas típicos de garantir a qualidade do código (especialmente ao lidar com vários desenvolvedores trabalhando em uma única base de código) e nos proteger contra regressões que podem quebrar partes de uma API (problemas genéricos que todos os programadores precisam enfrentar), mas também temos o problema de determinar se nosso código funciona em todos os navegadores que escolhemos oferecer suporte.

Neste capítulo, veremos ferramentas e técnicas para depurar código JavaScript, gerar testes com base nesses resultados e construir um conjunto de testes para executar esses testes de maneira confiável. Vamos começar.

### *Ferramentas de desenvolvedor web*

Por muito tempo, o desenvolvimento de aplicativos JavaScript foi prejudicado pela falta de uma infraestrutura básica de depuração. A única maneira de depurar o código JavaScript era espalhar alertas declarando que nos notificariam sobre o valor da expressão alertada, em torno do código que estava agindo de forma estranha. Como você pode imaginar, isso tornou a depuração (difícilmente uma atividade divertida) ainda mais difícil.

Felizmente, o Firebug, uma extensão do Firefox, foi desenvolvido em 2007. Firebug ocupa um lugar especial no coração de muitos desenvolvedores da web, porque foi a primeira ferramenta que forneceu uma experiência de depuração que se equiparou a depuração de última geração ambientes de desenvolvimento integrado (IDEs), como Visual Studio ou Eclipse. Além disso, o Firebug inspirou o desenvolvimento de ferramentas de desenvolvedor semelhantes para todos os principais navegadores: Ferramentas de desenvolvedor F12, incluídas no Internet Explorer e Microsoft Edge; WebKit Inspector, incluído no Safari; Ferramentas de desenvolvedor do Firefox, incluídas no Firefox; e Chrome DevTools incluídos no Chrome e Opera. Vamos explorá-los um pouco.

#### FIREBUG

Firebug, a primeira ferramenta avançada de depuração de aplicativos da web, está disponível exclusivamente para Firefox e pode ser acessada pressionando a tecla F12 (ou clicando com o botão direito em qualquer lugar da página e selecionando Inspecionar elemento com Firebug). Você pode instalar o Firebug abrindo a página no Firefox (<https://getfirebug.com/>) e seguindo as instruções. Figura

B.1 mostra Firebug.

O Firebug oferece funcionalidades avançadas de depuração, algumas das quais foram pioneiras. Por exemplo, podemos explorar facilmente o estado atual do DOM usando o painel HTML (o painel mostrado na figura B.1), executar o código JavaScript personalizado dentro do contexto da página atual usando o console (parte inferior da figura B.1), explore o estado do nosso código JavaScript usando o painel Script e até explore as comunicações de rede a partir do painel Net.

#### FIREFOX DEVELOPER TOOLS

Além do Firebug, se você for um usuário do Firefox, poderá usar o Firefox DevTools integrado, mostrado na figura B.2. Como você pode ver, a aparência geral das ferramentas de desenvolvedor do Firefox é semelhante ao Firebug (além de alguns layout e rótulo menores

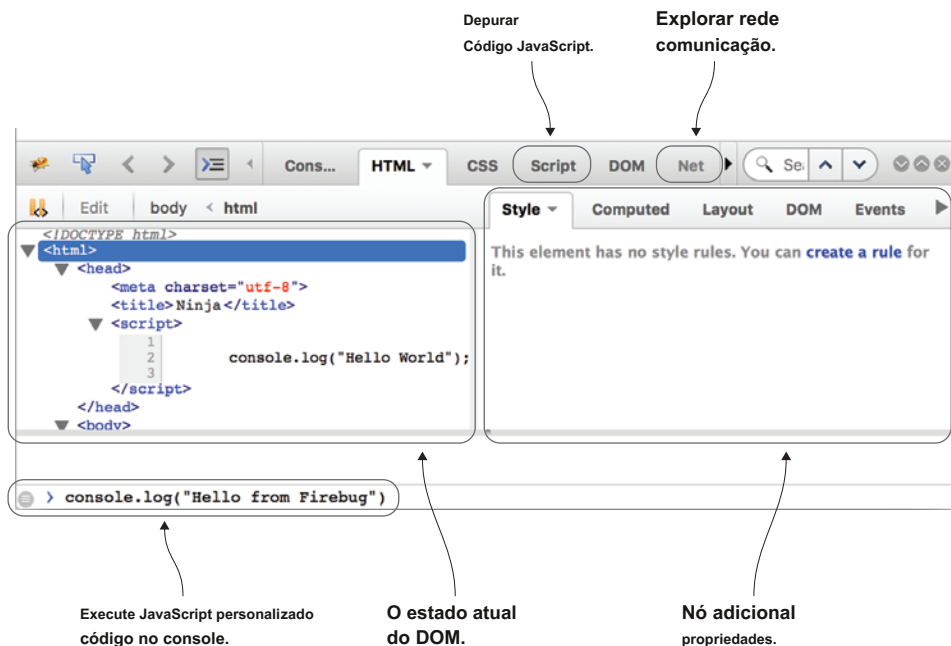


Figura B.1 O Firebug, disponível apenas no Firefox, foi a primeira ferramenta de depuração avançada para aplicativos da web.

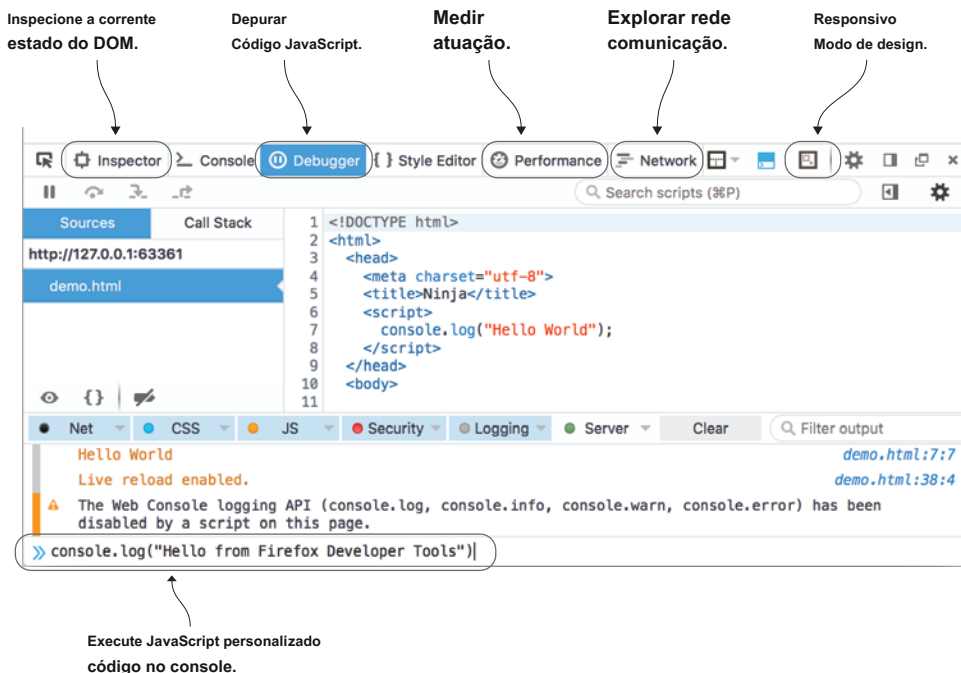


Figura B.2 As ferramentas de desenvolvedor do Firefox, integradas ao Firefox, oferecem todos os recursos do Firebug e mais alguns.

diferenças; por exemplo, o painel HTML do Firebug é denominado Inspector nas ferramentas de desenvolvedor do Firefox).

As ferramentas de desenvolvedor do Firefox são construídas pela equipe da Mozilla, que aproveitou essa integração com o Firefox trazendo alguns recursos úteis adicionais. O painel Desempenho, por exemplo, fornece uma visão detalhada sobre o desempenho de nossos aplicativos da web. Além disso, as ferramentas de desenvolvedor do Firefox são construídas com a web moderna em mente. Por exemplo, eles oferecem o modo de design responsivo, que nos ajuda a explorar a aparência de nossos aplicativos da web em diferentes tamanhos de tela, o que é algo com que devemos ter cuidado, porque hoje em dia os usuários acessam os aplicativos da web não apenas de seus PCs, mas também de dispositivos móveis, tablets e até TVs.

#### F12 DEVELOPER TOOLS

Se você está no campo do Internet Explorer (IE), ficará feliz em saber que o IE e o Microsoft Edge (o sucessor do IE) oferecem suas próprias ferramentas de desenvolvedor, as ferramentas de desenvolvedor F12. (Rapidamente, tente adivinhar qual tecla os ativa e desativa.) Essas ferramentas são mostradas na figura B.3.

Novamente, observe as semelhanças entre as ferramentas de desenvolvedor F12 e as ferramentas de desenvolvedor do Firefox (com apenas pequenas diferenças nos rótulos). As ferramentas F12 também nos permitem explorar o estado atual do DOM (o painel DOM Explorer, figura B.3), executar o código JavaScript personalizado por meio do console, depurar nosso código JavaScript (o painel Depurador), analisar o tráfego de rede (Rede), lidar com design responsivo (UI Responsiveness) e analisar o desempenho e o consumo de memória (Profiler e Memory).

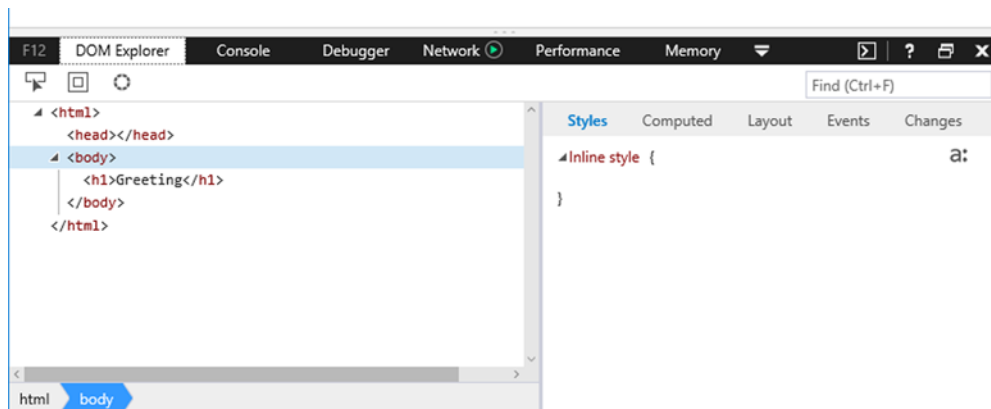


Figura B.3 As ferramentas de desenvolvedor F12 (alternadas pressionando F12) estão disponíveis no Internet Explorer e no Edge.



C EB K INSPETOR DE TI

Se você for um usuário do OS X, pode usar o WebKit Inspector, que é oferecido pelo Safari, conforme mostrado na figura B.4. Embora a IU do WebKit Inspector do Safari seja um pouco diferente

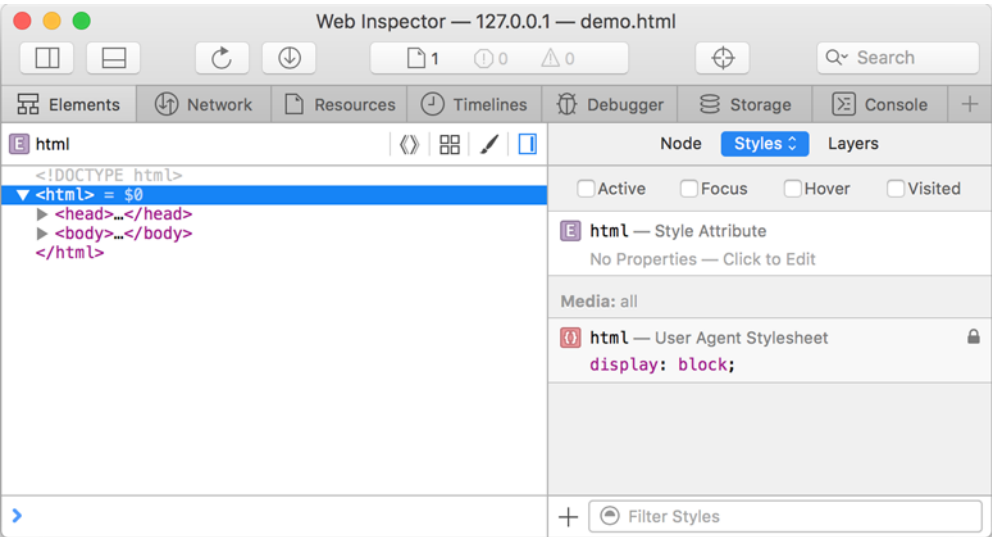


Figura B.4 WebKit Inspector, disponível no Safari

das ferramentas de desenvolvedor F12 ou ferramentas de desenvolvedor do Firefox, tenha certeza de que o WebKit Inspector também oferece suporte a todos os recursos de depuração importantes.

C HROME D EV T OOLS

Concluiremos nossa pequena pesquisa de ferramentas para desenvolvedores com o Chrome DevTools - em nossa opinião, o carro-chefe atual das ferramentas para desenvolvedores de aplicativos da web que está gerando muitas inovações ultimamente. Como você pode ver na figura B.5, a IU e os recursos básicos são semelhantes ao restante das ferramentas do desenvolvedor.

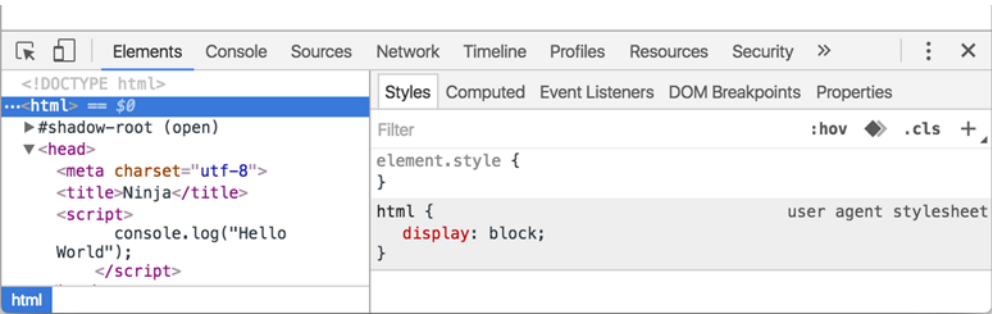


Figura B.5 Chrome DevTools, disponível no Chrome e Opera

Ao longo deste livro, usamos Chrome DevTools, por uma questão de convenção. Mas, como você viu ao longo desta seção, a maioria das ferramentas do desenvolvedor oferece recursos semelhantes (e se uma delas oferece algo novo, as outras o atualizam rapidamente). Você pode usar facilmente as ferramentas de desenvolvedor oferecidas pelo navegador de sua escolha.

Agora que você teve uma introdução às ferramentas que pode usar para depurar código, vamos explorar algumas técnicas de depuração.

### Código de depuração

Uma parte significativa do tempo de desenvolvimento de software é gasta na remoção de bugs irritantes. Embora isso às vezes possa ser interessante, quase como resolver um mistério polêmico, normalmente queremos nosso código funcionando corretamente e sem bugs o mais rápido possível.

Depurar JavaScript tem dois aspectos importantes:

- *Exploração madeireira*, que imprime o que está acontecendo enquanto nosso código está sendo executado
- *Breakpoints*, que nos permitem pausar temporariamente a execução de nosso código e explorar o estado atual do aplicativo

Ambos são úteis para responder à pergunta importante, "O que está acontecendo em meu código?" mas cada um o aborda de um ângulo diferente. Vamos começar examinando o registro.

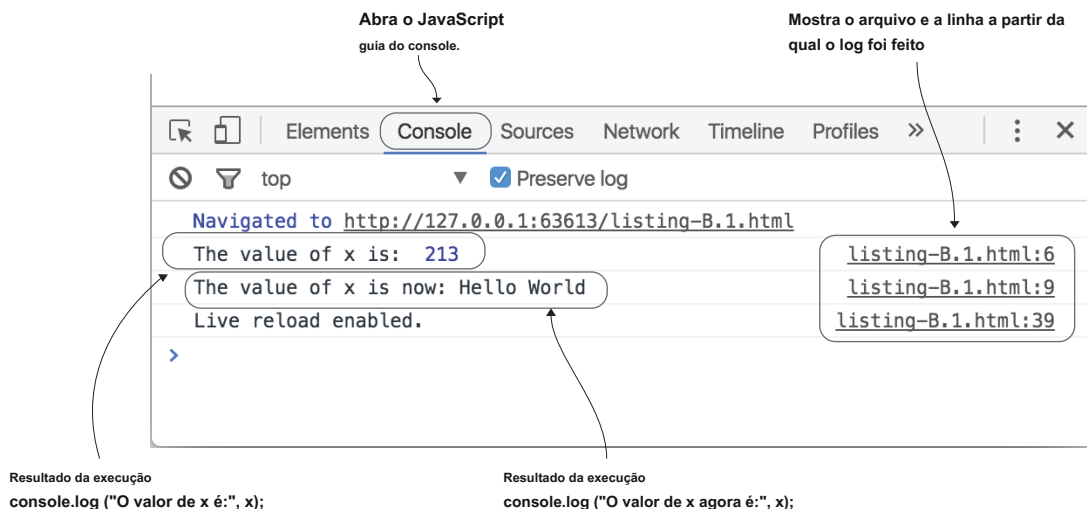
#### Exploração madeireira

*Exploração madeireira* instruções são usadas para enviar mensagens durante a execução do programa, sem impedir o fluxo normal do programa. Quando adicionamos instruções de log ao nosso código (por exemplo, usando o `console.log`), nos beneficiamos de ver as mensagens no console do navegador. Por exemplo, se quisermos saber o valor de uma variável chamada `x` em certos pontos da execução do programa, podemos escrever algo como a lista a seguir.

#### Listagem B.1 Registrando o valor da variável `x` em vários pontos de execução do programa

```
<!DOCTYPE html>
1: <html>
2: <head>
3: <title> Registro </title>
4: <script>
5: var x = 213;
6: console.log ("O valor de x é:", x);
7:
8: x = "Olá" + "Mundo";
9: console.log ("O valor de x agora é:", x); </script>
10:
11: </head>
12: <body> </body>
13: </html>
```

A Figura B.6 mostra o resultado da execução desse código no navegador Chrome com o console JavaScript habilitado.



**Figura B.6** O registro nos permite ver o estado de nosso código enquanto ele está sendo executado. Nesse caso, podemos ver que o valor de 213 é registrado na linha 6 e o valor de "Hello World" na linha 9 da listagem B.1. Todas as ferramentas de desenvolvedor, incluindo Chrome DevTools mostradas aqui, têm uma guia Console para fins de registro.

Como você pode ver, o navegador registra as mensagens diretamente no console JavaScript, mostrando a mensagem registrada e a linha na qual a mensagem foi registrada.

Este é um exemplo simples de registro de um valor de uma variável em diferentes pontos de execução do programa. Mas, em geral, você pode usar o registro para explorar várias facetas de seus aplicativos em execução, como a execução de funções importantes, a alteração de uma propriedade de objeto importante ou a ocorrência de um evento específico.

O registro é muito bom para ver o estado das coisas enquanto o código está sendo executado, mas às vezes queremos parar a ação e dar uma olhada. É aí que entram os pontos de interrupção.

## Breakpoints

Usando *pontos de interrupção* podem ser mais complexos do que o registro, mas possuem uma vantagem notável: eles interrompem a execução de um script em uma linha de código específica, parando o navegador. Isso nos permite investigar vagarosamente o estado de todos os tipos de coisas no momento do intervalo.

Digamos que temos uma página que registra uma saudação a um ninja famoso, conforme mostrado na lista a seguir.

Listagem B.2 Uma página simples de "cumprimentar um ninja"

```
<!DOCTYPE html>
<html>
 <head>
 <title> Saudação Ninja </title>
 <script>
 function logGreeting (name) {
 console.log ("Saudações ao grande" + nome);
```

```

 }
 var ninja = "Hattori Hanzo";
 logGreeting (ninja);
 </script>
</head>
<body>
</body>
</html>

```

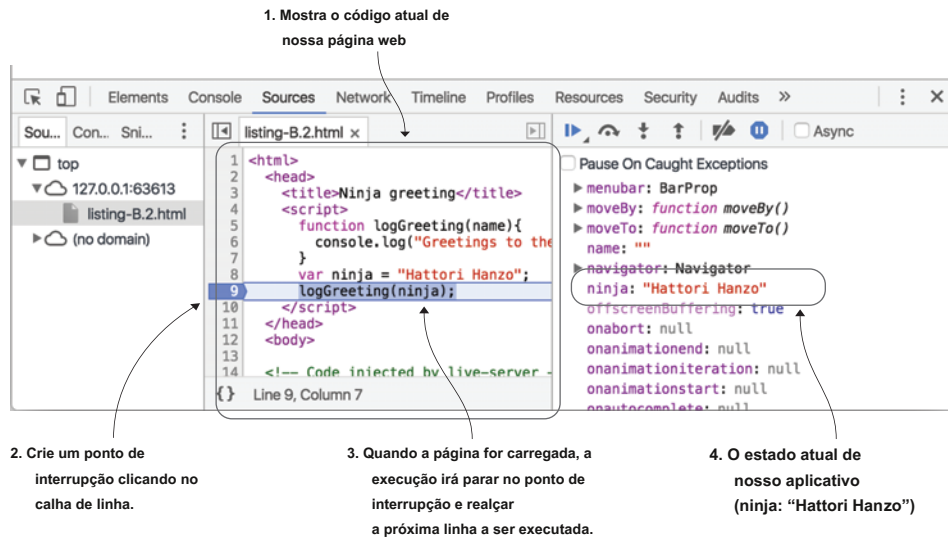
← Linha onde quebraremos

Digamos que definimos um ponto de interrupção usando o Chrome DevTools na linha anotada que chama o logGreeting função na listagem B.2 (clcando na medianiz do número da linha no painel Depurador) e atualize a página para fazer com que o código seja executado. O depurador então pararia a execução nessa linha e nos mostraria a tela na figura B.7.

O painel à direita mostra o estado do aplicativo no qual nosso código está sendo executado, incluindo o valor do ninja variável ( Hattori Hanzo). O depurador quebra em uma linha *antes* a linha de breakpoint é executada; neste exemplo, a chamada para o logGreeting função ainda não foi executada.

#### ENTRANDO EM UMA FUNÇÃO

Se estivermos tentando depurar um problema com nosso logGreeting função, podemos querer *entrar* essa função para ver o que está acontecendo dentro dela. Enquanto nossa execução está pausada no logGreeting chamada (com um ponto de interrupção que definimos anteriormente), clicamos no botão Step Into (mostrado como uma seta apontando para um ponto na maioria dos depuradores) ou pressionamos F11, o que fará com que o depurador seja executado até a primeira linha de nosso logGreeting função. A Figura B.8 mostra o resultado.



**Figura B.7** Quando definimos um ponto de interrupção em uma linha de código (clcando na medianiz da linha) e carregamos a página, o navegador para de executar o código JavaScript antes que essa linha seja executada. Em seguida, você pode explorar vagorosamente o estado atual do aplicativo no painel à direita.

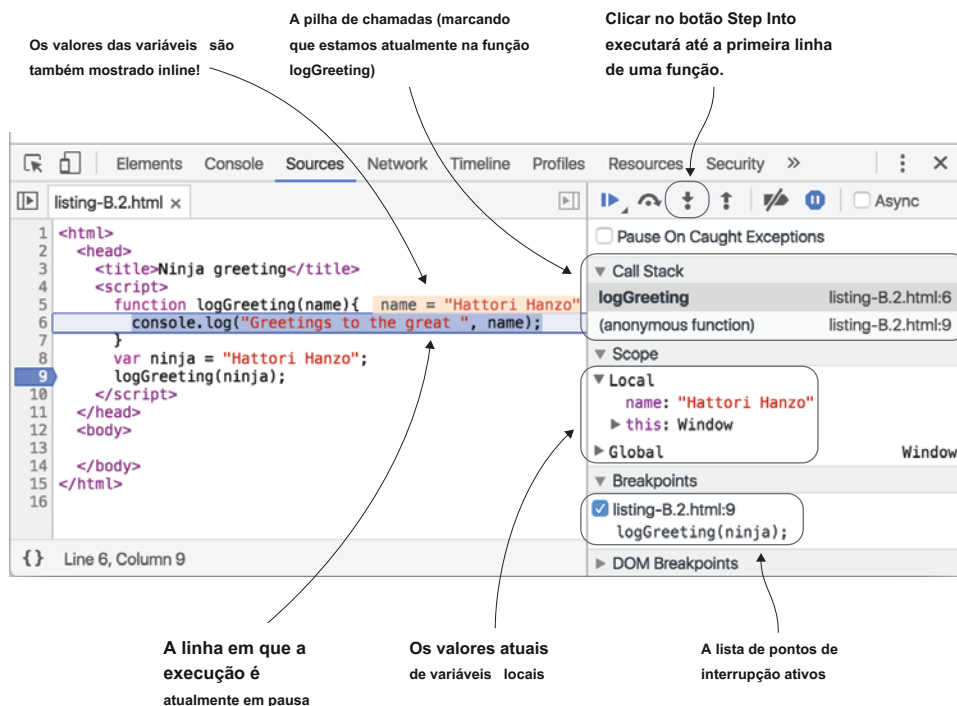
Observe que a aparência do Chrome DevTools mudou um pouco (em comparação com a figura B.7) para nos permitir vasculhar o estado do aplicativo em que o `logGreeting` a função é executada. Por exemplo, agora podemos explorar facilmente as variáveis locais de nosso `logGreeting` função e ver se temos um nome variável com o valor `Hattori Hanzo` (os valores das variáveis são mesmo mostrados em linha, com o código-fonte à esquerda). Observe também que no canto superior direito está um painel `Call Stack`, que mostra que estamos atualmente no `logGreeting` função, que foi chamada pelo código global.

### Avançar e sair

Além do comando `Step Into`, podemos usar `Step Over` e `Step Out`.

O comando `Step Over` executa nosso código linha por linha. Se o código na linha executada contiver uma chamada de função, o depurador passa por cima da função (a função será executada, mas o depurador não saltará para seu código).

Se tivermos pausado a execução de uma função, clicar no botão `Step Out` executará o código até o final da função, e o depurador fará uma nova pausa logo após a execução ter deixado essa função.



**Figura B.8** Entrar em uma função nos permite ver o novo estado em que a função é executada. Podemos explorar a posição atual estudando o `Call Stack` e os valores atuais das variáveis locais.

## C PONTOS DE INTERVALO ONDICIONAIS

Os pontos de interrupção padrão fazem com que o depurador pare a execução do aplicativo sempre que um depurador atingir esse ponto específico na execução do programa. Em certos casos, isso pode ser cansativo. Considere a seguinte lista.

## Listagem B.3 Contando Ninjas e pontos de interrupção condicionais

```
<!DOCTYPE html>
<html>
 <head>
 <script>
 para (var i = 0; i < 100; i++) {
 console.log("Ninjas:" + i);
 }
 </script>
 </head>
 <body>
 </body>
</html>
```

E se quisermos explorar o estado do aplicativo ao contar o 50° ninja? Devemos ter que passar tediosamente pelos primeiros 49?

Imagine que queremos explorar o estado do aplicativo ao contar o 50° ninja. Seria cansativo ter que visitar todos os 49 ninjas antes de finalmente chegar ao que queremos?

Bem-vindo aos pontos de interrupção condicionais! Ao contrário dos pontos de interrupção tradicionais, que param toda vez que a linha do ponto de interrupção é executada, um *pontos de interrupção condicional* faz com que o depurador seja interrompido apenas se uma expressão associada aos pontos de interrupção condicionais for satisfeita. Você pode definir um ponto de interrupção condicional clicando com o botão direito do mouse na sarjeta do número da linha e escolhendo Adicionar (consulte a figura B.9 para saber como isso é feito no Chrome).

Ao associar a expressão: `i == 49` com um ponto de interrupção condicional, o depurador irá parar apenas quando essa condição for satisfeita. Dessa forma, podemos pular imediatamente para o ponto da execução do aplicativo em que estamos interessados e ignorar os menos interessantes.

Até agora, você viu como usar várias ferramentas de desenvolvedor de diferentes navegadores para depurar nosso código com registro e pontos de interrupção. Todas essas são ótimas ferramentas que

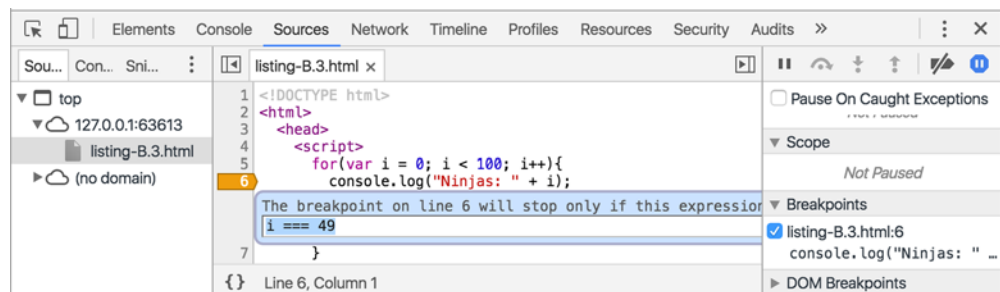


Figura B.9 Clique com o botão direito na margem do número da linha para definir um ponto de interrupção como condicional; observe que eles são mostrados em uma cor diferente, geralmente laranja.

ajude-nos a localizar bugs específicos e a compreender melhor a execução de um aplicativo específico. Mas, além disso, queremos ter uma infraestrutura que nos ajude a detectar bugs o mais rápido possível. Isso pode ser alcançado com testes.

### *Criação de testes*

Robert Frost escreveu que “boas cercas fazem bons vizinhos”, mas no mundo dos aplicativos da web e, na verdade, em qualquer disciplina de programação, bons testes são bons códigos. Observe a ênfase na palavra *Boa*. É possível ter um conjunto de testes extenso que não ajuda em nada a qualidade do nosso código se os testes forem mal construídos.

Bons testes exibem três características importantes:

- *Repetibilidade* —Nossos resultados de teste devem ser altamente reproduzíveis. Os testes executados repetidamente sempre devem produzir exatamente os mesmos resultados. Se os resultados do teste não forem determinísticos, como saberemos quais resultados são válidos e quais são inválidos? Além disso, a reprodutibilidade garante que nossos testes não dependam de fatores externos, como cargas de rede ou CPU.
- *Simplicidade* —Nossos testes devem se concentrar em testar *1* coisa. Devemos nos esforçar para remover o máximo de marcação HTML, CSS ou JavaScript possível, sem interromper a intenção do caso de teste. Quanto mais removemos, maior a probabilidade de que o caso de teste seja influenciado apenas pelo código específico que estamos testando.
- *Independência* —Nossos testes devem ser executados isoladamente. Devemos evitar tornar os resultados de um teste dependentes de outro. Dividir os testes nas menores unidades possíveis nos ajudará a determinar a origem exata de um bug quando ocorre um erro.

Podemos usar várias abordagens para construir testes. As duas abordagens principais são desconstrutivas e construtivas:

- *Casos de teste desconstrutivo* —O código existente é reduzido (desconstruído) para isolar um problema, eliminando qualquer coisa que não seja pertinente ao problema. Isso ajuda a atingir as três características listadas anteriormente. Podemos começar com um site completo, mas depois de remover a marcação extra, CSS e JavaScript, chegaremos a um caso menor que reproduz o problema.
- *Casos de teste construtivos* —Começamos de um caso reduzido e bom conhecido e continuamos até sermos capazes de reproduzir o bug em questão. Para usar esse estilo de teste, precisaremos de alguns arquivos de teste simples a partir dos quais construir testes e uma maneira de gerar esses novos testes com uma cópia limpa de nosso código.

Vejamos um exemplo de teste construtivo.

Ao criar casos de teste reduzidos, podemos começar com alguns arquivos HTML com funcionalidade mínima já incluída neles. Podemos até ter arquivos iniciais diferentes para várias áreas funcionais; por exemplo, um para manipulação de DOM, um para testes Ajax, um para animações e assim por diante.

Por exemplo, a lista a seguir mostra um caso de teste DOM simples usado para testar o jQuery.

Listagem B.4 Um caso de teste DOM reduzido para jQuery

```
<style>
#teste {largura: 100px; altura: 100px; fundo: vermelho; } </style>

<div id = "teste"> </div>
<script src = "dist / jquery.js"> </script>
<script>
$(document).ready(function () {
 $("#teste").append("teste");
});
</script>
```

Outra alternativa é usar um serviço pré-construído projetado para criar casos de teste simples, por exemplo JSFiddle ( <http://jsfiddle.net/> ), CodePen ( <http://codepen.io/> ) ou JS Bin ( <http://jsbin.com/> ) Todos têm funcionalidade semelhante; eles nos permitem construir casos de teste que se tornam disponíveis em um URL exclusivo. (E você pode até incluir cópias de bibliotecas populares.) Um exemplo no JSFiddle é mostrado na figura B.10.

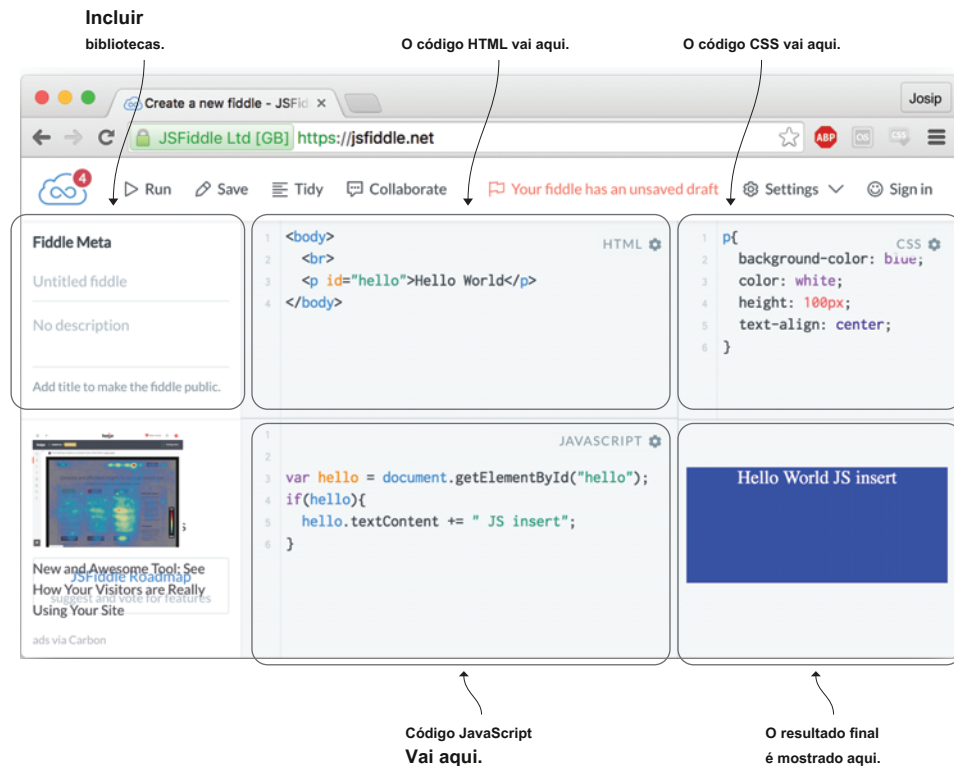


Figura B.10 O JSFiddle nos permite testar combinações de trechos de HTML, CSS e JavaScript em uma sandbox para ver se tudo funciona como planejado.



Usar o JSFiddle (ou ferramentas semelhantes) é muito bom e prático quando temos que fazer um teste rápido de um determinado conceito, especialmente porque você pode compartilhá-lo facilmente com outras pessoas, e talvez até obter algum feedback útil. Infelizmente, a execução de tais testes requer que você abra manualmente o teste e verifique seu resultado, o que pode ser bom se você tiver apenas alguns testes, mas normalmente devemos ter muitos e muitos testes que verificam cada canto e fissura de nosso código. Por este motivo, queremos automatizar nossos testes o máximo possível. Vamos ver como fazer isso.

## *Os fundamentos de uma estrutura de teste*

O objetivo principal de uma estrutura de teste é permitir-nos especificar testes individuais que podem ser agrupados em uma única unidade, para que possam ser executados em massa, fornecendo um único recurso que pode ser executado de forma fácil e repetida.

Para entender melhor como funciona uma estrutura de teste, faz sentido observar como ela é construída. Talvez surpreendentemente, as estruturas de teste de JavaScript são fáceis de construir.

Você teria que perguntar, porém, "Por que eu iria querer construir uma nova estrutura de teste?" Na maioria dos casos, escrever sua própria estrutura de teste de JavaScript não é necessário, porque muitas outras de boa qualidade já estão disponíveis (como você verá em breve). Mas construir sua própria estrutura de teste pode servir como uma boa experiência de aprendizado.

## *A afirmação*

O núcleo de uma estrutura de teste de unidade é o seu método de asserção, normalmente denominado afirmar. Este método geralmente leva um *valor* - uma expressão cuja premissa é *afirmou* - e uma descrição do propósito da afirmação. Se o valor for avaliado como verdade, a afirmação passa; caso contrário, é considerado um fracasso. A mensagem associada geralmente é registrada com um indicador de aprovação / reprovação apropriado.

Uma implementação simples desse conceito pode ser vista na lista a seguir.

### Listagem B.5 Uma implementação simples de uma asserção JavaScript

```
<!DOCTYPE html>
<html>
 <head>
 <title> Test Suite </title>
 <script>
 função assert (valor, desc) {
 var li = document.createElement ("li");
 li.className = valor? "passa": "falha";
 li.appendChild (document.createTextNode (desc));
 document.getElementById ("resultados"). appendChild (li);
 }
 window.onload = function () {
 assert (true, "The test suite is running."); assert (false, "Fail!");
 };
 </script>
 </style>
```

**Define a afirmação  
método**

**Executa testes  
usando asserções**

```
resultados li.pass {color: green; }
resultados li.fail {color: red; } </style>
```

Define estilos  
para resultados

```
</head>
<body>
 <ul id = "results">
</body>
</html>
```

← Retém os resultados do teste

A função chamada afirmar é quase surpreendentemente simples. Isso cria um novo `<li>` elemento contendo a descrição, atribui uma classe chamada passar ou falhou, dependendo do valor do parâmetro de asserção ( valor), e anexa o novo elemento a um elemento de lista no corpo do documento.

O conjunto de testes consiste em dois testes triviais: um que sempre terá sucesso e outro que sempre falhará:

```
assert (true, "The test suite is running."); // Sempre passará
assert (false, "Fail!"); // Sempre falhará
```

Regras de estilo para o passar e falhou classes indicam visualmente sucesso ou fracasso usando cores.

O resultado da execução de nosso conjunto de testes no Chrome é mostrado na figura B.11.

**GORJETA** Se você está procurando algo rápido, você pode usar o integrado `console.assert ()` método (ver figura B.12).

Agora que construímos nossa própria estrutura de teste rudimentar, vamos conhecer algumas das estruturas de teste mais populares e amplamente disponíveis.

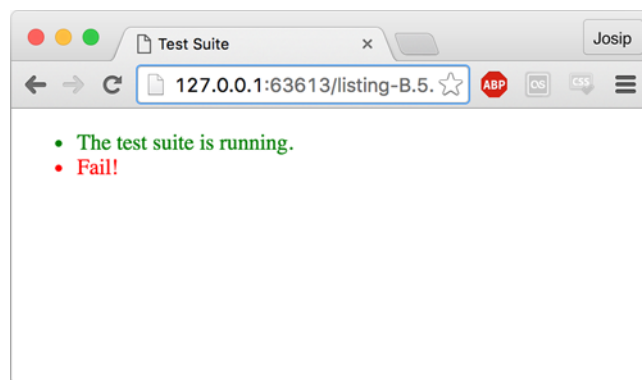


Figura B.11 O resultado da execução de nosso primeiro conjunto de testes

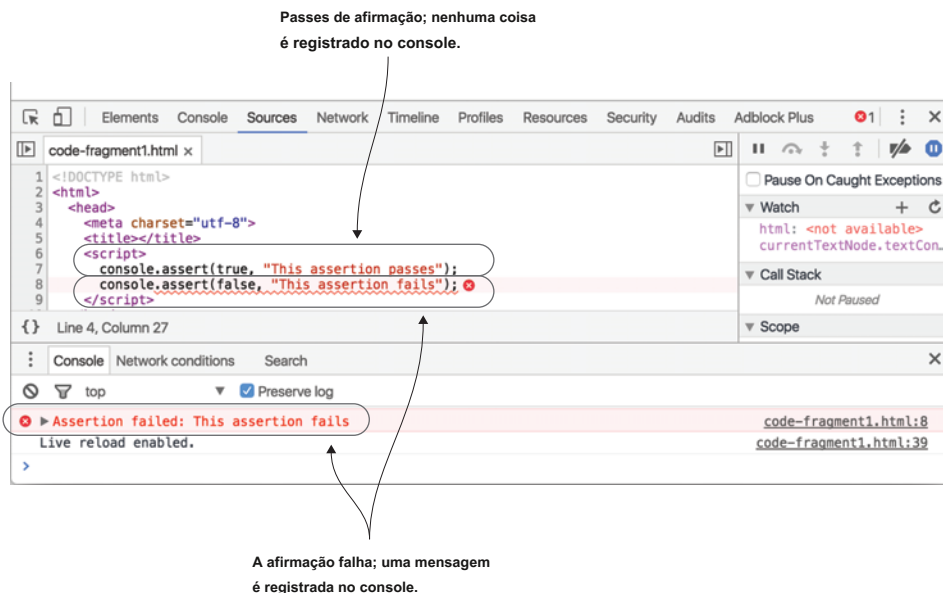


Figura B.12 Você pode usar o integrado **console.assert** como uma maneira rápida de testar o código. A mensagem de falha é registrada no console apenas se uma assertão falhar.

### Estruturas de teste populares

Uma estrutura de teste deve ser uma parte fundamental de seu fluxo de trabalho de desenvolvimento, portanto, você deve escolher uma estrutura que funcione particularmente bem para seu estilo de codificação e sua base de código. Uma estrutura de teste de JavaScript deve atender a uma única necessidade: exibir os resultados dos testes e tornar mais fácil determinar quais testes foram aprovados ou reprovados. Os frameworks de teste podem nos ajudar a alcançar esse objetivo sem ter que nos preocupar com nada além de criar os testes e organizá-los em coleções chamadas *suites de teste*.

Existem vários recursos que podemos querer procurar em uma estrutura de teste de unidade JavaScript, dependendo das necessidades dos testes. Alguns desses recursos incluem o seguinte:

- A capacidade de simular o comportamento do navegador (cliques, pressionamentos de tecla e assim por diante)
- Controle interativo de testes (pausar e retomar testes)
- Tratamento de tempos limite de teste assíncrono
- A capacidade de filtrar quais testes devem ser executados

Vamos conhecer as duas estruturas de teste mais populares atualmente: QUnit e Jasmine.

#### QUNIT

QUnit é a estrutura de teste de unidade construída originalmente para testar o jQuery. Desde então, ele se expandiu além de seus objetivos iniciais e agora é uma estrutura de teste de unidade autônoma.

QUnit foi projetado principalmente para ser uma solução simples para testes de unidade, fornecendo uma API mínima, mas fácil de usar. As características distintivas do QUnit são as seguintes:

- API simples
- Suporta teste assíncrono
- Não limitado a código jQuery ou jQuery Especialmente
- adequado para testes de regressão

Vamos veja um exemplo de teste QUnit na lista a seguir que testa se nós desenvolvemos uma função que diz "Oi" com precisão para um ninja.

#### Listagem B.6 exemplo de teste QUnit

```
<!DOCTYPE html>
<html>
 <head>
 <link rel = "stylesheet" href = "qunit / qunit-git.css" />
 <script src = "qunit / qunit-git.js"> </script>
 </head>
 <body>
 <div id = "qunit"> </div>
 </body>
</html>
```

**Inclui QUnit**  
código e estilos

Cria um elemento HTML que o QUnit  
preenche com os resultados do teste

```
function sayHiToNinja (ninja) {
 retornar "Oi" + ninja;
}
```

**Declara a função**  
que queremos testar

```
QUnit.test ("Ninja hello test", function (assert) {
 assert.ok (sayHiToNinja ("Hatori") == "Oi Hatori", "Aprovado"); assert.ok (falso, "Falha");
});
```

**Especifica um**  
**Caso de teste QUnit**

Testa uma afirmação de passagem

Testa uma afirmação com falha

Ao abrir este exemplo em um navegador, você deve obter os resultados mostrados na figura B.13, com uma afirmação de passagem de execução da linha digaHiToNinja ("Hatori"), e uma afirmação falha de assert.ok (falso, "Falha").

Mais informações sobre o QUnit podem ser encontradas em <http://qunitjs.com/>.

#### J COMO O MEU

Jasmine é outra estrutura de teste popular, construída em bases ligeiramente diferentes do QUnit. As principais partes da estrutura são as seguintes:

- O descrever função, que descreve suítes de teste
- O isto função, que especifica testes individuais
- O Espero função, que verifica as afirmações individuais

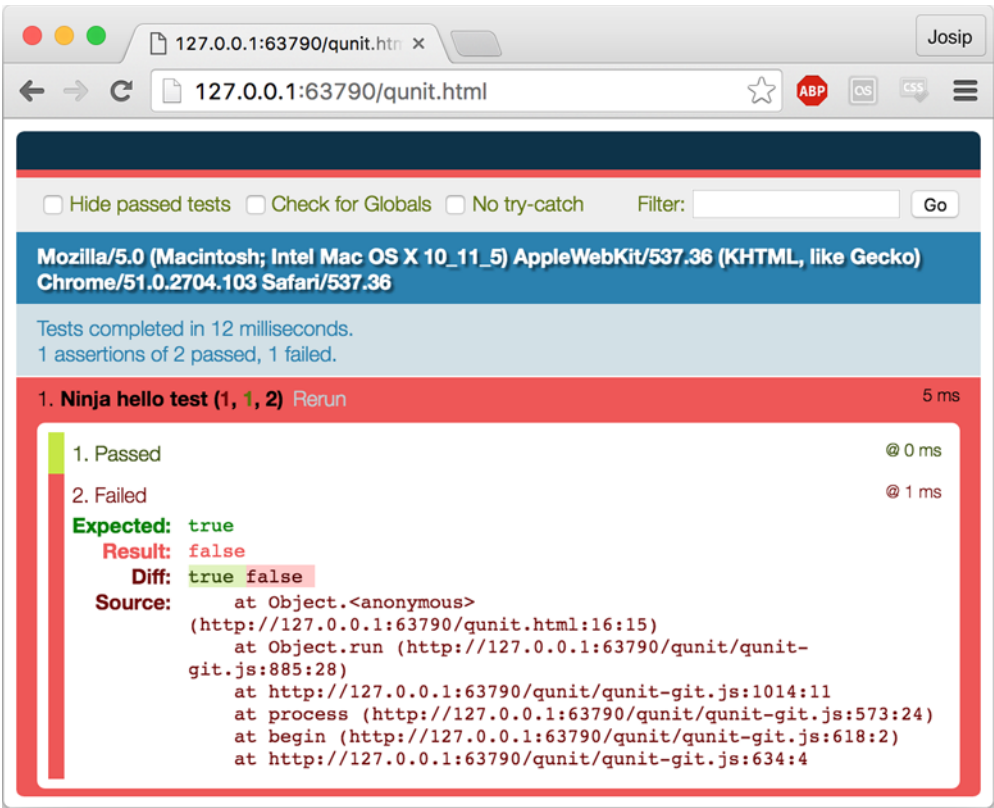


Figura B.13 Um exemplo de execução de teste QUnit. Como parte do nosso teste, temos uma afirmação que passa e uma que falha (uma das duas afirmações passou, uma falhou). Os resultados exibidos colocam uma ênfase muito maior no teste com falha, para garantir que o consertemos o mais rápido possível.

A combinação e a nomenclatura dessas funções são voltadas para tornar o conjunto de testes de natureza quase coloquial. Por exemplo, a lista a seguir mostra como testar o diga HiToNinja função usando Jasmine.

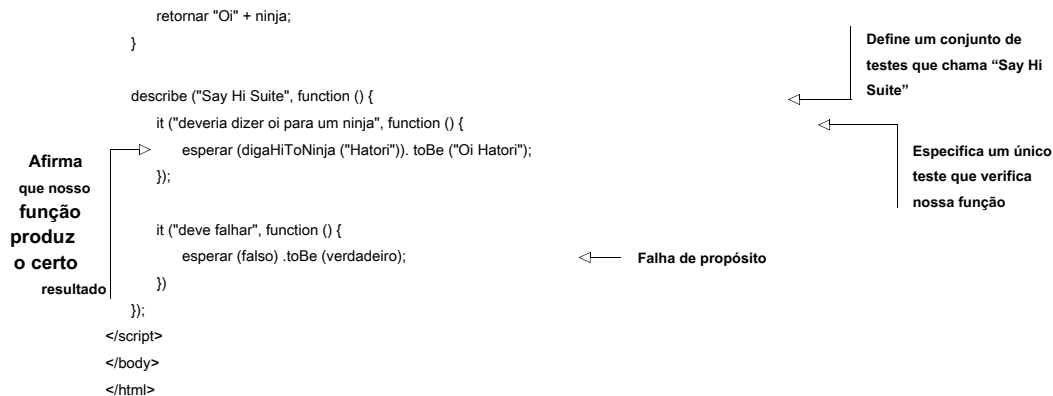
Listagem B.7 Exemplo de teste Jasmine

```
<!DOCTYPE html>
<html>
<head>
 <link rel = "stylesheet" href = "lib / jasmine-2.2.0 / jasmine.css">

 <script src = "lib / jasmine-2.2.0 / jasmine.js"> </script>
 <script src = "lib / jasmine-2.2.0 / jasmine-html.js"> </script> <script src = "lib / jasmine-2.2.0 / boot.js">
 </script>
</head>
<body>
<script>
 function sayHiToNinja (ninja) {
```

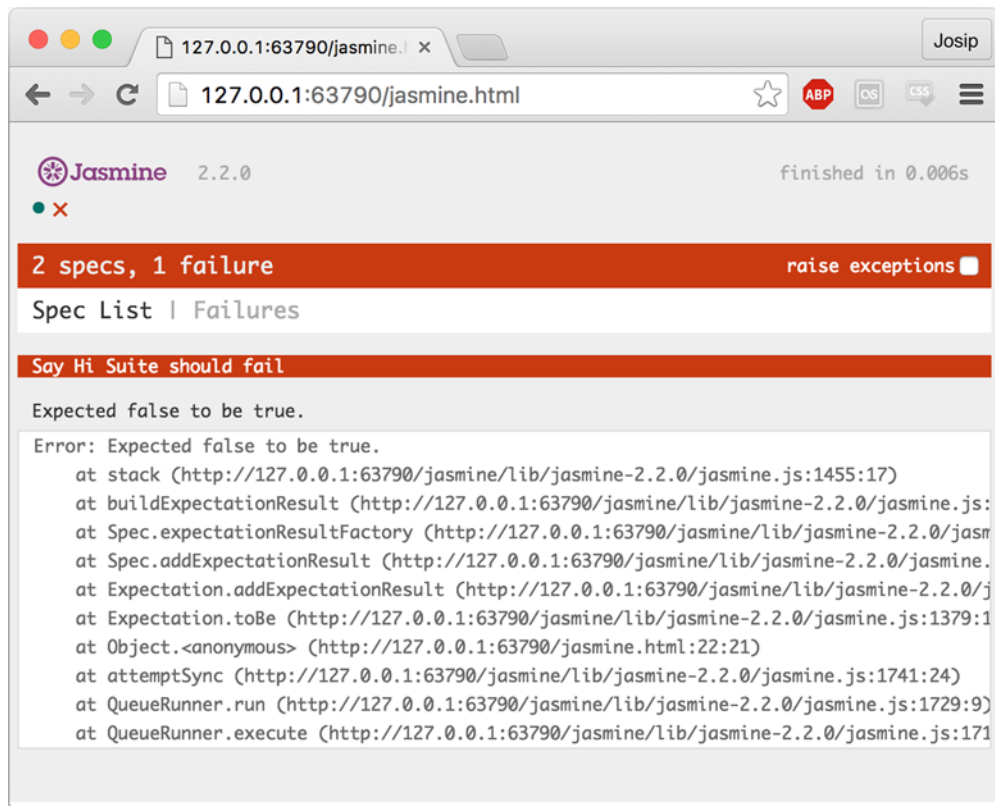
Inclui  
Arquivos Jasmine

Declara a função  
que queremos testar



O resultado da execução deste conjunto de testes Jasmine no navegador é mostrado na figura B.14.

Mais informações sobre Jasmine podem ser encontradas em <http://jasmine.github.io/>.



**Figura B.14** O resultado da execução de um conjunto de testes Jasmine no navegador. Temos dois testes: um aprovado e outro reprovado (duas especificações, uma falha).

## M COBERTURA DE CÓDIGO DE FACILIDADE

É difícil dizer o que torna um determinado conjunto de testes *Boa*. Idealmente, devemos testar todos os caminhos de execução possíveis de nossos programas. Infelizmente, exceto nos casos mais triviais, isso não é possível. Um passo na direção certa é tentar testar o máximo de código possível, e uma métrica que nos diz o grau em que um conjunto de testes cobre nosso código é chamada *Cobertura de código*.

Por exemplo, dizer que uma suíte de teste tem 80% de cobertura de código significa que 80% do código do nosso programa é executado pela suíte de teste, enquanto 20% do nosso código não é. Embora não possamos ter certeza absoluta de que 80% do código não contém bugs (podemos ter perdido um caminho de execução que leva a um), estamos completamente no escuro sobre os 20% que nem foram executados. É por isso que devemos medir a cobertura de código de nossos conjuntos de teste.

No desenvolvimento de JavaScript, podemos usar várias bibliotecas para medir a cobertura de nossos conjuntos de teste, mais notavelmente Blanket.js ( <https://github.com/alex-seville/blanket> ) e Istanbul ( <https://github.com/gotwarlost/istanbul> ) A configuração dessas bibliotecas vai além do escopo deste livro, mas suas respectivas páginas da web oferecem todas as informações de que podemos precisar para configurá-las adequadamente.

# apêndice C

## Respostas de exercícios

---

### Capítulo 2. Construindo a página no tempo de execução

- 1 Quais são as duas fases do ciclo de vida de um aplicativo da web do lado do cliente?

R: As duas fases do ciclo de vida de um aplicativo da Web do lado do cliente são a construção de páginas e o tratamento de eventos. Na fase de construção da página, a interface do usuário de nossa página é construída processando o código HTML e executando o código JavaScript da linha principal. Depois que o último nó HTML é processado, a página entra na fase de tratamento de eventos, na qual vários eventos são processados.

- 2 Qual é a principal vantagem de usar o `addEventListener` método para registrar um manipulador de eventos versus atribuir um manipulador a uma propriedade de elemento específico?

R: Ao atribuir manipuladores de eventos a propriedades de elementos específicos, podemos registrar apenas um manipulador de eventos; `addEventListener`, por outro lado, nos permite registrar quantos manipuladores de eventos forem necessários.

- 3 Quantos eventos podem ser processados de uma vez?

R: JavaScript é baseado em um modelo de execução de thread único, no qual os eventos são processados um de cada vez.

- 4 Em que ordem os eventos da fila de eventos são processados?

R: Os eventos são processados na ordem em que foram gerados: primeiro a entrar, primeiro a sair.

### Capítulo 3. Funções de primeira classe para o novato: definições e argumentos

- 1 No trecho de código a seguir, quais funções são funções de retorno de chamada?

```
// sortAsc é um retorno de chamada porque o mecanismo JavaScript // o chama para comparar itens da matriz
numbers.sort (função sortAsc (a, b) {
```



```

 retornar a - b;
 });

// Não é um retorno de chamada; ninja é chamado como uma função de função padrão ninja () {}

ninja ();

var myButton = document.getElementById ("myButton"); // handleClick é um callback, a função é
chamada // sempre que myButton é clicado

myButton.addEventListener ("click", função handleClick () {
 alerta ("clicado");
});

```

- 2 No snippet a seguir, categorize as funções de acordo com seu tipo (função de declaração, expressão de função ou função de seta).

```

// expressão da função como argumento para outra função numbers.sort (função sortAsc (a, b) {

 retornar a - b;
 });

// função de seta como argumento para outra função numbers.sort ((a, b) => b - a);

// expressão de função como o receptor em uma expressão de chamada (função(){}());

// declaração de função
function outer () {
 // declaração de função
 função interna () {}
 retorno interno;
}

// chamada de expressão de função envolvida em uma expressão (função(){}());

// função de seta como callee (() => " Yoshi ") ();

```

- 3 Depois de executar o seguinte trecho de código, quais são os valores das variáveis **samurai** e **ninja**?

```

// "Tomoe", o valor do corpo da expressão da função seta var samurai = (() => "Tomoe") ();

// indefinido, no caso do corpo de uma função de seta ser uma instrução de bloco // o valor é o valor da instrução de retorno.

// Como não há instrução de retorno, o valor é indefinido. var ninja = (() => {"Yoshi"}) ();

```

- 4 Dentro do corpo do teste função, quais são os valores dos parâmetros **a**, **b**, e **c** para as duas chamadas de função?

```
teste de função (a, b, ... c) {/ * a, b, c * /}

// a = 1; b = 2; c = [3, 4, 5] teste (1, 2, 3, 4, 5);

// a = undefined; b = indefinido; c = [] teste ();
```

- 5 Depois de executar o seguinte snippet de código, quais são os valores do mensagem1 e mensagem2 variáveis?

```
function getNinjaWieldingWeapon (ninja, weapon = "katana") {
 retornar ninja + "" + katana;
}

// "Yoshi katana" - há apenas um argumento na chamada // então o padrão da arma é "katana"

var message1 = getNinjaWieldingWeapon ("Yoshi");

// "Yoshi wakizashi" - enviamos dois argumentos, o valor // padrão não é levado em consideração

var message2 = getNinjaWieldingWeapon ("Yoshi", "wakizashi");
```

## Capítulo 4. Funções para o jornalista: entendendo a invocação de função

- 1 A função a seguir calcula a soma dos argumentos passados usando o argumentos objeto. Usando os parâmetros restantes introduzidos no capítulo anterior, reescreva o soma função para que não use o argumentos objeto.

```
function sum () {
 var sum = 0;
 para (var i = 0; i < argument.length; i++) {
 soma + = argumentos [i];
 }
 soma de retorno;
}

assert (sum (1, 2, 3) === 6, 'A soma dos três primeiros números é 6'); assert (sum (1, 2, 3, 4) === 10, 'A soma dos quatro primeiros números é 10');
```

R: Adicione um parâmetro de descanso à definição da função e ajuste ligeiramente o corpo da função:

```
soma de funções (... números){
 var sum = 0;
 para (var i = 0; i < números. comprimento; i++) {
 soma + = números[eu];
 }
 soma de retorno;
}

assert (sum (1, 2, 3) === 6, 'A soma dos três primeiros números é 6'); assert (sum (1, 2, 3, 4) === 10, 'A soma dos quatro primeiros números é 10');
```

- 2 Depois de executar o seguinte código no navegador, quais são os valores das variáveis `ninja` e `samurai`?

```
function getSamurai (samurai) {
 "use estrito"

 argumentos [0] = "Ishida";

 retorno samurai;
}

function getNinja (ninja) {
 argumentos [0] = "Fuma";
 retornar ninja;
}

var samurai = getSamurai ("Toyotomi"); var ninja = getNinja ("Yoshi");
```

UMA: `samurai` terá o valor `Toyotomi`, e `ninja` terá o valor `Fuma`.

Porque o `getSamurai` função está em modo estrito, o `argumentos` parâmetro não é um alias de parâmetros de função, portanto, alterar o valor do primeiro argumento não mudará o valor do `samurai` parâmetro.

Porque o `getNinja` função está em modo não estrito, quaisquer alterações feitas no `argumentos` parâmetro será refletido nos parâmetros da função.

- 3 Ao executar o código a seguir, qual das afirmações será aprovada?

```
function whoAml1 () {
 "use estrito";
 devolva isso;
}

function whoAml2 () {
 devolva isso;
}

assert (whoAml1 () === window, "Window?"); // falhou
assert (whoAml2 () === window, "Window?"); // passar
```

R: O `whoAml1` a função está no modo estrito; quando é chamado como uma função, o valor do esta parâmetro será `Indefinido` ( e não janela). A segunda afirmação será aprovada: Se uma função no modo não estrito for chamada como uma função, esta refere-se ao objeto global (o janela objeto, ao executar o código no navegador).

- 4 Ao executar o código a seguir, qual das afirmações será aprovada?

```
var ninja1 = {
 whoAml: function () {
 devolva isso;
 }
}
```

```

};

var ninja2 = {
 whoAml: ninja1.whoAml
};

var identificar = ninja2.whoAml;

// passar: whoAml chamado como um método de ninja1 assert (ninja1.whoAml () ===
ninja1, "ninja1?");

// falha: whoAml chamado como um método de ninja2 assert (ninja2.whoAml () === ninja1, "ninja1
novamente?");

// falha: identificar chamadas a função como uma função
// porque estamos no modo não estrito, isso se refere à declaração da janela (identifique () === ninja1, "ninja1
novamente?");

// passar: usando chamada para fornecer o contexto da função // isso se refere a ninja2

assert (ninja1.whoAml.call (ninja2) === ninja2, "ninja2 aqui?");

```

5 Ao executar o código a seguir, qual das afirmações será aprovada?

```

function Ninja () {
 this.whoAml = () => isso;
}

var ninja1 = novo Ninja (); var ninja2 = {

 whoAml: ninja1.whoAml
};

// pass: whoAml é uma função de seta herda o contexto da função // do contexto em que foi criado.

// Porque foi criado durante a construção do ninja1 // isso sempre apontará para ninja1

assert (ninja1.whoAml () === ninja1, "ninja1 aqui?");

// falso: sempre se refere a ninja1
assert (ninja2.whoAml () === ninja2, "ninja2 aqui?");

```

6 Qual das afirmações a seguir será aprovada?

```

function Ninja () {
 this.whoAml = function () {
 devolva isso;
 }.bind (este);
}

var ninja1 = novo Ninja (); var ninja2 = {

 whoAml: ninja1.whoAml
};

```

```
// passar: a função atribuída a whoAml é um limite de função
// para ninja1 (o valor deste quando o construtor foi chamado) // isso sempre se referirá a ninja1

assert (ninja1.whoAml () === ninja1, "ninja1 aqui?"); // falha: em whoAml sempre se refere a ninja1
// porque whoAml é uma função limitada.

assert (ninja2.whoAml () === ninja2, "ninja2 aqui?");
```

## Capítulo 5. Funções para o mestre: fechamentos e escopos

- 1 Fechamentos permitem funções para

R: Acesse variáveis externas que estão no escopo quando a função é definida (opção a)

- 2 Fechamentos vêm com

R: Custos de memória (os fechamentos mantêm vivas as variáveis que estão no escopo quando a função é definida) (opção b)

- 3 No exemplo de código a seguir, marque os identificadores acessados por meio de fechamentos:

```
function Samurai (nome) {
 var arma = "katana";

 this.getWeapon = function () {
 // acessa a variável local: weapon return weapon;

 };

 this.getName = function () {
 // acessa o parâmetro da função: nome nome do retorno;

 }

 esta.mensagem = nome + "empunhando uma" + arma;

 this.getMessage = function () {
 // esta. mensagem não é acessada por meio de um fechamento // é uma propriedade do
 objeto (e não uma variável) return this.message;

 }
}

var samurai = novo Samurai ("Hattori");

samurai.getWeapon ();
samurai.getName ();
samurai.getMessage ();
```

- 4 No código a seguir, quantos contextos de execução são criados e qual é o maior tamanho da pilha de contexto de execução?

```
function perfom (ninja) {
 sneak (ninja);
 infiltrar (ninja);
```

```

 }

 função sneak (ninja) {
 retornar ninja + "esquiva";
 }

 função infiltrar (ninja) {
 retornar ninja + "infiltrando";
 }

 perform ("Kuma");

```

R: O maior tamanho de pilha é 3, nas seguintes situações:

- código global -> executar -> esgueirar
- código global -> executar -> infiltrar

- 5 Qual palavra-chave em JavaScript nos permite definir variáveis que não podem ser reatribuídas a um valor completamente novo?

UMA: const variáveis não podem ser reatribuídas a novos valores.

- 6 Qual é a diferença entre var e deixei?

R: A palavra-chave var é usado para definir apenas variáveis de escopo de função ou global, enquanto deixei nos permite definir variáveis de escopo de bloco, escopo de função e escopo global.

- 7 Onde e por que o código a seguir lançará uma exceção?

```

getNinja ();
getSamurai (); // lança uma exceção

function getNinja () {
 retornar "Yoshi";
}

var getSamurai = () => "Hattori";

```

R: Uma exceção será lançada ao tentar invocar o getSamurai função. O getNinja function é definida com uma declaração de função e será criada antes que qualquer código seja executado; podemos chamá-lo de “antes” de sua declaração ser alcançada em código. O getSamurai função, por outro lado, é uma função de seta criada quando a execução a alcança, portanto, será indefinida quando tentarmos invocá-la.

## Capítulo 6. Funções para o futuro: geradores e promessas

- 1 Depois de executar o código a seguir, quais são os valores das variáveis a1 para a4?

```

function * EvenGenerator () {
 deixe num = 2;
 enquanto (verdadeiro) {

```

```

 rendimento num;
 num = num + 2;
 }
}

let generator = EvenGenerator ();

// 2 o primeiro valor gerado
deixe a1 = generator.next (). value;

// 4 o segundo valor gerado
deixe a2 = generator.next (). value;

// 2, porque iniciamos um novo gerador let a3 = EvenGenerator (). Next (). Value; // 6,
voltamos para o primeiro gerador let a4 = generator.next (). Value;
```

- 2 Qual é o conteúdo do ninjas array depois de executar o seguinte código? (Dica: Pense em como o para de loop pode ser implementado com um enquanto ciclo.)

```

function * NinjaGenerator () {
 produzir "Yoshi";
 retornar "Hattori";
 produzir "Hanzo";
}

var ninjas = [];
para (deixe ninja de NinjaGenerator ()) {
 ninjas.push (ninja);
}

ninjas;
```

R: O ninjas array irá conter apenas Yoshi. Isso acontece porque o para de loop itera sobre um gerador até que o gerador diga que está pronto (sem incluir o valor passado junto com done). Isso acontece quando não há mais código no gerador para executar ou quando um Retorna declaração é encontrada.

- 3 Qual é o valor das variáveis a1 e a2, depois de executar o seguinte código?

```

function * Gen (val) {
 val = rendimento val * 2; rendimento
 val;
}

deixe gerador = Gen (2);

// 4. O valor do primeiro valor passado por next: 3 é ignorado // porque o gerador ainda não iniciou sua execução e // não há nenhuma
expressão de rendimento em espera.

// Como o gerador é criado com val sendo 2 // o primeiro rendimento ocorre para val * 2, ou seja, 2
* 2 == 4 let a1 = generator.next (3) .value;
```

```
// 5: passando 5 como um argumento para o próximo
// significa que a expressão produzida em espera obterá o valor 5 // (yield val * 2) == 5

// porque esse valor é então atribuído a val, a próxima expressão de rendimento // rendimento val;

// retornará 5
deixe a2 = generator.next (5) .value;
```

#### 4 Qual é a saída do código a seguir?

```
promessa const = nova promessa ((resolver, rejeitar) => {
 rejeitar ("Hattori"); // a promessa foi rejeitada explicitamente});

// o manipulador de erros será invocado
promessa.então (val => alert ("Sucesso:" + val))
 . catch (e => alert ("Erro:" + e));
```

#### 5 Qual é a saída do código a seguir?

```
promessa const = nova promessa ((resolver, rejeitar) => {
 // a promessa foi explicitamente resolvida resolve ("Hattori");

 // uma vez que uma promessa foi estabelecida, ela não pode ser alterada // rejeitá-la após 500
 ms não terá efeito setTimeout (() => rejeitar ("Yoshi", 500);

});

// o manipulador de sucesso será invocado promessa.then (val => alert ("Sucesso:" +
val))
 . catch (e => alert ("Erro:" + e));
```

## Capítulo 7. Orientação a objetos com protótipos

#### 1 Qual das seguintes propriedades aponta para um objeto que será pesquisado se o objeto de destino não tem a propriedade procurada? UMA: protótipo ( opção c)

#### 2 Qual é o valor da variável a1 depois que o código a seguir é executado?

```
função Ninja () {}
Ninja.prototype.talk = function () {
 retornar "Olá";
};

const ninja = novo Ninja ();
const a1 = ninja.talk (); // "Olá"
```

R: O valor da variável a1 será Olá. Mesmo que o objeto ninja não possui o conversar método, seu protótipo faz.

#### 3 Qual é o valor de a1 depois de executar o seguinte código?



```

função Ninja () {}
Ninja.message = "Olá";

const ninja = novo Ninja ();

const a1 = ninja.mensagem;

```

R: O valor da variável a1 será indefinido. O mensagem propriedade é definida na função do construtor Ninja, e não é acessível através do ninja objeto.

- 4 Explique a diferença entre o getFullName método nestes dois códigos fragmentos:

```

// Primeiro fragmento
função Pessoa (primeiroNome, últimoNome) {
 this.firstName = primeiroNome;
 this.lastName = últimoNome;

 this.getFullName = function () {
 retorna this.firstName + " " + this.lastName;
 }
}

// segundo fragmento
função Pessoa (primeiroNome, últimoNome) {
 this.firstName = primeiroNome;
 this.lastName = últimoNome;
}

Person.prototype.getFullName = function () {
 retorna this.firstName + " " + this.lastName;
}

```

R: No primeiro fragmento, o getFullName método é definido diretamente na instância criada com o Pessoa construtor.

Cada objeto criado com o

Pessoa construtor obtém seu próprio getFullName método. No segundo fragmento, o getFullName método é definido no protótipo do Pessoa

função. Todas as instâncias criadas com o Pessoa função terá acesso a este único método.

- 5 Depois de executar o código a seguir, o que ninja.constructor aponta para?

```

function Person () {} function Ninja () {}

const ninja = novo Ninja ();

```

R: Ao acessar ninja.constructor, a propriedade constructor é encontrada em ninja protótipo de. Porque ninja foi criado com o Ninja função do construtor, a propriedade do construtor aponta para o Ninja função.

- 6 Depois de executar o código a seguir, o que `ninja.constructor` aponta para?

```
function Person () {} function Ninja () {}

Ninja.prototype = nova pessoa (); const ninja = novo Ninja ();
```

R: O construtor `property` é a propriedade do objeto de protótipo que foi criado com a função de construtor. Neste exemplo, substituímos o protótipo integrado do `Ninja` funcionar com um novo `Pessoa` objeto. Portanto, quando um `ninja` objeto é criado com o `Ninja` construtor, seu protótipo é definido para o novo `pessoa` objeto. Finalmente, quando acessamos o construtor `property` no `ninja` objeto, porque o `ninja` objeto não tem seu próprio construtor `property`, seu protótipo, o novo `pessoa` objeto, é consultado. O `pessoa` objeto também não tem um construtor `property`, então seu protótipo, o `Person.prototype` objeto, é consultado. Esse objeto tem um construtor `property`, referenciando o `Pessoa`

função. Este exemplo ilustra perfeitamente por que devemos ter cuidado ao usar o construtor `property`: Mesmo que nosso `ninja` objeto foi criado com o `Ninja` função, o construtor `property`, devido ao soluço de substituir o padrão `Ninja.prototype`, aponta para o `Pessoa` função.

- 7 Explique como o instancia de operador funciona no exemplo a seguir.

```
função Guerreiro () {}

função Samurai () {}
Samurai.prototype = novo Guerreiro ();

var samurai = novo Samurai ();

exemplo de samurai do guerreiro; //Explicar
```

R: O instancia de O operador verifica se o protótipo da função do lado direito está na cadeia de protótipos do objeto do lado esquerdo. O objeto à esquerda é criado com o `Samurai` função, e seu protótipo tem um novo `Guerreiro` objeto, cujo protótipo é o protótipo do `Guerreiro`

função ( `Warrior.prototype`). À direita temos o `Guerreiro` função. Então, neste exemplo, o instancia de operador irá retornar verdade, porque o protótipo da função à direita, `Warrior.prototype`, pode ser encontrado na cadeia de protótipos do objeto à esquerda.

- 8 Traduza o seguinte código ES6 para o código ES5.

```
class Guerreiro {
 constructor (arma) {
 this.weapon = arma;
 }

 manejar() {
```

```

 return "Wielding" + this.weapon;
 }

 duelo estático (guerreiro1, guerreiro2) {
 return warrior1.wield () + "" + warrior2.wield ();
 }
}

```

R: Podemos traduzir o código da seguinte maneira:

```

função Guerreiro (arma) {
 this.weapon = arma;
}

Warrior.prototype.wield = function () {
 return "Wielding" + this.weapon;
};

Warrior.duel = function (warrior1, warrior2) {
 return warrior1.wield () + "" + warrior2.wield ();
};

```

## Capítulo 8. Controlando o acesso aos objetos

- 1 Depois de executar o código a seguir, qual das seguintes expressões lançará uma exceção e por quê?

```

const ninja = {
 obter nome () {
 retornar "Akiyama";
 }
}

```

R: Ligando `ninja.name ()` lança uma exceção porque `ninja` não tem um método de nome (opção a).

Acessando `ninja.name` no `const name = ninja.name`

Funciona como um encanto; o getter é ativado e o nome da variável obtém o valor `Akiyama`.

- 2 No código a seguir, qual mecanismo permite que os getters acessem um objeto privado variável?

```

function Samurai () {
 const _weapon = "katana";
 Object.defineProperty (this, "weapon", {
 obter: () => _weapon});
}

Const samurai = novo Samurai ();
assert (samurai.weapon === "katana", "Um samurai empunhando uma katana");

```

R: Os fechamentos permitem que os getters acessem as variáveis do objeto privado. Neste caso, o obter método cria um fechamento em torno do `_ arma` variável privada definida na função construtora, que mantém o `_ arma` variável viva.

### 3 Qual das afirmações a seguir será aprovada?

```
const daimyo = {nome: "Matsu", clã: "Takasu"}; const proxy = new Proxy (daimyo, {

 obter: (alvo, chave) => {
 if (chave === "clã") {
 retornar "Tokugawa";
 }
 }
});

assert (daimyo.clan === "Takasu", "Matsu do clã Takasu"); assert (proxy.clan === "Tokugawa", "Matsu do clã Tokugawa"); // passar

proxy.clan = "Tokugawa";

assert (daimyo.clan === "Takasu", "Matsu do clã Takasu"); // falhou
assert (proxy.clan === "Tokugawa", "Matsu do clã Tokugawa?"); // passar
```

R: A primeira afirmação é aprovada porque daimyo tem um clã propriedade com valor Takasu. A segunda afirmação passa, porque acessamos a propriedade clã por meio de um proxy com uma armadilha get que sempre retorna Tokugawa como o valor do clã propriedade.

Quando a expressão `proxy.clan = "Tokugawa"` é avaliado, o valor Tokugawa é armazenado no daimyo de clã propriedade porque o proxy não tem uma armadilha definida, então a ação padrão de definir a propriedade é realizada no destino, daimyo objeto.

A terceira afirmação falha, porque o daimyo a propriedade do clã de tem o valor Tokugawa e não Takasu.

A quarta asserção é aprovada, porque o proxy sempre retorna Tokugawa, independentemente do valor armazenado no objeto de destino clã propriedade.

### 4 Qual das afirmações a seguir será aprovada?

```
const daimyo = {nome: "Matsu", clã: "Takasu", armySize: 10000}; const proxy = new Proxy (daimyo, {

 definir: (alvo, chave, valor) => {
 if (key === "armySize") {
 número const = Number.parseInt (valor); if (! Number.isNaN (number)) {

 alvo [chave] = número;
 }
 }
 } outro {
 alvo [chave] = valor;
 }
 },
});
```

```
//passar
assert (daimyo.armySize === 10000, "Matsu tem 10.000 homens armados"); //passar

assert (proxy.armySize === 10000, "Matsu tem 10.000 homens armados");

proxy.armySize = "grande";
assert (daimyo.armySize === "grande", "Matsu tem um grande exército"); // falhou

daimyo.armySize = "grande";
assert (daimyo.armySize === "grande", "Matsu tem um grande exército"); // passar
```

R: A primeira afirmação é aprovada; o valor de daimyo de armySize propriedade é 10.000.

A segunda afirmação também é aprovada; o proxy não tem um get trap definido, então o valor do alvo, daimyo de armySize propriedade, é retornado.

Quando a expressão proxy.armySize = "grande"; é avaliada, a armadilha definida do proxy é ativada. O configurador verifica se o valor passado é um número e, somente se for, o valor é atribuído à propriedade do destino. Nesse caso, o valor passado não é um número, portanto, nenhuma alteração é feita no armySize propriedade. Por esse motivo, a terceira asserção, que pressupõe a mudança, falha.

A expressão daimyo.armySize = "grande"; escreve diretamente para o armySize propriedade, ignorando o proxy. Portanto, a afirmação final é aprovada.

## Capítulo 9. Lidando com cobranças

- 1 Depois de executar o código a seguir, qual é o conteúdo do samurai variedade?

```
const samurai = ["Oda", "Tomoe"]; samurai[3] = "Hattori";
```

R: O valor do samurai é [" Oda ", " Tomoe ", indefinido, " Hattori "]. O

array começa com Oda e Tomoe em índices 0 e 1 Em seguida, adicionamos um novo samurai, Hattori, no índice 3, que "expande" a matriz e deixa o índice 2 com unde multado.

- 2 Depois de executar o código a seguir, qual é o conteúdo do ninjas variedade?

```
const ninjas = [];

ninjas.push ("Yoshi");
ninjas.unshift ("Hattori");

ninjas.length = 3;

ninjas.pop ();
```

R: O valor de ninjas é [" Hattori ", " Yoshi "]. Começamos com uma matriz vazia, Empurre adiciona Yoshi até o fim, e não mudar adiciona Hattori para o início.

Definindo explicitamente comprimento para 3 expande a matriz com Indefinido no índice 2 Chamando pop remove isso Indefinido da matriz, deixando apenas [" Hattori "," Yoshi "].

- 3 Depois de executar o código a seguir, qual é o conteúdo do samurai variedade?

```
const samurai = [];

samurai.push ("Oda");
samurai.unshift ("Tomoe");
samurai.splice (1, 0, "Hattori", "Takeda"); samurai.pop ();
```

R: O valor de samurai é [" Tomoe "," Hattori "," Takeda "]. A matriz começa vazio; Empurre adiciona Oda até o fim, e não mudar adiciona Tomoe para o início; emenda remove o item no índice 1 (Oda) e adiciona Hattori e Takeda em vez de.

- 4 Depois de executar o código a seguir, o que está armazenado nas variáveis primeiro segundo, e terceiro?

```
const ninjas = [{nome: "Yoshi", idade: 18},
 {nome: "Hattori", idade: 19},
 {nome: "Yagyu", idade: 20}];

const primeiro = pessoas.map (ninja => ninja.age); const segundo = primeiro.filtro (idade =>
idade% 2 == 0);
const third = first.reduce ((agregado, item) => agregado + item, 0);
```

A: primeiro: [18, 19, 20]; segundo: [18, 20]; terceiro: 57

- 5 Depois de executar o código a seguir, o que está armazenado nas variáveis primeiro e segundo?

```
const ninjas = [{nome: "Yoshi", idade: 18},
 {nome: "Hattori", idade: 19}, {nome: "Yagyu", idade:
20}];

const primeiro = ninjas.some (ninja => ninja.age% 2 == 0); segundo const = ninjas.every (ninja => ninja.age%
2 == 0);
```

A: primeiro: verdadeiro; segundo: falso

- 6 Qual das afirmações a seguir será aprovada?

```
const samuraiClanMap = new Map ();

const samurai1 = {nome: "Toyotomi"}; const samurai2 = {nome:
"Takeda"}; const samurai3 = {nome: "Akiyama"};

const oda = {clã: "Oda"};
const tokugawa = {clã: "Tokugawa"}; const takeda = {clã: "Takeda"};
```

```

samuraiClanMap.set (samurai1, oda);
samuraiClanMap.set (samurai2, tokugawa);
samuraiClanMap.set (samurai2, takeda);

assert (samuraiClanMap.size === 3, "Existem três mapeamentos"); assert (samuraiClanMap.has (samurai1), "O
primeiro samurai tem um mapeamento");

assert (samuraiClanMap.has (samurai3), "O terceiro samurai tem um mapeamento");

```

R: A primeira afirmação falha, porque um mapeamento para samurai2 foi criado duas vezes. A segunda afirmação é aprovada, porque um mapeamento para samurai1 foi adicionado. E a terceira afirmação falha, porque um mapeamento para samurai3 nunca foi criado.

7 Qual das afirmações a seguir será aprovada?

```

const samurai = novo Conjunto ("Toyotomi", "Takeda", "Akiyama", "Akiyama"); assert (samurai.size === 4, "Existem quatro
samurais no conjunto");

samurai.add ("Akiyama");
assert (samurai.size === 5, "Existem cinco samurais no conjunto");

assert (samurai.has ("Toyotomi", "Toyotomi está dentro!"); assert (samurai.has ("Hattori", "Hattori
está dentro!");

```

R: A primeira afirmação falha porque Akiyama é adicionado apenas uma vez ao conjunto. A segunda afirmação também falha, porque tentar adicionar Akiyama oncemore não mudará o conjunto (nem seu comprimento). As duas últimas afirmações serão aprovadas.

## Capítulo 10. *Wrangling expressões regulares*

1 Em JavaScript, as expressões regulares podem ser criadas com qual das seguintes opções? R: Literais de expressão regular (opção a) e usando o integrado RegExp construtor (opção B). A resposta c está incorreta; não embutido Expressão regular construtor existe.

2 Qual das opções a seguir é um literal de expressão regular?

R: Em JavaScript, um literal de expressão regular é colocado entre duas barras: /teste/ ( opção a).

3 Escolha os sinalizadores de expressão regular corretos.

R: Com literais de expressão regular, os sinalizadores de expressão são colocados após a barra de fechamento: / teste / g ( opção a). Com RegExp construtores, eles são passados como um segundo argumento: novo RegExp ("teste", "gi"); ( opção c). A expressão regular / def / corresponde a qual das

4 seguintes strings? R: A expressão regular / def / jogos apenas def: *d* seguido pela *e*, seguido pela *f*

(opção b).

5 A expressão regular / [^ abc]/ corresponde a qual das seguintes?

R: A expressão regular / [^ abc]/ corresponde a uma das strings d, e, f - um personagem que não é a, b, ou c (opção b).

- 6 Qual das seguintes expressões regulares corresponde à string Olá?

R: As opções a, b e c correspondem. / Olá/ corresponde apenas à string exata Olá. /Olá/ corresponde também Olá ou Olá ( o segundo eu é opcional). / Olá/, depois do primeiro eu, corresponde a qualquer número da letra eu.

- 7 A expressão regular / ( cd) + (de) \* / corresponde a qual das seguintes strings? R: As opções a, c, d e f estão corretas. / ( cd) + (de) \* / corresponde a uma ou mais ocorrências de CD seguido por qualquer número de ocorrências de de.

- 8 Em expressões regulares, podemos expressar alternativas com qual das seguintes? R: Usamos a barra vertical, |, para expressar alternativas em expressões regulares (opção c).

- 9 Na expressão regular / ([ 0-9]) 2 /, podemos fazer referência ao primeiro dígito correspondente com qual dos seguintes?

UMA: \ 1 ( opção d)

- 10 A expressão regular / ([ 0-5]) 6 \ 1 / corresponderá a qual dos seguintes? R: Na expressão regular / ([ 0-5]) 6 \ 1 /, o primeiro caractere é um dígito de 0 a 5, o segundo caractere é o dígito 6 e o terceiro caractere é o primeiro dígito correspondente, então ambos 060 e 565 corresponder (opções a e d). A expressão regular / (? : ninja) - (truque)? - \ 1 / corresponderá a qual dos seguintes?

11

R: Na expressão regular / (? : ninja) - (truque)? - \ 1 /, o primeiro grupo

(? : ninja) não é de captura, enquanto o segundo grupo é de captura, o que é opcional ( truque)? . Mas se este segundo grupo for encontrado, no final temos uma referência anterior a ele. Portanto, ninja- e truque-truque-ninja corresponder (opções a e c).

- 12 Qual é o resultado de executar " 012675 ".substituir (/ 0-5 / g, " a ")?

R: O código substitui todas as ocorrências de dígitos de 0 a 5 pela letra a, então aaa67a resultados (opção a).

## Capítulo 11. Técnicas de modularização de código

- 1 Qual mecanismo habilita variáveis de módulo privadas no padrão de módulo?

R: No padrão do módulo, os fechamentos nos permitem ocultar os internos do módulo: Os métodos da API pública do módulo mantêm os internos do módulo vivos (opção b).

- 2 No código a seguir que usa módulos ES6, quais identificadores podem ser acessados se o módulo for importado?

```
const spy = "Yagyu"; comando de
função () {
 retorno geral + "ordena que você faça guerra!";
}
export const general = "Minamoto";
```



R: De fora do módulo, podemos acessar apenas o em geral identificador, porque é o único identificador que foi explicitamente exportado (opção c).

- 3 No código a seguir que usa módulos ES6, quais identificadores podem ser acessados quando o módulo é importado?

```
const ninja = "Yagyu"; comando de função
() {
 retorno geral + "ordena que você faça guerra!";
}
const general = "Minamoto";

exportar {ninja como espião};
```

R: De fora do módulo, podemos acessar apenas o espião identificador: este é o único identificador que foi exportado como um alias do ninja variável (opção a).

- 4 Quais das seguintes importações são permitidas?

```
// Arquivo: staff.js
const ninja = "Yagyu"; comando de função
() {
 retorno geral + "ordena que você faça guerra!";
}
const general = "Minamoto";

exportar {ninja como espião};
```

R: A primeira importação não é permitida porque o pessoal módulo não exporta ninja e em geral identificadores (opção a). A segunda importação é permitida porque importamos todo o módulo, que é acessível através do objeto Pessoal ( opção b). A terceira importação também é permitida, pois importamos o exportado espião identificador (opção c).

- 5 Se tivermos o seguinte código de módulo, qual instrução importará o Ninja aula?

```
//Ninja.js
exportar classe padrão Ninja {
 skulk () {return "skulking"; }
}
```

R: A primeira importação é permitida: Importamos a exportação padrão (opção a). A segunda importação é permitida: Importamos todo o módulo (opção b). A terceira importação não é permitida porque não está sintaticamente correta (após o \* deve vir a parte "as Name") (opção c).

## Capítulo 12. Trabalhando o DOM

- 1 No código a seguir, qual das afirmações a seguir será aprovada?

```
<div id = "samurai"> </div>
<script>
 elemento const = document.querySelector ("# samurai");
```

```

assert (element.id === "samurai", "id da propriedade é samurai"); assert (element.getAttribute ("id") === "samurai",
 "id do atributo é samurai");

element.id = "novoSamurai";

assert (element.id === "newSamurai", "id da propriedade é newSamurai"); assert (element.getAttribute ("id") ===
 "newSamurai",
 "id do atributo é newSamurai");
</script>

```

R: Neste código, todas as afirmações são aprovadas. O eu ia atributo e o eu ia propriedade estão vinculadas; uma mudança em um deles se reflete no outro.

2 Dado o código a seguir, como podemos acessar o elemento largura da borda estilo

propriedade?

```

<div id = "element" style = "border-width: 1px;
 estilo de borda: sólido; border-color: red ">

</div>
<script>
 elemento const = document.querySelector ("# elemento"); </script>

```

R: O expressão `element.border-width` não faz muito sentido. Ele calcula a diferença entre `element.border` e uma variável largura, o que definitivamente não é algo que queremos. A próxima opção, `element.getAttribute ("border-width")` ;; busca o atributo do elemento HTML, e não a propriedade de estilo. Finalmente, as duas últimas opções fornecem o valor de 1px ( opções ce d).

3 Qual método integrado pode ter todos os estilos aplicados a um determinado elemento (estilos fornecidos pelo navegador, estilos aplicados por meio de folhas de estilo e propriedades definidas por meio do atributo style)?

R: Apenas a última opção, `getComputedStyle`, é um método integrado que pode ser usado para obter o estilo calculado de um determinado elemento HTML (opção c). Os outros três métodos não estão incluídos na API padrão.

4 Quando ocorre a eliminação do layout?

R: O descarte do layout ocorre quando nosso código executa uma série de leituras e gravações consecutivas no DOM, forçando sempre o navegador a recalculas as informações do layout. Isso leva a aplicativos da web mais lentos e menos responsivos.

### Capítulo 13. Eventos de sobrevivência

1 Por que é importante que a adição de tarefas à fila de tarefas aconteça fora do loop de eventos?

R: Se o processo de adicionar tarefas à fila de tarefas fizer parte do loop de eventos, quaisquer eventos que ocorram enquanto o código JavaScript estiver sendo executado serão ignorados. Isso definitivamente seria uma má ideia.

- 2 Por que é importante que cada iteração do loop de eventos não leve muito mais do que cerca de 16 ms?

R: Para obter aplicativos de execução suave, o navegador tenta realizar a renderização cerca de 60 vezes por segundo. Como a renderização é realizada no final do loop de eventos, cada iteração não deve durar muito mais do que 16 ms, a menos que queiramos criar aplicativos lentos e irregulares.

- 3 Qual é o resultado da execução do código a seguir por 2 segundos?

```
setTimeout (function () {
 console.log ("Tempo limite");
}, 1000);

setInterval (function () {
 console.log ("Intervalo");
}, 500);
```

UMA: Intervalo Tempo Limite Intervalo Intervalo Intervalo ( opção b). O definir-Intervalo método chama o manipulador com pelo menos o atraso fixo entre cada chamada, até que o intervalo seja explicitamente limpo. O setTimeout método, por outro lado, chama o retorno de chamada apenas uma vez, após o atraso especificado ter decorrido. Neste exemplo, primeiro o setInterval o retorno de chamada é disparado uma vez após 500 ms. Então o setTimeout retorno de chamada é invocado após 1000 ms, e outro setInterval

imediatamente depois. Nosso exame termina com mais dois setInterval invocações de retorno de chamada, uma a 1.500 ms e a outra a 2.000 ms.

- 4 Qual é o resultado da execução do código a seguir por 2 segundos?

```
const timeoutId = setTimeout (function () {
 console.log ("Tempo limite");
}, 1000);

setInterval (function () {
 console.log ("Intervalo");
}, 500);

clearTimeout (timeoutId);
```

UMA: Interval Interval Interval Interval ( opção c). O setTimeout chamar-volta é liberada antes de ter a chance de disparar, portanto, neste caso, temos apenas quatro execuções do setInterval ligar de volta.

- 5 Qual é o resultado de executar o código a seguir e clicar no elemento com o ID interno?

```
<body>
 <div id = "outer">
 <div id = "inner"> </div>
 </div>
</script>

const innerElement = document.querySelector ("# inner"); const outerElement = document.querySelector ("# outer");
```

```

const bodyElement = document.querySelector ("body"); innerElement.addEventListener ("click",
function () {
 console.log ("Interno");
});

outerElement.addEventListener ("click", function () {
 console.log ("Externo");
}, verdade);

bodyElement.addEventListener ("click", function () {
 console.log ("Corpo");
});
<script>
</body>

```

UMA: Corpo Interno Externo ( opção c). Os manipuladores de clique no innerElement e a bodyElement são registrados no modo bubbling, enquanto o manipulador de cliques no outerElement é registrado no modo de captura. Ao processar o evento, ele primeiro desce do topo e chama todos os manipuladores de eventos no modo de captura. A primeira mensagem será Exterior. Depois que a meta do evento é alcançada, no nosso caso, o elemento com ID interno, o borbulhar do evento acontece, e o evento borbulha. Portanto, a segunda mensagem será Interno, e o terceiro será Corpo.

#### Capítulo 14. Desenvolvimento de estratégias para vários navegadores

- 1 O que devemos levar em consideração ao decidir quais navegadores oferecer suporte? R: Ao decidir quais navegadores oferecer suporte, devemos pelo menos levar em consideração o seguinte:

- As expectativas e necessidades do público-alvo
- A participação de mercado do navegador
- A quantidade de esforço necessária para suportar o navegador Explique o

- 2 problema de IDs gananciosos.

R: Ao trabalhar com elementos de formulário, o navegador adiciona propriedades ao elemento de formulário para cada elemento descendente com um ID, para que possamos acessar esses elementos facilmente por meio do elemento de formulário. Infelizmente, isso pode substituir algumas das propriedades integradas do formulário, como ação ou enviar.

- 3 O que é detecção de recurso?

R: A detecção de recurso funciona determinando se um determinado objeto ou propriedade de objeto existe e, em caso afirmativo, presume que ele fornece a funcionalidade implícita. Em vez de testar se o usuário está usando um navegador específico e, em seguida, implementar soluções alternativas com base nessas informações, testamos se um determinado recurso funciona como deveria.

- 4 O que é um polyfill do navegador?

R: Se quisermos usar uma determinada funcionalidade que não é compatível com todos os navegadores-alvo, podemos usar a detecção de recursos. Se um navegador atual não suportar uma determinada funcionalidade, fornecemos nossa própria implementação, e isso é chamado de polyfilling.



## Símbolos

``` Character 388  
`^` Caracteres 263-264
`!` personagem 255
`?` personagem 265
`.` personagem 263, 277
`()` operadora 68
`*` personagem 129, 264, 266 \
 personagem 262, 264,
 269, 316-317
`+` operador 266, 273
`<` operador 241
`=>` operador 50
`>` operador 241
`>>>` operadora 380
`|` character 266, 427 \$
 character 263-264
`$ {}` sintaxe 388

UMA

acesso a objetos, controlando respostas
 de exercícios 422-424
 visão geral 210-213
 propriedades, acesso com getters e
 setters 200-210 definindo getters
 e
 setters 202-207
 usando getters e setters para
 definir computado
 propriedades 208-210
 usando getters e setters para
 validar propriedade
 valores 207-208

usando proxies
 para registro 214-215
 para medir
 desempenho 215-217
 visão geral 210-223
 custos de desempenho 220-223
 preencher automaticamente
 propriedades 217-218
 para implementar matriz negativa
 índices 218-220
 método de acesso 95-96
 atributo de ação 375
 Adicionar ponto de interrupção condicional
 opção 401
 método `addEventListener`
 26, 353-354, 356-357, 359,
 376-377, 411
 métodos avançados de matriz 5 agregando
 itens de matriz 241-242 método `alert()` 9

apelido
 objetos de argumentos como alias para
 parâmetros de função 65
 evitando 66-67
 propriedades de amálgama 322
 AMD (Módulo Assíncrono
 Definição) 291-292
 funções anônimas 39
 anti-recurso 374
 Apache Cordova 11
 APIs
 incongruente 383
 teste de desempenho 383
 aplicativos, para embutidos
 dispositivos 11
 aplique a armadilha 213, 216-217

método `apply()`
 forçando o contexto da função em
 callbacks 83
 invocando funções com
 77-83
 argumentos
 parâmetros de função e
 52-60
 parâmetros padrão e
 55-60
 parâmetros de descanso e visão geral
 54-55 52
 fatiar 54
 parâmetros de argumentos 62-67
 como alias para parâmetros
 de função 65
 evitando aliases 66-67
 visão geral 413
`argument.length` propriedade 63
 literais de matriz 225-226
 Objeto de matriz 225-226, 243
`Array.prototype.find`
 método 379
`Array.prototype.push()`
 método 244
 matrizes 225-244
 adicionar e remover itens em qualquer
 localização da matriz 230-232 adicionar e
 remover itens em qualquer extremidade de
 227-229
 agregando itens da matriz
 241-242
 criando 225-227
 iterando mais de 232-233
 mapeamento 233-235

matrizes (*continua*)

- reutilizando array embutido
- funções 242-244**
- pesquisando 237-240
- classificação 39, 240-241
- teste de itens de matriz 235-236
- funções de seta 34, 44, 50-52, 72, 83-86, 160
- como palavra-chave 298-300
- método `assert ()` 9-10, 37-38, 108, 404-405
- método de asserção, teste
- código 404-405
- suposições, cross-browser
- estratégias 383-385
- asterisco 129
- função assíncrona 159-162
- eventos assíncronos 25
- tratamento assíncrono 152
- Módulo Assíncrono
- Definição. *Ver* AMD
- método `attachEvent` 376
- atributos, DOM 313-315
- módulos de aumento 287-290
- propriedades de preenchimento automático
- visão geral 200, 214
- usando proxies para 217-218 `await`
- palavra-chave 161

B

Babel 7

- caractere barra invertida 264
- backticks 388
- melhores práticas 8-10
- depuração 9**
- análise de desempenho 10
- testando 9-10
- método `bind ()`**
- corrigir o problema dos contextos de
- função com 86-90
- visão geral 83
- variáveis com escopo de bloco, usando `let`
- e `const` palavras-chave para
- especificar 111-113
- Propriedade booleana 176
- border-properties 322**
- propriedade `borderWidth` 317, 429 pontos de
- interrupção
- condicional 401-402
- definido 398-399
- navegador
- APIs 8, 20
- correções de bugs 371-372
- compatibilidade 8

visão geral 7-8

Veja também problemas de navegador não

- testáveis
- eventos do navegador 25
- modo borbulhante 431
- fase de borbulhamento 356
- página de construção em tempo de execução,
- exercícios de resposta 411
- funções de array integradas,
- reutilizando 242-244
- `<button>` elemento 79
- objeto de botão 78-79, 84-87 método
- 78 de `button.click ()`

C

- pilha de chamadas 100, 102
- método de `call ()` 68, 79-80, 82-83, 244
- funções de retorno de chamada
- visão geral 36-38
- classificação com comparador
- 39-40
- parâmetro de retorno de chamada 38
- callbacks**
- conceito de 36
- forçando o contexto da função
- em 83
- visão geral 160
- problemas com 147-149
- usando fechamentos com 96-99
- capturas 266
- capturando fase 356, 431 caractere
- circunflexo 263
- Folhas de estilo em cascata. *Ver* Bloco de captura
- de CSS 128, 138
- método `catch` 152-153, 156, 160 `catch`
- declaração 108
- encadeando promessas 155-156
- operador de classe de caracteres 263
- verificar a função 113
- elementos sem filhos 308
- Chrome DevTools 9, 102, 396-397
- classe palavra-chave 168, 190-191 classes,
- usando em ES6 190-197 implementação
- herança 193-197
- usando a palavra-chave `class` 190-193
- `className` argumento 269
- método `clearInterval`
- 122, 344-345
- Método `clearTimeout` 344
- função de clique 16, 26, 28, 78, 85

clientHeight propriedade 329

- `clientLeft` propriedade 329
- propriedade 329 do `clientTop`
- propriedade `clientWidth` 328-329
- operação `cloneNode` 305**
- acoplamento próximo 361
- fechamentos
- exemplo 122-125**
- respostas do exercício 416-417
- manter o controle de identificadores com
- léxico
- ambientes 103-106**
- visão geral 92-95
- variáveis privadas,
- imitando 95-96, 117-121
- monitorar a execução do código com
- contextos de execução 99-102
- usando com `callbacks` 96-99
- modularização de código
- técnicas. *Ver* módulos
- aninhamento de código 103-106
- `<col>` elemento 309
- `<colgroup>` elemento 309
- coleções**
- matrizes 225-244
- adicionar e remover itens**
- em qualquer matriz
- localização 230-232
- adicionar e remover itens**
- em qualquer extremidade
- de 227-229
- agregando matriz**
- itens 241-242
- criando 225-227
- iterando mais de 232-233
- mapeamento 233-235
- reutilizando array embutido
- funções 242-244**
- pesquisando 237-240
- classificação 240-241
- testando itens de matriz 235-236 respostas
- de exercícios 424-426
- mapas 244-251
- criando 247-250
- iterando mais de 250-251
- objetos como, não use
- 245-247**
- conjuntos 251-256
- criando 252-253
- diferença de 255-256
- intersecção de 255
- união de 253-254
- propriedade de cor 317, 321
- CommonJS, módulos 292-293

comunicando, com
 geradores 136-139
 enviando valores como argumentos da
 função geradora
 136-137
 lançando exceções 138-139
 usando o próximo método para enviar valores
 para o gerador 137-138 comparador,
 classificando o retorno de chamada
 funções com 39-40
 computacionalmente caro
 processamento, loop de evento
 350-353
 estilos computados
 DOM 319-322
 visão geral 315
 variável `computedStyles` 321
 pontos de interrupção condicionais
 401-402
 chave configurável 185
 objeto de console 10
 método `console.assert()` 405
`console.log` método 103, 397
`console.time` método 10, 217
`console.timeEnd` método 10
`const` palavra-chave, especificando
 variáveis com escopo de bloco
 com 111-113
 variáveis `const` 107-109
 construir armadilha 213
 casos de teste construtivo 402 propriedade
 do construtor 173, 177,
 179-181, 183-184, 186, 193,
 246, 420-421
 construtores
 invocar funções como 72-77
 considerações de codificação
 para 76-77
 considerações para 76-77
 visão geral 72-74, 76
 valores de retorno 74-76
 digitação de objetos por meio da visão
 geral 179-181 172, 245
 contexto, para as funções 80, 83
`createElement` 309
 Cria o objeto 70
 Crockford, Douglas 287
 estratégias para vários navegadores
 correções de bug do navegador 371-372
 exercícios 385
 código externo e
 marcação 373-376
 Regras CSS 376
 DOM derrotando 374-375

encapsulamento 373-374
 trabalhando com existente
 código 374
 detecção de recurso 379-381
 corrige 378-379
 visão geral 8, 368-370
 polyfills 379-381
 reduzindo suposições
 383-385
 regressões 376-377
 navegador não testável
 edições 381-383
 Desempenho da API 383
 o navegador trava 382
 Efeitos de propriedade CSS 382 disparo
 de eventos 382
 ligações de manipulador de eventos 381
 APIs incongruentes 383
 Visão geral de CSS (Cascading Style
 Sheets) 306
 efeitos de propriedade 382
 regras 376
 eventos personalizados 360-364
 Ajax 361-362
 criando 362-364
 acoplamento solto 361

D

travessão 275
 Painel do depurador 395, código de
 depuração 399 397-402
 pontos de interrupção 398-399
 pontos de interrupção condicionais
 401-402
 registro 397-398
 entrando em função
 399-400
 declarações (função) 45-46
 casos de teste destrutivos 402
 exportações padrão, ES6
 módulos 297-298
 parâmetros padrão
 (função) 55-60
 definir função 291
 definição de interfaces de módulo 284 método de
 exclusão 249
 aplicativos de desktop 11
 desestruturação 387, 389-390
 Ferramentas do desenvolvedor, Firefox
 9, 393-395
 DevTools 9, 102, 396-397
 dicionários. *Ver* mapas
 difícil tratamento de erros 148

propriedade de exibição 323
 itens distintos 251
`<div>` elemento 268, 309, 314,
 316, 325
`doc.createDocumentFrag-`
 método de `ment()` 312
 manipulador de clique de documento
 355, 357
 objeto de documento 20, 22 Modelo de
 objeto de documento. *Ver*
 DOM
`document.getElementById`
 método 246
 Dojo toolkit 291
 caractere cifrado 264 DOM
 (objeto de documento
 Modelo) 305-331
 atributos e
 propriedades 313-315
 edifício 18-20
 estratégias para vários navegadores
 374-375
 respostas do exercício 428-429
 injetar HTML em 306-313 HTML
 wrapping 308-311
 inserir elementos em
 documento 311-313
 visão geral 307-311
 pré-processamento de HTML
 string de origem 307-308
 layout thrashing 327-331
 visão geral 7, 17
 propagando eventos
 até 354-360
 atributos de estilo 315-326
 estilos calculados 319-322
 convertendo pixel
 valores 322-323
 altura e largura
 propriedades 323
 nomeando 318-319
 visão geral 315-317
 atravessando usando
 geradores 135-136
 Painel do Explorador DOM 395
 fragmentos DOM 306, método 311
 DOM 313-315
 Especificação DOM 19
`DOMContentLoaded`
 manipulador 314
 feito propriedade 132
 caractere barra invertida dupla
 262, 264
 colchetes duplos 104

caractere de escape duplo 269
 natureza dinâmica do JavaScript,
 efeitos colaterais de 176-178

E

ECMAScript 6. *Ver* ES6 elem
 variável 98
 elipses 54, 309
 valores embutidos 269
 encapsulamento 373-374
 literais de objeto aprimorados
 387, 390-391
 chave enumerável 185
 enumerar armadilha 213
 Propriedade [[Ambiente]]
 106, 118
 erros, travando em cadeia
 promete 156
 ES6 (ECMAScript 6)
 desestruturação 389-390
 literais de objeto aprimorados
 390-391
 exportando e importando
 funcionalidade 294-302
 exportações padrão 297-298
 renomeando exportações e
 importações 298-302
 literais de modelo 387-388
 usando classes em 190-197
 implementação
 herança 193-197
 usando a palavra-chave *class*
 190–193
 caracteres de escape 278
 evento borbulhando 355-356,
 359-360, 364
 captura de eventos 355-356, 364 tratamento
 de eventos
 visão geral 14-15
 construção de página 23-29
 loop de eventos 152, 333, 335-344
 microtarefas 336-344
 microtarefas 339-344
 temporizadores 344-353
 computacionalmente caro
 processamento 350-353
 visão geral 345-350
 tempo limite e intervalos 350 fila de
 eventos 14, 24-25, 28 aplicativos orientados a
 eventos 8
 registro de manipulador de eventos 26
 propriedade *event.target* 359
 eventos
 eventos personalizados 360-364

Ajax 361-362
 criando 362-364
 acoplamento solto 361
 evento disparando 382
 associações do manipulador de eventos 381
 respostas do exercício 429-431
 exercícios 364-365
 propagando através de
 DOM 354-360
 Veja também loop de eventos
 a cada método 235
 exceções, jogando 138-139
 ponto de exclamação 255
 método *exec ()* 271, 275
 executando o estado 139
 execução 267
 contextos de execução
 execução do código de rastreamento
 com 99-102
 geradores de rastreamento
 usando 141-146
 função do executor 147, 151-152 respostas
 do exercício 411-431
 acesso a objetos,
 controlando 422-424
 página de construção na modularização do
 código 411 em tempo de execução
 técnicas 427-428
 coleções 424-426
 estratégias para vários navegadores 431
 DOM 428-429
 eventos 429-431
 funções
 fechamentos e escopos
 416-417
 definições e
 argumentos 411-413
 geradores e
 promessas 417-419
 invocação 413-416
 protótipos, orientação a objetos
 com 419-422
 expressões regulares 426-427
 esperar função 407
 rejeitando explicitamente as promessas 152 exportar
 palavra-chave 294–295, 297–
 298
 exporta propriedade 292
 exportações, renomeando 298-302
 expressões (função) 46-48
 estende a palavra-chave 194-195
 código externo e marcação,
 navegador cruzado
 estratégias 373-376
 Regras CSS 376

DOM derrotando 374-375
 encapsulamento 373-374
 trabalhando com existente
 código 374

F

F12 Developer Tools 9, 395 reprovam
 classe 405
 falha na função 108
 falha de retorno de chamada 147
 FastDom 329
 operador *fat-arrow* 50-51
 detecção de recursos, navegador cruzado
 estratégias 379-381
 método de filtro 238-239, 255, método de
 localização 237-238, 240, 244,
 379-380
 método *findIndex* 240
 Firebug 9, 393
 Ferramentas de desenvolvedor do Firefox
 9, 393-395
 objetos de primeira classe 5, 34-35, 38,
 44, 58, 160
 Fitzgerald, Michael 261
 para loop 10, 104, 110,
 112, 232–233
 para declaração 81
 loop *for-in* 185-186
 declarações *for-in* 213
 loop *for-of* 129-130, 132-133,
 136, 163, 418
 método *forEach ()* 81-82,
 233-234
<form> elemento 375
 parâmetro de fragmento 311
 Friedl, Jeffrey 261
 estado cumprido 150
 encerramentos de funções 5
 código de função 21, 99
 construtores de função 44, 72
 contextos de função
 corrigindo o problema de 83-90 usando o
 método de ligação 86-90 com as funções de
 seta 83-86 forçando em retornos de chamada
 80-83 visão geral 67, 100
 declarações de função 44
 contexto de execução de função 99
 expressões de função 44, 47 palavra-chave
 de função 45, 50,
 128-130, 162
 literal de função 44
 declaração de função *()* 49

programação funcional 36

funções

argumentos 52-60

parâmetros padrão e
55-60parâmetros de descanso e 54-55
fatiamento 54

como objetos de primeira classe 35-36

como módulos 284-285

como objetos 40-44

auto-memorável

funções 42-44

armazenamento de funções 40-42

callback 36-40

chamando funções antes de suas
declarações 115encerramentos, exercício
respostas 416-417

declarando 44

definindo 44-52

funções de seta 50-52

declarações de função
45-46

expressões de função 46-48

funções imediatas 48-50

corrigindo o problema dos contextos de
função 83-90

usando o método de ligação 86-90 com

geradores de funções de seta 83-86,

exercício

respostas 417-419

função implícita

parâmetros 62-67

parâmetro de argumentos
62-67

contexto de função 67

importância de nas funções

anônimas do JavaScript 39

conceito de callback 36

invocando 67, 72, 77-83 como

construtores 72-77

como métodos 69-72

considerações para 76-77

respostas do exercício 413-416

visão geral 68-69, 72, 76 com

aplicar e ligar

métodos 77-83

substituindo 115-116

parâmetros para 53, 67

promessas, exercício

respostas 417-419

escopos, respostas de exercícios
416-417

auto-lembrando 42

simulando métodos de array
com 242entrando em função
399-400

armazenando 42

G

g bandeira 262, 271

funções do gerador 44

geradores 129-146

combinando com

promete 158-164

comunicando-se com 136-139

enviando valores como gerador

função

argumentos 136-137

lançando exceções

138-139

usando o próximo método para enviar

valores em

gerador 137-138

controlando por meio de iterador

objeto 130-133

iterador de iteração 132

cedendo a outro

gerador 133

respostas do exercício 417-419

tornando o código assíncrono elegante com
127-129

visão geral 5

rastreamento usando execução

contextos 141-146

usando para gerar IDs

133-135

usando para atravessar DOM

135-136

obter a palavra-chave 185, 203

get trap 211-213, 215, 218, 220

getAttribute () método 313-315

getBoundingClientRect

propriedade 329

Método getComputedStyle

329, 429

Método getElementById

16, 21-22

getElementsByTagName

método 268

getPrototypeOf 213

função getter 185, 203 getters

e setters

acesso a propriedades

usando 200-210

definindo getters e

setters 202-207

usando getters e setters para
definir computado

propriedades 208-210

usando getters e setters para
validar propriedade

valores 207-208

visão geral 96

código global 21, 99

Goyvaerts, janeiro 261

suporte de navegador graduado 368

maior do que operador 241

operadores gananciosos 265

H

tratamento de eventos 25

tem o método 248-249, elemento

principal 253 18-19

propriedade de altura, DOM 323-326 internos

do módulo de ocultação 284 içamento 116

HTML

injetar no DOM 306-313 HTML

wrapping 308-311

inserir elementos em

documento 311-313

visão geral 307-311

pré-processamento de HTML

string de origem 307-308

análise de 18-20

elemento html 18, 372

painel HTML 393

eu

eu sinalizo 262

i variável 110-111

id atributo 314, 374, 429 id

valores 41, 134, 278 identificador

resolução 102

identificadores

registrando-se no léxico

ambientes 113-116

chamando funções antes

declarações 115

substituindo funções

115-116

processo 113-115

rastreamento com léxico

ambientes 103-106

IDEs (desenvolvimento integrado

ambientes) 393

IDs, gerando usando

geradores 133-135

IE (Internet Explorer) 395 IIFE
(invocado imediatamente
expressão de função) 49
funções imediatas 48-50
função implícita
parâmetros 62-67
parâmetros de argumentos 62-67
contexto de função 67
parâmetros implícitos 62
importar palavra-chave 294-296
anotação importante 317
importações, renomeando 298-302
método `indexOf` 239
herança 169
`input` # `action` element 375
propriedades da instância 173-176
operador instância
180, 187–190, 213, 421
ambiente de desenvolvimento integrado
ronments. *Ver* IDEs
interfaces, módulo 285-287
Internet Explorer. *Ver* Interseção do IE,
de conjuntos de intervalos de 255,
invocação de loop de evento 350,
exercício
respostas 413-416
invocando funções 67-83
como método 69-72
construtor 72-77
considerações de codificação
para 76-77
visão geral 72-74
valores de retorno 74-76
visão geral 68-69
com métodos de aplicação e
chamada 77-83
iterando sobre mapas 250-251 objeto
iterador, controlando
eradores até 130-133
iterador de iteração 132
cedendo a outro
gerador 133

J

Estrutura de teste Jasmine
407-409
JavaScript
melhores práticas 8-10
depuração 9
análise de desempenho 10
testando 9-10
aumentando a habilidade
transferibilidade 10-12

navegador 7-8
características do idioma
evolução de 6
visão geral 4-5
transpiladores 6-7
função `jQuery` 373
Método `JSON.parse` 155
método `juggle` () 79

K

método de chaves 251
Kleene, Stephen 261
objeto kuma 169-170

eu

funções lambda 44
layout thrashing, parâmetro de comprimento 64
do DOM 327-331
propriedade de comprimento 63, 225-227,
243-244, 380
menor que o operador 241
deixe a palavra-chave
visão geral 417
especificando com escopo de bloco
variáveis com 111-113
Levithan, Steven 261
ambientes lexicais
manter o controle de identificadores com
103-106
registrando identificadores
dentro de 113-116
chamando funções antes
declarações 115
substituindo funções
115-116
processo 113-115
 elemento 23, 405
ciclo de vida, construção de página 14-17
propriedade `location.href` 249–250
exploração madeireira
visão geral 397-398
usando proxies para 214-215
acoplamento solto 361

M

m bandeira 262
macrotarefas
loop de evento 336-344
visão geral 333-334
função `makeLoggable` 215

Construtor de mapa 248
função de mapa 234
matrizes de mapeamento 233-235
mapas 244–251
criando 247-250
iterando mais de 250-251
objetos como, não use 245-247 visão
geral 6
método `match` () 270-272
segmentos correspondentes, capturando
grupos de não captura 273
executando simples
captura 269
referenciando capturas 272
MDN (Mozilla Developer
Rede) 379
medição de desempenho, proxies
para 215-217
memoização 40
métodos, invocando funções
como 69-72
microtarefas
loop de eventos 339-344
visão geral 334, 336
minimizando suposições 383
aplicativos móveis, com
frameworks 11
parâmetro do módulo 288
padrão de módulo 286-287,
289-291, 293, 295
variável de módulo
286, 292-293, 295
`module.exports` object 293, 295 modules

ES6 294-302
respostas do exercício 427-428
no módulo pré-ES6 JavaScript 283-293
Asynchronous
Definição 291-292
módulos de aumento
287-290
CommonJS 292-293
funciona como 284-285
interfaces de módulo 285-287
evento `mouseenter` 382
evento `mouseleave` 382
evento `mousemove` 15-16,
27-28, 39
evento `mouseout` 382
evento `mouseover` 382
Mozilla Developer Network. *Ver*
MDN
cordas multilinha 387

N

exportação nomeada 296, índices de matriz negativa 298–299,
 implementando usando
 proxies 218–220
 eventos de rede 25
 novo operador 171–173
 novas linhas, correspondendo a 277
 próximo método 130–134, 136–138,
 140, 143–144, 158–160
 código de retorno de chamada não bloqueante 129
 geradores não bloqueantes 129
 grupos de não captura 273–274
 elemento não exibido 323
 variáveis não exportadas 296
 operadores não agudos 265
 variáveis não importadas 296
 modo não estrito 67–69, 205, 414

O

literais de objeto
 funções de seta e 85–86
 aprimorado 390–391
 métodos de objeto 192
 orientação do objeto, usando
 protótipos 419–422
 Object.defineProperty
 método 185–186, 193, 202,
 205–206
 Object.setPrototypeOf
 método 170
 objetos
 acesso, controlando 422–424 como
 mapas, não use construção de
 245–247, protótipos
 e 171–181
 propriedades da instância
 173–176
 digitação de objetos via
 construtores 179–181
 efeitos colaterais da dinâmica
 natureza de
 JavaScript 176–178
 funciona como 40–44
 auto-memorável
 funções 42–44
 armazenamento de funções 40–42
 propriedade offsetHeight 323, 329
 propriedade offsetLeft 329
 propriedade offsetParent 329
 propriedade offsetTop 329
 propriedade offsetWidth 323, 329

propriedade offsetX 329
 propriedade offsetY 329
 evento onerror 154
 onload evento 154
 onload propriedade 26
 <option> elemento 307–308
 exterior (pai) léxico
 ambiente 104
 substituindo funções 115–116

P

construção de página
 construção DOM 18–20
 tratamento de eventos 23–29
 exercícios 29
 Código JavaScript
 executando 21–23
 objetos globais 20
 tipos de 21
 ciclo de vida 14–17
 visão geral 14–15
 analisar HTML 18–20
 parâmetros, para as funções 53, 67
 método parseFloat 323
 subexpressão passiva
 273–274, 277
 variável padrão 262
 padrões 261
 estado pendente 149, 151–152
 desempenho
 medindo proxies para
 215–217
 visão geral 306
 caractere ponto 263, caractere 277
 barra vertical 266, valores de 427
 pixels, polyfills DOM 322–323 379–381
 método pop 227, 229
 propriedade de posição 323
 pré-compilar regular
 expressões 269
 pré-construção regular
 expressões 269
 personagem predefinido
 classes 265
 hash anterior 326
 valor principal 43
 variáveis privadas, imitando
 95–96
 Construtor de promessa
 146–147, 151
 Método Promise.all 157
 Método Promise.race 157

Método Promise.resolve () 342

promessas
 encadeamento 155–156
 combinando com
 geradores 158–164
 criando 154–155
 respostas do exercício 417–419
 tornando o código assíncrono elegante com
 127–129
 visão geral 149–152
 rejeitando 152–153
 esperando pelo número de
 156–158
 propriedades
 acesso a com getters e setters
 200–210
 definindo getters e
 setters 202–207
 usando getters e setters para
 definir computado
 propriedades 208–210
 usando getters e setters para
 validar propriedade
 valores 207–208
 DOM 313–315
 descritor de propriedade 185
 cadeia de protótipo 170, 175, 182
 métodos de protótipo 192
 propriedade de protótipo
 170, 173–174, 183
 objeto baseado em protótipo
 orientação 5
 protótipos
 alcançando herança
 181–190
 operador instância
 187–190
 problema de anular
 constructor
 propriedade 184–187
 instanciando usando referência
 para o construtor 179
 construção de objetos e
 171–181
 propriedades da instância
 173–176
 digitação de objetos via
 construtores 179–181
 efeitos colaterais da dinâmica
 natureza de
 JavaScript 176–178
 orientação de objeto usando
 417–422
 visão geral 168–171

protótipos (*continua*)

- usando classes JavaScript no ES6 190-197
- implementando herança 193-197
- usando a palavra-chave class 190–193

procuradores

- custos de desempenho de 220-223 usando para perfilagem 214-215 usando para medição
- desempenho 215-217
- usando para preencher automaticamente propriedades 217-218
- usando para implementar índices de matriz negativos 218-220

Construtor de proxy 210-211, 213

método push 227-229, 244

Q

Estrutura de teste QUnit 406-407

R

- reconciliando propriedade referências 178
- reduzir método 242
- referenciando capturas, capturando segmentos correspondentes 272
- Método RegExp () 262, regressões 269, navegador cruzado estratégias 376-377
- expressões regulares
 - benefícios de 260
 - capturando correspondência segmentos 269-274
 - correspondência usando global expressões 271
 - grupos de não captura 273
 - executando simples captura 269
 - referenciando capturas 272-273
 - usando global expressões 272
- classes de personagens 263
- compilando 267
- fim da string 264
- exemplos usando
 - correspondência de novas linhas 277
 - Unicode correspondente caracteres 277-278

- respostas do exercício 426-427
- visão geral 260
- substituindo usando funções 274
- resolvendo problemas comuns
 - com
 - correspondência escapou personagens 278
 - correspondência de novas linhas 277
 - Unicode 277 correspondente
- termos e operadores
 - alternância (ou) 266
 - referências anteriores 266
 - começa e termina 264
 - escapando 263
 - correspondência exata 263
 - agrupamento 266
 - correspondência da classe de
 - personagens 263
 - personagem predefinido classes 265
 - ocorrências repetidas 264
- Expressão regular
 - construtor 426
- rejeitar função 146-147, 150, 152
- rejeitando promessas 152-153
- operação removeChild 305
- ocorrências repetidas, regulares expressões e 264
- método replace () 272, 274, 276, 321
- objeto representativo 211
- requer função 293
- Requer JS 291
- função de resolução 146-147, 149, 152
- Modo de design responsivo 395 parâmetros de descanso (função) 54-55
- declaração de retorno 50, 52, 130, 140, 146, 312
- valores de retorno, construtores 74-76
- tempo de execução, página de construção em 411

S

- escopos 5, 92, 416-417
- elemento de script 17–18, 20, 22–23
- propriedade scroll 329
- scrollBy property 329
- propriedade scrollByLines 329
- propriedade scrollByPages 329
- propriedade scrollHeight 329
- Propriedade scrollIntoView 329

- propriedade scrollLeft 329, 372
- propriedade scrollTo 329
- propriedade scrollTop 329, 372
- propriedade scrollWidth 329
- propriedade scrollY 329
- procurando matrizes 237-240
- <select> elemento 308-309
- funções de auto-memorização 42-44
- aplicativos do lado do servidor 11
- definir palavra-chave 202–203
- definir operador 263
- set trap 211, 213, 215, 220 setAttribute () método 314-315
- método setInterval 98, 122, 344-345, 347, 349, 364, 430
- setPrototypeOf 213
- conjuntos 251-256
 - criando 252-253
 - diferença de 255–256
 - intersecção de 255
 - visão geral 251
 - união de 253-254
- função setter 185
- método setTimeout 344, 347, 350, 364, 430
- método de deslocamento 227, 230
- irmãos 18
- elementos simples 266
- simulando com base em classe herança 190
- execução single-threaded modelo 24
- propriedade de tamanho 249, 253
- argumentos de corte 54
- método de classificação 39, 50, 240
- matrizes de classificação 39, 240-241
- elemento 268
- método de emenda 231-232
- colchetes 263
- pilha 100
- entrando na função 399-400 armazenar variável 41
- armazenamento de funções 40-42
- modo estrito 66
- interpolação de string 387
- Objeto string 271, 274, 276 atributo de estilo 315-317
 - e estilos computados 322 conversão de valores de pixel 323 obtendo propriedades da visão geral 317 315-316
- <style> elemento 316
- propriedade de estilo 315-317, 319-322, 331

método `style()` 322

atributos de estilo, DOM 315-326 estilos

- computados 319-322
- convertendo valores de pixel 322-323
- altura e largura
 - propriedades 323-326
- nomeando 318-319
- visão geral 315-317

retorno de chamada de sucesso 147

super palavra-chave 195

Método da SuperClass 182

Estado de início suspenso 139 Estado de rendimento suspenso 145

T

objeto alvo 210-211, 213, 215-216, 219

propriedade alvo 354, 359, 364

elemento `<tbody>` 309

literais de modelo 387-388

casos de teste 402

função de teste 412

independência de teste 402

módulo de teste 283

repetibilidade de teste 402

testar a simplicidade 402

suítes de teste 406

método `test()` 269

testando

- itens de matriz 235-236
- visão geral 9

código de teste 402-410

- método de afirmação 404-405
- criando testes 402-404
- fundamentos de 404-410
- Jasmine 407-409
- medição de cobertura de código 410 QUnit 406-407

elemento `textarea` 262

o método 146-147, 150-153, 155, 160, 340

método de lançamento 138-139, 159

exceções de lançamento 138-139

método de tempo 10

tempos limite, loop de evento 350

método `timeEnd` 10

eventos de cronômetro 25

variável do temporizador 98

temporizadores, loop de evento 344-353

- caro computacionalmente
- processamento 350-353
- visão geral 345-350

método `toString` 247

Traceur 7

transformar propriedade 269-270

transpiladores 6-7

armadilhas 211

atravessando o DOM, usando geradores 135-136

bloco `try-catch` 138

declarações `try-catch` 103, 147, 149, 153, 155, 161

atributo de tipo 378, 383 operador `typeof` 56, 180, 213

você

u sinalizar 262

Capacidade de resposta da IU 395

`ul` element 23

string indefinida 56

Unicode, correspondendo a 277

união, dos conjuntos 253-254

promessa não resolvida 149

método `unshift` 227-230

problemas de navegador não testáveis 381-383

- Desempenho da API 383
- o navegador trava 382
- Efeitos de propriedade CSS 382 disparo de eventos 382
- ligações de manipulador de eventos 381
- APIs incongruentes 383

recursos não testáveis 383

eventos do usuário 25

V

argumento de valor 319

chave de valor 185

métodos de valores 251

valores, enviando para gerador 137-138

palavra-chave `var`, variáveis 109-111

çamento variável 116

variáveis 106-116

- variáveis `const` 107-109
- visão geral 106

registrando identificadores dentro de ambientes lexicais 113-116

chamando funções antes declarações 115

substituindo funções 115-116

processo 113-115

usando palavras-chave `let` e `const` para especificar o escopo do bloco variáveis 111-113

usando `var` palavra-chave 109-111

mutabilidade variável 107-109

propriedade de visibilidade 323

C

ferramentas de desenvolvedor da web 393-397

- Chrome DevTools 396-397
- F12 395
- Firebug 393
- Firefox 393-395
- WebKit Inspector 9, 396 WebKit Inspector 9, 396 while loop 132, 134, 163, 418 caractere de espaço em branco 266, 269, 277

propriedade de largura 323-326, 329, 378

objeto de janela 20, 23, 26, 68, 414

`window.getComputedStyle()` método 319

chave gravável 185

X

Objeto XMLHttpRequest 154

Y

bandeira y 262

expressão de rendimento 131, 137-138, 140, 146, 160

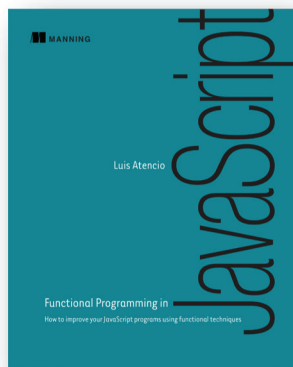
palavra-chave de rendimento 128-129, 131, 145

rendimento * operador 133

Z

operador zero-fill shift right-shift 380

MAIS TÍTULOS DEMANNING



Programação Funcional em JavaScript

por Luis Atencio

ISBN: 9781617292828

272 páginas

\$ 44,99

Junho de 2016



Design de aplicativo JavaScript

Uma abordagem Build First

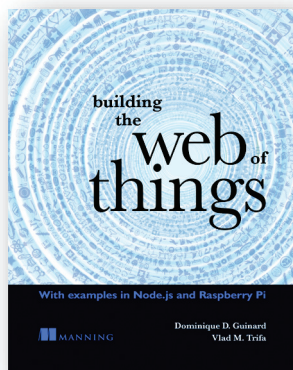
por Nicolas G. Bevacqua

ISBN: 9781617291951

344 páginas

\$ 39,99

Janeiro de 2015



Construindo a Teia das Coisas

Com exemplos em Node.js e Raspberry Pi

por Dominique D. Guinard e Vlad M. Trifa

ISBN: 9781617292682

344 páginas

\$ 34,99

Junho de 2016

Para obter informações sobre pedidos, acesse www.manning.com

Folha de dicas ES6 (continuação)

Aulas agir como um açúcar sintático em torno dos protótipos do JavaScript:

```
aula Pessoa {
  construtor( nome) {this.name = nome; } dance () {return true; }
}
classe Ninja estende Pessoa {
  construtor (nome, nível) {
    super (nome);
    this.level = level;
  }
  estático compare (ninja1, ninja2) {
    retornar ninja1.level - ninja2.level;
  }
}
```

Proxies controlar o acesso a outros objetos. Ações personalizadas podem ser executadas quando um objeto é interagido (por exemplo, quando uma propriedade é lida ou uma função é chamada):

```
const p = novo Proxy ( alvo, {
  obter: ( target, key) => {/ * Chamado quando propriedade acessada por meio de proxy * /},
  definir: ( destino, chave, valor) => {/ * Chamado quando a propriedade definida por meio de proxy * /}});
```

Mapas são mapeamentos entre uma chave e um valor:

- novo mapa () cria um novo mapa.
- Use o definir método para adicionar um novo mapeamento, o obter método para buscar um mapeamento, o tem método para verificar se existe um mapeamento, e o excluir método para remover um mapeamento.

Jogos são coleções de itens exclusivos:

- novo conjunto() cria um novo conjunto.
- Use o adicionar método para adicionar um novo item, o excluir método para remover um item, e o Tamanho propriedade para verificar o número de itens em um conjunto.

para de os loops iteram sobre coleções e geradores.

Destruição extrai dados de objetos e matrizes:

- **const {nome: ninjaName} = ninja;**
- **const [firstNinja] = [" Yoshi "];**

Módulos são unidades maiores de código de organização que nos permitem dividir programas em clusters:

```
exportar classe Ninja {}; // Exportar um item
exportação padrão class Ninja {} // Exportar exportação padrão {ninja}; // Exportar
exportação de variáveis existentes {ninja como samurai}; // Renomear uma
exportação

importar Ninja de "Ninja.js"; // Importar uma importação de exportação padrão {ninja} de "Ninja.js"; //
Importar exportações nomeadas
importar * como Ninja de "Ninja.js"; // Importar todas as exportações nomeadas import {ninja como iNinja} de "Ninja.js"; //
Importar com um novo nome
```


Segredos do JavaScript Ninja Segunda edição

Resig .Bibeault .Maras



Jtodo tipo de aplicativo, seja na web, no desktop, na nuvem ou em dispositivos móveis. Quando você

Se tornar um profissional de JavaScript, você tem um conjunto de habilidades poderoso que pode ser usado em todos esses domínios.

Segredos do JavaScript Ninja, segunda edição usa prático

exemplos para ilustrar claramente cada conceito e técnica principal. Esta edição completamente revisada mostra como fazer uma chave mestra

Conceitos de JavaScript, como funções, closures, objetos, prototipos e promessas. Abrange APIs como DOM, eventos, e temporizadores. Você descobrirá técnicas de práticas recomendadas, como teste e desenvolvimento em vários navegadores, todas ensinadas da perspectiva de profissionais qualificados de JavaScript.

O que há dentro

- Escrever código mais eficaz com funções, objetos e encerramentos
- Aprendendo a evitar armadilhas de aplicativos JavaScript
- Usando expressões regulares para escrever código de processamento de texto sucinto
- Gerenciando código assíncrono com promessas
- Totalmente revisado para cobrir os conceitos de ES6 e ES7

Você não precisa ser um ninja para ler este livro - apenas esteja disposto a se tornar um. Você está pronto?

John Resig é uma autoridade reconhecida em JavaScript e criadora da biblioteca jQuery. **Bear Bibeault** é um desenvolvedor web e autor da primeira edição, bem como co-autor de *Ajax na prática*, *protótipo* e *Scriptaculous em ação*, e *jQuery em ação*.

Josip Maras é pesquisador pós-doutorado e professor.

“
deve lopo...
técnicas poderosas para
melhore o seu JavaScript.”

- Becky Huett, Big Shovel Labs

Excelente e abrangente
visão sobre a magia das funções e
fechamentos para o
uso eficiente de JavaScript.”

- Gerd Klevesaat, Siemens

“
msuas habilidades essenciais para desenvolver
para o próximo nível.”

- David Starkey, Bluemix

“
Ajuda você a dominar as
técnicas fundamentais e usadas
de JavaScript moderno.”

- Christopher Haupt
New Relic Inc.

Para baixar seu e-book gratuito nos formatos PDF, ePub e Kindle,
os proprietários deste livro devem visitar
manning.com/books/secrets-of-the-javascript-ninja-second-edition

ISBN-13: 978-1-61729-285-9
ISBN-10: 1-61729-285-0



9 781617 292859